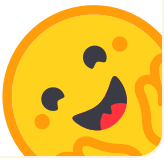
Search models, datasets, users...

Deep RL Course documentation
(Automatic) Curriculum Learning for RL

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

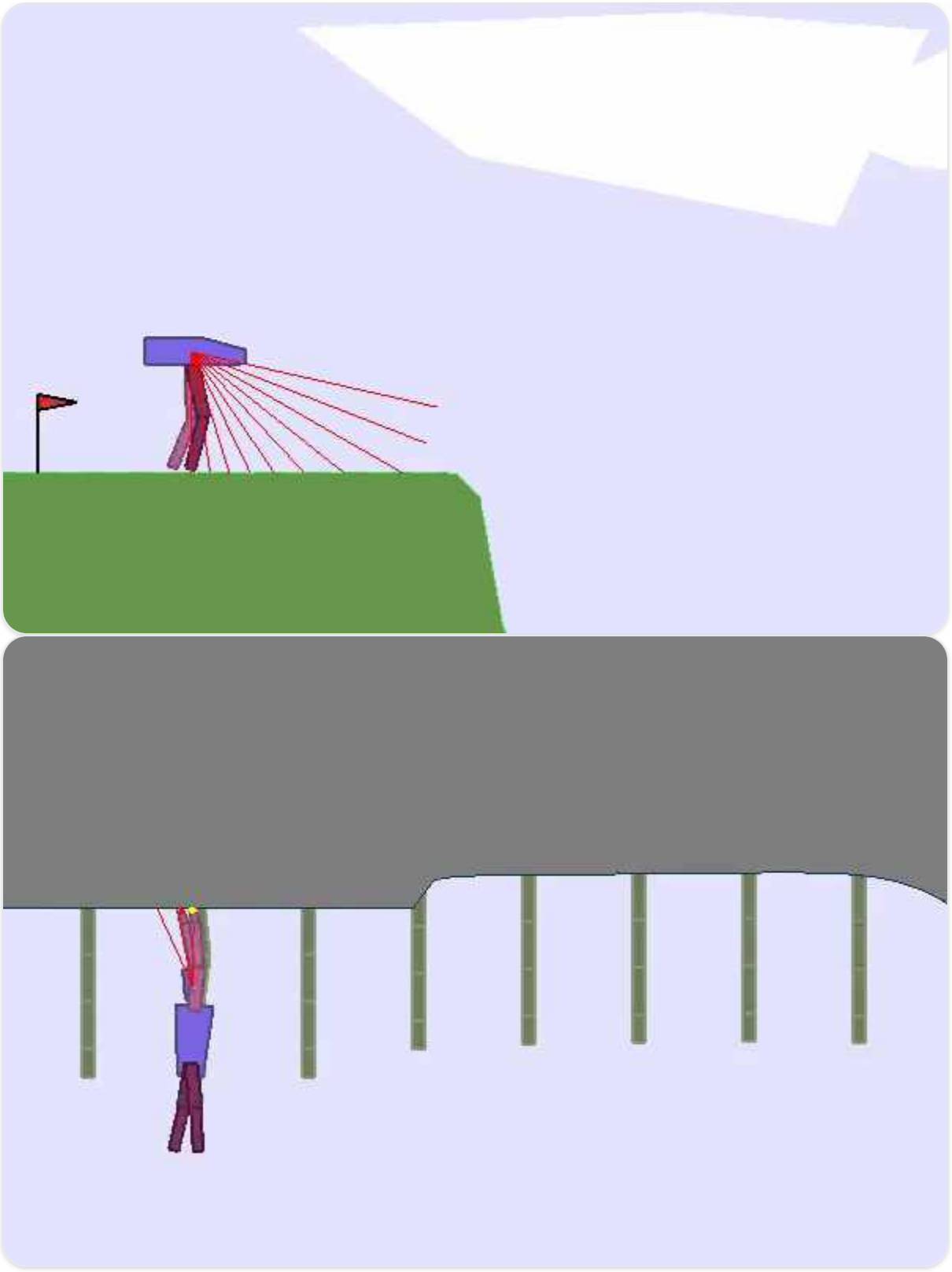


(Automatic) Curriculum Learning for RL

While most of the RL methods seen in this course work well in practice, there are some cases where using them alone fails. This can happen, for instance, when:

- the task to learn is hard and requires an **incremental acquisition of skills** (for instance when one wants to make a bipedal agent learn to go through hard obstacles, it must first learn to stand, then walk, then maybe jump…)
- there are variations in the environment (that affect the difficulty) and one wants its agent to be **robust** to them

Ask HuggingChat



TeachMyAgent

In such cases, it seems needed to propose different tasks to our RL agent and organize them such that the agent progressively acquires skills. This approach is called **Curriculum Learning** and usually implies a hand-designed curriculum (or set of t

practice, one can, for instance, control the generation of the environment, the initial states, or use Self-Play and control the level of opponents proposed to the RL agent.

As designing such a curriculum is not always trivial, the field of **Automatic Curriculum Learning (ACL) proposes to design approaches that learn to create such an organization of tasks in order to maximize the RL agent's performances**. Portelas et al. proposed to define ACL as:

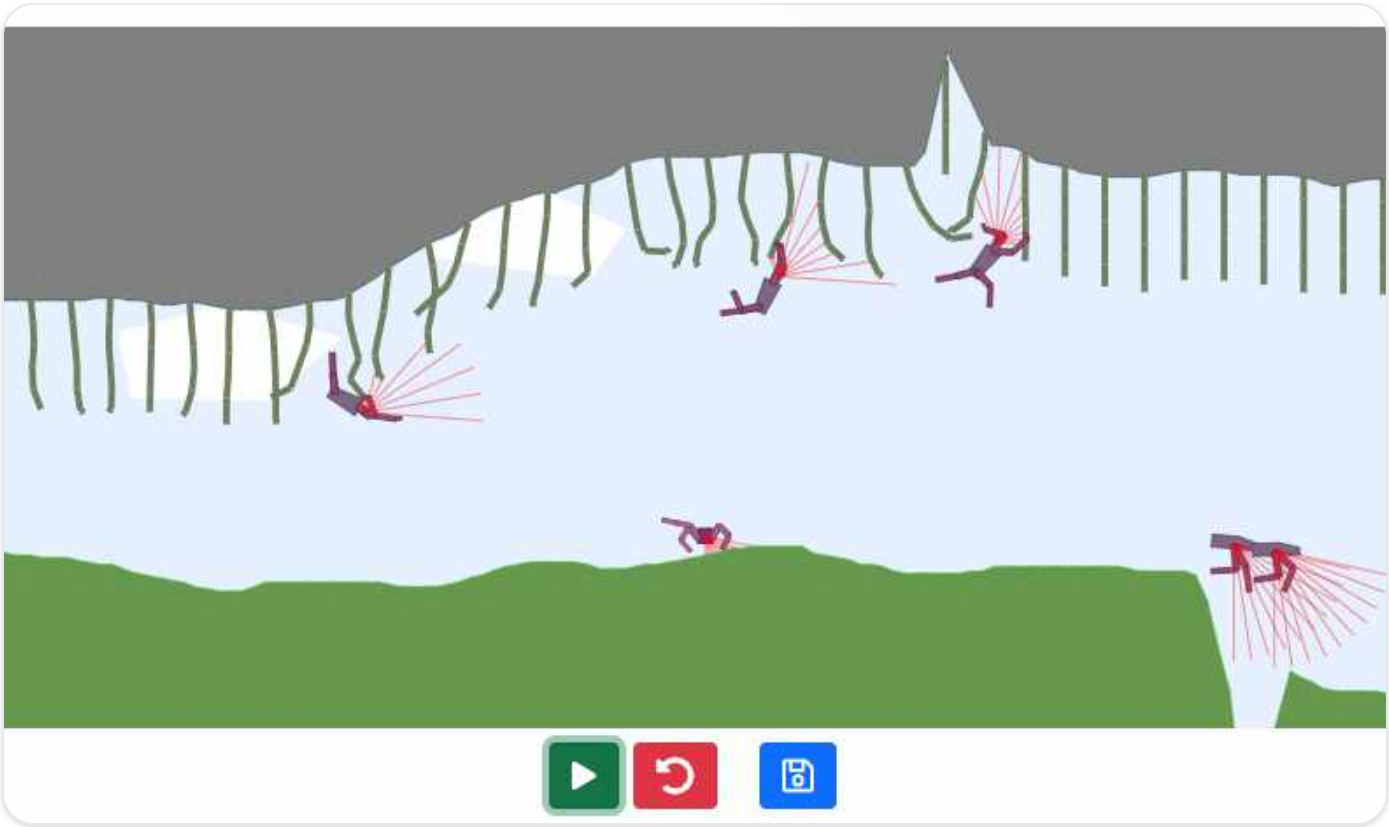
> *"… a family of mechanisms that automatically adapt the distribution of training data by learning to adjust the selection of learning situations to the capabilities of RL agents."*

As an example, OpenAI used **Domain Randomization** (they applied random variations on the environment) to make a robot hand solve Rubik's Cubes.



OpenAI - Solving Rubik's Cube with a Robot Hand

Finally, you can play with the robustness of agents trained in the TeachMyAgent benchmark by controlling environment variations or even drawing the



https://huggingface.co/spaces/flowers-team/Interactive_DeepRL_Demo

Further reading

For more information, we recommend that you check out the following resources:

Overview of the field

- Automatic Curriculum Learning For Deep RL: A Short Survey
- Curriculum for Reinforcement Learning

Recent methods

- Evolving Curricula with Regret-Based Environment Design
- Curriculum Reinforcement Learning via Constrained Optimal Transport
- Prioritized Level Replay

Author

This section was written by Clément Romac

<> Update on GitHub

← Language models in RL

Interesting environments to try →

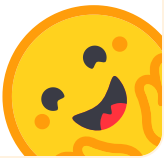
Search models, datasets, users...

Deep RL Course documentation
(Optional) How to customize the environment

Join the Hugging Face community

and get access to the augmented documentation experience

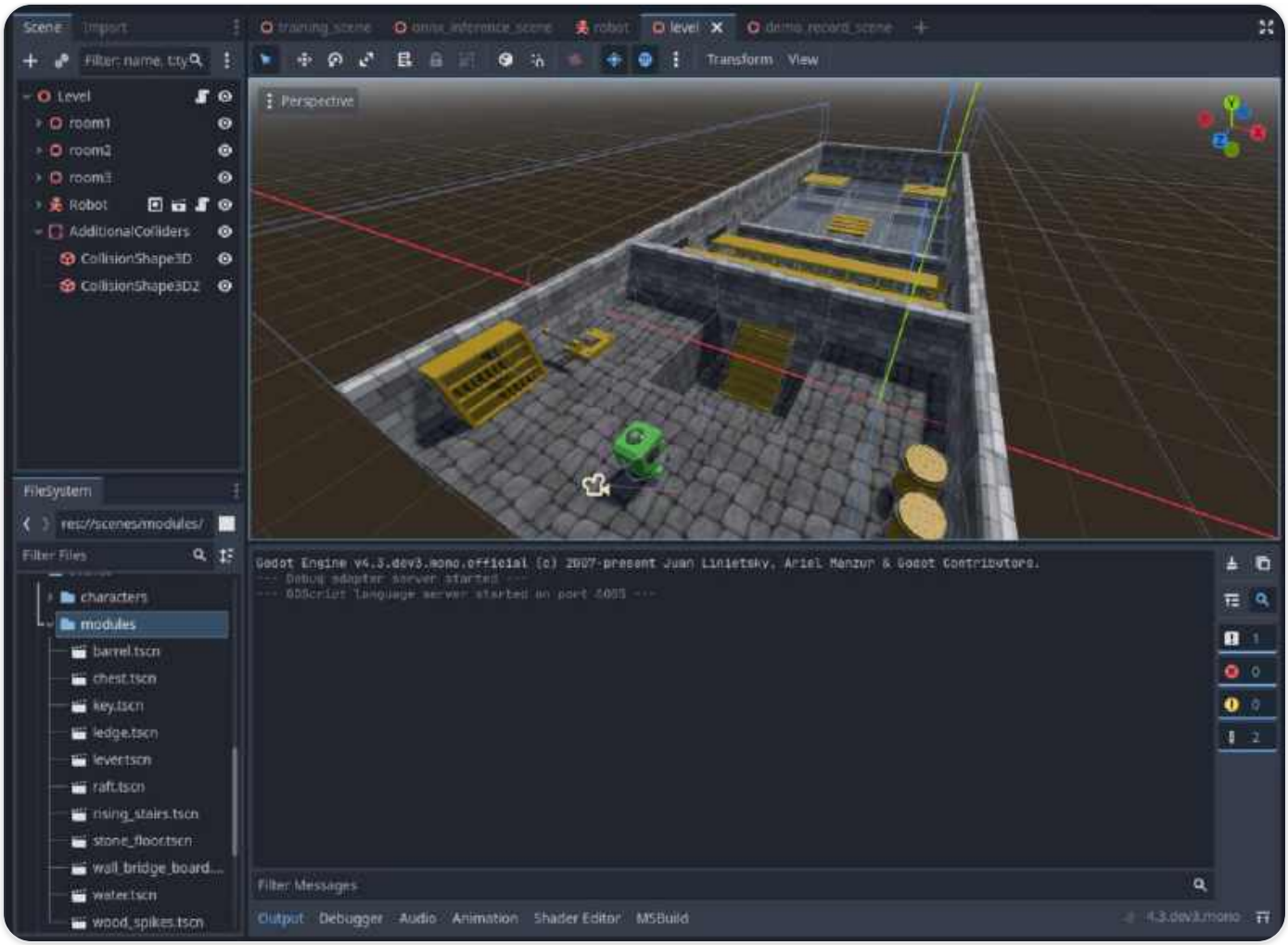
Sign Up to get started



(Optional) How to customize the environment

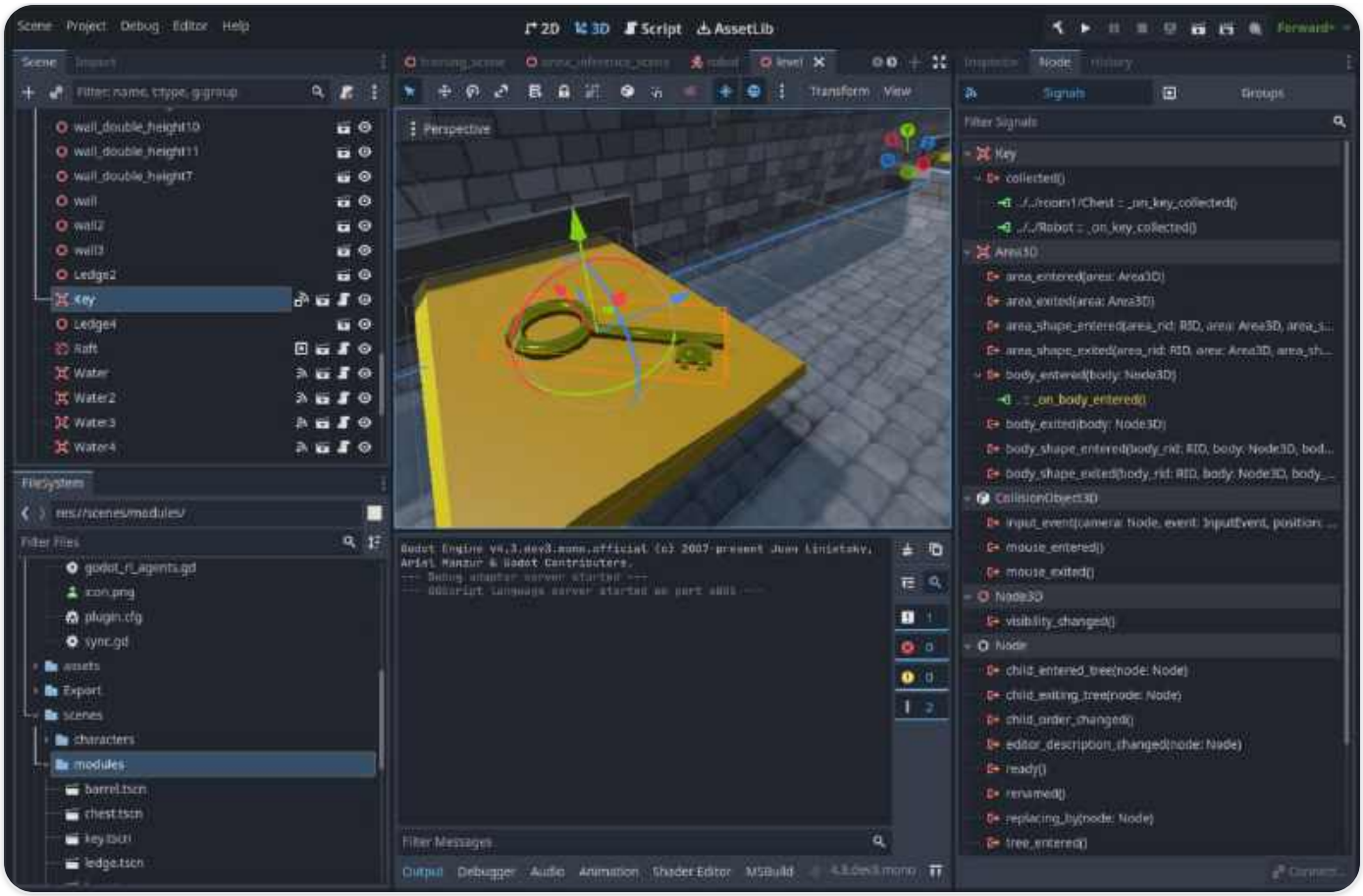
If you'd like to customize the game level, open the level scene `res://scenes/level.tscn`, then open the `res://scenes/modules/` folder in the Godot FileSystem:

Ask HuggingChat

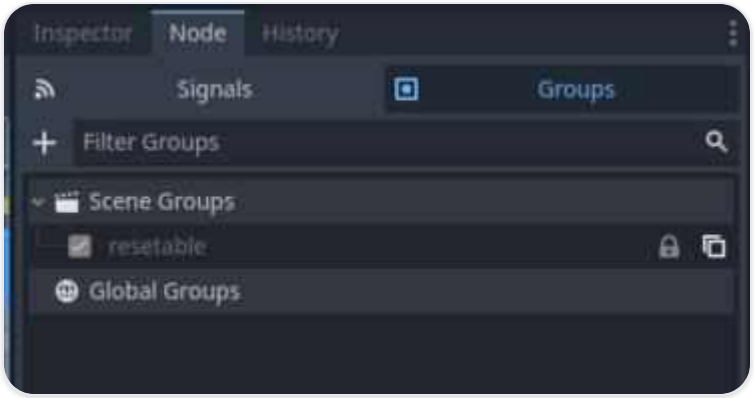


The level contains 3 rooms made using the modules, robot, and some additional colliders which prevent the ability to complete the level by climbing on a wall in the first room and reaching the key that way. By adding the modules to the scene, you can add new rooms and items.

If you click on the Key node (it's in `Room3`, you can also search for it), then click on `Node > Signals`, you will see that the `collected` signal is connected to both the robot and the chest. We use this to track whether the robot has collected the key, and to unlock the chest. The same system is applied for using the lever to activate the stairs, and if you add more levers/stairs/keys, you can connect them using signals.



If you switch to `Groups`, you will see that the key is a member of the `resetable` group. In the same group we have the raft, lever, chest, player, and can add any node that needs to be reset when the episode resets.



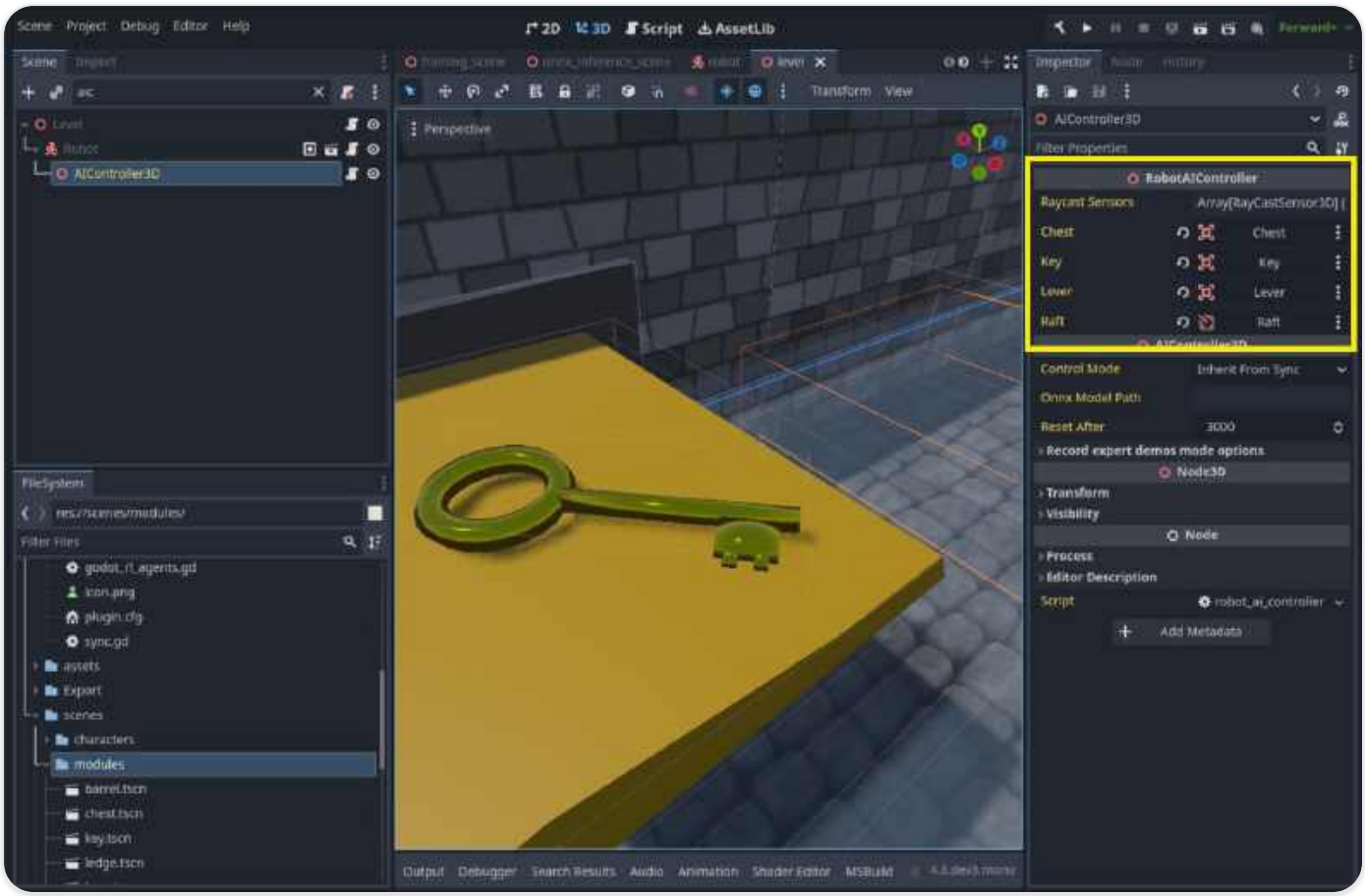
For this to work, every object that is in the `resetable` group also needs to implement the `reset()` method, which takes care of resetting that object.

Because we have multiple instances of the level scene f
but only those within the same scene. In `level_manager.gd`, we have a method

`reset_all_resetables()` that takes care of this, and it is called by the robot script when resetting is needed.

After changing the level size, updating the `level_size` variable in `robot_ai_controller.gd` is also needed. For this, just roughly measure the longest dimension of the level, and update the variable.

If you change the amount of objects that need to be tracked by the `AIController` (levers, rafts, etc.), you will need to update the relevant code in the script, include export properties for those objects, then connect them in the inspector properties of `AIController` in the level scene:



After this, you may also need to update the same properties of the `AIController` in the demo record scene as well.

<> Update on GitHub

← Train our robot

Conclusion →

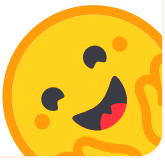
Search models, datasets, users...

Deep RL Course documentation
(Optional) the Policy Gradient Theorem

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



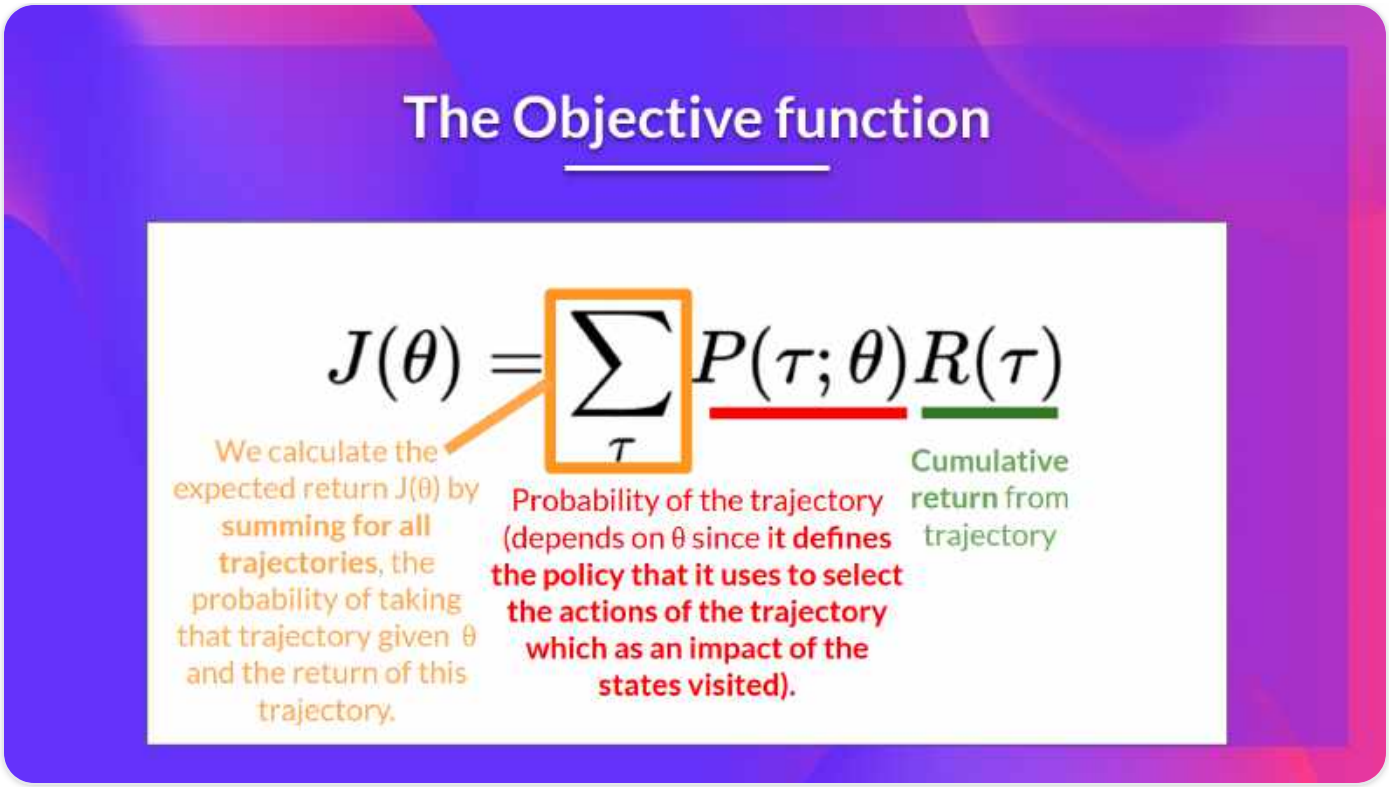
(Optional) the Policy Gradient Theorem

In this optional section where we're **going to study how we differentiate the objective function that we will use to approximate the policy gradient**.

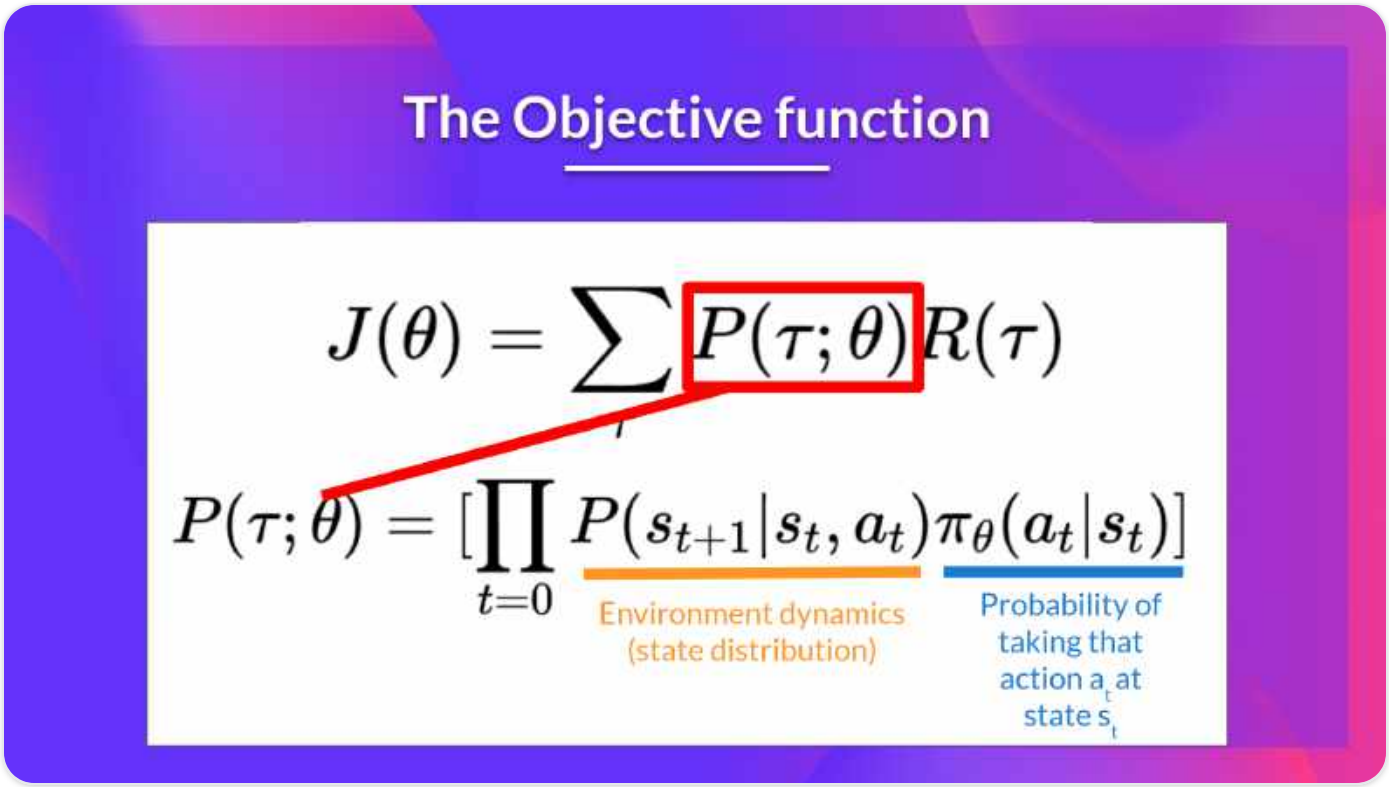
Let's first recap our different formulas:

1. The Objective function

Ask HuggingChat



2. The probability of a trajectory (given that action comes from π_θ):



So we have: $\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta) R(\tau)$

We can rewrite the gradient of the sum as the sum of the gradient: $= \sum_\tau \nabla_\theta (P(\tau;\theta)R(\tau)) = \sum_\tau \nabla_\theta P(\tau;\theta)R(\tau)$ as $R(\tau)$ is not dependent on θ

We then multiply every term in the sum by $\frac{P(\tau;\theta)}{P(\tau;\theta)}$ (which is possible since it's = 1) $= \sum_\tau \frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta)R(\tau)$

We can simplify further this since $\frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta) = P(\tau;\theta) \frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}$.

Thus we can rewrite the sum as $P(\tau;\theta) \frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} = \sum_\tau P(\tau;\theta) \frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} R(\tau)$

We can then use the *derivative log trick* (also called *likelihood ratio trick* or *REINFORCE trick*), a simple rule in calculus that implies that $\nabla_x log f(x) = \frac{\nabla_x f(x)}{f(x)}$

So given we have $\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}$ we transform it as $\nabla_\theta log P(\tau|\theta)$

So this is our likelihood policy gradient: $\nabla_\theta J(\theta) = \sum_\tau P(\tau;\theta) \nabla_\theta log P(\tau;\theta) R(\tau)$

Thanks for this new formula, we can estimate the gradient using trajectory samples (we can approximate the likelihood ratio policy gradient with sample-based estimate if you prefer).
$\nabla_\theta J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta log P(\tau^{(i)};\theta) R(\tau^{(i)})$ where each $\tau^{(i)}$ is a sampled trajectory.

But we still have some mathematics work to do there: we need to simplify $\nabla_\theta log P(\tau|\theta)$

We know that: $\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta log[\mu(s_0) \prod_{t=0}^{H} P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)}) \pi_\theta(a_t^{(i)}|s_t^{(i)})]$

Where $\mu(s_0)$ is the initial state distribution and $P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)})$ is the state transition dynamics of the MDP.

We know that the log of a product is equal to the sum of the logs: $\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta \left[log \mu(s_0) + \sum_{t=0}^{H} log P(s_{t+1}^{(i)}|s_t^{(i)} a_t^{(i)}) + \sum_{t=0}^{H} log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \right]$

We also know that the gradient of the sum is equal to the sum of gradient: $\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta log \mu(s_0) + \nabla_\theta \sum_{t=0}^{H} log P(s_{t+1}^{(i)}|s_t^{(i)} a_t^{(i)}) + \nabla_\theta \sum_{t=0}^{H} log \pi_\theta(a_t^{(i)}|s_t^{(i)})$

Since neither initial state distribution or state transition dynamics of the MDP are dependent ofθ, the derivate of both terms are 0. So we can remove them:

Since:$\nabla_\theta \sum_{t=0}^{H} logP(s_{t+1}^{(i)}|s_t^{(i)}a_t^{(i)}) = 0$ and$\nabla_\theta \mu(s_0) = 0$ $\nabla_\theta logP(\tau^{(i)};\theta) = \nabla_\theta \sum_{t=0}^{H} log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

We can rewrite the gradient of the sum as the sum of gradients: $\nabla_\theta logP(\tau^{(i)};\theta) = \sum_{t=0}^{H} \nabla_\theta log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

So, the final formula for estimating the policy gradient is: $\nabla_\theta J(\theta) = \hat{g} = \frac{1}{m}\sum_{i=1}^{m}\sum_{t=0}^{H} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)})R(\tau^{(i)})$

<> Update on GitHub

← Diving deeper into policy-gradient

Glossary →

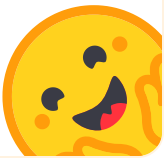
Search models, datasets, users...

Deep RL Course documentation
(Optional) What is Curiosity in Deep Reinforcement Learning?

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



(Optional) What is Curiosity in Deep Reinforcement Learning?

This is an (optional) introduction to Curiosity. If you want to learn more, you can read two additional articles where we dive into the mathematical details:

- Curiosity-Driven Learning through Next State Prediction
- Random Network Distillation: a new take on Curiosity-Driven Learning

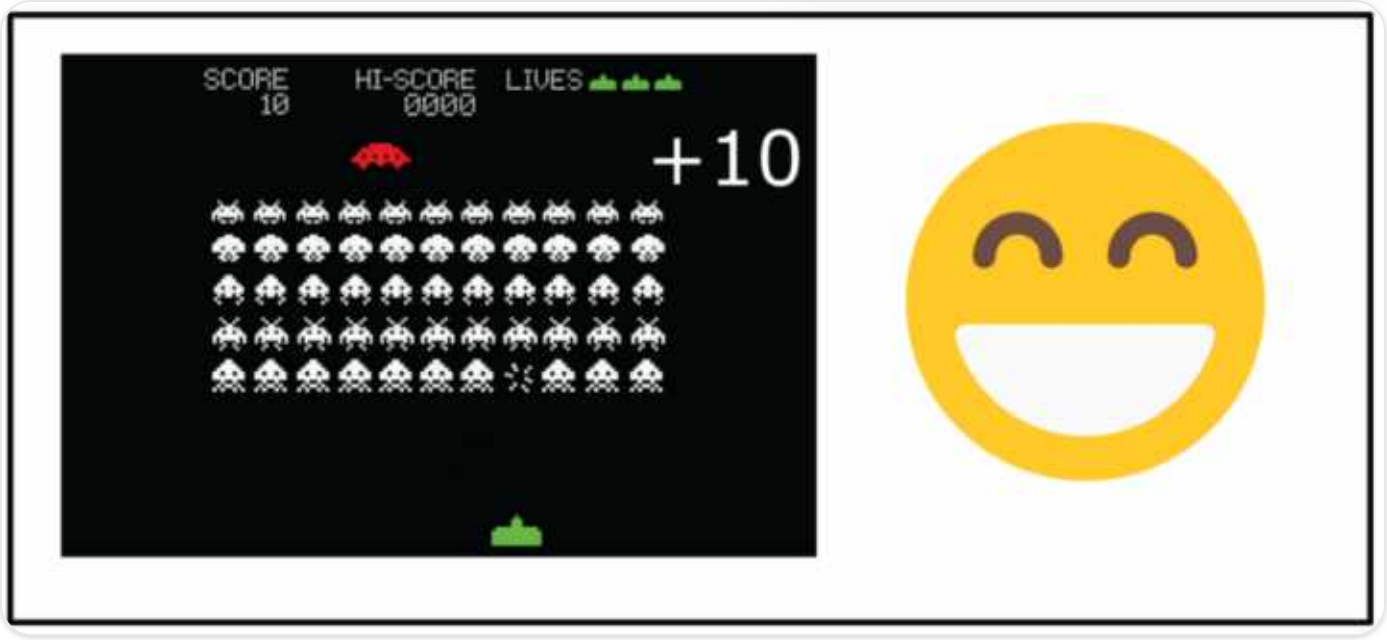
Two Major Problems in Modern RL

To understand what Curiosity is, we first need to understand the two major problems with RL:

First, the *sparse rewards problem:* that is, **most rewards do not contain information, and hence are set to zero**.

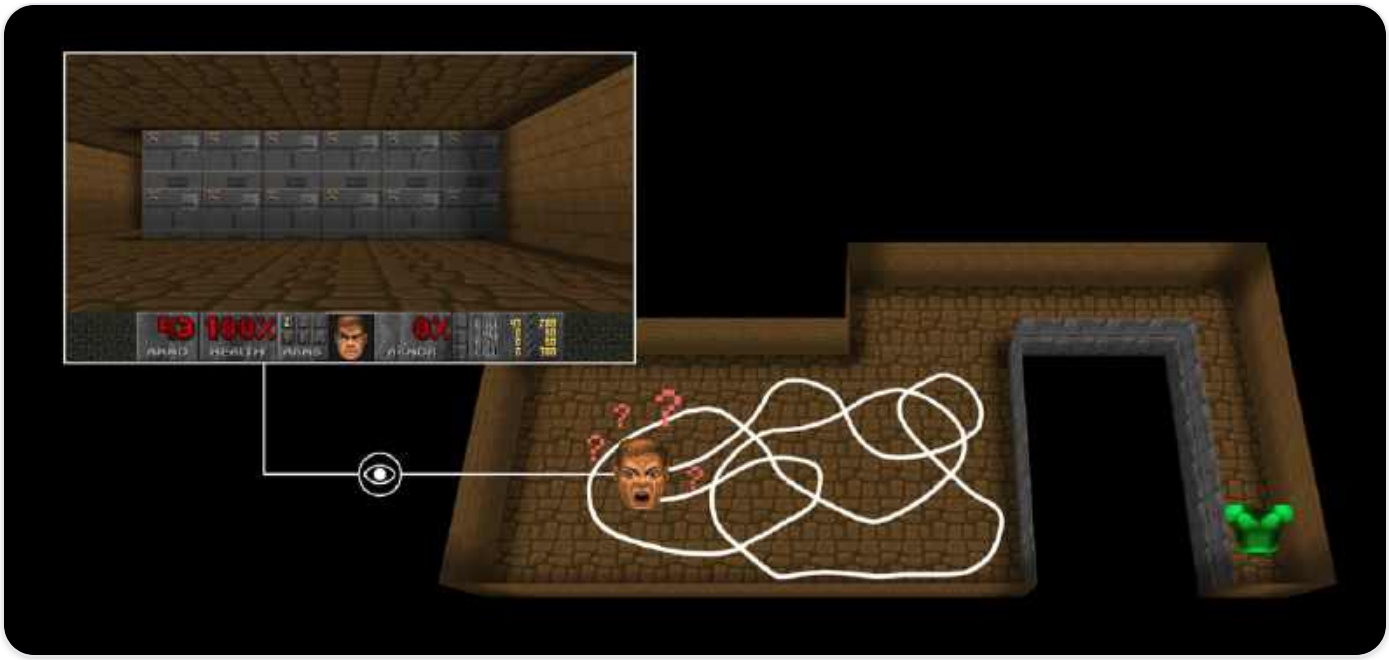
Remember that RL is based on the *reward hypothesis*, which is the idea that each goal can be described as the maximization of the rewards. Therefore, rewards act as feedback for RL agents; **if they don't receive any, their knowledge of which action is appropriate (or not) cannot change**.

Ask HuggingChat



Source: Thanks to the reward, our agent knows that this action at that state was good

For instance, in Vizdoom, a set of environments based on the game Doom "DoomMyWayHome," your agent is only rewarded **if it finds the vest**. However, the vest is far away from your starting point, so most of your rewards will be zero. Therefore, if our agent does not receive useful feedback (dense rewards), it will take much longer to learn an optimal policy, and **it can spend time turning around without finding the goal**.



The second big problem is that **the extrinsic reward function is handmade; in each environment, a human has to implement a reward function**. But how we can scale that in big and complex environments?

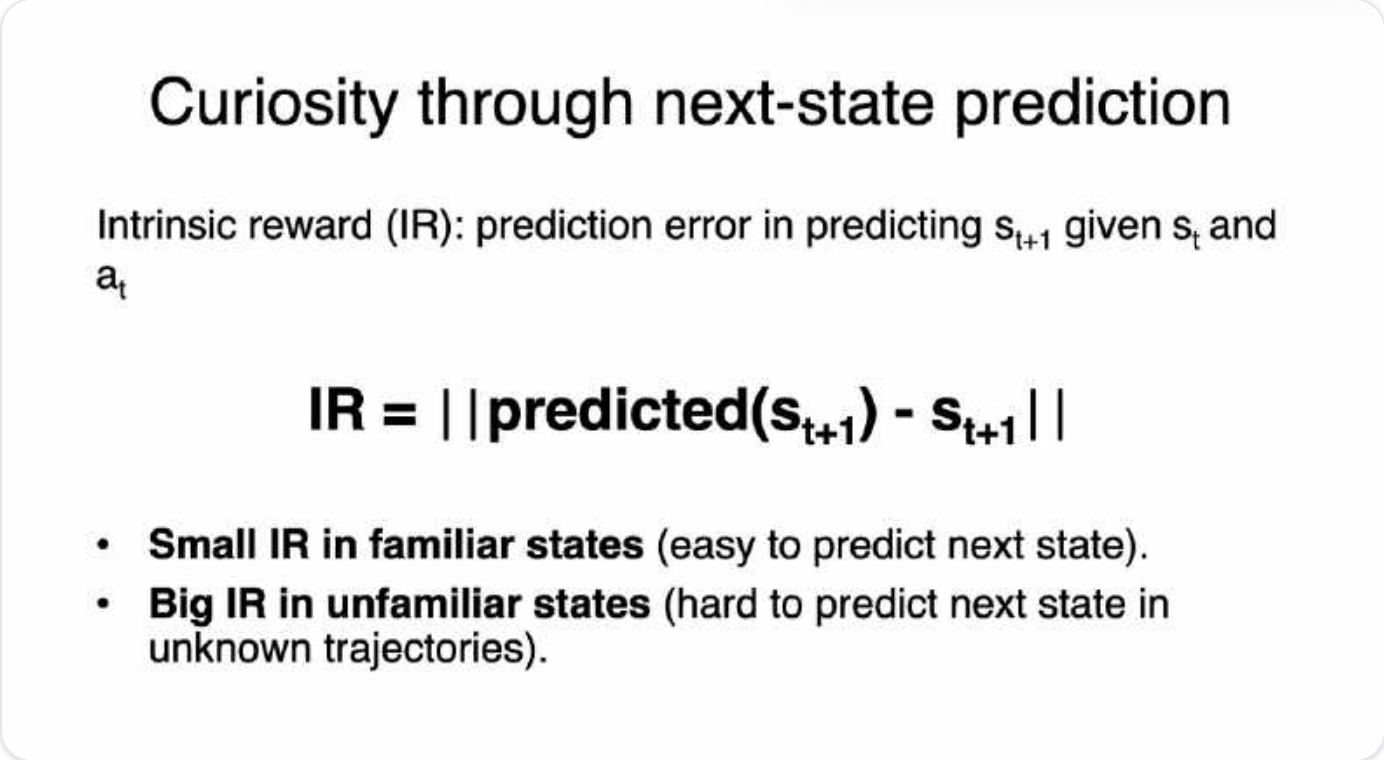
So what is Curiosity?

A solution to these problems is **to develop a reward function intrinsic to the agent, i.e., generated by the agent itself**. The agent will act as a self-learner since it will be the student and its own feedback master.

This intrinsic reward mechanism is known as Curiosity because this reward pushes the agent to explore states that are novel/unfamiliar. To achieve that, our agent will receive a high reward when exploring new trajectories.

This reward is inspired by how humans act. **We naturally have an intrinsic desire to explore environments and discover new things**.

There are different ways to calculate this intrinsic reward. The classical approach (Curiosity through next-state prediction) is to calculate Curiosity **as the error of our agent in predicting the next state, given the current state and action taken**.



Because the idea of Curiosity is to **encourage our agent to perform actions that reduce the uncertainty in the agent's ability to predict the consequences of its actions** (uncertainty will be higher in areas where the agent has spent less time or in areas with complex dynamics).

If the agent spends a lot of time on these states, it will be good at predicting the next state (low Curiosity). On the other hand, if it's in a new, unexplored state, it will be hard to predict the following state (high Curiosity).



Using Curiosity will push our agent to favor transitions with high prediction error (which will be higher in areas where the agent has spent less time, or in areas with complex dynamics) and **consequently better explore our environment**.

There's also **other curiosity calculation methods**. ML-Agents uses a more advanced one called Curiosity through random network distillation. This is out of the scope of the tutorial but if you're interested I wrote an article explaining it in detail.

<> Update on GitHub

← The Pyramids environment

Hands-on →

Search models, datasets, users...

Deep RL Course documentation
A Q-Learning example

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

A Q-Learning example

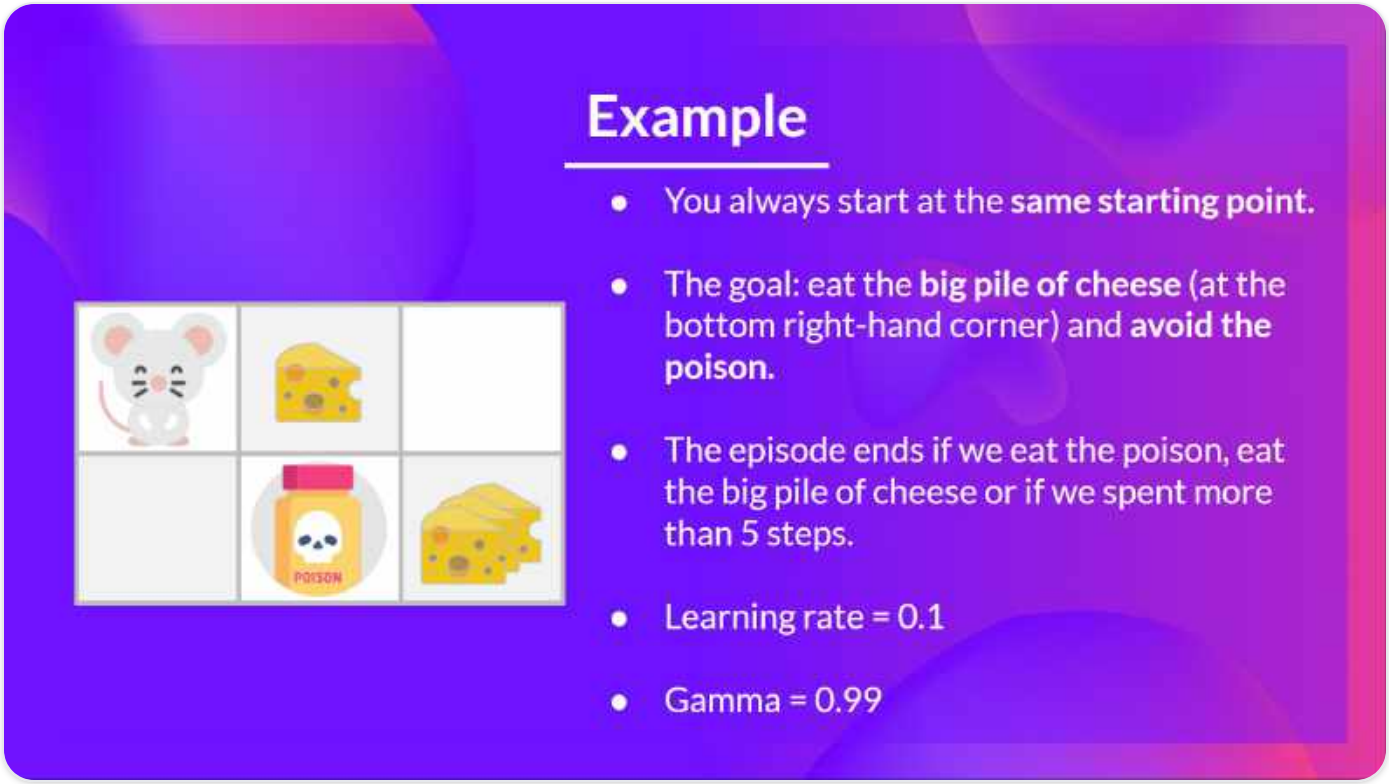
To better understand Q-Learning, let's take a simple example:



- You're a mouse in this tiny maze. You always **start at the same starting point.**
- The goal is **to eat the big pile of cheese at the bottom right-hand corner** and avoid the poison. After all, who doesn't like cheese?
- The episode ends if we eat the poison, **eat the big** steps.

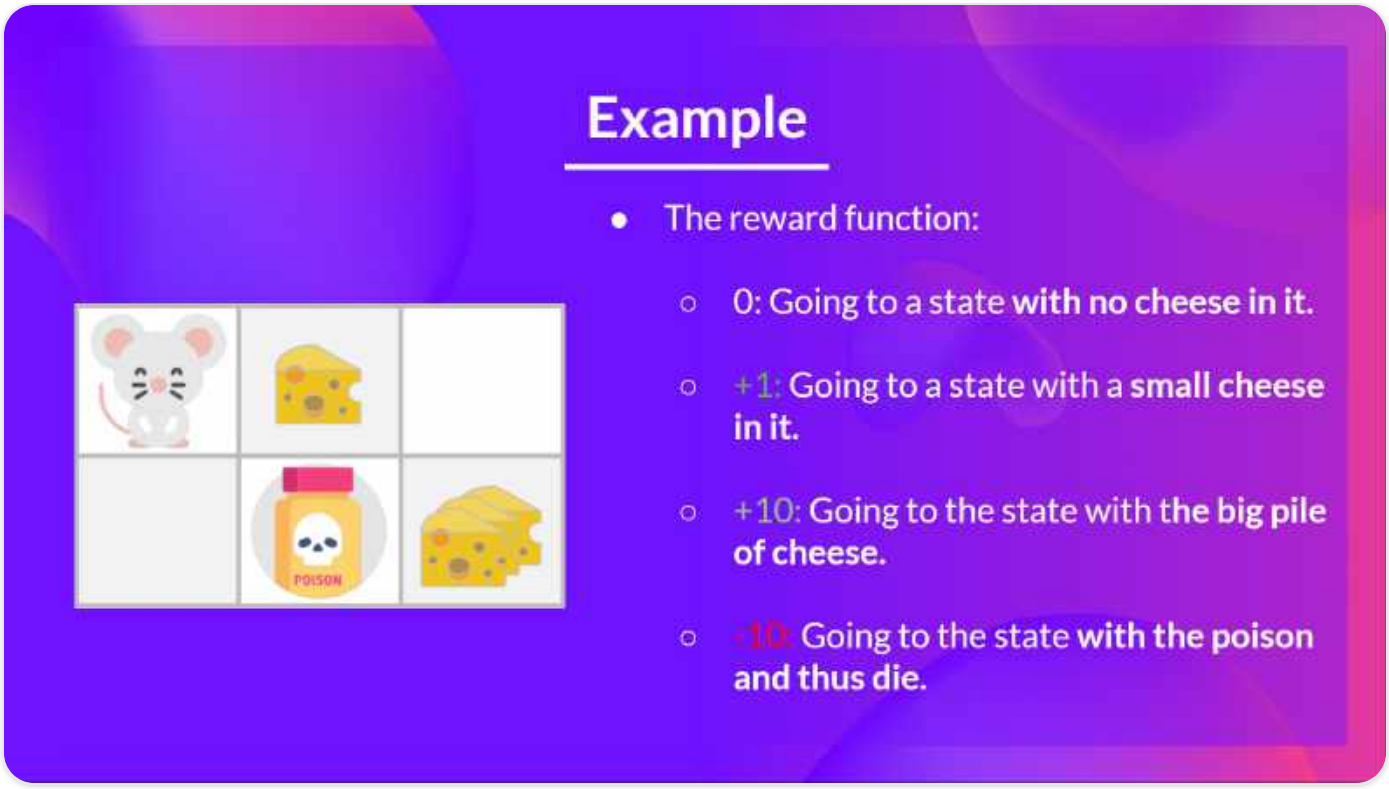
Ask HuggingChat

- The learning rate is 0.1
- The discount rate (gamma) is 0.99



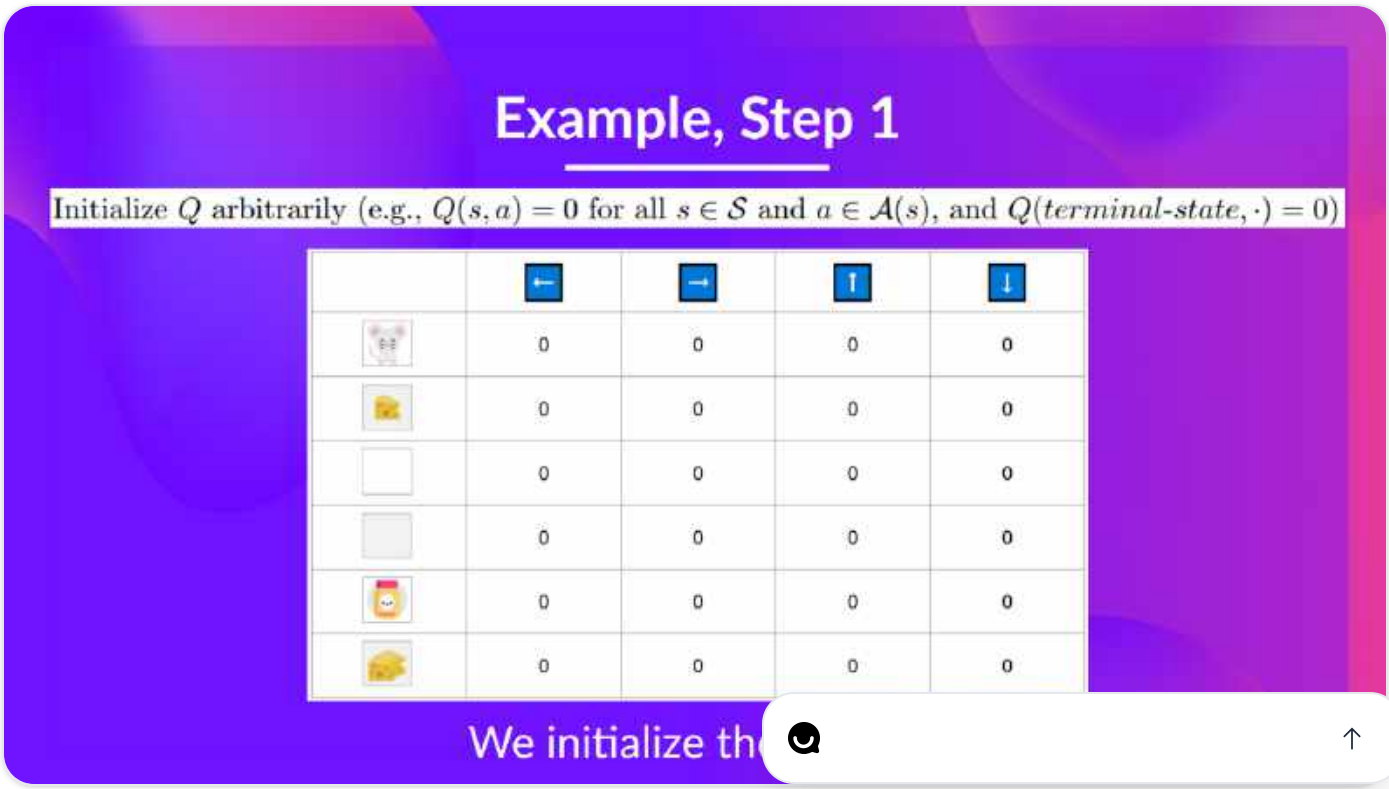
The reward function goes like this:

- **+0:** Going to a state with no cheese in it.
- **+1:** Going to a state with a small cheese in it.
- **+10:** Going to the state with the big pile of cheese.
- **-10:** Going to the state with the poison and thus dying.
- **+0** If we take more than five steps.



To train our agent to have an optimal policy (so a policy that goes right, right, down), **we will use the Q-Learning algorithm**.

Step 1: Initialize the Q-table



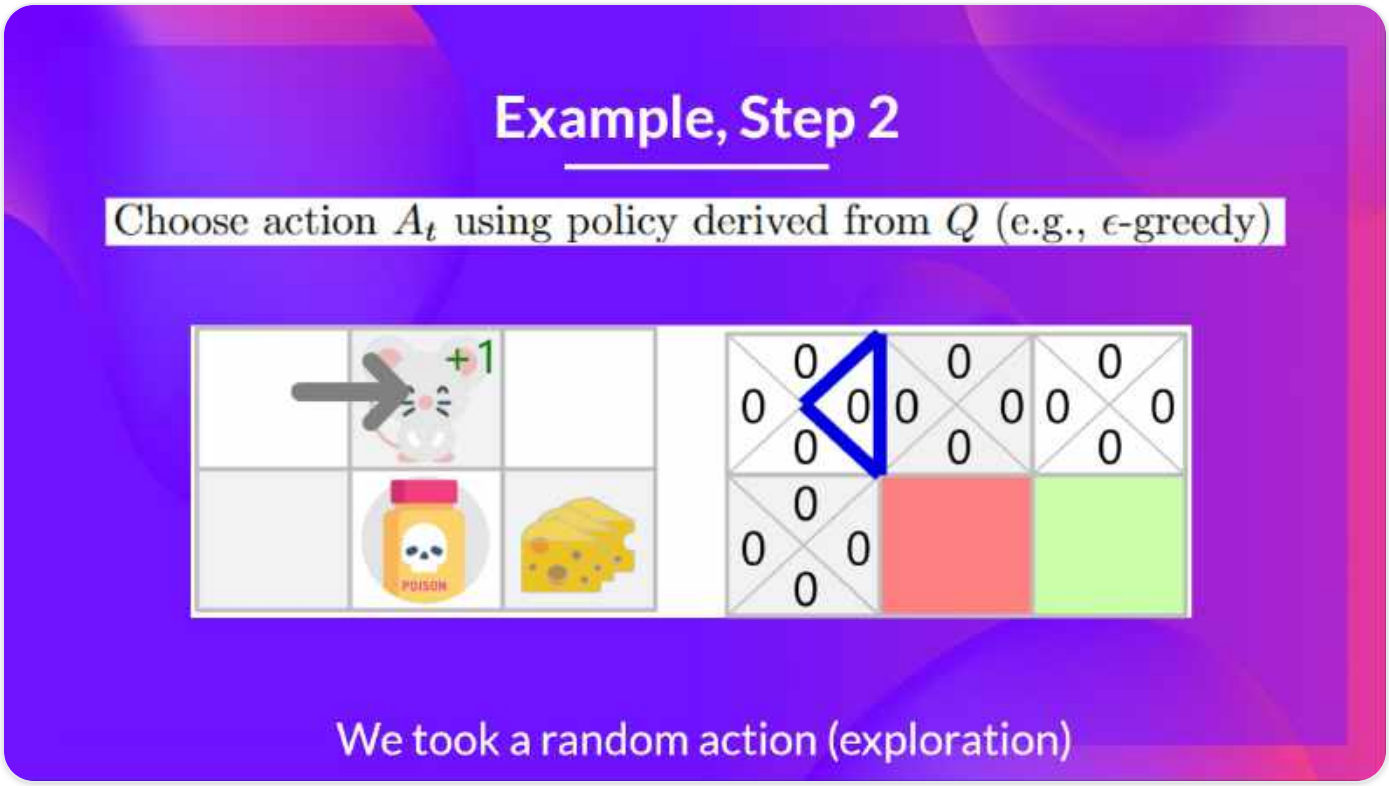
So, for now, **our Q-table is useless**; we need **to train our Q-function using the Q-Learning algorithm.**

Let's do it for 2 training timesteps:

Training timestep 1:

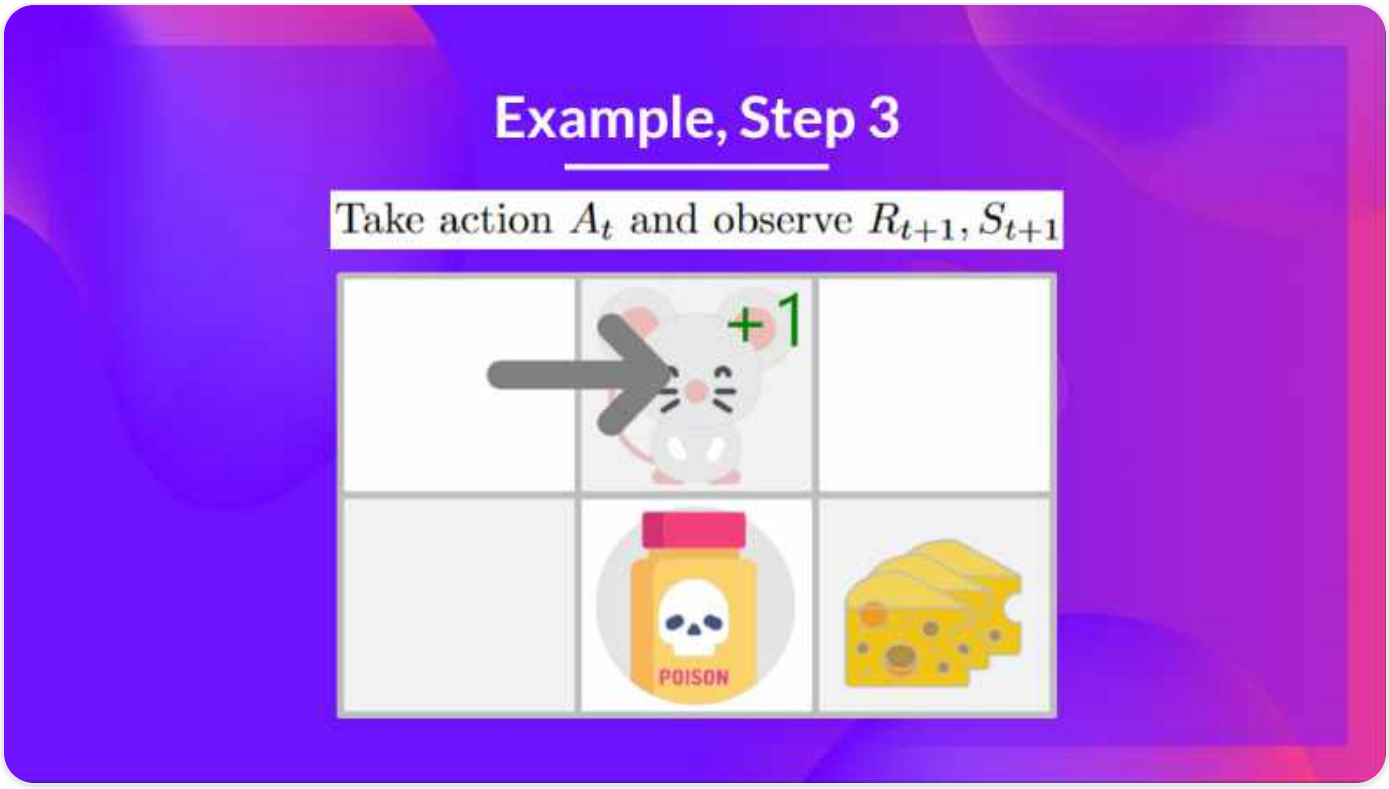
Step 2: Choose an action using the Epsilon Greedy Strategy

Because epsilon is big (= 1.0), I take a random action. In this case, I go right.



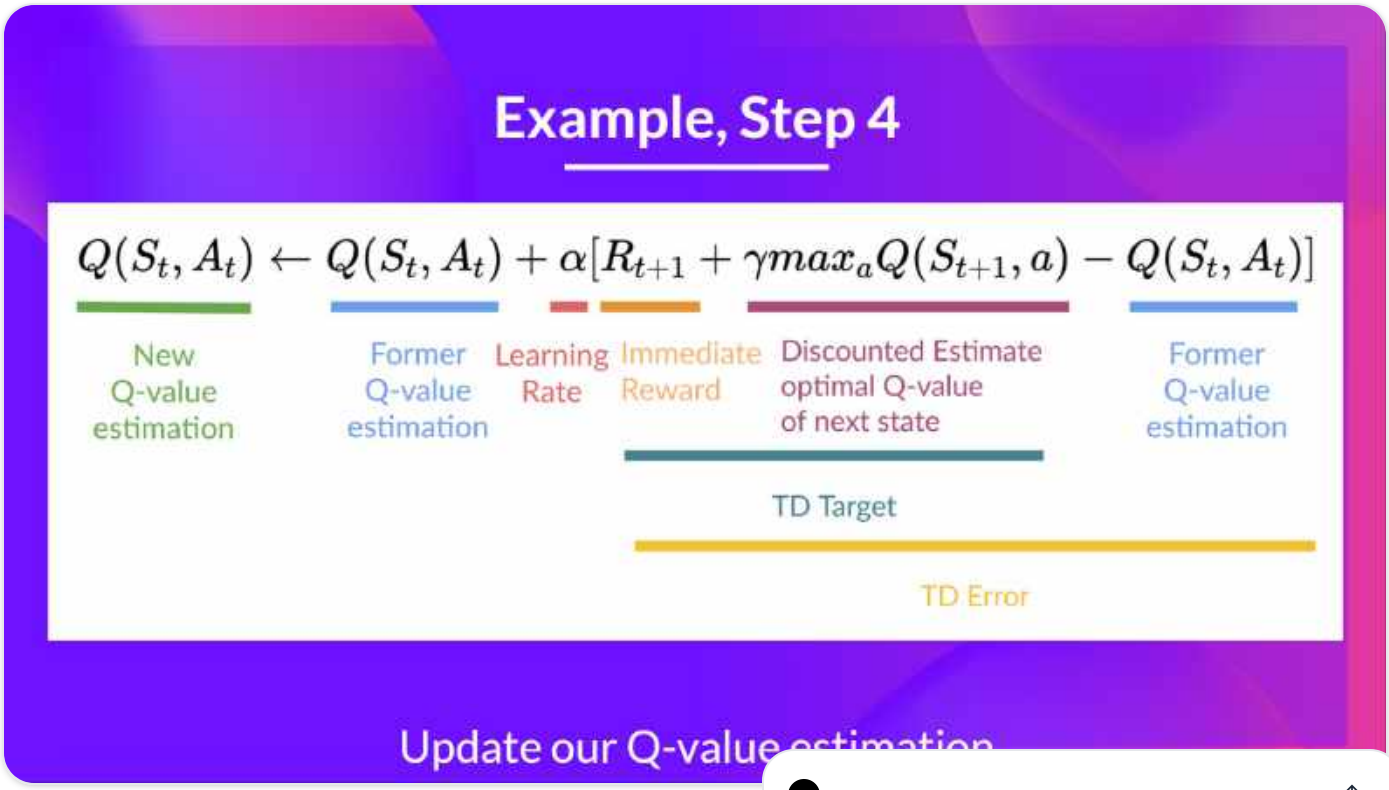
Step 3: Perform action At, get Rt+1 and St+1

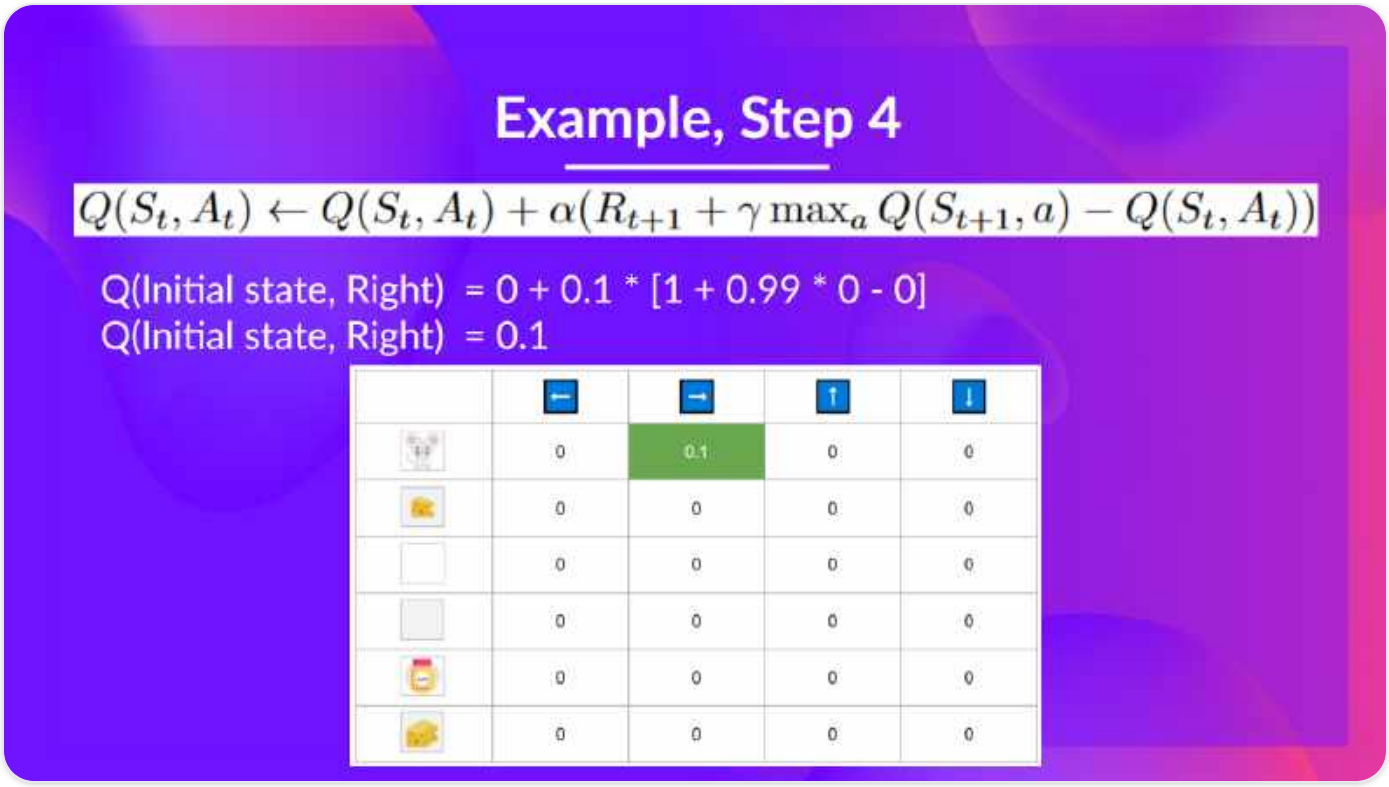
By going right, I get a small cheese, so $R_{t+1} = 1$ and I'm in a new state.



Step 4: Update Q(St, At)

We can now update$Q(S_t, A_t)$ using our formula.



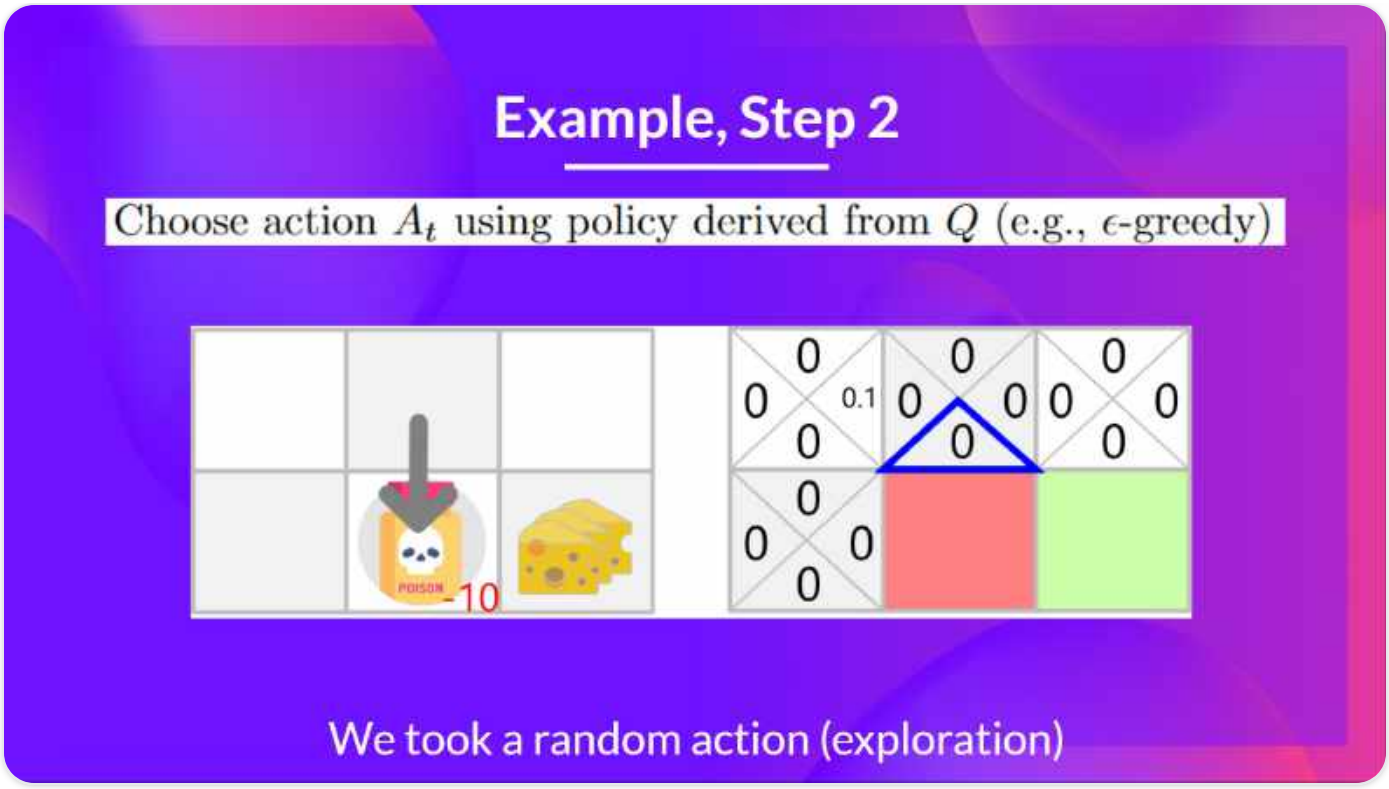


Training timestep 2:

Step 2: Choose an action using the Epsilon Greedy Strategy

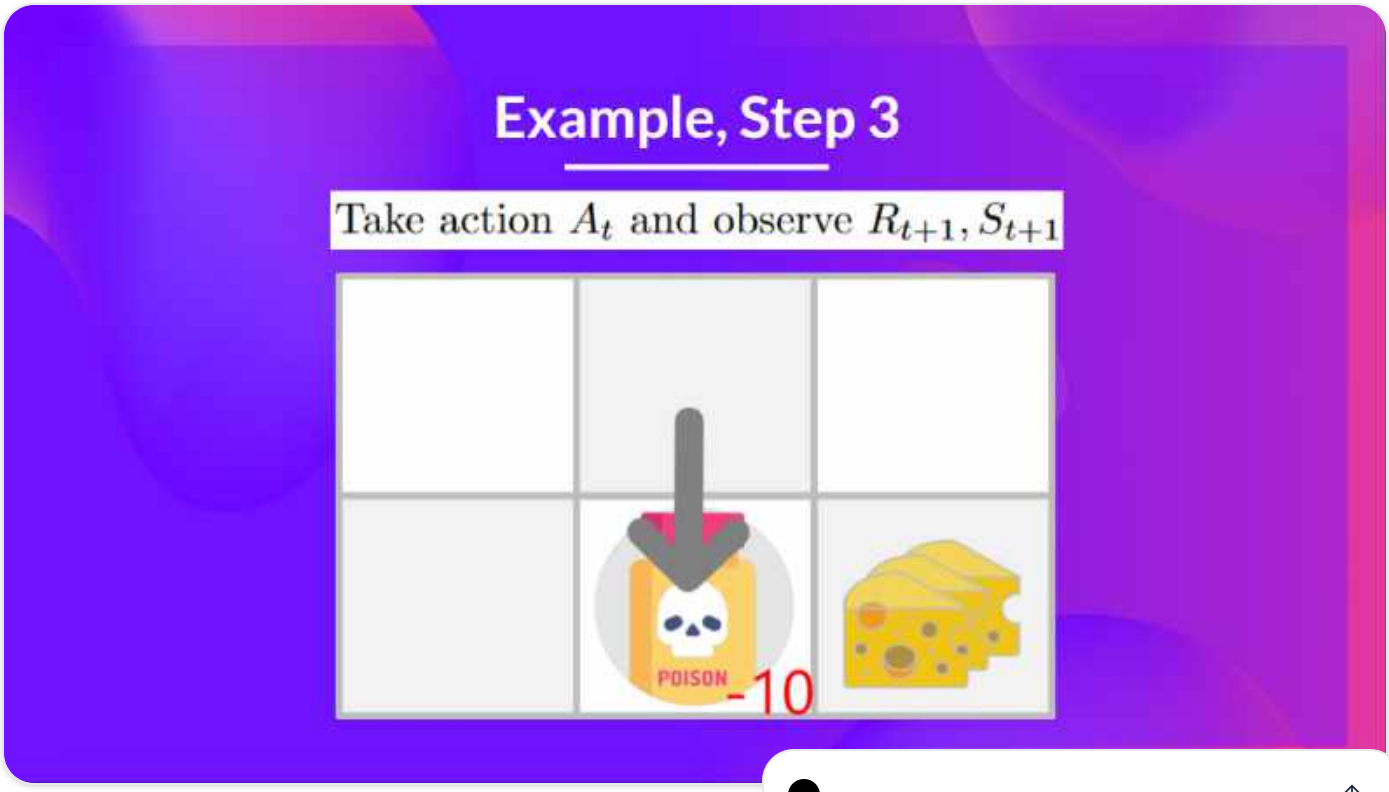
I take a random action again, since epsilon=0.99 is big. (Notice we decay epsilon a little bit because, as the training progress, we want less and less exploration).

I took the action ‘down’. **This is not a good action since it leads me to the poison.**

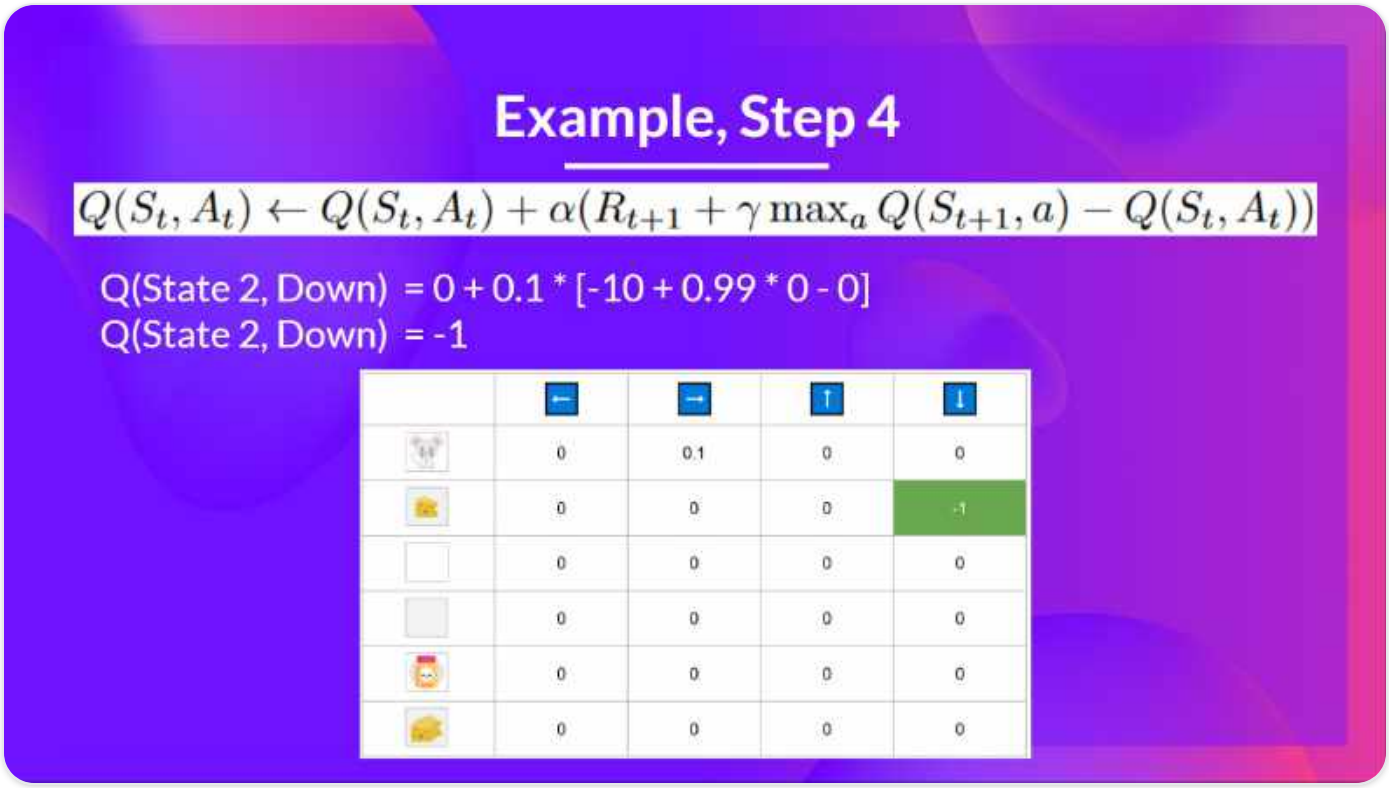


Step 3: Perform action At, get Rt+1 and St+1

Because I ate poison, **I get** $R_{t+1} = -10$**, and I die.**



Step 4: Update Q(St, At)



Because we're dead, we start a new episode. But what we see here is that, **with two explorations steps, my agent became smarter.**

As we continue exploring and exploiting the environment and updating Q-values using the TD target, the **Q-table will give us a better and better approximation. At the end of the training, we'll get an estimate of the optimal Q-function.**

<> Update on GitHub

← Introducing Q-Learning Q-Learning Recap →

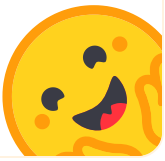
Search models, datasets, users...

Deep RL Course documentation
Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym

Ask a question Open in Colab

Now that you've studied the theory behind Advantage Actor Critic (A2C), **you're ready to train your A2C agent** using Stable-Baselines3 in a robotic environment. And train a:

- A robotic arm to move to the correct position.

We're going to use

- panda-gym

To validate this hands-on for the certification process, you need to push your two trained models to the Hub and get the following results:

- `PandaReachDense-v3` get a result of >= -3.5.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

For more information about the certification process, check this section 👉
https://huggingface.co/deep-rl-course/en/unit0/introd

Ask HuggingChat

To start the hands-on click on Open In Colab button 👇 :

Open in Colab

Unit 6: Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym 🤖

🎮 Environments:

- Panda-Gym

📚 RL-Library:

- Stable-Baselines3

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Objectives of this notebook 🏆

At the end of the notebook, you will:

- Be able to use **Panda-Gym**, the environment library.
- Be able to **train robots using A2C**.
- Understand why **we need to normalize the input**.
- Be able to **push your trained agent and the code to the Hub** with a nice video replay and an evaluation score 🔥 .

Prerequisites 🏗️

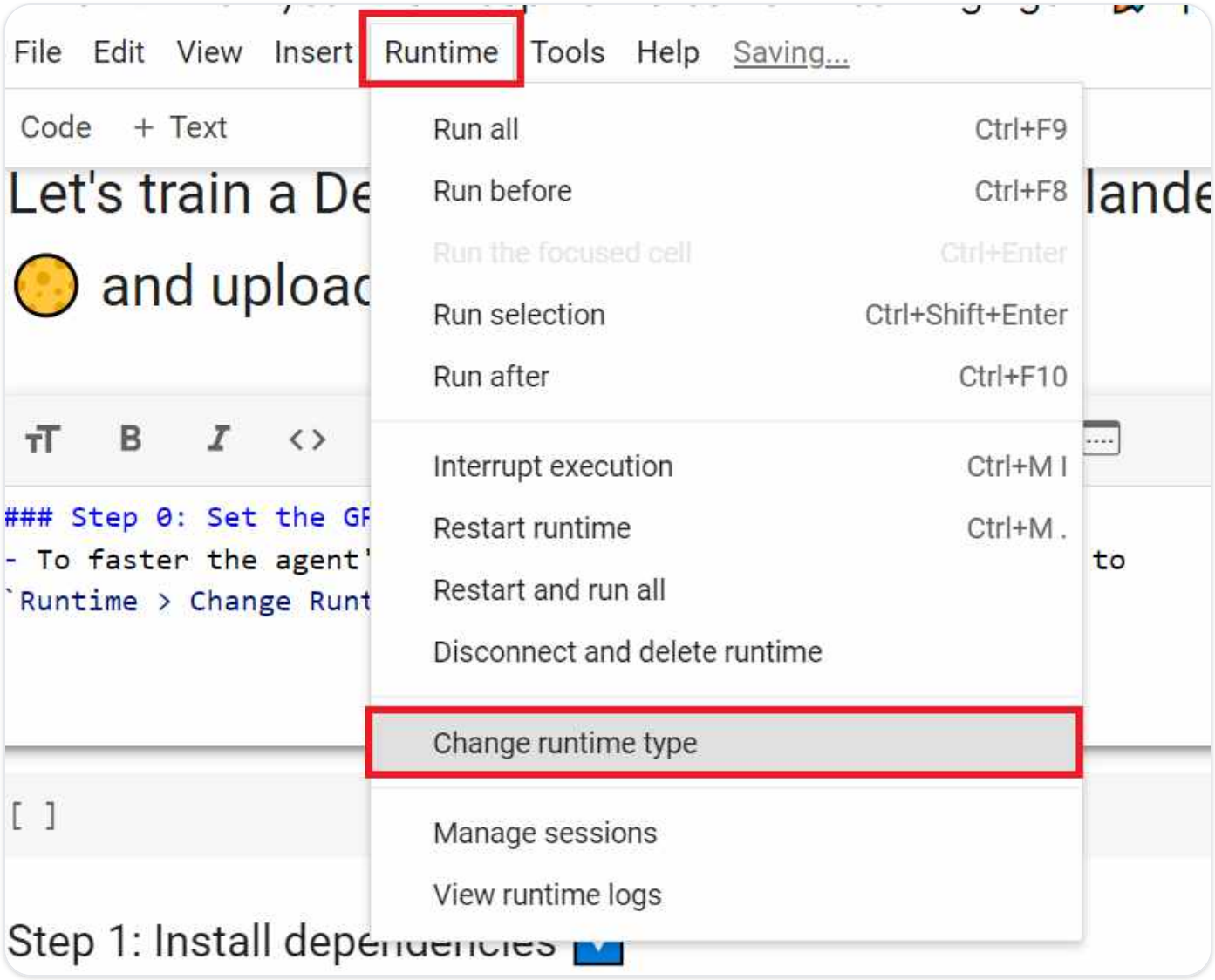
Before diving into the notebook, you need to:

🔲 📚 Study Actor-Critic methods by reading Unit 6 🤗

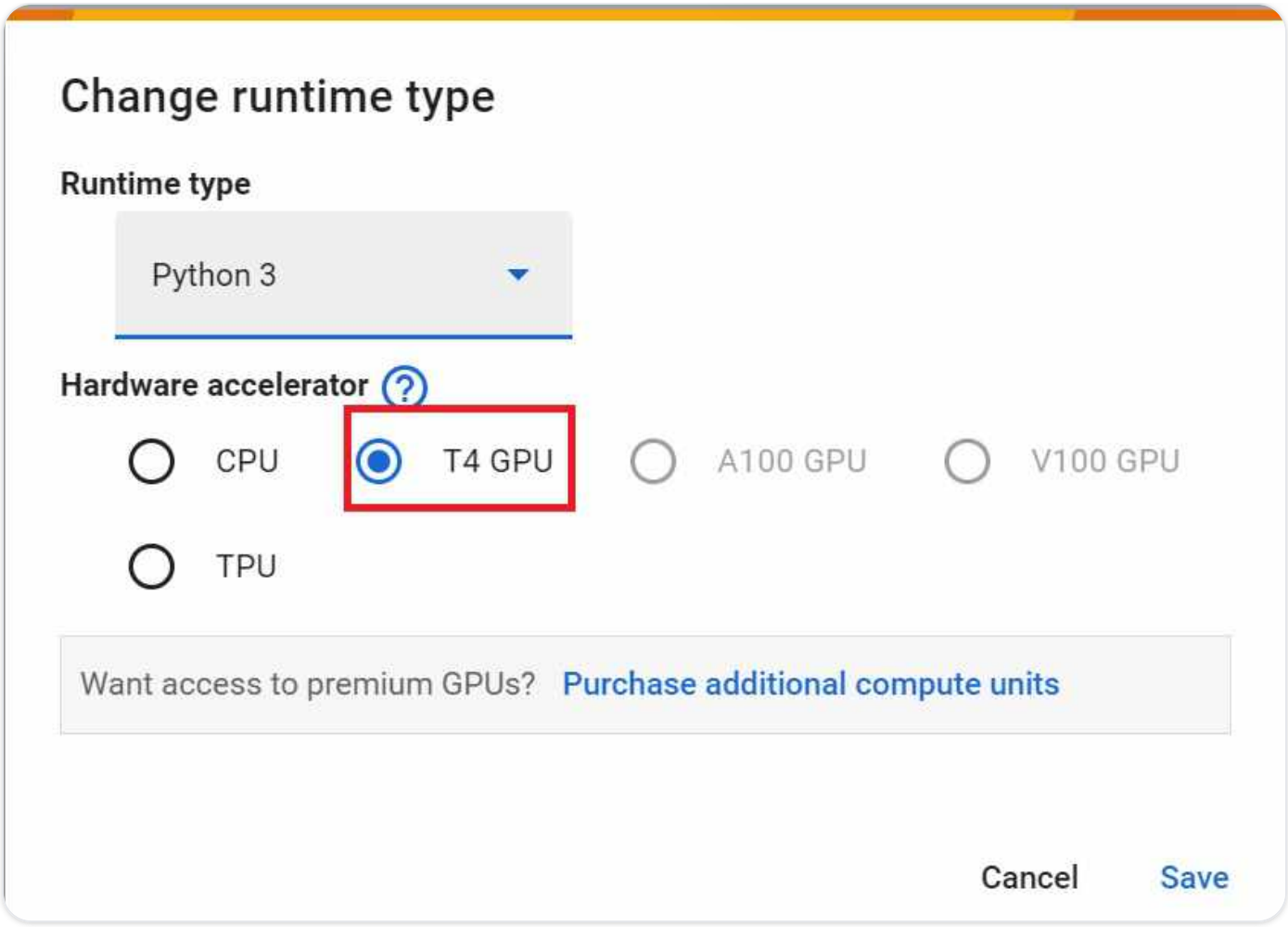
Let's train our first robots 🤖

Set the GPU 💪

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`



- `Hardware Accelerator > GPU`



Create a virtual display 🔽

During the notebook, we'll need to generate a replay video. To do so, with colab, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

The following cell will install the librairies and create and run a virtual screen 🖥️

```
%%capture
!apt install python-opengl
!apt install ffmpeg
!apt install xvfb
!pip3 install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display
```

```
virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Install dependencies 🔽

We'll install multiple ones:

- `gymnasium`
- `panda-gym`: Contains the robotics arm environments.
- `stable-baselines3`: The SB3 deep reinforcement learning library.
- `huggingface_sb3`: Additional code for Stable-baselines3 to load and upload models from the Hugging Face 🤗 Hub.
- `huggingface_hub`: Library allowing anyone to work with the Hub repositories.

```
!pip install stable-baselines3[extra]
!pip install gymnasium
!pip install huggingface_sb3
!pip install huggingface_hub
!pip install panda_gym
```

Import the packages 📦

```
import os

import gymnasium as gym
import panda_gym

from huggingface_sb3 import load_from_hub, package_to_hub

from stable_baselines3 import A2C
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
from stable_baselines3.common.env_util import make_vec_env

from huggingface_hub import notebook_login
```

PandaReachDense-v3 🦾

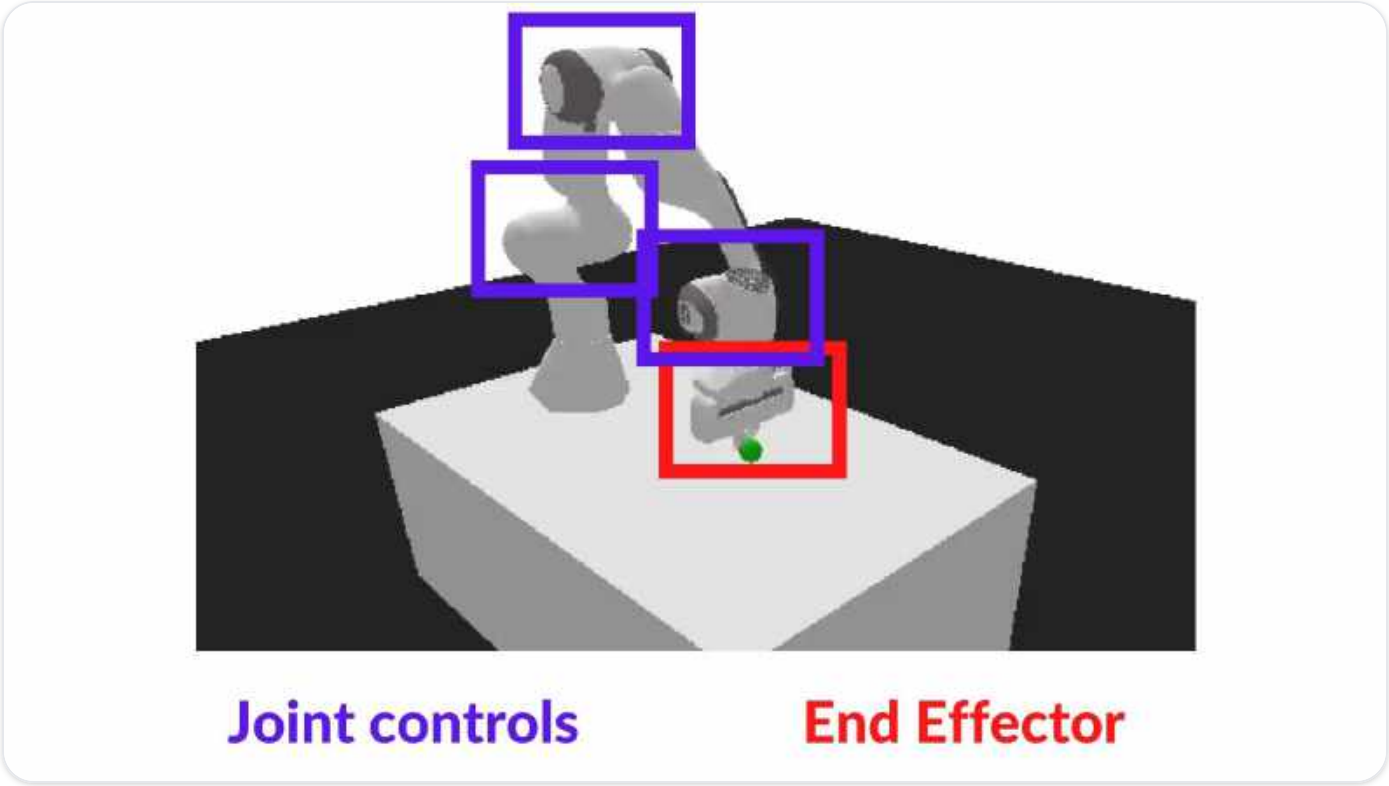
The agent we're going to train is a robotic arm that needs to do controls (moving the arm and using the end-effector).

In robotics, the *end-effector* is the device at the end of a robotic arm designed to interact with the environment.

In `PandaReach`, the robot must place its end-effector at a target position (green ball).

We're going to use the dense version of this environment. It means we'll get a *dense reward function* that **will provide a reward at each timestep** (the closer the agent is to completing the task, the higher the reward). Contrary to a *sparse reward function* where the environment **return a reward if and only if the task is completed**.

Also, we're going to use the *End-effector displacement control*, it means the **action corresponds to the displacement of the end-effector**. We don't control the individual motion of each joint (joint control).



This way **the training will be easier**.

Create the environment

The environment 🎮

In `PandaReachDense-v3` the robotic arm must place its end-effector at a target position (green ball).

```
env_id = "PandaReachDense-v3"

# Create the env
env = gym.make(env_id)

# Get the state space and action space
s_size = env.observation_space.shape
a_size = env.action_space
```

```
print("_____OBSERVATION SPACE_____ \n")
print("The State Space is: ", s_size)
print("Sample observation", env.observation_space.sample()) # Get a random observation
```

The observation space **is a dictionary with 3 different elements**:

- `achieved_goal`: (x,y,z) position of the goal.
- `desired_goal`: (x,y,z) distance between the goal position and the current object position.
- `observation`: position (x,y,z) and velocity of the end-effector (vx, vy, vz).

Given it's a dictionary as observation, **we will need to use a MultiInputPolicy policy instead of MlpPolicy**.

```
print("\n _____ACTION SPACE_____ \n")
print("The Action Space is: ", a_size)
print("Action Space Sample", env.action_space.sample()) # Take a random action
```

The action space is a vector with 3 values:

- Control x, y, z movement

Normalize observation and rewards

A good practice in reinforcement learning is to normalize input features.

For that purpose, there is a wrapper that will compute a running average and standard deviation of input features.

We also normalize rewards with this same wrapper by adding `norm_reward = True`

You should check the documentation to fill this cell

```
env = make_vec_env(env_id, n_envs=4)

# Adding this wrapper to normalize the observation and the reward
env = # TODO: Add the wrapper
```

Solution

```
env = make_vec_env(env_id, n_envs=4)

env = VecNormalize(env, norm_obs=True, norm_reward=True, clip_obs=10.)
```

Create the A2C Model

For more information about A2C implementation with StableBaselines3 check: https://stable-baselines3.readthedocs.io/en/master/modules/a2c.html#notes

To find the best parameters I checked the official trained agents by Stable-Baselines3 team.

```
model = # Create the A2C model and try to find the best parameters
```

Solution

```
model = A2C(policy = "MultiInputPolicy",
            env = env,
```

```
            verbose=1)
```

Train the A2C agent 🏃

- Let's train our agent for 1,000,000 timesteps, don't forget to use GPU on Colab. It will take approximately ~25-40min

```
model.learn(1_000_000)
```

```
# Save the model and  VecNormalize statistics when saving the agent
model.save("a2c-PandaReachDense-v3")
env.save("vec_normalize.pkl")
```

Evaluate the agent 📈

- Now that's our agent is trained, we need to **check its performance**.
- Stable-Baselines3 provides a method to do that: `evaluate_policy`

```
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

# Load the saved statistics
eval_env = DummyVecEnv([lambda: gym.make("PandaReachDense-v3")])
eval_env = VecNormalize.load("vec_normalize.pkl", eval_env)

# We need to override the render_mode
eval_env.render_mode = "rgb_array"

#  do not update them at test time
eval_env.training = False
# reward normalization is not needed at test time
eval_env.norm_reward = False

# Load the agent
model = A2C.load("a2c-PandaReachDense-v3")

mean_reward, std_reward = evaluate_policy(model,
```

```
print(f"Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}")
```

Publish your trained model on the Hub 🔥

Now that we saw we got good results after the training, we can publish our trained model on the Hub with one line of code.

📚 The libraries documentation 👉 https://github.com/huggingface/huggingface_sb3/tree/main#hugging-face—x-stable-baselines3-v20

By using `package_to_hub`, as we already mentionned in the former units, **you evaluate, record a replay, generate a model card of your agent and push it to the hub**.

This way:

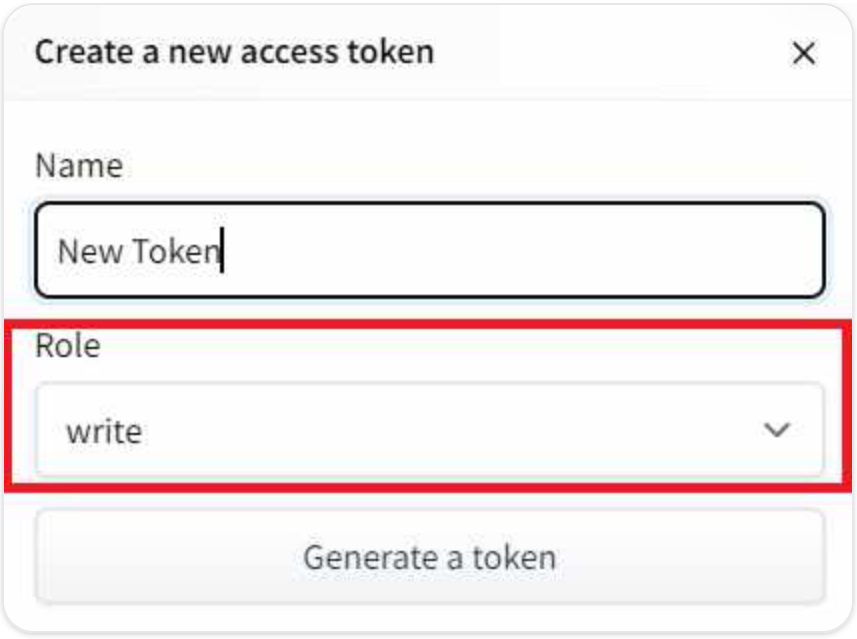
- You can **showcase our work** 🔥
- You can **visualize your agent playing** 👀
- You can **share with the community an agent that others can use** 💾
- You can **access a leaderboard** 🏆 **to see how well your agent is performing compared to your classmates** 👉 https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard

To be able to share your model with the community there are three more steps to follow:

1️⃣ (If it's not already done) create an account to HF ➡️ https://huggingface.co/join

2️⃣ Sign in and then, you need to store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**



- Copy the token
- Run the cell below and paste the token

```
notebook_login()
!git config --global credential.helper store
```

If you don't want to use a Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

3️⃣ We're now ready to push our trained agent to the 🤗 Hub 🔥 using `package_to_hub()` function. For this environment, **running this cell can take approximately 10min**

```
from huggingface_sb3 import package_to_hub

package_to_hub(
    model=model,
    model_name=f"a2c-{env_id}",
    model_architecture="A2C",
    env_id=env_id,
    eval_env=eval_env,
    repo_id=f"ThomasSimonini/a2c-{env_id}", # Change the username
    commit_message="Initial commit",
)
```

Some additional challenges 🏆

The best way to learn **is to try things by your own**! Why not trying `PandaPickAndPlace-v3`?

If you want to try more advanced tasks for panda-gym, you need to check what was done using **TQC or SAC** (a more sample-efficient algorithm suited for robotics tasks). In real robotics, you'll use a more sample-efficient algorithm for a simple reason: contrary to a simulation **if you move your robotic arm too much, you have a risk of breaking it**.

PandaPickAndPlace-v1 (this model uses the v1 version of the environment): https://huggingface.co/sb3/tqc-PandaPickAndPlace-v1

And don't hesitate to check panda-gym documentation here: https://panda-gym.readthedocs.io/en/latest/usage/train_with_sb3.html

We provide you the steps to train another agent (optional):

1. Define the environment called "PandaPickAndPlace-v3"
2. Make a vectorized environment
3. Add a wrapper to normalize the observations and rewards. Check the documentation
4. Create the A2C Model (don't forget verbose=1 to print the training logs).
5. Train it for 1M Timesteps
6. Save the model and VecNormalize statistics when saving the agent
7. Evaluate your agent
8. Publish your trained model on the Hub 🔥 with `package_to_hub`

Solution (optional)

```
# 1 - 2
env_id = "PandaPickAndPlace-v3"
env = make_vec_env(env_id, n_envs=4)

# 3
env = VecNormalize(env, norm_obs=True, norm_rewa
```

```
# 4
model = A2C(policy = "MultiInputPolicy",
            env = env,
            verbose=1)
# 5
model.learn(1_000_000)
```

```
# 6
model_name = "a2c-PandaPickAndPlace-v3";
model.save(model_name)
env.save("vec_normalize.pkl")

# 7
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

# Load the saved statistics
eval_env = DummyVecEnv([lambda: gym.make("PandaPickAndPlace-v3")])
eval_env = VecNormalize.load("vec_normalize.pkl", eval_env)

#  do not update them at test time
eval_env.training = False
# reward normalization is not needed at test time
eval_env.norm_reward = False

# Load the agent
model = A2C.load(model_name)

mean_reward, std_reward = evaluate_policy(model, eval_env)

print(f"Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}")

# 8
package_to_hub(
    model=model,
    model_name=f"a2c-{env_id}",
    model_architecture="A2C",
    env_id=env_id,
    eval_env=eval_env,
    repo_id=f"ThomasSimonini/a2c-{env_id}", # TODO: Change the username
```

```
    commit_message="Initial commit",
)
```

See you on Unit 7! 🔥

Keep learning, stay awesome 🤗

<> Update on GitHub

← Advantage Actor Critic (A2C) Quiz →

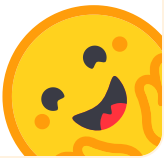
Search models, datasets, users...

Deep RL Course documentation
Advantage Actor-Critic (A2C)

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Advantage Actor-Critic (A2C)

Reducing variance with Actor-Critic methods

The solution to reducing the variance of the Reinforce algorithm and training our agent faster and better is to use a combination of Policy-Based and Value-Based methods: *the Actor-Critic method*.

To understand the Actor-Critic, imagine you're playing a video game. You can play with a friend that will provide you with some feedback. You're the Actor and your friend is the Critic.

Ask HuggingChat



You don't know how to play at the beginning, **so you try some actions randomly**. The Critic observes your action and **provides feedback**.

Learning from this feedback, **you'll update your policy and be better at playing that game.**

On the other hand, your friend (Critic) will also update their way to provide feedback so it can be better next time.

This is the idea behind Actor-Critic. We learn two function approximations:

- *A policy* that **controls how our agent acts**: $\pi_\theta(s)$
- *A value function* to assist the policy update by measuring how good the action taken is: $\hat{q}_w(s, a)$

The Actor-Critic Process

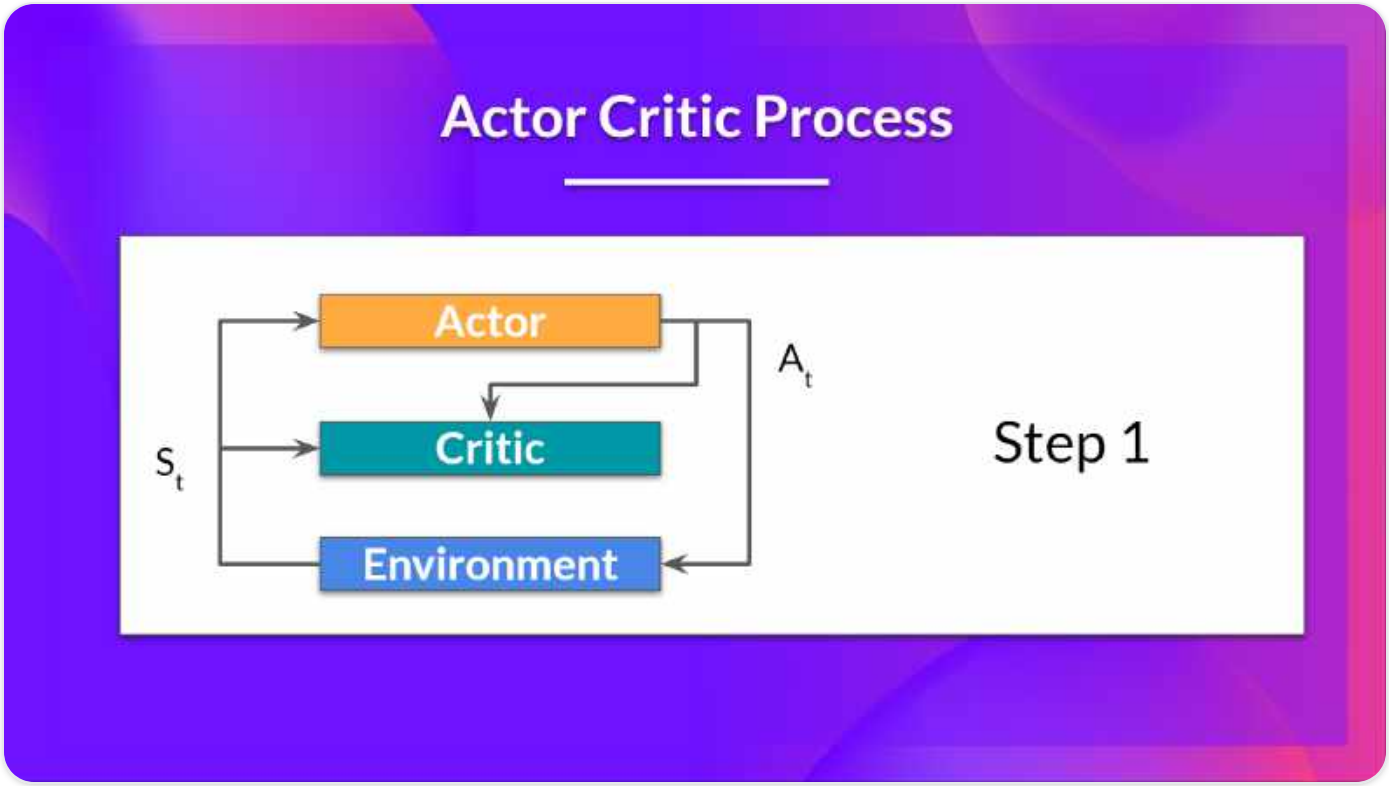
Now that we have seen the Actor Critic's big picture, let's dive deeper to understand how the Actor and Critic improve together during the training.

As we saw, with Actor-Critic methods, there are two function approximations (two neural networks):

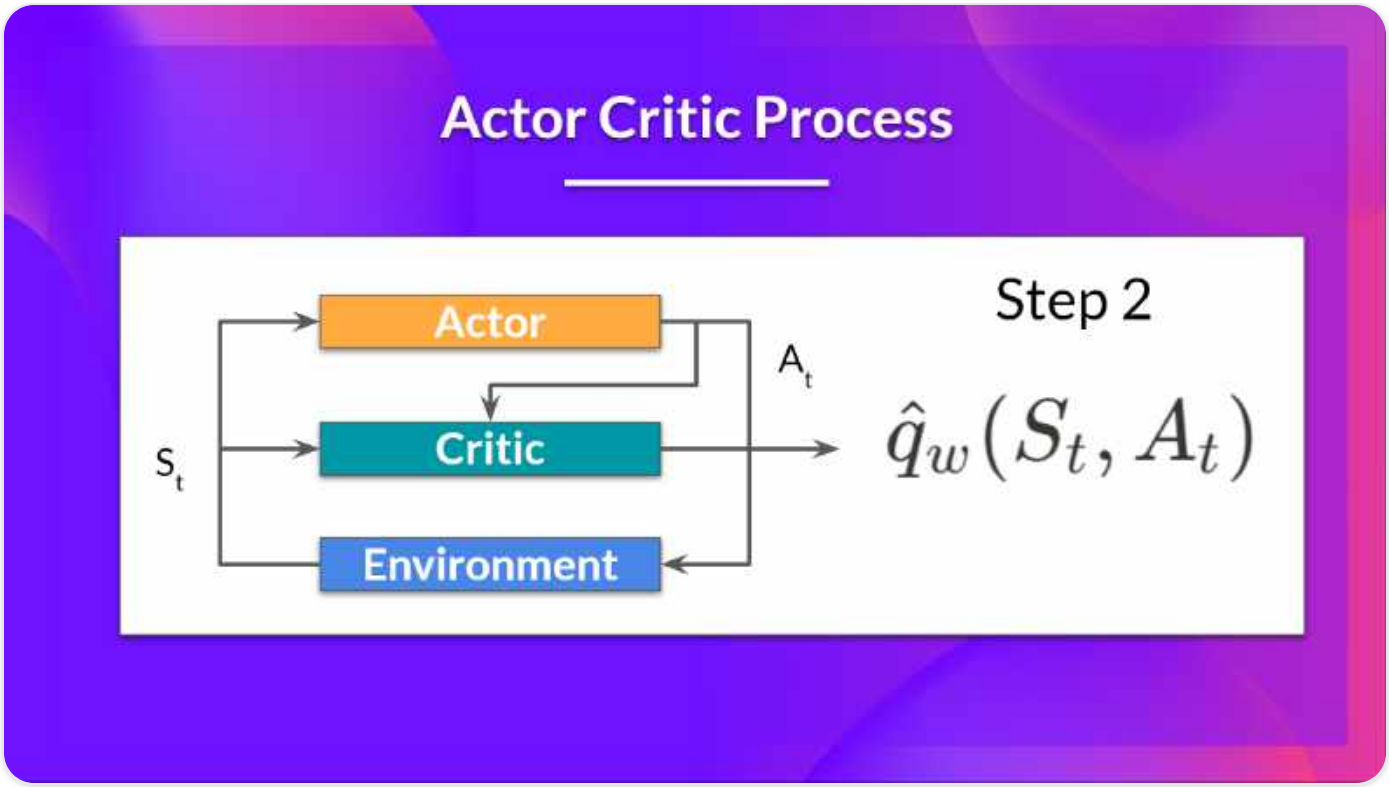
- *Actor*, a **policy function** parameterized by theta: $\pi_\theta(s)$
- *Critic*, a **value function** parameterized by w: $\hat{q}_w(s, a)$

Let's see the training process to understand how the Actor and Critic are optimized:

- At each timestep, t, we get the current state S_t from the environment and **pass it as input through our Actor and Critic**.
- Our Policy takes the state and **outputs an action** A_t.



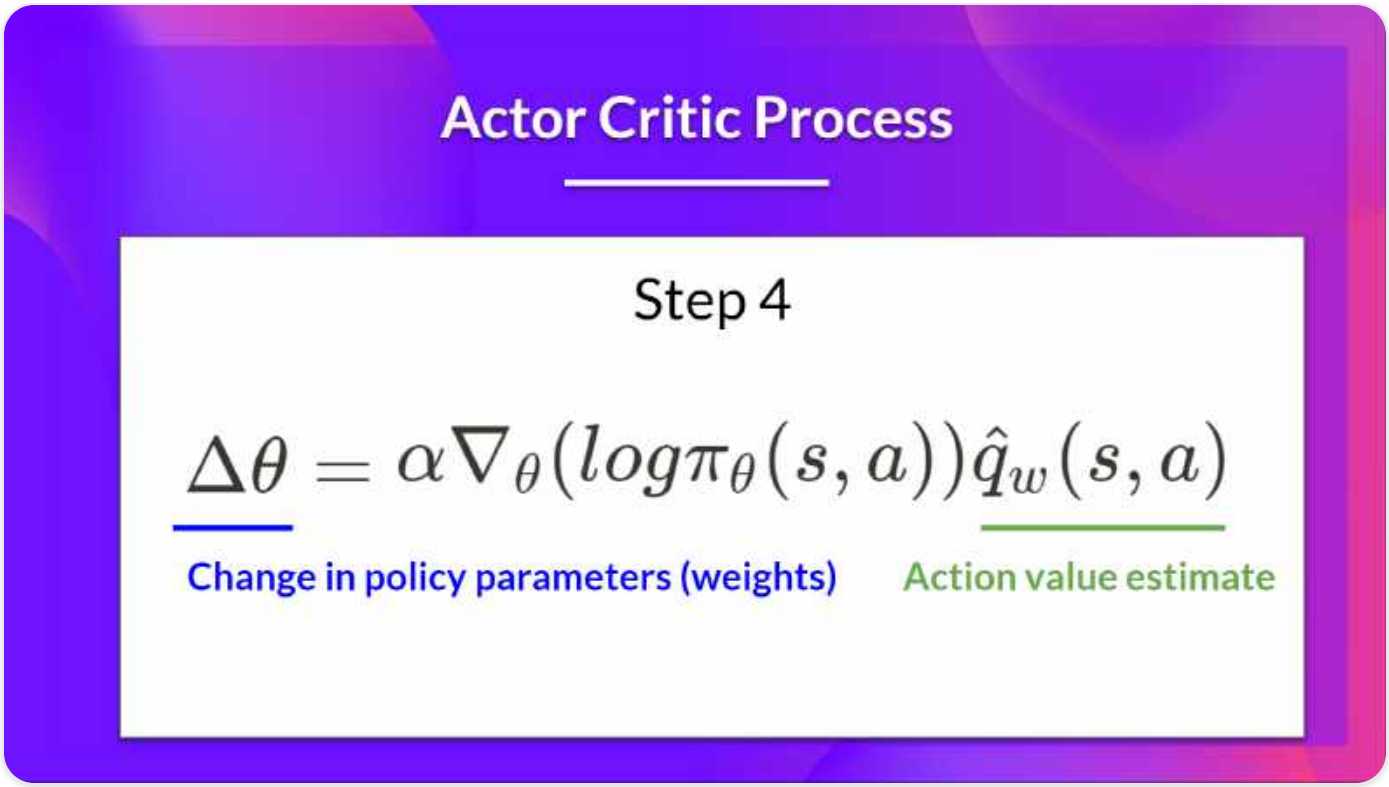
- The Critic takes that action also as input and, using S_t and A_t, **computes the value of taking that action at that state: the Q-value**.



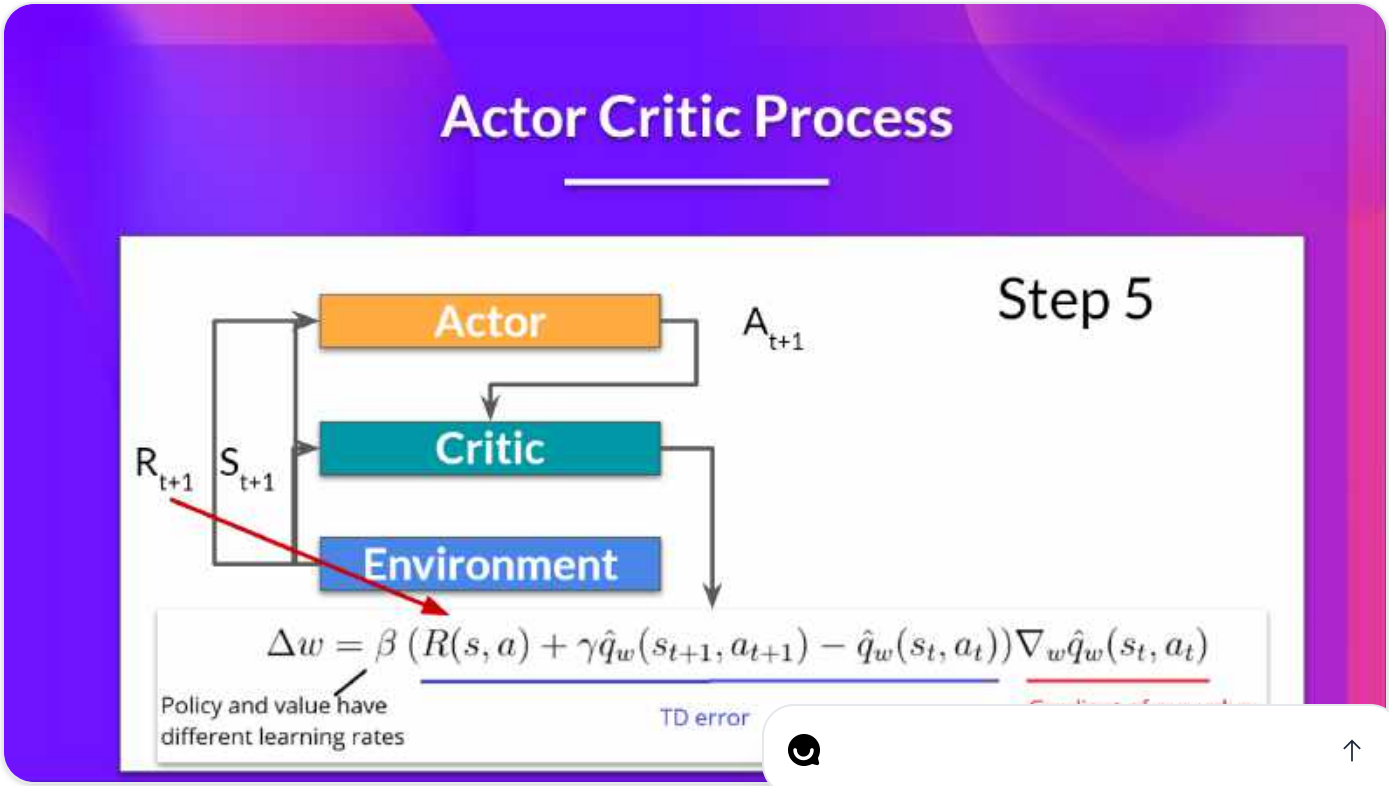
- The action A_t performed in the environment outputs a new state S_{t+1} and a reward R_{t+1} .



- The Actor updates its policy parameters using the (



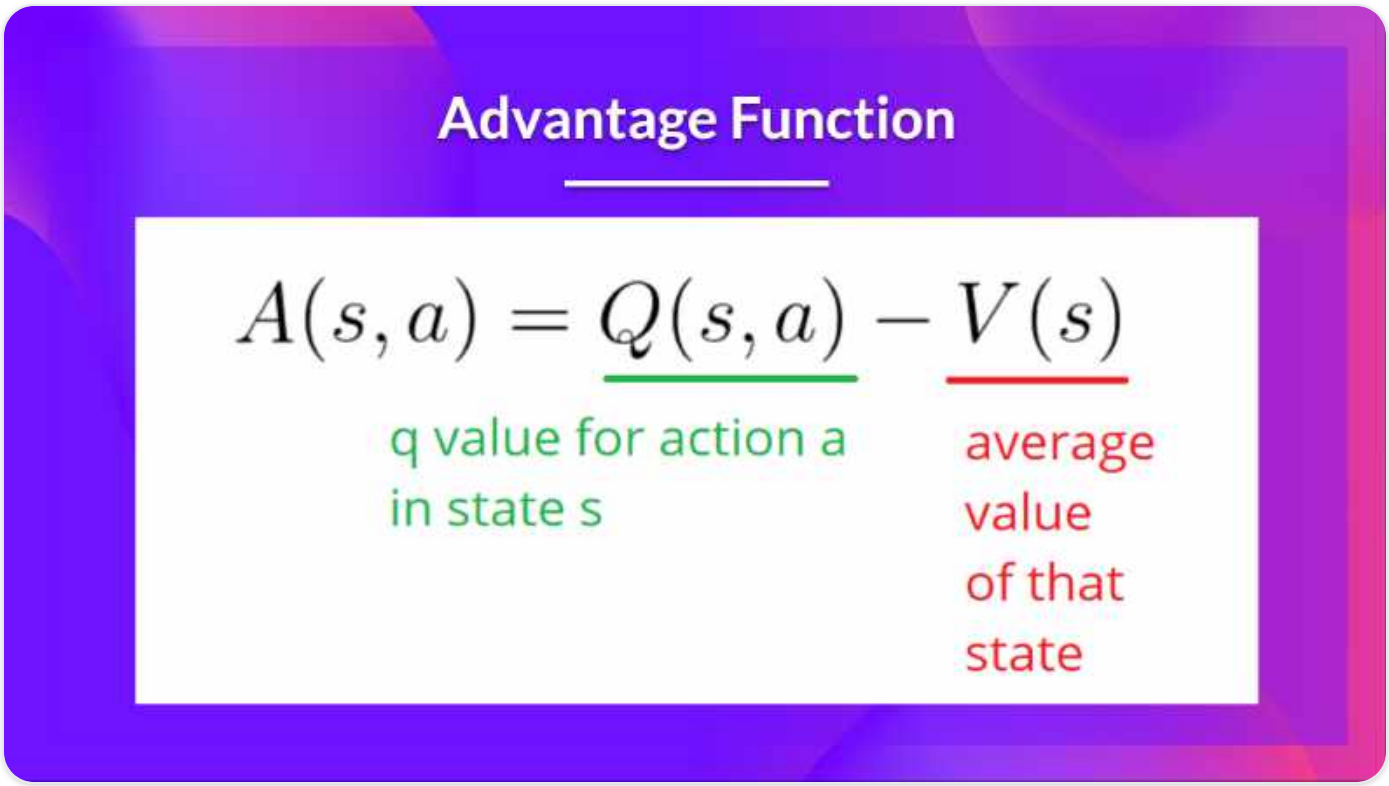
- Thanks to its updated parameters, the Actor produces the next action to take at A_{t+1} given the new state S_{t+1}.
- The Critic then updates its value parameters.



Adding Advantage in Actor-Critic (A2C)

We can stabilize learning further by **using the Advantage function as Critic instead of the Action value function**.

The idea is that the Advantage function calculates the relative advantage of an action compared to the others possible at a state: **how taking that action at a state is better compared to the average value of the state**. It's subtracting the mean value of the state from the state action pair:

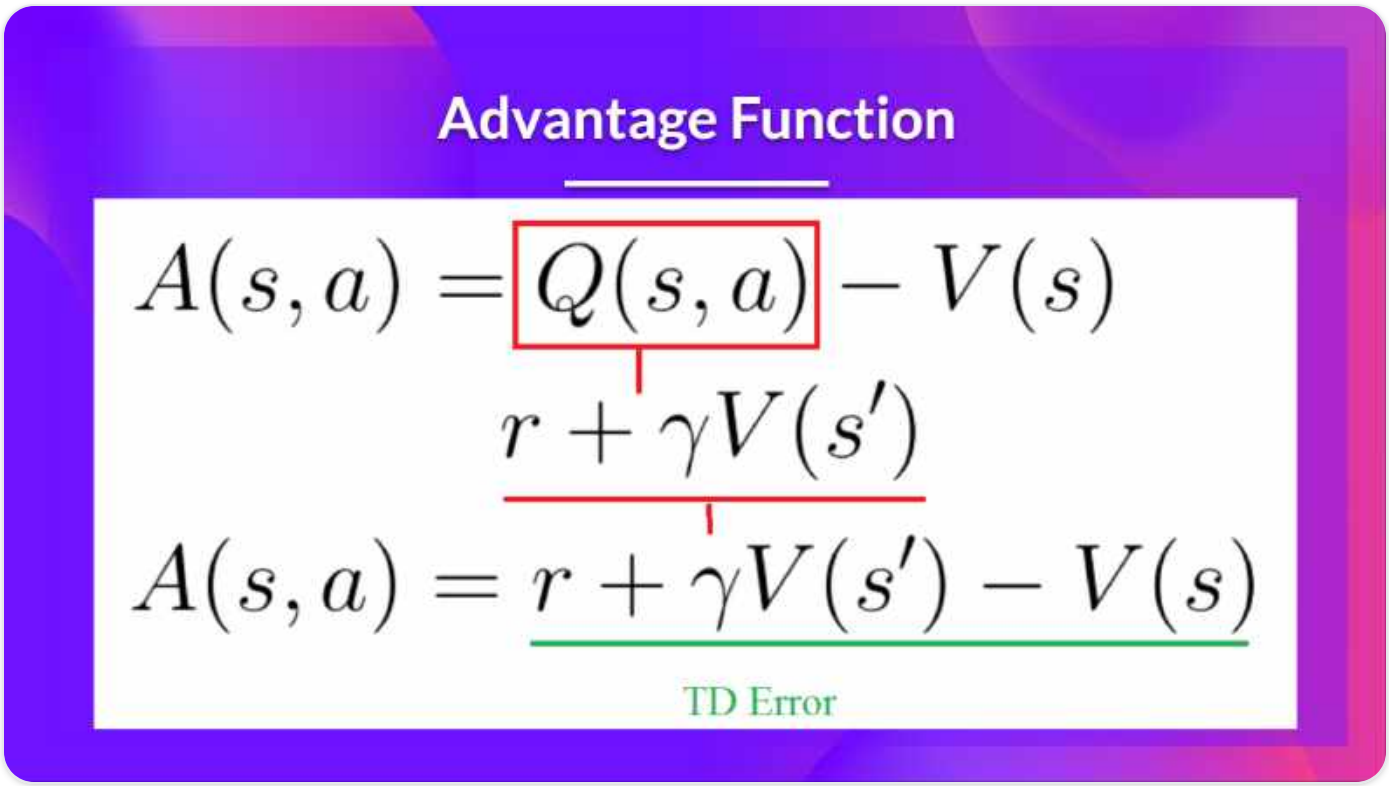


In other words, this function calculates **the extra reward we get if we take this action at that state compared to the mean reward we get at that state**.

The extra reward is what's beyond the expected value of that state.

- If A(s,a) > 0: our gradient is **pushed in that direction**.
- If A(s,a) < 0 (our action does worse than the average value of that state), **our gradient is pushed in the opposite direction**.

The problem with implementing this advantage function is that it requires two value functions — $Q(s, a)$ and $V(s)$. Fortunately, **we can use the TD error as a good estimator of the advantage function.**



<> Update on GitHub

← The Problem of Variance in Reinforce

Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym →

Search models, datasets, users...

Deep RL Course documentation
An introduction to Multi-Agents Reinforcement Learning (MARL)

Join the Hugging Face community

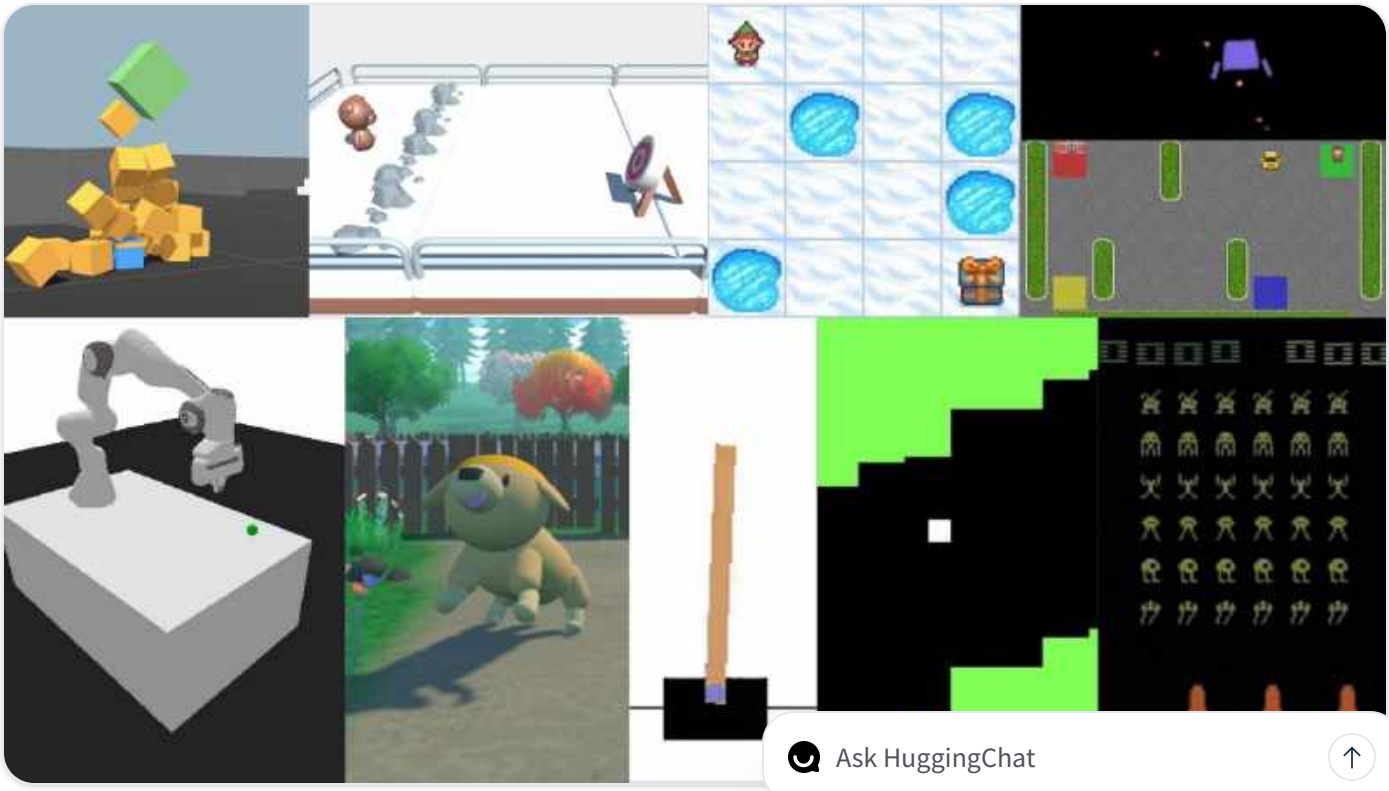
and get access to the augmented documentation experience

Sign Up to get started

An introduction to Multi-Agents Reinforcement Learning (MARL)

From single agent to multiple agents

In the first unit, we learned to train agents in a single-agent system. When our agent was alone in its environment: **it was not cooperating or collaborating with other agents**.



Ask HuggingChat

A patchwork of all the environments you've trained your agents on since the beginning of the course

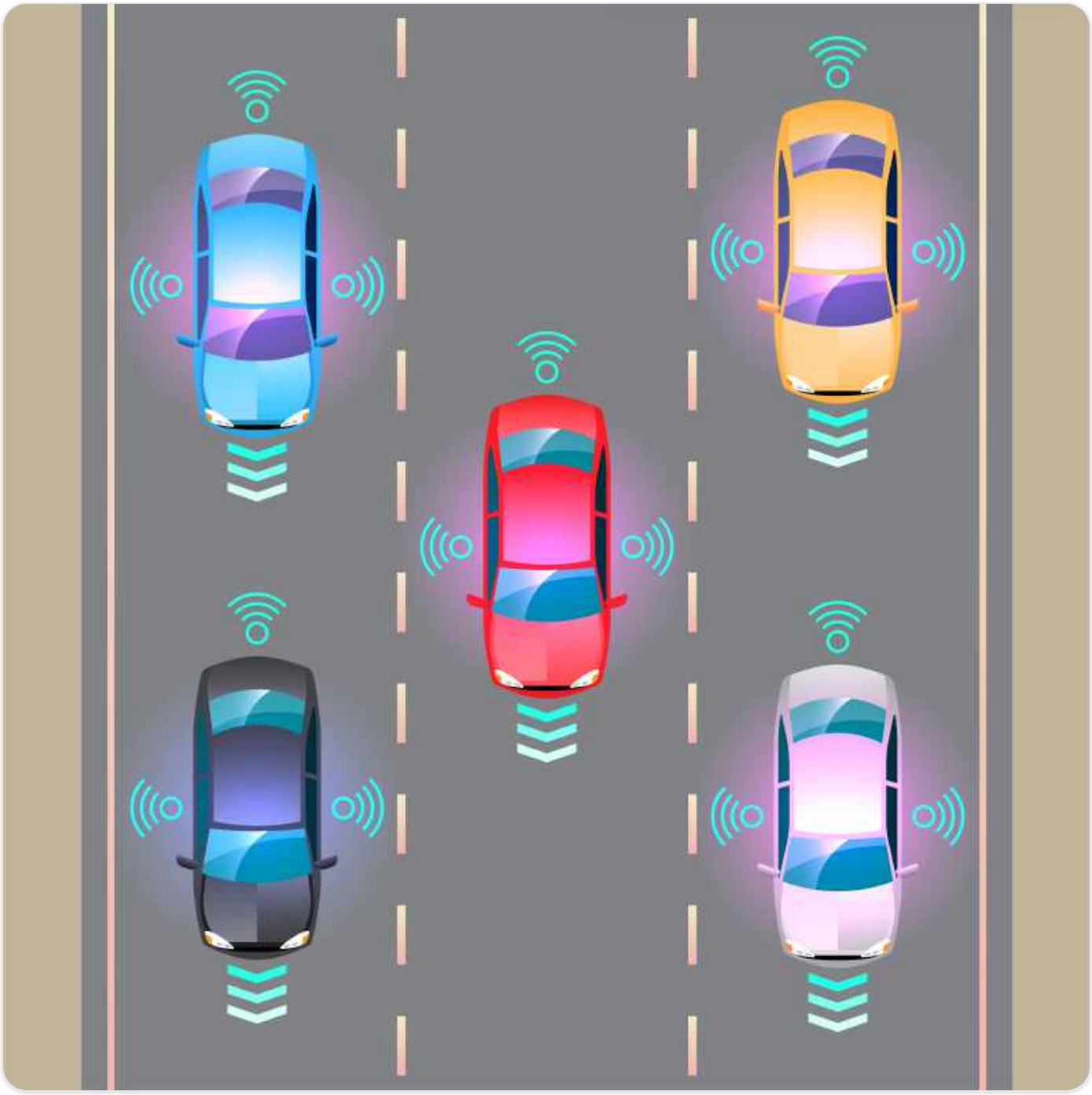
When we do multi-agents reinforcement learning (MARL), we are in a situation where we have multiple agents **that share and interact in a common environment**.

For instance, you can think of a warehouse where **multiple robots need to navigate to load and unload packages**.



[Image by upklyak](https://www.freepik.com/free-vector/robots-warehouse-interior-automated-machines_32117680.htm#query=warehouse robot&position=17&from_view=keyword) on Freepik

Or a road with **several autonomous vehicles**.



[Image by jcomp](https://www.freepik.com/free-vector/autonomous-smart-car-automatic-wireless-sensor-driving-road-around-car-autonomous-smart-car-goes-scans-roads-observe-distance-automatic-braking-system_26413332.htm#query=self driving cars highway&position=34&from_view=search&track=ais) on Freepik

In these examples, we have **multiple agents interacting in the environment and with the other agents**. This implies defining a multi-agents system. But first, let's understand the different types of multi-agent environments.

Different types of multi-agent environments

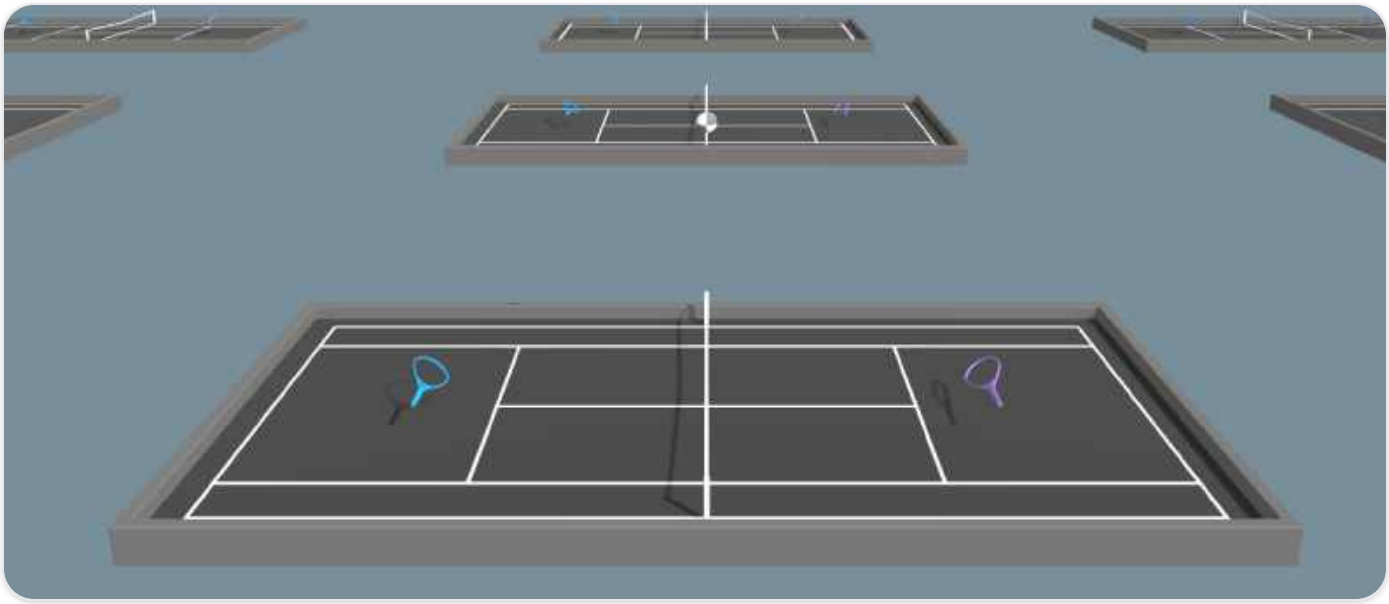
Given that, in a multi-agent system, agents interact with other agents, we can have different types of environments:

- *Cooperative environments*: where your agents need **to maximize the common benefits**.

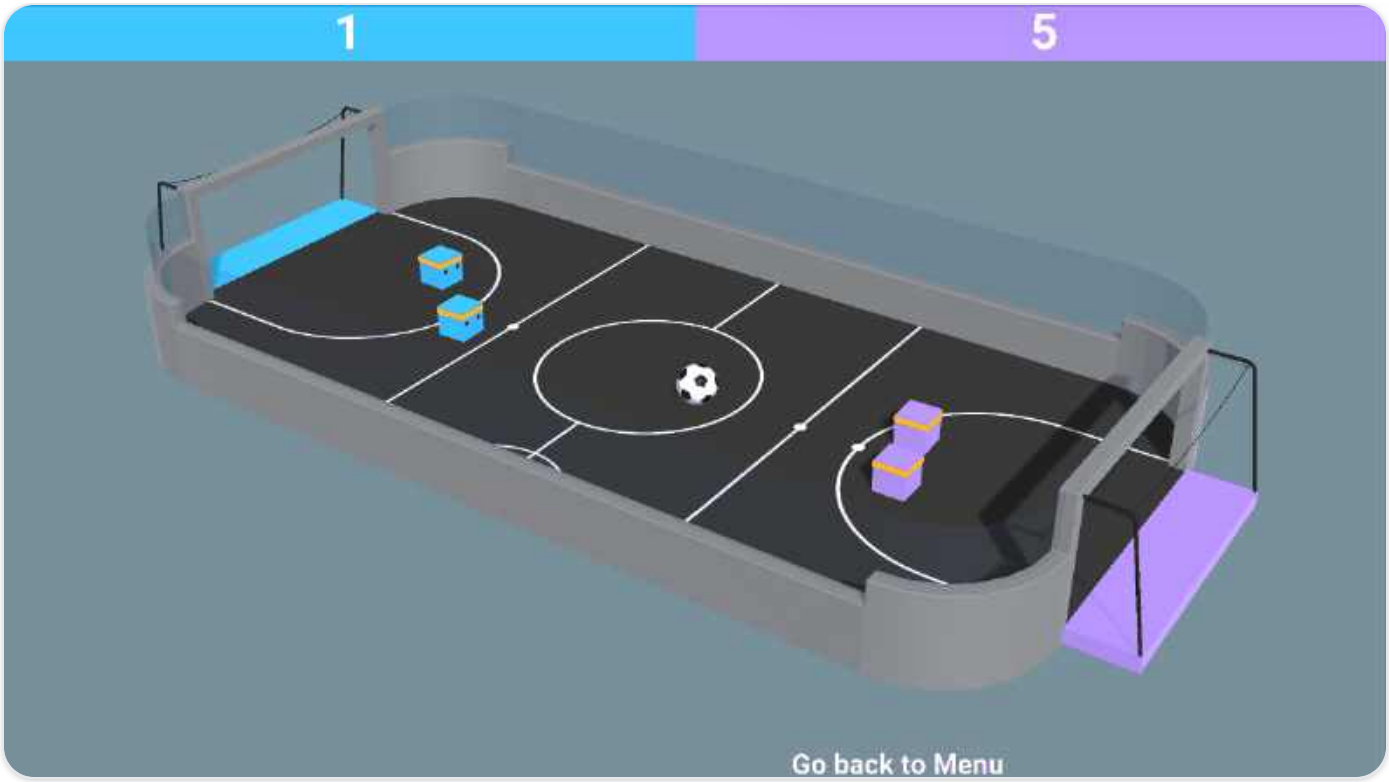
For instance, in a warehouse, **robots must collaborate to load and unload the packages efficiently (as fast as possible)**.

- *Competitive/Adversarial environments*: in this case, your agent **wants to maximize its benefits by minimizing the opponent's**.

For example, in a game of tennis, **each agent wants to beat the other agent**.



- *Mixed of both adversarial and cooperative*: like in our SoccerTwos environment, two agents are part of a team (blue or purple): they need to cooperate with each other and beat the opponent team.



This environment was made by the Unity MLAgents Team

So now we might wonder: how can we design these multi-agent systems? Said differently, **how can we train agents in a multi-agent setting** ?

<> Update on GitHub

← Introduction

Designing Multi-Agents systems →

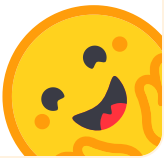
Search models, datasets, users...

Deep RL Course documentation
An Introduction to Unreal Learning Agents

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



An Introduction to Unreal Learning Agents

Learning Agents is an Unreal Engine (UE) plugin that allows you **to train AI characters using machine learning (ML) in Unreal**.

It's an exciting new plugin where you can create unique environments using Unreal Engine and train your agents.

Let's see how you can **get started and train a car to drive in an Unreal Engine Environment**.

Ask HuggingChat



Source: [Learning Agents Driving Car Tutorial]
(https://dev.epicgames.com/community/learning/tutorials/qj2O/unreal-engine-learning-to-drive)

Case 1: I don’t know anything about Unreal Engine and Beginners in Unreal Engine

If you’re new to Unreal Engine, don’t be scared! We listed two courses you need to follow to be able to use Learning Agents:

1. Master the Basics: Begin by watching this course your first hour in Unreal Engine 5. This comprehensive course will **lay down the foundational knowledge you need to use Unreal**.

2. Dive into Blueprints: Explore the world of Blueprints, the visual scripting component of Unreal Engine. This video course will familiarize you with this essential tool.

Armed with the basics, **you’re now prepared to play with Learning Agents**:

3. Get the Big Picture of Learning Agents by reading this informative overview.

4. Teach a Car to Drive using Reinforcement Learning in Learning Agents.

5. Check Imitation Learning with the Unreal Engine 5.

Case 2: I’m familiar with Unreal

For those already acquainted with Unreal Engine, you can jump straight into Learning Agents with these two tutorials:

1. Get the Big Picture of Learning Agents by reading this informative overview.
2. Teach a Car to Drive using Reinforcement Learning in Learning Agents. .
3. Check Imitation Learning with the Unreal Engine 5.3 Learning Agents Plugin

<> Update on GitHub

← Interesting environments to try

An Introduction to Godot RL →

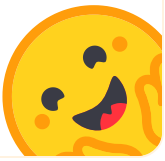
Search models, datasets, users...

Deep RL Course documentation
Bonus: Learn to create your own environments with Unity and MLAgents

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Bonus: Learn to create your own environments with Unity and MLAgents

You can create your own reinforcement learning environments with Unity and MLAgents. Using a game engine such as Unity can be intimidating at first, but here are the steps you can take to learn smoothly.

Step 1: Know how to use Unity

- The best way to learn Unity is to do "Create with Code" course: it's a series of videos for beginners where **you will create 5 small games with Unity**.

Step 2: Create the simplest environment with this tutorial

- Then, when you know how to use Unity, you can create your first basic RL environment using this tutorial.

Step 3: Iterate and create nice environments

- Now that you've created your first simple environ
using the MLAgents documentation (especially Designing Agents and Agent part)

Ask HuggingChat

- In addition, you can take this free course “Create a hummingbird environment” by Adam Kelly

Have fun! And if you create custom environments don’t hesitate to share them to the `#rl-i-made-this` discord channel.

<> Update on GitHub

← Hands-on Quiz →

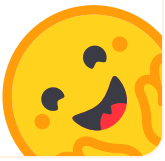
Search models, datasets, users...

Deep RL Course documentation
Brief introduction to RL documentation

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Brief introduction to RL documentation

In this advanced topic, we address the question: **how should we monitor and keep track of powerful reinforcement learning agents that we are training in the real world and interfacing with humans?**

As machine learning systems have increasingly impacted modern life, the **call for the documentation of these systems has grown**.

Such documentation can cover aspects such as the training data used — where it is stored, when it was collected, who was involved, etc. — or the model optimization framework — the architecture, evaluation metrics, relevant papers, etc. — and more.

Today, model cards and datasheets are becoming increasingly available. For example, on the Hub (see documentation here).

If you click on a popular model on the Hub, you can learn about its creation process.

These model and data specific logs are designed to be completed when the model or dataset are created, leaving them to go un-updated when these models are built into evolving systems in the future.

Ask HuggingChat

Motivating Reward Reports

Reinforcement learning systems are fundamentally designed to optimize based on measurements of reward and time. While the notion of a reward function can be mapped nicely to many well-understood fields of supervised learning (via a loss function), understanding of how machine learning systems evolve over time is limited.

To that end, the authors introduce *Reward Reports for Reinforcement Learning* (the pithy naming is designed to mirror the popular papers *Model Cards for Model Reporting* and *Datasheets for Datasets*). The goal is to propose a type of documentation focused on the **human factors of reward** and **time-varying feedback systems**.

Building on the documentation frameworks for model cards and datasheets proposed by Mitchell et al. and Gebru et al., we argue the need for Reward Reports for AI systems.

Reward Reports are living documents for proposed RL deployments that demarcate design choices.

However, many questions remain about the applicability of this framework to different RL applications, roadblocks to system interpretability, and the resonances between deployed supervised machine learning systems and the sequential decision-making utilized in RL.

At a minimum, Reward Reports are an opportunity for RL practitioners to deliberate on these questions and begin the work of deciding how to resolve them in practice.

Capturing temporal behavior with documentation

The core piece specific to documentation designed for RL and feedback-driven ML systems is a *change-log*. The change-log updates information from the designer (changed training parameters, data, etc.) along with noticed changes from the user (harmful behavior, unexpected responses, etc.).

The change log is accompanied by update triggers that encourage monitoring these effects.

Contributing

Some of the most impactful RL-driven systems are multi-stakeholder in nature and behind the closed doors of private corporations. These corporations are largely without regulation, so the burden of documentation falls on the public.

If you are interested in contributing, we are building Reward Reports for popular machine learning systems on a public record on GitHub.For further reading, you can visit the Reward Reports paper or look an example report.

Author

This section was written by Nathan Lambert

<> Update on GitHub

← Students projects

Introduction →

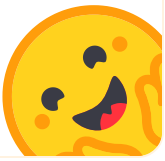
Search models, datasets, users...

Deep RL Course documentation
Conclusion

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Conclusion

Congrats on finishing this unit! **That was the biggest one**, and there was a lot of information. And congrats on finishing the tutorial. You've just trained your first Deep RL agents and shared them with the community! 🥳

It's **normal if you still feel confused by some of these elements**. This was the same for me and for all people who studied RL.

Take time to really grasp the material before continuing. It's important to master these elements and have a solid foundation before entering the fun part.

Naturally, during the course, we're going to use and explain these terms again, but it's better to understand them before diving into the next units.

In the next (bonus) unit, we're going to reinforce what we just learned by **training Huggy the Dog to fetch a stick**.

You will then be able to play with him 🤗.

Ask HuggingChat



Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then, please 👉 fill this form

Keep Learning, stay awesome 🤗

<> Update on GitHub

← Quiz

Additional Readings →

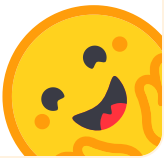
Search models, datasets, users...

Deep RL Course documentation
Decision Transformers

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Decision Transformers

The Decision Transformer model was introduced by "Decision Transformer: Reinforcement Learning via Sequence Modeling" by Chen L. et al. It abstracts Reinforcement Learning as a conditional-sequence modeling problem.

The main idea is that instead of training a policy using RL methods, such as fitting a value function, that will tell us what action to take to maximize the return (cumulative reward), **we use a sequence modeling algorithm (Transformer) that, given a desired return, past states, and actions, will generate future actions to achieve this desired return**. It's an autoregressive model conditioned on the desired return, past states, and actions to generate future actions that achieve the desired return.

This is a complete shift in the Reinforcement Learning paradigm since we use generative trajectory modeling (modeling the joint distribution of the sequence of states, actions, and rewards) to replace conventional RL algorithms. This means that in Decision Transformers, we don't maximize the return but rather generate a series of future actions that achieve the desired return.

The 🤗 Transformers team integrated the Decision Transformer, an Offline Reinforcement Learning method, into the library as well as the Hugging Face Hub.

Ask HuggingChat

Learn about Decision Transformers

To learn more about Decision Transformers, you should read the blogpost we wrote about it Introducing Decision Transformers on Hugging Face

Train your first Decision Transformers

Now that you understand how Decision Transformers work thanks to Introducing Decision Transformers on Hugging Face, you're ready to learn to train your first Offline Decision Transformer model from scratch to make a half-cheetah run.

Start the tutorial here 👉 https://huggingface.co/blog/train-decision-transformers

Further reading

For more information, we recommend that you check out the following resources:

- Decision Transformer: Reinforcement Learning via Sequence Modeling
- Online Decision Transformer

Author

This section was written by Edward Beeching

<> Update on GitHub

← Reinforcement Learning from Human Feedback Language models in RL →

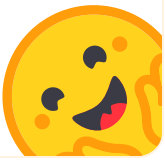
Search models, datasets, users...

Deep RL Course documentation
Designing Multi-Agents systems

Join the Hugging Face community

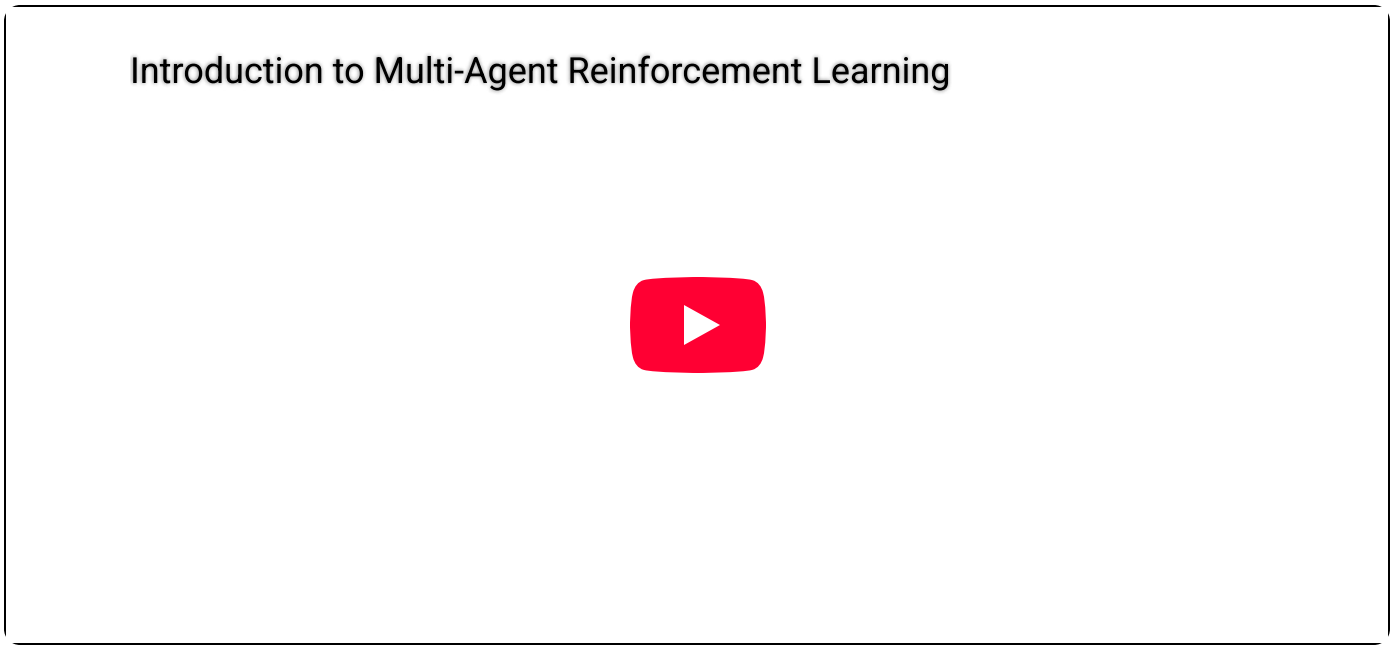
and get access to the augmented documentation experience

Sign Up to get started



Designing Multi-Agents systems

For this section, you're going to watch this excellent introduction to multi-agents made by Brian Douglas .

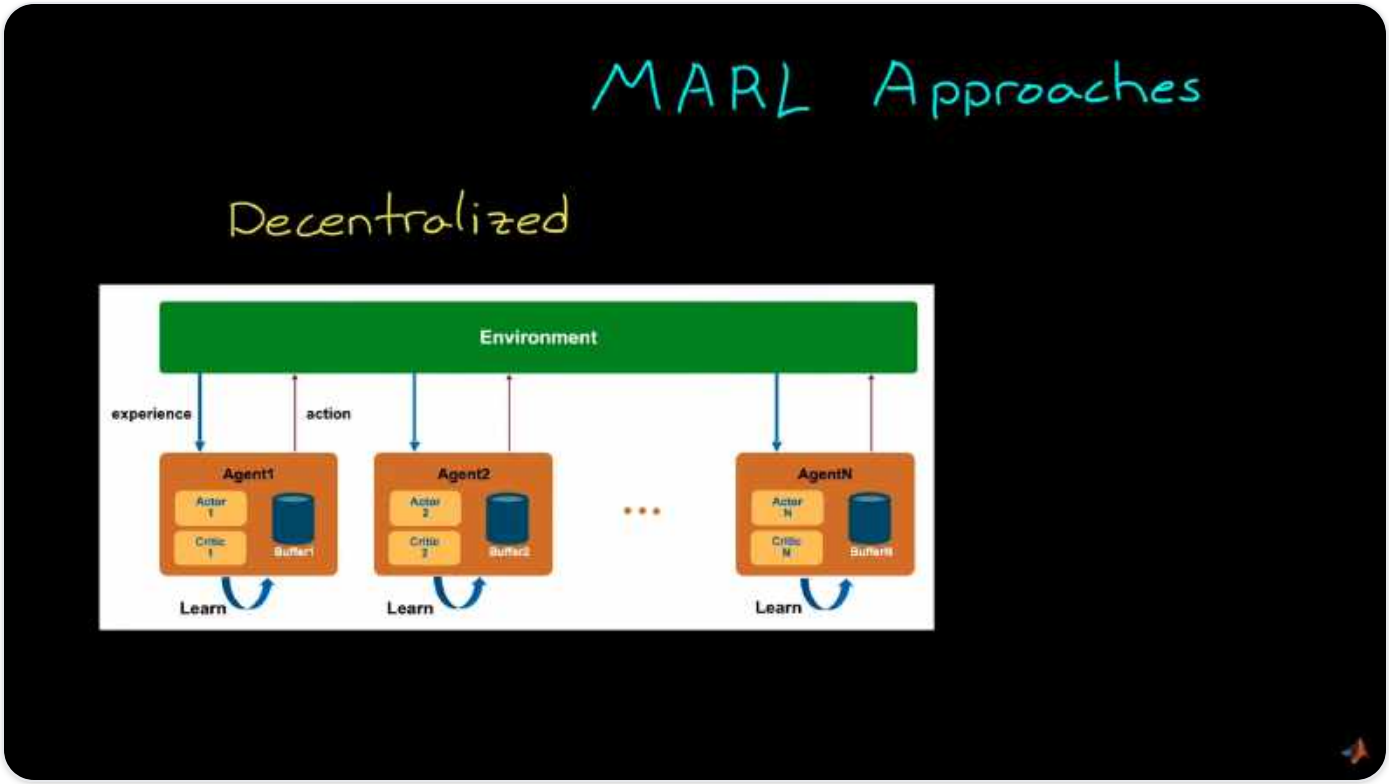


In this video, Brian talked about how to design multi-agent systems. He specifically took a multi-agents system of vacuum cleaners and asked: **how can can cooperate with each other**?

Ask HuggingChat

We have two solutions to design this multi-agent reinforcement learning system (MARL):

Decentralized system



Source: Introduction to Multi-Agent Reinforcement Learning

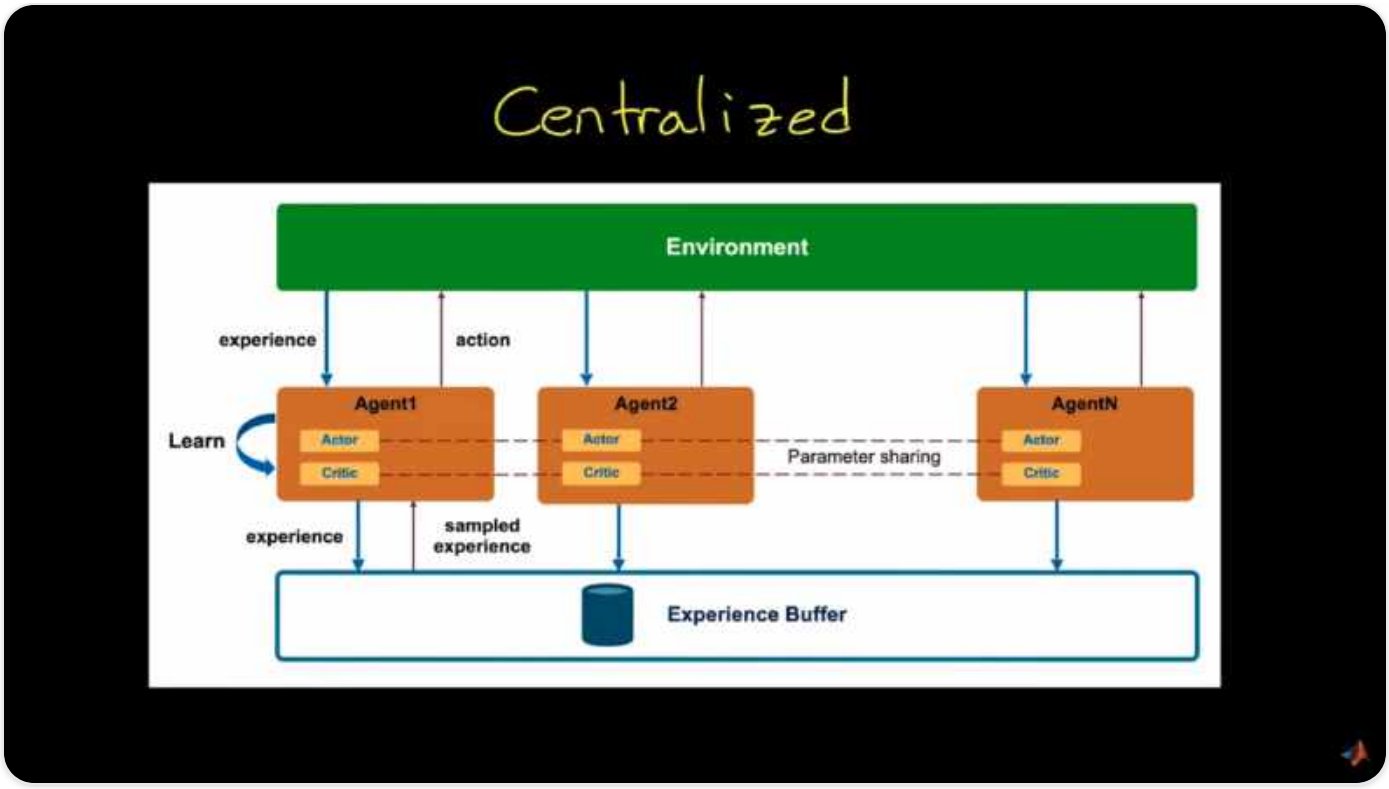
In decentralized learning, **each agent is trained independently from the others**. In the example given, each vacuum learns to clean as many places as it can **without caring about what other vacuums (agents) are doing**.

The benefit is that **since no information is shared between agents, these vacuums can be designed and trained like we train single agents**.

The idea here is that **our training agent will consider other agents as part of the environment dynamics**. Not as agents.

However, the big drawback of this technique is that it will **make the environment non-stationary** since the underlying Markov decision process changes over time as other agents are also interacting in the environment. And this is problematic for many Reinforcement Learning algorithms **that can't reach a global optimum with a non-stationary environment**.

Centralized approach



Source: Introduction to Multi-Agent Reinforcement Learning

In this architecture, **we have a high-level process that collects agents' experiences**: the experience buffer. And we'll use these experiences **to learn a common policy**.

For instance, in the vacuum cleaner example, the observation will be:

- The coverage map of the vacuums.
- The position of all the vacuums.

We use that collective experience **to train a policy that will move all three robots in the most beneficial way as a whole**. So each robot is learning from their common experience. We now have a stationary environment since all the agents are treated as a larger entity, and they know the change of other agents' policies (since it's the same as theirs).

If we recap:

- In a *decentralized approach*, we **treat all agents independently without considering the existence of the other agents.**
  - In this case, all agents **consider others agents**

    - **It's a non-stationarity environment condition**, so has no guarantee of convergence.

- In a *centralized approach*:
    - A **single policy is learned from all the agents**.
    - Takes as input the present state of an environment and the policy outputs joint actions.
    - The reward is global.

<> Update on GitHub

← An introduction to Multi-Agents Reinforcement Learning (MARL) Self-Play →

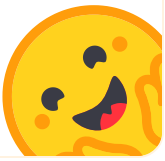
Search models, datasets, users...

Deep RL Course documentation
Diving deeper into policy-gradient methods

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Diving deeper into policy-gradient methods

Getting the big picture

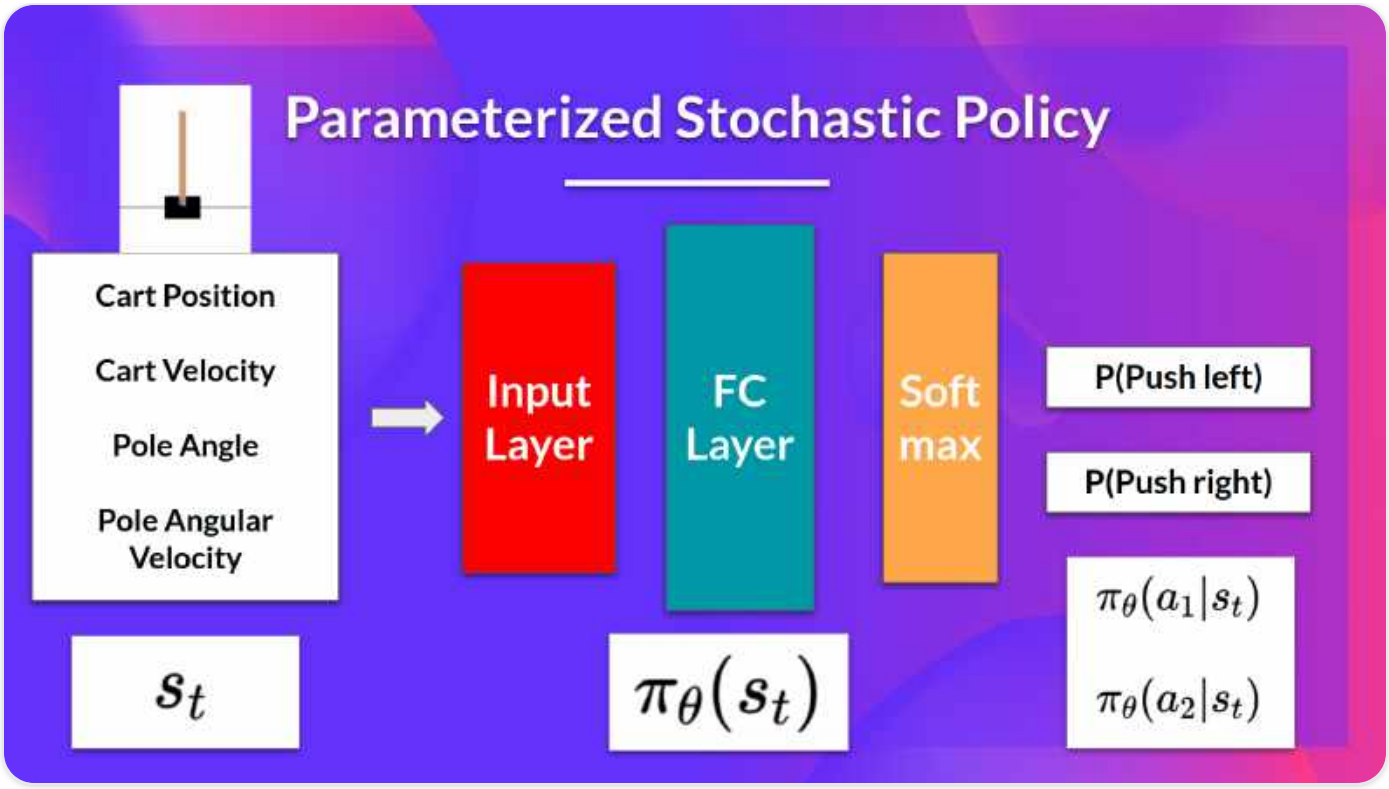
We just learned that policy-gradient methods aim to find parameters θ that **maximize the expected return**.

The idea is that we have a *parameterized stochastic policy*. In our case, a neural network outputs a probability distribution over actions. The probability of taking each action is also called the *action preference*.

If we take the example of CartPole-v1:

- As input, we have a state.
- As output, we have a probability distribution over actions at that state.

Ask HuggingChat



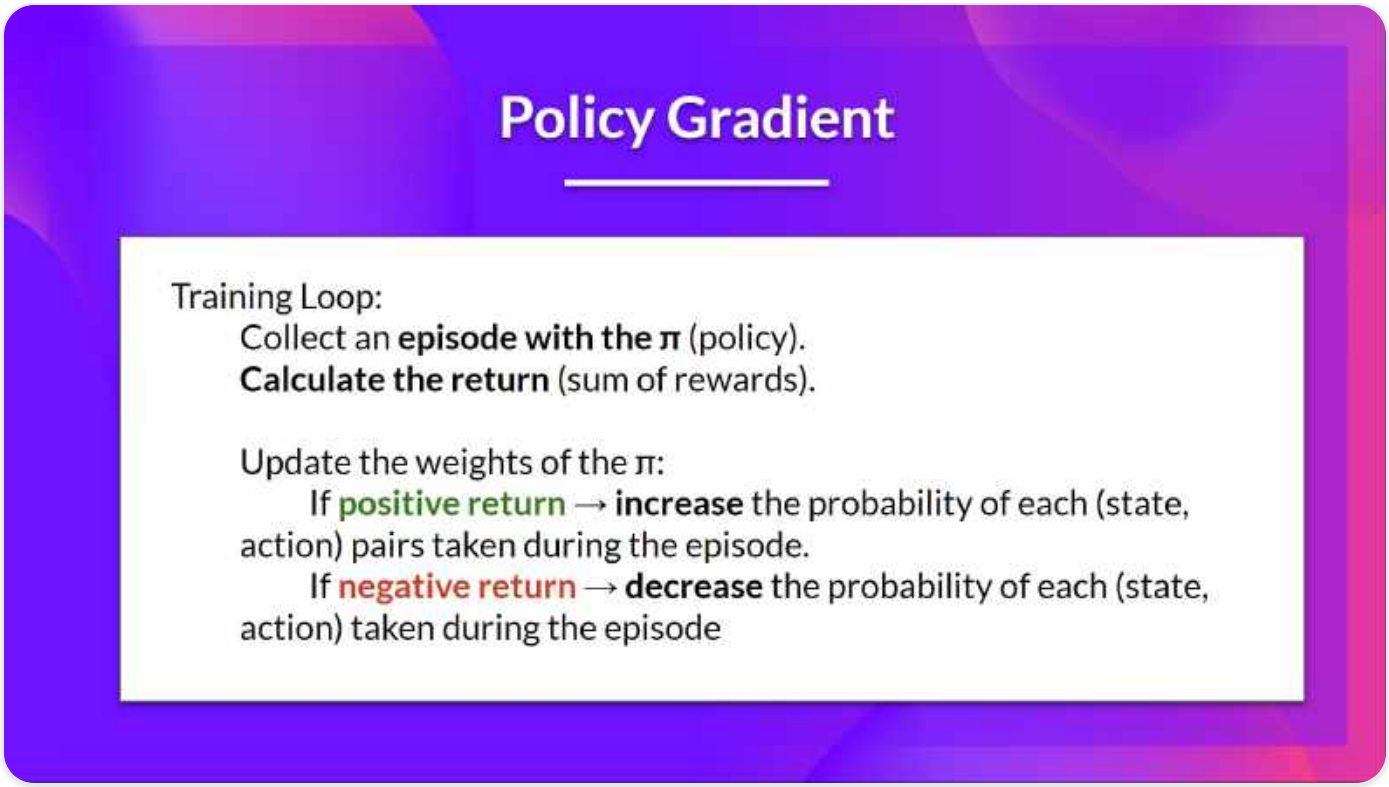
Our goal with policy-gradient is to **control the probability distribution of actions** by tuning the policy such that **good actions (that maximize the return) are sampled more frequently in the future.** Each time the agent interacts with the environment, we tweak the parameters such that good actions will be sampled more likely in the future.

But **how are we going to optimize the weights using the expected return**?

The idea is that we're going to **let the agent interact during an episode**. And if we win the episode, we consider that each action taken was good and must be more sampled in the future since they lead to win.

So for each state-action pair, we want to increase the $P(a|s)$: the probability of taking that action at that state. Or decrease if we lost.

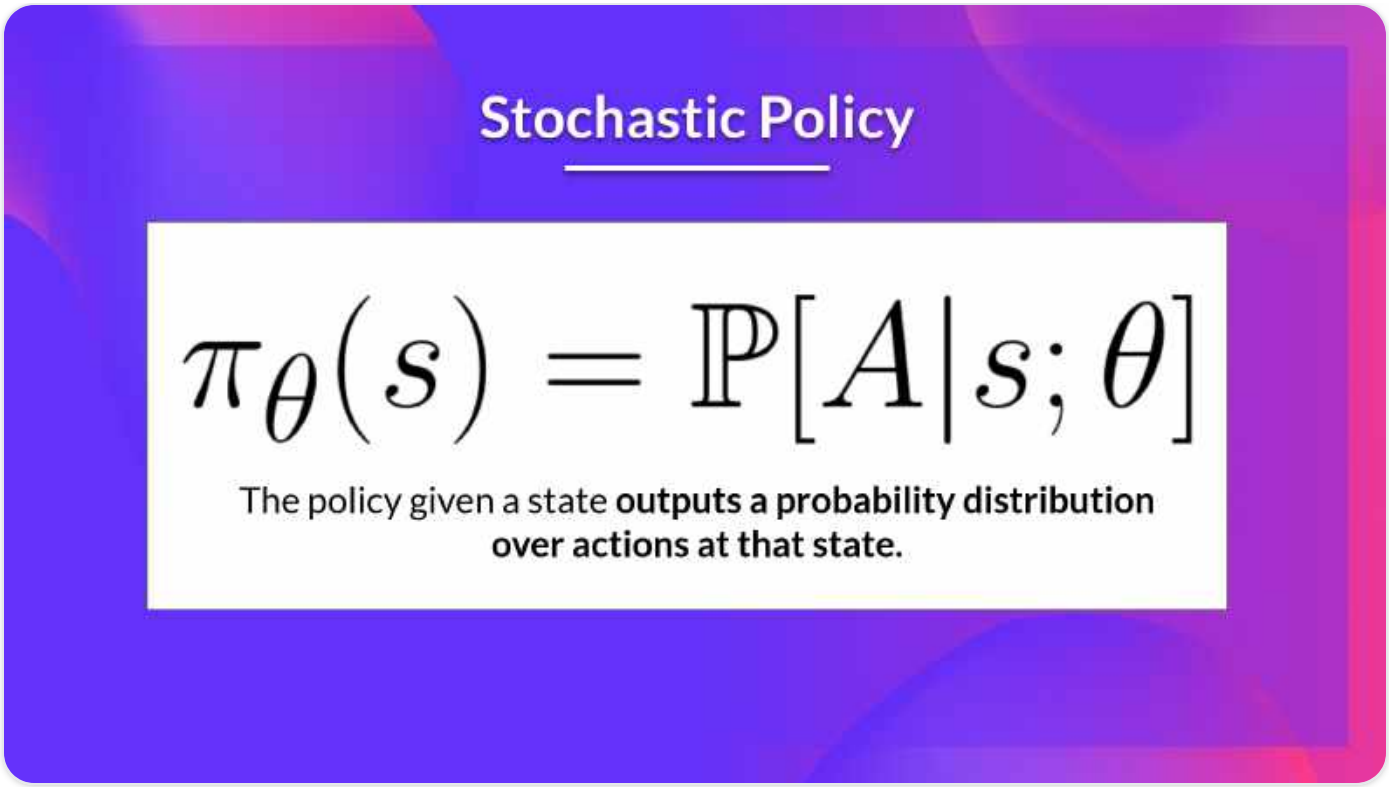
The Policy-gradient algorithm (simplified) looks like this:



Now that we got the big picture, let's dive deeper into policy-gradient methods.

Diving deeper into policy-gradient methods

We have our stochastic policy π which has a parameter θ. This π, given a state, **outputs a probability distribution of actions**.

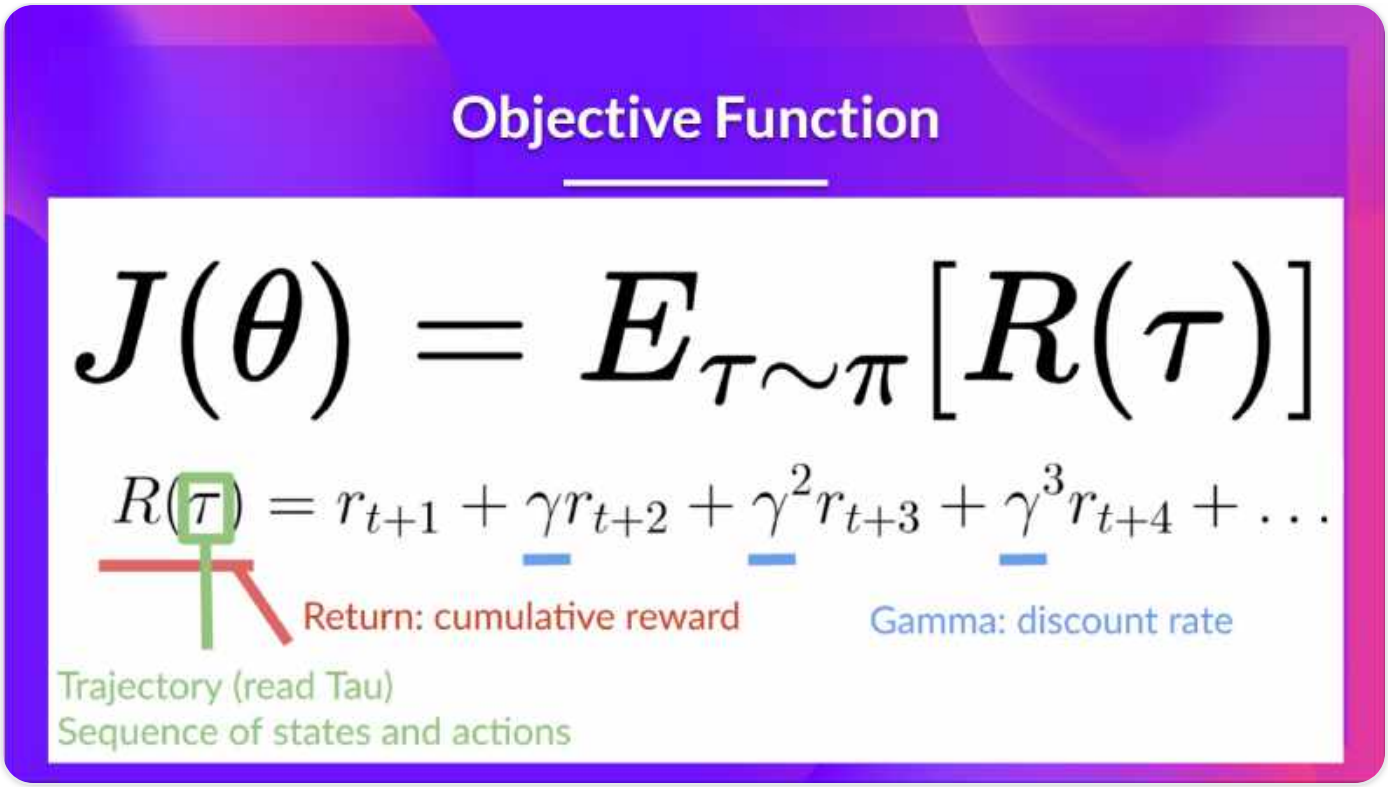


Where $\pi_\theta(a_t|s_t)$ is the probability of the agent selecting action a_t from state s_t given our policy.

But how do we know if our policy is good? We need to have a way to measure it. To know that, we define a score/objective function called $J(\theta)$.

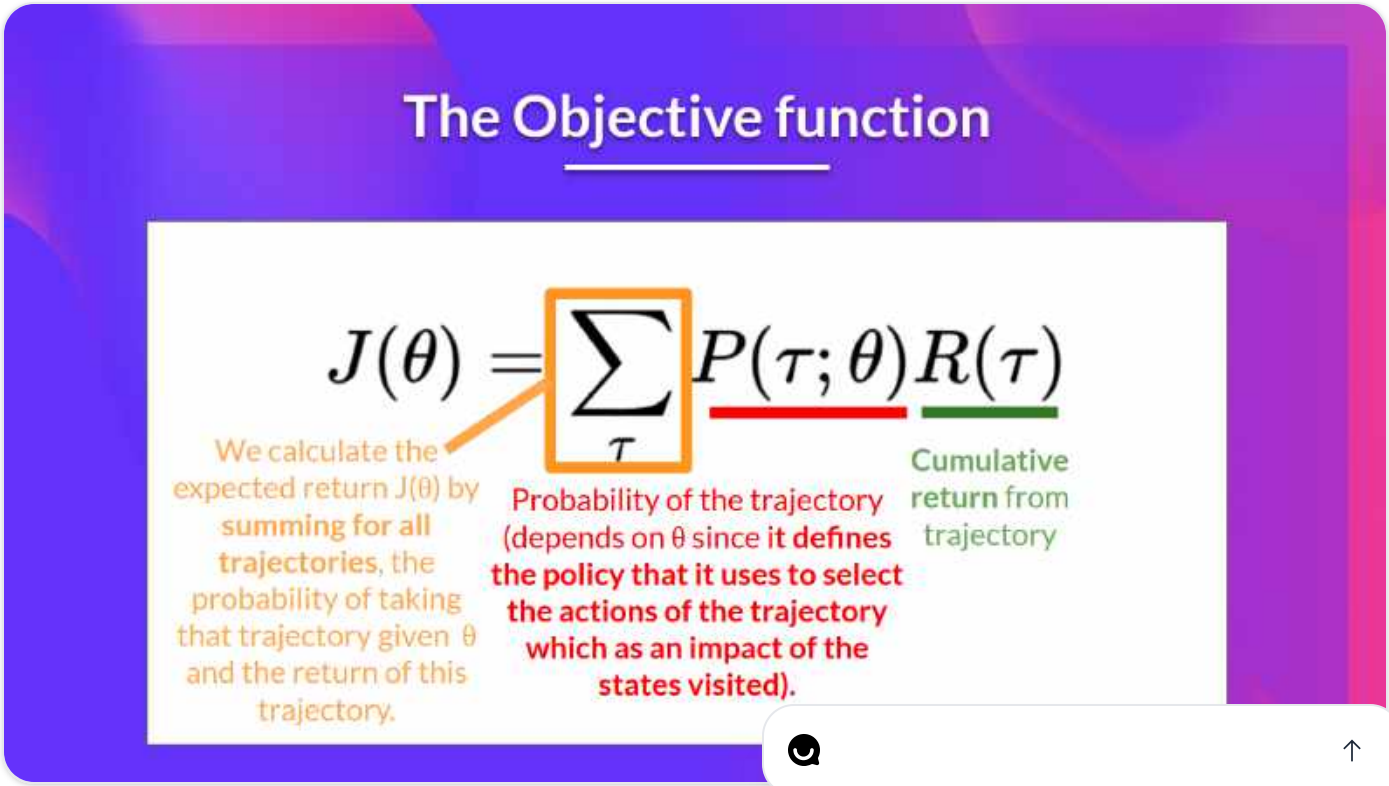
The objective function

The *objective function* gives us the **performance of the agent** given a trajectory (state action sequence without considering reward (contrary to an episode)), and it outputs the *expected cumulative reward*.



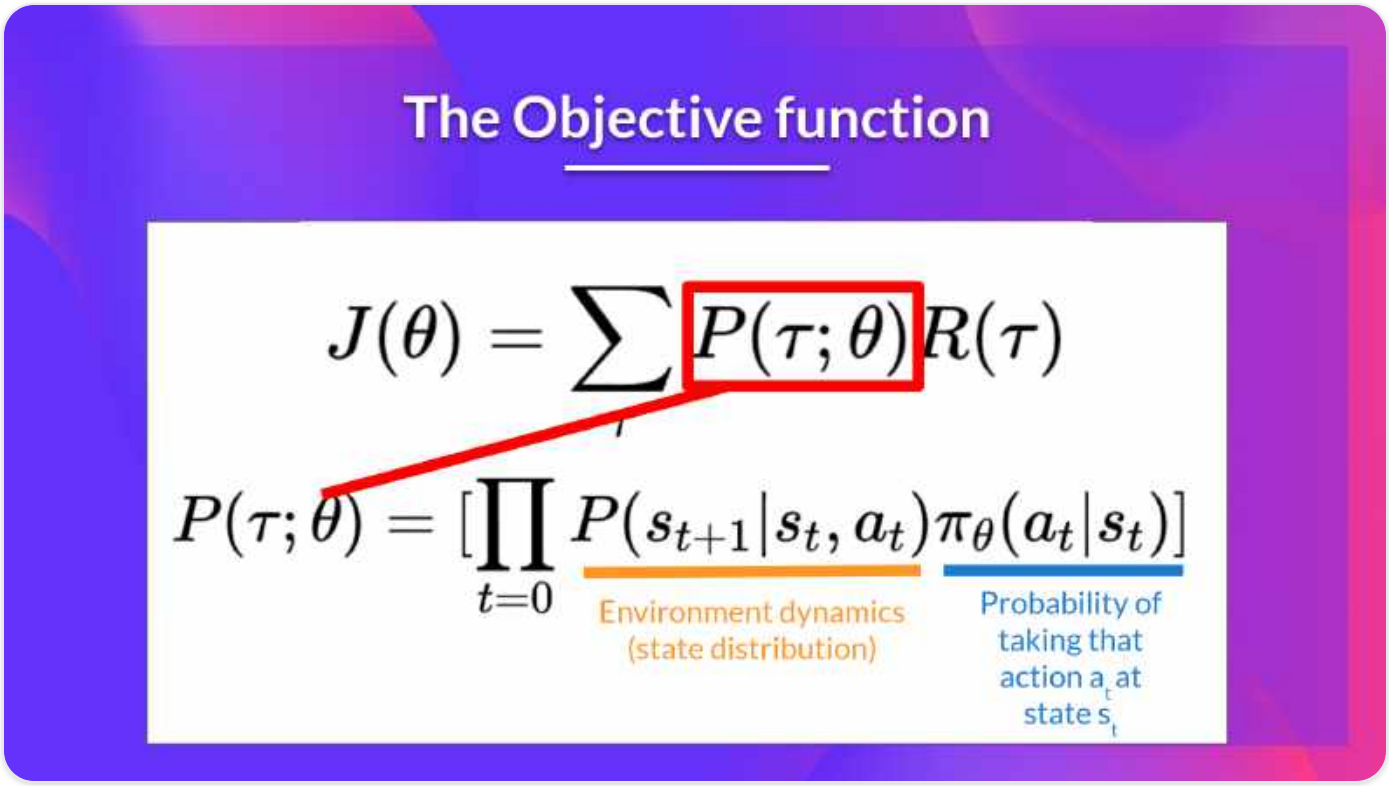
Let's give some more details on this formula:

- The *expected return* (also called expected cumulative reward), is the weighted average (where the weights are given by $P(\tau;\theta)$ of all possible values that the return $R(\tau)$ can take).



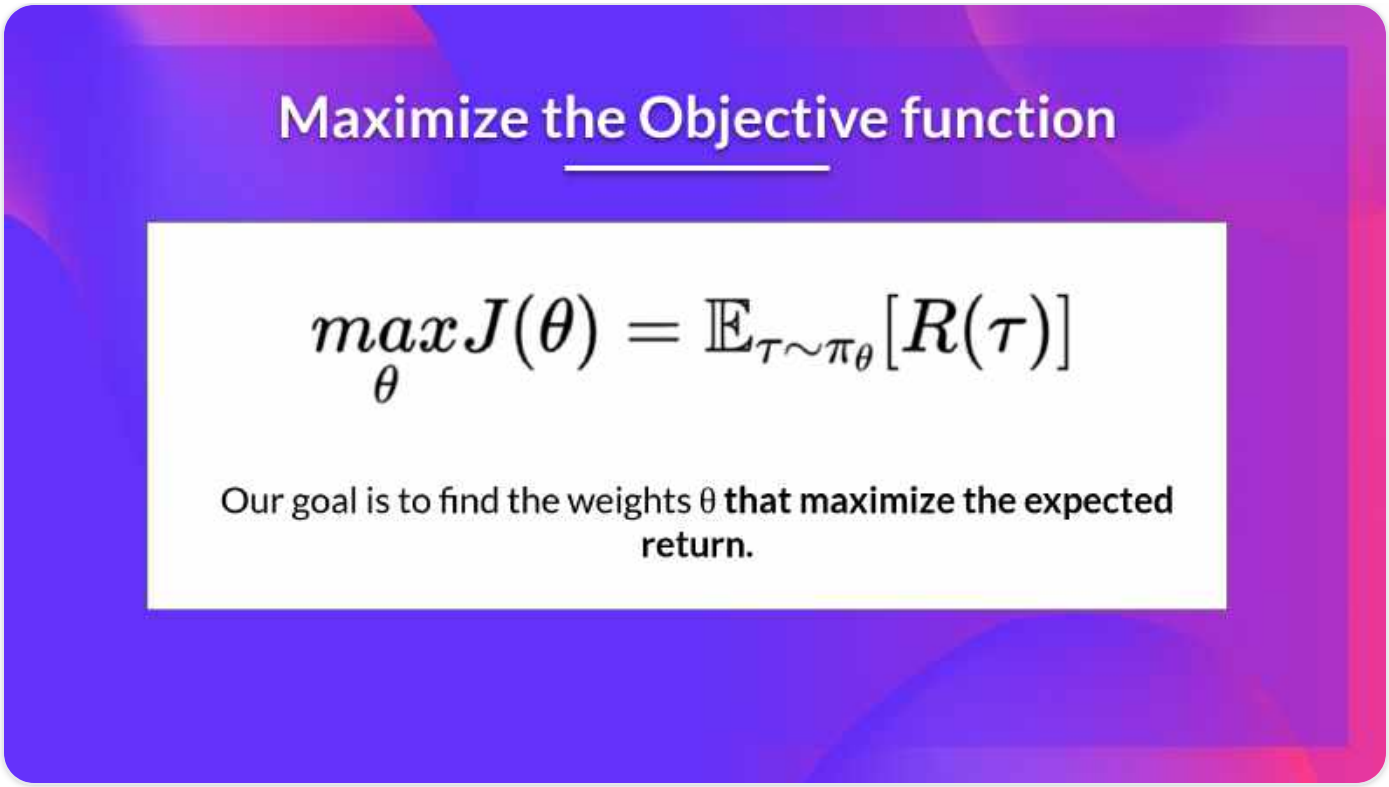
-$R(\tau)$: Return from an arbitrary trajectory. To take this quantity and use it to calculate the expected return, we need to multiply it by the probability of each possible trajectory.

-$P(\tau;\theta)$: Probability of each possible trajectory τ (that probability depends on θ since it defines the policy that it uses to select the actions of the trajectory which has an impact of the states visited).



-$J(\theta)$: Expected return, we calculate it by summing for all trajectories, the probability of taking that trajectory given θ multiplied by the return of this trajectory.

Our objective then is to maximize the expected cumulative reward by finding the θ that will output the best action probability distributions:



Gradient Ascent and the Policy-gradient Theorem

Policy-gradient is an optimization problem: we want to find the values of θ that maximize our objective function $J(\theta)$, so we need to use **gradient-ascent**. It's the inverse of *gradient-descent* since it gives the direction of the steepest increase of $J(\theta)$.

(If you need a refresher on the difference between gradient descent and gradient ascent check this and this).

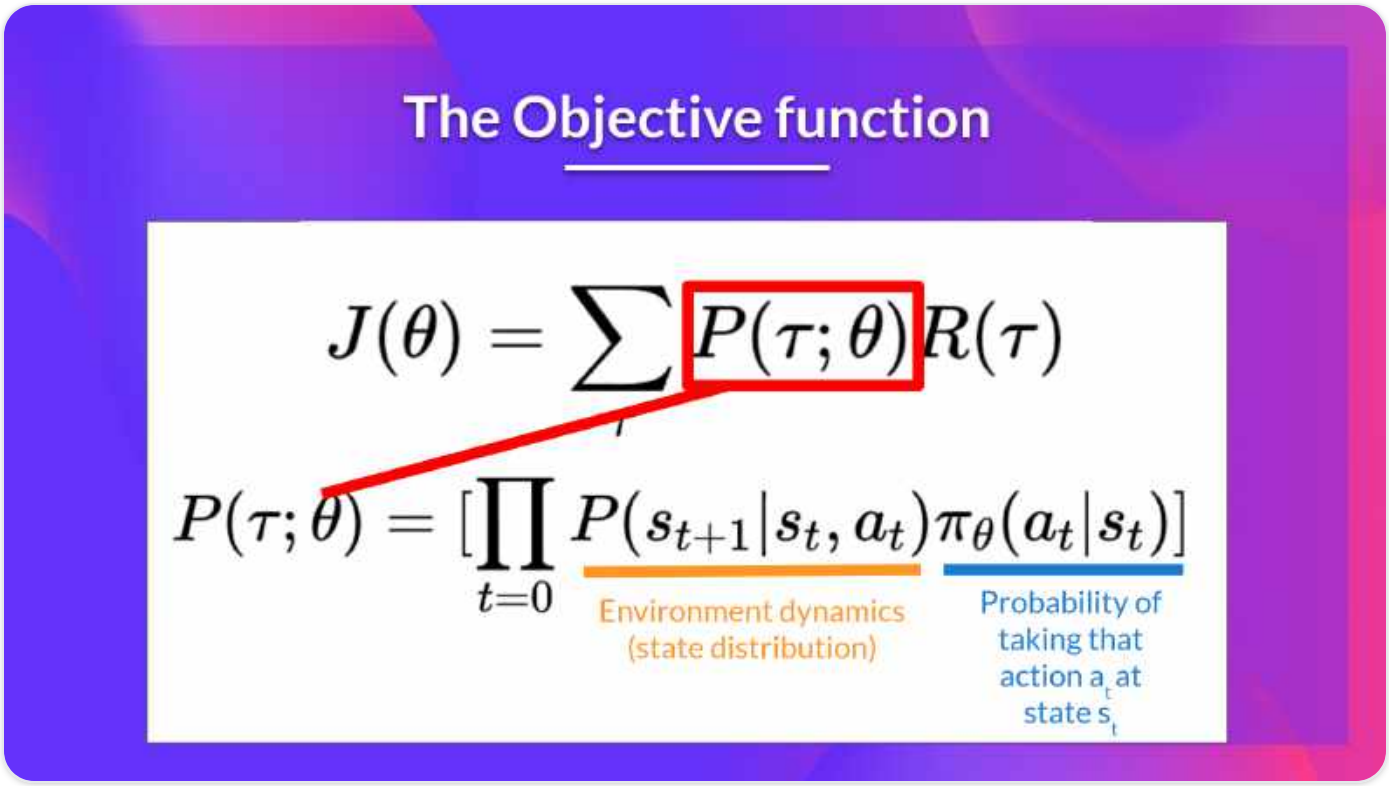
Our update step for gradient-ascent is: $\theta \leftarrow \theta + \alpha * \nabla_\theta J(\theta)$

We can repeatedly apply this update in the hopes that θ converges to the value that maximizes $J(\theta)$.

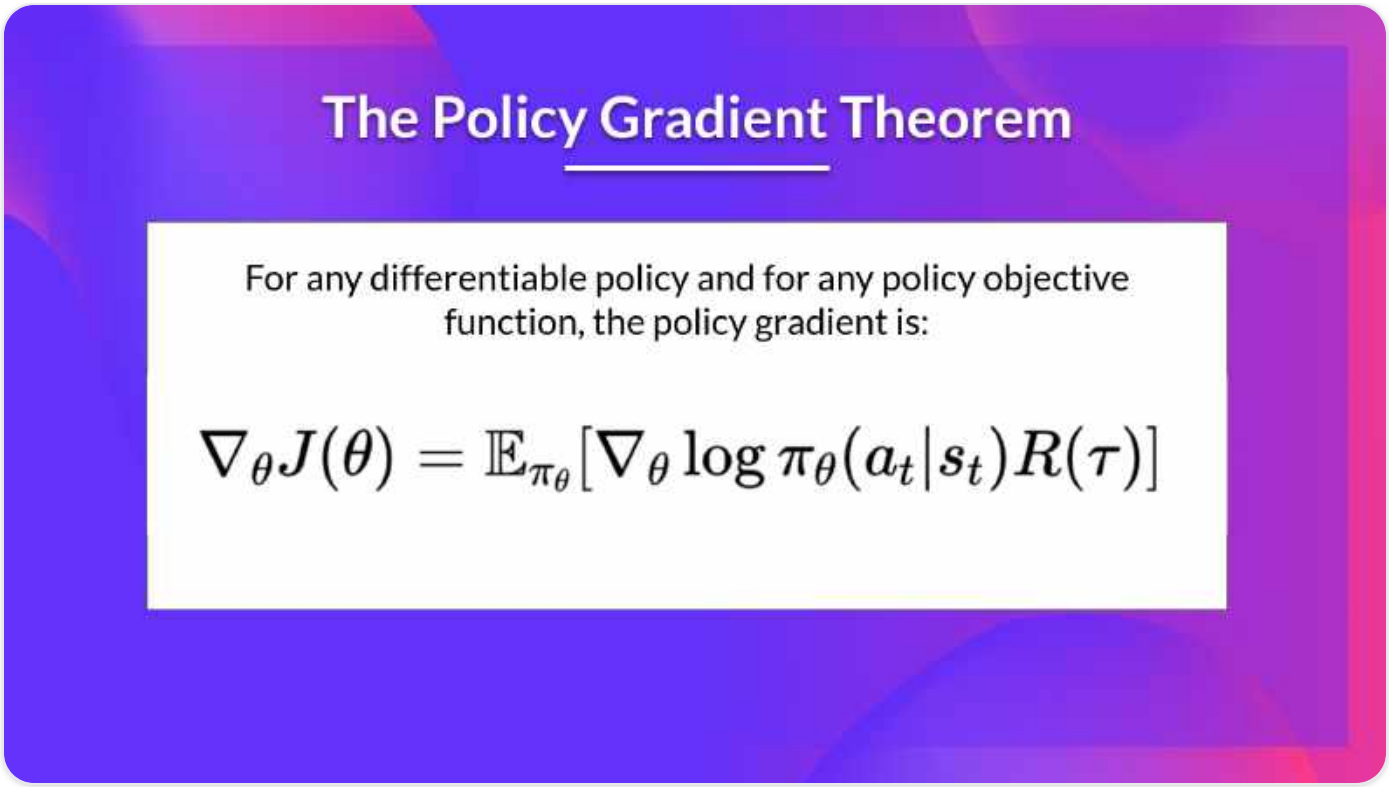
However, there are two problems with computing the derivative of $J(\theta)$:

1. We can't calculate the true gradient of the objective function since it requires calculating the probability of each possible trajectory, which is computationally super expensive. So we want to **calculate a gradient estimation with a sample-**

2. We have another problem that I explain in the next optional section. To differentiate this objective function, we need to differentiate the state distribution, called the Markov Decision Process dynamics. This is attached to the environment. It gives us the probability of the environment going into the next state, given the current state and the action taken by the agent. The problem is that we can't differentiate it because we might not know about it.



Fortunately we're going to use a solution called the Policy Gradient Theorem that will help us to reformulate the objective function into a differentiable function that does not involve the differentiation of the state distribution.



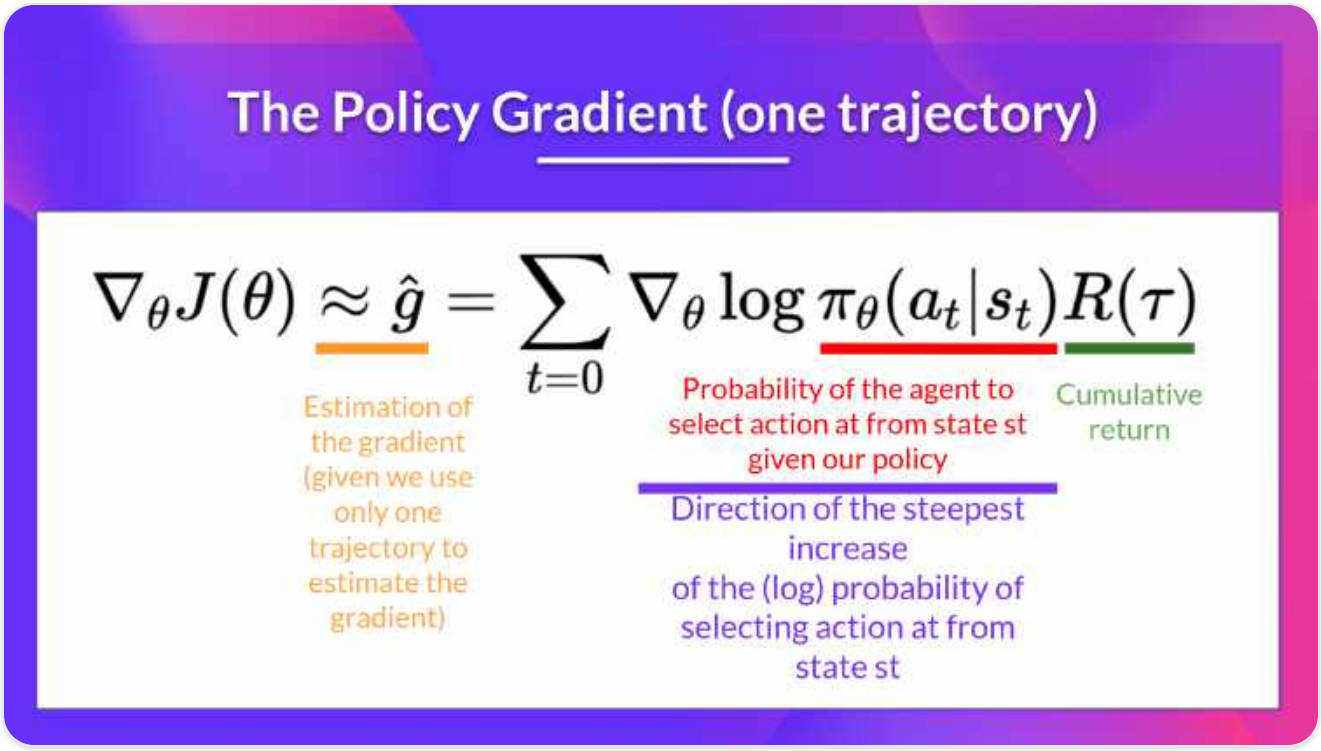
If you want to understand how we derive this formula for approximating the gradient, check out the next (optional) section.

The Reinforce algorithm (Monte Carlo Reinforce)

The Reinforce algorithm, also called Monte-Carlo policy-gradient, is a policy-gradient algorithm that **uses an estimated return from an entire episode to update the policy parameter** θ:

In a loop:

- Use the policy π_θ to collect an episode τ
- Use the episode to estimate the gradient $\hat{g} = \nabla_\theta J(\theta)$



- Update the weights of the policy: $\theta \leftarrow \theta + \alpha \hat{g}$

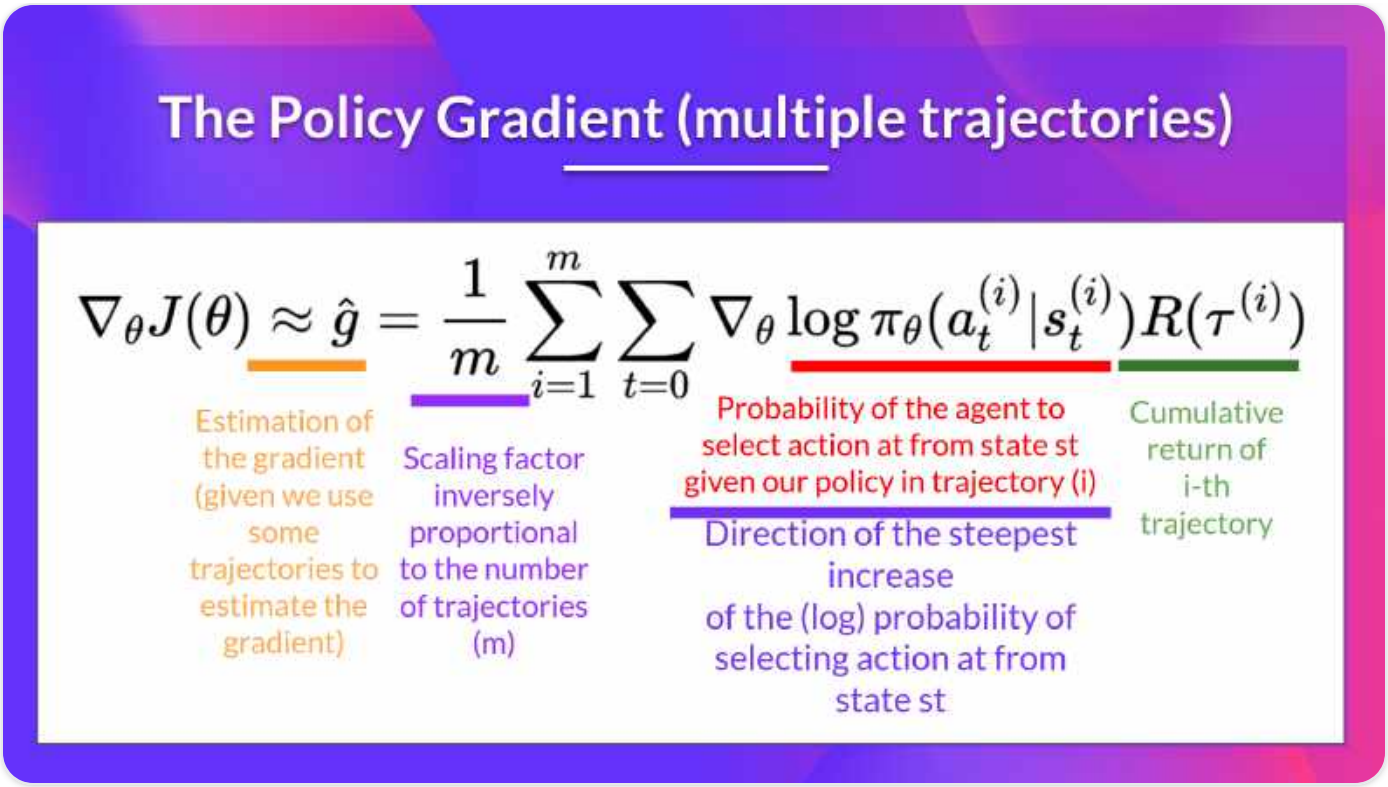
We can interpret this update as follows:

-$\nabla_\theta log \pi_\theta(a_t|s_t)$ is the direction of **steepest increase of the (log) probability** of selecting action a_t from states_t. This tells us **how we should change the weights of policy** if we want to increase/decrease the log probability of selecting actiona_t at states_t.

-$R(\tau)$: is the scoring function:

- If the return is high, it will **push up the probabilities** of the (state, action) combinations.
- Otherwise, if the return is low, it will **push down the probabilities** of the (state, action) combinations.

We can also **collect multiple episodes (trajectories)** to estimate the gradient:



<> Update on GitHub

← The advantages and disadvantages of policy-gradient methods

(Optional) the Policy Gradient Theorem →

Search models, datasets, users...

Deep RL Course documentation
From Q-Learning to Deep Q-Learning

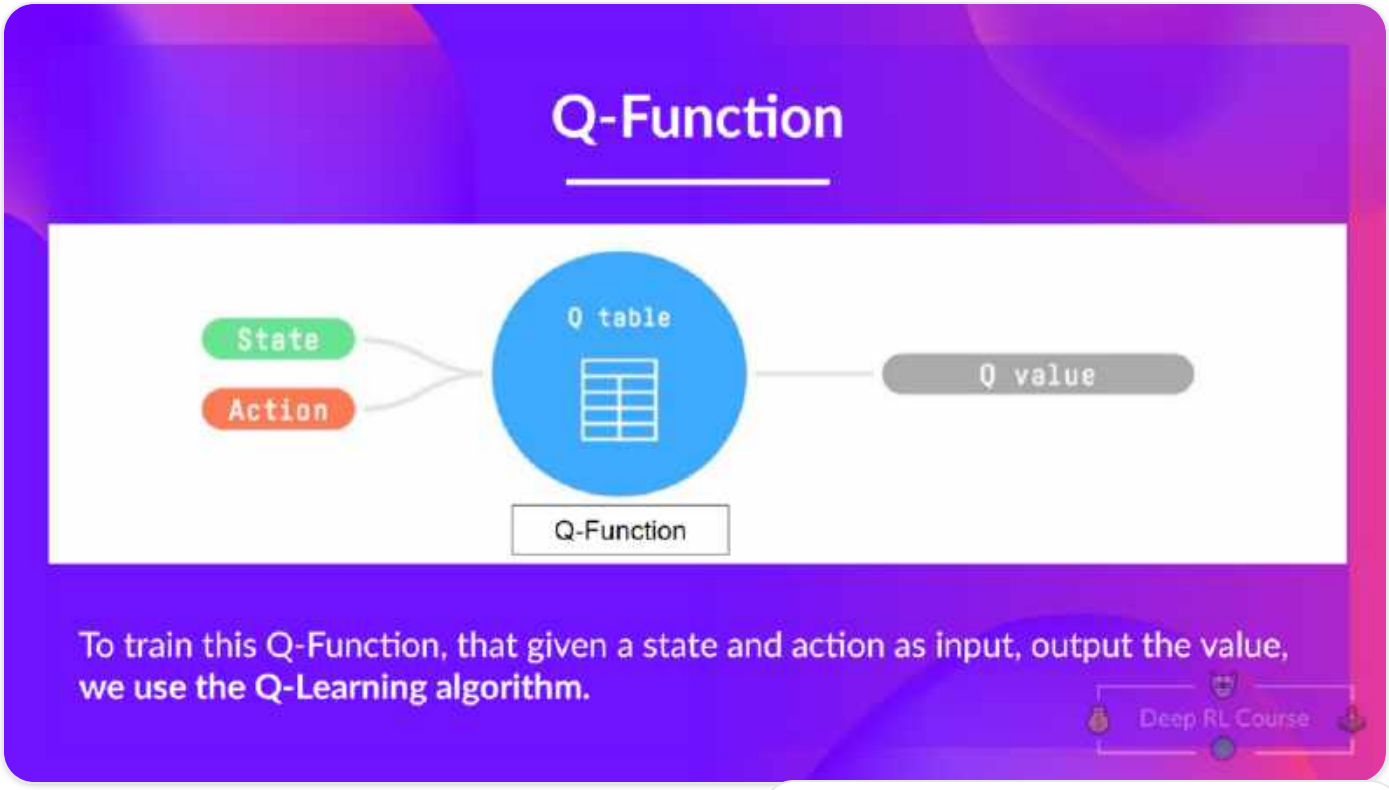
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

From Q-Learning to Deep Q-Learning

We learned that **Q-Learning is an algorithm we use to train our Q-Function**, an **action-value function** that determines the value of being at a particular state and taking a specific action at that state.



Ask HuggingChat

The **Q comes from "the Quality" of that action at that state.**

Internally, our Q-function is encoded by **a Q-table, a table where each cell corresponds to a state-action pair value.** Think of this Q-table as **the memory or cheat sheet of our Q-function.**

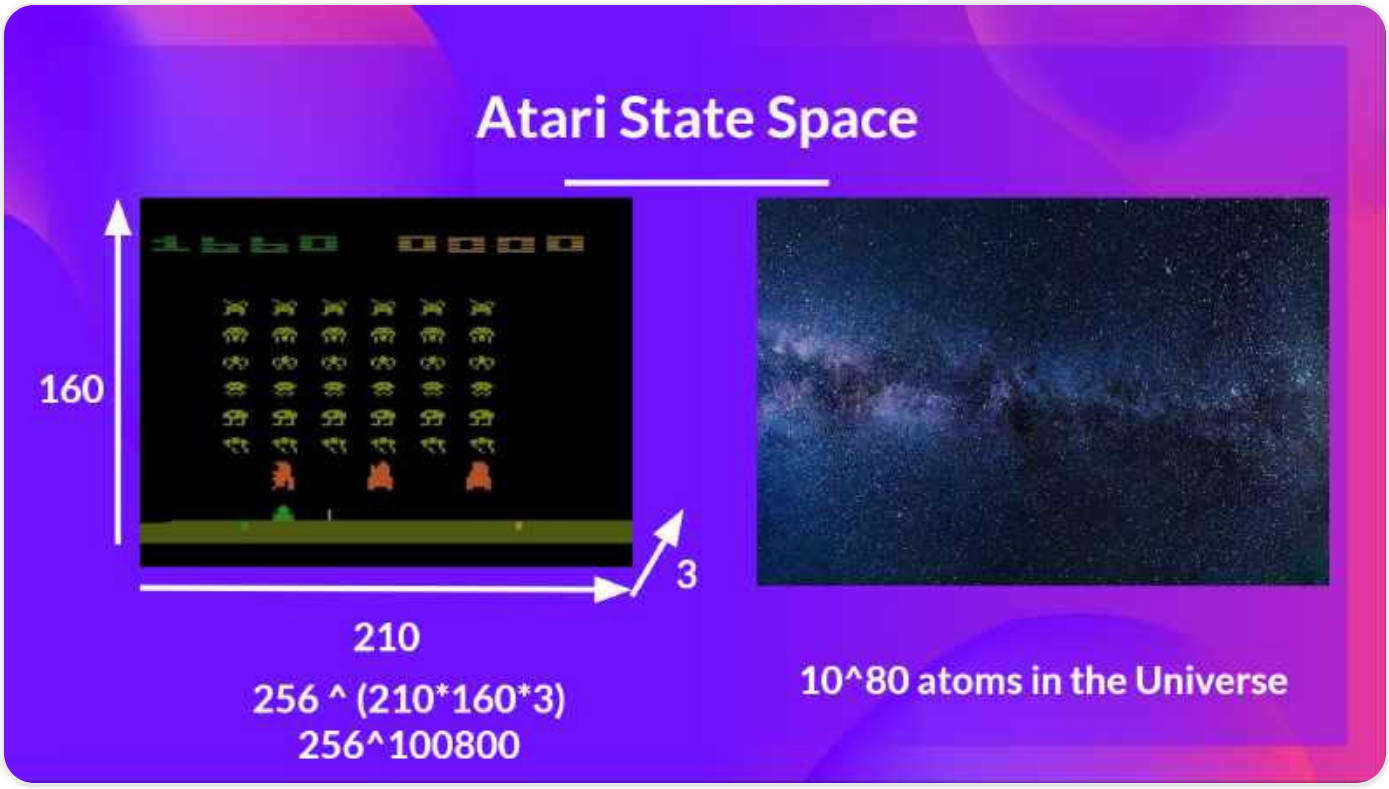
The problem is that Q-Learning is a *tabular method*. This becomes a problem if the states and actions spaces **are not small enough to be represented efficiently by arrays and tables**. In other words: it is **not scalable**. Q-Learning worked well with small state space environments like:

- FrozenLake, we had 16 states.
- Taxi-v3, we had 500 states.

But think of what we're going to do today: we will train an agent to learn to play Space Invaders, a more complex game, using the frames as input.

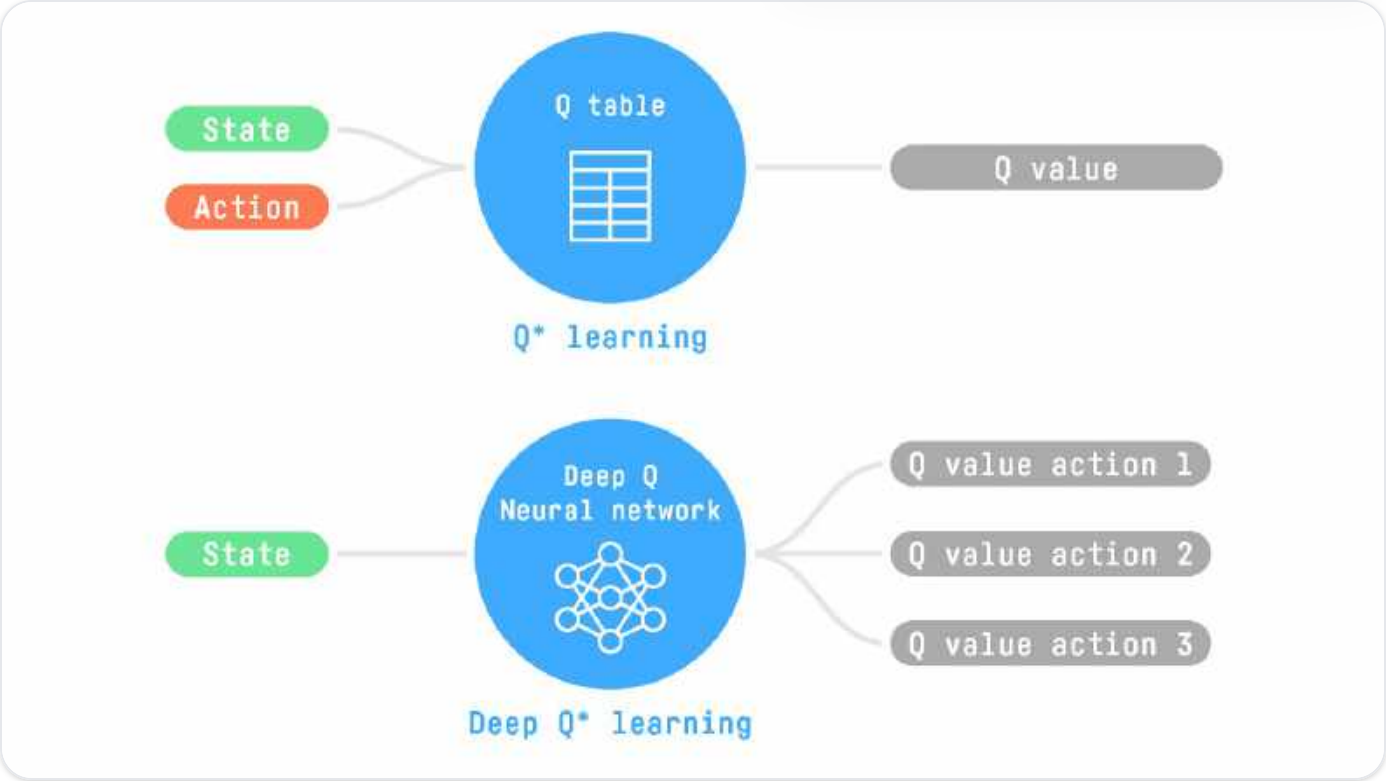
As Nikita Melkozerov mentioned**, Atari environments** have an observation space with a shape of (210, 160, 3)*, containing values ranging from 0 to 255 so that gives us$256^{210 \times 160 \times 3} = 256^{100800}$ possible observations (for comparison, we have approximately10^{80} atoms in the observable universe).

- A single frame in Atari is composed of an image of 210x160 pixels. Given that the images are in color (RGB), there are 3 channels. This is why the shape is (210, 160, 3). For each pixel, the value can go from 0 to 255.



Therefore, the state space is gigantic; due to this, creating and updating a Q-table for that environment would not be efficient. In this case, the best idea is to approximate the Q-values using a parametrized Q-function $Q_\theta(s, a)$.

This neural network will approximate, given a state, the different Q-values for each possible action at that state. And that's exactly what Deep Q-Learning does.



Now that we understand Deep Q-Learning, let's dive deeper into the Deep Q-Network.

<> Update on GitHub

← Introduction

The Deep Q-Network (DQN) →

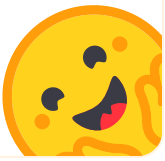
Search models, datasets, users...

Deep RL Course documentation
Generalization in Reinforcement Learning

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Generalization in Reinforcement Learning

Generalization plays a pivotal role in the realm of Reinforcement Learning. While **RL algorithms demonstrate good performance in controlled environments**, the real world presents a **unique challenge due to its non-stationary and open-ended nature**.

As a result, the development of RL algorithms that stay robust in the face of environmental variations, coupled with the capability to transfer and adapt to uncharted yet analogous tasks and settings, becomes fundamental for real world application of RL.

If you're interested to dive deeper into this research subject, we recommend exploring the following resource:

- Generalization in Reinforcement Learning by Robert Kirk: this comprehensive survey provides an insightful **overview of the concept of generalization in RL**, making it an excellent starting point for your exploration.

- Improving Generalization in Reinforcement Learning using Policy Similarity Embeddings

<> Update on GitHub

Ask HuggingChat

← Offline vs. Online Reinforcement Learning Reinforcement Learning from Human Feedback →

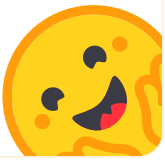
Search models, datasets, users...

Deep RL Course documentation
Getting started:

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Getting started:

To get started, download the project from here (click on the download icon next to `GDRL-IL-Project.zip`). The zip file features both the “Starter” and “Complete” projects.

The game code is already implemented in the starter project and the nodes are configured. We will focus on:

- Implementing the code for the AIController node,
- Recording expert demonstrations,
- Training the agent and exporting an .onnx file which we can use for inference in Godot.

Open the starter project in Godot

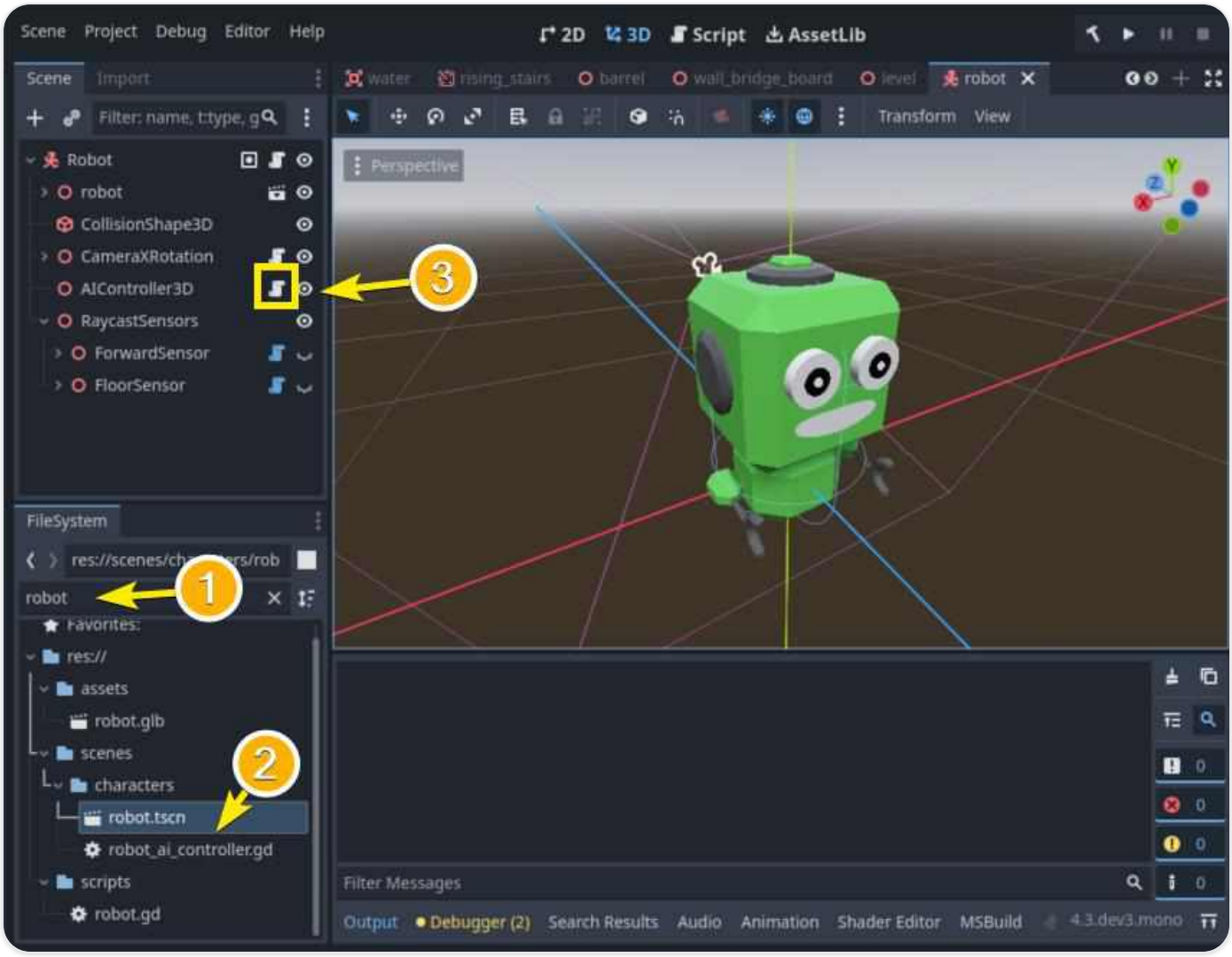
Extract the zip file, open Godot, click “Import” and navigate to the `Starter\Godot` folder of the extracted archive.

Open the robot scene

> You can search for “robot” in the FileSystem search.

Ask HuggingChat

This scene contains a couple of different nodes, including the `robot` node, which contains the visual shape of the robot, `CameraXRotation` node which is used to rotate the camera “up-down” using the mouse in human control modes. The AI agent does not control this node since it is not necessary for learning the task. `RaycastSensors` node contains two Raycast sensors that help the agent to “sense” parts of the game world, including walls, floors, etc.



Click on the scroll next to AIController3D to open the script for editing

> You might have to collapse the “robot” branch to find it more easily, or you can type `aicontroller` in the Filter box above the `Robot` node.

Replace the get_obs() and get_reward() methods

```
func get_obs() -> Dictionary:
    var observations: Array[float] = []
    for raycast_sensor in raycast_sensors:
        observations.append_array(raycast_sensor.get_observation())

    var level_size = 16.0

    var chest_local = to_local(chest.global_position)
    var chest_direction = chest_local.normalized()
    var chest_distance = clampf(chest_local.length(), 0.0, level_size)

    var lever_local = to_local(lever.global_position)
    var lever_direction = lever_local.normalized()
    var lever_distance = clampf(lever_local.length(), 0.0, level_size)

    var key_local = to_local(key.global_position)
    var key_direction = key_local.normalized()
    var key_distance = clampf(key_local.length(), 0.0, level_size)

    var raft_local = to_local(raft.global_position)
    var raft_direction = raft_local.normalized()
    var raft_distance = clampf(raft_local.length(), 0.0, level_size)

    var player_speed = player.global_basis.inverse() * player.velocity.limit_length(5.0)

    (
        observations
        .append_array(
            [
                chest_direction.x,
                chest_direction.y,
                chest_direction.z,
                chest_distance,
                lever_direction.x,
                lever_direction.y,
                lever_direction.z,
                lever_distance,
                key_direction.x,
                key_direction.y,
                key_direction.z,
                key_distance,
                raft_direction.x,
```

```
                raft_direction.y,
                raft_direction.z,
                raft_distance,
                raft.movement_direction_multiplier,
                float(player._is_lever_pulled),
                float(player._is_chest_opened),
                float(player._is_key_collected),
                float(player.is_on_floor()),
                player_speed.x,
                player_speed.y,
                player_speed.z,
            ]
        )
    )
    return {"obs": observations}

func get_reward() -> float:
    return reward
```

In `get_obs()`, we first get the obs from the two Raycast sensors added to the `AIController3D` node in the inspector, and add them to the obs, then we get the relative position vectors to chest, lever, key, and raft, which we separate into directions and distances, and then we add them to the obs as well.

We also add other game state info to the obs:

- has the lever has been pulled,
- was the key collected,
- was the chest opened,
- is the player on floor (also determines whether the player can jump),
- the normalized local velocity of the player.

We convert boolean values such as `_is_lever_pulled` to floats (0 or 1).

In `get_reward()`, we only need to return the current reward.

Replace the _physics_process() and reset() methods with the implementation below:

```
func _physics_process(delta: float) -> void:
    # Reset on timeout, this is implemented in parent class to set needs_reset to true,
    # we are re-implementing here to call player.game_over() that handles the game reset.
    n_steps += 1
    if n_steps > reset_after:
        player.game_over()

    # In training or onnx inference modes, this method will be called by sync node with a
    # For expert demo recording mode, it will be called without any actions (as we set th
    # For human control mode the method will not be called, so we call it here without ar
    if control_mode == ControlModes.HUMAN:
        set_action()

    # Reset the game faster if the lever is not pulled.
    steps_without_lever_pulled += 1
    if steps_without_lever_pulled > 200 and (not player._is_lever_pulled):
        player.game_over()

func reset():
    super.reset()
    steps_without_lever_pulled = 0
```

Replace the get_action_space() , get_action() , and set_action() methods with the implementation below:

```
# Defines the actions for the AI agent ("size": 2 means 2 floats for this action)
func get_action_space() -> Dictionary:
    return {
        "movement": {"size": 2, "action_type": "continuous"},
        "rotation": {"size": 1, "action_type": "continuous"},
        "jump": {"size": 1, "action_type": "continuous"},
        "use_action": {"size": 1, "action_type": "continuous"}
    }

# We return the action values in the same order as defined in get_action_space() (importa
# For actions of size 1, we return 1 float in th
# set_action is called just before get_action by
func get_action():
```

```
    return [
        # "movement" action values
        player.requested_movement.x,
        player.requested_movement.y,
        # "rotation" action value
        player.requested_rotation.x,
        # "jump" action value (-1 if not requested, 1 if requested)
        -1.0 + 2.0 * float(player.jump_requested),
        # "use_action" action value (-1 if not requested, 1 if requested)
        -1.0 + 2.0 * float(player.use_action_requested)
    ]

# Here we set human control and AI control actions to the robot
func set_action(action = null) -> void:
    # If there's no action provided, it means that AI is not controlling the robot (humar
    if not action:
        # Only rotate if the mouse has moved since the last set_action call
        if previous_mouse_movement == mouse_movement:
            mouse_movement = Vector2.ZERO

        player.requested_movement = Input.get_vector(
            "move_left", "move_right", "move_forward", "move_back"
        )
        player.requested_rotation = mouse_movement

        var use_action = Input.is_action_pressed("requested_action")
        var jump = Input.is_action_pressed("requested_jump")

        player.use_action_requested = use_action
        player.jump_requested = jump

        previous_mouse_movement = mouse_movement
    else:
        # If there is action provided, we set the actions received from the AI agent
        player.requested_movement = Vector2(action.movement[0], action.movement[1])
        # The agent only rotates the robot along the Y axis, no need to rotate the camera
        player.requested_rotation = Vector2(action.rotation[0], 0.0)
        player.jump_requested = bool(action.jump[0] > 0)
        player.use_action_requested = bool(action.use_action[0] > 0)
```

For `get_action()` (only needed if using the demo record mode), we need to provide the actions that we want the agent to send when it encounters the same state. It is important for the values to be in the correct range (`-1.0 to 1.0`), which is why we have the `-1 + 2 * variable` for boolean states, and in the correct order, as defined in `get_action_space()`.

In demo record mode, `set_action()` is called without providing actions, as we need to set the action values based on human input. In training/inference modes, the method is called with an `action` argument containing values for all of the actions provided by the RL model, so we have an `if/else` to handle both cases.

More info is included in the code comments.

Replace the _input method with the implementation below:

```
# Record mouse movement for human and demo_record modes
# We don't directly rotate in input to allow for frame skipping (action_repeat setting) w
# will also be applied to the AI agent in training/inference modes.
func _input(event):
    if not (heuristic == "human" or heuristic == "demo_record"):
        return

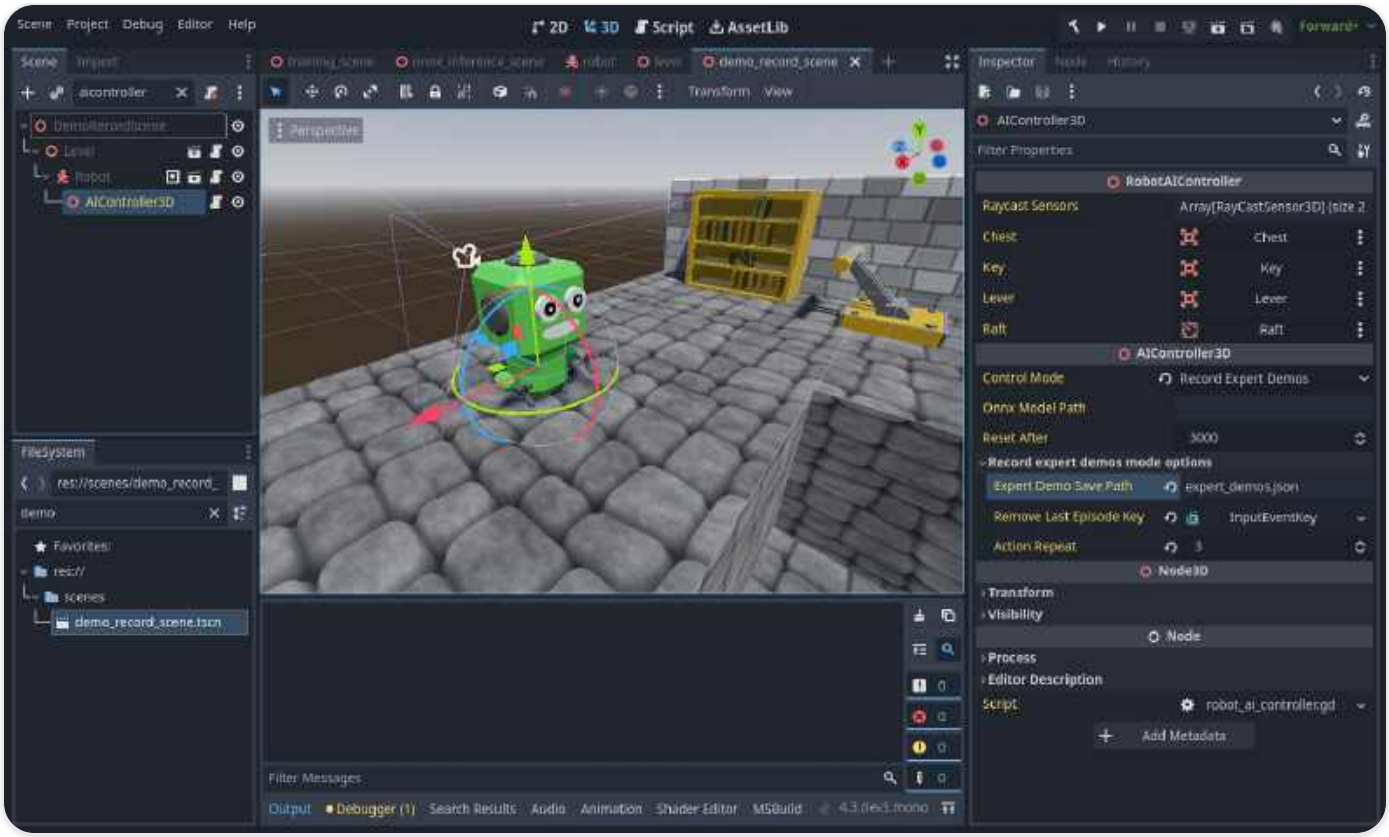
    if event is InputEventMouseMotion:
        var movement_scale: float = 0.005
        mouse_movement.y = clampf(event.relative.y * movement_scale, -1.0, 1.0)
        mouse_movement.x = clampf(event.relative.x * movement_scale, -1.0, 1.0)
```

This code part records mouse movement in case of human control and demo record modes.

Finally, save the script. We are ready for the next step.

Open the demo record scene, and click on AIController3D node

> You can search for “demo” in the FileSystem search, and you can search for “aicontroller” in the scene's filter box.



You don’t need to make any changes as everything is preset, but let’s go over the things you would need to set in your own env:

The scene contains modified `Level > Robot > AIController3D` node settings:

- `Control Mode` is set to `Record Expert Demos`
- `Expert Demo Save Path` is filled out
- `Action Repeat` is set to the same value as is set for the `Sync` node in `training_scene` and `onnx_inference_scene`. This means that every action set by the agent is repeated for 3 physics frames. The setting in `AIController` adds the same action repeat to the human input (which introduces some lag) to match the same behavior. This is a fairly low value which doesn’t introduce much lag. If you change this value, make sure to change it in all 3 places.
- `Remove Last Episode` key allows us to set a key that can be used to remove a failed episode during recording, without having to restart the entire session. E.g. if the robot falls in the water and the game resets, we can use this key to remove the previously recorded episode while recording the next one. It is set to `R`, but you can change it to any key by clicking on it, and then clicking on the `Configure` button.

Another way to make episode recording easier in challenging environments is to slow down the environment during recording. This can easily be done by clicking on the `Sync` node in the scene, and adjusting the `Speed Up` property (set to 1 by default).

Let's record some demos:

> Note that the demos will only be saved if we have recorded at least one complete episode and closed the game window by clicking on "X" or pressing ALT+F4. Using the stop button in Godot editor will not save the demos. It's best to try recording just one episode first, then check if you see "expert_demos.json" in the filesystem or in the Godot project folder.

Make sure that you are still in the `demo_record_scene`, `press F6` and the demo recording will start.

Controls:

- mouse controls the camera (if you need to adjust mouse sensitivity, open the `robot` scene, click on the `Robot` node and adjust the `Rotation Speed`, keep it the same value for recording demos, training and inference),
- `WASD` controls the player movement,
- `SPACE` jumps,
- `E` activates the lever and opens the chest

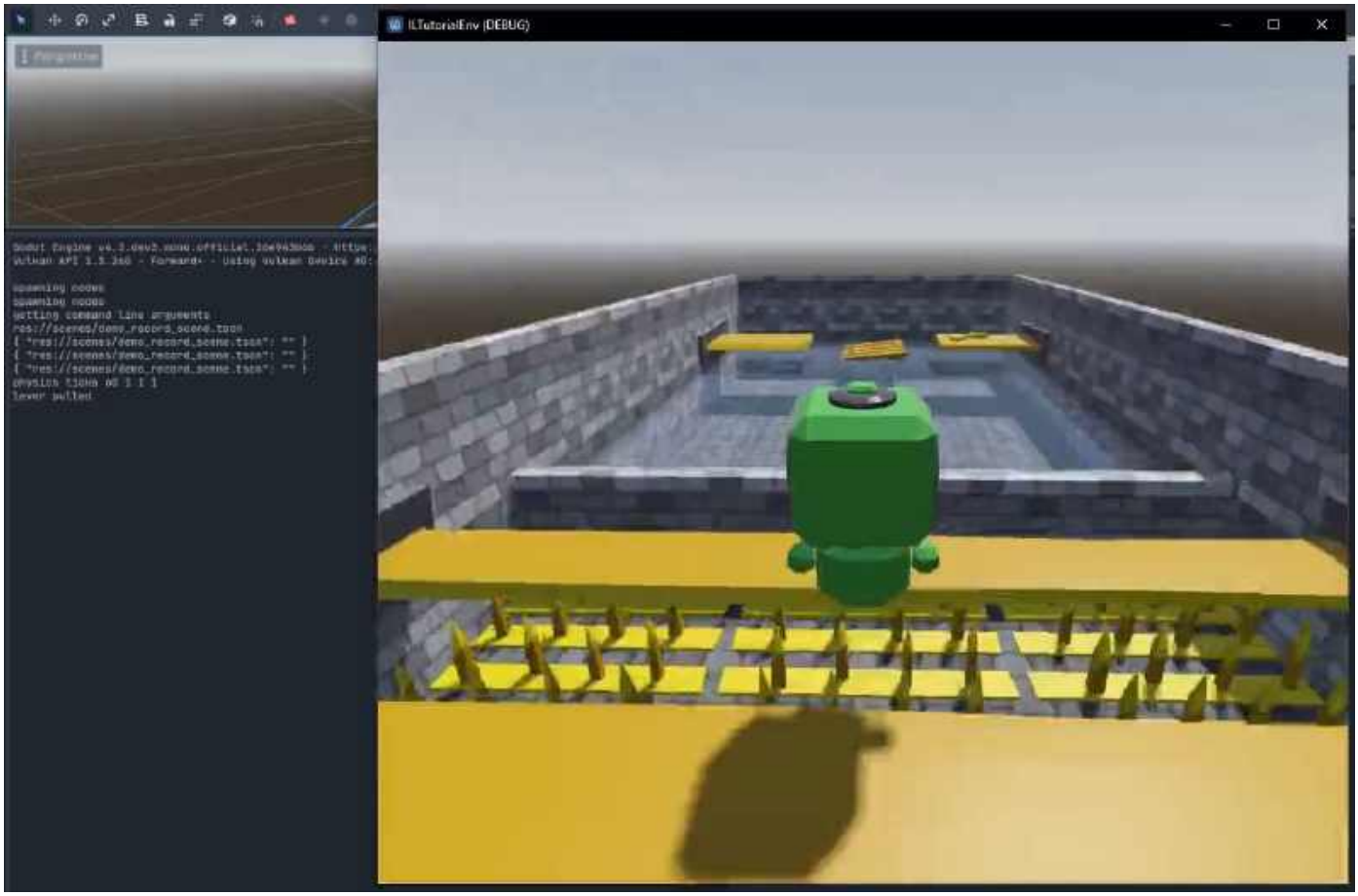
You can take a few practice first to get familiar with the env. If you wish to skip recording demos, you can also find the pre-recorded demos in the completed project and use the `expert_demos.json` file from there.

The recorded demos should include at least 22-24 complete successful episodes. Multiple demo files can also be used in the training stage, so you don't have to record all demos in one go (you can change the file name using the `Expert Demo Save Path` property mentioned before).

Recording 23 episodes took me ~10 minutes (as the key
24 would provide an equal distribution of key positions

approaching the lever or chest, I pressed and held the `E` key slightly longer to ensure the action is recorded for multiple steps when near those objects. I also removed a couple of episodes that I didn't complete successfully by pressing the `R` key during the following episode.

Here's a sped-up video of the demo recording process:



Export the game for training:

You can export the game from Godot using `Project > Export`.

<> Update on GitHub

← The environment

Train our robot →

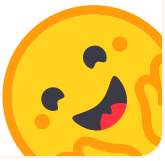
Search models, datasets, users...

Deep RL Course documentation
Glossary

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Glossary

This is a community-created glossary. Contributions are welcome!

Agent

An agent learns to **make decisions by trial and error, with rewards and punishments from the surroundings**.

Environment

An environment is a simulated world **where an agent can learn by interacting with it**.

Markov Property

It implies that the action taken by our agent is **conditional solely on the present state and independent of the past states and actions**.

Observations/State

- **State**: Complete description of the state of the world.
- **Observation**: Partial description of the state of the

Ask HuggingChat

Actions

- **Discrete Actions**: Finite number of actions, such as left, right, up, and down.
- **Continuous Actions**: Infinite possibility of actions; for example, in the case of self-driving cars, the driving scenario has an infinite possibility of actions occurring.

Rewards and Discounting

- **Rewards**: Fundamental factor in RL. Tells the agent whether the action taken is good/bad.
- RL algorithms are focused on maximizing the **cumulative reward**.
- **Reward Hypothesis**: RL problems can be formulated as a maximisation of (cumulative) return.
- **Discounting** is performed because rewards obtained at the start are more likely to happen as they are more predictable than long-term rewards.

Tasks

- **Episodic**: Has a starting point and an ending point.
- **Continuous**: Has a starting point but no ending point.

Exploration v/s Exploitation Trade-Off

- **Exploration**: It's all about exploring the environment by trying random actions and receiving feedback/returns/rewards from the environment.
- **Exploitation**: It's about exploiting what we know about the environment to gain maximum rewards.
- **Exploration-Exploitation Trade-Off**: It balances how much we want to **explore** the environment and how much we want to **exploit** what we know about the environment.

Policy

- **Policy**: It is called the agent's brain. It tells us what action to take, given the state.

- **Optimal Policy**: Policy that **maximizes** the **expected return** when an agent acts according to it. It is learned through *training*.

Policy-based Methods:

- An approach to solving RL problems.
- In this method, the Policy is learned directly.
- Will map each state to the best corresponding action at that state. Or a probability distribution over the set of possible actions at that state.

Value-based Methods:

- Another approach to solving RL problems.
- Here, instead of training a policy, we train a **value function** that maps each state to the expected value of being in that state.

Contributions are welcome 🤗

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- @lucifermorningstar1305
- @daspartho
- @misza222

<> Update on GitHub

← Summary Hands-on →

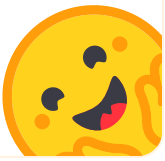
Search models, datasets, users...

Deep RL Course documentation
Glossary

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Glossary

This is a community-created glossary. Contributions are welcomed!

- **Tabular Method:** Type of problem in which the state and action spaces are small enough to approximate value functions to be represented as arrays and tables. **Q-learning** is an example of tabular method since a table is used to represent the value for different state-action pairs.
- **Deep Q-Learning:** Method that trains a neural network to approximate, given a state, the different **Q-values** for each possible action at that state. It is used to solve problems when observational space is too big to apply a tabular Q-Learning approach.
- **Temporal Limitation** is a difficulty presented when the environment state is represented by frames. A frame by itself does not provide temporal information. In order to obtain temporal information, we need to **stack** a number of frames together.
- **Phases of Deep Q-Learning:**
  - **Sampling:** Actions are performed, and observed experience tuples are stored in a **replay memory**.
  - **Training:** Batches of tuples are selected randomly and the neural network updates its weights using gradient descent.

Ask HuggingChat

- **Solutions to stabilize Deep Q-Learning:**
  - **Experience Replay:** A replay memory is created to save experiences samples that can be reused during training. This allows the agent to learn from the same experiences multiple times. Also, it helps the agent avoid forgetting previous experiences as it gets new ones.
  - **Random sampling** from replay buffer allows to remove correlation in the observation sequences and prevents action values from oscillating or diverging catastrophically.
  - **Fixed Q-Target:** In order to calculate the **Q-Target** we need to estimate the discounted optimal **Q-value** of the next state by using Bellman equation. The problem is that the same network weights are used to calculate the **Q-Target** and the **Q-value**. This means that everytime we are modifying the **Q-value**, the **Q-Target** also moves with it. To avoid this issue, a separate network with fixed parameters is used for estimating the Temporal Difference Target. The target network is updated by copying parameters from our Deep Q-Network after certain **C steps**.
  - **Double DQN:** Method to handle **overestimation** of **Q-Values**. This solution uses two networks to decouple the action selection from the target **Value generation**:
    - **DQN Network** to select the best action to take for the next state (the action with the highest **Q-Value**)
    - **Target Network** to calculate the target **Q-Value** of taking that action at the next state. This approach reduces the **Q-Values** overestimation, it helps to train faster and have more stable learning.

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- Dario Paez

<> Update on GitHub

← The Deep Q Algorithm

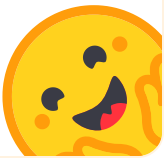
Search models, datasets, users...

Deep RL Course documentation
Glossary

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Glossary

This is a community-created glossary. Contributions are welcomed!

Strategies to find the optimal policy

- **Policy-based methods.** The policy is usually trained with a neural network to select what action to take given a state. In this case it is the neural network which outputs the action that the agent should take instead of using a value function. Depending on the experience received by the environment, the neural network will be re-adjusted and will provide better actions.
- **Value-based methods.** In this case, a value function is trained to output the value of a state or a state-action pair that will represent our policy. However, this value doesn't define what action the agent should take. In contrast, we need to specify the behavior of the agent given the output of the value function. For example, we could decide to adopt a policy to take the action that always leads to the biggest reward (Greedy Policy). In summary, the policy is a Greedy Policy (or whatever decision the user takes) that uses the values of the value-function to decide the actions to take.

Among the value-based methods, we can find two main strategies

Ask HuggingChat

- **The state-value function.** For each state, the state-value function is the expected return if the agent starts in that state and follows the policy until the end.
- **The action-value function.** In contrast to the state-value function, the action-value calculates for each state and action pair the expected return if the agent starts in that state, takes that action, and then follows the policy forever after.

Epsilon-greedy strategy:

- Common strategy used in reinforcement learning that involves balancing exploration and exploitation.
- Chooses the action with the highest expected reward with a probability of 1-epsilon.
- Chooses a random action with a probability of epsilon.
- Epsilon is typically decreased over time to shift focus towards exploitation.

Greedy strategy:

- Involves always choosing the action that is expected to lead to the highest reward, based on the current knowledge of the environment. (Only exploitation)
- Always chooses the action with the highest expected reward.
- Does not include any exploration.
- Can be disadvantageous in environments with uncertainty or unknown optimal actions.

Off-policy vs on-policy algorithms

- **Off-policy algorithms:** A different policy is used at training time and inference time
- **On-policy algorithms:** The same policy is used during training and inference

Monte Carlo and Temporal Difference learning strategies

- **Monte Carlo (MC):** Learning at the end of the episode. With Monte Carlo, we wait until the episode ends and then we update the value functi[illegible] episode.

- **Temporal Difference (TD):** Learning at each step. With Temporal Difference Learning, we update the value function (or policy function) at each step without requiring a complete episode.

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- Ramón Rueda
- Hasarindu Perera
- Arkady Arkhangorodsky

<> Update on GitHub

← Q-Learning Recap Hands-on →

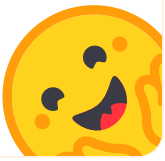
Search models, datasets, users...

Deep RL Course documentation
Glossary

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Glossary

This is a community-created glossary. Contributions are welcome!

- **Deep Q-Learning:** A value-based deep reinforcement learning algorithm that uses a deep neural network to approximate Q-values for actions in a given state. The goal of Deep Q-learning is to find the optimal policy that maximizes the expected cumulative reward by learning the action-values.

- **Value-based methods:** Reinforcement Learning methods that estimate a value function as an intermediate step towards finding an optimal policy.

- **Policy-based methods:** Reinforcement Learning methods that directly learn to approximate the optimal policy without learning a value function. In practice they output a probability distribution over actions.

  The benefits of using policy-gradient methods over value-based methods include:

  - simplicity of integration: no need to store action values;
  - ability to learn a stochastic policy: the agent explores the state space without always taking the same trajectory, and avoids the problem of perceptual aliasing;
  - effectiveness in high-dimensional and contin

Ask HuggingChat

    - improved convergence properties.

- **Policy Gradient:** A subset of policy-based methods where the objective is to maximize the performance of a parameterized policy using gradient ascent. The goal of a policy-gradient is to control the probability distribution of actions by tuning the policy such that good actions (that maximize the return) are sampled more frequently in the future.

- **Monte Carlo Reinforce:** A policy-gradient algorithm that uses an estimated return from an entire episode to update the policy parameter.

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- Diego Carpintero

<> Update on GitHub

← (Optional) the Policy Gradient Theorem Hands-on →

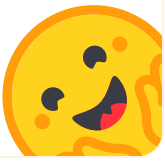
Search models, datasets, users...

Deep RL Course documentation
Godot RL Agents

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Godot RL Agents

Godot RL Agents is an Open Source package that allows video game creators, AI researchers, and hobbyists the opportunity **to learn complex behaviors for their Non Player Characters or agents**.

The library provides:

- An interface between games created in the Godot Engine and Machine Learning algorithms running in Python
- Wrappers for four well known rl frameworks: StableBaselines3, CleanRL, Sample Factory and Ray RLLib
- Support for memory-based agents with LSTM or attention based interfaces
- Support for *2D and 3D games*
- A suite of *AI sensors* to augment your agent's capacity to observe the game world
- Godot and Godot RL Agents are **completely free and open source under a very permissive MIT license**. No strings attached, no royalties, nothing.

You can find out more about Godot RL agents on their GitHub page or their AAAI-2022 Workshop paper. The library's creator, Ed Beeching, is a Research Scientist here at Hugging Face.

Ask HuggingChat

Installation of the library is simple: `pip install godot`

Create a custom RL environment with Godot RL Agents

In this section, you will **learn how to create a custom environment in the Godot Game Engine** and then implement an AI controller that learns to play with Deep Reinforcement Learning.

The example game we create today is simple, **but shows off many of the features of the Godot Engine and the Godot RL Agents library**. You can then dive into the examples for more complex environments and behaviors.

The environment we will be building today is called Ring Pong, the game of pong but the pitch is a ring and the paddle moves around the ring. The **objective is to keep the ball bouncing inside the ring**.



Installing the Godot Game Engine

The Godot game engine is an open source tool for the **creation of video games, tools and user interfaces**.

Godot Engine is a feature-packed, cross-platform game engine designed to create 2D and 3D games from a unified interface. It provides a comprehensive se
making games without having to reinvent the wheel.

number of platforms, including the major desktop platforms (Linux, macOS, Windows) as well as mobile (Android, iOS) and web-based (HTML5) platforms.

While we will guide you through the steps to implement your agent, you may wish to learn more about the Godot Game Engine. Their documentation is thorough, and there are many tutorials on YouTube we would also recommend GDQuest, KidsCanCode and Bramwell as sources of information.

In order to create games in Godot, **you must first download the editor**. Godot RL Agents supports the latest version of Godot, Godot 4.0.

Which can be downloaded at the following links:

- Windows
- Mac
- Linux

Loading the starter project

We provide two versions of the codebase:

- A starter project, to download and follow along for this tutorial
- A final version of the project, for comparison and debugging.

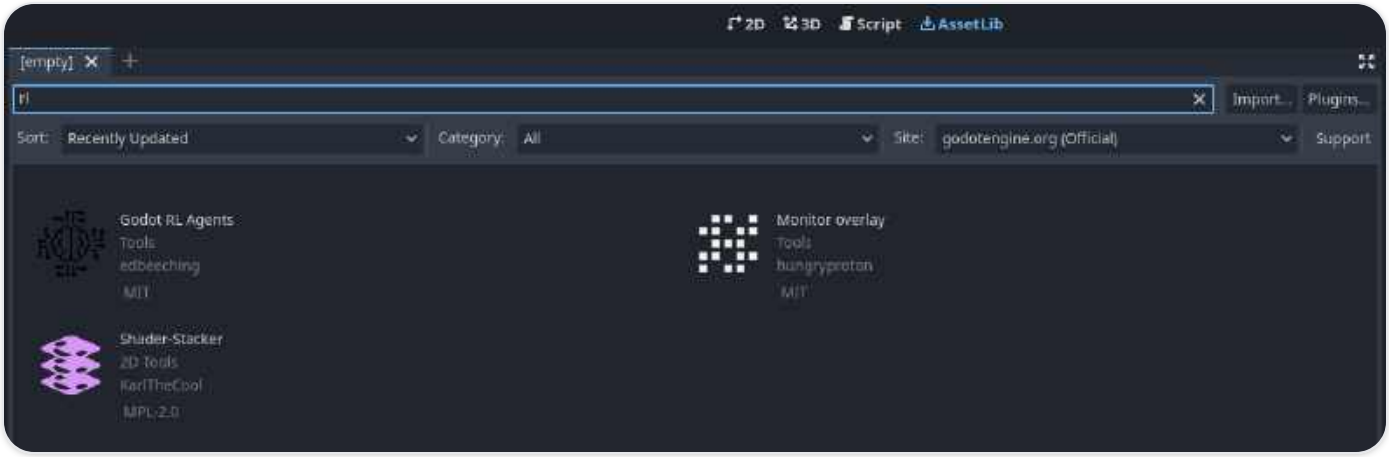
To load the project, in the Godot Project Manager click **Import**, navigate to where the files are located and load the **project.godot** file.

If you press F5 or play in the editor, you should be able to play the game in human mode. There are several instances of the game running, this is because we want to speed up training our AI agent with many parallel environments.

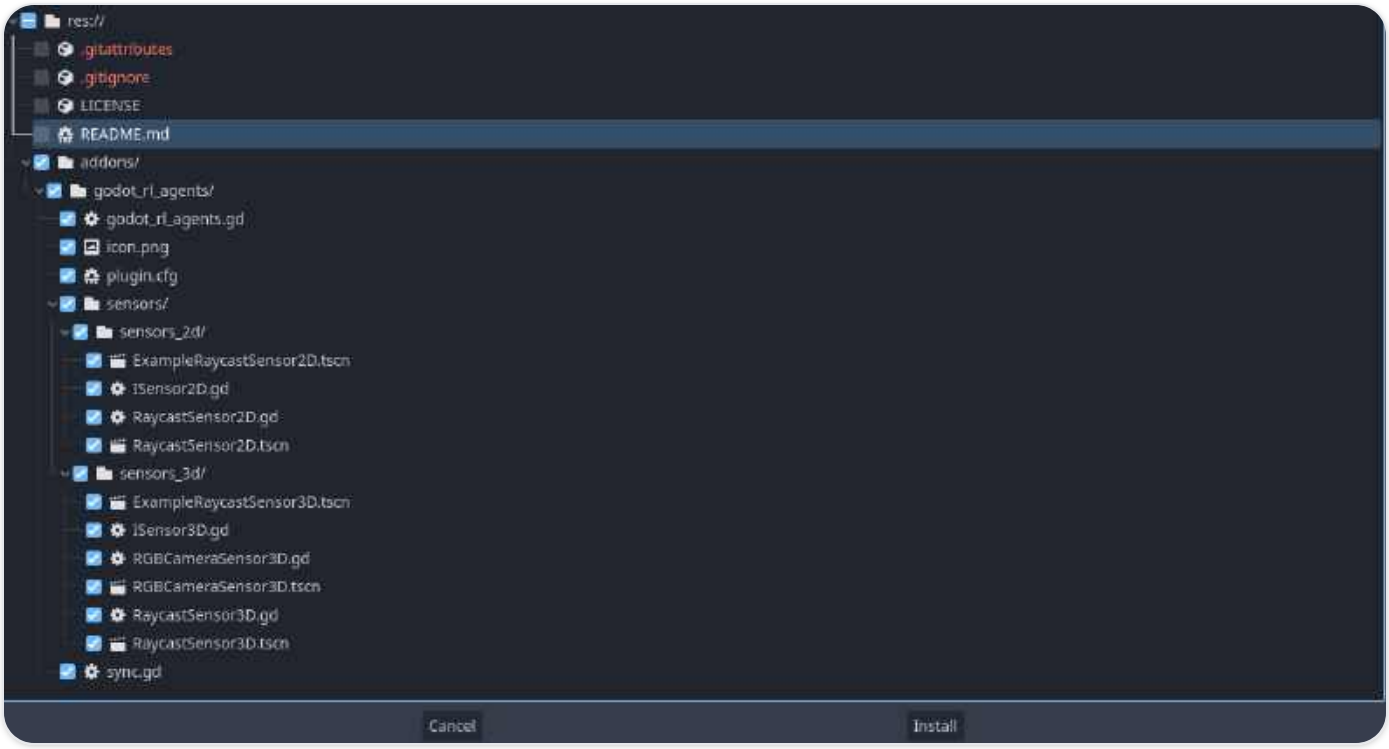
Installing the Godot RL Agents plugin

The Godot RL Agents plugin can be installed from the G
editor.

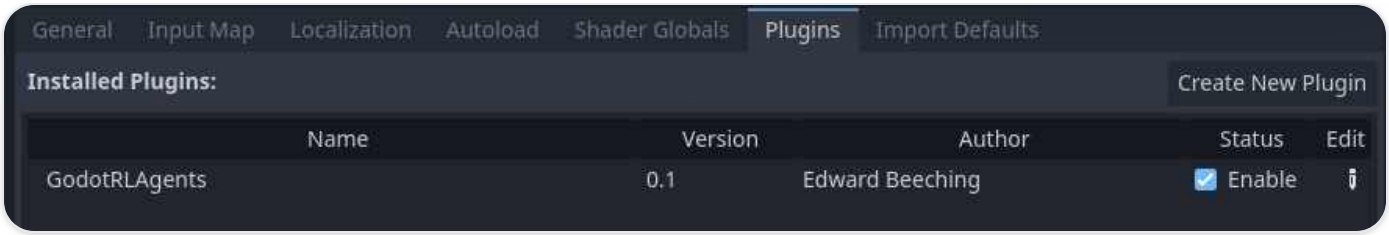
First click on the AssetLib and search for “rl”



Then click on Godot RL Agents, click Download and unselect the LICENSE and README .md files. Then click install.

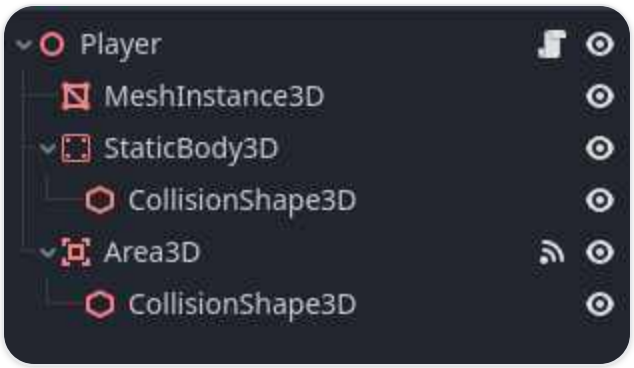


The Godot RL Agents plugin is now downloaded to your machine. Now click on Project → Project settings and enable the addon:

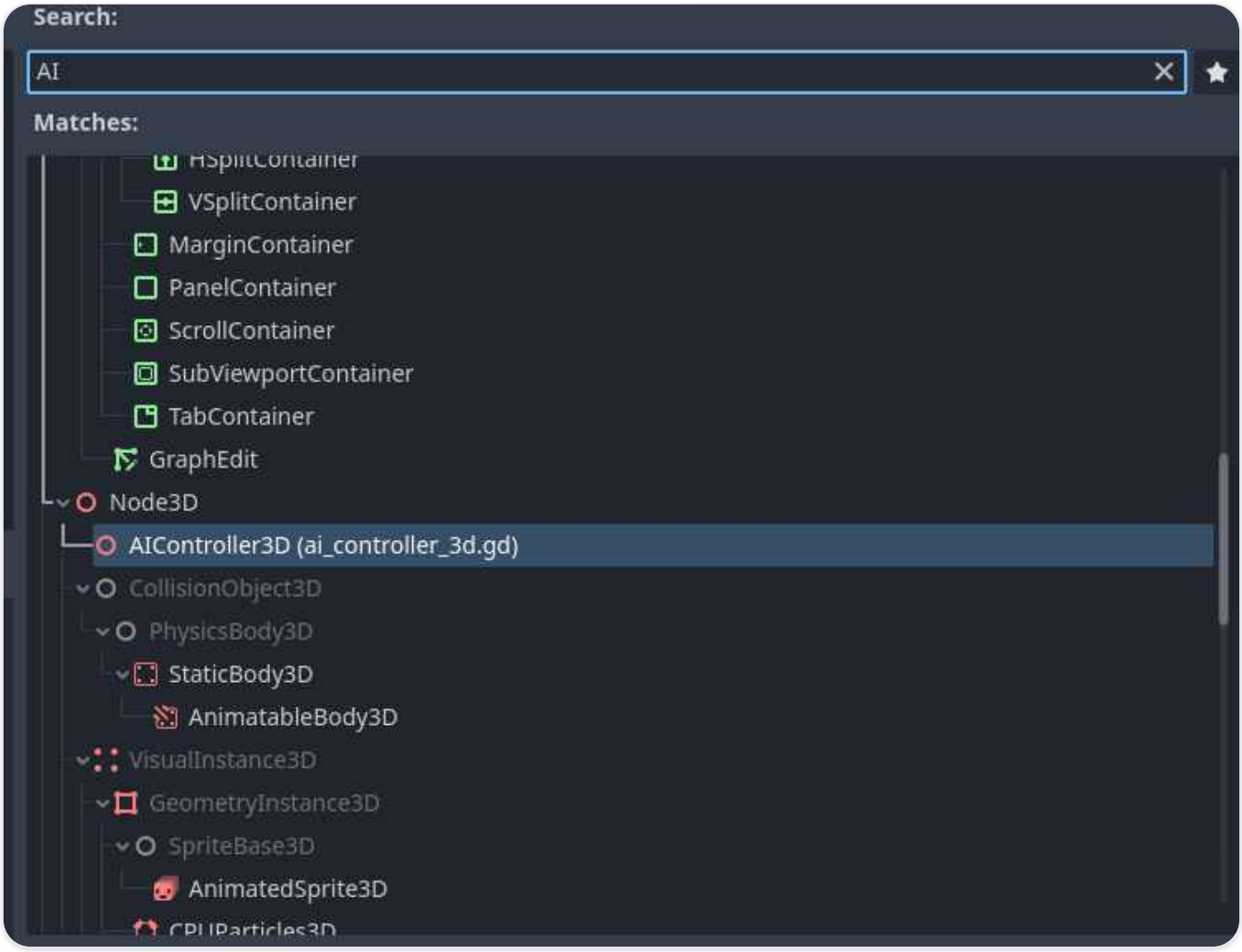


Adding the AI controller

We now want to add an AI controller to our game. Open the player.tscn scene, on the left you should see a hierarchy of nodes that looks like this:



Right click the **Player** node and click **Add Child Node.** There are many nodes listed here, search for AIController3D and create it.



The AI Controller Node should have been added to the scene tree, next to it is a scroll. Click on it to open the script that is attached to the AIController. The Godot game engine uses a scripting language called GDScript, which is syntactically similar to python. The script contains methods that need to be implemented in order to get our AI controller working.

```
#-- Methods that need implementing using the "extend script" option in Godot --#
func get_obs() -> Dictionary:
    assert(false, "the get_obs method is not implemented when extending from ai_controlle
    return {"obs":[]}

func get_reward() -> float:
    assert(false, "the get_reward method is not implemented when extending from ai_contro
    return 0.0

func get_action_space() -> Dictionary:
    assert(false, "the get get_action_space meth
```

```
    return {
        "example_actions_continous" : {
            "size": 2,
            "action_type": "continuous"
        },
        "example_actions_discrete" : {
            "size": 2,
            "action_type": "discrete"
        },
        }

func set_action(action) -> void:
    assert(false, "the get set_action method is not implemented when extending from ai_cc
# -----------------------------------------------------------------------------#
```

In order to implement these methods, we will need to create a class that inherits from AIController3D. This is easy to do in Godot, and is called “extending” a class.

Right click the AIController3D Node and click “Extend Script” and call the new script `controller.gd`. You should now have an almost empty script file that looks like this:

```
extends AIController3D

# Called when the node enters the scene tree for the first time.
func _ready():
    pass # Replace with function body.

# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta):
    pass
```

We will now implement the 4 missing methods, delete this code, and replace it with the following:

```
extends AIController3D

# Stores the action sampled for the agent's policy, running in python
var move_action : float = 0.0

func get_obs() -> Dictionary:
```

```
    # get the balls position and velocity in the paddle's frame of reference
    var ball_pos = to_local(_player.ball.global_position)
    var ball_vel = to_local(_player.ball.linear_velocity)
    var obs = [ball_pos.x, ball_pos.z, ball_vel.x/10.0, ball_vel.z/10.0]

    return {"obs":obs}

func get_reward() -> float:
    return reward

func get_action_space() -> Dictionary:
    return {
        "move_action" : {
            "size": 1,
            "action_type": "continuous"
        },
        }

func set_action(action) -> void:
    move_action = clamp(action["move_action"][0], -1.0, 1.0)
```

We have now defined the agent's observation, which is the position and velocity of the ball in its local coordinate space. We have also defined the action space of the agent, which is a single continuous value ranging from -1 to +1.

The next step is to update the Player's script to use the actions from the AIController, edit the Player's script by clicking on the scroll next to the player node, update the code in `Player.gd` to the following:

```
extends Node3D

@export var rotation_speed = 3.0
@onready var ball = get_node("../Ball")
@onready var ai_controller = $AIController3D

func _ready():
    ai_controller.init(self)

func game_over():
    ai_controller.done = true
```

```
    ai_controller.needs_reset = true

func _physics_process(delta):
    if ai_controller.needs_reset:
        ai_controller.reset()
        ball.reset()
        return

    var movement : float
    if ai_controller.heuristic == "human":
        movement = Input.get_axis("rotate_anticlockwise", "rotate_clockwise")
    else:
        movement = ai_controller.move_action
    rotate_y(movement*delta*rotation_speed)

func _on_area_3d_body_entered(body):
    ai_controller.reward += 1.0
```

We now need to synchronize between the game running in Godot and the neural network being trained in Python. Godot RL agents provides a node that does just that. Open the train.tscn scene, right click on the root node, and click “Add child node”. Then, search for “sync” and add a Godot RL Agents Sync node. This node handles the communication between Python and Godot over TCP.

You can run training live in the editor, by first launching the python training with `gdrl`.

In this simple example, a reasonable policy is learned in several minutes. You may wish to speed up training, click on the Sync node in the train scene, and you will see there is a “Speed Up” property exposed in the editor:



Try setting this property up to 8 to speed up training. This can be a great benefit on more complex environments, like the multi-player FPS we will learn about in the next chapter.

Export model

Reference doc

Let's put aside the Godot editor for now. We'll need to use terminals to run some commands in order to save models we trained.

The latest version of the Godot RL library provides experimental support for onnx models with the Stable Baselines 3, rllib, and CleanRL training frameworks.

For example, let's use the Stable Baselines 3 as the framework. Train your agent using the sb3 example (instructions for using the script), enabling the option `--onnx_export_path=model.onnx`

Below is an example command line to execute:

```
cd <....> # go into this Godot project directory
python stable_baselines3_example.py --timesteps=100_000 --onnx_export_path=model.onnx --s
```

If things work correctly, you should see messages printed out in the terminal like below:

```
No game binary has been provided, please press PLAY in the Godot editor
waiting for remote GODOT connection on port 11008
```

> *"If you encounter failures about import error in stable_baselines3_example script: "ImportError: cannot import name 'export_model_as_onnx' from 'godot_rl.wrappers.onnx.stable_baselines_export'", follow the answer in this issue here."*

Now it's time to switch back to the Godot editor, and hit PLAY on the top right corner. Once you hit that, the game scene will pop up showing AI training. In the meantime, the terminal will start to print out metrics. Wait for the training to finish, and if t
find the file `model.onnx` in the Godot project directory.

Apply AI in the game!

Now let's apply this trained model to the game!

In the Godot editor, find the Sync node in `train.tscn`:

- change the control mode to `Onnx Inference` from the dropdown
- set `Onnx Model Path` to the model file name, in our case here it's `model.onnx`

To run this game, we need the mono version (i.e., the .NET version) of the Godot editor, you can download it from the Godot official page. We need to install .NET as well.

Most likely you wil encounter errors in the first attempt. Below are the scenarios to help you resolve the errors.

1. issue about `Invalid Call. Nonexistent function 'new' in base 'CSharpScript'`: solution
2. errors about `onnxruntime` on MacOS: solution

There's more!

We have only scratched the surface of what can be achieved with Godot RL Agents, the library includes custom sensors and cameras to enrich the information available to the agent. Take a look at the examples to find out more!

For the ability to export the trained model to .onnx so that you can run inference directly from Godot without the Python server, and other useful training options, take a look at the advanced SB3 tutorial.

Author

This section was written by Edward Beeching

<> Update on GitHub

← An introduction to Unreal Learning Agents

Students projects →

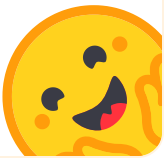
Search models, datasets, users...

Deep RL Course documentation
Hands-on

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Hands-on

Now that you learned the basics of multi-agents, you're ready to train your first agents in a multi-agent system: **a 2vs2 soccer team that needs to beat the opponent team**.

And you're going to participate in AI vs. AI challenges where your trained agent will compete against other classmates' **agents every day and be ranked on a new leaderboard.**

To validate this hands-on for the certification process, you just need to push a trained model. There **are no minimal results to attain to validate it.**

For more information about the certification process, check this section 👉
https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

This hands-on will be different since to get correct results **you need to train your agents from 4 hours to 8 hours**. And given the risk of timeout in Colab, we advise you to train on your computer. You don't need a supercomputer: a simple laptop is good enough for this exercise.

Let's get started! 🔥

What is AI vs. AI?

Ask HuggingChat

AI vs. AI is an open-source tool we developed at Hugging Face to compete agents on the Hub against one another in a multi-agent setting. These models are then ranked in a leaderboard.

The idea of this tool is to have a robust evaluation tool: **by evaluating your agent with a lot of others, you'll get a good idea of the quality of your policy.**

More precisely, AI vs. AI is three tools:

- A *matchmaking process* defining the matches (which model against which) and running the model fights using a background task in the Space.
- A *leaderboard* getting the match history results and displaying the models' ELO ratings: https://huggingface.co/spaces/huggingface-projects/AIvsAI-SoccerTwos
- A *Space demo* to visualize your agents playing against others: https://huggingface.co/spaces/unity/ML-Agents-SoccerTwos

In addition to these three tools, your classmate cyllum created a 🤗 SoccerTwos Challenge Analytics where you can check the detailed match results of a model: https://huggingface.co/spaces/cyllum/soccertwos-analytics

We're wrote a blog post to explain this AI vs. AI tool in detail, but to give you the big picture it works this way:

- Every four hours, our algorithm **fetches all the available models for a given environment (in our case ML-Agents-SoccerTwos).**
- It creates a **queue of matches with the matchmaking algorithm.**
- We simulate the match in a Unity headless process and **gather the match result** (1 if the first model won, 0.5 if it's a draw, 0 if the second model won) in a Dataset.
- Then, when all matches from the matches queue are done, **we update the ELO score for each model and update the leaderboard.**

Competition Rules

This first AI vs. AI competition **is an experiment**: the go
your feedback. So some **breakups can happen during the challenge**. But don't worry **all the**

results are saved in a dataset so we can always restart the calculation correctly without losing information.

In order for your model to get correctly evaluated against others you need to follow these rules:

1. **You can't change the observation space or action space of the agent.** By doing that your model will not work during evaluation.
2. You **can't use a custom trainer for now,** you need to use the Unity MLAgents ones.
3. We provide executables to train your agents. You can also use the Unity Editor if you prefer **, but to avoid bugs, we advise that you use our executables**.

What will make the difference during this challenge are **the hyperparameters you choose**.

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Chat with your classmates, share advice and ask questions on Discord

- We created a new channel called `ai-vs-ai-challenge` to exchange advice and ask questions.
- If you didn't join the discord server yet, you can join here

Step 0: Install MLAgents and download the correct executable

We advise you to use conda as a package manager and create a new environment.

With conda, we create a new environment called rl with **Python 3.10.12**:

```
conda create --name rl python=3.10.12
conda activate rl
```

To be able to train our agents correctly and push to the Hub, we need to install ML-Agents

```
git clone https://github.com/Unity-Technologies/
```

When the cloning is done (it takes 2.63 GB), we go inside the repository and install the package

```
cd ml-agents
pip install -e ./ml-agents-envs
pip install -e ./ml-agents
```

Mac users on Apple Silicon may encounter troubles with the installation (e.g. ONNX wheel build failing), you should first try to install grpcio:

```
conda install grpcio
```

This github issue in the official ml-agent repo might also help you.

Finally, you need to install git-lfs: https://git-lfs.com/

Now that it's installed, we need to add the environment training executable. Based on your operating system you need to download one of them, unzip it and place it in a new folder inside `ml-agents` that you call `training-envs-executables`

At the end your executable should be in `ml-agents/training-envs-executables/SoccerTwos`

Windows: Download this executable

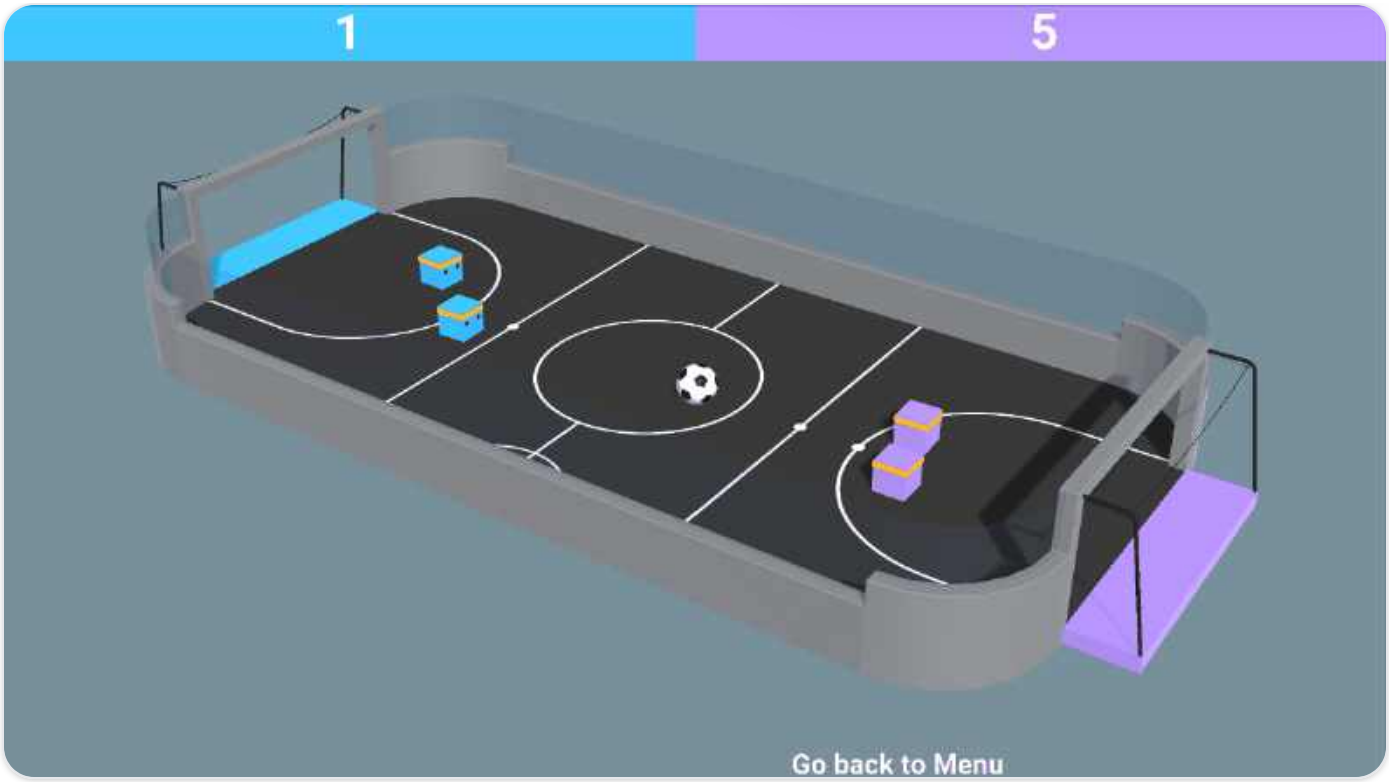
Linux (Ubuntu): Download this executable

Mac: Download this executable ⚠️ For Mac you need also to call this `xattr -cr training-envs-executables/SoccerTwos/SoccerTwos.app` to be able to run SoccerTwos

Step 1: Understand the environment

The environment is called `SoccerTwos`. The Unity MLAgents Team made it. You can find its documentation here

The goal in this environment **is to get the ball into the o** **from entering your own goal.**



This environment was made by the Unity MLAgents Team

The reward function

The reward function is:



The observation space

The observation space is composed of vectors of size 336:

- 11 ray-casts forward distributed over 120 degrees (264 state dimensions)
- 3 ray-casts backward distributed over 90 degrees (72 state dimensions)
- Both of these ray-casts can detect 6 objects:
    - Ball
    - Blue Goal
    - Purple Goal
    - Wall
    - Blue Agent
    - Purple Agent

The action space

The action space is three discrete branches:



Step 2: Understand MA-POCA

We know how to train agents to play against others: **we can use self-play.** This is a perfect technique for a 1vs1.

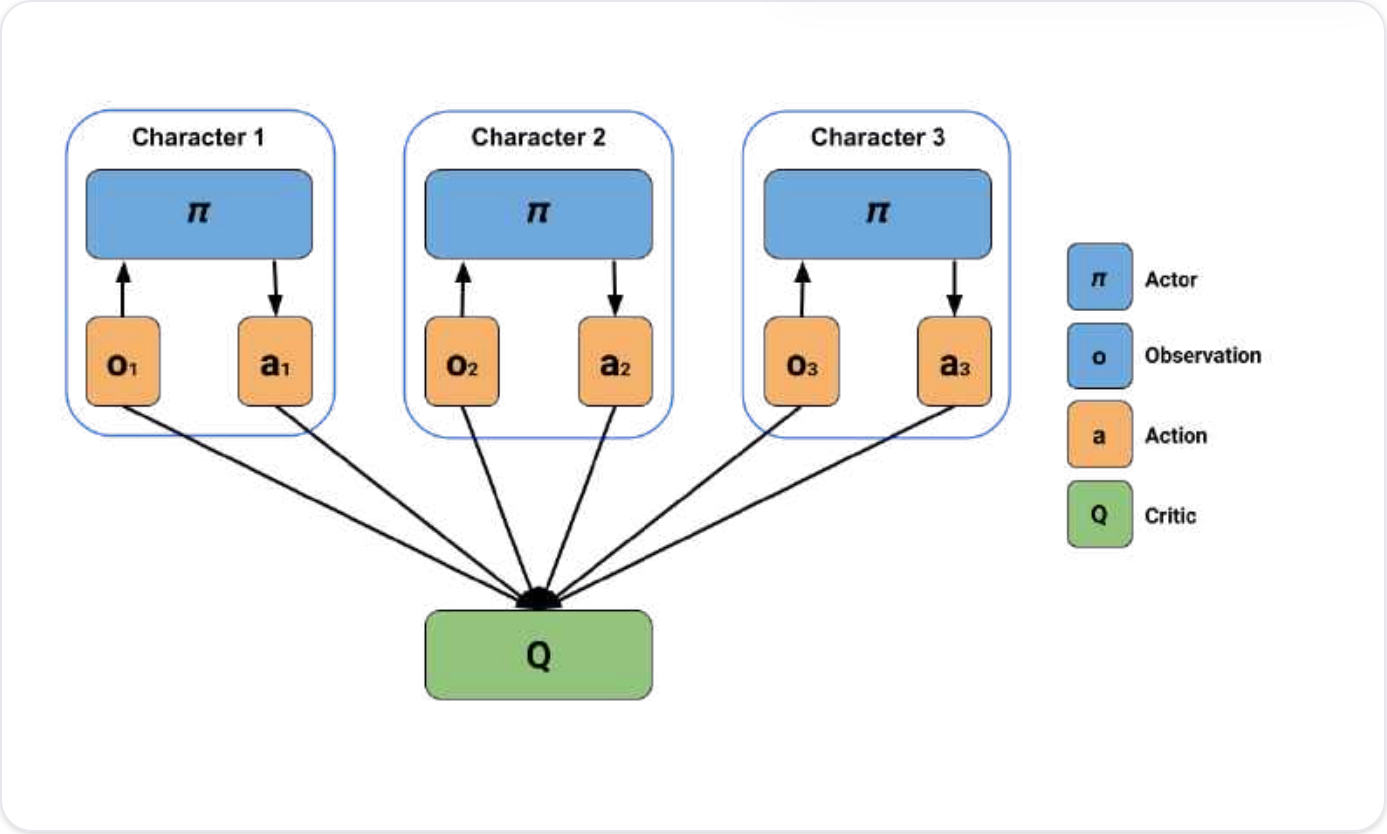
But in our case we're 2vs2, and each team has 2 agents. How then can we **train cooperative behavior for groups of agents?**

As explained in the Unity Blog, agents typically receive a reward as a group (+1 - penalty) when the team scores a goal. This implies that **every agent on the team is rewarded even if each agent didn't contribute the same to the win**, which makes it difficult to learn what to do independently.

The Unity MLAgents team developed the solution in a new multi-agent trainer called *MA-POCA (Multi-Agent POsthumous Credit Assignment)*.

The idea is simple but powerful: a centralized critic **processes the states of all agents in the team to estimate how well each agent is doing**. Think of this critic as a coach.

This allows each agent to **make decisions based only on what it perceives locally**, and **simultaneously evaluate how good its behavior is in the context of the whole group**.



This illustrates MA-POCA's centralized learning and decentralized execution. Source: MLAgents Plays Dodgeball

The solution then is to use Self-Play with an MA-POCA trainer (called poca). The poca trainer will help us to train cooperative behavior and self-play to win against an opponent team.

If you want to dive deeper into this MA-POCA algorithm, you need to read the paper they published here and the sources we put on the additional readings section.

Step 3: Define the config file

We already learned in Unit 5 that in ML-Agents, you define **the training hyperparameters in `config.yaml` files.**

There are multiple hyperparameters. To understand them better, you should read the explanations for each of them in the documentation

The config file we're going to use here is in `./config/poca/SoccerTwos.yaml`. It looks like this:

```
behaviors:
  SoccerTwos:
    trainer_type: poca
    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: constant
    network_settings:
      normalize: false
      hidden_units: 512
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
    keep_checkpoints: 5
    max_steps: 5000000
    time_horizon: 1000
    summary_freq: 10000
    self_play:
      save_steps: 50000
      team_change: 200000
      swap_steps: 2000
      window: 10
      play_against_latest_model_ratio: 0.5
      initial_elo: 1200.0
```

Compared to Pyramids or SnowballTarget, we have new hyperparameters with a self-play part. How you modify them can be critical in getting good results.

The advice I can give you here is to check the explanation and recommended value for each parameters (especially self-play ones) against the documentation.

Now that you've modified our config file, you're ready t

Step 4: Start the training

To train the agents, we need to **launch mlagents-learn and select the executable containing the environment.**

We define four parameters:

1. `mlagents-learn <config>`: the path where the hyperparameter config file is.
2. `-env`: where the environment executable is.
3. `-run_id`: the name you want to give to your training run id.
4. `-no-graphics`: to not launch the visualization during the training.

Depending on your hardware, 5M timesteps (the recommended value, but you can also try 10M) will take 5 to 8 hours of training. You can continue using your computer in the meantime, but I advise deactivating the computer standby mode to prevent the training from being stopped.

Depending on the executable you use (windows, ubuntu, mac) the training command will look like this (your executable path can be different so don't hesitate to check before running).

For Windows, it might look like this:

```
mlagents-learn ./config/poca/SoccerTwos.yaml --env=./training-envs-executables/SoccerTwos
```

For Mac, it might look like this:

```
mlagents-learn ./config/poca/SoccerTwos.yaml --env=./training-envs-executables/SoccerTwos
```

The executable contains 8 copies of SoccerTwos.

⚠️ It's normal if you don't see a big increase of ELO score (and even a decrease below 1200) before 2M timesteps, since your agents will spend most of their time moving randomly on the field before being able to goal.

⚠️ You can stop the training with Ctrl + C but beware of typing this command only once to stop the training since MLAgents needs to generate a final .onnx file before closing the run.

Step 5: Push the agent to the Hugging Face Hub

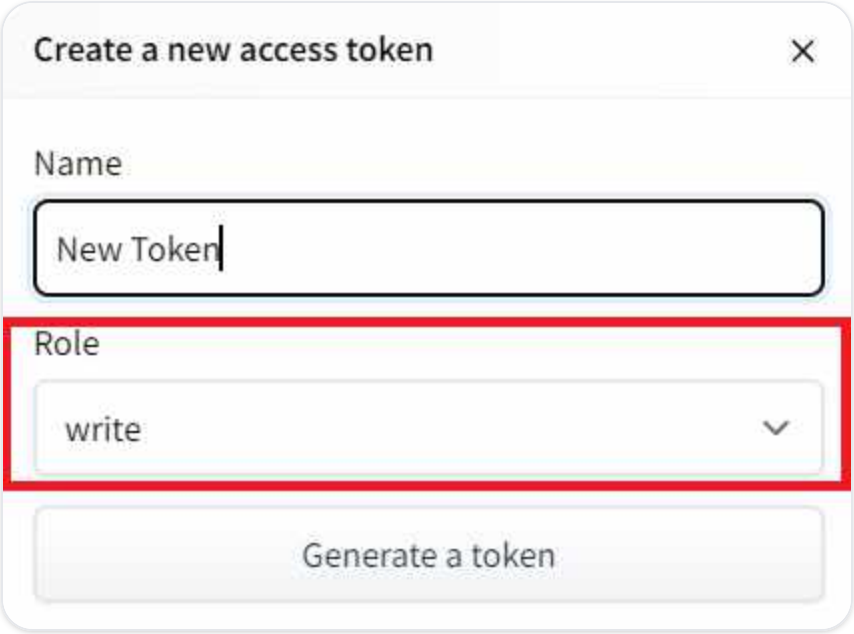
Now that we trained our agents, we're **ready to push them to the Hub to be able to participate in the AI vs. AI challenge and visualize them playing on your browser 🔥 .**

To be able to share your model with the community, there are three more steps to follow:

1️⃣ (If it's not already done) create an account to HF ➡️ https://huggingface.co/join

2️⃣ Sign in and store your authentication token from the Hugging Face website.

Create a new token (https://huggingface.co/settings/tokens) **with write role**



Copy the token, run this, and paste the token

```
huggingface-cli login
```

Then, we need to run `mlagents-push-to-hf`.

And we define four parameters:

1. `-run-id`: the name of the training run id.
2. `-local-dir`: where the agent was saved, it's results/<run_id name>, so in my case results/First Training.
3. `-repo-id`: the name of the Hugging Face repo you want to create or update. It's always <your huggingface username>/<the repo name> If the repo does not exist **it will be created automatically**
4. `--commit-message`: since HF repos are git repositories you need to give a commit message.

In my case

```
mlagents-push-to-hf  --run-id="SoccerTwos" --local-dir="./results/SoccerTwos" --repo-id="
```

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-i
```

If everything worked you should see this at the end of the process (but with a different url 😆) :

Your model is pushed to the Hub. You can view your model here: https://huggingface.co/ThomasSimonini/poca-SoccerTwos

It's the link to your model. It contains a model card that explains how to use it, your Tensorboard, and your config file. **What's awesome is that it's a git repository, which means you can have different commits, update your repository with a new push, etc.**

Step 6: Verify that your model is ready for AI vs AI Challenge

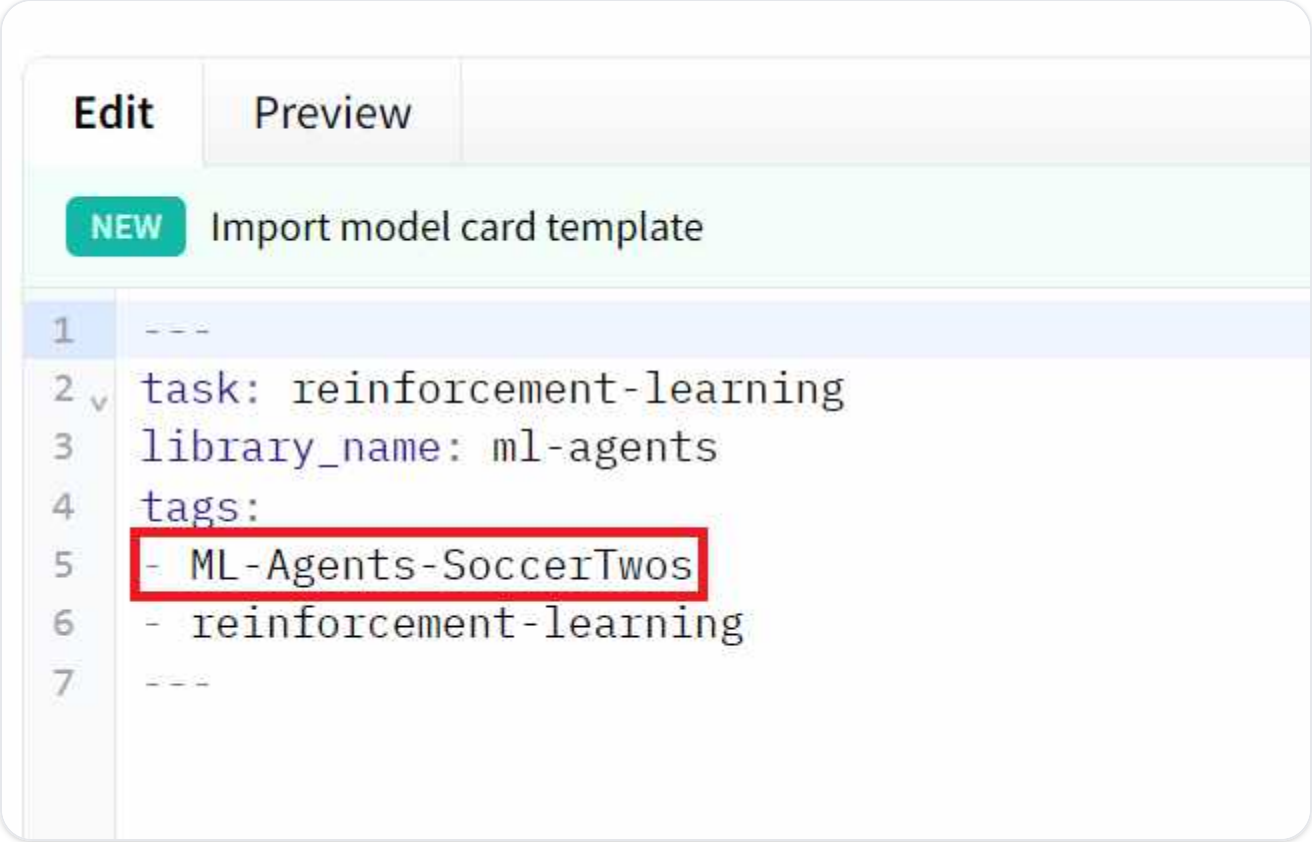
Now that your model is pushed to the Hub, **it's going to be added automatically to the AI vs AI Challenge model pool.** It can take a little bit of time before your model is added to the leaderboard given we do a run of matches every 4h.

But to ensure that everything works perfectly you need to check:

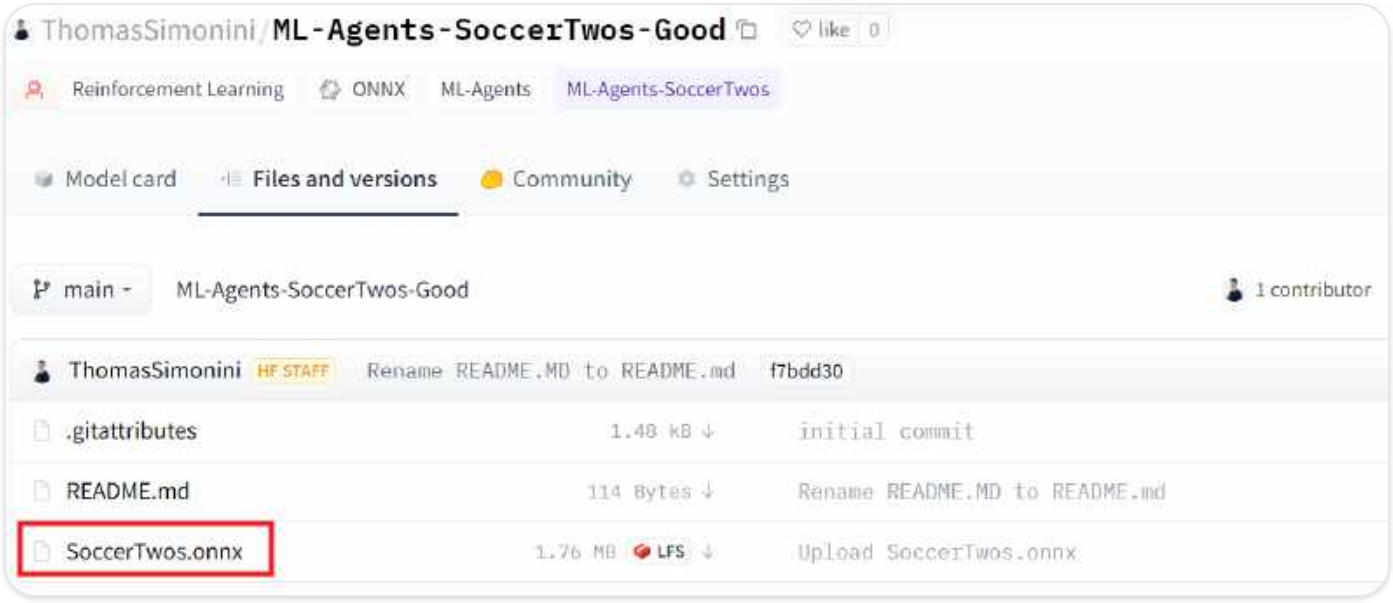
1. That you have this tag in your model: ML-Agents-SoccerTwos. This is the tag we use to select models to be added to the challenge pool. To do that go to your model and check the tags



If it's not the case you just need to modify the readme and add it



2. That you have a `SoccerTwos.onnx` file



We strongly suggest that you create a new model when you push to the Hub if you want to train it again or train a new version.

Step 7: Visualize some match in our demo

Now that your model is part of AI vs AI Challenge, **you can visualize how good it is compared to others**: https://huggingface.co/spaces/unity/ML-Agents-SoccerTwos

In order to do that, you just need to go to this demo:

- Select your model as team blue (or team purple if you prefer) and another model to compete against. The best opponents to compare your model to are either whoever is on top of the leaderboard or the baseline model

The matches you see live are not used in the calculation of your result **but they are a good way to visualize how good your agent is**.

And don't hesitate to share the best score your agent gets on discord in the #rl-i-made-this channel 🔥

<> Update on GitHub

← Self-Play

Quiz →

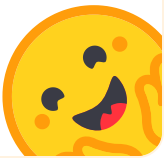
Search models, datasets, users...

Deep RL Course documentation
Hands-on

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Hands-on

Ask a question Open in Colab

Now that we studied the theory behind PPO, the best way to understand how it works **is to implement it from scratch.**

Implementing an architecture from scratch is the best way to understand it, and it's a good habit. We have already done it for a value-based method with Q-Learning and a Policy-based method with Reinforce.

So, to be able to code it, we're going to use two resources:

- A tutorial made by Costa Huang. Costa is behind CleanRL, a Deep Reinforcement Learning library that provides high-quality single-file implementation with research-friendly features.
- In addition to the tutorial, to go deeper, you can read the 13 core implementation details: https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/

Then, to test its robustness, we're going to train it in:

- LunarLander-v2

Ask HuggingChat



And finally, we will push the trained model to the Hub to evaluate and visualize your agent playing.

LunarLander-v2 is the first environment you used when you started this course. At that time, you didn’t know how it worked, and now you can code it from scratch and train it. **How incredible is that 🤩.**

via GIPHY

Let’s get started! 🚀

The colab notebook:

Open in Colab

Unit 8: Proximal Policy Gradient (PPO) with PyTorch



In this notebook, you'll learn to **code your PPO agent from scratch with PyTorch using CleanRL implementation as model**.

To test its robustness, we're going to train it in:

- LunarLander-v2

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Objectives of this notebook

At the end of the notebook, you will:

- Be able to **code your PPO agent from scratch using PyTorch**.
- Be able to **push your trained agent and the code to the Hub** with a nice video replay and an evaluation score 🔥.

Prerequisites 🏗️

Before diving into the notebook, you need to:

🔲 📚 Study PPO by reading Unit 8 🤗

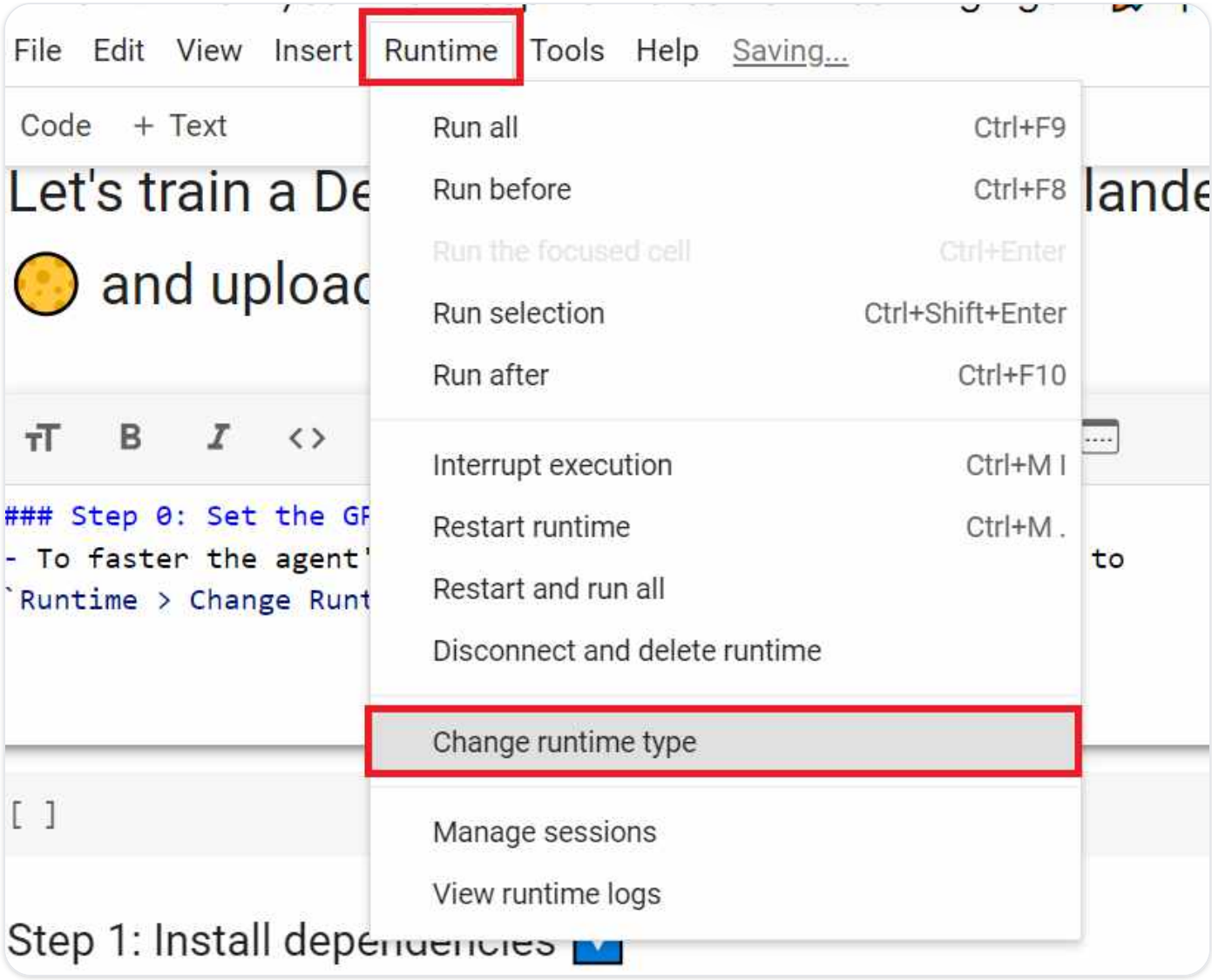
To validate this hands-on for the certification process, you need to push one model, we don't ask for a minimal result but we **advise you to try different hyperparameters settings to get better results**.

If you don't find your model, **go to the bottom of the page and click on the refresh button**

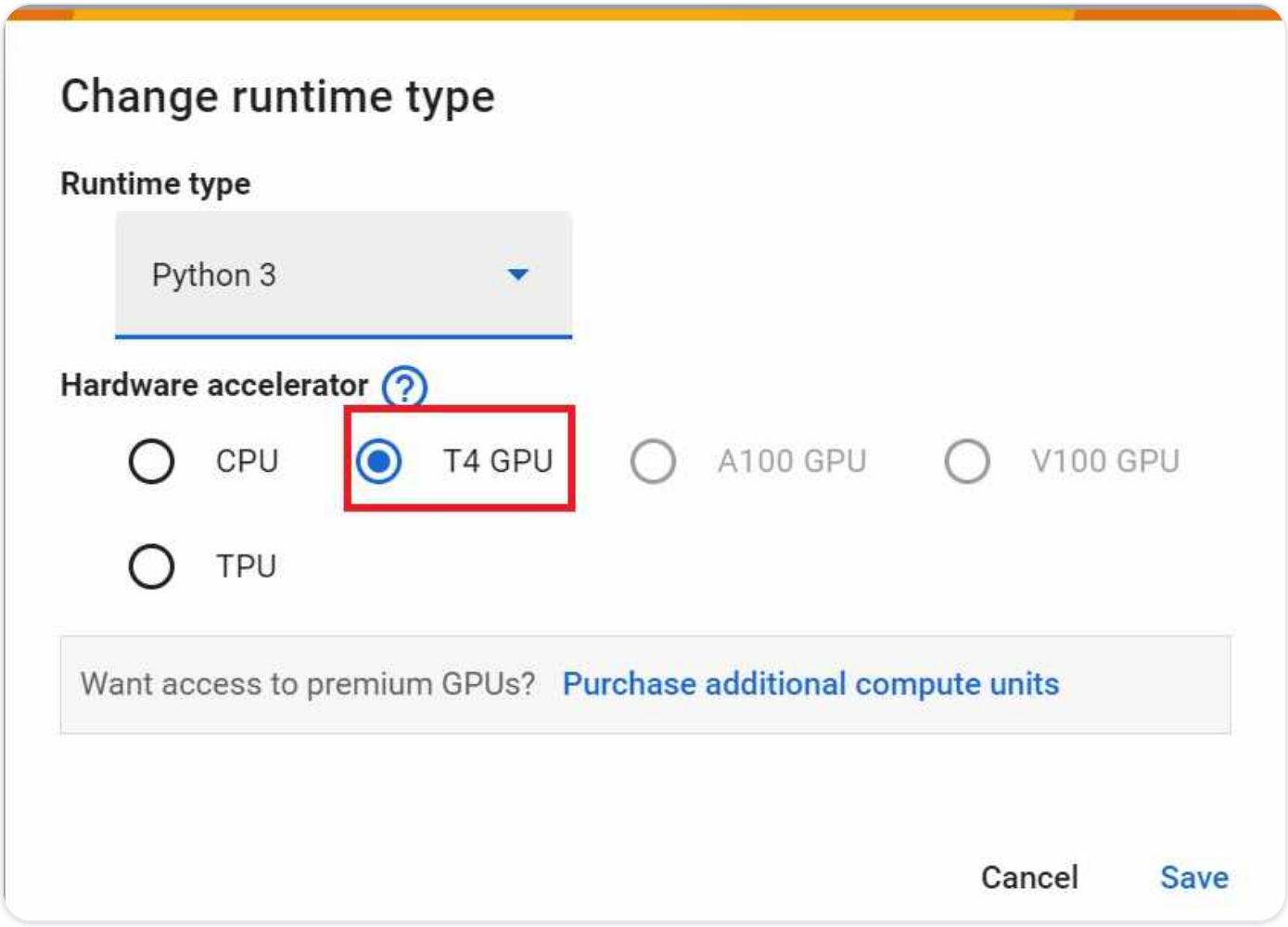
For more information about the certification process, check this section 👉 https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

Set the GPU 💪

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`



- `Hardware Accelerator > GPU`



Create a virtual display 🔽

During the notebook, we'll need to generate a replay video. To do so, with colab, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

Hence the following cell will install the librairies and create and run a virtual screen 🖥️

```
apt install python-opengl
apt install ffmpeg
apt install xvfb
pip install pyglet==1.5
pip install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display
```

```
virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Install dependencies 🔽

For this exercise, we use `gym==0.21` because the video was recorded with Gym.

```
pip install gym==0.22
pip install imageio-ffmpeg
pip install huggingface_hub
pip install gym[box2d]==0.22
```

Let's code PPO from scratch with Costa Huang's tutorial

- For the core implementation of PPO we're going to use the excellent Costa Huang tutorial.
- In addition to the tutorial, to go deeper you can read the 37 core implementation details: https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/

👉 The video tutorial: https://youtu.be/MEt6rrxH8W4

```
from IPython.display import HTML

HTML(
    '<iframe width="560" height="315" src="https://www.youtube.com/embed/MEt6rrxH8W4" tit
)
```

Add the Hugging Face Integration 🤗

- In order to push our model to the Hub, we need to define a function `package_to_hub`
- Add dependencies we need to push our model to the Hub

```
from huggingface_hub import HfApi, upload_folder
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import tempfile
import json
import shutil
import imageio

from wasabi import Printer

msg = Printer()
```

- Add new argument in `parse_args()` function to define the repo-id where we want to push the model.

```
# Adding HuggingFace argument
parser.add_argument(
    "--repo-id",
    type=str,
    default="ThomasSimonini/ppo-CartPole-v1",
    help="id of the model repository from the Hugging Face Hub {username/repo_name}",
)
```

- Next, we add the methods needed to push the model to the Hub
- These methods will:
  - `_evalutate_agent()` : evaluate the agent.
  - `_generate_model_card()` : generate the model card of your agent.
  - `_record_video()` : record a video of your agent.

```
def package_to_hub(
    repo_id,
    model,
    hyperparameters,
    eval_env,
```

```
    video_fps=30,
    commit_message="Push agent to the Hub",
    token=None,
    logs=None,
):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the hub
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param model: trained model
    :param eval_env: environment used to evaluate the agent
    :param fps: number of fps for rendering the video
    :param commit_message: commit message
    :param logs: directory on local machine of tensorboard logs you'd like to upload
    """
    msg.info(
        "This function will save, evaluate, generate a video of your agent, "
        "create a model card and push everything to the hub. "
        "It might take up to 1min. \n "
        "This is a work in progress: if you encounter a bug, please open an issue."
    )
    # Step 1: Clone or create the repo
    repo_url = HfApi().create_repo(
        repo_id=repo_id,
        token=token,
        private=False,
        exist_ok=True,
    )

    with tempfile.TemporaryDirectory() as tmpdirname:
        tmpdirname = Path(tmpdirname)

        # Step 2: Save the model
        torch.save(model.state_dict(), tmpdirname / "model.pt")

        # Step 3: Evaluate the model and build JSON
        mean_reward, std_reward = _evaluate_agent(eval_env, 10, model)

        # First get datetime
```

```
        eval_datetime = datetime.datetime.now()
        eval_form_datetime = eval_datetime.isoformat()

        evaluate_data = {
            "env_id": hyperparameters.env_id,
            "mean_reward": mean_reward,
            "std_reward": std_reward,
            "n_evaluation_episodes": 10,
            "eval_datetime": eval_form_datetime,
        }

        # Write a JSON file
        with open(tmpdirname / "results.json", "w") as outfile:
            json.dump(evaluate_data, outfile)

        # Step 4: Generate a video
        video_path = tmpdirname / "replay.mp4"
        record_video(eval_env, model, video_path, video_fps)

        # Step 5: Generate the model card
        generated_model_card, metadata = _generate_model_card(
            "PPO", hyperparameters.env_id, mean_reward, std_reward, hyperparameters
        )
        _save_model_card(tmpdirname, generated_model_card, metadata)

        # Step 6: Add logs if needed
        if logs:
            _add_logdir(tmpdirname, Path(logs))

        msg.info(f"Pushing repo {repo_id} to the Hugging Face Hub")

        repo_url = upload_folder(
            repo_id=repo_id,
            folder_path=tmpdirname,
            path_in_repo="",
            commit_message=commit_message,
            token=token,
        )

        msg.info(f"Your model is pushed to the Hub. You can view your model here: {repo_u
    return repo_url
```

```
def _evaluate_agent(env, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and st
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The agent
    """
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        while done is False:
            state = torch.Tensor(state).to(device)
            action, _, _, _ = policy.get_action_and_value(state)
            new_state, reward, done, info = env.step(action.cpu().numpy())
            total_rewards_ep += reward
            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward


def record_video(env, policy, out_directory, fps=30):
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        state = torch.Tensor(state).to(device)
        # Take the action (index) that have the maximum expected future reward given that
        action, _, _, _ = policy.get_action_and_value(state)
        state, reward, done, info = env.step(
            action.cpu().numpy()
        )  # We directly put next_state = state
        img = env.render(mode="rgb_array")
```

```
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=f


def _generate_model_card(model_name, env_id, mean_reward, std_reward, hyperparameters):
    """
    Generate the model card for the Hub
    :param model_name: name of the model
    :env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    :hyperparameters: training arguments
    """
    # Step 1: Select the tags
    metadata = generate_metadata(model_name, env_id, mean_reward, std_reward)

    # Transform the hyperparams namespace to string
    converted_dict = vars(hyperparameters)
    converted_str = str(converted_dict)
    converted_str = converted_str.split(", ")
    converted_str = "\n".join(converted_str)

    # Step 2: Generate the model card
    model_card = f"""
  # PPO Agent Playing {env_id}

  This is a trained model of a PPO agent playing {env_id}.

  # Hyperparameters
  """
    return model_card, metadata


def generate_metadata(model_name, env_id, mean_reward, std_reward):
    """
    Define the tags for the model card
    :param model_name: name of the model
    :param env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    """
    metadata = {}
    metadata["tags"] = [
```

```
        env_id,
        "ppo",
        "deep-reinforcement-learning",
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-course",
    ]

    # Add metrics
    eval = metadata_eval_result(
        model_pretty_name=model_name,
        task_pretty_name="reinforcement-learning",
        task_id="reinforcement-learning",
        metrics_pretty_name="mean_reward",
        metrics_id="mean_reward",
        metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
        dataset_pretty_name=env_id,
        dataset_id=env_id,
    )

    # Merges both dictionaries
    metadata = {**metadata, **eval}

    return metadata


def _save_model_card(local_path, generated_model_card, metadata):
    """Saves a model card for the repository.
    :param local_path: repository directory
    :param generated_model_card: model card generated by _generate_model_card()
    :param metadata: metadata
    """
    readme_path = local_path / "README.md"
    readme = ""
    if readme_path.exists():
        with readme_path.open("r", encoding="utf8") as f:
            readme = f.read()
    else:
        readme = generated_model_card

    with readme_path.open("w", encoding="utf-8") as f:
        f.write(readme)
```

```
    # Save our metrics to Readme metadata
    metadata_save(readme_path, metadata)


def _add_logdir(local_path: Path, logdir: Path):
    """Adds a logdir to the repository.
    :param local_path: repository directory
    :param logdir: logdir directory
    """
    if logdir.exists() and logdir.is_dir():
        # Add the logdir to the repository under new dir called logs
        repo_logdir = local_path / "logs"

        # Delete current logs if they exist
        if repo_logdir.exists():
            shutil.rmtree(repo_logdir)

        # Copy logdir into repo logdir
        shutil.copytree(logdir, repo_logdir)
```

- Finally, we call this function at the end of the PPO training

```
# Create the evaluation environment
eval_env = gym.make(args.env_id)

package_to_hub(
    repo_id=args.repo_id,
    model=agent,  # The model we want to save
    hyperparameters=args,
    eval_env=gym.make(args.env_id),
    logs=f"runs/{run_name}",
)
```

- Here's what the final ppo.py file looks like:

```
# docs and experiment results can be found at https://docs.cleanrl.dev/rl-algorithms/ppo/

import argparse
import os
```

```
import random
import time
from distutils.util import strtobool

import gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions.categorical import Categorical
from torch.utils.tensorboard import SummaryWriter

from huggingface_hub import HfApi, upload_folder
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import tempfile
import json
import shutil
import imageio

from wasabi import Printer

msg = Printer()


def parse_args():
    # fmt: off
    parser = argparse.ArgumentParser()
    parser.add_argument("--exp-name", type=str, default=os.path.basename(__file__).rstrip
        help="the name of this experiment")
    parser.add_argument("--seed", type=int, default=1,
        help="seed of the experiment")
    parser.add_argument("--torch-deterministic", type=lambda x: bool(strtobool(x)), defau
        help="if toggled, `torch.backends.cudnn.deterministic=False`")
    parser.add_argument("--cuda", type=lambda x: bool(strtobool(x)), default=True, nargs=
        help="if toggled, cuda will be enabled by default")
    parser.add_argument("--track", type=lambda x: bool(strtobool(x)), default=False, narg
        help="if toggled, this experiment will be tracked with Weights and Biases")
    parser.add_argument("--wandb-project-name", type=str, default="cleanRL",
        help="the wandb's project name")
    parser.add_argument("--wandb-entity", type=s
```

```
        help="the entity (team) of wandb's project")
    parser.add_argument("--capture-video", type=lambda x: bool(strtobool(x)), default=Fal
        help="weather to capture videos of the agent performances (check out `videos` fol

    # Algorithm specific arguments
    parser.add_argument("--env-id", type=str, default="CartPole-v1",
        help="the id of the environment")
    parser.add_argument("--total-timesteps", type=int, default=50000,
        help="total timesteps of the experiments")
    parser.add_argument("--learning-rate", type=float, default=2.5e-4,
        help="the learning rate of the optimizer")
    parser.add_argument("--num-envs", type=int, default=4,
        help="the number of parallel game environments")
    parser.add_argument("--num-steps", type=int, default=128,
        help="the number of steps to run in each environment per policy rollout")
    parser.add_argument("--anneal-lr", type=lambda x: bool(strtobool(x)), default=True, n
        help="Toggle learning rate annealing for policy and value networks")
    parser.add_argument("--gae", type=lambda x: bool(strtobool(x)), default=True, nargs="
        help="Use GAE for advantage computation")
    parser.add_argument("--gamma", type=float, default=0.99,
        help="the discount factor gamma")
    parser.add_argument("--gae-lambda", type=float, default=0.95,
        help="the lambda for the general advantage estimation")
    parser.add_argument("--num-minibatches", type=int, default=4,
        help="the number of mini-batches")
    parser.add_argument("--update-epochs", type=int, default=4,
        help="the K epochs to update the policy")
    parser.add_argument("--norm-adv", type=lambda x: bool(strtobool(x)), default=True, na
        help="Toggles advantages normalization")
    parser.add_argument("--clip-coef", type=float, default=0.2,
        help="the surrogate clipping coefficient")
    parser.add_argument("--clip-vloss", type=lambda x: bool(strtobool(x)), default=True,
        help="Toggles whether or not to use a clipped loss for the value function, as per
    parser.add_argument("--ent-coef", type=float, default=0.01,
        help="coefficient of the entropy")
    parser.add_argument("--vf-coef", type=float, default=0.5,
        help="coefficient of the value function")
    parser.add_argument("--max-grad-norm", type=float, default=0.5,
        help="the maximum norm for the gradient clipping")
    parser.add_argument("--target-kl", type=float, default=None,
        help="the target KL divergence threshold")

    # Adding HuggingFace argument
```

```
    parser.add_argument("--repo-id", type=str, default="ThomasSimonini/ppo-CartPole-v1",

    args = parser.parse_args()
    args.batch_size = int(args.num_envs * args.num_steps)
    args.minibatch_size = int(args.batch_size // args.num_minibatches)
    # fmt: on
    return args


def package_to_hub(
    repo_id,
    model,
    hyperparameters,
    eval_env,
    video_fps=30,
    commit_message="Push agent to the Hub",
    token=None,
    logs=None,
):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the hub
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param model: trained model
    :param eval_env: environment used to evaluate the agent
    :param fps: number of fps for rendering the video
    :param commit_message: commit message
    :param logs: directory on local machine of tensorboard logs you'd like to upload
    """
    msg.info(
        "This function will save, evaluate, generate a video of your agent, "
        "create a model card and push everything to the hub. "
        "It might take up to 1min. \n "
        "This is a work in progress: if you encounter a bug, please open an issue."
    )
    # Step 1: Clone or create the repo
    repo_url = HfApi().create_repo(
        repo_id=repo_id,
        token=token,
```

```
        private=False,
        exist_ok=True,
    )

    with tempfile.TemporaryDirectory() as tmpdirname:
        tmpdirname = Path(tmpdirname)

        # Step 2: Save the model
        torch.save(model.state_dict(), tmpdirname / "model.pt")

        # Step 3: Evaluate the model and build JSON
        mean_reward, std_reward = _evaluate_agent(eval_env, 10, model)

        # First get datetime
        eval_datetime = datetime.datetime.now()
        eval_form_datetime = eval_datetime.isoformat()

        evaluate_data = {
            "env_id": hyperparameters.env_id,
            "mean_reward": mean_reward,
            "std_reward": std_reward,
            "n_evaluation_episodes": 10,
            "eval_datetime": eval_form_datetime,
        }

        # Write a JSON file
        with open(tmpdirname / "results.json", "w") as outfile:
            json.dump(evaluate_data, outfile)

        # Step 4: Generate a video
        video_path = tmpdirname / "replay.mp4"
        record_video(eval_env, model, video_path, video_fps)

        # Step 5: Generate the model card
        generated_model_card, metadata = _generate_model_card(
            "PPO", hyperparameters.env_id, mean_reward, std_reward, hyperparameters
        )
        _save_model_card(tmpdirname, generated_model_card, metadata)

        # Step 6: Add logs if needed
        if logs:
            _add_logdir(tmpdirname, Path(logs))
```

```
        msg.info(f"Pushing repo {repo_id} to the Hugging Face Hub")

        repo_url = upload_folder(
            repo_id=repo_id,
            folder_path=tmpdirname,
            path_in_repo="",
            commit_message=commit_message,
            token=token,
        )

        msg.info(f"Your model is pushed to the Hub. You can view your model here: {repo_u
    return repo_url


def _evaluate_agent(env, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and st
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The agent
    """
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        while done is False:
            state = torch.Tensor(state).to(device)
            action, _, _, _ = policy.get_action_and_value(state)
            new_state, reward, done, info = env.step(action.cpu().numpy())
            total_rewards_ep += reward
            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward
```

```
def record_video(env, policy, out_directory, fps=30):
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        state = torch.Tensor(state).to(device)
        # Take the action (index) that have the maximum expected future reward given that
        action, _, _, _ = policy.get_action_and_value(state)
        state, reward, done, info = env.step(
            action.cpu().numpy()
        )  # We directly put next_state = state for recording logic
        img = env.render(mode="rgb_array")
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=f


def _generate_model_card(model_name, env_id, mean_reward, std_reward, hyperparameters):
    """
    Generate the model card for the Hub
    :param model_name: name of the model
    :env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    :hyperparameters: training arguments
    """
    # Step 1: Select the tags
    metadata = generate_metadata(model_name, env_id, mean_reward, std_reward)

    # Transform the hyperparams namespace to string
    converted_dict = vars(hyperparameters)
    converted_str = str(converted_dict)
    converted_str = converted_str.split(", ")
    converted_str = "\n".join(converted_str)

    # Step 2: Generate the model card
    model_card = f"""
  # PPO Agent Playing {env_id}

  This is a trained model of a PPO agent playing {env_id}.

  # Hyperparameters
```

```
    """
    return model_card, metadata


def generate_metadata(model_name, env_id, mean_reward, std_reward):
    """
    Define the tags for the model card
    :param model_name: name of the model
    :param env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    """
    metadata = {}
    metadata["tags"] = [
        env_id,
        "ppo",
        "deep-reinforcement-learning",
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-course",
    ]

    # Add metrics
    eval = metadata_eval_result(
        model_pretty_name=model_name,
        task_pretty_name="reinforcement-learning",
        task_id="reinforcement-learning",
        metrics_pretty_name="mean_reward",
        metrics_id="mean_reward",
        metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
        dataset_pretty_name=env_id,
        dataset_id=env_id,
    )

    # Merges both dictionaries
    metadata = {**metadata, **eval}

    return metadata


def _save_model_card(local_path, generated_model_card, metadata):
    """Saves a model card for the repository.
    :param local_path: repository directory
```

```
    :param generated_model_card: model card generated by _generate_model_card()
    :param metadata: metadata
    """
    readme_path = local_path / "README.md"
    readme = ""
    if readme_path.exists():
        with readme_path.open("r", encoding="utf8") as f:
            readme = f.read()
    else:
        readme = generated_model_card

    with readme_path.open("w", encoding="utf-8") as f:
        f.write(readme)

    # Save our metrics to Readme metadata
    metadata_save(readme_path, metadata)


def _add_logdir(local_path: Path, logdir: Path):
    """Adds a logdir to the repository.
    :param local_path: repository directory
    :param logdir: logdir directory
    """
    if logdir.exists() and logdir.is_dir():
        # Add the logdir to the repository under new dir called logs
        repo_logdir = local_path / "logs"

        # Delete current logs if they exist
        if repo_logdir.exists():
            shutil.rmtree(repo_logdir)

        # Copy logdir into repo logdir
        shutil.copytree(logdir, repo_logdir)


def make_env(env_id, seed, idx, capture_video, run_name):
    def thunk():
        env = gym.make(env_id)
        env = gym.wrappers.RecordEpisodeStatistics(env)
        if capture_video:
            if idx == 0:
                env = gym.wrappers.RecordVideo(e
        env.seed(seed)
```

```
        env.action_space.seed(seed)
        env.observation_space.seed(seed)
        return env

    return thunk


def layer_init(layer, std=np.sqrt(2), bias_const=0.0):
    torch.nn.init.orthogonal_(layer.weight, std)
    torch.nn.init.constant_(layer.bias, bias_const)
    return layer


class Agent(nn.Module):
    def __init__(self, envs):
        super().__init__()
        self.critic = nn.Sequential(
            layer_init(nn.Linear(np.array(envs.single_observation_space.shape).prod(), 64
            nn.Tanh(),
            layer_init(nn.Linear(64, 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, 1), std=1.0),
        )
        self.actor = nn.Sequential(
            layer_init(nn.Linear(np.array(envs.single_observation_space.shape).prod(), 64
            nn.Tanh(),
            layer_init(nn.Linear(64, 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, envs.single_action_space.n), std=0.01),
        )

    def get_value(self, x):
        return self.critic(x)

    def get_action_and_value(self, x, action=None):
        logits = self.actor(x)
        probs = Categorical(logits=logits)
        if action is None:
            action = probs.sample()
        return action, probs.log_prob(action), probs.entropy(), self.critic(x)


if __name__ == "__main__":
```

```
    args = parse_args()
    run_name = f"{args.env_id}__{args.exp_name}__{args.seed}__{int(time.time())}"
    if args.track:
        import wandb

        wandb.init(
            project=args.wandb_project_name,
            entity=args.wandb_entity,
            sync_tensorboard=True,
            config=vars(args),
            name=run_name,
            monitor_gym=True,
            save_code=True,
        )
    writer = SummaryWriter(f"runs/{run_name}")
    writer.add_text(
        "hyperparameters",
        "|param|value|\n|-|-|\n%s" % ("\n".join([f"|{key}|{value}|" for key, value in var
    )

    # TRY NOT TO MODIFY: seeding
    random.seed(args.seed)
    np.random.seed(args.seed)
    torch.manual_seed(args.seed)
    torch.backends.cudnn.deterministic = args.torch_deterministic

    device = torch.device("cuda" if torch.cuda.is_available() and args.cuda else "cpu")

    # env setup
    envs = gym.vector.SyncVectorEnv(
        [make_env(args.env_id, args.seed + i, i, args.capture_video, run_name) for i in r
    )
    assert isinstance(envs.single_action_space, gym.spaces.Discrete), "only discrete acti

    agent = Agent(envs).to(device)
    optimizer = optim.Adam(agent.parameters(), lr=args.learning_rate, eps=1e-5)

    # ALGO Logic: Storage setup
    obs = torch.zeros((args.num_steps, args.num_envs) + envs.single_observation_space.sha
    actions = torch.zeros((args.num_steps, args.num_envs) + envs.single_action_space.shap
    logprobs = torch.zeros((args.num_steps, args.num_envs)).to(device)
    rewards = torch.zeros((args.num_steps, args.
    dones = torch.zeros((args.num_steps, args.nu
```

```
    values = torch.zeros((args.num_steps, args.num_envs)).to(device)

    # TRY NOT TO MODIFY: start the game
    global_step = 0
    start_time = time.time()
    next_obs = torch.Tensor(envs.reset()).to(device)
    next_done = torch.zeros(args.num_envs).to(device)
    num_updates = args.total_timesteps // args.batch_size

    for update in range(1, num_updates + 1):
        # Annealing the rate if instructed to do so.
        if args.anneal_lr:
            frac = 1.0 - (update - 1.0) / num_updates
            lrnow = frac * args.learning_rate
            optimizer.param_groups[0]["lr"] = lrnow

        for step in range(0, args.num_steps):
            global_step += 1 * args.num_envs
            obs[step] = next_obs
            dones[step] = next_done

            # ALGO LOGIC: action logic
            with torch.no_grad():
                action, logprob, _, value = agent.get_action_and_value(next_obs)
                values[step] = value.flatten()
            actions[step] = action
            logprobs[step] = logprob

            # TRY NOT TO MODIFY: execute the game and log data.
            next_obs, reward, done, info = envs.step(action.cpu().numpy())
            rewards[step] = torch.tensor(reward).to(device).view(-1)
            next_obs, next_done = torch.Tensor(next_obs).to(device), torch.Tensor(done).t

            for item in info:
                if "episode" in item.keys():
                    print(f"global_step={global_step}, episodic_return={item['episode']['
                    writer.add_scalar("charts/episodic_return", item["episode"]["r"], glo
                    writer.add_scalar("charts/episodic_length", item["episode"]["l"], glo
                    break

        # bootstrap value if not done
        with torch.no_grad():
            next_value = agent.get_value(next_ob
```

```
        if args.gae:
            advantages = torch.zeros_like(rewards).to(device)
            lastgaelam = 0
            for t in reversed(range(args.num_steps)):
                if t == args.num_steps - 1:
                    nextnonterminal = 1.0 - next_done
                    nextvalues = next_value
                else:
                    nextnonterminal = 1.0 - dones[t + 1]
                    nextvalues = values[t + 1]
                delta = rewards[t] + args.gamma * nextvalues * nextnonterminal - valu
                advantages[t] = lastgaelam = delta + args.gamma * args.gae_lambda * n
            returns = advantages + values
        else:
            returns = torch.zeros_like(rewards).to(device)
            for t in reversed(range(args.num_steps)):
                if t == args.num_steps - 1:
                    nextnonterminal = 1.0 - next_done
                    next_return = next_value
                else:
                    nextnonterminal = 1.0 - dones[t + 1]
                    next_return = returns[t + 1]
                returns[t] = rewards[t] + args.gamma * nextnonterminal * next_return
            advantages = returns - values

    # flatten the batch
    b_obs = obs.reshape((-1,) + envs.single_observation_space.shape)
    b_logprobs = logprobs.reshape(-1)
    b_actions = actions.reshape((-1,) + envs.single_action_space.shape)
    b_advantages = advantages.reshape(-1)
    b_returns = returns.reshape(-1)
    b_values = values.reshape(-1)

    # Optimizing the policy and value network
    b_inds = np.arange(args.batch_size)
    clipfracs = []
    for epoch in range(args.update_epochs):
        np.random.shuffle(b_inds)
        for start in range(0, args.batch_size, args.minibatch_size):
            end = start + args.minibatch_size
            mb_inds = b_inds[start:end]

            _, newlogprob, entropy, newvalue
```

```
            b_obs[mb_inds], b_actions.long()[mb_inds]
        )
        logratio = newlogprob - b_logprobs[mb_inds]
        ratio = logratio.exp()

        with torch.no_grad():
            # calculate approx_kl http://joschu.net/blog/kl-approx.html
            old_approx_kl = (-logratio).mean()
            approx_kl = ((ratio - 1) - logratio).mean()
            clipfracs += [((ratio - 1.0).abs() > args.clip_coef).float().mean().i

        mb_advantages = b_advantages[mb_inds]
        if args.norm_adv:
            mb_advantages = (mb_advantages - mb_advantages.mean()) / (mb_advantag

        # Policy loss
        pg_loss1 = -mb_advantages * ratio
        pg_loss2 = -mb_advantages * torch.clamp(ratio, 1 - args.clip_coef, 1 + ar
        pg_loss = torch.max(pg_loss1, pg_loss2).mean()

        # Value loss
        newvalue = newvalue.view(-1)
        if args.clip_vloss:
            v_loss_unclipped = (newvalue - b_returns[mb_inds]) ** 2
            v_clipped = b_values[mb_inds] + torch.clamp(
                newvalue - b_values[mb_inds],
                -args.clip_coef,
                args.clip_coef,
            )
            v_loss_clipped = (v_clipped - b_returns[mb_inds]) ** 2
            v_loss_max = torch.max(v_loss_unclipped, v_loss_clipped)
            v_loss = 0.5 * v_loss_max.mean()
        else:
            v_loss = 0.5 * ((newvalue - b_returns[mb_inds]) ** 2).mean()

        entropy_loss = entropy.mean()
        loss = pg_loss - args.ent_coef * entropy_loss + v_loss * args.vf_coef

        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(agent.parameters(), args.max_grad_norm)
        optimizer.step()
```

```
            if args.target_kl is not None:
                if approx_kl > args.target_kl:
                    break

        y_pred, y_true = b_values.cpu().numpy(), b_returns.cpu().numpy()
        var_y = np.var(y_true)
        explained_var = np.nan if var_y == 0 else 1 - np.var(y_true - y_pred) / var_y

        # TRY NOT TO MODIFY: record rewards for plotting purposes
        writer.add_scalar("charts/learning_rate", optimizer.param_groups[0]["lr"], global
        writer.add_scalar("losses/value_loss", v_loss.item(), global_step)
        writer.add_scalar("losses/policy_loss", pg_loss.item(), global_step)
        writer.add_scalar("losses/entropy", entropy_loss.item(), global_step)
        writer.add_scalar("losses/old_approx_kl", old_approx_kl.item(), global_step)
        writer.add_scalar("losses/approx_kl", approx_kl.item(), global_step)
        writer.add_scalar("losses/clipfrac", np.mean(clipfracs), global_step)
        writer.add_scalar("losses/explained_variance", explained_var, global_step)
        print("SPS:", int(global_step / (time.time() - start_time)))
        writer.add_scalar("charts/SPS", int(global_step / (time.time() - start_time)), gl

    envs.close()
    writer.close()

    # Create the evaluation environment
    eval_env = gym.make(args.env_id)

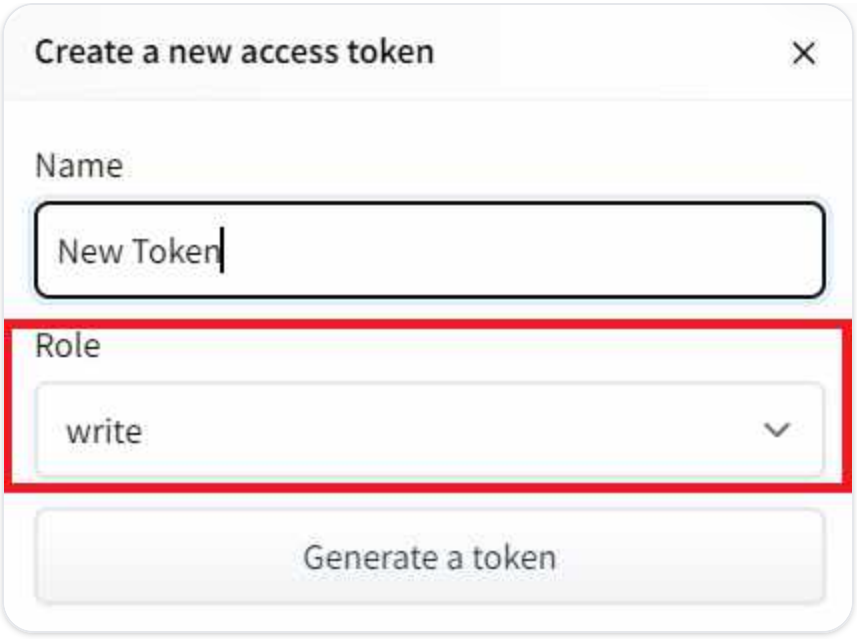
    package_to_hub(
        repo_id=args.repo_id,
        model=agent,  # The model we want to save
        hyperparameters=args,
        eval_env=gym.make(args.env_id),
        logs=f"runs/{run_name}",
    )
```

To be able to share your model with the community there are three more steps to follow:

1️⃣ (If it's not already done) create an account to HF ➡️ https://huggingface.co/join

2️⃣ Sign in and get your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**



- Copy the token
- Run the cell below and paste the token

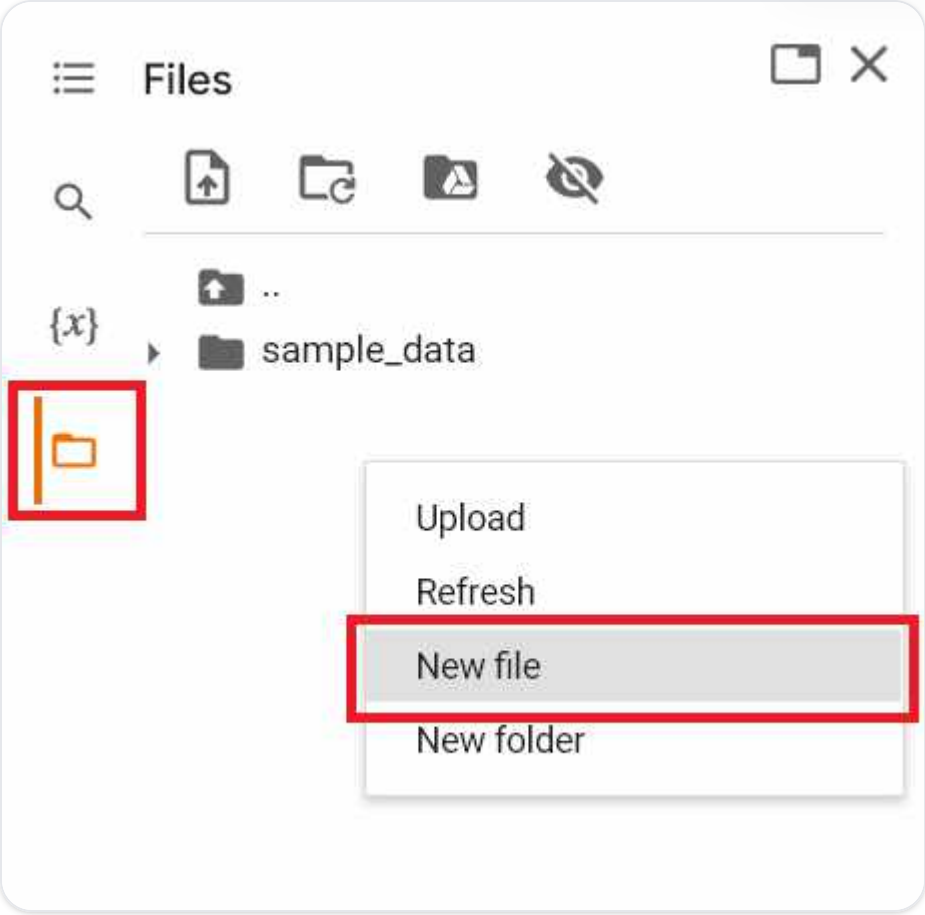
```
from huggingface_hub import notebook_login
notebook_login()
!git config --global credential.helper store
```

If you don't want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

Let's start the training 🔥

⚠️ ⚠️ ⚠️ Don't use **the same repo id with the one you used for the Unit 1**

- Now that you've coded PPO from scratch and added the Hugging Face Integration, we're ready to start the training 🔥
- First, you need to copy all your code to a file you create called `ppo.py`


Files
..
sample_data
Upload
Refresh
New file
New folder

ppo.py

```python
# docs and experiment results can be found

import argparse
import os
import random
import time
from distutils.util import strtobool

import gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions.categorical import
from torch.utils.tensorboard import Summar

from huggingface_hub import HfApi, upload_
from huggingface_hub.repocard import metad

from pathlib import Path
```

- Now we just need to run this python script using `python <name-of-python-script>.py` with the additional parameters we defined using `argparse`
- You should modify more hyperparameters otherwise the training will not be super stable.

```
!python ppo.py --env-id="LunarLander-v2" --repo-id="YOUR_REPO_ID" --total-timesteps=50000
```

Some additional challenges 🏆

The best way to learn **is to try things on your own**! Why not try another environment? Or why not trying to modify the implementation to work with Gym

See you in Unit 8, part 2 where we're going to train agents to play Doom 🔥

Keep learning, stay awesome 🤗

<> Update on GitHub

← Visualize the Clipped Surrogate Objective Function

Conclusion →

Search models, datasets, users...

Deep RL Course documentation
Hands-on

Join the Hugging Face community

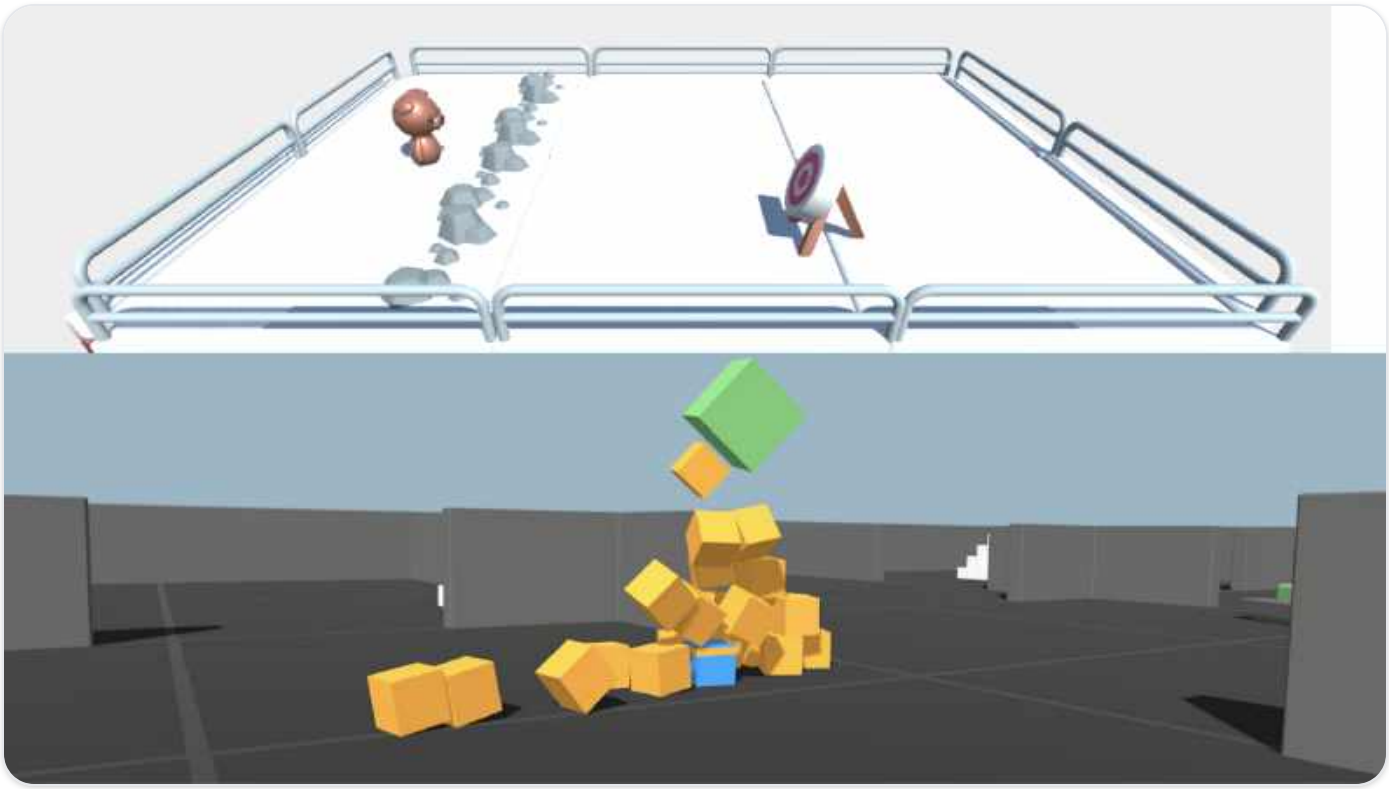
and get access to the augmented documentation experience

Sign Up to get started

Hands-on

Ask a question Open in Colab

We learned what ML-Agents is and how it works. We also studied the two environments we're going to use. Now we're ready to train our agents!



Ask HuggingChat

To validate this hands-on for the certification process, you **just need to push your trained models to the Hub.** There are **no minimum results to attain** in order to validate this Hands On. But if you want to get nice results, you can try to reach the following:

- For Pyramids: Mean Reward = 1.75
- For SnowballTarget: Mean Reward = 15 or 30 targets shoot in an episode.

For more information about the certification process, check this section 👉 https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

To start the hands-on, click on Open In Colab button 👇 :



We strongly **recommend students use Google Colab for the hands-on exercises** instead of running them on their personal computers.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects** of setting up your environments.

Unit 5: An Introduction to ML-Agents



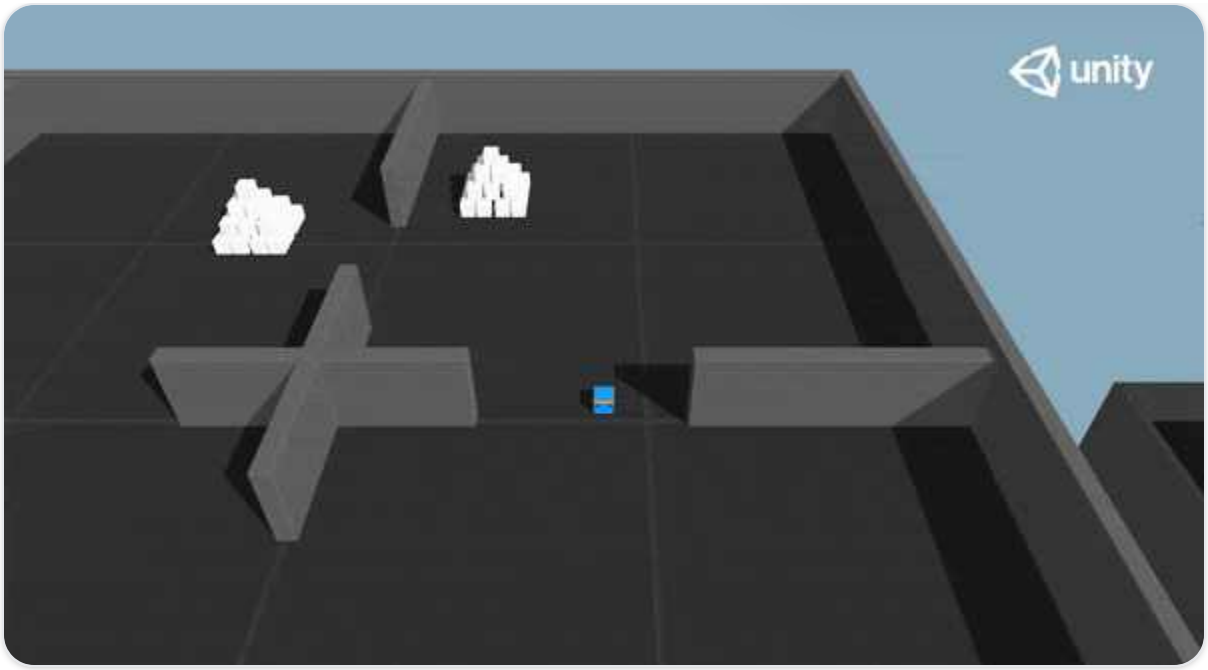
In this notebook, you'll learn about ML-Agents and train two agents.

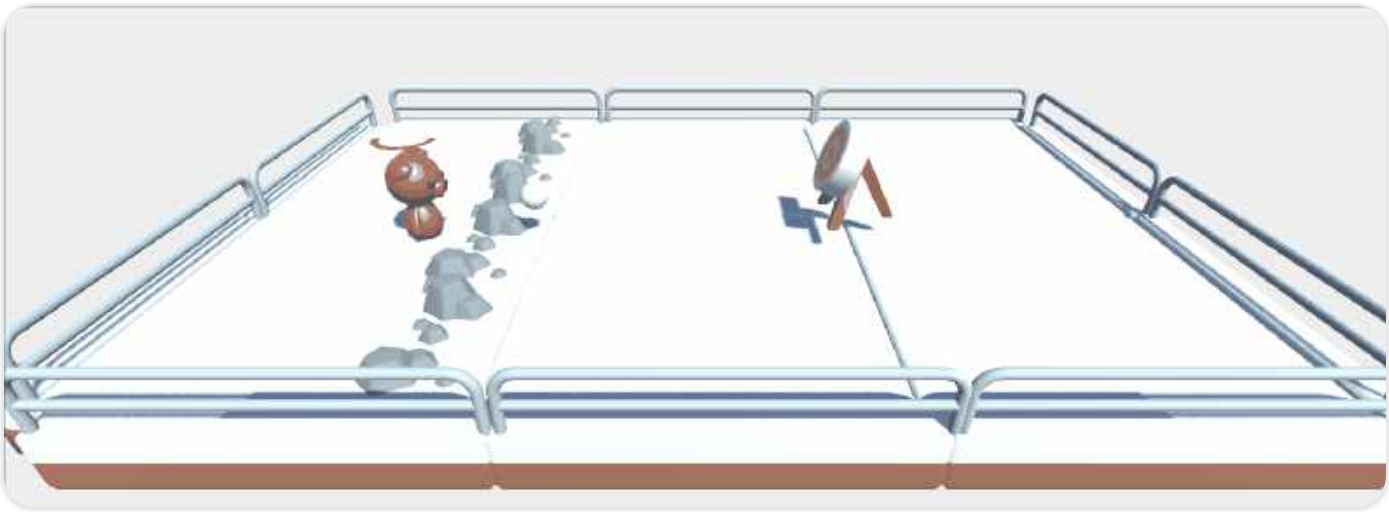
- The first one will learn to **shoot snowballs onto spawning targets**.
- The second needs to press a button to spawn a pyramid, then navigate to the pyramid, knock it over, **and move to the gold brick at the top**. To do that, it will need to explore its environment, and we will use a technique called curiosity.

After that, you'll be able **to watch your agents playing directly on your browser**.

For more information about the certification process, check this section 👉 https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

⬇️ Here is an example of what **you will achieve at the end of this unit.** ⬇️





🎮 Environments:

- Pyramids
- SnowballTarget

📚 RL-Library:

- ML-Agents

We’re constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Objectives of this notebook 🏆

At the end of the notebook, you will:

- Understand how **ML-Agents** works and the environment library.
- Be able to **train agents in Unity Environments**.

Prerequisites 🏗️

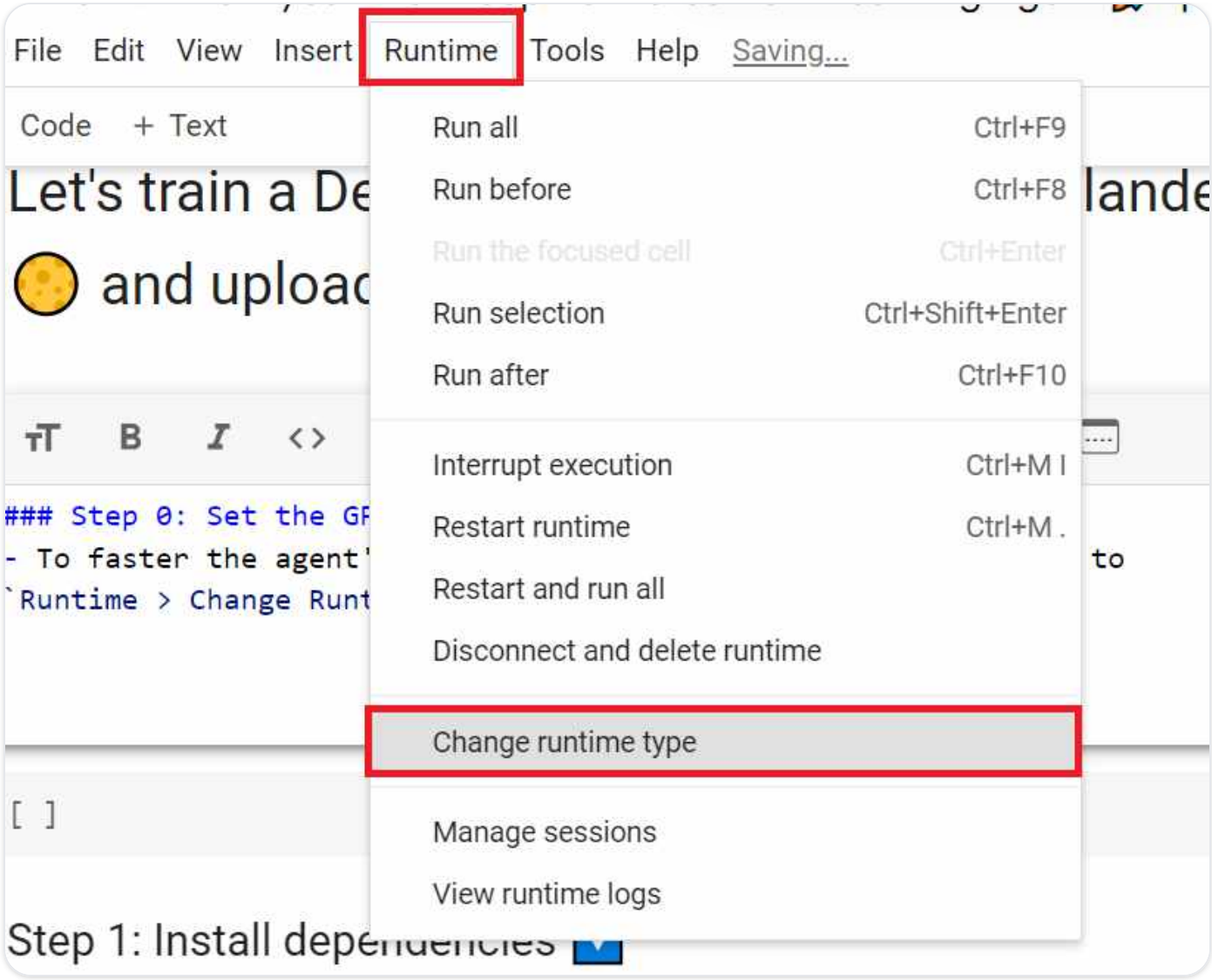
Before diving into the notebook, you need to:

🔲 📚 **Study** what ML-Agents is and how it works by reading Unit 5 🤗

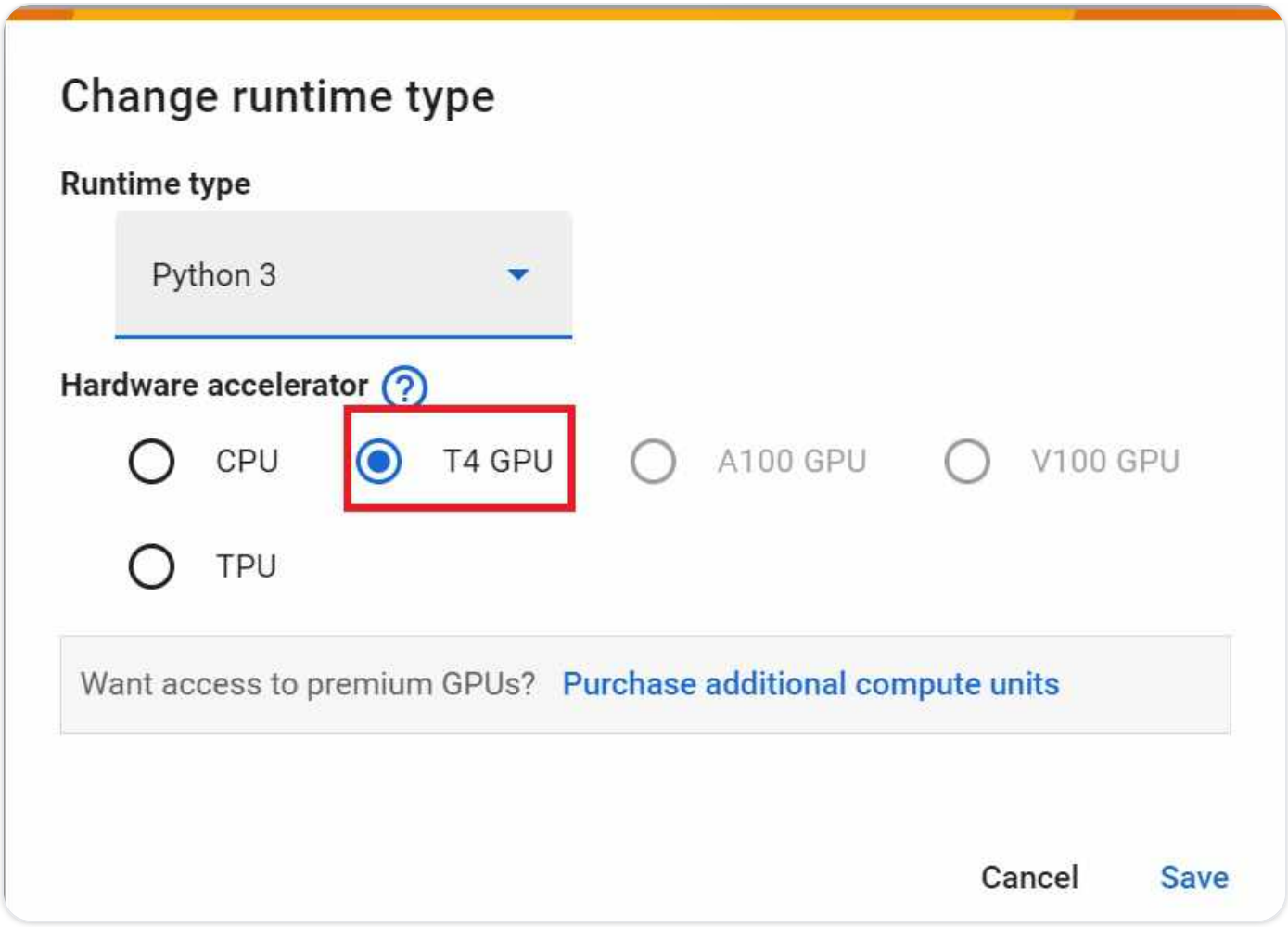
Let's train our agents 🚀

Set the GPU 💪

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`



- `Hardware Accelerator > GPU`



Clone the repository 🔽

- We need to clone the repository, that contains **ML-Agents.**

```
# Clone the repository (can take 3min)
git clone --depth 1 https://github.com/Unity-Technologies/ml-agents
```

Setup the Virtual Environment 🔽

- In order for the **ML-Agents** to run successfully in Colab, Colab's Python version must meet the library's Python requirements.

- We can check for the supported Python version under the `python_requires` parameter in the `setup.py` files. These files are required to set up th
found in the following locations:

    - `/content/ml-agents/ml-agents/setup.py`
    - `/content/ml-agents/ml-agents-envs/setup.py`

- Colab's Current Python version(can be checked using `!python --version`) doesn't match the library's `python_requires` parameter, as a result installation may silently fail and lead to errors like these, when executing the same commands later:
    - `/bin/bash: line 1: mlagents-learn: command not found`
    - `/bin/bash: line 1: mlagents-push-to-hf: command not found`

- To resolve this, we'll create a virtual environment with a Python version compatible with the **ML-Agents** library.

`Note:` *For future compatibility, always check the `python_requires` parameter in the installation files and set your virtual environment to the maximum supported Python version in the given below script if the Colab's Python version is not compatible*

```
# Colab's Current Python Version (Incompatible with ML-Agents)
!python --version
```

```
# Install virtualenv and create a virtual environment
!pip install virtualenv
!virtualenv myenv

# Download and install Miniconda
!wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
!chmod +x Miniconda3-latest-Linux-x86_64.sh
!./Miniconda3-latest-Linux-x86_64.sh -b -f -p /usr/local

# Activate Miniconda and install Python ver 3.10.12
!source /usr/local/bin/activate
!conda install -q -y --prefix /usr/local python=3.10.12 ujson  # Specify the version here

# Set environment variables for Python and conda paths
!export PYTHONPATH=/usr/local/lib/python3.10/site-packages/
!export CONDA_PREFIX=/usr/local/envs/myenv
```

```
# Python Version in New Virtual Environment (Compatible with ML-Agents)
!python --version
```

Installing the dependencies 🔽

```
# Go inside the repository and install the package (can take 3min)
%cd ml-agents
pip3 install -e ./ml-agents-envs
pip3 install -e ./ml-agents
```

SnowballTarget ⛄

If you need a refresher on how this environment works check this section 👉
https://huggingface.co/deep-rl-course/unit5/snowball-target

Download and move the environment zip file in ./training-envs-executables/linux/

- Our environment executable is in a zip file.
- We need to download it and place it to `./training-envs-executables/linux/`
- We use a linux executable because we use colab, and colab machines OS is Ubuntu (linux)

```
# Here, we create training-envs-executables and linux
mkdir ./training-envs-executables
mkdir ./training-envs-executables/linux
```

We downloaded the file SnowballTarget.zip from https://github.com/huggingface/Snowball-Target using `wget`

```
wget "https://github.com/huggingface/Snowball-Target/raw/main/SnowballTarget.zip" -O ./tr
```

We unzip the executable.zip file

```
unzip -d ./training-envs-executables/linux/ ./training-envs-executables/linux/SnowballTar
```

Make sure your file is accessible

```
chmod -R 755 ./training-envs-executables/linux/SnowballTarget
```

Define the SnowballTarget config file

- In ML-Agents, you define the **training hyperparameters in config.yaml files.**

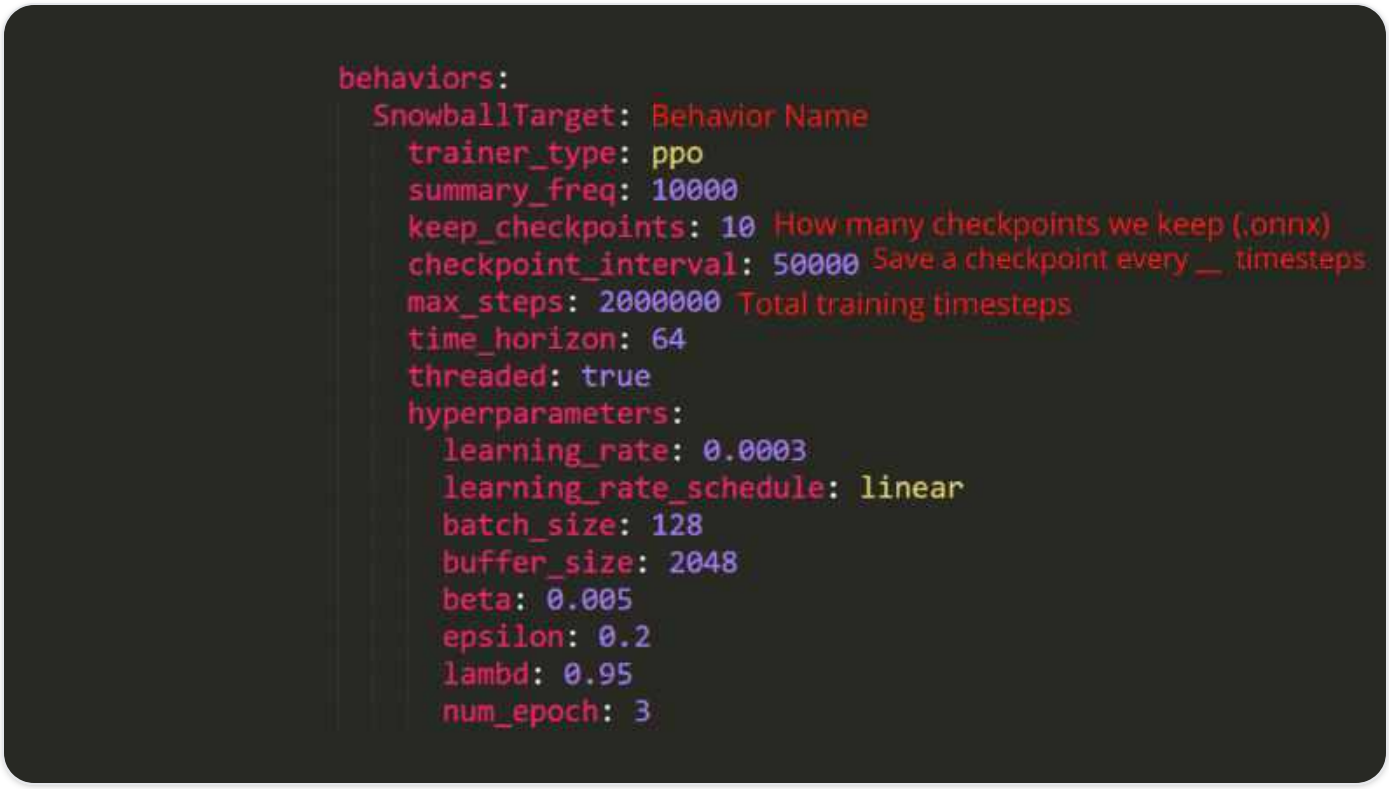
There are multiple hyperparameters. To understand them better, you should read the explanation for each one in the documentation

You need to create a `SnowballTarget.yaml` config file in ./content/ml-agents/config/ppo/

We'll give you a preliminary version of this config (to copy and paste into your `SnowballTarget.yaml file`), **but you should modify it**.

```
behaviors:
  SnowballTarget:
    trainer_type: ppo
    summary_freq: 10000
    keep_checkpoints: 10
    checkpoint_interval: 50000
    max_steps: 200000
    time_horizon: 64
    threaded: true
    hyperparameters:
      learning_rate: 0.0003
      learning_rate_schedule: linear
      batch_size: 128
      buffer_size: 2048
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
    network_settings:
```

```
    normalize: false
    hidden_units: 256
    num_layers: 2
    vis_encode_type: simple
  reward_signals:
    extrinsic:
      gamma: 0.99
      strength: 1.0
```





As an experiment, try to modify some other hyperparameters. Unity provides very good documentation explaining each of them here.

Now that you've created the config file and understand what most hyperparameters do, we're ready to train our agent 🔥 .

Train the agent

To train our agent, we need to **launch mlagents-learn and select the executable containing the environment.**

We define four parameters:

1. `mlagents-learn <config>`: the path where the hyperparameter config file is.
2. `--env`: where the environment executable is.
3. `--run_id`: the name you want to give to your training run id.
4. `--no-graphics`: to not launch the visualization during the training.



Train the model and use the `--resume` flag to continue training in case of interruption.

> *"It will fail the first time if and when you use `--resume`. Try rerunning the block to bypass the error."*

The training will take 10 to 35min depending on your config. Go take a 🧉 you deserve it 🤗.

```
mlagents-learn ./config/ppo/SnowballTarget.yaml --env=./training-envs-executables/linux/S
```

Push the agent to the Hugging Face Hub

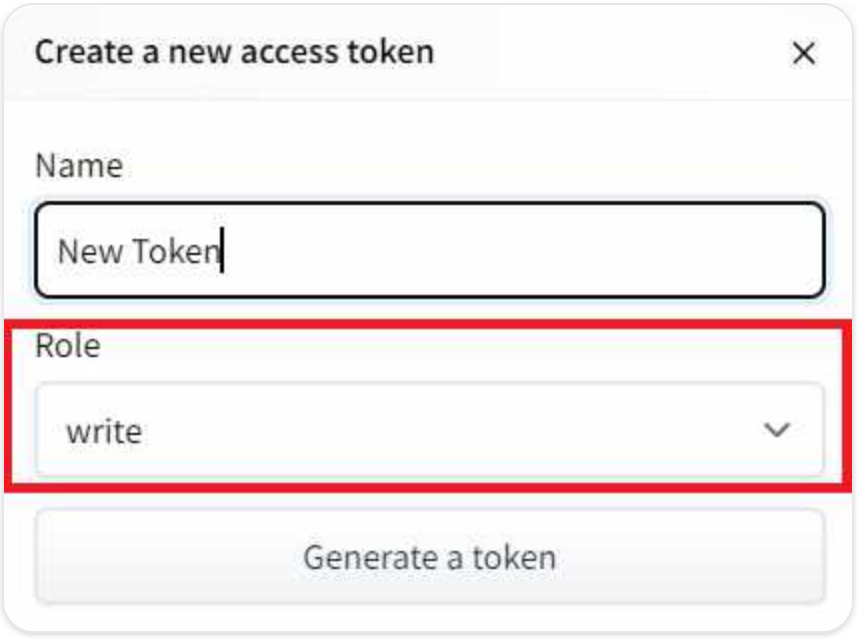
- Now that we've trained our agent, we're **ready to push it to the Hub and visualize it playing on your browser 🔥 .**

To be able to share your model with the community, there are three more steps to follow:

1️⃣ (If it's not already done) create an account to HF ➡️ https://huggingface.co/join

2️⃣ Sign in and store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**



- Copy the token
- Run the cell below and paste the token

```
from huggingface_hub import notebook_login

notebook_login()
```

If you don’t want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

Then we need to run `mlagents-push-to-hf`.

And we define four parameters:

1. `--run-id`: the name of the training run id.
2. `--local-dir`: where the agent was saved, it’s results/<run_id name>, so in my case results/First Training.
3. `--repo-id`: the name of the Hugging Face repo you want to create or update. It’s always <your huggingface username>/<the repo name> If the repo does not exist **it will be created automatically**
4. `--commit-message`: since HF repos are git reposit



For instance:

```
mlagents-push-to-hf --run-id="SnowballTarget1" --local-dir="./results/SnowballTarget1" --repo-id="ThomasSimonini/ppo-SnowballTarget" --commit-message="First Push"
```

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-i
```

If everything worked you should see this at the end of the process (but with a different url 😆) :

```
Your model is pushed to the hub. You can view your model here: https://huggingface.co/Thc
```

It's the link to your model. It contains a model card that explains how to use it, your Tensorboard, and your config file. **What's awesome is that it's a git repository, which means you can have different commits, update your repository with a new push, etc.**

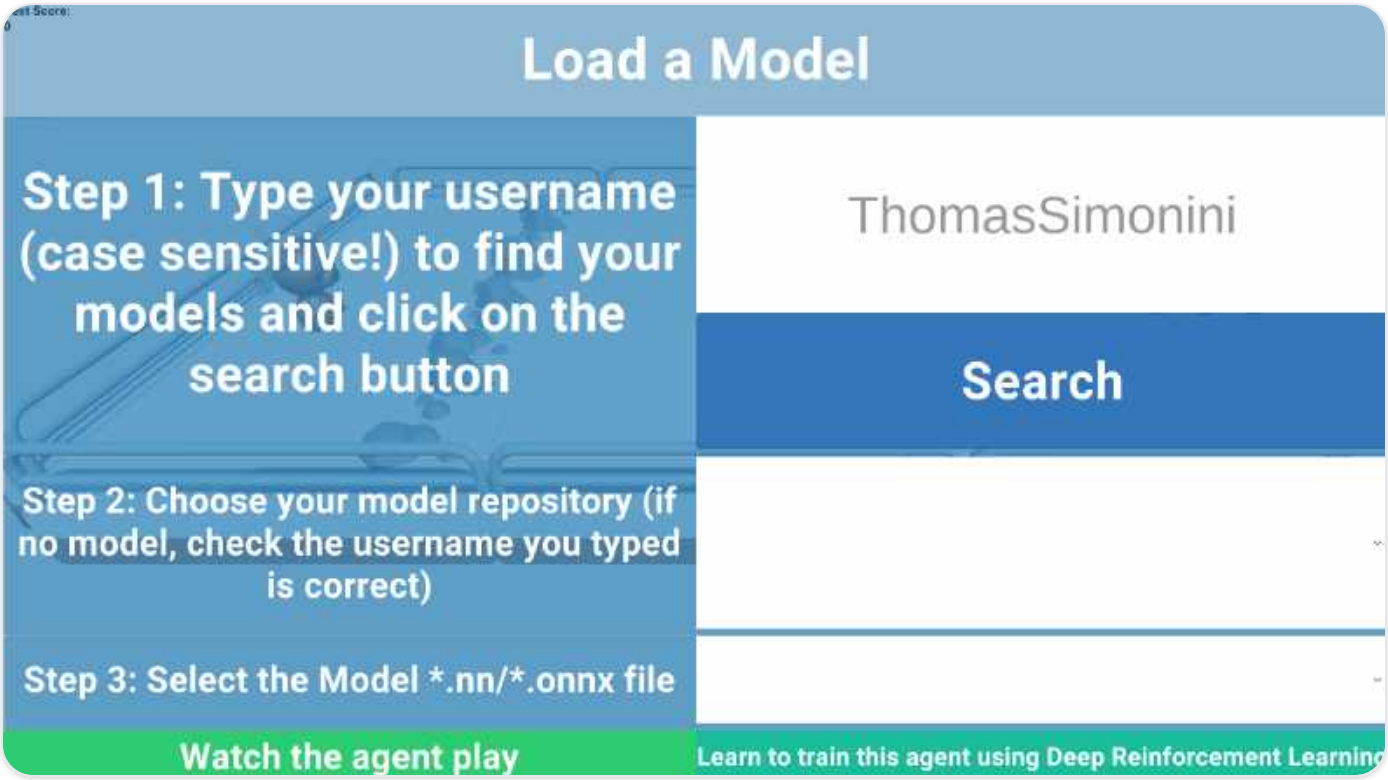
But now comes the best: **being able to visualize your agent online 👀 .**

Watch your agent playing 👀

This step it's simple:

1. Go here: https://huggingface.co/spaces/ThomasSimonini/ML-Agents-SnowballTarget

2. Launch the game and put it in full screen by clicking on the bottom right button



1. In step 1, type your username (your username is case sensitive: for instance, my username is ThomasSimonini not thomassimonini or ThOmasImoNInI) and click on the search button.

2. In step 2, select your model repository.

3. In step 3, **choose which model you want to replay**:

- I have multiple ones, since we saved a model every 500000 timesteps.
- But since I want the most recent one, I choose `SnowballTarget.onnx`

👉 It's good **to try with different models steps to see the improvement of the agent.**

And don't hesitate to share the best score your agent gets on discord in the #rl-i-made-this channel 🔥

Now let's try a more challenging environment called Py

Pyramids 🏆

Download and move the environment zip file in ./training-envs-executables/linux/

- Our environment executable is in a zip file.
- We need to download it and place it into `./training-envs-executables/linux/`
- We use a linux executable because we're using colab, and the colab machine's OS is Ubuntu (linux)

We downloaded the file Pyramids.zip from from https://huggingface.co/spaces/unity/ML-Agents-Pyramids/resolve/main/Pyramids.zip using `wget`

```
wget "https://huggingface.co/spaces/unity/ML-Agents-Pyramids/resolve/main/Pyramids.zip" -
```

Unzip it

```
unzip -d ./training-envs-executables/linux/ ./training-envs-executables/linux/Pyramids.zi
```

Make sure your file is accessible

```
chmod -R 755 ./training-envs-executables/linux/Pyramids/Pyramids
```

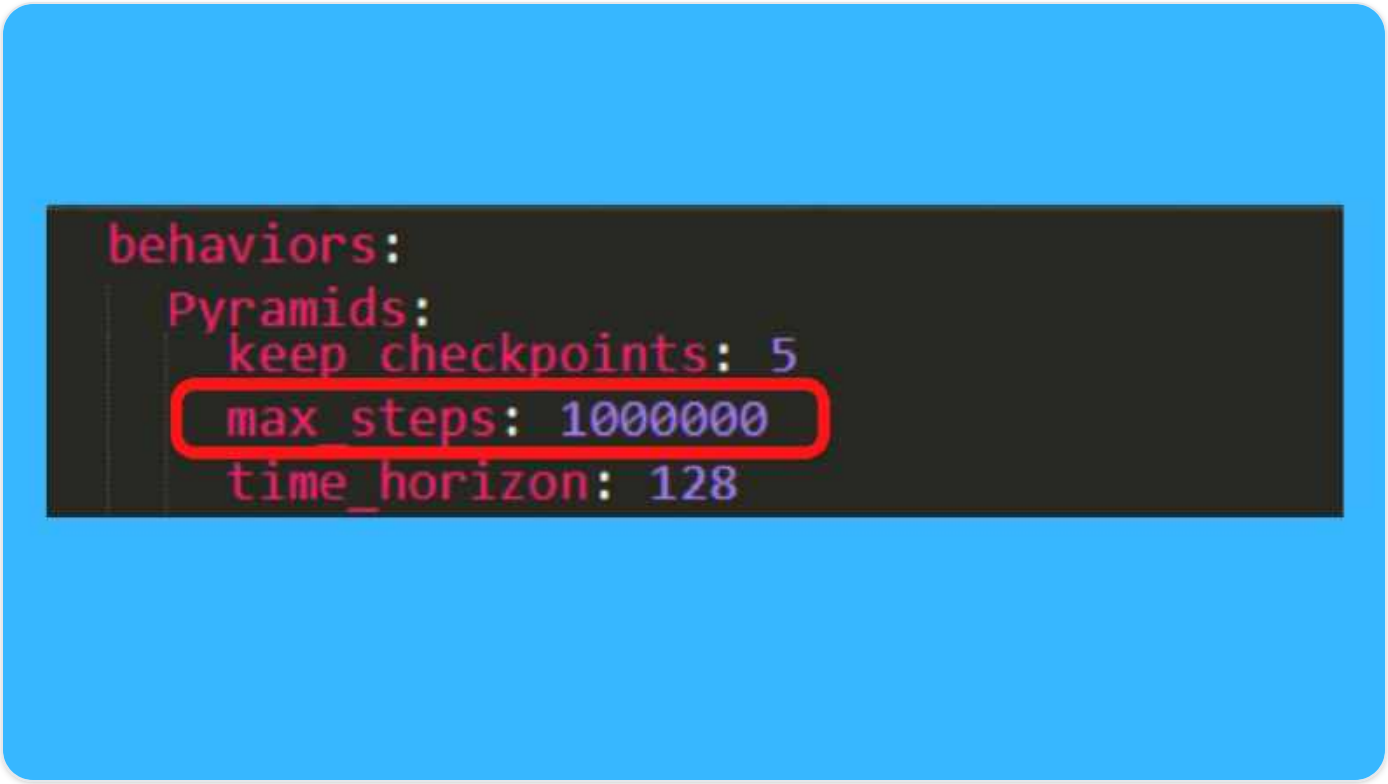
Modify the PyramidsRND config file

- Contrary to the first environment, which was a custom one, **Pyramids was made by the Unity team**.
- So the PyramidsRND config file already exists and is in ./content/ml-agents/config/ppo/PyramidsRND.yaml
- You might ask why "RND" is in PyramidsRND. RND stands for *random network distillation* it's a way to generate curiosity rewards. If you want to kn

explaining this technique: https://medium.com/data-from-the-trenches/curiosity-driven-learning-through-random-network-distillation-488ffd8e5938

For this training, we'll modify one thing:

- The total training steps hyperparameter is too high since we can hit the benchmark (mean reward = 1.75) in only 1M training steps. 👉 To do that, we go to config/ppo/PyramidsRND.yaml,**and change max_steps to 1000000.**



As an experiment, you should also try to modify some other hyperparameters. Unity provides very good documentation explaining each of them here.

We're now ready to train our agent 🔥.

Train the agent

The training will take 30 to 45min depending on your machine, go take a 🧉 you deserve it 🤗.

```
mlagents-learn ./config/ppo/PyramidsRND.yaml --e
```

Push the agent to the Hugging Face Hub

- Now that we trained our agent, we're **ready to push it to the Hub to be able to visualize it playing on your browser 🔥 .**

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-i
```

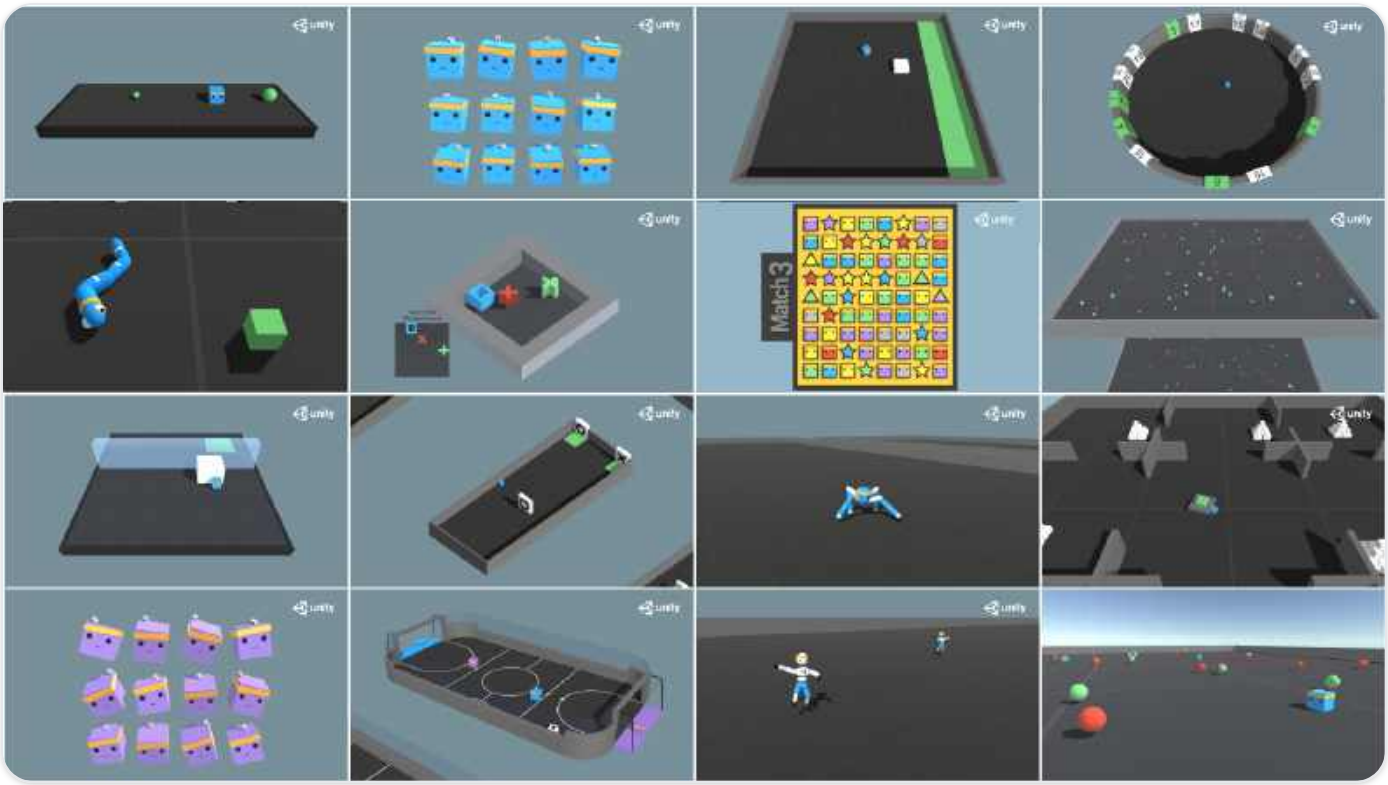
Watch your agent playing 👀

👉 https://huggingface.co/spaces/unity/ML-Agents-Pyramids

🎁 Bonus: Why not train on another environment?

Now that you know how to train an agent using MLAgents, **why not try another environment?**

MLAgents provides 17 different environments and we're building some custom ones. The best way to learn is to try things on your own, have fun.



You have the full list of the one currently available environments on Hugging Face here 👉 https://github.com/huggingface/ml-agents#the-environments

For the demos to visualize your agent 👉 https://huggingface.co/unity

For now we have integrated:

- Worm demo where you teach a **worm to crawl**.
- Walker demo where you teach an agent **to walk towards a goal**.

That's all for today. Congrats on finishing this tutorial!

The best way to learn is to practice and try stuff. Why not try another environment? ML-Agents has 18 different environments, but you can also create your own. Check the documentation and have fun!

See you on Unit 6 🔥,

Keep Learning, Stay awesome 🤗

<> Update on GitHub

← (Optional) What is curiosity in Deep Reinforcement Learning?

Bonus. Learn to create your own environments with Unity and MLAgents →

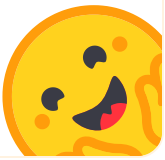
Search models, datasets, users...

Deep RL Course documentation
Hands-on: advanced Deep Reinforcement Learning. Using Sample Factory to play Doom from pixels

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Hands-on: advanced Deep Reinforcement Learning. Using Sample Factory to play Doom from pixels

Ask a question Open in Colab

The colab notebook: Open in Colab

Unit 8 Part 2: Advanced Deep Reinforcement Learning. Using Sample Factory to play Doom from pixels

Ask HuggingChat



In this notebook, we will learn how to train a Deep Neural Network to collect objects in a 3D environment based on the game of Doom, a video of the resulting policy is shown below. We train this policy using Sample Factory, an asynchronous implementation of the PPO algorithm.

Please note the following points:

- Sample Factory is an advanced RL framework and **only functions on Linux and Mac** (not Windows).
- The framework performs best on a **GPU machine with many CPU cores**, where it can achieve speeds of 100k interactions per second. The resources available on a standard Colab notebook **limit the performance of this library**. So the speed in this setting **does not reflect the real-world performance**.
- Benchmarks for Sample Factory are available in a number of settings, check out the examples if you want to find out more.

```
from IPython.display import HTML

HTML(
```

```
    """<video width="640" height="480" controls>
  <source src="https://huggingface.co/edbeeching/doom_health_gathering_supreme_3333/resol
  type="video/mp4">Your browser does not support the video tag.</video>"""
)
```

To validate this hands-on for the certification process, you need to push one model:

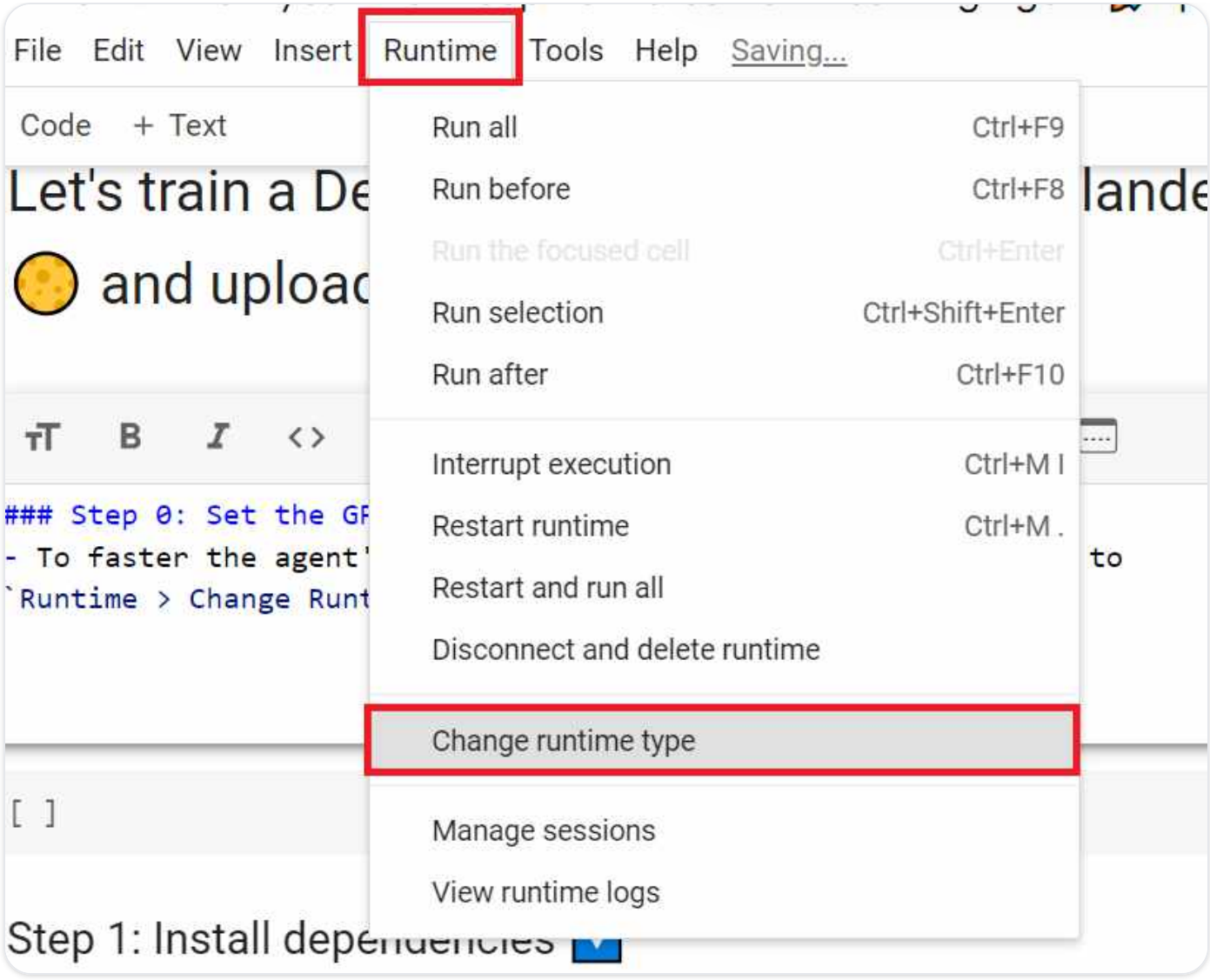
To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

If you don't find your model, **go to the bottom of the page and click on the refresh button**

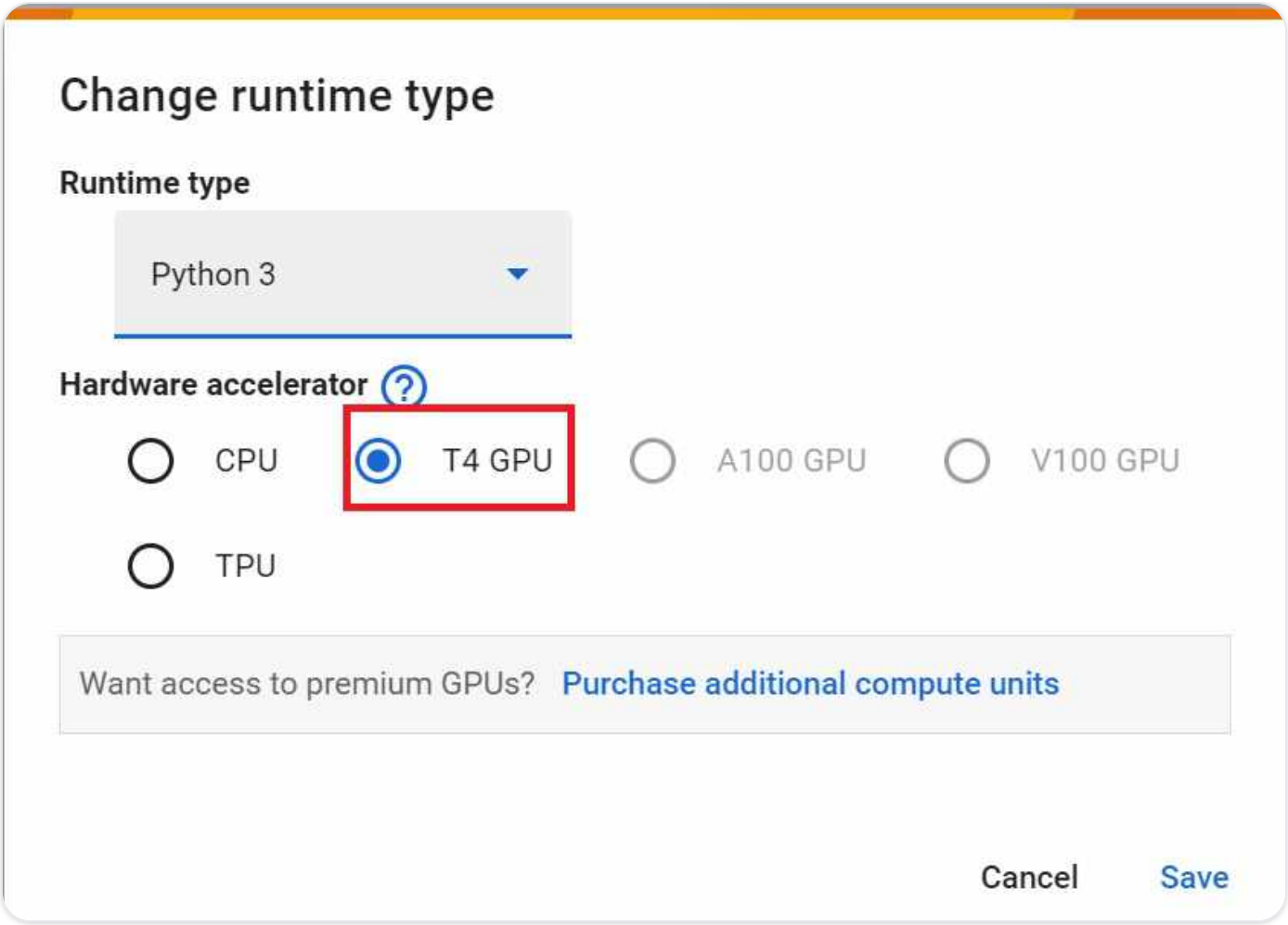
For more information about the certification process, check this section 👉 https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

Set the GPU 💪

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`



- `Hardware Accelerator > GPU`

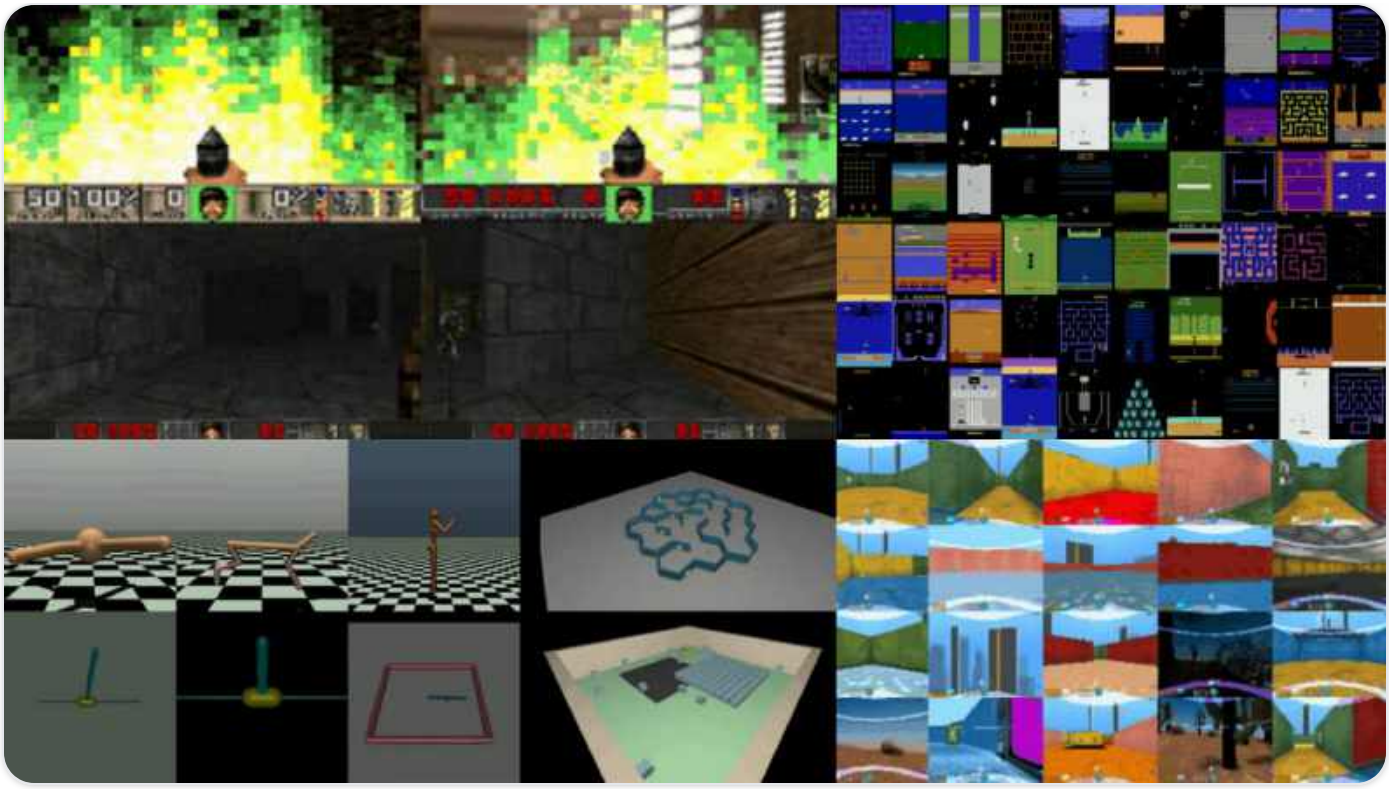


Before starting to train our agent, let's **study the library and environments we're going to use**.

Sample Factory

Sample Factory is one of the **fastest RL libraries focused on very efficient synchronous and asynchronous implementations of policy gradients (PPO)**.

Sample Factory is thoroughly **tested, used by many researchers and practitioners**, and is actively maintained. Our implementation is known to **reach SOTA performance in a variety of domains while minimizing RL experiment training time and hardware requirements**.



Key features

- Highly optimized algorithm architecture for maximum learning throughput
- Synchronous and asynchronous training regimes
- Serial (single-process) mode for easy debugging
- Optimal performance in both CPU-based and GPU-accelerated environments
- Single- & multi-agent training, self-play, supports training multiple policies at once on one or many GPUs
- Population-Based Training (PBT)
- Discrete, continuous, hybrid action spaces
- Vector-based, image-based, dictionary observation spaces
- Automatically creates a model architecture by parsing action/observation space specification. Supports custom model architectures
- Designed to be imported into other projects, custom environments are first-class citizens
- Detailed WandB and Tensorboard summaries, cust
- HuggingFace 🤗 integration (upload trained models and metrics to the Hub)

- Multiple example environment integrations with tuned parameters and trained models

All of the above policies are available on the 🤗 hub. Search for the tag sample-factory

How sample-factory works

Sample-factory is one of the **most highly optimized RL implementations available to the community**.

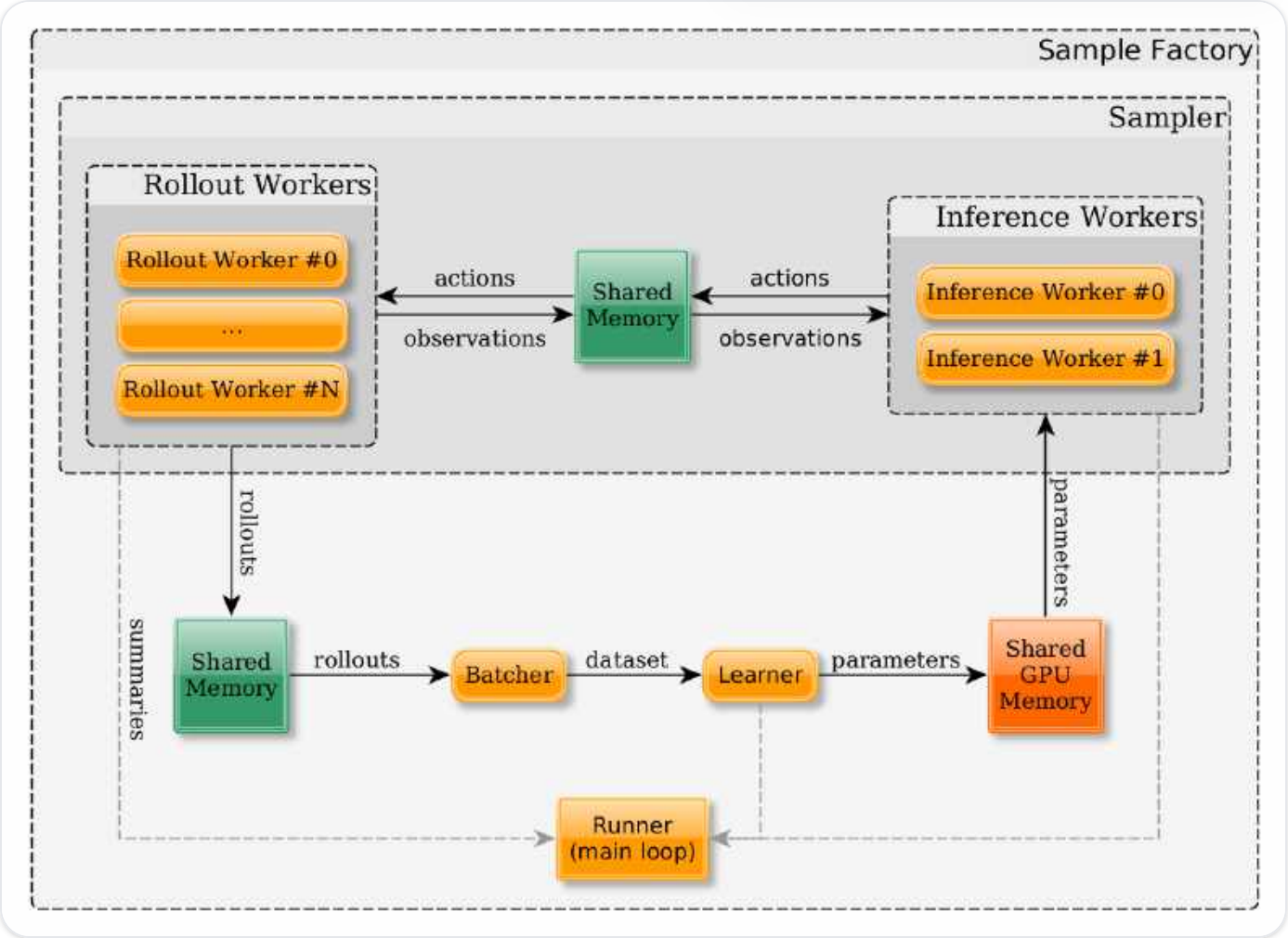
It works by **spawning multiple processes that run rollout workers, inference workers and a learner worker**.

The *workers* **communicate through shared memory, which lowers the communication cost between processes**.

The *rollout workers* interact with the environment and send observations to the *inference workers*.

The *inferences workers* query a fixed version of the policy and **send actions back to the rollout worker**.

After *k* steps the rollout works send a trajectory of experience to the learner worker, **which it uses to update the agent's policy network**.



Actor Critic models in Sample-factory

Actor Critic models in Sample Factory are composed of three components:

- **Encoder** - Process input observations (images, vectors) and map them to a vector. This is the part of the model you will most likely want to customize.
- **Core** - Intergrate vectors from one or more encoders, can optionally include a single- or multi-layer LSTM/GRU in a memory-based agent.
- **Decoder** - Apply additional layers to the output of the model core before computing the policy and value outputs.

The library has been designed to automatically support any observation and action spaces. Users can easily add their custom models. You can find out more in the documentation.

ViZDoom

ViZDoom is an **open-source python interface for the Doom Engine**.

The library was created in 2016 by Marek Wydmuch, Michal Kempka at the Institute of Computing Science, Poznan University of Technology, Poland.

The library enables the **training of agents directly from the screen pixels in a number of scenarios**, including team deathmatch, shown in the video below. Because the ViZDoom environment is based on a game the was created in the 90s, it can be run on modern hardware at accelerated speeds, **allowing us to learn complex AI behaviors fairly quickly**.

The library includes feature such as:

- Multi-platform (Linux, macOS, Windows),
- API for Python and C++,
- OpenAI Gym environment wrappers
- Easy-to-create custom scenarios (visual editors, scripting language, and examples available),
- Async and sync single-player and multiplayer modes,
- Lightweight (few MBs) and fast (up to 7000 fps in sync mode, single-threaded),
- Customizable resolution and rendering parameters,
- Access to the depth buffer (3D vision),
- Automatic labeling of game objects visible in the frame,
- Access to the audio buffer
- Access to the list of actors/objects and map geometry,
- Off-screen rendering and episode recording,
- Time scaling in async mode.

We first need to install some dependencies that are required for the ViZDoom environment

Now that our Colab runtime is set up, we can start by in
ViZDoom on linux.

If you are following on your machine on Mac, you will want to follow the installation instructions on the github page.

```
# Install ViZDoom deps from
# https://github.com/mwydmuch/ViZDoom/blob/master/doc/Building.md#-linux

apt-get install build-essential zlib1g-dev libsdl2-dev libjpeg-dev \
nasm tar libbz2-dev libgtk2.0-dev cmake git libfluidsynth-dev libgme-dev \
libopenal-dev timidity libwildmidi-dev unzip ffmpeg

# Boost libraries
apt-get install libboost-all-dev

# Lua binding dependencies
apt-get install liblua5.1-dev
```

Then we can install Sample Factory and ViZDoom

- This can take 7min

```
pip install sample-factory
pip install vizdoom
```

Setting up the Doom Environment in sample-factory

```
import functools

from sample_factory.algo.utils.context import global_model_factory
from sample_factory.cfg.arguments import parse_full_cfg, parse_sf_args
from sample_factory.envs.env_utils import register_env
from sample_factory.train import run_rl

from sf_examples.vizdoom.doom.doom_model import make_vizdoom_encoder
from sf_examples.vizdoom.doom.doom_params import add_doom_env_args, doom_override_default
from sf_examples.vizdoom.doom.doom_utils import DOOM_ENVS, make_doom_env_from_spec
```

```
# Registers all the ViZDoom environments
def register_vizdoom_envs():
    for env_spec in DOOM_ENVS:
        make_env_func = functools.partial(make_doom_env_from_spec, env_spec)
        register_env(env_spec.name, make_env_func)


# Sample Factory allows the registration of a custom Neural Network architecture
# See https://github.com/alex-petrenko/sample-factory/blob/master/sf_examples/vizdoom/doc
def register_vizdoom_models():
    global_model_factory().register_encoder_factory(make_vizdoom_encoder)


def register_vizdoom_components():
    register_vizdoom_envs()
    register_vizdoom_models()


# parse the command line args and create a config
def parse_vizdoom_cfg(argv=None, evaluation=False):
    parser, _ = parse_sf_args(argv=argv, evaluation=evaluation)
    # parameters specific to Doom envs
    add_doom_env_args(parser)
    # override Doom default values for algo parameters
    doom_override_defaults(parser)
    # second parsing pass yields the final configuration
    final_cfg = parse_full_cfg(parser, argv)
    return final_cfg
```

Now that the setup if complete, we can train the agent. We have chosen here to learn a ViZDoom task called `Health Gathering Supreme`.

The scenario: Health Gathering Supreme



The objective of this scenario is to **teach the agent how to survive without knowing what makes it survive**. The Agent know only that **life is precious** and death is bad so **it must learn what prolongs its existence and that its health is connected with survival**.

The map is a rectangle containing walls and with a green, acidic floor which **hurts the player periodically**. Initially there are some medkits spread uniformly over the map. A new medkit falls from the skies every now and then. **Medkits heal some portions of player's health** - to survive, the agent needs to pick them up. The episode finishes after the player's death or on timeout.

Further configuration:

- Living_reward = 1
- 3 available buttons: turn left, turn right, move forward
- 1 available game variable: HEALTH
- death penalty = 100

You can find out more about the scenarios available in ViZDoom here.

There are also a number of more complex scenarios that have been create for ViZDoom, such as the ones detailed on this github page.

Training the agent

- We're going to train the agent for 4000000 steps. It will take approximately 20min

```
## Start the training, this should take around 15 minutes
register_vizdoom_components()

# The scenario we train on today is health gathering
# other scenarios include "doom_basic", "doom_two_colors_easy", "doom_dm", "doom_dwango5"
env = "doom_health_gathering_supreme"
cfg = parse_vizdoom_cfg(
    argv=[f"--env={env}", "--num_workers=8", "--num_envs_per_worker=4", "--train_for_env_
)

status = run_rl(cfg)
```

Let's take a look at the performance of the trained policy and output a video of the agent.

```
from sample_factory.enjoy import enjoy

cfg = parse_vizdoom_cfg(
    argv=[f"--env={env}", "--num_workers=1", "--save_video", "--no_render", "--max_num_ep
)
status = enjoy(cfg)
```

Now lets visualize the performance of the agent

```
from base64 import b64encode
from IPython.display import HTML
```

```
mp4 = open("/content/train_dir/default_experiment/replay.mp4", "rb").read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

The agent has learned something, but its performance could be better. We would clearly need to train for longer. But let's upload this model to the Hub.

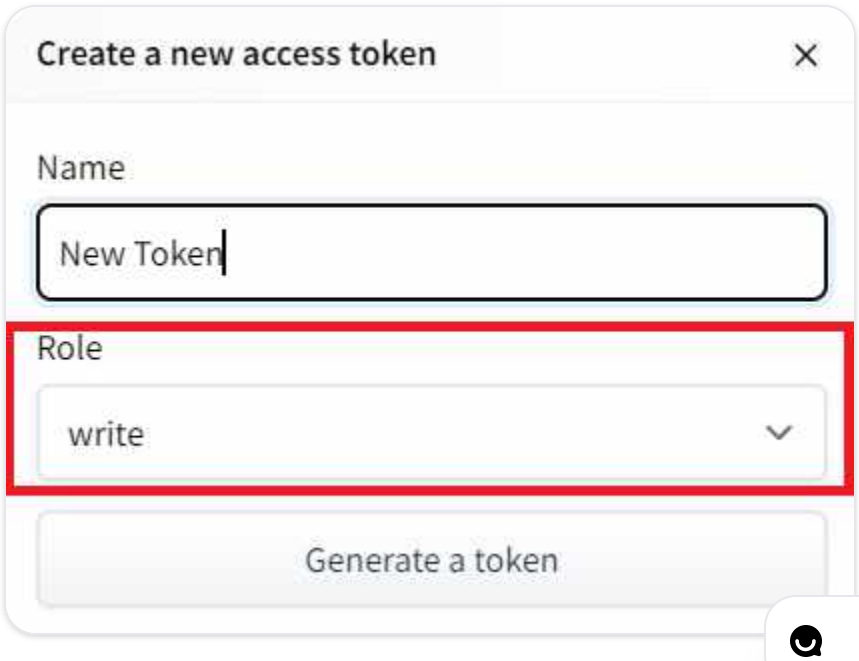
Now lets upload your checkpoint and video to the Hugging Face Hub

To be able to share your model with the community there are three more steps to follow:

1️⃣ (If it's not already done) create an account to HF ➡️ https://huggingface.co/join

2️⃣ Sign in and get your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**



- Copy the token
- Run the cell below and paste the token

If you don't want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

```
from huggingface_hub import notebook_login
notebook_login()
!git config --global credential.helper store
```

```
from sample_factory.enjoy import enjoy

hf_username = "ThomasSimonini"  # insert your HuggingFace username here

cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=10",
        "--max_num_frames=100000",
        "--push_to_hub",
        f"--hf_repository={hf_username}/rl_course_vizdoom_health_gathering_supreme",
    ],
    evaluation=True,
)
status = enjoy(cfg)
```

Let's load another model

This agent's performance was good, but we can do better! Let's download and visualize an agent trained for 10B timesteps from the hub.

```
#download the agent from the hub
python -m sample_factory.huggingface.load_from_h
```

```
ls train_dir/doom_health_gathering_supreme_2222
```

```
env = "doom_health_gathering_supreme"
cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=10",
        "--experiment=doom_health_gathering_supreme_2222",
        "--train_dir=train_dir",
    ],
    evaluation=True,
)
status = enjoy(cfg)
```

```
mp4 = open("/content/train_dir/doom_health_gathering_supreme_2222/replay.mp4", "rb").reac
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

Some additional challenges 🏆 : Doom Deathmatch

Training an agent to play a Doom deathmatch **takes many hours on a more beefy machine than is available in Colab**.

Fortunately, we have have **already trained an agent in this scenario and it is available in the 🤗 Hub!** Let's download the model and visualize the agent's performance

```
# Download the agent from the hub
python -m sample_factory.huggingface.load_from_hub -r edbeeching/doom_deathmatch_bots_222
```

Given the agent plays for a long time the video generation can take **10 minutes**.

```
from sample_factory.enjoy import enjoy

register_vizdoom_components()
env = "doom_deathmatch_bots"
cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=1",
        "--experiment=doom_deathmatch_bots_2222",
        "--train_dir=train_dir",
    ],
    evaluation=True,
)
status = enjoy(cfg)
mp4 = open("/content/train_dir/doom_deathmatch_bots_2222/replay.mp4", "rb").read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

You **can try to train your agent in this environment** using the code above, but not on colab. **Good luck** 🤞

If you prefer an easier scenario, **why not try training in another ViZDoom scenario such as** `doom_deadly_corridor` **or** `doom_defend_the_center`

This concludes the last unit. But we are not finished yet! 🤗 The following **bonus section include some of the most interesting, advanced, and cutting edge work in Deep Reinforcement Learning**.

Keep learning, stay awesome 🤗

<> Update on GitHub

← Introduction

Conclusion →

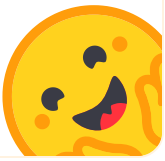
Search models, datasets, users...

Deep RL Course documentation
How do Unity ML-Agents work?

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



How do Unity ML-Agents work?

Before training our agent, we need to understand **what ML-Agents is and how it works**.

What is Unity ML-Agents?

Unity ML-Agents is a toolkit for the game engine Unity that **allows us to create environments using Unity or use pre-made environments to train our agents**.

It's developed by Unity Technologies, the developers of Unity, one of the most famous Game Engines used by the creators of Firewatch, Cuphead, and Cities: Skylines.

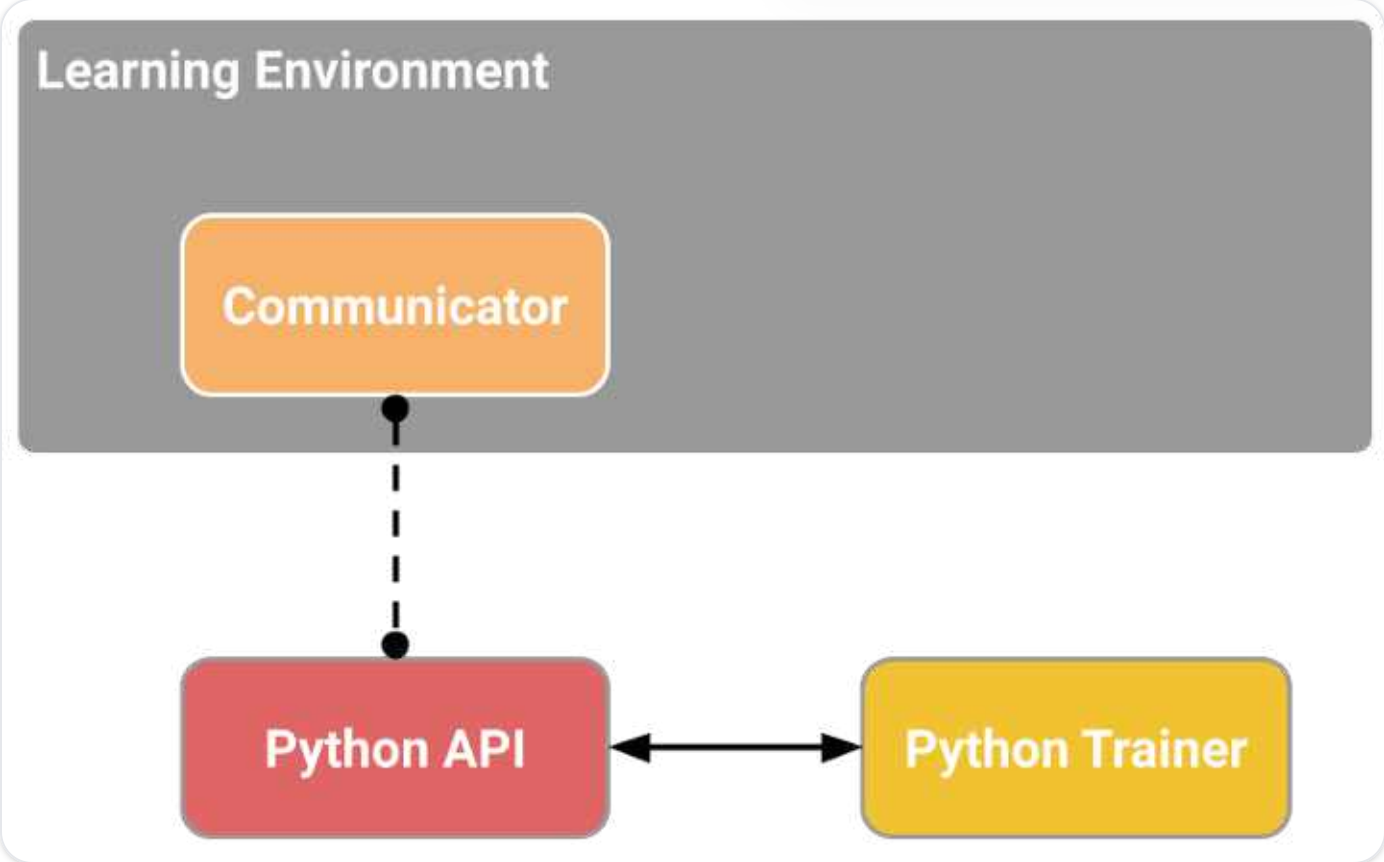
Ask HuggingChat



Firewatch was made with Unity

The six components

With Unity ML-Agents, you have six essential components:



Source: Unity ML-Agents Documentation

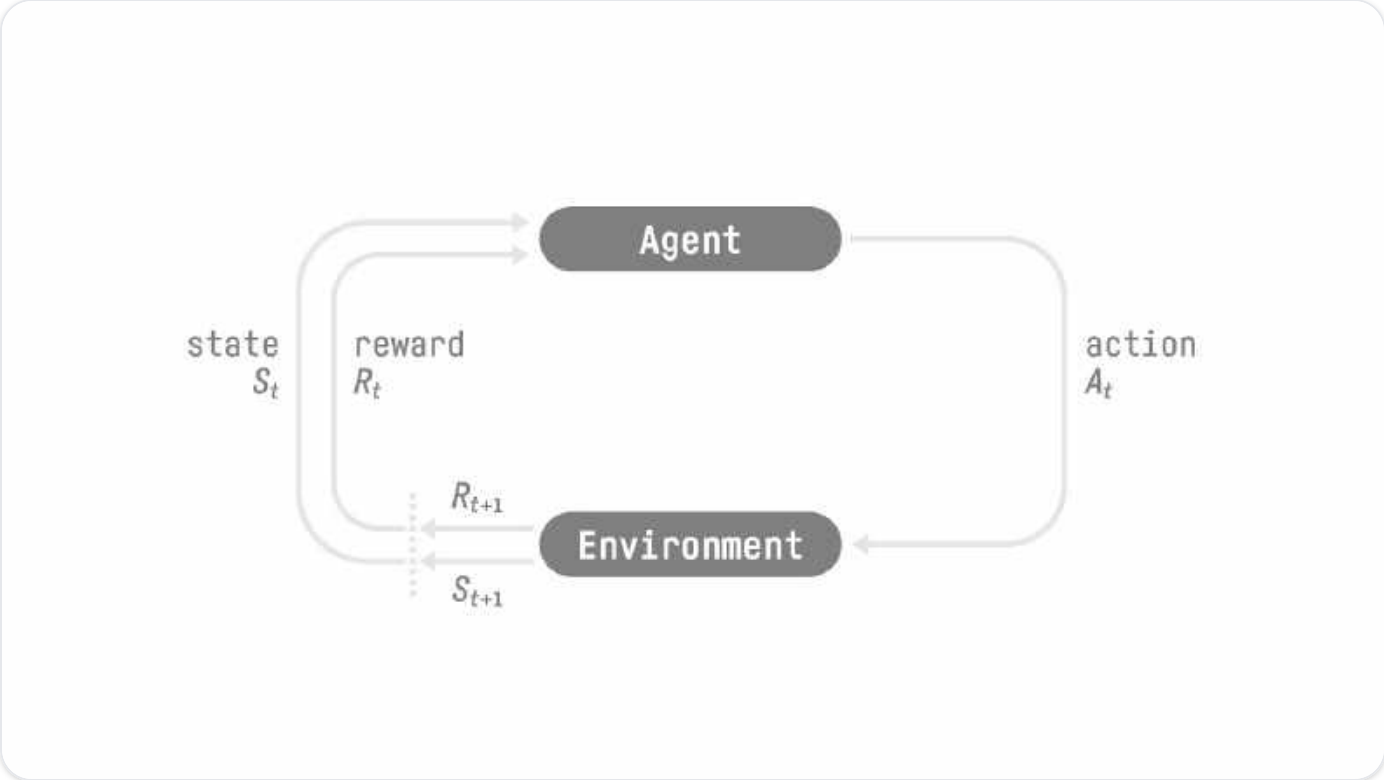
- The first is the *Learning Environment*, which contains **the Unity scene (the environment) and the environment elements** (game characters).
- The second is the *Python Low-level API*, which contains **the low-level Python interface for interacting and manipulating the environment**. It's the API we use to launch the training.
- Then, we have the *External Communicator* that **connects the Learning Environment (made with C#) with the low level Python API (Python)**.
- The *Python trainers*: the **Reinforcement algorithms made with PyTorch (PPO, SAC…)**.
- The *Gym wrapper*: to encapsulate the RL environment in a gym wrapper.
- The *PettingZoo wrapper*: PettingZoo is the multi-agents version of the gym wrapper.

Inside the Learning Component

Inside the Learning Component, we have **two important elements**:

- The first is the *agent component*, the actor of the scene. We'll **train the agent by optimizing its policy** (which will tell us what action to take in each state). The policy is called the *Brain*.
- Finally, there is the *Academy*. This component **orchestrates agents and their decision-making processes**. Think of this Academy as a teacher who handles Python API requests.

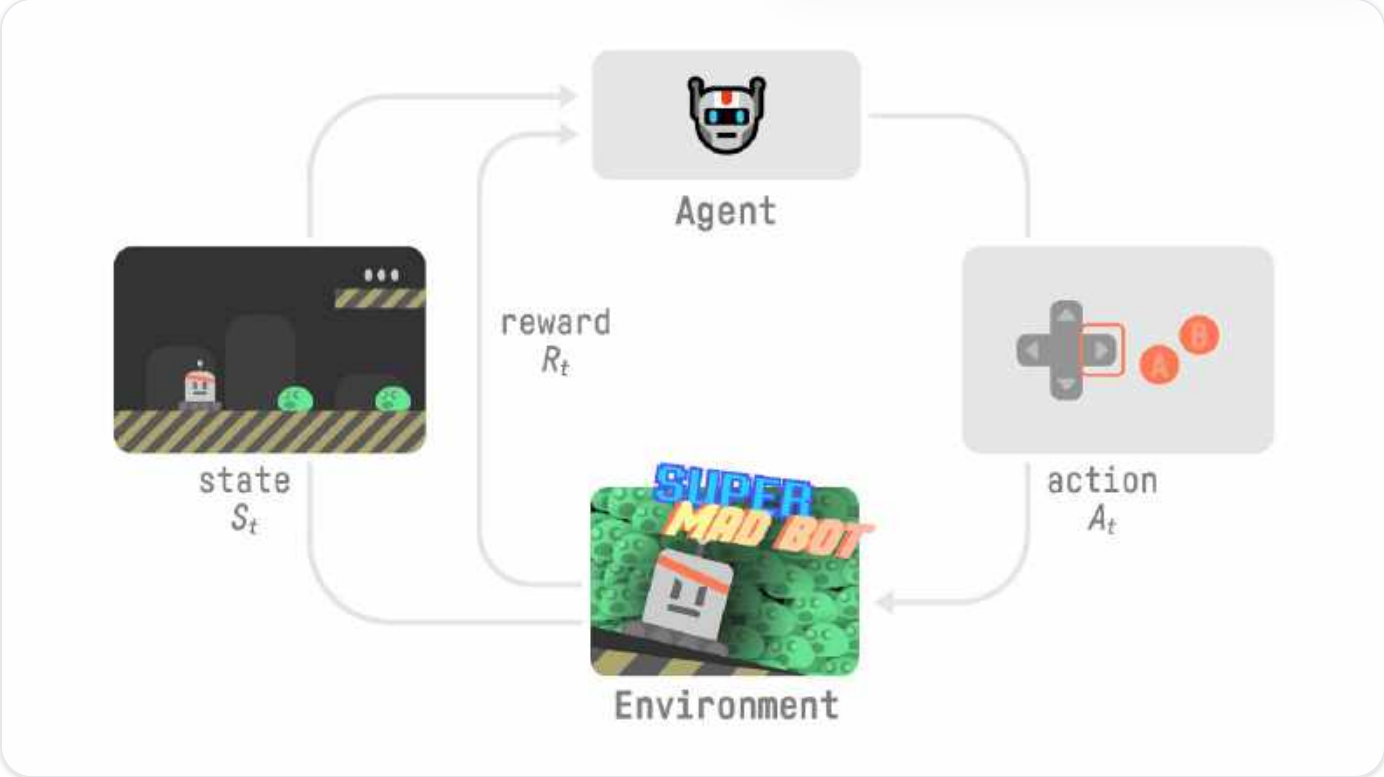
To better understand its role, let's remember the RL process. This can be modeled as a loop that works like this:



The RL Process: a loop of state, action, reward and next state

Source: Reinforcement Learning: An Introduction, Richard Sutton and Andrew G. Barto

Now, let's imagine an agent learning to play a platform game. The RL process looks like this:

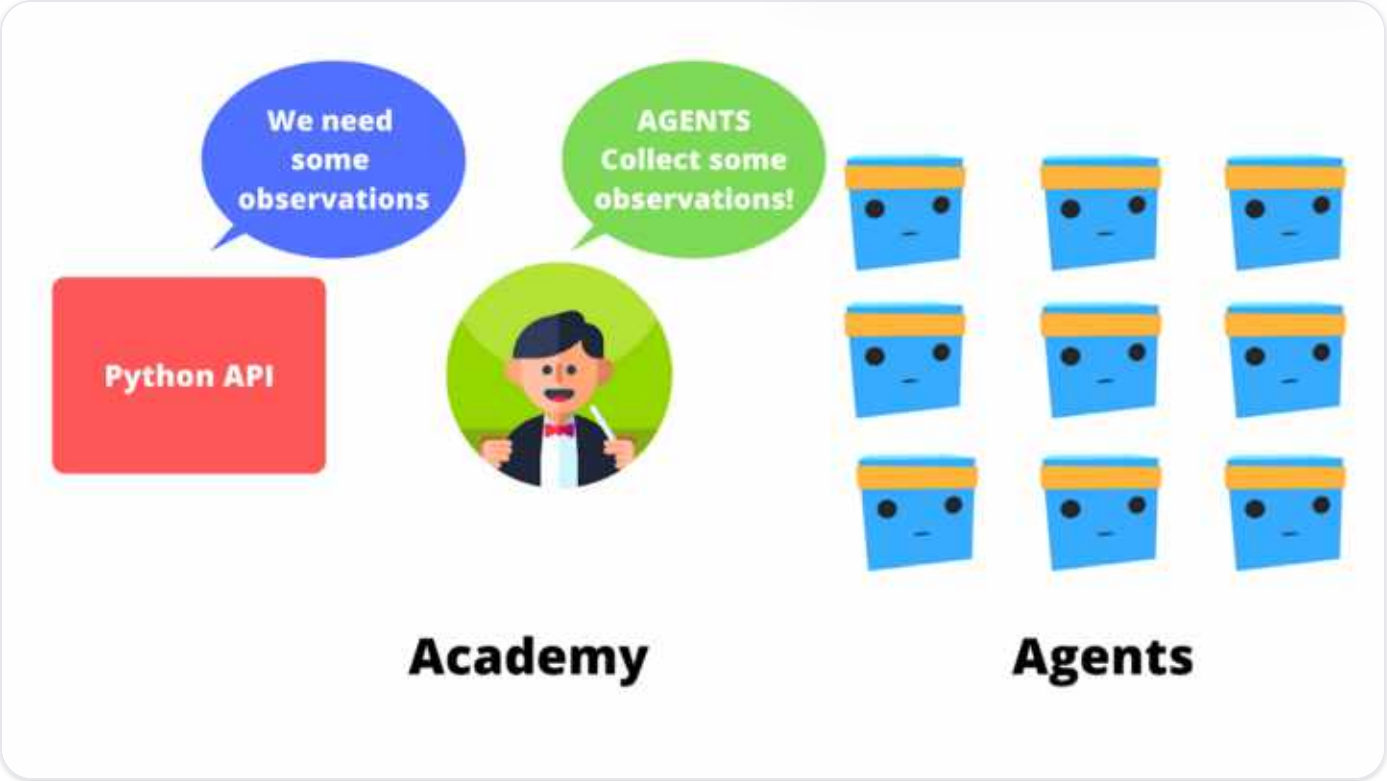


- Our Agent receives **state** S_0 from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state** S_0**,** the Agent takes **action** A_0 — our Agent will move to the right.
- The environment goes to a **new state** S_1 — new frame.
- The environment gives some **reward** R_1 to the Agent — we're not dead *(Positive Reward +1)*.

This RL loop outputs a sequence of **state, action, reward and next state.** The goal of the agent is to **maximize the expected cumulative reward**.

The Academy will be the one that will **send the order to our Agents and ensure that agents are in sync**:

- Collect Observations
- Select your action using your policy
- Take the Action
- Reset if you reached the max step or if you're done.



Now that we understand how ML-Agents works, **we're ready to train our agents.**

<> Update on GitHub

← Introduction

The SnowballTarget environment →

Search models, datasets, users...

Deep RL Course documentation
Interesting Environments to try

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

Interesting Environments to try

Here we provide a list of interesting environments you can try to train your agents on:

DIAMBRA Arena



Ask HuggingChat

DIAMBRA Arena is a software package featuring a collection of high-quality environments for Reinforcement Learning research and experimentation. It provides a standard interface to popular arcade emulated video games, offering a Python API fully compliant with OpenAI Gym/Gymnasium format, that makes its adoption smooth and straightforward.

It supports all major Operating Systems (Linux, Windows and MacOS) and can be easily installed via Python PIP. It is completely free to use, the user only needs to register on the official website.

In addition, its GitHub repository provides a collection of examples covering main use cases of interest that can be run in just a few steps.

Main Features

All environments are episodic Reinforcement Learning tasks, with discrete actions (gamepad buttons) and observations composed by screen pixels plus additional numerical data (RAM values like characters health bars or characters stage side).

They all support both single player (1P) as well as two players (2P) mode, making them the perfect resource to explore Standard RL, Competitive Multi-Agent, Competitive Human-Agent, Self-Play, Imitation Learning and Human-in-the-Loop.

Interfaced games have been selected among the most popular fighting retro-games. While sharing the same fundamental mechanics, they provide different challenges, with specific features such as different type and number of characters, how to perform combos, health bars recharging, etc.

DIAMBRA Arena is built to maximize compatibility will all major Reinforcement Learning libraries. It natively provides interfaces with the two most important packages: Stable Baselines 3 and Ray RLlib, while Stable Baselines is also available but deprecated. Their usage is illustrated in the official documentation and in the DIAMBRA Agents examples repository. It can easily be interfaced with any other package in a similar way.

Competition Platform

DIAMBRA also provides a competition platform fully integrated with the Hugging Face Hub, on which you can submit your trained agents and compet

video games tournaments!

It features a public leaderboard where users are ranked by the best score achieved by their agents in our different environments.

It also offers the possibility to unlock cool achievements depending on the performances of your agent.

Submitted agents are evaluated and their episodes are streamed on DIAMBRA Twitch channel.

References

To start using this environment, check these resources:

- Official Docs
- Competition Platform
- GitHub
- Discord

MineRL



MineRL is a Python library that provides a Gym interface for interacting with the video game Minecraft, accompanied by datasets of human gameplay. Every year there are challenges with this library. Check the website

To start using this environment, check these resources:

- What is MineRL?
- First steps in MineRL
- MineRL documentation and tutorials

DonkeyCar Simulator



Donkey is a Self Driving Car Platform for hobby remote control cars. This simulator version is built on the Unity game platform. It uses their internal physics and graphics and connects to a donkey Python process to use our trained model to control the simulated Donkey (car).

To start using this environment, check these resources:

- DonkeyCar Simulator documentation
- Learn to Drive Smoothly (Antonin Raffin's tutorial) Part 1
- Learn to Drive Smoothly (Antonin Raffin's tutorial) Part 2
- Learn to Drive Smoothly (Antonin Raffin's tutorial) Part 3
- Pretrained agents:
  - https://huggingface.co/araffin/tqc-donkey-mountain-track-v0
  - https://huggingface.co/araffin/tqc-donkey-avc-sparkfun-v0
  - https://huggingface.co/araffin/tqc-donkey-minimonaco-track-v0

Starcraft II



Starcraft II is a famous *real-time strategy game*. DeepMind has used this game for their Deep Reinforcement Learning research with Alphastar

To start using this environment, check these resources:

- Starcraft gym
- A. I. Learns to Play Starcraft 2 (Reinforcement Learning) tutorial

Author

This section was written by Thomas Simonini

<> Update on GitHub

← (Automatic) Curriculum Learning for RL

An introduction to Unreal Learning Agents →

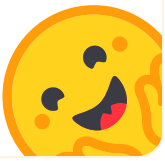
Search models, datasets, users...

Deep RL Course documentation
Introducing Q-Learning

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Introducing Q-Learning

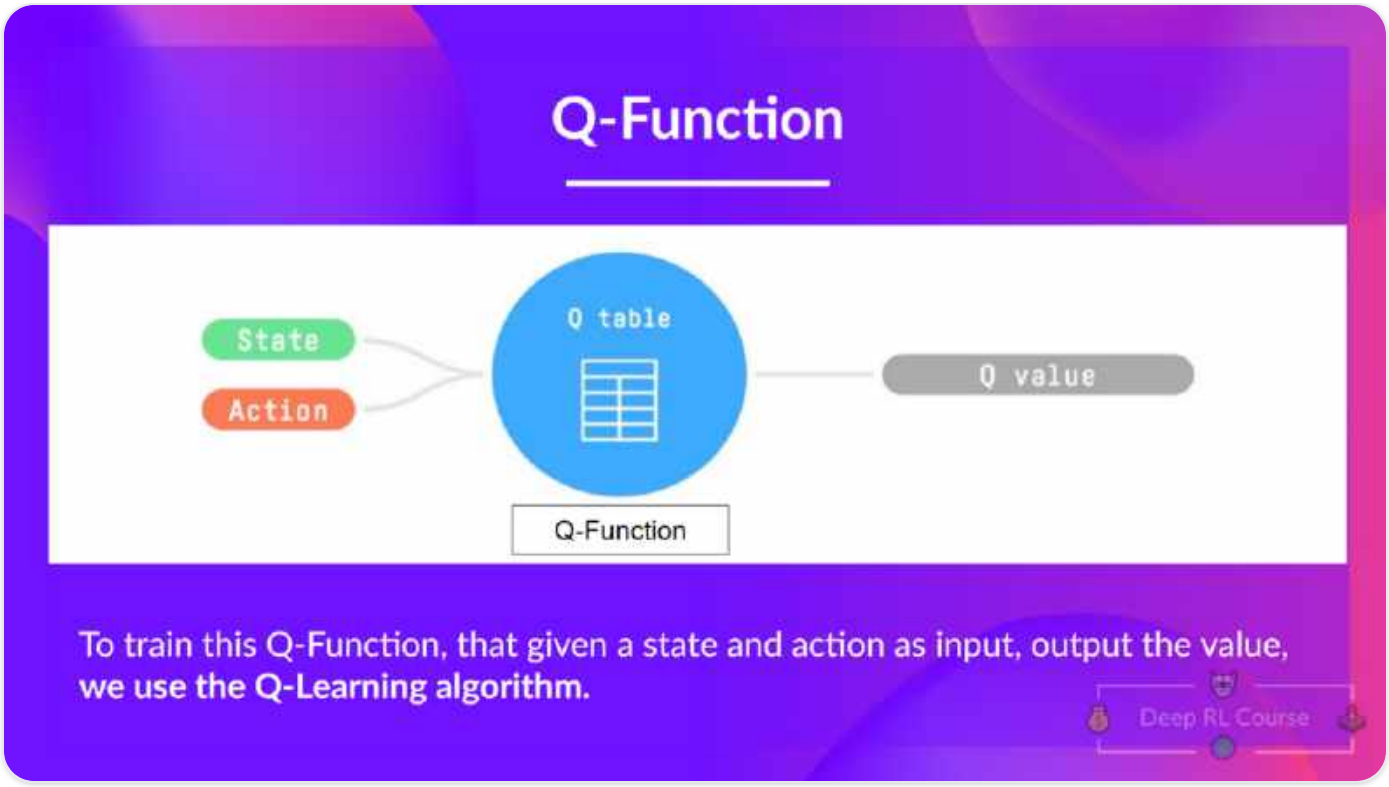
What is Q-Learning?

Q-Learning is an **off-policy value-based method that uses a TD approach to train its action-value function:**

- *Off-policy*: we'll talk about that at the end of this unit.
- *Value-based method*: finds the optimal policy indirectly by training a value or action-value function that will tell us **the value of each state or each state-action pair.**
- *TD approach:* **updates its action-value function at each step instead of at the end of the episode.**

Q-Learning is the algorithm we use to train our Q-function, an **action-value function** that determines the value of being at a particular state and taking a specific action at that state.

Ask HuggingChat



Given a state and action, our Q Function outputs a state-action value (also called Q-value)

The **Q comes from "the Quality" (the value) of that action at that state.**

Let's recap the difference between value and reward:

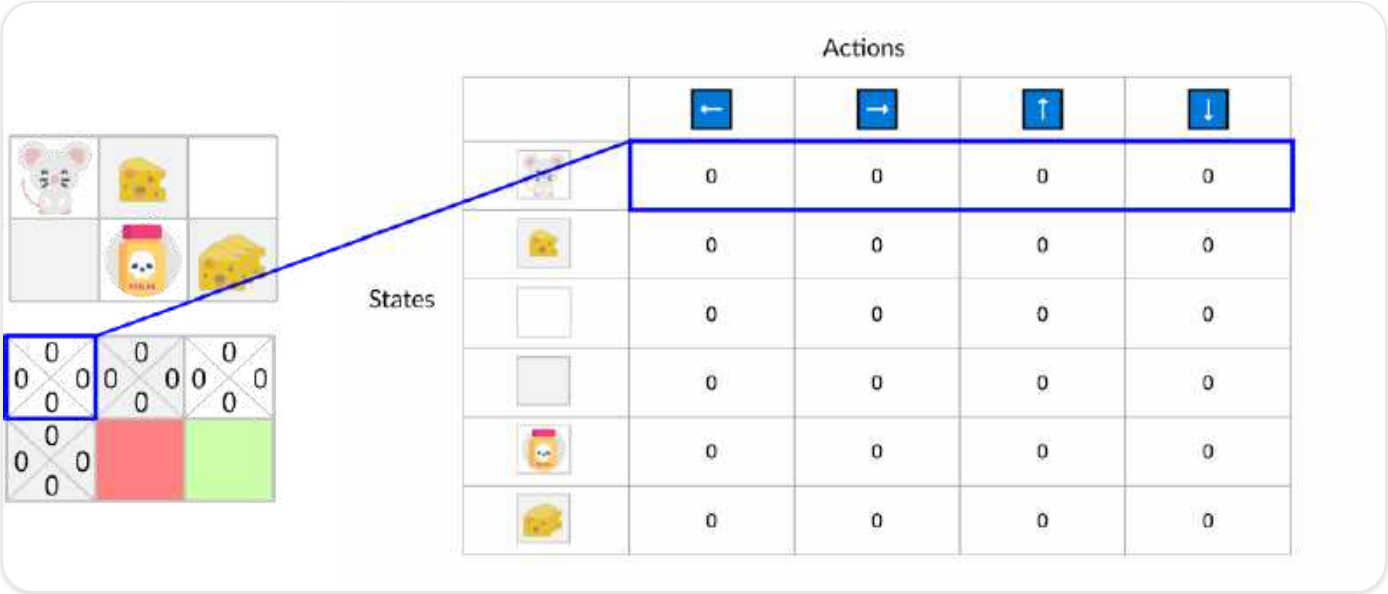
- The *value of a state*, or a *state-action pair* is the expected cumulative reward our agent gets if it starts at this state (or state-action pair) and then acts accordingly to its policy.
- The *reward* is the **feedback it gets from the environment** after performing an action at a state.

Internally, our Q-function is encoded by **a Q-table, a table where each cell corresponds to a state-action pair value.** Think of this Q-table as **the memory or cheat sheet of our Q-function.**

Let's go through an example of a maze.



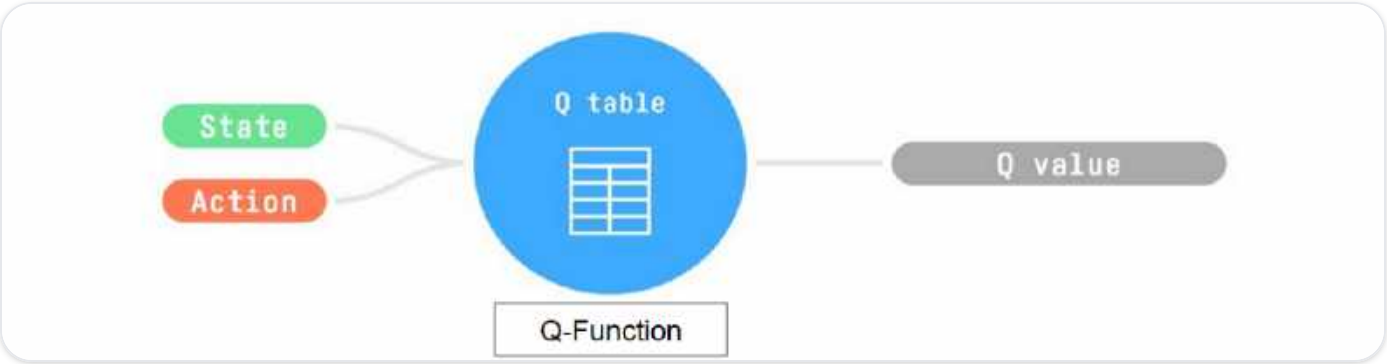
The Q-table is initialized. That's why all values are = 0. This table **contains, for each state and action, the corresponding state-action values.** For this simple example, the state is only defined by the position of the mouse. Therefore, we have 2*3 rows in our Q-table, one row for each possible position of the mouse. In more complex scenarios, the state could contain more information than the position of the actor.



Here we see that the **state-action value of the initial state and going up is 0:**

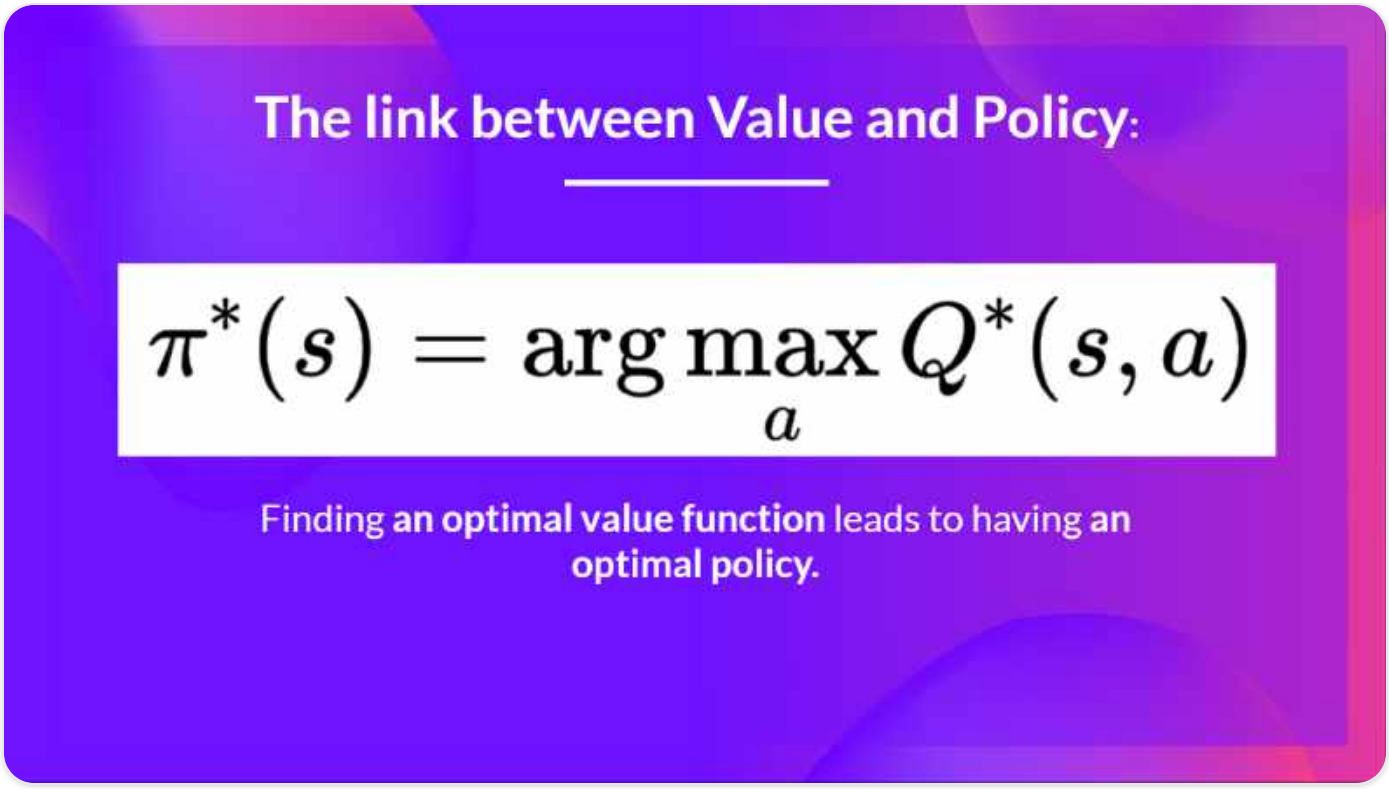


So: the Q-function uses a Q-table **that has the value of each state-action pair.** Given a state and action, **our Q-function will search inside its Q-table to output the value.**

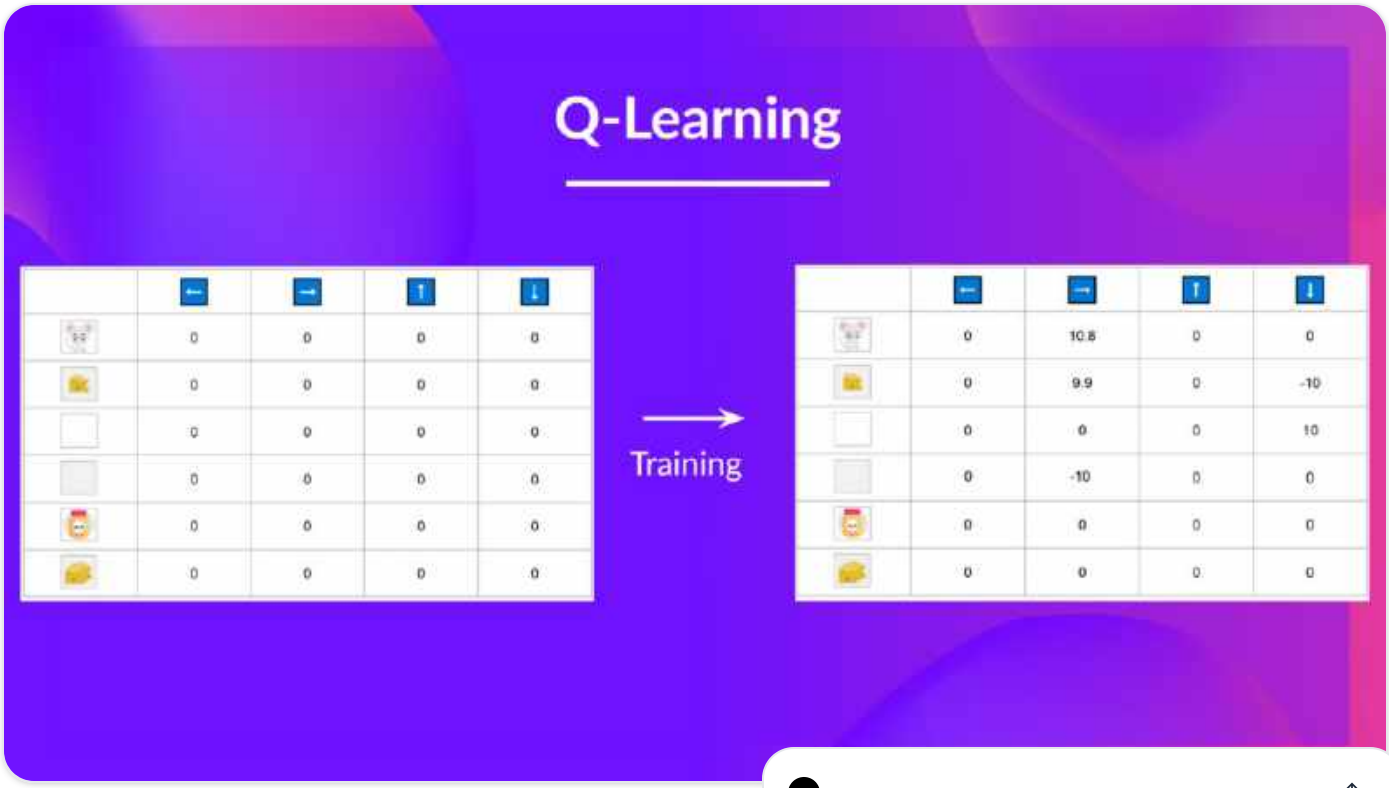


If we recap, *Q-Learning* **is the RL algorithm that:**

- Trains a *Q-function* (an **action-value function**), which internally is a **Q-table that contains all the state-action pair values.**
- Given a state and action, our Q-function **will search its Q-table for the corresponding value.**
- When the training is done, **we have an optimal Q-function, which means we have optimal Q-table.**
- And if we **have an optimal Q-function**, we **have an optimal policy** since we **know the best action to take at each state.**



In the beginning, **our Q-table is useless since it gives arbitrary values for each state-action pair** (most of the time, we initialize the Q-table to 0). As the agent **explores the environment and we update the Q-table, it will give us a better and better approximation** to the optimal policy.

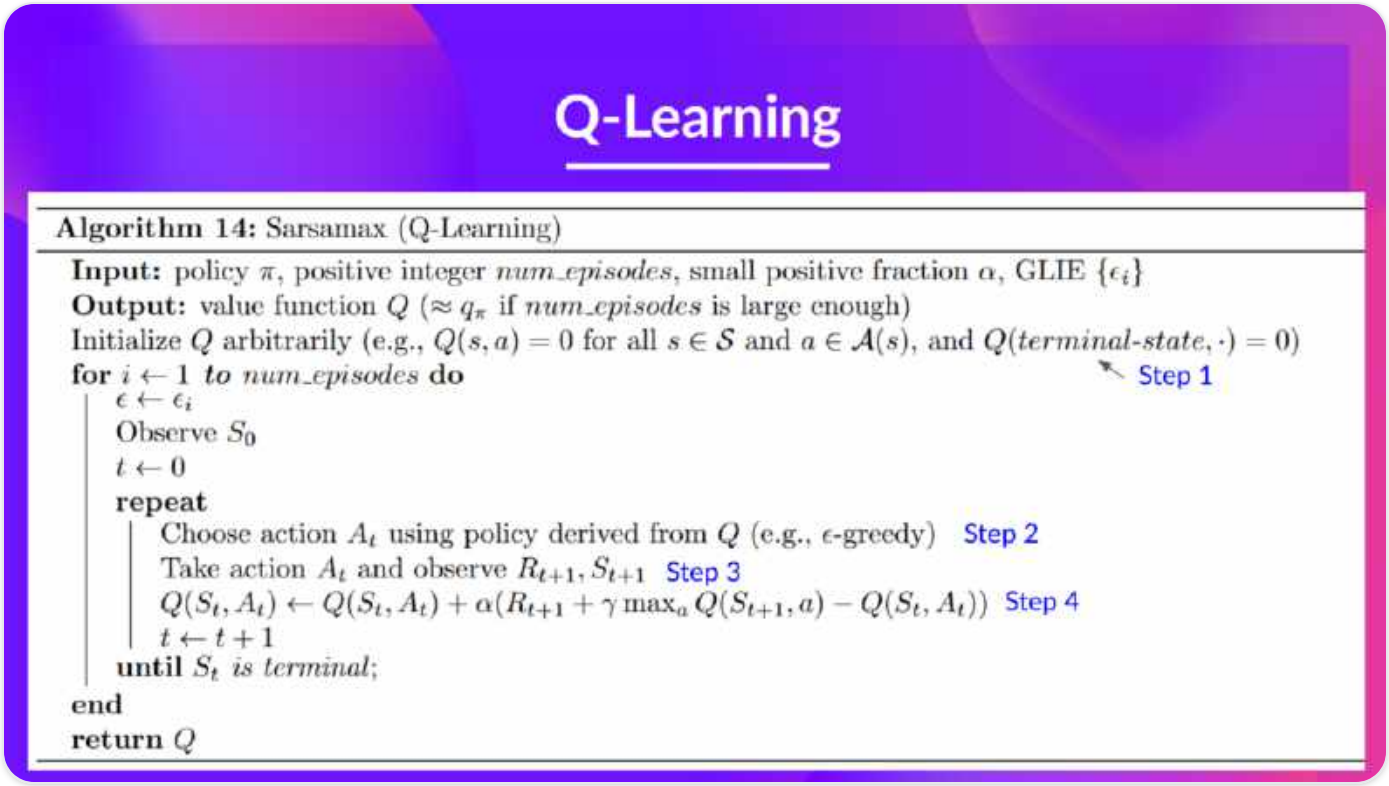


We see here that with the training, our Q-table is better since, thanks to it, we can know the value of each state-action pair.

Now that we understand what Q-Learning, Q-functions, and Q-tables are, **let's dive deeper into the Q-Learning algorithm**.

The Q-Learning algorithm

This is the Q-Learning pseudocode; let's study each part and **see how it works with a simple example before implementing it.** Don't be intimidated by it, it's simpler than it looks! We'll go over each step.

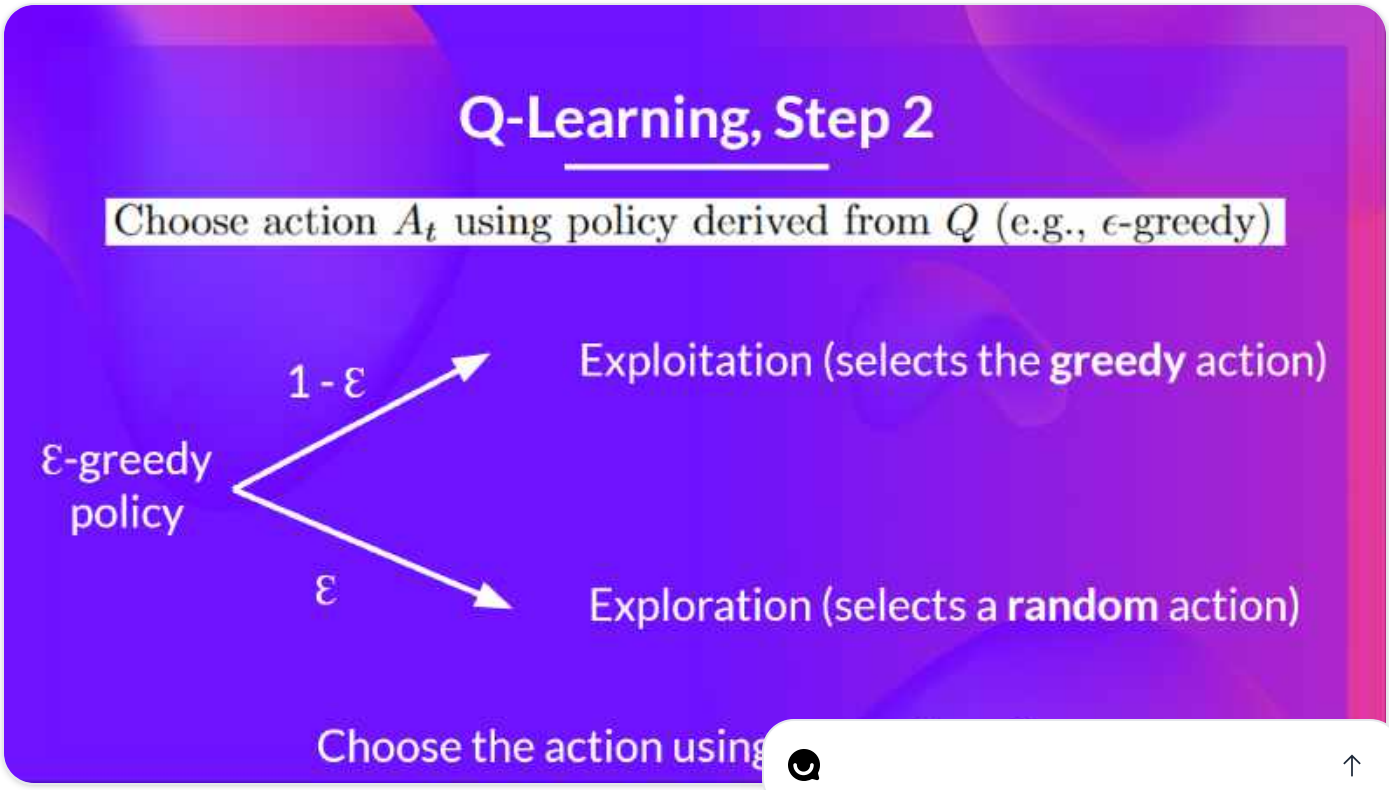


Step 1: We initialize the Q-table



We need to initialize the Q-table for each state-action pair. **Most of the time, we initialize with values of 0.**

Step 2: Choose an action using the epsilon-greedy strategy

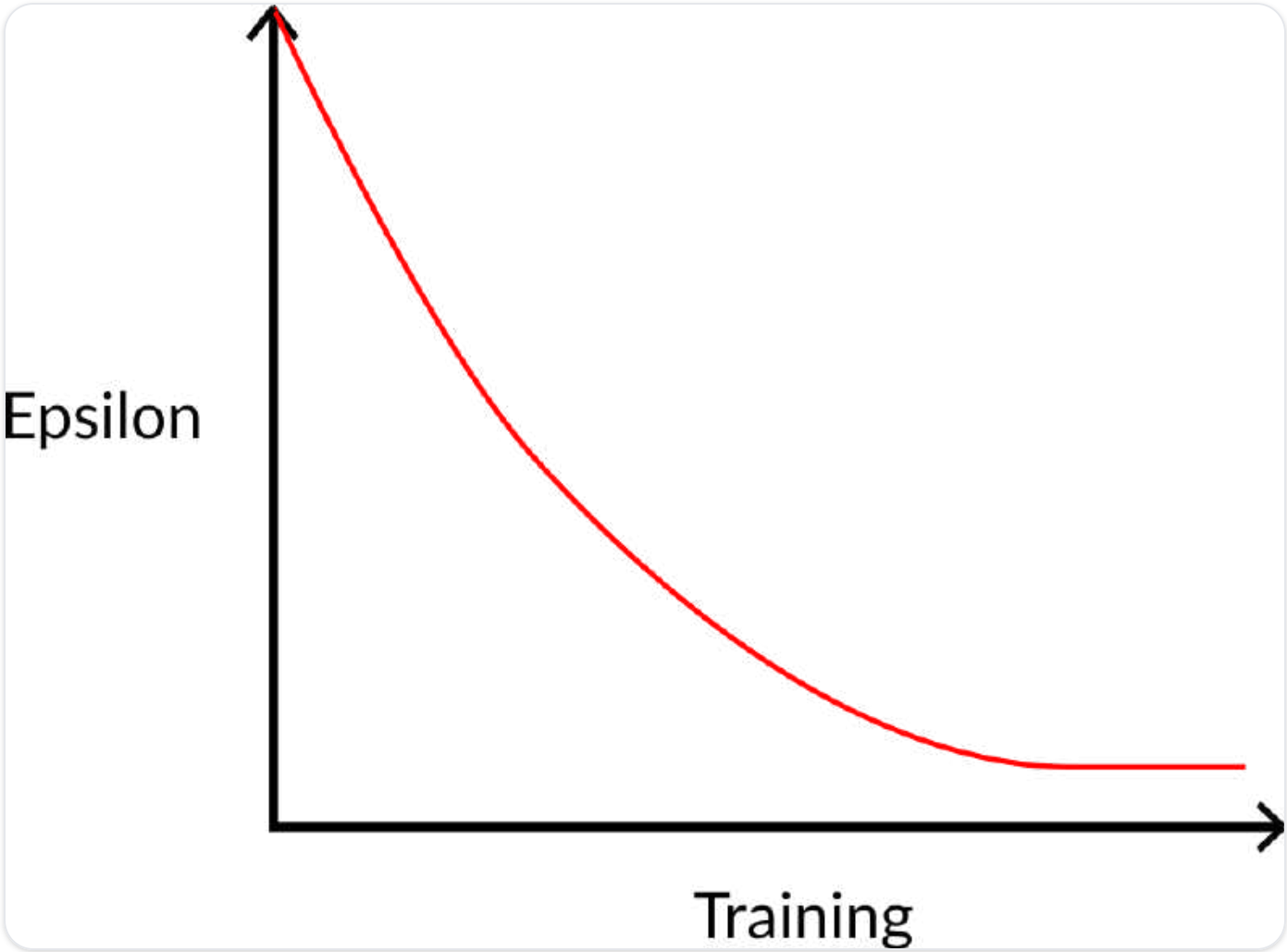


The epsilon-greedy strategy is a policy that handles the exploration/exploitation trade-off.

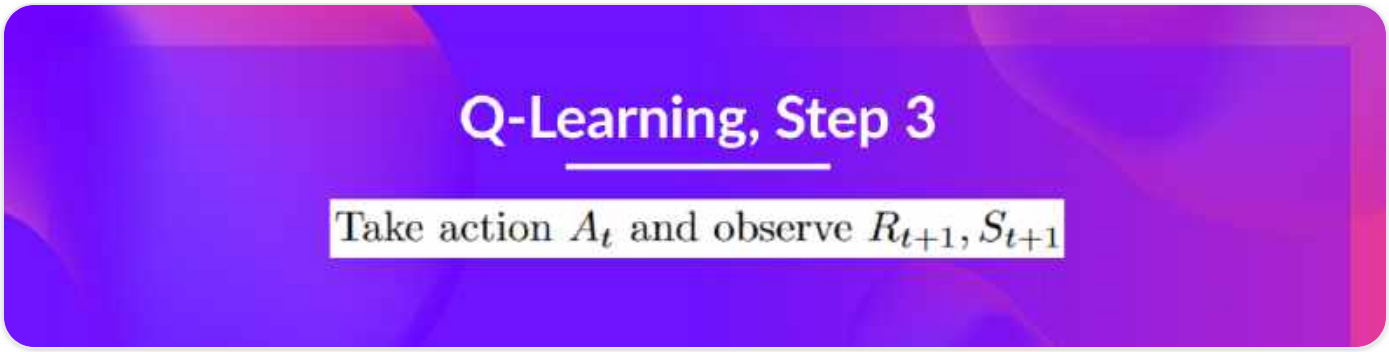
The idea is that, with an initial value of ε = 1.0:

- *With probability 1 — ε* : we do **exploitation** (aka our agent selects the action with the highest state-action pair value).
- With probability ε: **we do exploration** (trying random action).

At the beginning of the training, **the probability of doing exploration will be huge since ε is very high, so most of the time, we'll explore.** But as the training goes on, and consequently our **Q-table gets better and better in its estimations, we progressively reduce the epsilon value** since we will need less and less exploration and more exploitation.



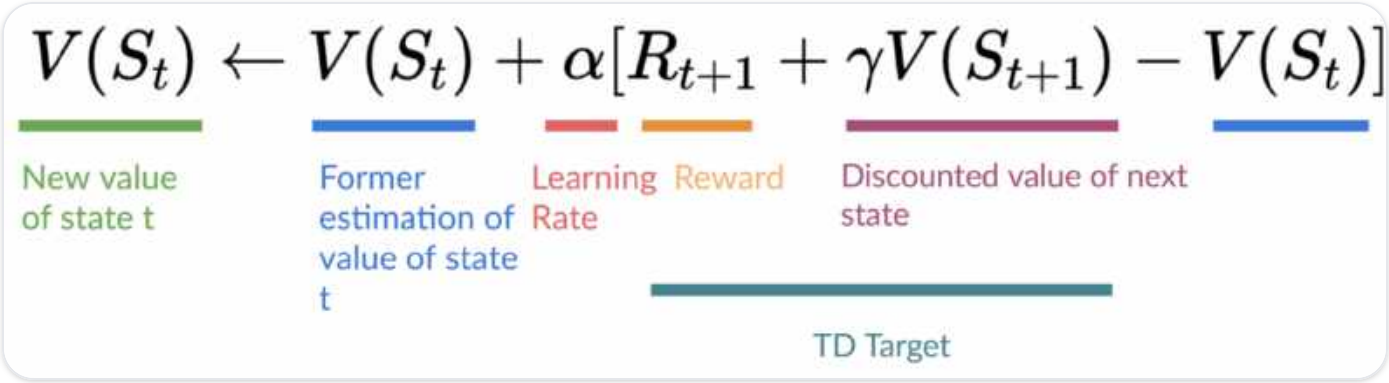
Step 3: Perform action At, get reward Rt+1 and ne



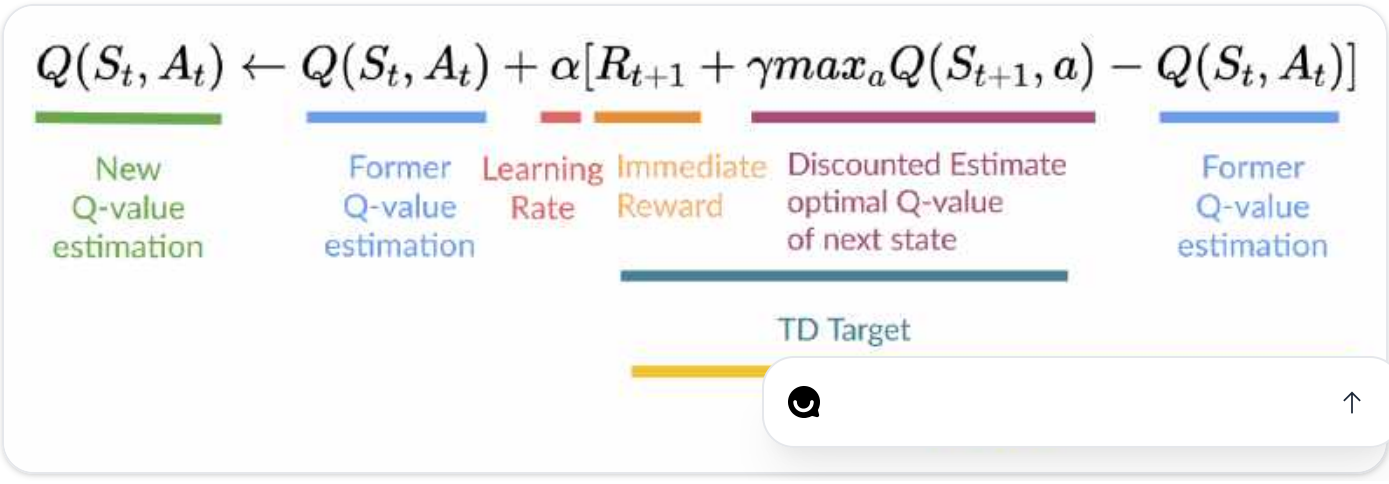
Step 4: Update Q(St, At)

Remember that in TD Learning, we update our policy or value function (depending on the RL method we choose) **after one step of the interaction.**

To produce our TD target, **we used the immediate reward** R_{t+1} **plus the discounted value of the next state**, computed by finding the action that maximizes the current Q-function at the next state. (We call that bootstrap).



Therefore, our $Q(S_t, A_t)$ **update formula goes like this:**



This means that to update our $Q(S_t, A_t)$:

- We need $S_t, A_t, R_{t+1}, S_{t+1}$.
- To update our Q-value at a given state-action pair, we use the TD target.

How do we form the TD target?

1. We obtain the reward R_{t+1} after taking the action A_t.
2. To get the **best state-action pair value** for the next state, we use a greedy policy to select the next best action. Note that this is not an epsilon-greedy policy, this will always take the action with the highest state-action value.

Then when the update of this Q-value is done, we start in a new state and select our action **using a epsilon-greedy policy again.**

This is why we say that Q Learning is an off-policy algorithm.

Off-policy vs On-policy

The difference is subtle:

- *Off-policy*: using **a different policy for acting (inference) and updating (training).**

For instance, with Q-Learning, the epsilon-greedy policy (acting policy), is different from the greedy policy that is **used to select the best next-state action value to update our Q-value (updating policy).**

$$\text{Choose action } A_t \text{ using policy derived from } Q \text{ (e.g., } \epsilon\text{-greedy)}$$

Acting Policy

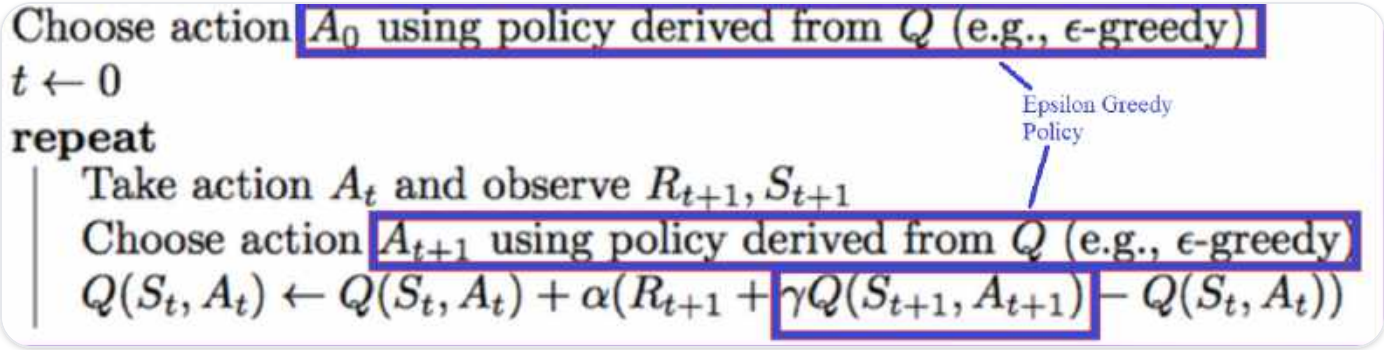
Is different from the policy we use during the training part:

$$\gamma \max_a Q(S_{t+1}, a)$$

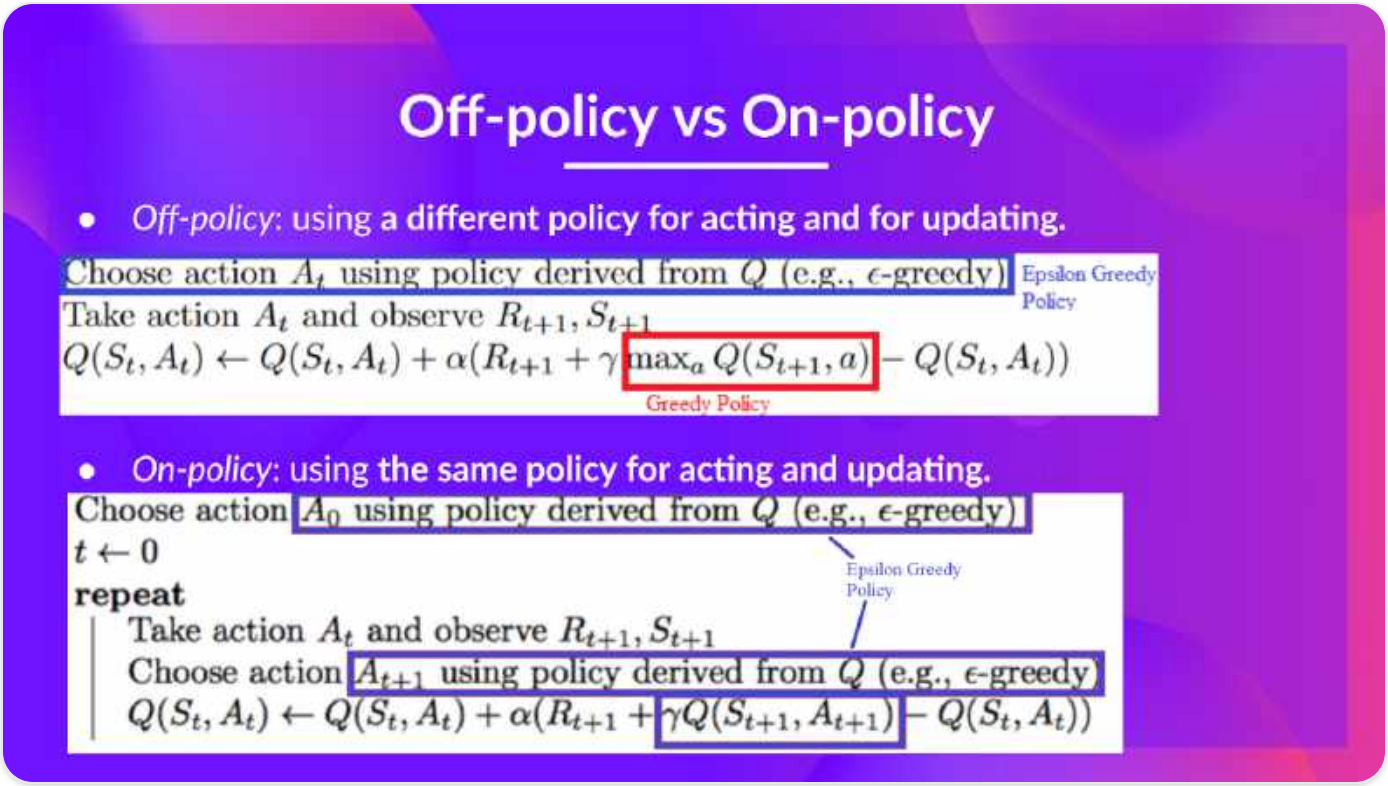
Updating policy

- *On-policy:* using the **same policy for acting and updating.**

For instance, with Sarsa, another value-based algorithm, **the epsilon-greedy policy selects the next state-action pair, not a greedy policy.**



Sarsa



<> Update on GitHub

← Mid-way Quiz

A Q-Learning example →

Search models, datasets, users...

Deep RL Course documentation
Introducing the Clipped Surrogate Objective Function

Join the Hugging Face community

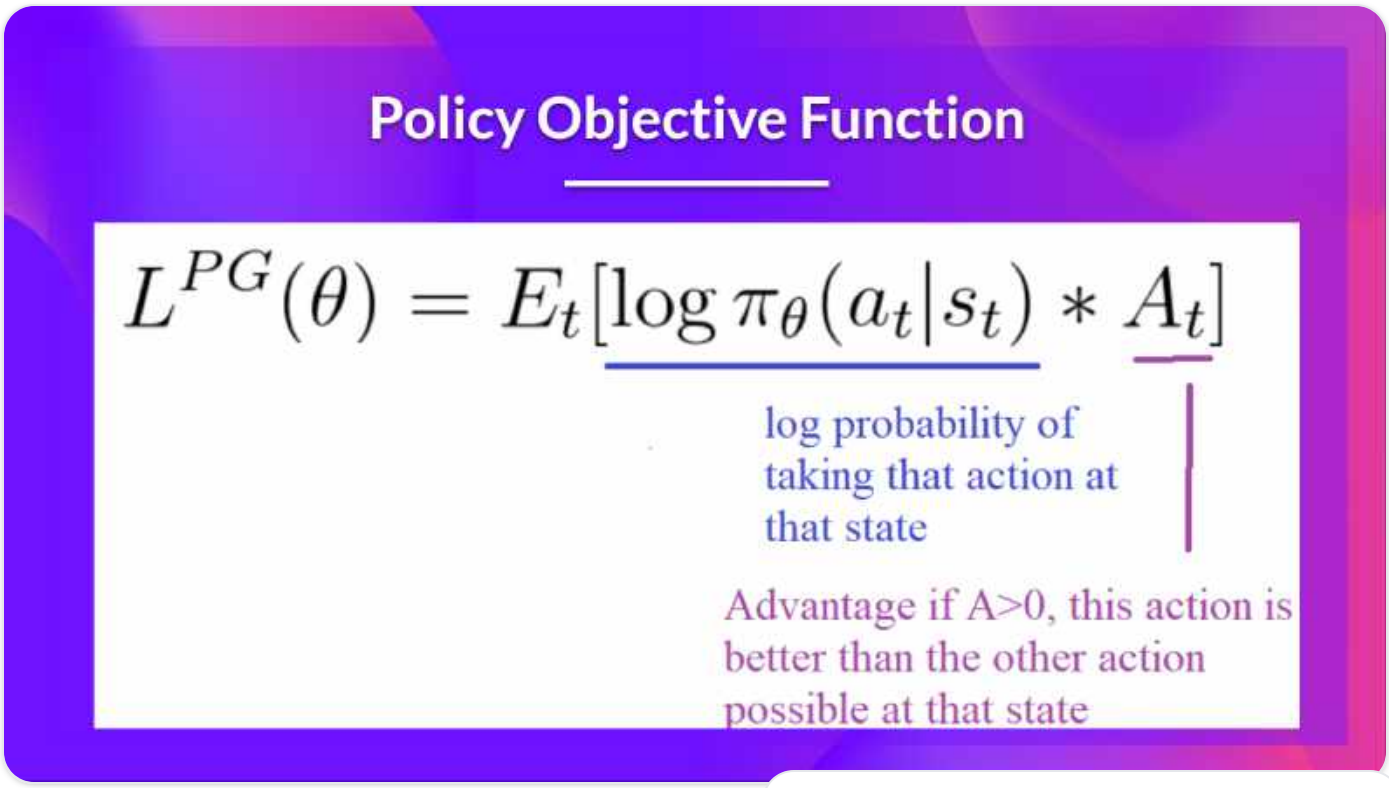
and get access to the augmented documentation experience

Sign Up to get started

Introducing the Clipped Surrogate Objective Function

Recap: The Policy Objective Function

Let’s remember what the objective is to optimize in Reinforce:



Ask HuggingChat

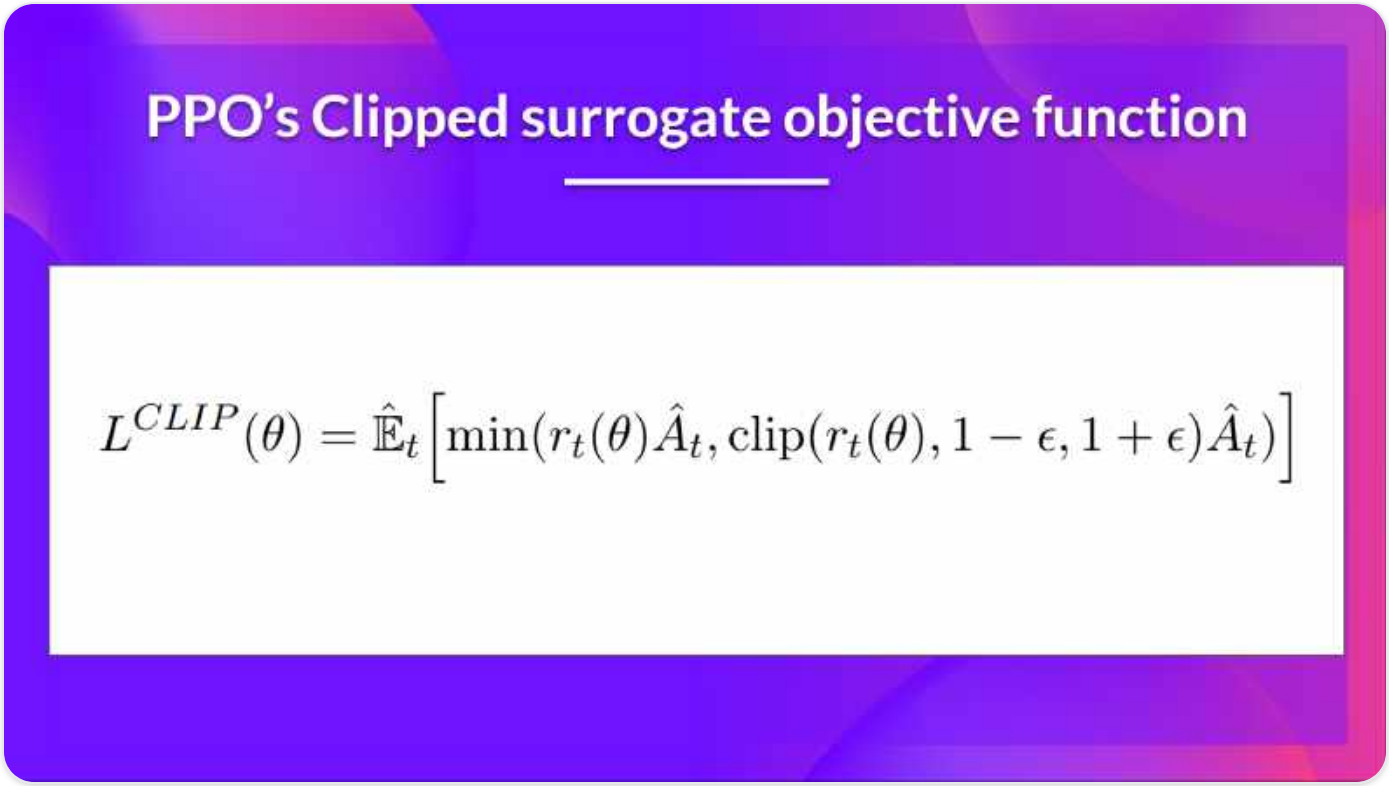
The idea was that by taking a gradient ascent step on this function (equivalent to taking gradient descent of the negative of this function), we would **push our agent to take actions that lead to higher rewards and avoid harmful actions.**

However, the problem comes from the step size:

- Too small, **the training process was too slow**
- Too high, **there was too much variability in the training**

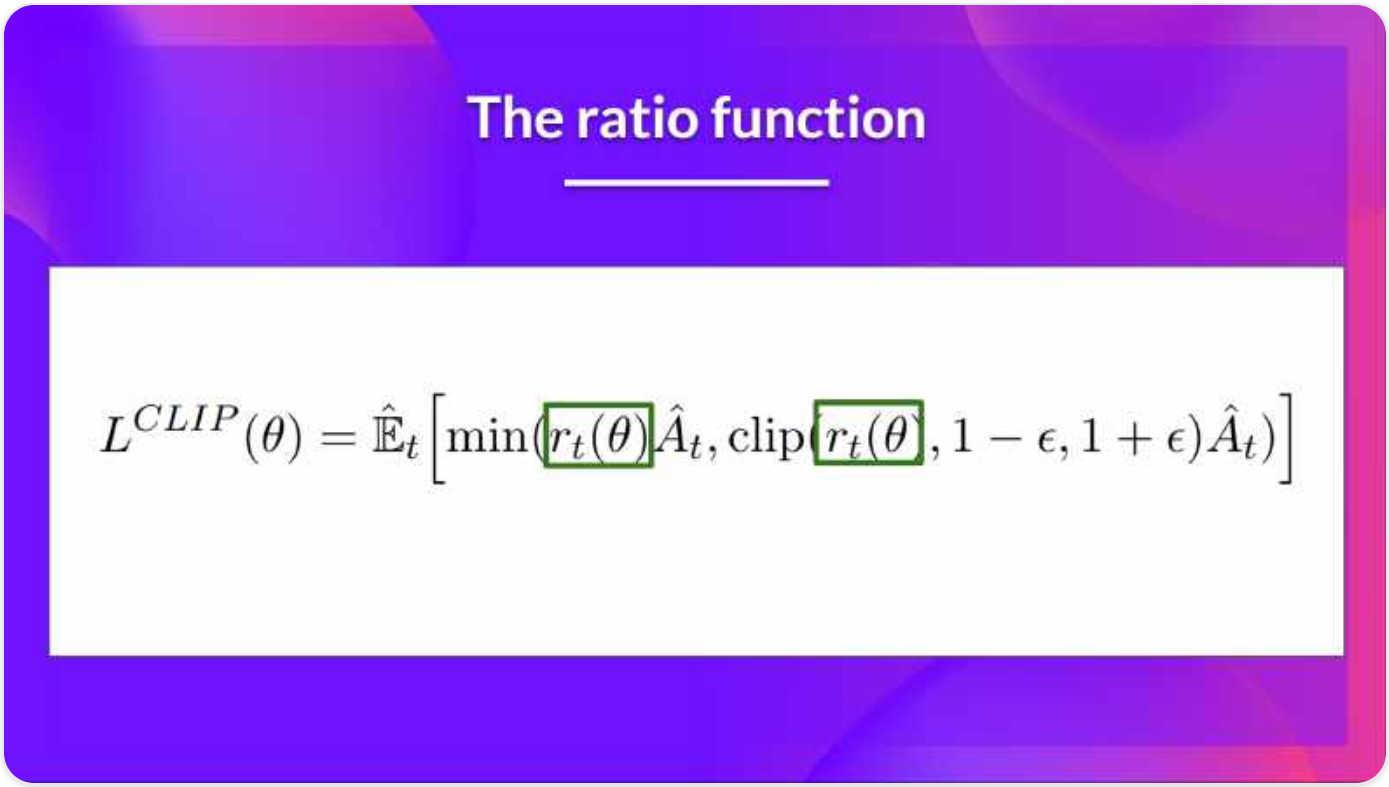
With PPO, the idea is to constrain our policy update with a new objective function called the *Clipped surrogate objective function* that **will constrain the policy change in a small range using a clip.**

This new function **is designed to avoid destructively large weights updates** :

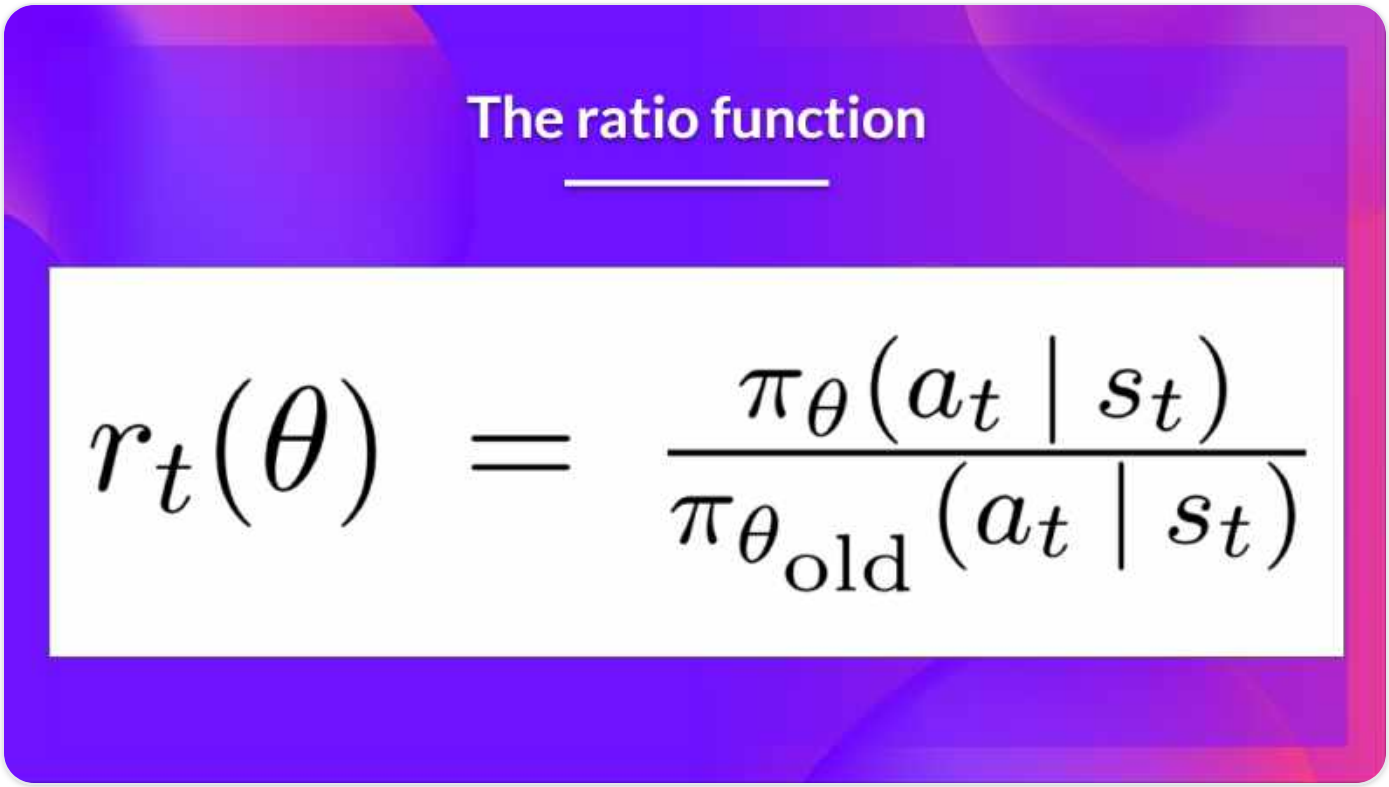


Let's study each part to understand how it works.

The Ratio Function



This ratio is calculated as follows:



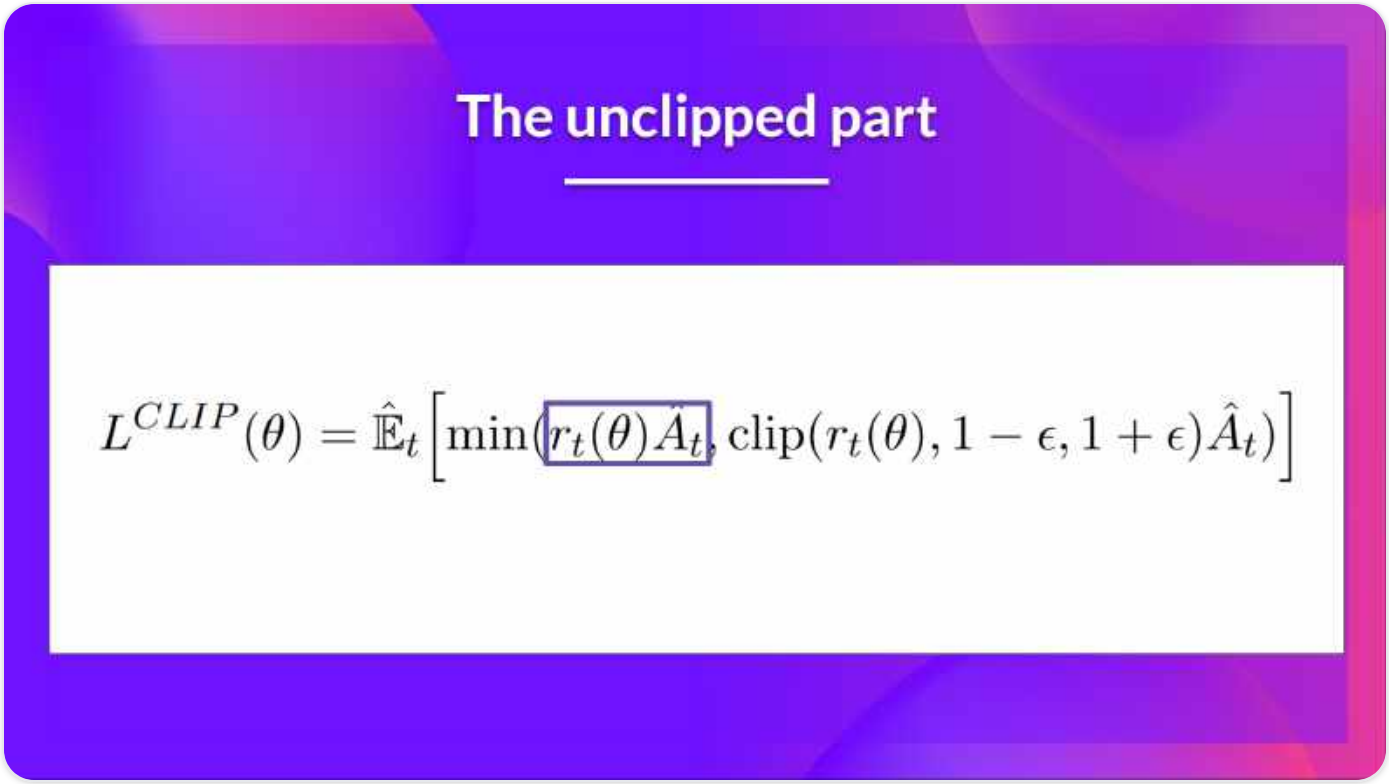
It's the probability of taking action a_t at state s_t in the c
previous policy.

As we can see, $r_t(\theta)$ denotes the probability ratio between the current and old policy:

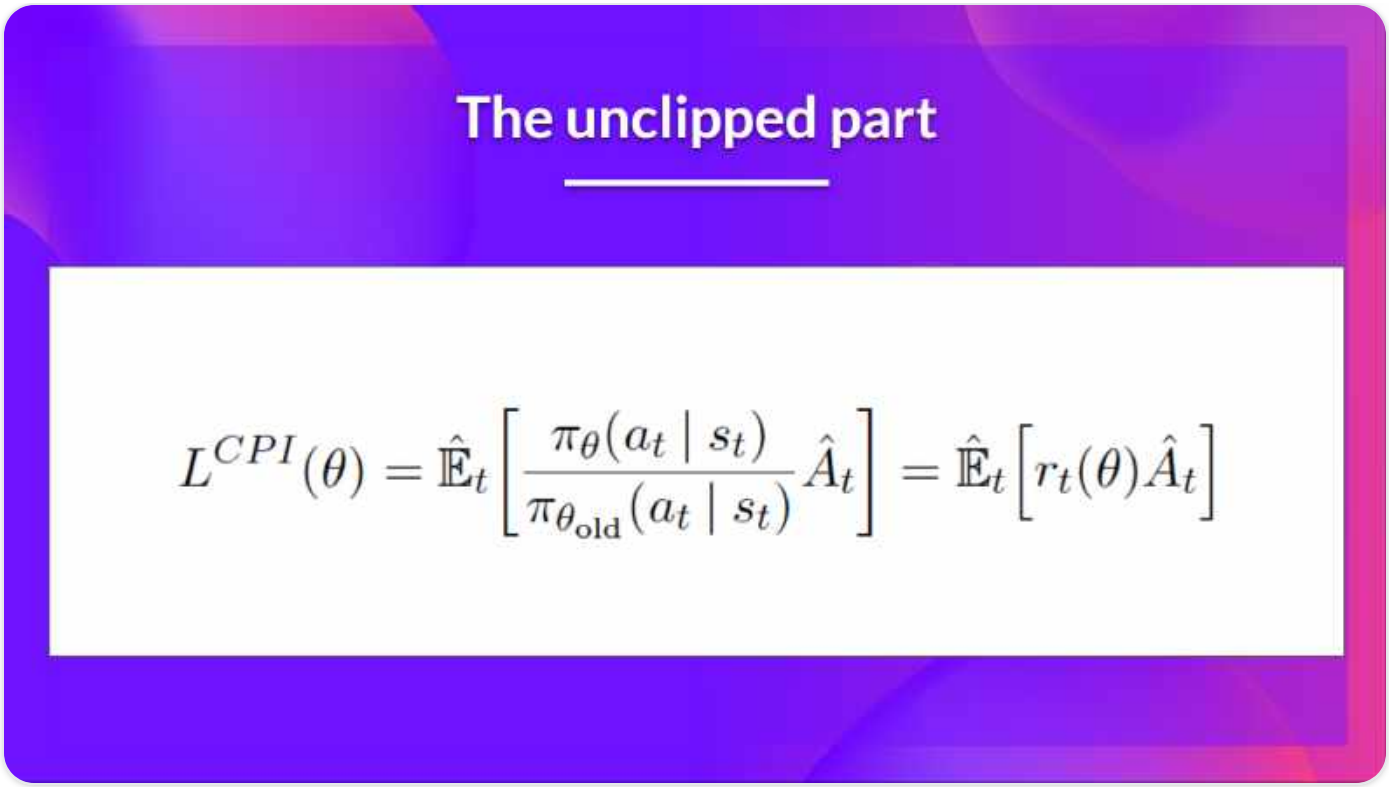
- If $r_t(\theta) > 1$, the **action a_t at state s_t is more likely in the current policy than the old policy.**
- If $r_t(\theta)$ is between 0 and 1, the **action is less likely for the current policy than for the old one**.

So this probability ratio is an **easy way to estimate the divergence between old and current policy.**

The unclipped part of the Clipped Surrogate Objective function



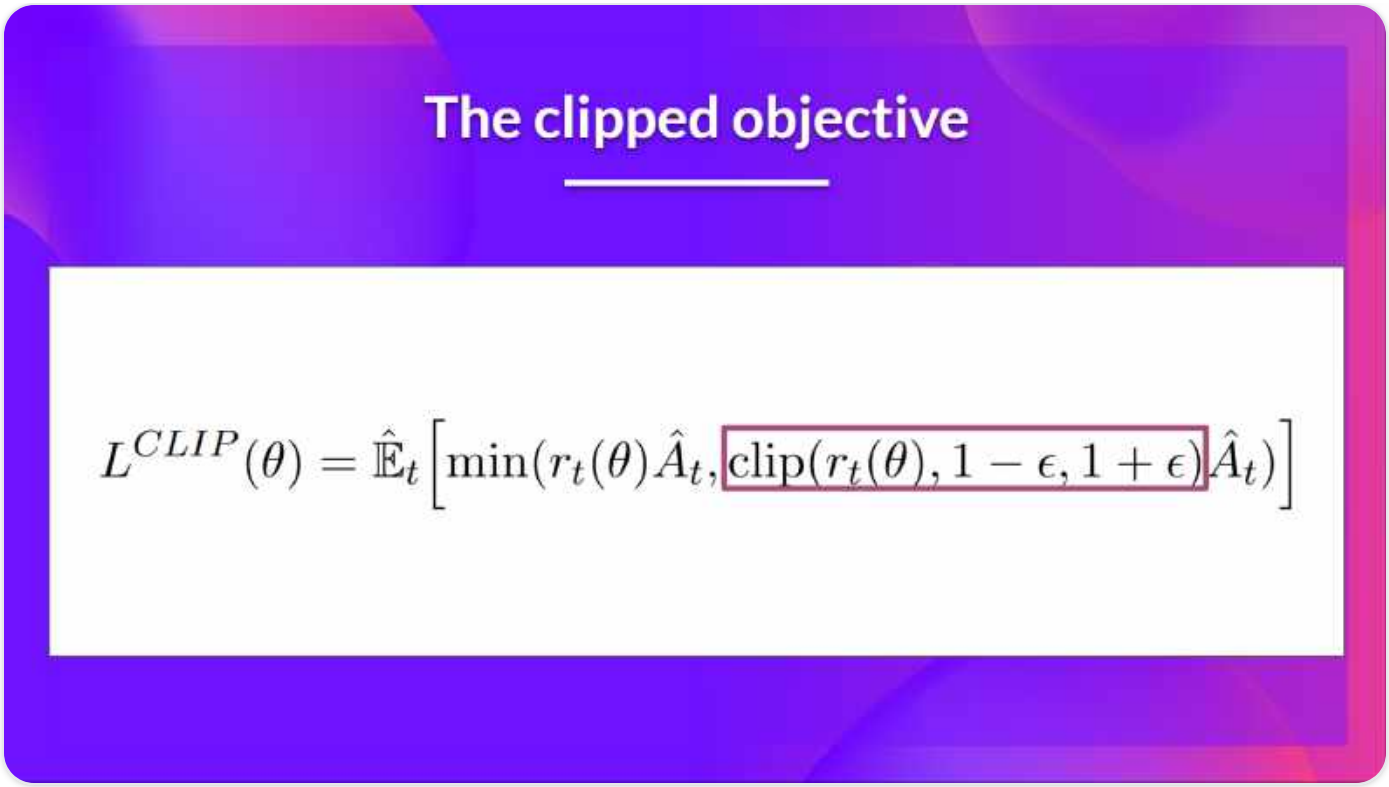
This ratio **can replace the log probability we use in the policy objective function**. This gives us the left part of the new objective function: multiplying the ratio by the advantage.



Proximal Policy Optimization Algorithms

However, without a constraint, if the action taken is much more probable in our current policy than in our former, **this would lead to a significant policy gradient step** and, therefore, an **excessive policy update.**

The clipped Part of the Clipped Surrogate Objective function

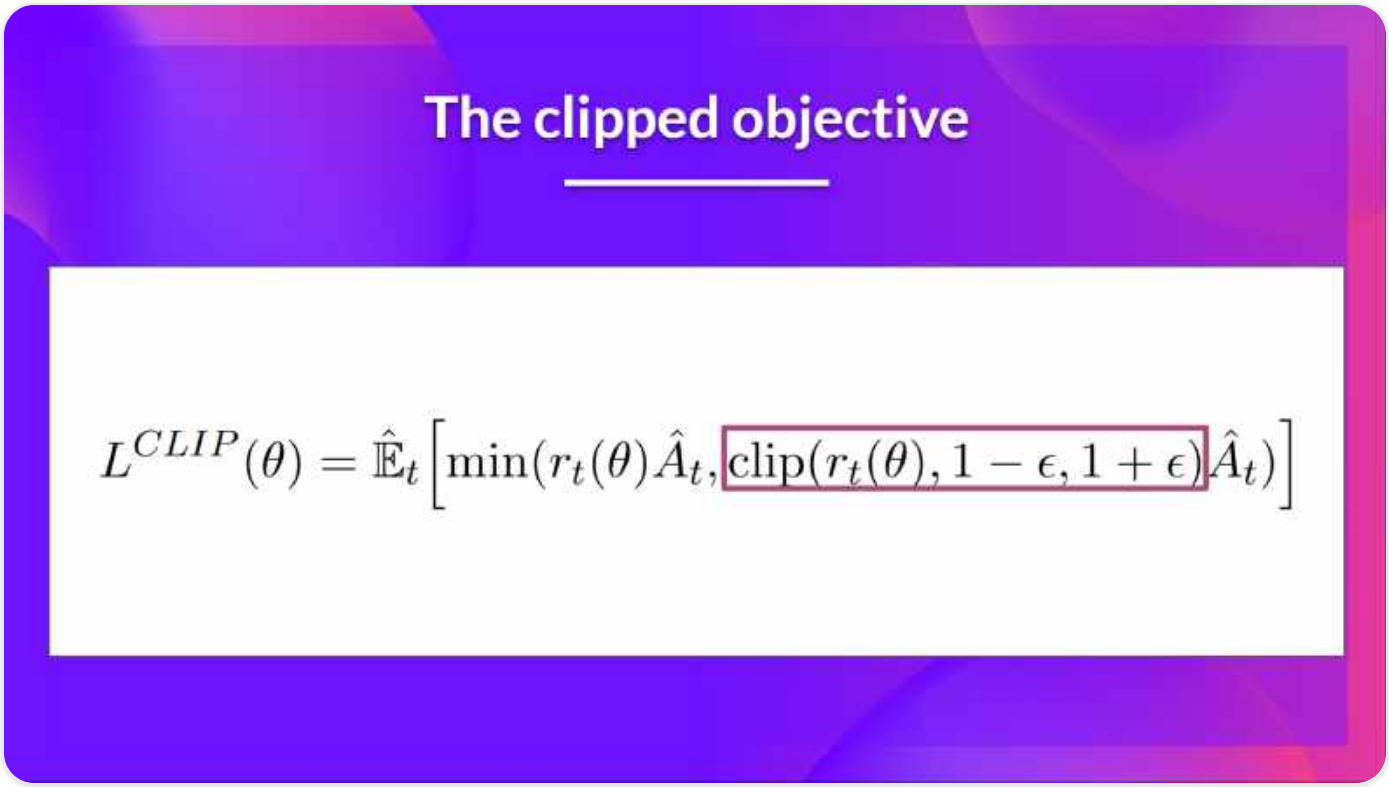


Consequently, we need to constrain this objective function by penalizing changes that lead to a ratio far away from 1 (in the paper, the ratio can only vary from 0.8 to 1.2).

By clipping the ratio, we ensure that we do not have a too large policy update because the current policy can't be too different from the older one.

To do that, we have two solutions:

- *TRPO (Trust Region Policy Optimization)* uses KL divergence constraints outside the objective function to constrain the policy update. But this method **is complicated to implement and takes more computation time.**
- *PPO* clip probability ratio directly in the objective function with its **Clipped surrogate objective function.**



This clipped part is a version where$r_t(\theta)$ is clipped between $[1 - \epsilon, 1 + \epsilon]$.

With the Clipped Surrogate Objective function, we have two probability ratios, one non-clipped and one clipped in a range between $[1 - \epsilon, 1 + \epsilon]$, epsilon is a hyperparameter that helps us to define this clip range (in the paper $\epsilon = 0.2$.).

Then, we take the minimum of the clipped and non-clipped objective, **so the final objective is a lower bound (pessimistic bound) of the unclipped objective.**

Taking the minimum of the clipped and non-clipped objective means **we'll select either the clipped or the non-clipped objective based on the ratio and advantage situation**.

<> Update on GitHub

← The intuition behind PPO Visualize the Clipped Surrogate Objective Function →

LangChain Docs

LangGraph overview

LangGraph overview

Copy page

Gain control with LangGraph to design agents that reliably handle complex tasks

Trusted by companies shaping the future of agents— including Klarna, Replit, Elastic, and more— LangGraph is a low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful agents.

LangGraph is very low-level, and focused entirely on agent **orchestration**. Before using LangGraph, we recommend you familiarize yourself with some of the components used to build agents, starting with **models** and **tools**.

We will commonly use **LangChain** components throughout the documentation to integrate models and tools, but you don't need to use LangChain to use LangGraph. If you are just getting started with agents or want a higher-level abstraction, we recommend you use LangChain's **agents** that provide pre-built architectures for common LLM and tool-calling loops.

LangGraph is focused on the underlying capabilities important for agent orchestration: durable execution, streaming, human-in-the-loop, and more.

Install

pip uv

```
pip install -U langgraph
```

Then, create a simple hello world example:

Assistant

Responses are generated using AI and may contain mistakes.

Ask a question...

scalable infrastructure designed to handle the unique challenges of stateful, long-running workflows.

LangGraph ecosystem

Assistant

Responses are generated using AI and may contain mistakes.

Acknowledgements

Assistant

Responses are generated using AI and may contain mistakes.

Trust Center

Assistant

Responses are generated using AI and may contain mistakes.

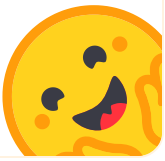
Search models, datasets, users...

Deep RL Course documentation
Language models in RL

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



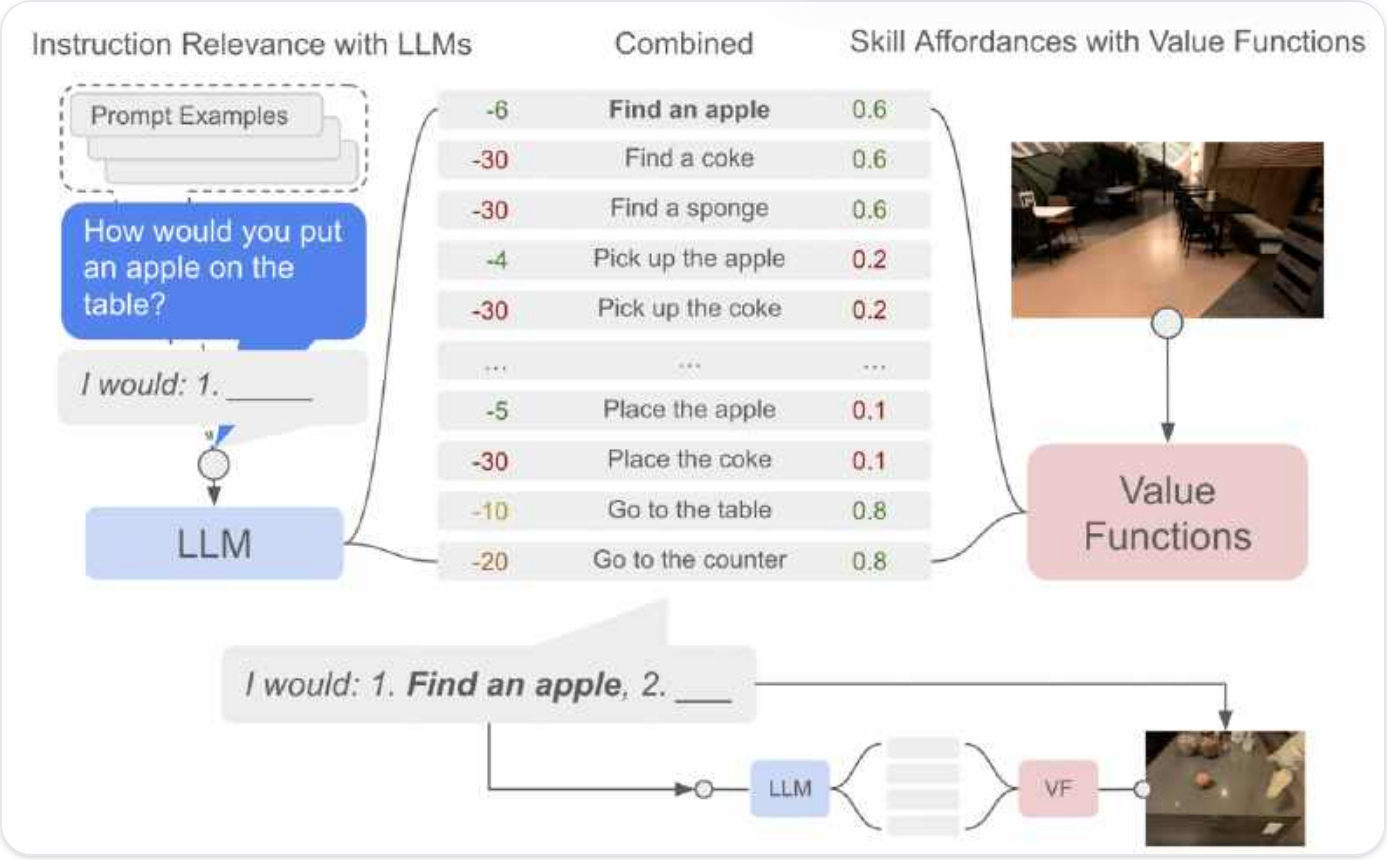
Language models in RL

LMs encode useful knowledge for agents

Language models (LMs) can exhibit impressive abilities when manipulating text such as question-answering or even step-by-step reasoning. Additionally, their training on massive text corpora allowed them to **encode various types of knowledge including abstract ones about the physical rules of our world** (for instance what is possible to do with an object, what happens when one rotates an object…).

A natural question recently studied was whether such knowledge could benefit agents such as robots when trying to solve everyday tasks. And while these works showed interesting results, the proposed agents lacked any learning method. **This limitation prevents these agent from adapting to the environment (e.g. fixing wrong knowledge) or learning new skills.**

Ask HuggingChat



Source: Towards Helpful Robots: Grounding Language in Robotic Affordances

LMs and RL

There is therefore a potential synergy between LMs which can bring knowledge about the world, and RL which can align and correct this knowledge by interacting with an environment. It is especially interesting from a RL point-of-view as the RL field mostly relies on the **Tabula-rasa** setup where everything is learned from scratch by the agent leading to:

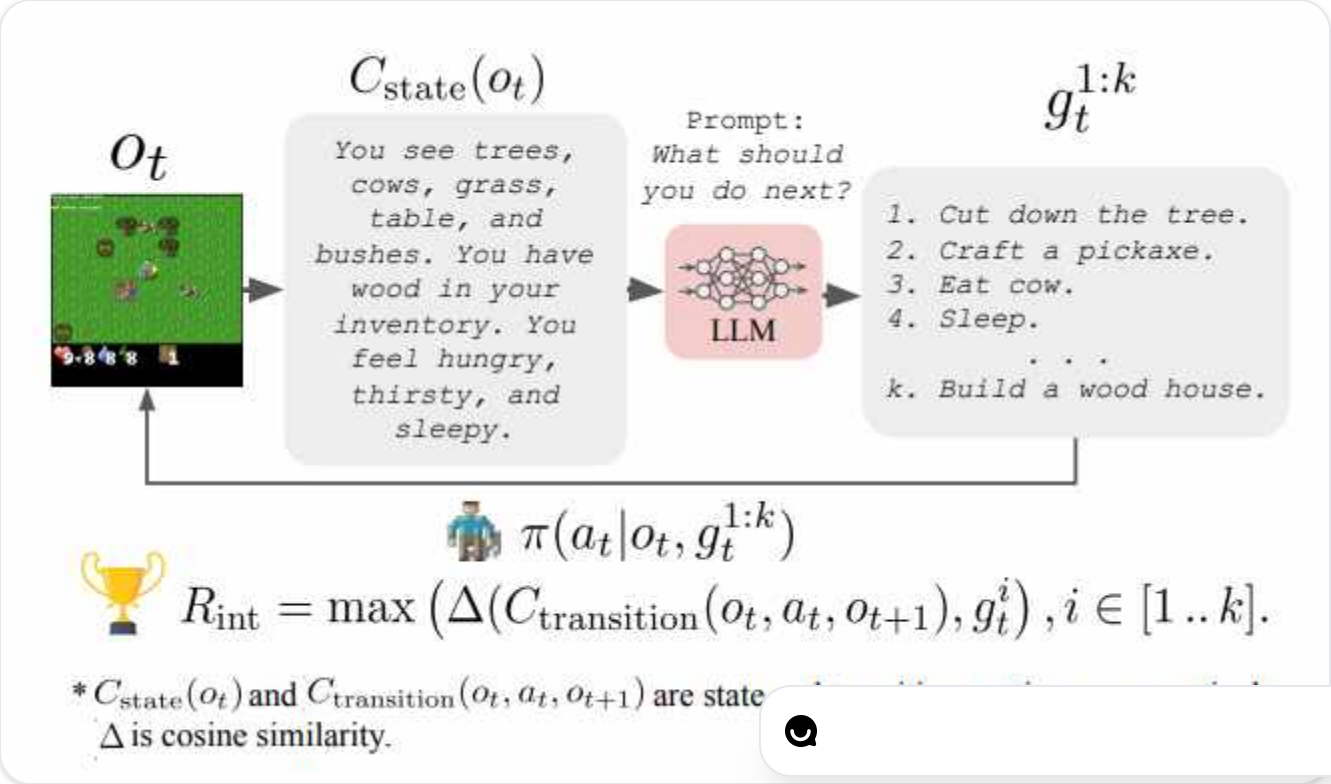
1) Sample inefficiency

2) Unexpected behaviors from humans' eyes

As a first attempt, the paper "Grounding Large Language Models with Online Reinforcement Learning" tackled the problem of **adapting or aligning a LM to a textual environment using PPO**. They showed that the knowledge encoded in the LM lead to a fast adaptation to the environment (opening avenues for sample efficient RL agents) but also that such knowledge allowed the LM to better generalize to new tasks once aligned.

0:00 / 2:19

Another direction studied in "Guiding Pretraining in Reinforcement Learning with Large Language Models" was to keep the LM frozen but leverage its knowledge to **guide an RL agent's exploration**. Such a method allows the RL agent to be guided towards human-meaningful and plausibly useful behaviors without requiring a human in the loop during training.



Source: Towards Helpful Robots: Grounding Language in Robotic Affordances

Several limitations make these works still very preliminary such as the need to convert the agent's observation to text before giving it to a LM as well as the compute cost of interacting with very large LMs.

Further reading

For more information we recommend you check out the following resources:

- Google Research, 2022 & beyond: Robotics
- Pre-Trained Language Models for Interactive Decision-Making
- Grounding Large Language Models with Online Reinforcement Learning
- Guiding Pretraining in Reinforcement Learning with Large Language Models

Author

This section was written by Clément Romac

<> Update on GitHub

← Decision Transformers and Offline RL (Automatic) Curriculum Learning for RL →

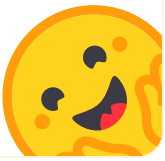
Search models, datasets, users...

Deep RL Course documentation
Mid-way Recap

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Mid-way Recap

Before diving into Q-Learning, let's summarize what we've just learned.

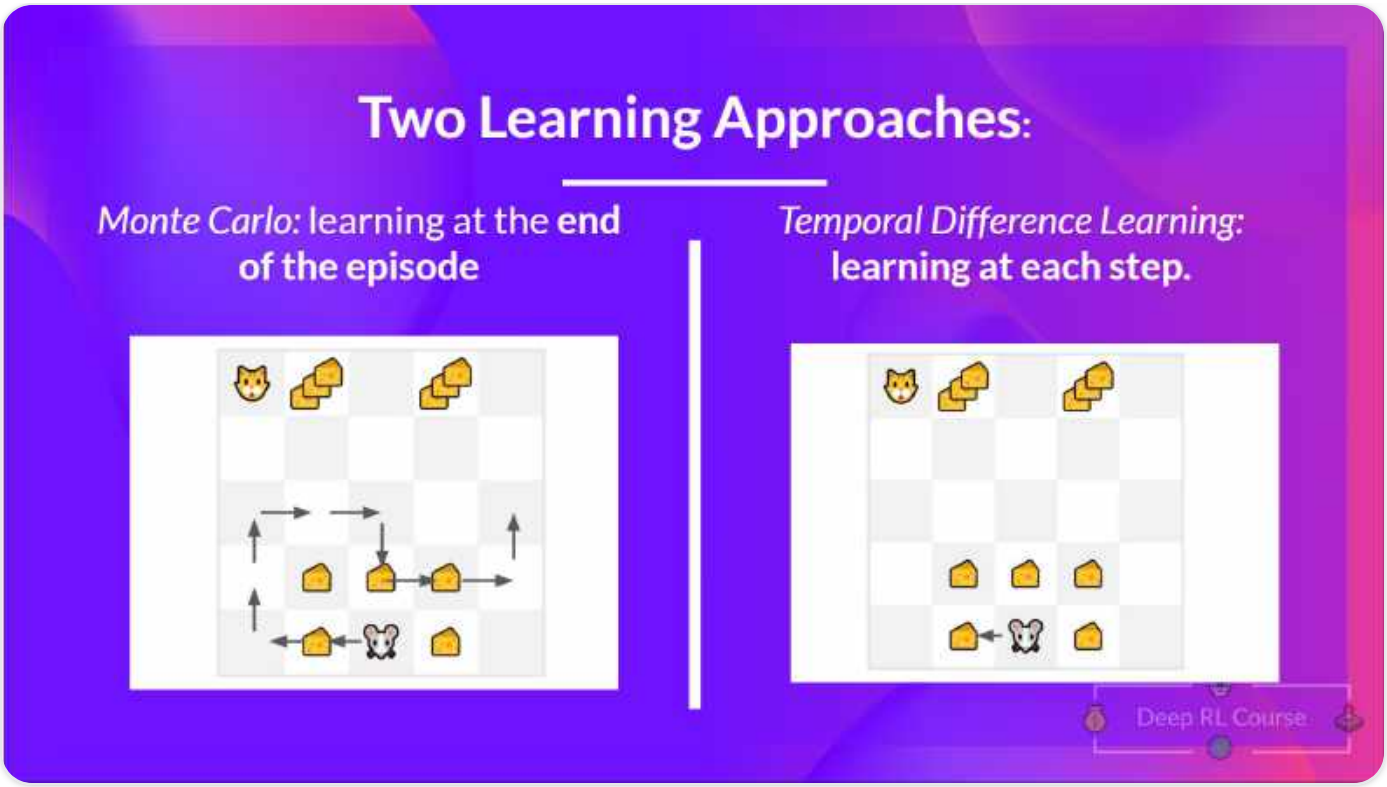
We have two types of value-based functions:

- State-value function: outputs the expected return if **the agent starts at a given state and acts according to the policy forever after.**
- Action-value function: outputs the expected return if **the agent starts in a given state, takes a given action at that state** and then acts accordingly to the policy forever after.
- In value-based methods, rather than learning the policy, **we define the policy by hand** and we learn a value function. If we have an optimal value function, we **will have an optimal policy.**

There are two types of methods to update the value function:

- With *the Monte Carlo method*, we update the value function from a complete episode, and so we **use the actual discounted return of this episode.**
- With *the TD Learning method,* we update the value function from a step, replacing the unknownG_t with **an estimated return called the TD target.**

Ask HuggingChat



<> Update on GitHub

← Monte Carlo vs Temporal Difference Learning

Mid-way Quiz →

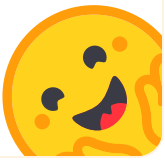
Search models, datasets, users...

Deep RL Course documentation
Model Based Reinforcement Learning (MBRL)

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Model Based Reinforcement Learning (MBRL)

Model-based reinforcement learning only differs from its model-free counterpart in learning a *dynamics model*, but that has substantial downstream effects on how the decisions are made.

The dynamics model usually models the environment transition dynamics, $s_{t+1} = f_\theta(s_t, a_t)$, but things like inverse dynamics models (mapping from states to actions) or reward models (predicting rewards) can be used in this framework.

Simple definition

- There is an agent that repeatedly tries to solve a problem, **accumulating state and action data**.
- With that data, the agent creates a structured learning tool, *a dynamics model*, to reason about the world.
- With the dynamics model, the agent **decides how to act by predicting the future**.
- With those actions, **the agent collects more data, improves said model, and hopefully improves future actions**.

Ask HuggingChat

Academic definition

Model-based reinforcement learning (MBRL) follows the framework of an agent interacting in an environment, **learning a model of said environment**, and then **leveraging the model for control (making decisions).

Specifically, the agent acts in a Markov Decision Process (MDP) governed by a transition function $s_{t+1} = f(s_t, a_t)$ and returns a reward at each step$r(s_t, a_t)$. With a collected dataset$D := s_i, a_i, s_{i+1}, r_i$, the agent learns a model,$s_{t+1} = f_\theta(s_t, a_t)$ **to minimize the negative log-likelihood of the transitions**.

We employ sample-based model-predictive control (MPC) using the learned dynamics model, which optimizes the expected reward over a finite, recursively predicted horizon,τ, from a set of actions sampled from a uniform distribution$U(a)$, (see paper or paper or paper).

Further reading

For more information on MBRL, we recommend you check out the following resources:

- A blog post on debugging MBRL.
- A recent review paper on MBRL,

Author

This section was written by Nathan Lambert

<> Update on GitHub

← Introduction Offline vs. Online Reinforcement Learning →

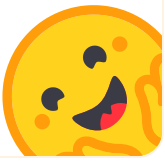
Search models, datasets, users...

Deep RL Course documentation
Monte Carlo vs Temporal Difference Learning

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Monte Carlo vs Temporal Difference Learning

The last thing we need to discuss before diving into Q-Learning is the two learning strategies.

Remember that an RL agent **learns by interacting with its environment.** The idea is that **given the experience and the received reward, the agent will update its value function or policy.**

Monte Carlo and Temporal Difference Learning are two different **strategies on how to train our value function or our policy function.** Both of them **use experience to solve the RL problem.**

On one hand, Monte Carlo uses **an entire episode of experience before learning.** On the other hand, Temporal Difference uses **only a step (S_t, A_t, R_{t+1}, S_{t+1}) to learn.**

We'll explain both of them **using a value-based method example.**

Monte Carlo: learning at the end of the episode

Monte Carlo waits until the end of the episode, calculates G_t (return) and uses it as **a target for updating $V(S_t)$.**

So it requires a **complete episode of interaction before updating our value function.**

Ask HuggingChat

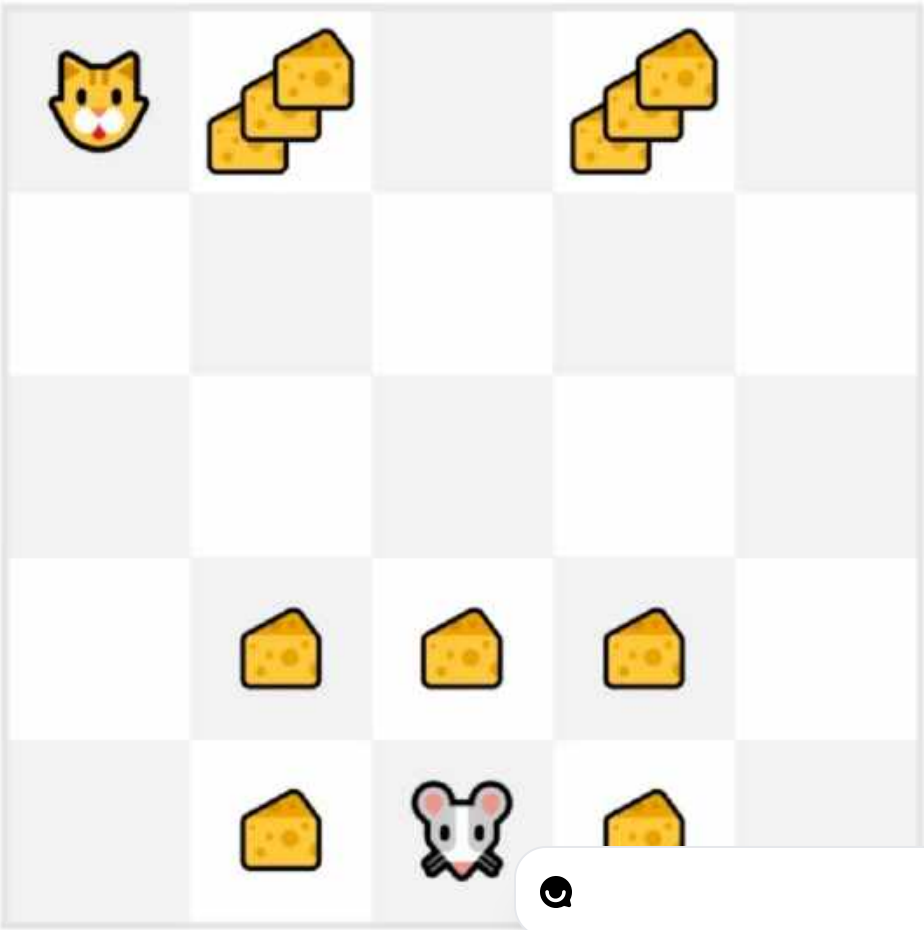
Monte Carlo Approach:

Monte Carlo: **waits until the end of the episode**, then calculates Gt (return) and uses it as a target for its value or policy.

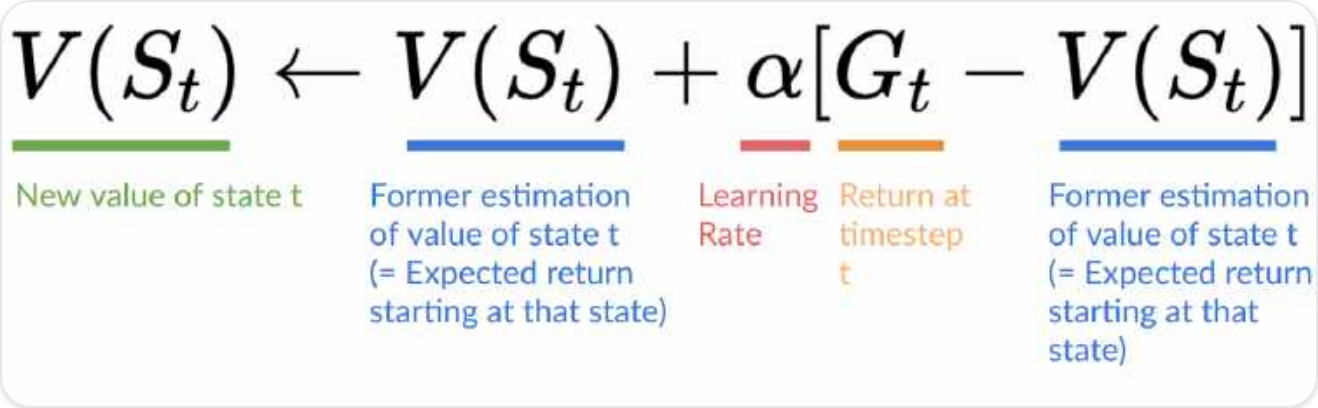
$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

- $V(S_t)$: New value of state t
- $V(S_t)$: Former estimation of value of state t (= Expected return starting at that state)
- α: Learning Rate
- G_t: Return at timestep t
- $V(S_t)$: Former estimation of value of state t (= Expected return starting at that state)

If we take an example:

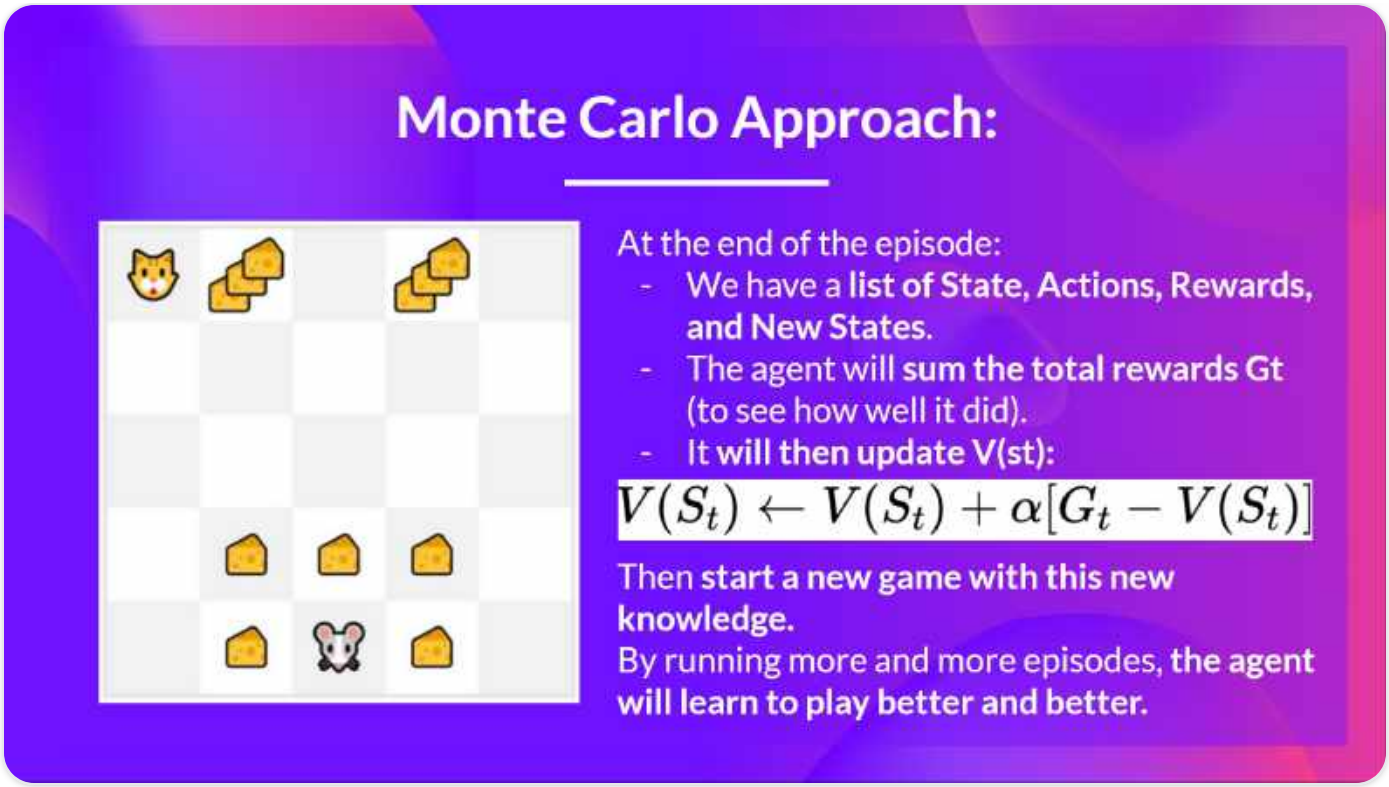


- We always start the episode **at the same starting point.**
- **The agent takes actions using the policy**. For instance, using an Epsilon Greedy Strategy, a policy that alternates between exploration (random actions) and exploitation.
- We get **the reward and the next state.**
- We terminate the episode if the cat eats the mouse or if the mouse moves > 10 steps.
- At the end of the episode, **we have a list of State, Actions, Rewards, and Next States tuples** For instance [[State tile 3 bottom, Go Left, +1, State tile 2 bottom], [State tile 2 bottom, Go Left, +0, State tile 1 bottom]…]
- **The agent will sum the total rewards** G_t (to see how well it did).
- It will then **update** $V(s_t)$ **based on the formula**



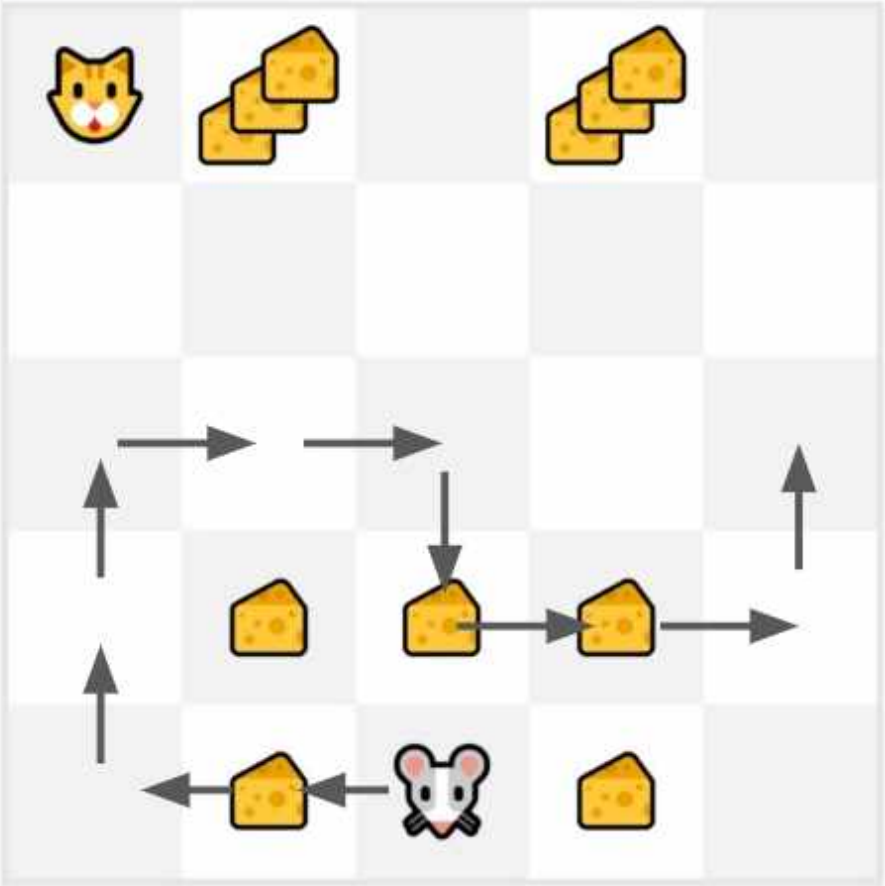
- Then **start a new game with this new knowledge**

By running more and more episodes, **the agent will learn to play better and better.**

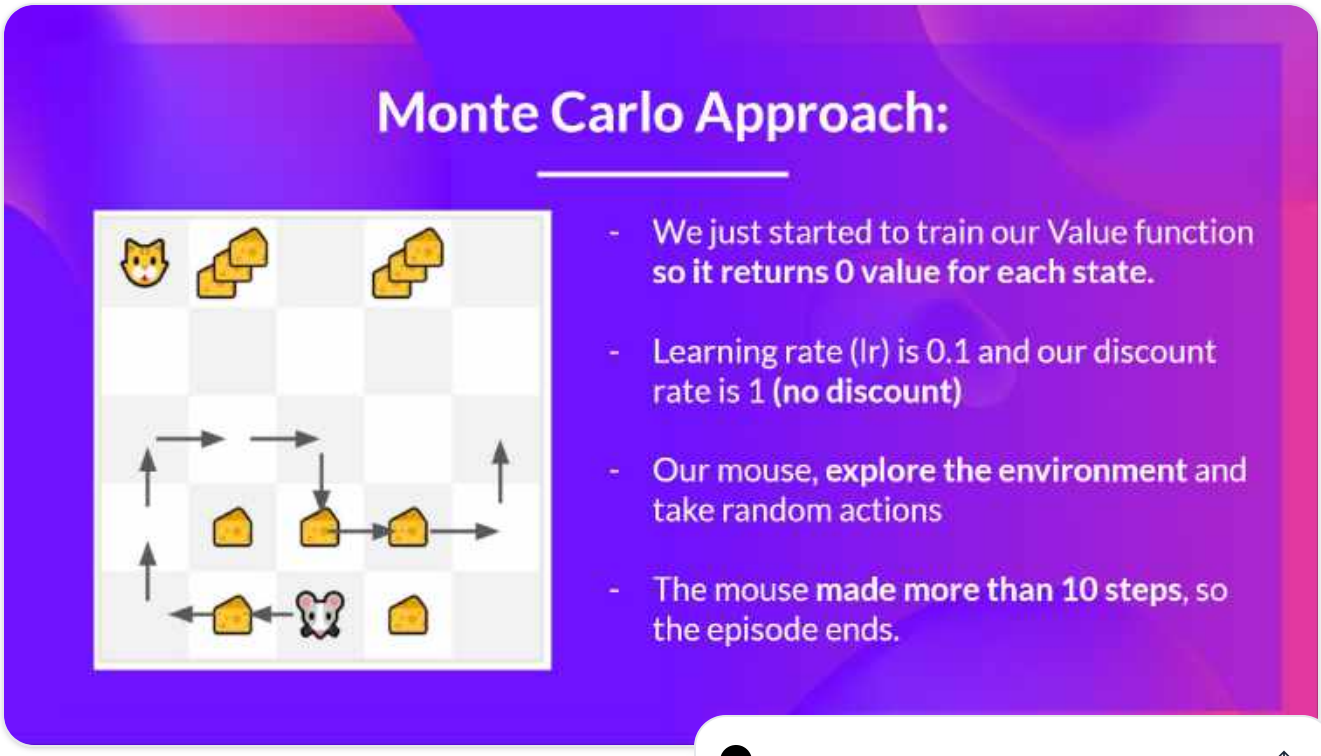


For instance, if we train a state-value function using Monte Carlo:

- We initialize our value function **so that it returns 0 value for each state**
- Our learning rate (lr) is 0.1 and our discount rate is 1 (= no discount)
- Our mouse **explores the environment and takes random actions**



- The mouse made more than 10 steps, so the episode ends .



- We have a list of state, action, rewards, next_state, **we need to calculate the return**$Gt = 0$ $G_t = R_{t+1} + R_{t+2} + R_{t+3}...$ (for simplicity, we don't discount the rewards) $G_0 = R_1 + R_2 + R_3 \ldots G_0 = 1 + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0\, G_0 = 3$

- We can now compute the **new**$V(S_0)$:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

New value of state t

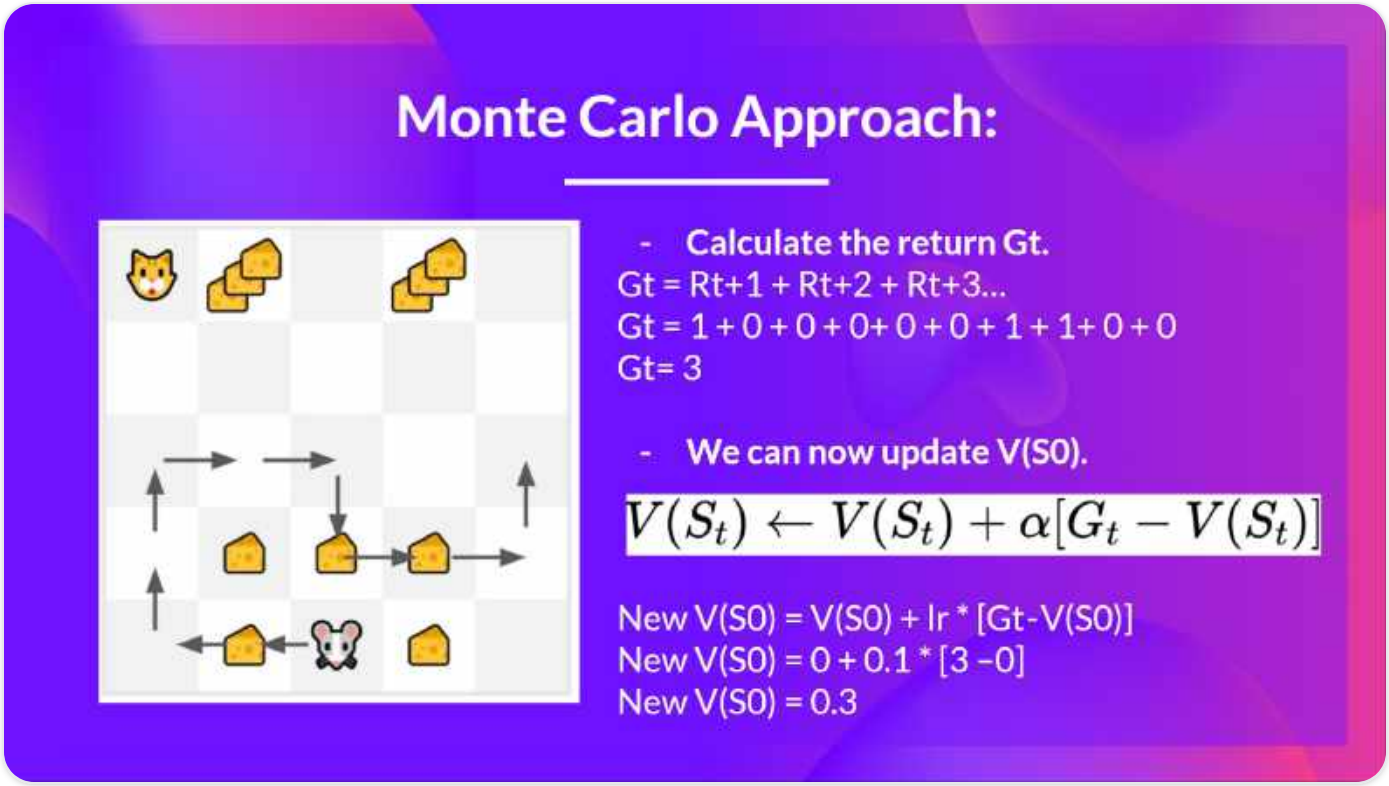
Former estimation of value of state t (= Expected return starting at that state)

Learning Rate

Return at timestep t

Former estimation of value of state t (= Expected return starting at that state)

$$V(S_0) = V(S_0) + lr * [G_0 — V(S_0)]\, V(S_0) = 0 + 0.1 * [3–0]\, V(S_0) = 0.3$$



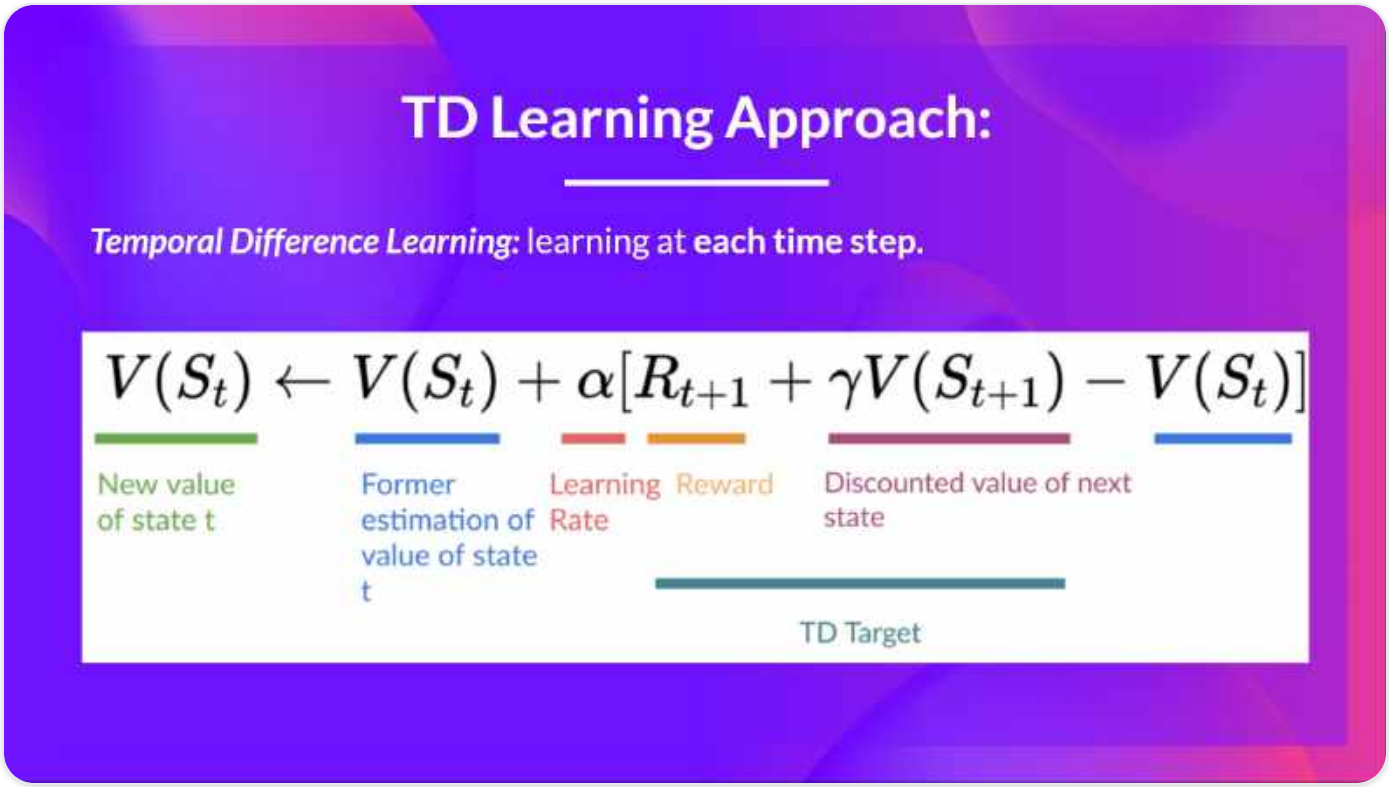
Temporal Difference Learning: learning at each step

Temporal Difference, on the other hand, waits for only one interaction (one step) S_{t+1} to form a TD target and update $V(S_t)$ using R_{t+1} and $\gamma * V(S_{t+1})$.

The idea with **TD is to update the** $V(S_t)$ **at each step.**

But because we didn't experience an entire episode, we don't have G_t (expected return). Instead, **we estimate** G_t **by adding** R_{t+1} **and the discounted value of the next state.**

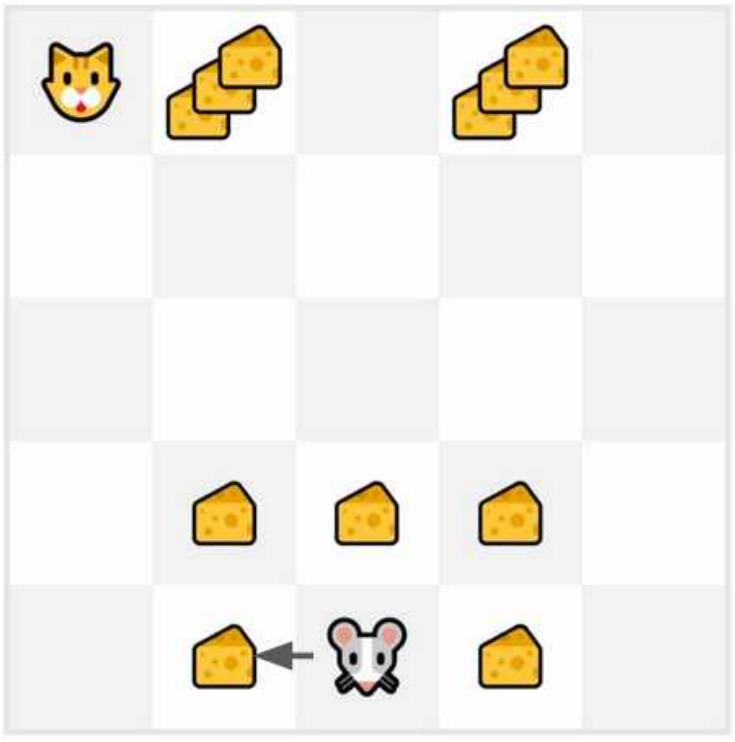
This is called bootstrapping. It's called this **because TD bases its update in part on an existing estimate** $V(S_{t+1})$ **and not a complete sample** G_t**.**



This method is called TD(0) or **one-step TD (update the value function after any individual step).**

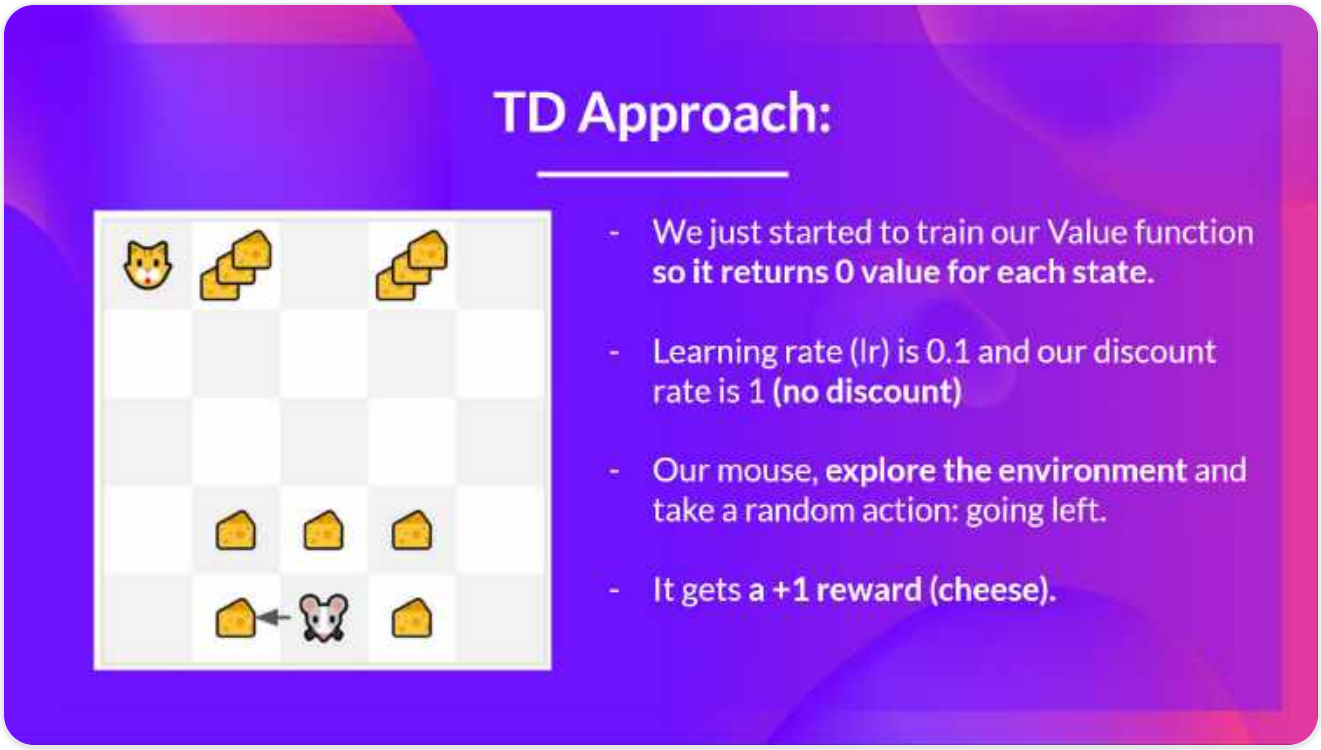


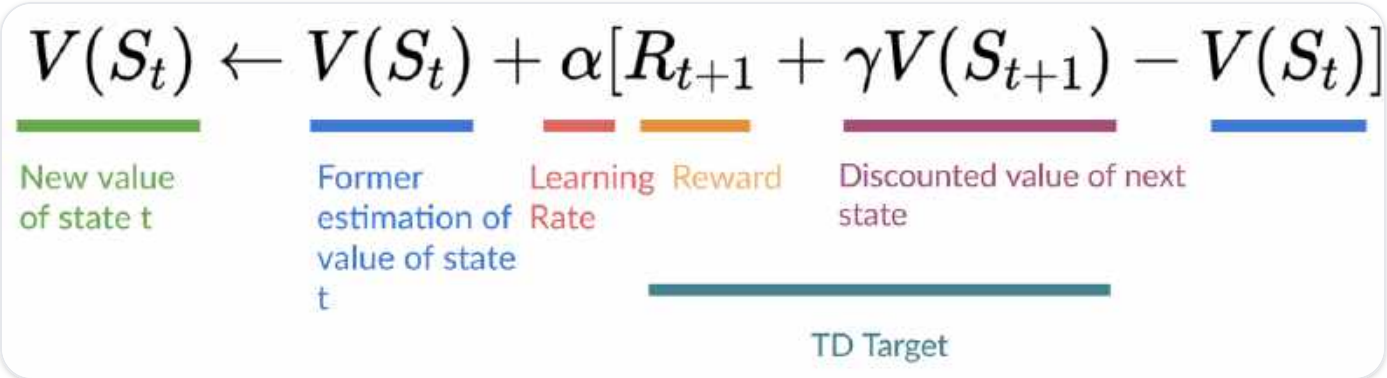
If we take the same example,



- We initialize our value function so that it returns 0

- Our learning rate (lr) is 0.1, and our discount rate is 1 (no discount).
- Our mouse begins to explore the environment and takes a random action: **going to the left**
- It gets a reward $R_{t+1} = 1$ since **it eats a piece of cheese**





We can now update $V(S_0)$:

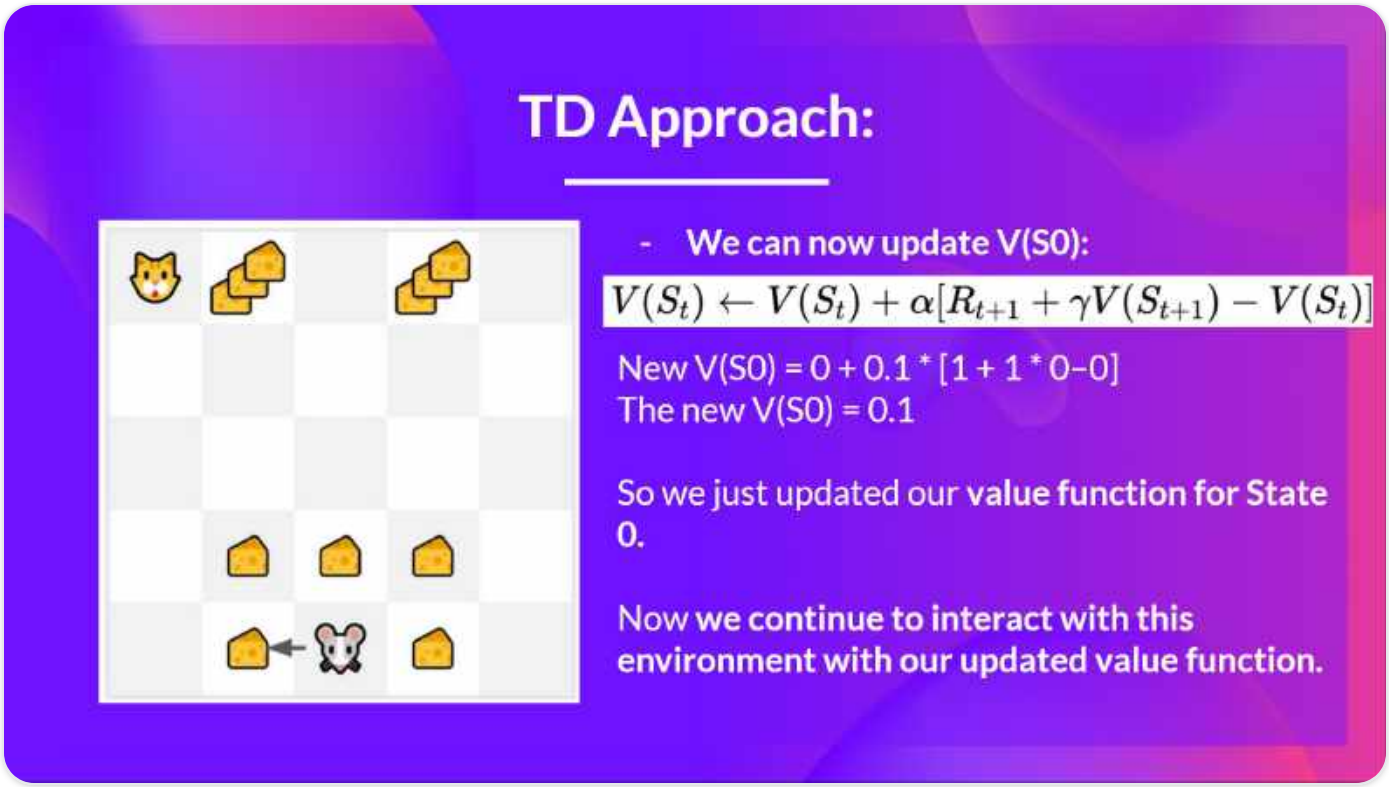
New $V(S_0) = V(S_0) + lr * [R_1 + \gamma * V(S_1) - V(S_0)]$

New $V(S_0) = 0 + 0.1 * [1 + 1 * 0–0]$

New $V(S_0) = 0.1$

So we just updated our value function for State 0.

Now we **continue to interact with this environment with our updated value function.**



To summarize:

- With *Monte Carlo*, we update the value function from a complete episode, and so we **use the actual accurate discounted return of this episode.**
- With *TD Learning*, we update the value function from a step, and we replace G_t, which we don't know, with **an estimated return called the TD target.**

Monte Carlo: $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$

TD Learning: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

<> Update on GitHub

← The Bellman Equation, simplify our value estimation

Mid-way Recap →

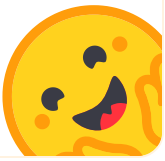
Search models, datasets, users...

Deep RL Course documentation
Offline vs. Online Reinforcement Learning

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



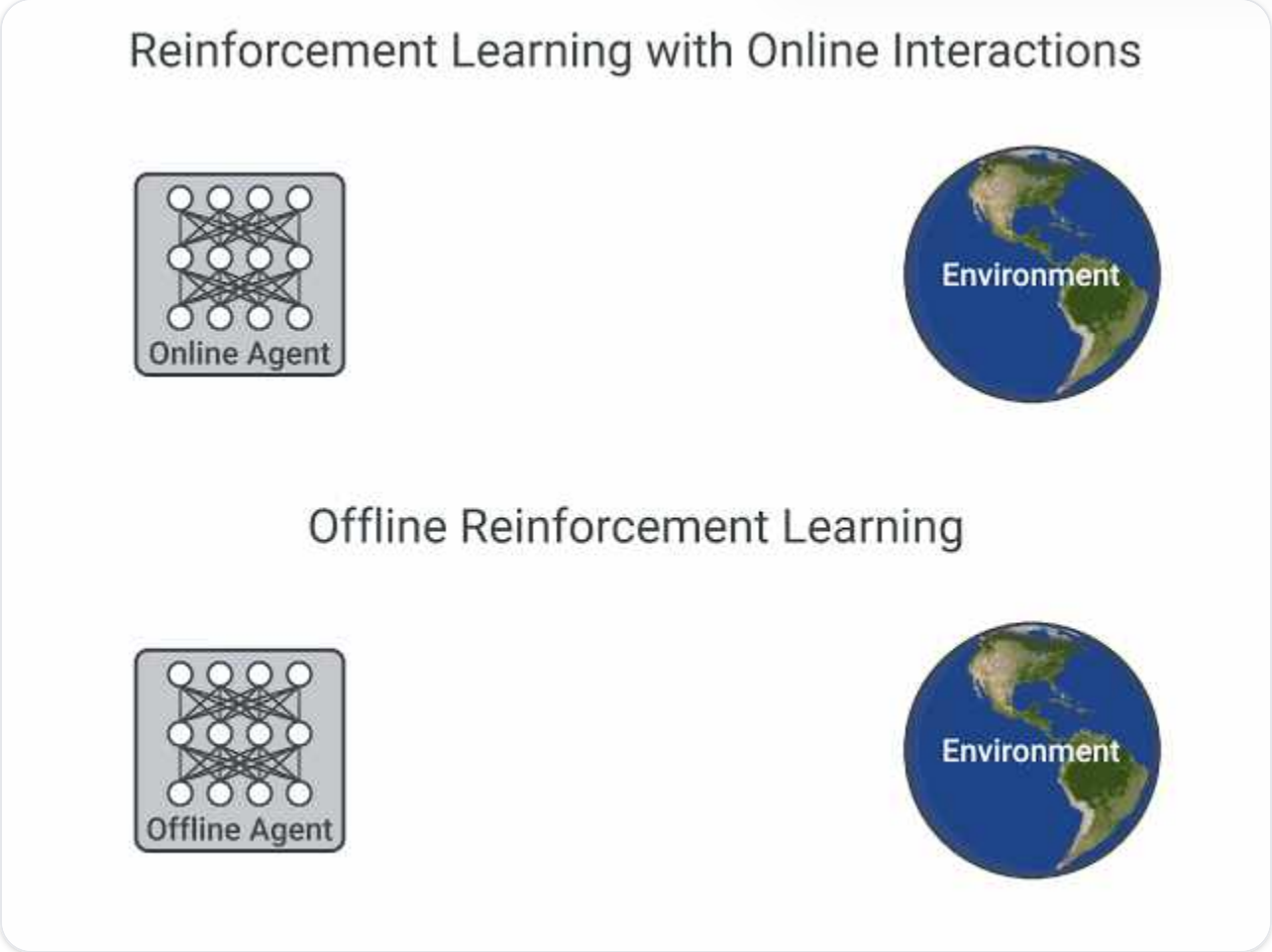
Offline vs. Online Reinforcement Learning

Deep Reinforcement Learning (RL) is a framework **to build decision-making agents**. These agents aim to learn optimal behavior (policy) by interacting with the environment through **trial and error and receiving rewards as unique feedback**.

The agent's goal **is to maximize its cumulative reward**, called return. Because RL is based on the *reward hypothesis*: all goals can be described as the **maximization of the expected cumulative reward**.

Deep Reinforcement Learning agents **learn with batches of experience**. The question is, how do they collect it?:

Ask HuggingChat



A comparison between Reinforcement Learning in an Online and Offline setting, figure taken from this post

- In *online reinforcement learning*, which is what we've learned during this course, the agent **gathers data directly**: it collects a batch of experience by **interacting with the environment**. Then, it uses this experience immediately (or via some replay buffer) to learn from it (update its policy).

But this implies that either you **train your agent directly in the real world or have a simulator**. If you don't have one, you need to build it, which can be very complex (how to reflect the complex reality of the real world in an environment?), expensive, and insecure (if the simulator has flaws that may provide a competitive advantage, the agent will exploit them).

- On the other hand, in *offline reinforcement learning*, the agent only **uses data collected from other agents or human demonstrations**. It does **not interact with the environment**.

The process is as follows:

- **Create a dataset** using one or more policies and/or human interactions.
- Run **offline RL on this dataset** to learn a policy

This method has one drawback: the *counterfactual queries problem*. What do we do if our agent **decides to do something for which we don't have the data?** For instance, turning right on an intersection but we don't have this trajectory.

There exist some solutions on this topic, but if you want to know more about offline reinforcement learning, you can watch this video

Further reading

For more information, we recommend you check out the following resources:

- Offline Reinforcement Learning, Talk by Sergei Levine
- Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems

Author

This section was written by Thomas Simonini

<> Update on GitHub

← Model-Based Reinforcement Learning Generalisation Reinforcement Learning →

DeepLearning.AI
THE BATCH

Published Mar 27, 2024
Reading time 12 min read

Dear friends,

Last week, I described four design patterns for AI agentic workflows that I believe will drive significant progress this year: Reflection, Tool use, Planning and Multi-agent collaboration. Instead of having an LLM generate its final output directly, an agentic workflow prompts the LLM multiple times, giving it opportunities to build step by step to higher-quality output. In this letter, I'd like to discuss Reflection. For a design pattern that's relatively quick to implement, I've seen it lead to surprising performance gains.

You may have had the experience of prompting ChatGPT/Claude/Gemini, receiving unsatisfactory output, delivering critical feedback to help the LLM improve its response, and then getting a better response. What if you automate the step of delivering critical feedback, so the model automatically criticizes its own output and improves its response? This is the crux of Reflection.

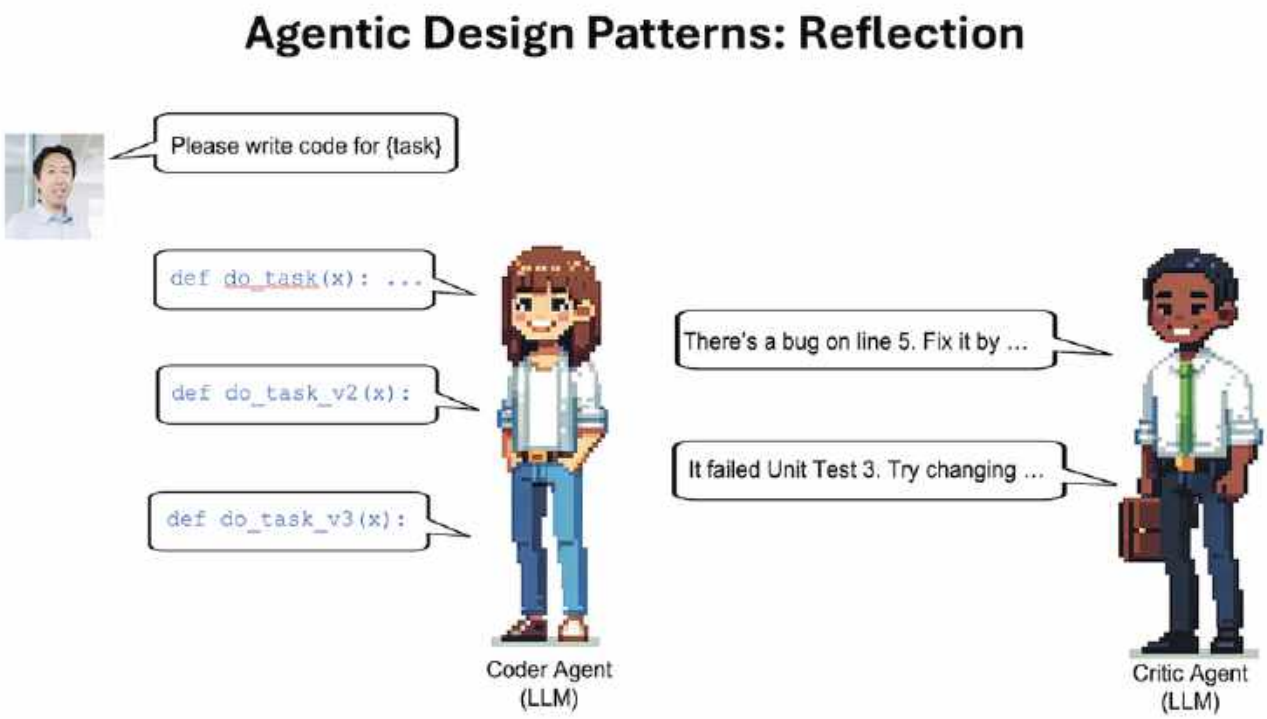
Take the task of asking an LLM to write code. We can prompt it to generate the desired code directly to carry out some task X. After that, we can prompt it to reflect on its own output, perhaps as follows:

Here's code intended for task X: [previously generated code]
Check the code carefully for correctness, style, and efficiency, and give constructive criticism for how to improve it.

Sometimes this causes the LLM to spot problems and come up with constructive suggestions. Next, we can prompt the LLM with context including (i) the previously generated code and the constructive feedback and (ii) ask it to use the feedback to rewrite the code. This can lead to a better response. Repeating the criticism/rewrite

DeepLearning.AI
THE BATCH

writing text, and answering questions.



And we can go beyond self-reflection by giving the LLM tools that help evaluate its output; for example, running its code through a few unit tests to check whether it generates correct results on test cases or searching the web to double-check text output. Then it can reflect on any errors it found and come up with ideas for improvement.

Further, we can implement Reflection using a multi-agent framework. I've found it convenient to create two different agents, one prompted to generate good outputs and the other prompted to give constructive criticism of the first agent's output. The resulting discussion between the two agents leads to improved responses.

Reflection is a relatively basic type of agentic workflow, but I've been delighted by how much it improved my applications' results in a few cases. I hope you will try it in your own work. If you're interested in learning more about reflection, I recommend these papers:

- "Self-Refine: Iterative Refinement with Self-Feedback," Madaan et al. (2023)

DeepLearning.AI
THE BATCH

- "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing," Gou et al. (2024)

I'll discuss the other agentic design patterns in future letters.

Keep learning!

Andrew

P.S. New JavaScript short course! Learn to build full-stack web applications that use RAG in "JavaScript RAG Web Apps with LlamaIndex," taught by Laurie Voss, VP of Developer Relations at LlamaIndex and co-founder of npm.

- Build a RAG application for querying your own data.
- Develop tools that interact with multiple data sources and use an agent to autonomously select the right tool for a given query.
- Create a full-stack web app step by step that lets you chat with your data.
- Dig further into production-ready techniques like how to persist your data, so you don't need to reindex constantly.

Sign up here!

News

DeepLearning.AI
THE BATCH



One Agent, Many Environments

AI agents are typically designed to operate a particular software environment. Recent work enabled a single agent to take actions in a variety of three-dimensional virtual worlds.

What's new: A team of 90 people at Google and University of British Columbia announced Scalable Instructable Multiworld Agent (SIMA), a system that learned to follow text instructions (such as “make a pile of rocks to mark this spot” or “see if you can jump over this chasm”) in seven commercial video games and four research environments.

How it works: SIMA’s architecture consists of several transformers and a vanilla neural network. The authors trained it to mimic human players using a dataset of gameplay broken into 10 second tasks, including onscreen images, text instructions, keyboard presses, and mouse motions. The video games included Goat Simulator 3 (a third-person game in which the player takes the form of a goat), No Man’s Sky (a first- or third-person game of exploration and survival in outer space), Hydroneer (a first-person game of mining and building), and others.

DeepLearning.AI
THE BATCH

similar text and images) produced text and image embeddings. Given recent frames, Phenaki (a transformer pretrained to predict future frames in a video) generated a video embedding.

- Given the image, text, and video embeddings, a collection of transformers learned to produce a representation of the game. (The authors don't fully describe this part of the architecture.)
- Given the game representation, a vanilla neural network learned to produce the corresponding keyboard and mouse actions.

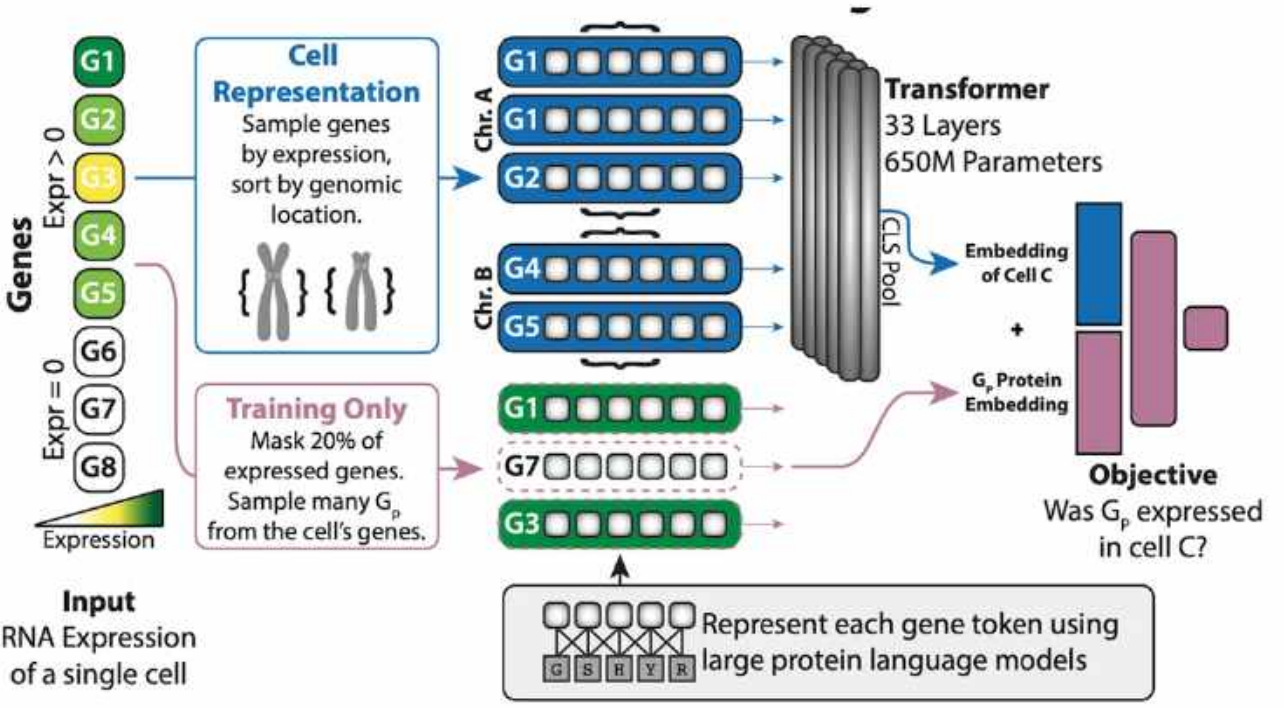
Results: Judges evaluated SIMA's success or failure at completing nearly 1,500 instructions that spanned tasks in nine categories like action ("jump"), navigation ("go to your ship"), and gathering resources ("get raspberries"). In Goat Simulator 3, SIMA completed 40 percent of the tasks. In No Man's Sky, the judges compared SIMA's performance to that of the human players whose gameplay produced the training data. SIMA was successful 34 percent of the time, while the players were successful 60 percent of the time. Judges also compared SIMA to versions that were trained to be experts in a single game. SIMA was successful more than 1.5 times more often than the specialized agents.

Behind the news: SIMA extends Google's earlier successes building agents that rival or beat human players at individual games including Go, classic Atari games, and StarCraft II.

Why it matters: Training agents to follow directions in various environments, seeing the same things humans would, is a step toward building instructable agents that can work in any situation. The authors point to potential applications in robotics, simulations, and gaming; wherever an agent might need to be guided through diverse challenges.

We're thinking: This work shows that an agent trained on multiple games can perform better than an agent trained on just one, and that the richer the language inputs in a gameworld, the better the agent can perform. With only a handful of training

DeepLearning.AI
THE BATCH



Cross-Species Cell Embeddings

Researchers used an AI system to identify animal cell types from gene sequences, including a cell type that conventional approaches had discovered only in the past year.

What's new: Biologists at Stanford trained a system to produce embeddings that represent individual cells in an organism. This enabled them to find cell types that have common function in different animals; for instance, the Norn cell, a type of kidney cell that biologists had previously theorized but discovered only in 2023.

How it works: Universal Cell Embedding (UCE) comprises two transformers that produce embeddings of genes and cells respectively, plus a classifier based on a vanilla neural network. The authors trained the classifier, given embeddings of a gene and cell, to classify whether or not the cell produces the protein coded by that gene.

structures.

- The authors represented each cell as a sequence of gene embeddings, laid out in the order in which they appear in the cell's genome. Instead of including all of a cell's genes, the authors sampled 1,024 genes known to encode proteins. A pretrained ESM-2 transformer computed each gene's embedding based on the protein(s) — that is, amino acid sequence(s) — it produces.
- The authors randomly masked 20 percent of the gene embeddings. Given the masked sequence, a vanilla transformer learned to compute an embedding of the cell.
- For each gene in the cell, the authors concatenated its embedding with the cell embedding. Given the combined embeddings, the vanilla neural network learned to classify whether the genes encoded a protein.

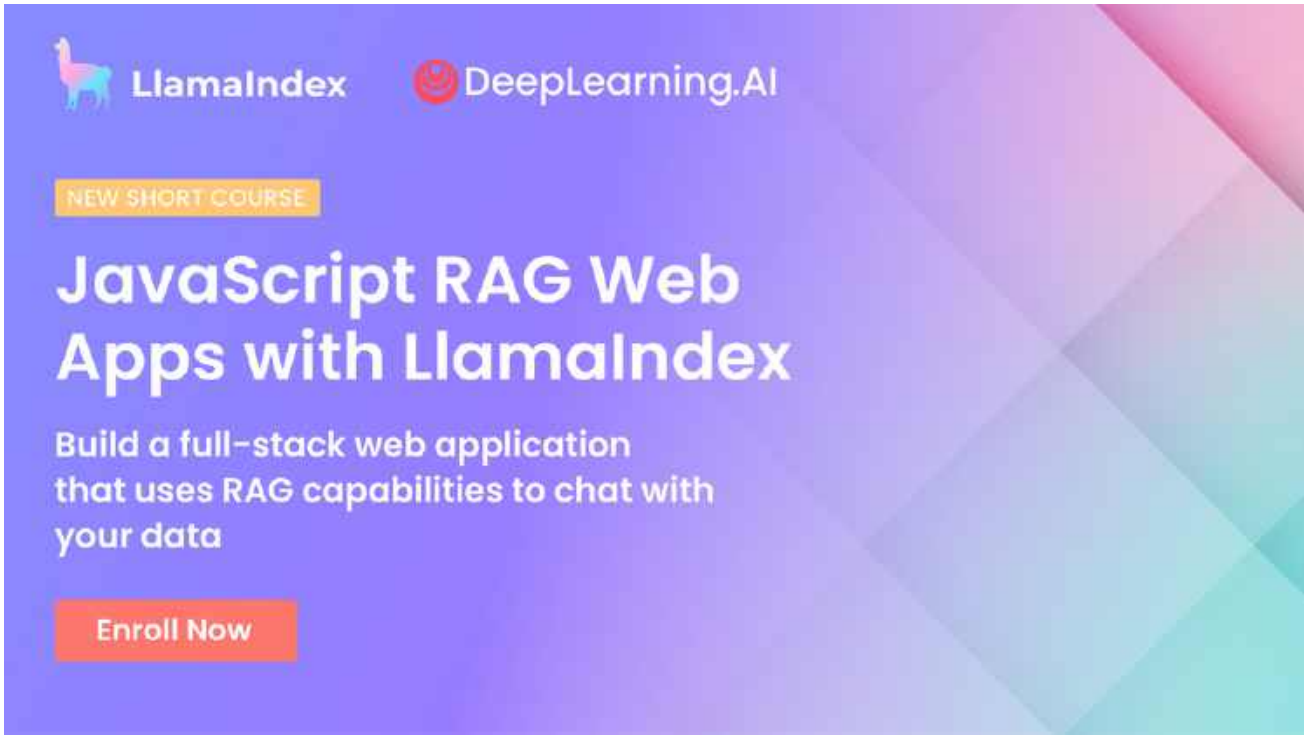
Results: Cell embeddings produced by UCE enabled the authors to identify cell types in animal species that weren't in the training set. For instance, the authors embedded a dataset of mouse cells and applied UMAP clustering to differentiate the types. They labeled the clusters as specific cell types (including Norn cells, which biologists took more than a century to find) based on the presence of certain genes that distinguish one cell type from another. Using the labels, they trained a logistic classifier. They applied the classifier to their training dataset and found Norn cells, among other cell types, in species other than mice. They verified the findings by looking for genes that tend to show up only in Norn cells.

Why it matters: UCE's embeddings encode biologically meaningful information about individual cells, enabling a clustering algorithm to group them into recognized cell types. The fact that the recently discovered Norn cell was among those clusters suggests that UCE may yield further discoveries that accelerate development of new medicines, lab processes, and research methods. In fact, the model found Norn cells — which are known to occur in the kidney — in organs where they have not been seen before. If this result turns out to be valid, UCE will have made a discovery that has eluded biologists to date.

DeepLearning.AI
THE BATCH

species, yet it generalized to others.

NEW FROM DEEPLEARNING.AI



Join our short course on “JavaScript RAG Web Apps with LlamaIndex” to learn how to build full-stack JavaScript web applications that let you chat with your data. Harness the capabilities of large language models and retrieval augmented generation (RAG)! Enroll for free

DeepLearning.AI
THE BATCH



United States Intelligence, Surveillance and Reconnaissance Division, Combined Air Operations Center, Al Udeid Air Base, Qatar, 2017

U.S. Deploys AI-Assisted Targeting

The United States military is using computer vision to target enemy positions in the Red Sea and elsewhere.

What's new: Maven, a system that analyzes satellite and geolocation data, has been used to identify targets in real-world conflicts, *Bloomberg* reported. The system was developed primarily by Palantir and integrates technology from Amazon, Microsoft, information technology firms ECS Federal and L3Harris, aerospace firms Maxar and Sierra Nevada, and other unnamed companies.

How it works: The 18th Airborne Corps, a U.S. Army unit organized for rapid deployment around the world, used Maven in live-fire training exercises. The system helped locate surface vessels in the Red Sea, rocket launchers in Yemen, and potential airstrike targets in Iraq and Syria. The U.S. used it to help Ukraine's armed forces to locate Russian equipment, anonymous sources said.

- Maven melds various data streams into a top-down image of a geographic area. Satellites provide still images and video, and radar and infrared observations enable the system to see through clouds and other obstructions. It can also

DeepLearning.AI
THE BATCH

- Computer vision models identify military equipment such as aircraft and tanks, highlighting significant changes to object locations. They can register a buildup of equipment that may indicate a new, or newly active, military base.
- The system displays a map that outlines potential targets in yellow and friendly forces, schools, hospitals, and other no-strike zones outlined in blue. Human decision-makers review output and authorize responses.

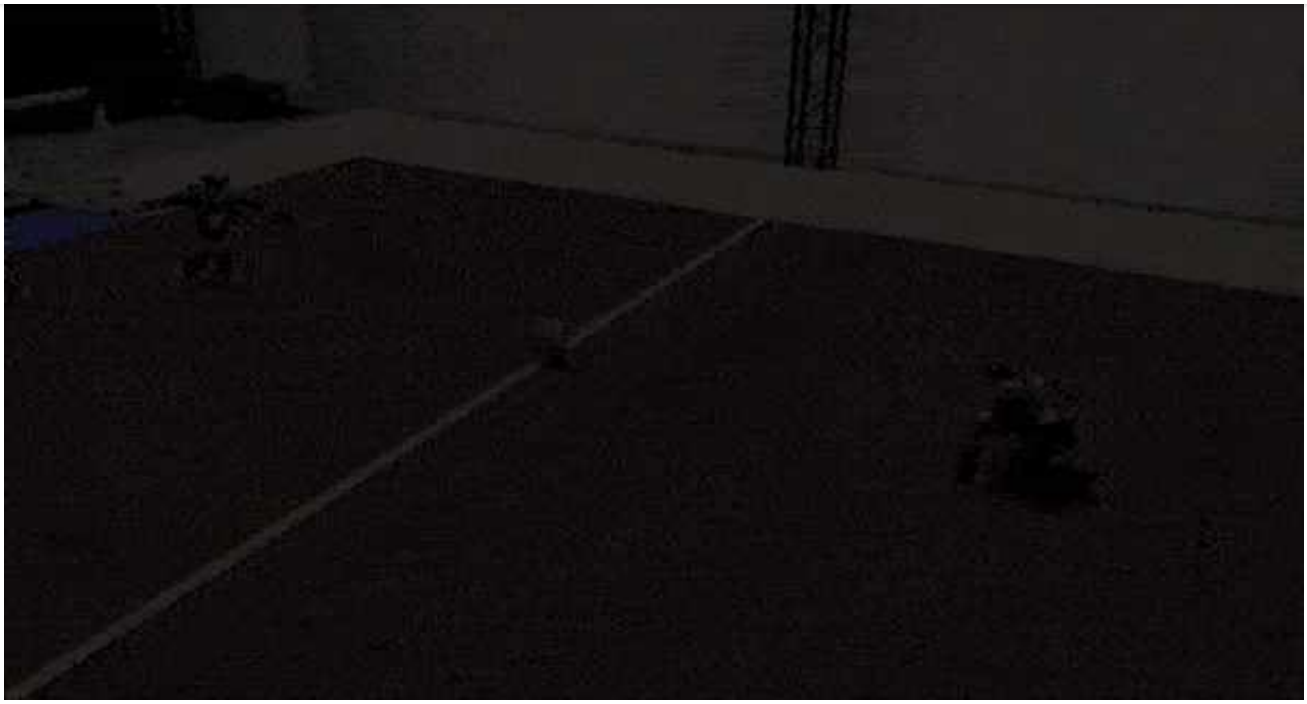
Behind the news: Google initially developed Maven for the U.S. Defense Department around 2017. Palantir inherited the project after Google, facing protests by employees who did not want to contribute to government intelligence systems, declined to renew its contract in 2018. The U.S. military now has more than 800 active AI projects with a wide range of technology partners and contractors. Other countries are deploying similar technology: Israel and Ukraine have used AI-assisted targeting in their ongoing conflicts.

Yes, but: Some U.S. military experts worry about Maven's accuracy. In tests, Maven successfully identified objects about 60 percent of the time, while human analysts working with the 18th Airborne Corps did so 84 percent of time. Moreover, the system's training data emphasizes deserts, and its success rate drops in other types of environments.

Why it matters: Maven and similar systems offer some advantages over human analysts. They can observe and integrate multiple data streams simultaneously, and they can identify potential targets much more quickly. It's likely that more data will make these systems more accurate. On the other hand, they represent a further step toward automated warfare in which automated assistance could come to displace human decision-making.

We're thinking: Automated targeting is increasingly used in military applications, and less-sophisticated systems have been in use for decades. However, humans should always be in control of decisions to fire. We support a global ban on fully autonomous weapons.

DeepLearning.AI
THE BATCH



Robo-Football From Simulation to Reality

Humanoid robots can play football (known as soccer in the United States) in the real world, thanks to reinforcement learning.

What's new: Tuomas Haarnoja and colleagues at Google and University of Oxford trained an agent to play one-on-one football in a simulated environment. They applied the agent to 20-inch hardware robots on a scaled-down field. You can see it in action here.

Key insight: In reinforcement learning, an agent improves as it explores various motions. However, such exploration risks damaging expensive hardware. By training in a simulation, the agent can attempt a diversity of motions without risking a physical robot. Once the agent is trained, it can make the leap from simulation to reality.

How it works: The agent learned in a virtual world to control the robot's motion given (i) a simulated robot's state (including the position, velocity, and acceleration of each of 20 joints), (ii) the current game state (including the location and velocity of the ball

stages.

- During the first stage of training, the authors trained two teachers, both of which were vanilla neural networks. (i) The first teacher learned to predict movements that help a simulated robot score goals against an untrained opponent that immediately fell over. The teacher earned rewards for scoring and was penalized for falling over or letting the opponent score, among other rewards and penalties. (ii) The second teacher learned to make a fallen simulated robot stand up. It received larger rewards for smaller differences, and smaller rewards for larger differences, between the robot's joint positions and the joint positions for key robot poses recorded during a manually designed process of standing up.
- The second stage of training involved another agent, also a vanilla neural network. This agent played a match against a previous version of itself in which each agent controlled a simulated robot. It received rewards for moving the robot's joints in ways that helped it win the match or resembled the two teachers' movements; this encouraged the agent to score goals and stand up after falling. To better approximate real-world conditions, the authors randomly perturbed the simulation, adding noise to the sensors that measured the robot's actions and delaying parts of the simulation. They also restricted the joints' range of motion to prevent the simulated robot from acting in ways that would damage a hardware robot.
- At inference, the trained agent controlled an off-the-shelf Robotis OP3 humanoid robot, which costs around $14,000.

Results: The agent learned not only to turn and kick but also to anticipate the ball's motion and block an opponent's shots. It scored penalties against a stationary goalie with 90 percent success in simulation and 70 percent success in the physical world. It stood up in 0.9 seconds on average, while a manually designed agent stood up in 2.5 seconds. Its maximum walking speed of 0.69 meters per second beat the manually designed agent's 0.27 meters per second. However, its kicks propelled the ball at 2.0 meters per second on average, slower than the manually designed agent's 2.1 meters per second.

require dedicated research. This work drives humanoid robots in complex motions by combining established training methods: training in a noisy simulation, self-play, and using teacher agents to reward particular actions.

We're thinking: This work demonstrates that robots get a kick out of machine learning.

Data Points

The latest AI updates of the week include:

👉 Stability AI's Stable Video 3D
👉 Sakana's evolution-inspired model merging technique
👉 The new Blackwell B200 GPU by Nvidia

And much more.

Read Data Points, your weekly AI news digest.

Share

Subscribe to The Batch

Stay updated with weekly AI News and Insights delivered to your inbox

DeepLearning.AI
THE BATCH

Keep me updated on the latest news, courses, and workshops and events announcements

Submit

Technical Orchestration for the code510 Vehicle Dynamics Transition: Integrating Powertrain, Tire Physics, and Aerodynamic Sub-systems within the AAdvancedRacingVehicle Foundation

The strategic transition from the v1.0.1 Chassis and Telemetry Core to the subsequent implementation phase—encompassing Powertrain, Tires, and Aerodynamics (PTA)—represents a pivotal expansion in the fidelity and complexity of the code510 simulation architecture. As the Orchestrator for this transition, the objective is to synchronize empirical engineering constants with the technical constraints of the Unreal Engine 5 Chaos Physics environment, while simultaneously formalizing the project management and build validation frameworks required for a professional-grade racing simulation. This evolution necessitates a rigorous adherence to the physics of vehicle system dynamics, particularly the non-linear interactions at the tire-road interface and the complex fluid dynamics governing aerodynamic forces.[1]

Fundamental Engineering Constants and Empirical Models for Tire Dynamics

The most critical component of the PTA phase is the tire model, as the contact patch is the sole interface through which all driving, braking, and steering forces are transmitted to the vehicle. The code510 architecture adopts the Pacejka Magic Formula (MF), a standard reference in both industrial vehicle dynamics and high-fidelity racing games due to its ability to fit experimental data with high accuracy.[1]

The Pacejka Magic Formula and Slip Phenomena

Tire friction is not a static coefficient but a dynamic function of slip. The Magic Formula characterizes this behavior through a set of non-linear equations that calculate longitudinal force (F_x), lateral force (F_y), and self-aligning torque (M_z) as functions of vertical load (F_z), slip ratio (σ), and slip angle (α).[1] The general form of the equation is expressed as a sine-based curve:

$$y(x) = D \sin$$

In this model, the parameters B, C, D, and E represent stiffness, shape, peak, and

curvature factors, respectively.[4] These constants are derived through curve-fitting experimental data collected from real-world tire testing. For simulation-grade accuracy, the code510 framework must account for the load-dependence of these factors. For example, the peak factor D is directly proportional to the vertical force F_z, modified by a friction coefficient (μ) that itself varies with load.[5]

Parameter	Identifier	Longitudinal Role	Typical Racing Range
Stiffness Factor	B	Slope at the origin	100 - 500 N/%
Shape Factor	C	Overall curve shape	1.4 - 1.8
Peak Factor	D	Maximum grip value	0.9 - 1.9 (μ)
Curvature Factor	E	Transition to sliding	-0.1 - +0.1

[5]

A significant challenge in implementing Pacejka models within computer code is the "low-speed divergence" problem. Because slip ratio calculations often involve a velocity term in the denominator, the formula can diverge as velocity approaches zero.[1] To mitigate this, the AAdvancedRacingVehicle foundation implements a velocity threshold, such as the VXLOW parameter, which saturates the denominator to ensure mathematical stability during pit-entry speeds or standing starts.[4]

Slip Velocity vs. Slip Ratio in Physics Engines

While standard Pacejka sets often utilize slip ratio (a percentage), some modern physics implementations, such as Vehicle Physics Pro (VPP), utilize slip velocity in meters per second (m/s).[7] For code510, converting between these units requires a normalized tire friction curve where the coefficient of friction (μ) is a function of the actual slip velocity at the contact patch. In these models, a small slip velocity (e.g., < 0.5 m/s) increases the coefficient of friction up to the tire's peak, after which increasing slip reduces grip.[7]

For racing slick tires, the peak μ typically ranges from 1.7 to 1.9, whereas standard road tires achieve a μ of approximately 1.0.[7] The implementation must also consider the alignment of wheel bones; for the physics engine to calculate these forces correctly, the front side of each wheel mesh must face the X-axis, and wheel centers must be accurately aligned

with the suspension axles.[8]

Aerodynamic Coefficients and High-Speed Stability

Aerodynamics in the code510 PTA phase serve two primary functions: reducing drag to maximize top speed and generating downforce to increase the vertical load on the tires, thereby enhancing grip without increasing the vehicle's mass.[7]

Drag and Lift Optimization

The aerodynamic performance of a vehicle is defined by its Coefficient of Drag (C_d) and Coefficient of Lift (C_l). For racing applications, a negative C_l value indicates downforce. The total aerodynamic force is calculated as a function of the square of the vehicle's velocity, meaning aerodynamic influence grows exponentially as speed increases.[9]

Vehicle Class	Drag Coefficient (Cd)	Lift Coefficient (Cl)	L/D Ratio
Typical Passenger Car	0.30 - 0.35	+0.10 to +0.20	Positive Lift
High-Performance Supercar	0.34 - 0.38	-0.10 to -0.30	Low Downforce
GT3 / GTO Race Car	0.45 - 0.55	-0.50 to -1.20	High Downforce
Prototype / Formula 1	0.60 - 0.90	-2.00 to -3.80	Extreme Downforce

[10]

The 1987 Ferrari F40, a benchmark for the code510 project, achieves a C_d of 0.34 through the use of an extremely low front end, a sealed underbelly, and a rear diffuser.[12] However, even with these features, its total C_l is approximately 0.02 (positive lift), indicating that the designers focused on minimizing lift rather than generating pure downforce.[11] In contrast, a modern GT3 car or a DIY track build with a high-mounted wing can achieve C_l values exceeding -1.0, dramatically shifting the aero balance toward the rear axle.[10]

Aero Balance and Downforce Distribution

The distribution of aerodynamic forces between the front and rear axles—known as aero

balance—is critical for high-speed handling. An imbalance can lead to terminal understeer or oversteer as speed increases. For the AAdvancedRacingVehicle, the implementation must account for the center of pressure and how it shifts relative to the center of gravity.[14]

For a balanced racing setup, the ideal C_l is zero over both axles, ensuring that the handling characteristics remain consistent across all speeds.[10] Professional racing teams use complex CFD (Computational Fluid Dynamics) simulations to analyze wake features and pressure zones, ensuring that the flow exiting the underbody diffusers contributes to stability without creating excessive drag.[9]

Powertrain Mechanics and Torque Delivery

The powertrain sub-system is responsible for generating and distributing torque to the wheels. This involves modeling the engine's torque curve, the transmission's gear ratios, and the final drive reduction.[16]

Engine Torque Curves and Transmission Logic

The engine's performance is defined by a Curve Float asset, which maps torque (Nm) against rotational speed (RPM). For the AAdvancedRacingVehicle, this curve typically follows an inverted U-shape, with peak torque occurring in the mid-range.[17]

Comparing the two primary benchmarks for the code510 project reveals distinct powertrain philosophies:

Specification	Ferrari F40 (1987)	Subaru WRX STI (2019)
Engine Layout	Mid-Longitudinal V8	Front-Longitudinal H4
Displacement	2936 cc	2457 cc
Peak Torque	577 Nm @ 4000 RPM	393 Nm @ 4000 RPM
Peak Power	478 hp @ 7000 RPM	310 hp @ 6000 RPM
Transmission	5-Speed Manual	6-Speed Manual
Final Drive	2.90:1	3.90:1

The Ferrari F40's torque is transmitted through a twin-plate clutch to a 5-speed manual gearbox, which features a tall final drive optimized for its 201 mph top speed.[23] The Subaru WRX STI, conversely, utilizes a shorter 3.90 final drive and a sophisticated All-Wheel Drive (AWD) system with a nominal 41:59 torque split managed by a Driver Controlled Center Differential (DCCD).[20]

Differential and Axle Modules

In the UE5 Chaos Physics environment, torque distribution is managed through the AxleModule and WheelModule. These modules must be configured to handle the specific needs of the drivetrain layout. For the STI's AWD system, the implementation requires a limited-slip center differential that can dynamically adjust the torque split based on steering angle and yaw rate sensors.[22] For the F40's RWD layout, a mechanical limited-slip differential (LSD) is modeled to prevent excessive wheel spin on the inside tire during corner exit.[21]

Advanced Suspension Rigging and Implementation Order

The transition to the PTA phase also involves a fundamental shift in how the vehicle's suspension is rigged and simulated. For the AAdvancedRacingVehicle foundation, which features a double-wishbone suspension, the standard linear wheel controller is insufficient for visual fidelity.[17]

Rigging the Double-Wishbone System

A realistic double-wishbone setup requires separating the simulated physics wheels from the rendered visual wheels. The physics engine tracks the position of a "simulated bone" along a Z-axis trace, while the visual components (arms, hubs, and wheels) are driven by an Animation Blueprint using LookAt and CopyBone nodes.[17]

The rigging process involves several critical steps:

- **Pivots and Hierarchy**: The simulated wheel joint, kingpin, hub, and visible wheel must share a common initial pivot location. The lower arm rotates around a chassis pivot, and its motion drives the position of the hub.[17]
- **LookAt Constraints**: The upper and lower arms use LookAt nodes to aim at targets on the kingpin, ensuring they remain mechanically connected as the hub moves vertically.[17]
- **Steering and Spin**: The CopyBone node is used to transfer rotation data (steering and tire spin) from the simulated physics bone to the visible wheel mesh.[17]

Implementation Roadmap for Chaos Vehicles

According to established best practices, the implementation of a Chaos Vehicle must follow a

specific sequence to ensure stability [17]:

1. **Plugin Activation**: Enable ChaosVehiclesPlugin and restart the editor. This system is incompatible with legacy PhysX.[17]
2. **Wheel Blueprints**: Create ChaosVehicleWheel classes for front and rear axles, defining radius, mass, and damping.[16]
3. **Torque Curve**: Define the engine's torque curve as a Curve Float asset.[17]
4. **Rigging and PhAT**: Import the skeletal mesh with correct axis alignment (X=Front, Z=Up) and configure the Physics Asset (PhAT) with simple primitives. Wheels should use sphere collisions, but collision responses for the wheels themselves are often disabled to prevent erratic interactions with the chassis.[8]
5. **Animation Blueprint**: Implement the VehicleAnimationInstance and WheelController nodes to drive the mesh based on physics data.[17]
6. **Vehicle Blueprint**: Assemble the WheeledVehiclePawn, assigning the skeletal mesh, animation class, and configuring the VehicleMovementComponent with the wheel setups and torque curve.[17]

Visual Fidelity and Material Science with UE5 Substrate

To complement the high-fidelity physics models, the code510 project utilizes the Substrate material system. Substrate moves away from traditional shading models in favor of "slabs of matter" parameterized by physical properties, enabling the creation of complex, layered surfaces such as oily or wet tarmac.[28]

Authoring the Oily Tarmac Material

Authoring an "Oily Tarmac" surface involves layering a translucent liquid slab over an opaque pavement slab. This is achieved using the Substrate Vertical Layer node, which calculates how light interacts with both layers.[28]

Technical parameters for the oily surface include:

- **Roughness**: The oil layer is assigned a low roughness (e.g., 0.05) to create sharp, glossy reflections, while the underlying tarmac uses a high roughness (e.g., 0.8).[28]
- **F0 and F90**: Reflectance at normal and grazing angles. For the oily layer (a dielectric), F0 is typically set between 0.02 and 0.04 to mimic the refractive index of petroleum-based liquids.[28]
- **Mean Free Path (MFP)**: This parameter defines the transmittance of the top layer. For an oily sheen, a high MFP allows the tarmac's color and texture to remain visible through the liquid.[28]
- **Top Thickness**: This scalar value (measured in cm) controls the depth of the oil film. In the material graph, this can be modulated by a noise texture to simulate irregular pooling

or drainage.[28]

The adoption of Adaptive GBuffer is necessary for these complex materials, as it allows for the high byte-per-pixel budget required to evaluate multiple material closures (e.g., diffuse, specular, and rough refraction) simultaneously.[28]

Project Management and Workflow Configuration (Document 116)

The code510 Orchestrator manages the transition phase through the revamped GitHub Projects system. This system provides an abstract management layer at the organization level, allowing the team to track progress across the various repositories involved in the PTA transition—including those for physics code, vehicle assets, and environment materials.[32]

Workflow Views and Status Columns

The project board is structured to provide different perspectives on the transition:

- **Kanban Board**: The primary horizontal view for task tracking. It includes the standard "To-Do," "In Progress," and "Done" columns, supplemented by a custom "Up Next" status (color-coded blue) representing the short list of prioritized upcoming tasks for the current sprint.[32]
- **Roadmap View**: Used for long-term planning, this view defines the time bars for major PTA milestones and identifies potential dependencies between sub-systems (e.g., the tire friction model must be validated before aerodynamic balance testing can commence).[32]
- **Table View**: A spreadsheet-style layout used for bulk editing of issue metadata, such as assigning labels for "Powertrain," "Tires," or "Aero" to individual tasks.[32]

Automation Rules for Issue Management

To streamline the workflow, the Orchestrator configures automation rules that automatically ingest and organize work items. The "Auto-Add to Project" rule is set to monitor the AAdvancedRacingVehicle repository, automatically adding any new issues to the project board if they match specific query filters (e.g., excluding pull requests and focusing on items with the "Phase-PTA" label).[32]

Synchronization between individual issues and the project board is bidirectional. When a task is moved to "In Progress" on the Kanban board, the change is reflected in the issue's metadata, ensuring that the engineering team has real-time visibility into the current focus of the Orchestrator.[32]

Build Validation and CI/CD for

AAdvancedRacingVehicle (Document 114)

The final requirement for the PTA transition is the establishment of a robust build validation pipeline. Automated building and packaging ensure that changes to the core physics or material shaders do not break the project's ability to run on target hardware.

UE5-Build-Project GitHub Action

The code510 project utilizes the UE5-Build-Project GitHub Action, which leverages Unreal Engine's RunUAT.bat script to automate the build lifecycle.[32] This action is essential for the continuous integration of the AAdvancedRacingVehicle foundation.

Configuration parameters for the build validation workflow include:

- **RUNUAT_PATH**: Must be set to the engine's batch files directory (e.g., Engine/Build/BatchFiles/RunUAT.bat).[32]
- **UPROJECT_PATH**: The full path to the AAdvancedRacingVehicle.uproject file.[32]
- **BUILD_CONFIG**: Typically set to Development for ongoing validation or Shipping for final performance testing.[32]
- **NULLRHI**: Set to true for cloud-based runners (such as GitHub-hosted runners) that lack a physical display or GPU output.[32]

Validation Stages and Process Flags

The build process is divided into several stages, each controlled by specific flags:

- **CLEAN**: Set to true to perform a full rebuild, ensuring no stale artifacts remain from previous versions of the chassis core.[32]
- **COOK**: This critical stage processes all assets (e.g., torque curves, skeletal meshes, and Substrate materials) into the optimized format required by the target platform (e.g., Win64).[32]
- **STAGE and PACKAGE**: These stages organize the cooked assets into a deployable package and create .pak files for efficient loading.[32]

Successful completion of this workflow serves as the "green light" for the Orchestrator to proceed with more granular testing, such as time attack runs to validate acceleration and high-speed stability against the F40 and STI benchmarks.[27]

Synthesis of Physics Constants and Simulator Validation

The integrity of the AAdvancedRacingVehicle simulation is verified by comparing its performance against empirical data and theoretical expectations. This synthesis ensures that the PTA sub-systems function as a unified physical entity rather than a collection of

independent modifiers.

Comparison of Static and Dynamic Loads

Validation begins with the static state. The vehicle's curb weight and distribution—e.g., 44/56 for the F40 or 50/50 for a generic racing sedan—are checked against the normal force (F_z) calculated at each tire patch.[7] Under dynamic conditions, the simulation must accurately account for longitudinal load transfer during acceleration and braking, as well as lateral transfer during cornering.[7]

Phenomenon	F40 Target Behavior	STI Target Behavior	Validation Metric
0-100 km/h (62 mph)	4.1 Seconds	4.5 - 5.1 Seconds	Acceleration Time
0-400 m (1/4 Mile)	11.9 Seconds	13.0 - 13.5 Seconds	Acceleration Distance
Top Speed	201 mph	155 - 165 mph	Max Velocity
Lateral Acceleration	1.1 - 1.2 G	0.9 - 1.0 G	Skidpad (300 ft)

18

Deviations from these targets often indicate a misconfiguration in the Pacejka stiffness factor or an incorrect aerodynamic C_d value. For instance, if the F40 fails to reach its top speed despite having the correct torque curve, the Orchestrator must investigate whether the drag coefficient or the final drive ratio is the limiting factor.[7]

Mechanical vs. Aerodynamic Grip

A nuanced insight from the research suggests that as the vehicle moves through its performance envelope, the source of its grip shifts. At low speeds, mechanical grip (governed by spring rates and tire μ) dominates.[7] As speed increases, aerodynamic downforce (F_z increase) begins to counteract the reduction in tire friction coefficient that occurs under high slip.

In the STI, the rear wing provides essential stability at high speeds, compensating for the lift naturally produced by its sedan body shape.[38] For the F40, the aerodynamics are tuned to achieve "near neutral" lift, ensuring that the car remains predictable at 200 mph without the

massive drag penalty associated with higher-downforce racing wings.[11]

Conclusions and Orchestrator Recommendations

The transition from the v1.0.1 Chassis / Telemetry Core to the PTA phase for the AAdvancedRacingVehicle foundation is now technically defined. The synthesis of empirical automotive engineering data with the Unreal Engine 5 Chaos Physics framework provides a robust path toward achieving simulator-grade fidelity.

The Orchestrator concludes that the implementation should prioritize the following key areas:

1. **Tire Model Fidelity**: The Pacejka Magic Formula must be the cornerstone of the physics implementation. Particular attention must be paid to the low-speed denominator saturation to ensure simulation stability during transition from rest.[1]
2. **Rigging Accuracy**: The double-wishbone suspension system requires a decoupled rigging approach using the Animation Blueprint's LookAt and CopyBone nodes to maintain visual mechanical integrity.[17]
3. **Aerodynamic Balance**: C_l and C_d values must be calibrated to replicate the specific high-speed stability of the F40 and STI benchmarks, with careful monitoring of the center of pressure relative to the center of gravity.[10]
4. **Workflow Automation**: The GitHub Project board must be used to strictly manage the multi-repository dependencies inherent in this transition, utilizing automation to ensure real-time status visibility across the engineering team.[32]
5. **Build Reliability**: A CI/CD pipeline using the UE5-Build-Project action must be the final gate for all physics and asset changes, ensuring that the simulation remains consistently packageable and valid across all target platforms.[32]

By strictly following this roadmap, the code510 project will successfully transition into its most complex implementation phase, establishing the AAdvancedRacingVehicle as a premiere high-fidelity digital twin for automotive research and racing simulation.[2]

Works cited

1. Hans B. Pacejka - Wikipedia, accessed January 15, 2026, https://en.wikipedia.org/wiki/Hans_B._Pacejka
2. Vehicle Engineering Simulation Software | Ansys, accessed January 15, 2026, https://www.ansys.com/industries/transportation-and-mobility/vehicle-engineering
3. Digital Twin in Automotive Industry: Transforming the Future | Neural Concept, accessed January 15, 2026, https://www.neuralconcept.com/post/digital-twin-in-automotive-industry-transforming-the-future
4. Tire-road dynamics given by Magic Formula coefficients - MATLAB - MathWorks, accessed January 15, 2026,

https://www.mathworks.com/help/sdl/ref/tireroadinteractionmagicformula.html
5. Pacejka '94 parameters explained – a comprehensive guide - Edy's Projects, accessed January 15, 2026, https://www.edy.es/dev/docs/pacejka-94-parameters-explained-a-comprehensive-guide/
6. How to program realistic vehicle physics for realtime environments / games, Part I: simple suspension and realistic tire dynamics (C++) | by Rem Voorhuis | Medium, accessed January 15, 2026, https://medium.com/@remvoorhuis/how-to-program-realistic-vehicle-physics-for-realtime-environments-games-part-i-simple-b4c2375dc7fa
7. Tires - Vehicle Physics Pro, accessed January 15, 2026, https://vehiclephysics.com/blocks/tires/
8. The Ultimate Chaos Vehicle Guide | Neutronio Games, accessed January 15, 2026, https://neutroniogames.wordpress.com/2024/01/17/the-ultimate-chaos-vehicle-guide/
9. Optimizing Vehicle Aerodynamics with Digital Twin and Automotive CFD - Tecnologia - Techsim.sk, accessed January 15, 2026, https://www.techsim.sk/2025/09/03/optimizing-vehicle-aerodynamics-with-digital-twin-and-automotive-cfd/
10. Thinking in Aerodynamic Coefficients – Occam's Racer, accessed January 15, 2026, https://occamsracers.com/2024/03/22/thinking-in-aerodynamic-coefficients/
11. *Ferrari F40* Aerodynamics - YouTube, accessed January 15, 2026, https://www.youtube.com/watch?v=m-HnB05jwGc
12. The Ferrari F40 - The Classic Supercar - Silodrome, accessed January 15, 2026, https://silodrome.com/ferrari-f40/
13. Tag: coefficient of drag - Occam's Racer, accessed January 15, 2026, https://occamsracers.com/tag/coefficient-of-drag/
14. car simulation lift and drag numbers don't make sense - CFD Online, accessed January 15, 2026, https://www.cfd-online.com/Forums/openfoam-solving/259225-car-simulation-lift-drag-numbers-dont-make-sense.html
15. Validation of the front and rear lift coefficient CL,i from CFD and... - ResearchGate, accessed January 15, 2026, https://www.researchgate.net/figure/Validation-of-the-front-and-rear-lift-coefficient-CL-i-from-CFD-and-wind-tunnel-tests-at_tbl7_301272945
16. Unreal Engine 5 Tutorial: Create a Working Chaos Vehicle (Step-by-Step) | by Shaun Fulton, accessed January 15, 2026, https://medium.com/@fulton_shaun/unreal-engine-5-tutorial-create-a-working-chaos-vehicle-step-by-step-ff30afad1d16
17. How to Set up Vehicles in Unreal Engine | Unreal Engine 5.7 ..., accessed January 15, 2026, https://dev.epicgames.com/documentation/en-us/unreal-engine/how-to-set-up-vehicles-in-unreal-engine

18. Ferrari F40 Specifications - SBR Engineering, accessed January 15, 2026, https://sbraceengineering.co.uk/ferrari-specialists/ferrari-models-1980-to-1989/ferrari-f40/
19. Ferrari F40 - Wikipedia, accessed January 15, 2026, https://en.wikipedia.org/wiki/Ferrari_F40
20. 2019 Subaru WRX STI, accessed January 15, 2026, https://www.mikeshawsubaru.com/2019-subaru-wrx-sti.htm
21. CURRENT BASE PRICE dollars x 1000 - FERRARI F40 - PORSCHE 928GT - Amazon S3, accessed January 15, 2026, https://s3.amazonaws.com/amv-prod-cad-assets/files/ferrari-f40-archived-test-review-car-and-driverferrari-f40-full-specs.pdf
22. 2019 Subaru WRX® STI®, accessed January 15, 2026, http://subarumedia.iconicweb.com/mediasite/attachments/2019_STI_specs-FINAL.PDF
23. 1990 Ferrari F40 US-Spec - Supercars.net, accessed January 15, 2026, https://www.supercars.net/blog/1990-ferrari-f40-us-spec/
24. Ferrari F40 (1987) - Ferrari.com, accessed January 15, 2026, https://www.ferrari.com/en-EN/auto/f40
25. Realistic Vehicle Simulation – ChaosVehicle vs Chaos Modular Vehicle? - Programming & Scripting - Epic Developer Community Forums, accessed January 15, 2026, https://forums.unrealengine.com/t/realistic-vehicle-simulation-chaosvehicle-vs-chaos-modular-vehicle/2608108
26. Chaos Physics Overview | Unreal Engine 4.27 Documentation | Epic Developer Community, accessed January 15, 2026, https://dev.epicgames.com/documentation/en-us/unreal-engine/chaos-physics-overview?application_version=4.27
27. [Chaos Vehicle] Experiences with the new vehicle implementation [+ SkidMark implementation] - World Creation - Epic Developer Community Forums, accessed January 15, 2026, https://forums.unrealengine.com/t/chaos-vehicle-experiences-with-the-new-vehicle-implementation-skidmark-implementation/151400
28. Overview of Substrate Materials in Unreal Engine - Epic Games Developers, accessed January 15, 2026, https://dev.epicgames.com/documentation/en-us/unreal-engine/overview-of-substrate-materials-in-unreal-engine
29. Exploring Substrate Materials Basic to Advanced Techniques [Unreal Fest Bali 2025] | ドクセル, accessed January 15, 2026, https://www.docswell.com/s/EpicGamesJapan/ZRE424-unreal-fest-bali-substrate
30. Authoring a Puddle of Water with Substrate Materials | Epic Developer Community, accessed January 15, 2026, https://dev.epicgames.com/community/learning/tutorials/m3Rq/unreal-engine-authoring-a-puddle-of-water-with-substrate-materials
31. Theory behind F0, F90, and Specular Color - A Deep Dive on Substrate Materials, accessed January 15, 2026,

https://dev.epicgames.com/community/learning/courses/Emv/unreal-engine-a-deep-dive-on-substrate-materials
32. Document 116.pdf
33. [Open Source] Ferrari F40 1987 (A Class) - Tuning - Official Forza Community Forums, accessed January 15, 2026, https://forums.forza.net/t/open-source-ferrari-f40-1987-a-class/6626
34. Tech Tip: Springs & Dampers, Part Two - OptimumG, accessed January 15, 2026, https://optimumg.com/wp-content/uploads/2020/01/SpringsDampers_Tech_Tip_2.pdf
35. Spring Rate Calculator - What Is It and How It Is Calculated - Call To Grid, accessed January 15, 2026, https://calltogrid.com/spring-rate-guide-calculator/
36. 2019 Subaru WRX STI: Price, Review, Photos and Specs - Read This Before You Buy, accessed January 15, 2026, https://driving.ca/subaru/wrx-sti/2019/
37. ChassisSim approach to lap time simulation – What makes us unique - Danny Nowlan Motorsport biography, accessed January 15, 2026, https://www.chassissim.com/wp-content/uploads/CSim_laptime_sim_approach.pdf
38. 2019 - WRX/WRX STI - Dealer E Process, accessed January 15, 2026, https://cdn.dealereprocess.org/cdn/brochures/subaru/2019-wrx.pdf
39. Ferrari F40 - AutoZine, accessed January 15, 2026, https://www.autozine.org/Archive/Ferrari/classic/F40.html
40. Digital Twin in Military Ground Vehicles: Design and Predictive Maintenance - Clemson OPEN, accessed January 15, 2026, https://open.clemson.edu/context/all_theses/article/5351/viewcontent/Eddy_Thesis.pdf

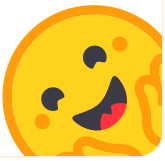
Search models, datasets, users...

Deep RL Course documentation
Q-Learning Recap

Join the Hugging Face community

and get access to the augmented documentation experience

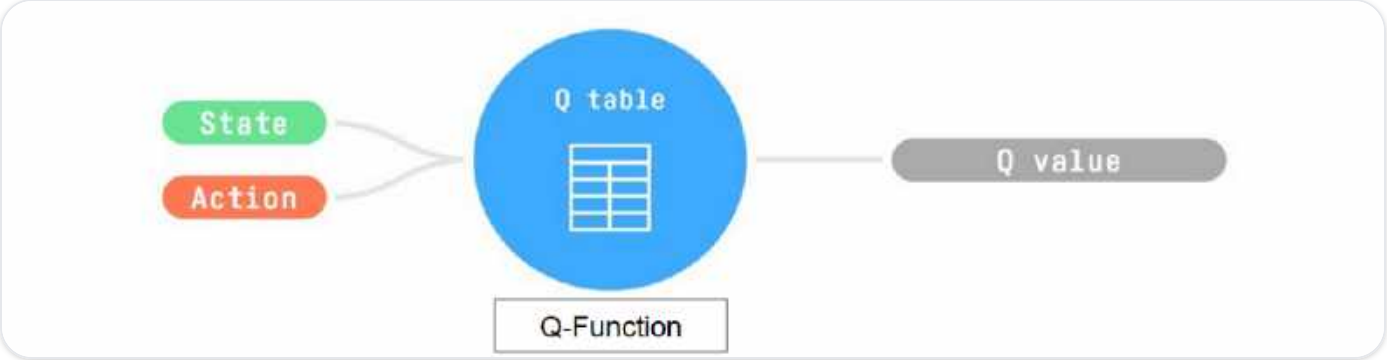
Sign Up to get started



Q-Learning Recap

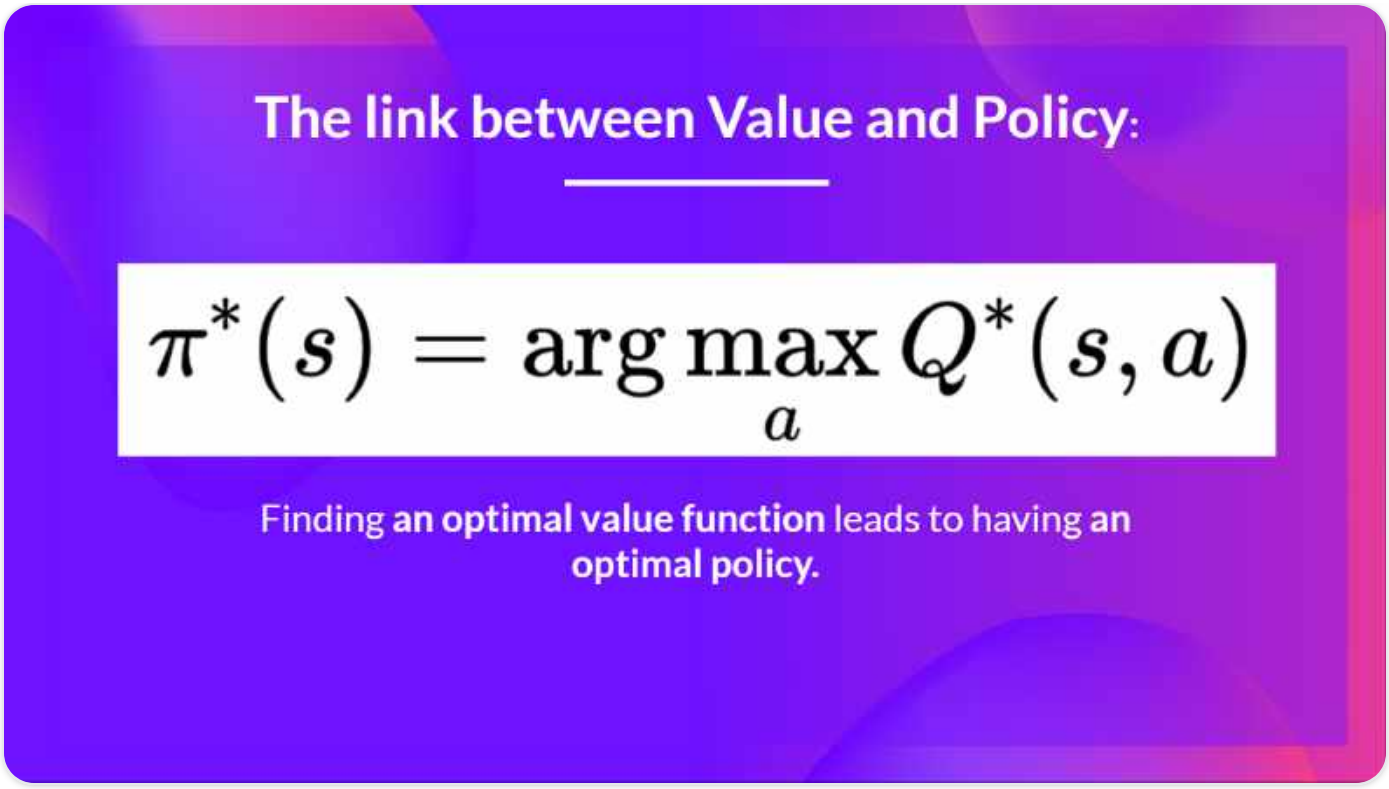
Q-Learning **is the RL algorithm that** :

- Trains a *Q-function*, an **action-value function** encoded, in internal memory, by a *Q-table* **containing all the state-action pair values.**
- Given a state and action, our Q-function **will search its Q-table for the corresponding value.**

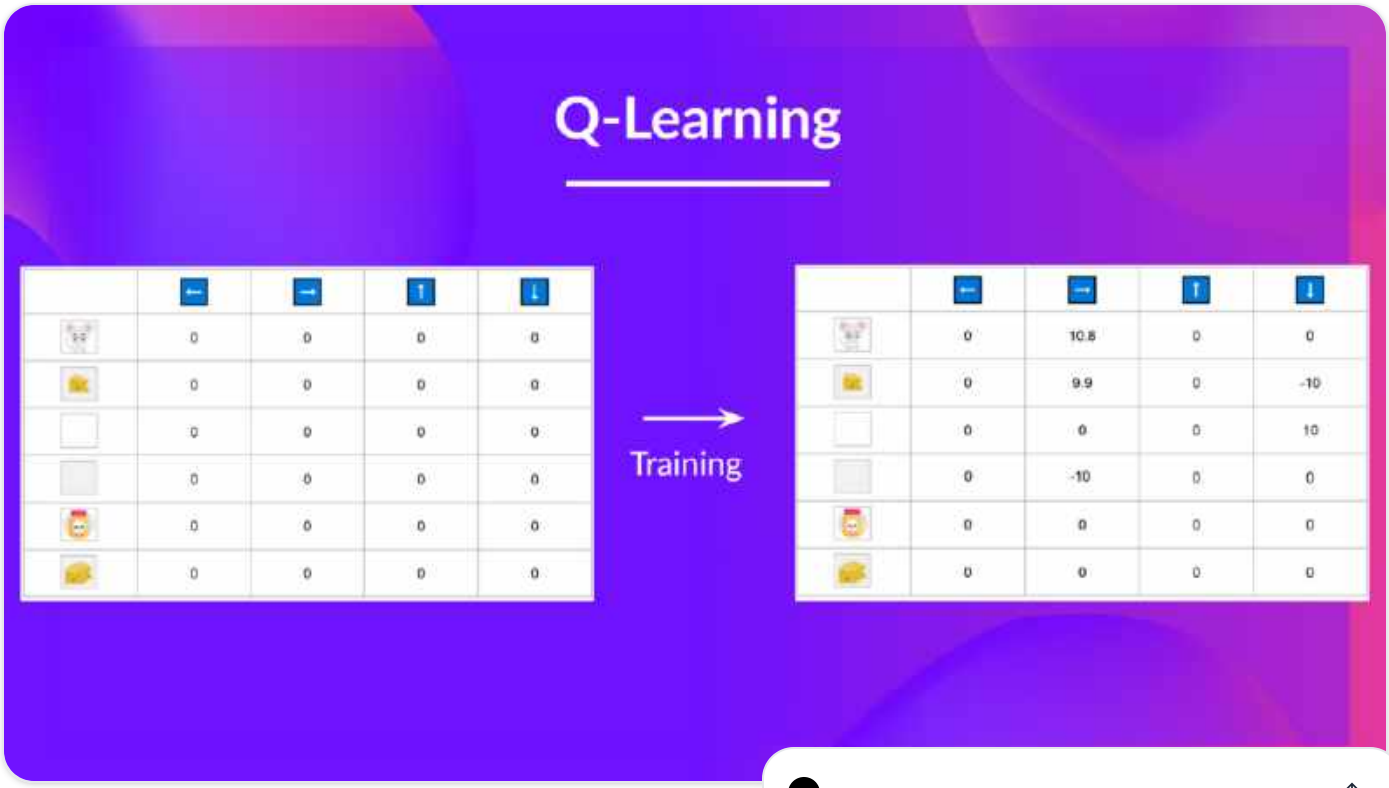


- When the training is done, **we have an optimal Q-function, or, equivalently, an optimal Q-table.**
- And if we **have an optimal Q-function**, we have an optimal policy, since we **know, for each state, the best action to take.**

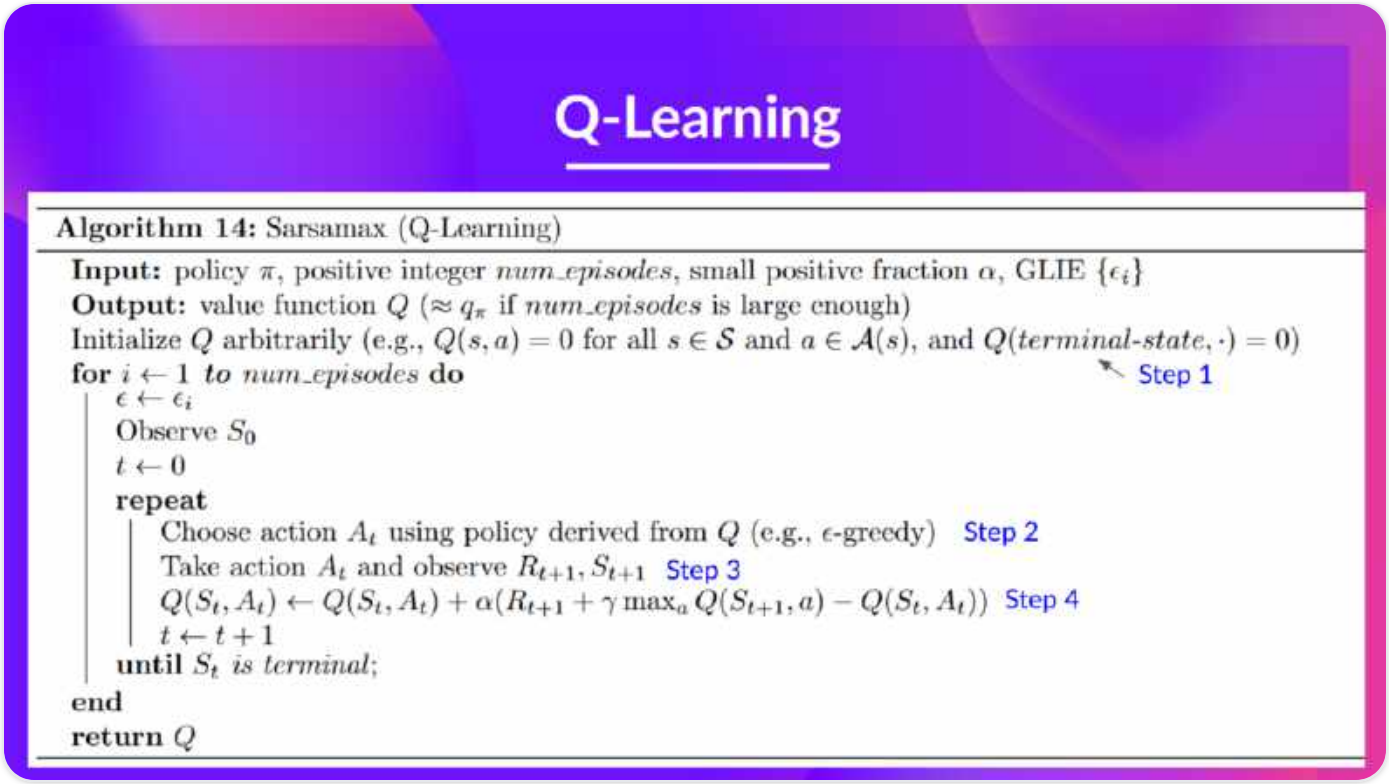
Ask HuggingChat



But, in the beginning, our **Q-table is useless since it gives arbitrary values for each state-action pair (most of the time we initialize the Q-table to 0 values)**. But, as we explore the environment and update our Q-table it will give us a better and better approximation.



This is the Q-Learning pseudocode:



<> Update on GitHub

← A Q-Learning example

Glossary →

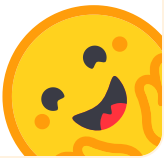
Search models, datasets, users...

Deep RL Course documentation
Quiz

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Quiz

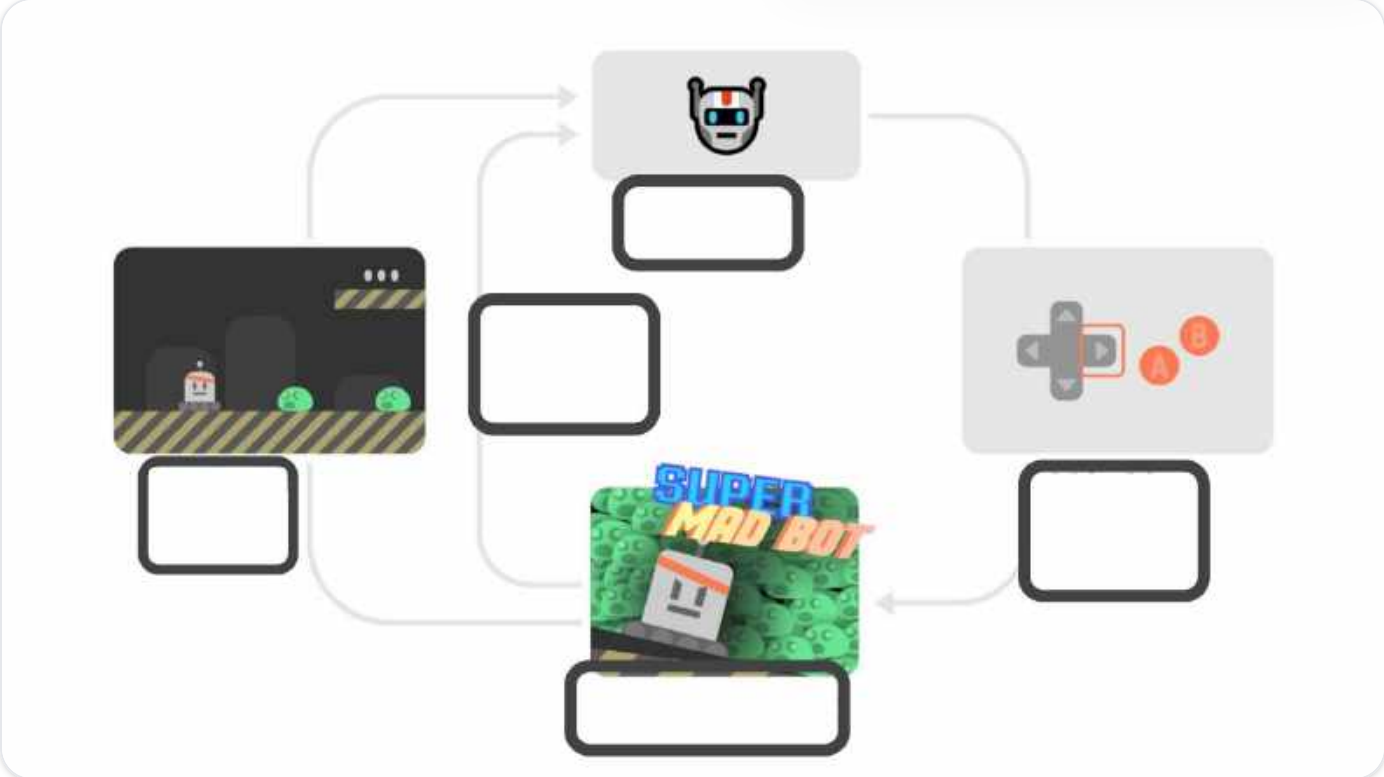
The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: What is Reinforcement Learning?

▶ Solution

Q2: Define the RL Loop

Ask HuggingChat



At every step:

- Our Agent receives __ from the environment
- Based on that __ the Agent takes an __
- Our Agent will move to the right
- The Environment goes to a __
- The Environment gives a __ to the Agent

- [ ] an action a0, action a0, state s0, state s1, reward r1
- [ ] state s0, state s0, action a0, new state s1, reward r1
- [ ] a state s0, state s0, action a0, state s1, action a1

Submit

Q3: What's the difference between a state and an observation?

- [ ] The state is a complete description of the state of the world (there is no hidden information)
- [ ] The state is a partial description of the state

- [ ] The observation is a complete description of the state of the world (there is no hidden information)
- [ ] The observation is a partial description of the state
- [ ] We receive a state when we play with chess environment
- [ ] We receive an observation when we play with chess environment
- [ ] We receive a state when we play with Super Mario Bros
- [ ] We receive an observation when we play with Super Mario Bros

Submit

Q4: A task is an instance of a Reinforcement Learning problem. What are the two types of tasks?

- [ ] Episodic
- [ ] Recursive
- [ ] Adversarial
- [ ] Continuing

Submit

Q5: What is the exploration/exploitation tradeoff?

▶ Solution

Q6: What is a policy?

▶ Solution

Q7: What are value-based methods?

▶ Solution

Q8: What are policy-based methods?

▶ Solution

Congrats on finishing this Quiz 🥳, if you missed some elements, take time to read again the chapter to reinforce (😉) your knowledge, but **do not worry**: during the course we'll go over again of these concepts, and you'll **reinforce your theoretical knowledge with hands-on**.

<> Update on GitHub

← Hands-on Conclusion →

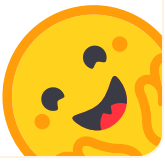
Search models, datasets, users...

Deep RL Course documentation
RLHF

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



RLHF

Reinforcement learning from human feedback (RLHF) is a **methodology for integrating human data labels into a RL-based optimization process**. It is motivated by the **challenge of modeling human preferences**.

For many questions, even if you could try and write down an equation for one ideal, humans differ on their preferences.

Updating models **based on measured data is an avenue to try and alleviate these inherently human ML problems**.

Start Learning about RLHF

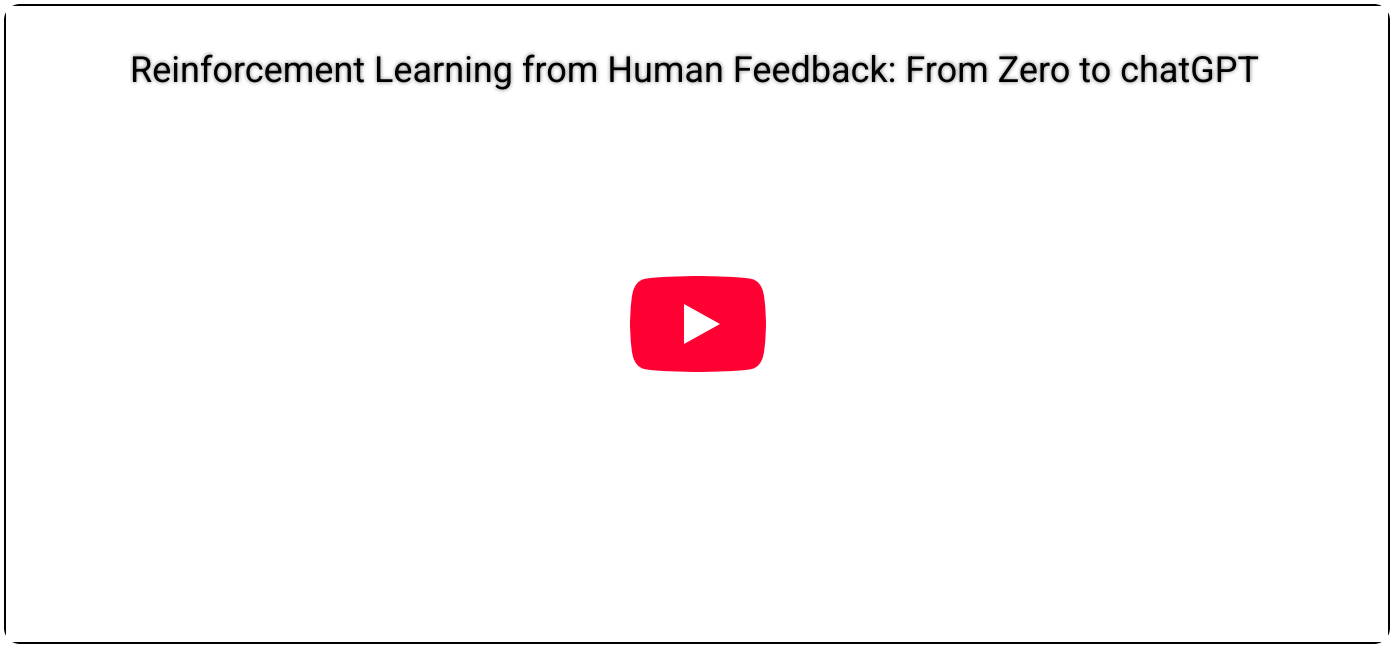
To start learning about RLHF:

1. Read this introduction: Illustrating Reinforcement Learning from Human Feedback (RLHF).

2. Watch the recorded live we did some weeks ago, where Nathan covered the basics of Reinforcement Learning from Human Feedback (RLHF) and how this technology is being used to enable state-of-the-art ML tools like ChatGPT. Most of the talk is an overview of the

Ask HuggingChat

interconnected ML models. It covers the basics of Natural Language Processing and RL and how RLHF is used on large language models. We then conclude with open questions in RLHF.



3. Read other blogs on this topic, such as Closed-API vs Open-source continues: RLHF, ChatGPT, data moats. Let us know if there are more you like!

Additional readings

Note, this is copied from the Illustrating RLHF blog post above. Here is a list of the most prevalent papers on RLHF to date. The field was recently popularized with the emergence of DeepRL (around 2017) and has grown into a broader study of the applications of LLMs from many large technology companies. Here are some papers on RLHF that pre-date the LM focus:

- TAMER: Training an Agent Manually via Evaluative Reinforcement (Knox and Stone 2008): Proposed a learned agent where humans provided scores on the actions taken iteratively to learn a reward model.
- Interactive Learning from Policy-Dependent Human Feedback (MacGlashan et al. 2017): Proposed an actor-critic algorithm, COACH, where human feedback (both positive and negative) is used to tune the advantage function.

- Deep Reinforcement Learning from Human Preferences (Christiano et al. 2017): RLHF applied on preferences between Atari trajectories.
- Deep TAMER: Interactive Agent Shaping in High-Dimensional State Spaces (Warnell et al. 2018): Extends the TAMER framework where a deep neural network is used to model the reward prediction.

And here is a snapshot of the growing set of papers that show RLHF's performance for LMs:

- Fine-Tuning Language Models from Human Preferences (Zieglar et al. 2019): An early paper that studies the impact of reward learning on four specific tasks.
- Learning to summarize with human feedback (Stiennon et al., 2020): RLHF applied to the task of summarizing text. Also, Recursively Summarizing Books with Human Feedback (OpenAI Alignment Team 2021), follow on work summarizing books.
- WebGPT: Browser-assisted question-answering with human feedback (OpenAI, 2021): Using RLHF to train an agent to navigate the web.
- InstructGPT: Training language models to follow instructions with human feedback (OpenAI Alignment Team 2022): RLHF applied to a general language model [Blog post on InstructGPT].
- GopherCite: Teaching language models to support answers with verified quotes (Menick et al. 2022): Train a LM with RLHF to return answers with specific citations.
- Sparrow: Improving alignment of dialogue agents via targeted human judgements (Glaese et al. 2022): Fine-tuning a dialogue agent with RLHF
- ChatGPT: Optimizing Language Models for Dialogue (OpenAI 2022): Training a LM with RLHF for suitable use as an all-purpose chat bot.
- Scaling Laws for Reward Model Overoptimization (Gao et al. 2022): studies the scaling properties of the learned preference model in RLHF.
- Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback (Anthropic, 2022): A detailed documentation of training a LM assistant with RLHF.
- Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned (Ganguli et al. 2022): A detailed documentation of efforts to "discover, measure, and attempt to reduce [language models] potentially h

- Dynamic Planning in Open-Ended Dialogue using Reinforcement Learning (Cohen at al. 2022): Using RL to enhance the conversational skill of an open-ended dialogue agent.
- Is Reinforcement Learning (Not) for Natural Language Processing?: Benchmarks, Baselines, and Building Blocks for Natural Language Policy Optimization (Ramamurthy and Ammanabrolu et al. 2022): Discusses the design space of open-source tools in RLHF and proposes a new algorithm NLPO (Natural Language Policy Optimization) as an alternative to PPO.

Author

This section was written by Nathan Lambert

<> Update on GitHub

← Generalisation Reinforcement Learning Decision Transformers and Offline RL →

Search models, datasets, users...

Deep RL Course documentation
Self-Play: a classic technique to train competitive agents in adversarial games

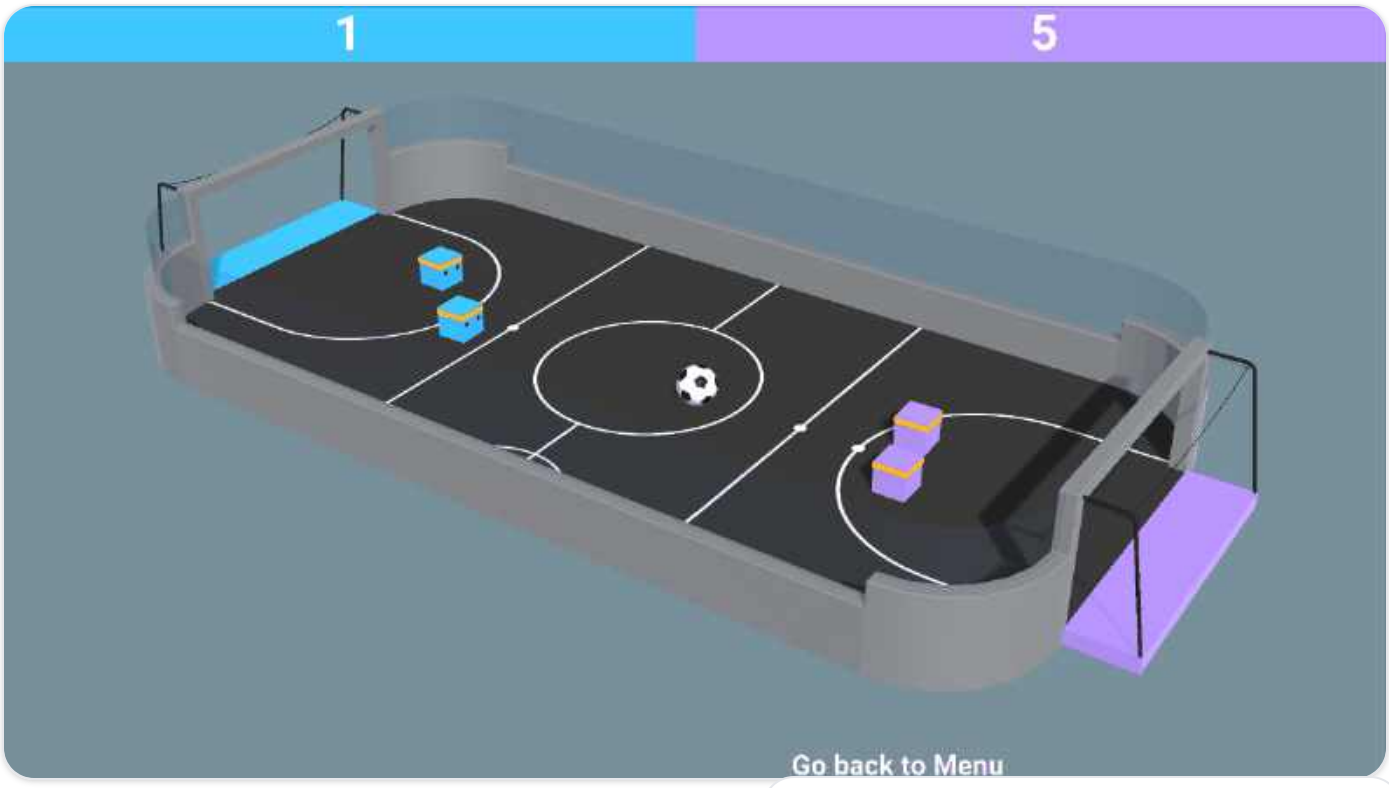
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

Self-Play: a classic technique to train competitive agents in adversarial games

Now that we've studied the basics of multi-agents, we're ready to go deeper. As mentioned in the introduction, we're going **to train agents in an adversarial game with SoccerTwos, a 2vs2 game**.



This environment was made by the Unity MLAgents Team

Ask HuggingChat

What is Self-Play?

Training agents correctly in an adversarial game can be **quite complex**.

On the one hand, we need to find how to get a well-trained opponent to play against your training agent. And on the other hand, if you find a very good trained opponent, how will your agent improve its policy when the opponent is too strong?

Think of a child that just started to learn soccer. Playing against a very good soccer player will be useless since it will be too hard to win or at least get the ball from time to time. So the child will continuously lose without having time to learn a good policy.

The best solution would be **to have an opponent that is on the same level as the agent and will upgrade its level as the agent upgrades its own**. Because if the opponent is too strong, we'll learn nothing; if it is too weak, we'll overlearn useless behavior against a stronger opponent then.

This solution is called *self-play*. In self-play, **the agent uses former copies of itself (of its policy) as an opponent**. This way, the agent will play against an agent of the same level (challenging but not too much), have opportunities to gradually improve its policy, and then update its opponent as it becomes better. It's a way to bootstrap an opponent and progressively increase the opponent's complexity.

It's the same way humans learn in competition:

- We start to train against an opponent of similar level
- Then we learn from it, and when we acquire some skills, we can move further with stronger opponents.

We do the same with self-play:

- We **start with a copy of our agent as an opponent** this way, this opponent is on a similar level.
- We **learn from it** and, when we acquire some skills, we **update our opponent with a more recent copy of our training policy**.

The theory behind self-play is not something new. It was already used by Arthur Samuel's checker player system in the fifties and by Gerald Tesauro's TD-Gammon in 1995. If you want to learn more about the history of self-play check out this very good blogpost by Andrew Cohen

Self-Play in MLAgents

Self-Play is integrated into the MLAgents library and is managed by multiple hyperparameters that we're going to study. But the main focus, as explained in the documentation, is the **tradeoff between the skill level and generality of the final policy and the stability of learning**.

Training against a set of slowly changing or unchanging adversaries with low diversity **results in more stable training. But there is a risk of overfitting if the change is too slow.**

So we need to control:

- How **often we change opponents** with the `swap_steps` and `team_change` parameters.
- The **number of opponents saved** with the `window` parameter. A larger value of `window` means that an agent's pool of opponents will contain a larger diversity of behaviors since it will contain policies from earlier in the training run.
- The **probability of playing against the current self vs opponent** sampled from the pool with `play_against_latest_model_ratio`. A larger value of `play_against_latest_model_ratio` indicates that an agent will be playing against the current opponent more often.
- The **number of training steps before saving a new opponent** with `save_steps` parameters. A larger value of `save_steps` will yield a set of opponents that cover a wider range of skill levels and possibly play styles since the policy receives more training.

To get more details about these hyperparameters, you definitely need to check out this part of the documentation

The ELO Score to evaluate our agent

What is ELO Score?

In adversarial games, tracking the **cumulative reward is not always a meaningful metric to track the learning progress:** because this metric is **dependent only on the skill of the opponent.**

Instead, we're using an ***ELO rating system*** (named after Arpad Elo) that calculates the **relative skill level** between 2 players from a given population in a zero-sum game.

In a zero-sum game: one agent wins, and the other agent loses. It's a mathematical representation of a situation in which each participant's gain or loss of utility **is exactly balanced by the gain or loss of the utility of the other participants.** We talk about zero-sum games because the sum of utility is equal to zero.

This ELO (starting at a specific score: frequently 1200) can decrease initially but should increase progressively during the training.

The Elo system is **inferred from the losses and draws against other players.** It means that player ratings depend **on the ratings of their opponents and the results scored against them.**

Elo defines an Elo score that is the relative skills of a player in a zero-sum game. **We say relative because it depends on the performance of opponents.**

The central idea is to think of the performance of a player **as a random variable that is normally distributed.**

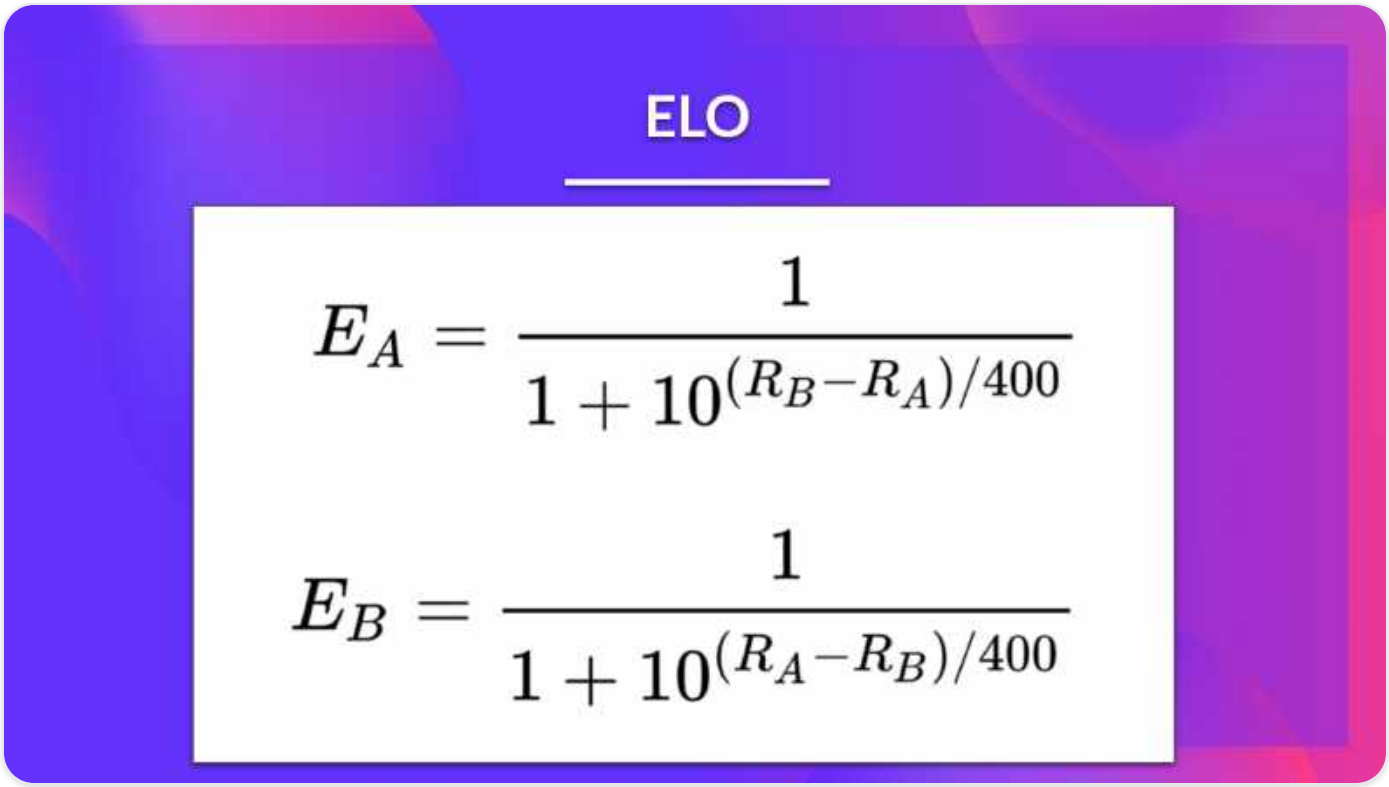
The difference in rating between 2 players serves as **the predictor of the outcomes of a match.** If the player wins, but the probability of winning is high, it will only win a few points from its opponent since it means that it is much stronger than it.

After every game:

- The winning player takes **points from the losing one.**
- The number of points is determined **by the difference in the 2 players ratings (hence relative).**
    - If the higher-rated player wins → few points will be taken from the lower-rated player.
    - If the lower-rated player wins → a lot of point

- If it's a draw → the lower-rated player gains a few points from the higher.

So if A and B have rating Ra, and Rb, then the **expected scores are** given by:

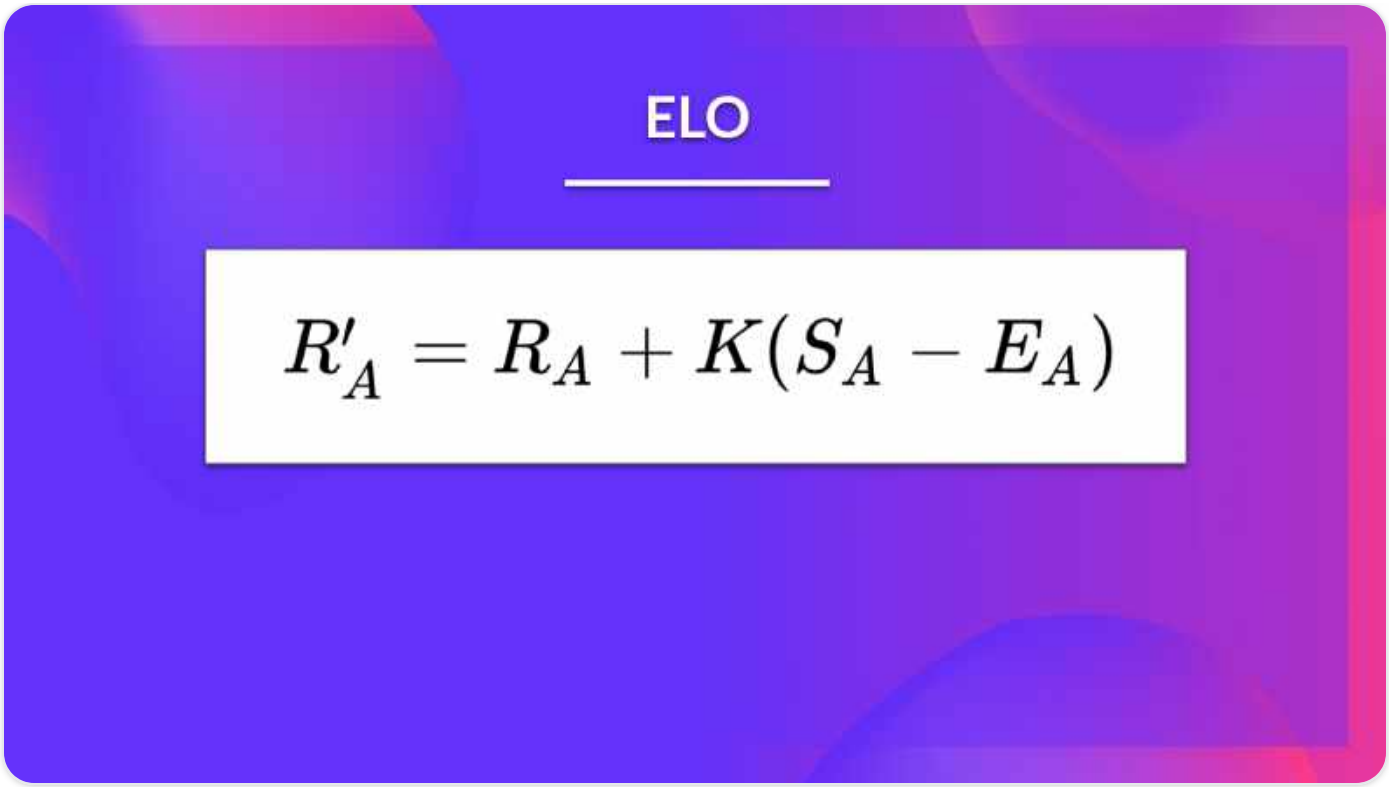


Then, at the end of the game, we need to update the player's actual Elo score. We use a linear adjustment **proportional to the amount by which the player over-performed or under-performed.**

We also define a maximum adjustment rating per game: K-factor.

- K=16 for master.
- K=32 for weaker players.

If Player A has Ea points but scored Sa points, then the player's rating is updated using the formula:



Example

If we take an example:

Player A has a rating of 2600

Player B has a rating of 2300

- We first calculate the expected score: $E_A = \frac{1}{1+10^{(2300-2600)/400}} = 0.849$ $E_B = \frac{1}{1+10^{(2600-2300)/400}} = 0.151$
- If the organizers determined that K=16 and A wins, the new rating would be: $ELO_A = 2600 + 16 * (1 - 0.849) = 2602$ $ELO_B = 2300 + 16 * (0 - 0.151) = 2298$
- If the organizers determined that K=16 and B wins, the new rating would be: $ELO_A = 2600 + 16 * (0 - 0.849) = 2586$ $ELO_B = 2300 + 16 * (1 - 0.151) = 2314$

The Advantages

Using the ELO score has multiple advantages:

- Points are **always balanced** (more points are exchanged when there is an unexpected outcome, but the sum is always the same).
- It is a **self-corrected system** since if a player wins against a weak player, they will only win a few points.
- It **works with team games**: we calculate the average for each team and use it in Elo.

The Disadvantages

- ELO **does not take into account the individual contribution** of each people in the team.
- Rating deflation: **a good rating requires skill over time to keep the same rating**.
- **Can’t compare rating in history**.

<> Update on GitHub

← Designing Multi-Agents systems

Let's train our soccer team to beat your classmates' teams (AI vs. AI) →

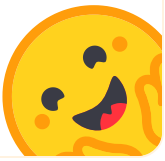
Search models, datasets, users...

Deep RL Course documentation
Student Works

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Student Works

Since the launch of the Deep Reinforcement Learning Course, **many students have created amazing projects that you should check out and consider participating in**.

If you've created an interesting project, don't hesitate to add it to this list by opening a pull request on the GitHub repository.

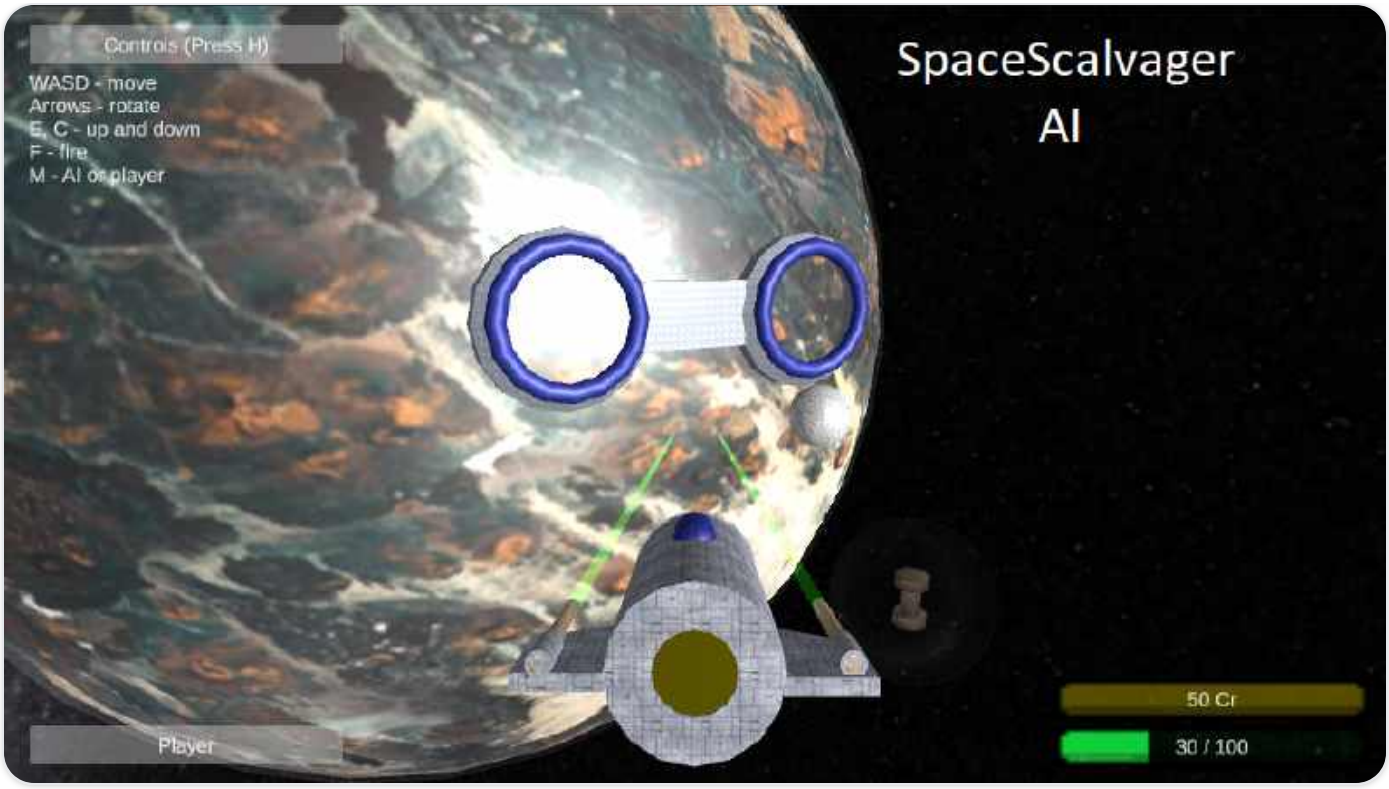
The projects are **arranged based on the date of publication in this page**.

Space Scavanger AI

This project is a space game environment with trained neural network for AI.

AI is trained by Reinforcement learning algorithm based on UnityMLAgents and RLlib frameworks.

Ask HuggingChat



Play the Game here 👉 https://swingshuffle.itch.io/spacescalvagerai

Check the Unity project here 👉 https://github.com/HighExecutor/SpaceScalvagerAI

Neural Nitro



In this project, Sookeyy created a low poly racing game

Check out the demo here 👉 https://sookeyy.itch.io/neuralnitro

Space War 🚀



In this project, Eric Dong recreates Bill Seiler's 1985 version of Space War in Pygame and uses reinforcement learning (RL) to train AI agents.

This project is currently in development!

Demo

Dev/Edge version:

- https://e-dong.itch.io/spacewar-dev

Stable version:

- https://e-dong.itch.io/spacewar
- https://huggingface.co/spaces/EricofRL/SpaceWarRL

Community blog posts

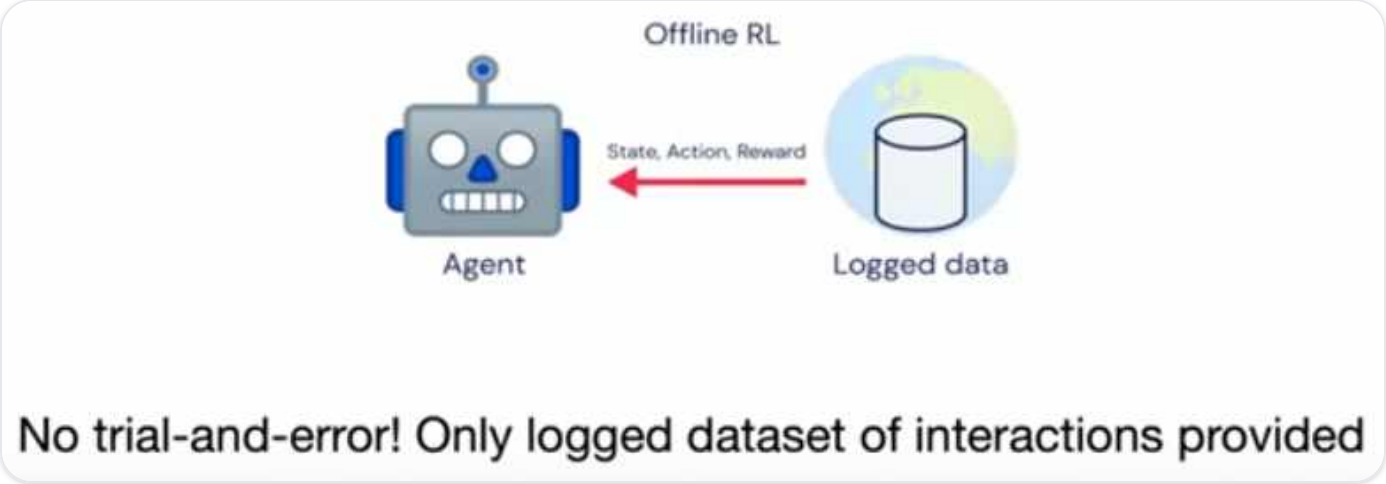
TBA

Other links

Check out the source here 👉 https://github.com/e-dong/space-war-rl
Check out his blog here 👉 https://dev.to/edong/space-war-rl-0-series-introduction-25dh

Decision Transformers for Trading

In this project, student has explored training a Decision Transformer for stock trading. In phase-1, offline training has been implemented. He intends to incorporate online fine-tuning in the next version.



Source: Stanford CS25: V1 I Decision Transformer: Reinforcement Learning via Sequence Modeling

Check out the source here 👉 https://github.com/ra9hur/Decision-Transformers-For-Trading

<> Update on GitHub

← An Introduction to Godot RL

Brief introduction to RL documentation →

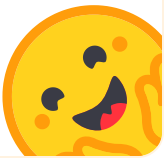
Search models, datasets, users...

Deep RL Course documentation
Summary

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Summary

That was a lot of information! Let's summarize:

- Reinforcement Learning is a computational approach of learning from actions. We build an agent that learns from the environment **by interacting with it through trial and error** and receiving rewards (negative or positive) as feedback.
- The goal of any RL agent is to maximize its expected cumulative reward (also called expected return) because RL is based on the **reward hypothesis**, which is that **all goals can be described as the maximization of the expected cumulative reward.**
- The RL process is a loop that outputs a sequence of **state, action, reward and next state.**
- To calculate the expected cumulative reward (expected return), we discount the rewards: the rewards that come sooner (at the beginning of the game) **are more probable to happen since they are more predictable than the long term future reward.**
- To solve an RL problem, you want to **find an optimal policy**. The policy is the "brain" of your agent, which will tell us **what action to take given a state.** The optimal policy is the one which **gives you the actions that maximize the expected return.**
- There are two ways to find your optimal policy:

Ask HuggingChat

1. By training your policy directly: **policy-based methods.**
2. By training a value function that tells us the expected return the agent will get at each state and use this function to define our policy: **value-based methods.**

- Finally, we speak about Deep RL because we introduce **deep neural networks to estimate the action to take (policy-based) or to estimate the value of a state (value-based)** hence the name “deep”.

<> Update on GitHub

← The “Deep” in Deep Reinforcement Learning Glossary →

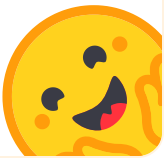
Search models, datasets, users...

Deep RL Course documentation
The “Deep” in Reinforcement Learning

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



The “Deep” in Reinforcement Learning

> What we've talked about so far is Reinforcement Learning. But where does the "Deep" come into play?

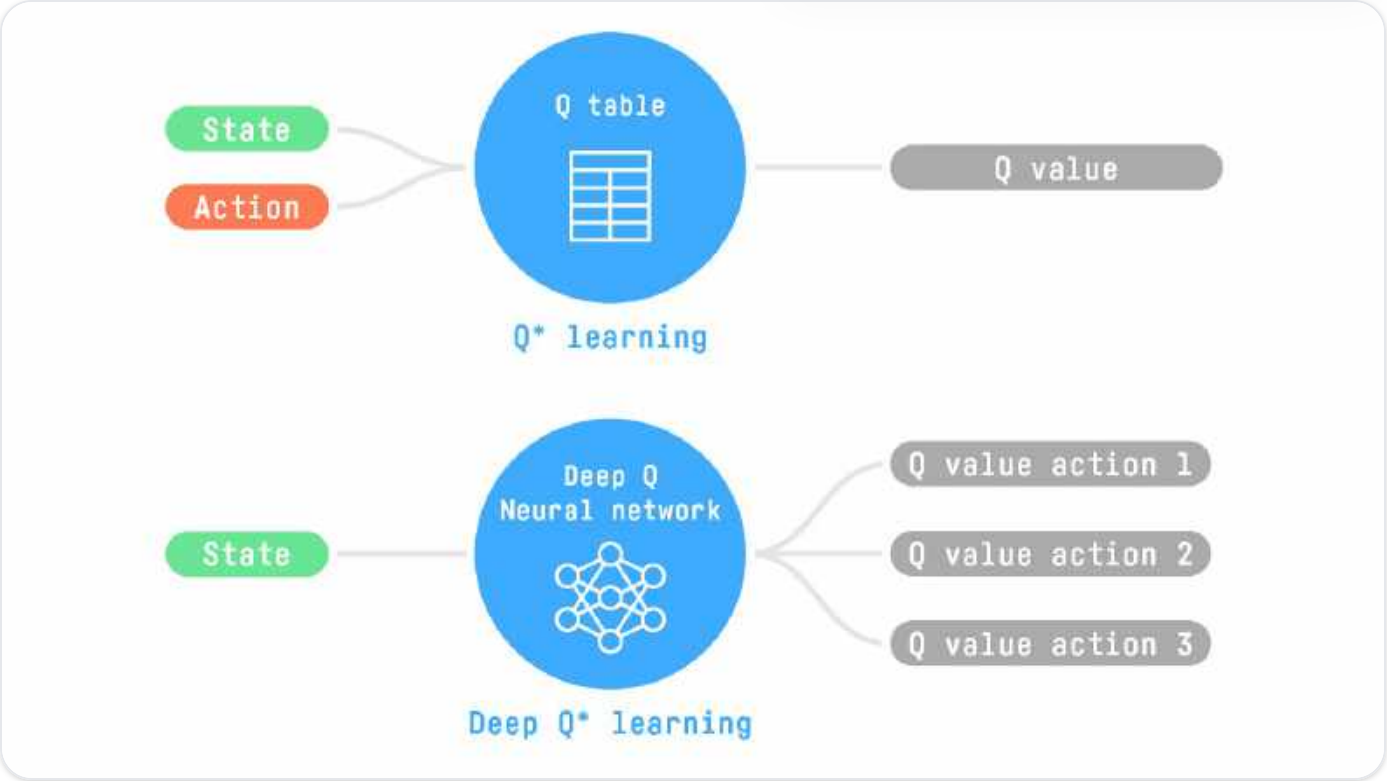
Deep Reinforcement Learning introduces **deep neural networks to solve Reinforcement Learning problems** — hence the name “deep”.

For instance, in the next unit, we’ll learn about two value-based algorithms: Q-Learning (classic Reinforcement Learning) and then Deep Q-Learning.

You’ll see the difference is that, in the first approach, **we use a traditional algorithm** to create a Q table that helps us find what action to take for each state.

In the second approach, **we will use a Neural Network** (to approximate the Q value).

Ask HuggingChat



Schema inspired by the Q learning notebook by Udacity

If you are not familiar with Deep Learning you should definitely watch the FastAI Practical Deep Learning for Coders (Free).

<> Update on GitHub

← The two main approaches for solving RL problems

Summary →

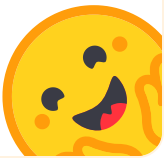
Search models, datasets, users...

Deep RL Course documentation
The advantages and disadvantages of policy-gradient methods

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



The advantages and disadvantages of policy-gradient methods

At this point, you might ask, “but Deep Q-Learning is excellent! Why use policy-gradient methods?“. To answer this question, let’s study the **advantages and disadvantages of policy-gradient methods**.

Advantages

There are multiple advantages over value-based methods. Let’s see some of them:

The simplicity of integration

We can estimate the policy directly without storing additional data (action values).

Policy-gradient methods can learn a stochastic policy

Policy-gradient methods can **learn a stochastic policy while value functions can’t**.

This has two consequences:

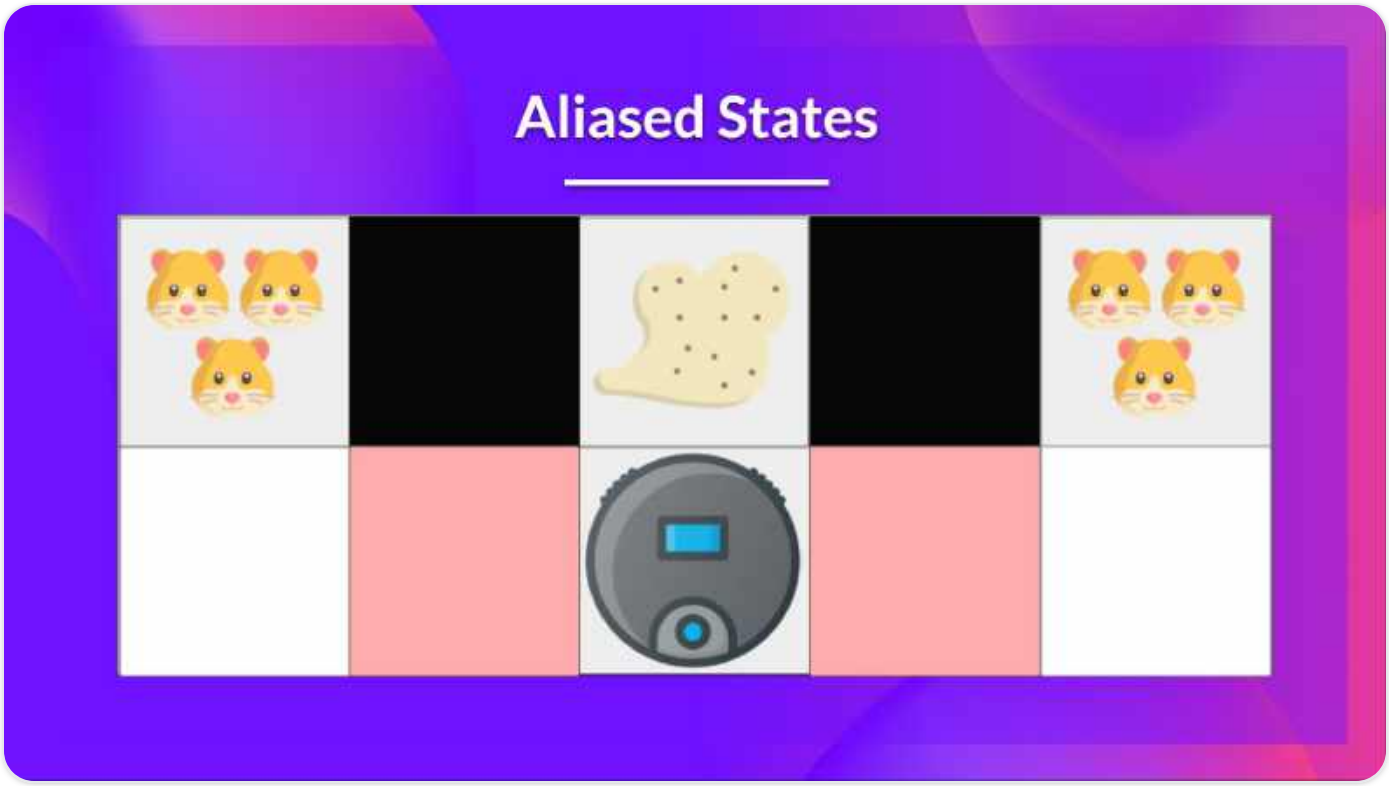
1. We **don’t need to implement an exploration/exploitation trade-off by hand**. Since we output a probability distribution over actions, the agent e

Ask HuggingChat

taking the same trajectory.

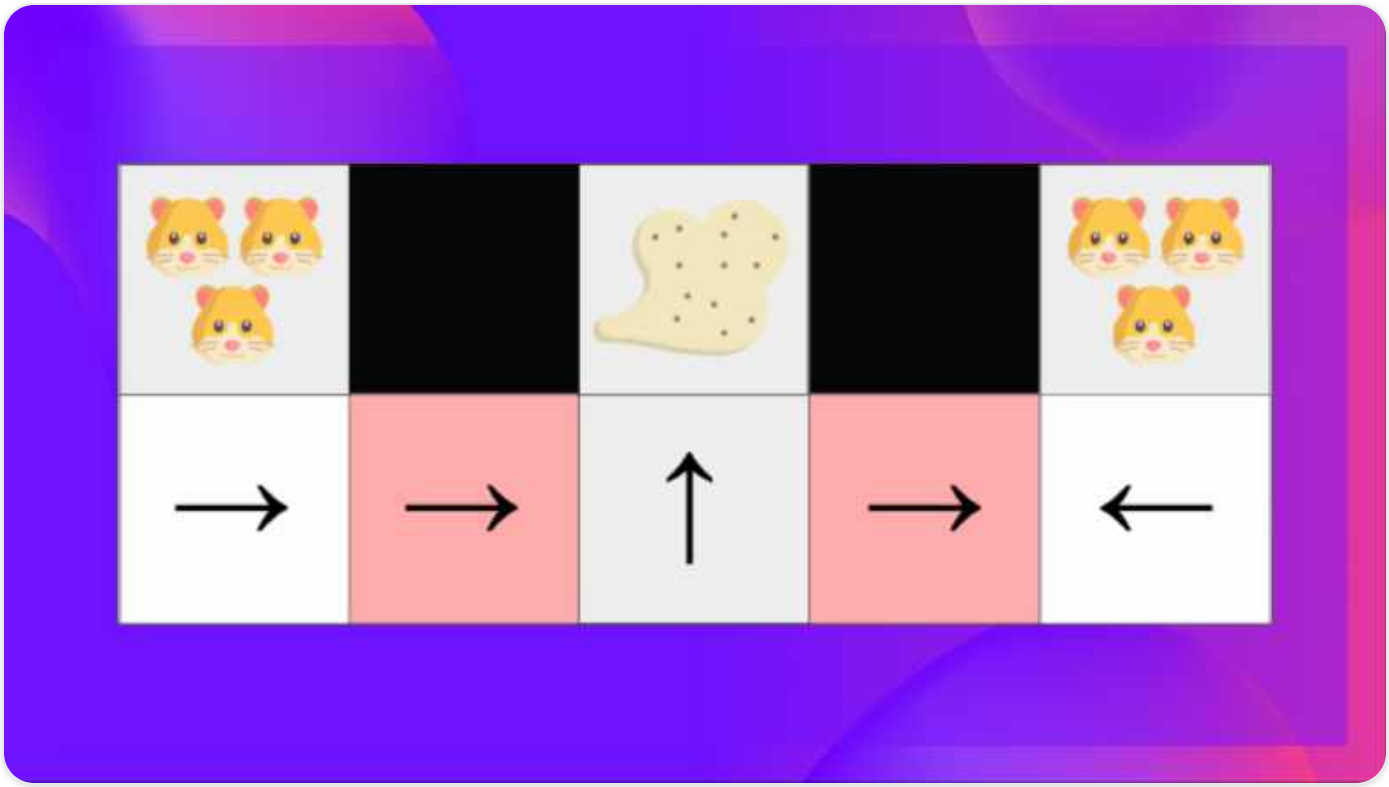
2. We also get rid of the problem of **perceptual aliasing**. Perceptual aliasing is when two states seem (or are) the same but need different actions.

Let's take an example: we have an intelligent vacuum cleaner whose goal is to suck the dust and avoid killing the hamsters.



Our vacuum cleaner can only perceive where the walls are.

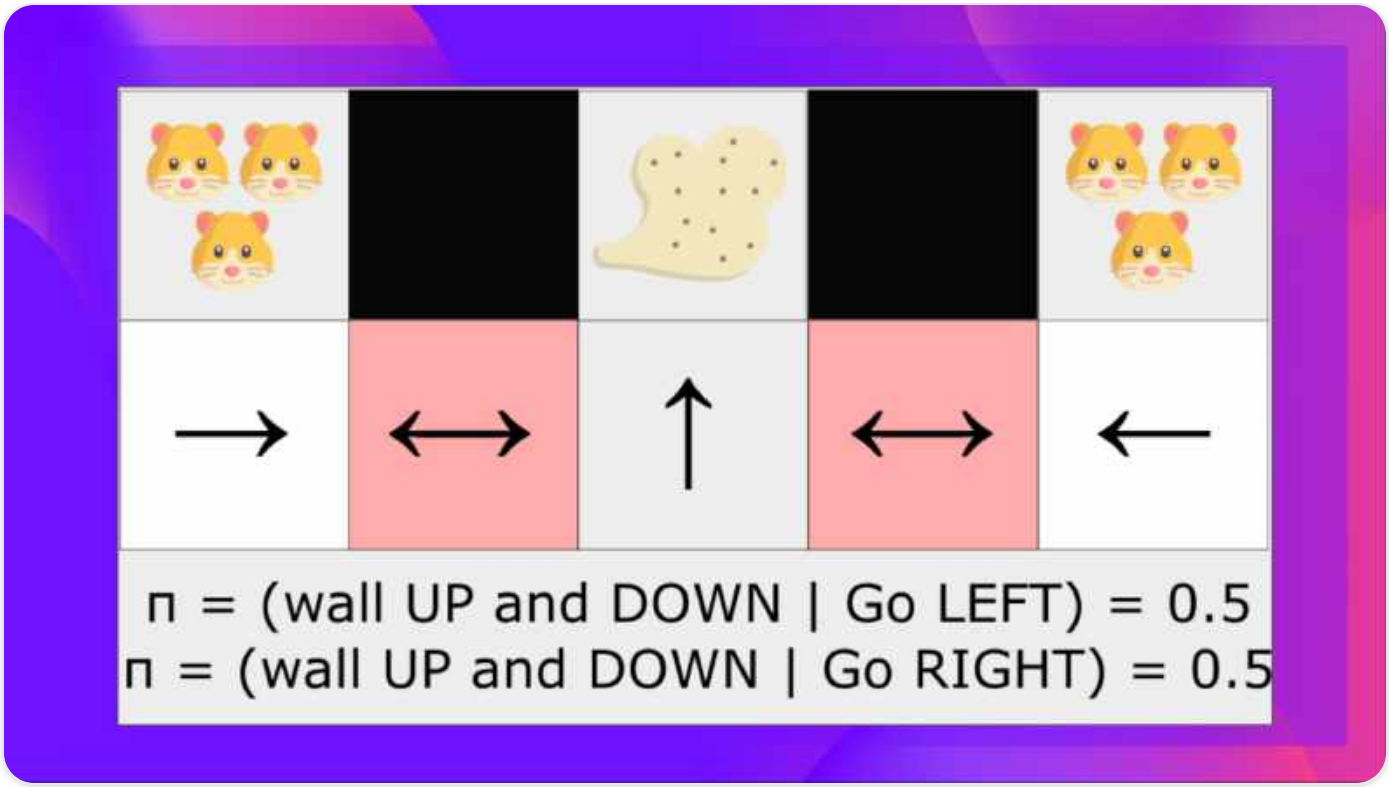
The problem is that the **two red (colored) states are aliased states because the agent perceives an upper and lower wall for each**.



Under a deterministic policy, the policy will either always move right when in a red state or always move left. **Either case will cause our agent to get stuck and never suck the dust**.

Under a value-based Reinforcement learning algorithm, we learn a **quasi-deterministic policy** ("greedy epsilon strategy"). Consequently, our agent can **spend a lot of time before finding the dust**.

On the other hand, an optimal stochastic policy **will randomly move left or right in red (colored) states**. Consequently, **it will not be stuck and will reach the goal state with a high probability**.



Policy-gradient methods are more effective in high-dimensional action spaces and continuous actions spaces

The problem with Deep Q-learning is that their **predictions assign a score (maximum expected future reward) for each possible action**, at each time step, given the current state.

But what if we have an infinite possibility of actions?

For instance, with a self-driving car, at each state, you can have a (near) infinite choice of actions (turning the wheel at 15°, 17.2°, 19,4°, honking, etc.). **We'll need to output a Q-value for each possible action**! And **taking the max action of a continuous output is an optimization problem itself**!

Instead, with policy-gradient methods, we output a **probability distribution over actions.**

Policy-gradient methods have better convergence properties

In value-based methods, we use an aggressive operator to **change the value function: we take the maximum over Q-estimates**. Consequently, the action

an arbitrarily small change in the estimated action values if that change results in a different action having the maximal value.

For instance, if during the training, the best action was left (with a Q-value of 0.22) and the training step after it's right (since the right Q-value becomes 0.23), we dramatically changed the policy since now the policy will take most of the time right instead of left.

On the other hand, in policy-gradient methods, stochastic policy action preferences (probability of taking action) **change smoothly over time**.

Disadvantages

Naturally, policy-gradient methods also have some disadvantages:

- **Frequently, policy-gradient methods converges to a local maximum instead of a global optimum.**
- Policy-gradient goes slower, **step by step: it can take longer to train (inefficient).**
- Policy-gradient can have high variance. We'll see in the actor-critic unit why, and how we can solve this problem.

👉 If you want to go deeper into the advantages and disadvantages of policy-gradient methods, you can check this video.

<> Update on GitHub

← What are the policy-based methods? Diving deeper into policy-gradient →

Search models, datasets, users...

Deep RL Course documentation
The Bellman Equation: simplify our value estimation

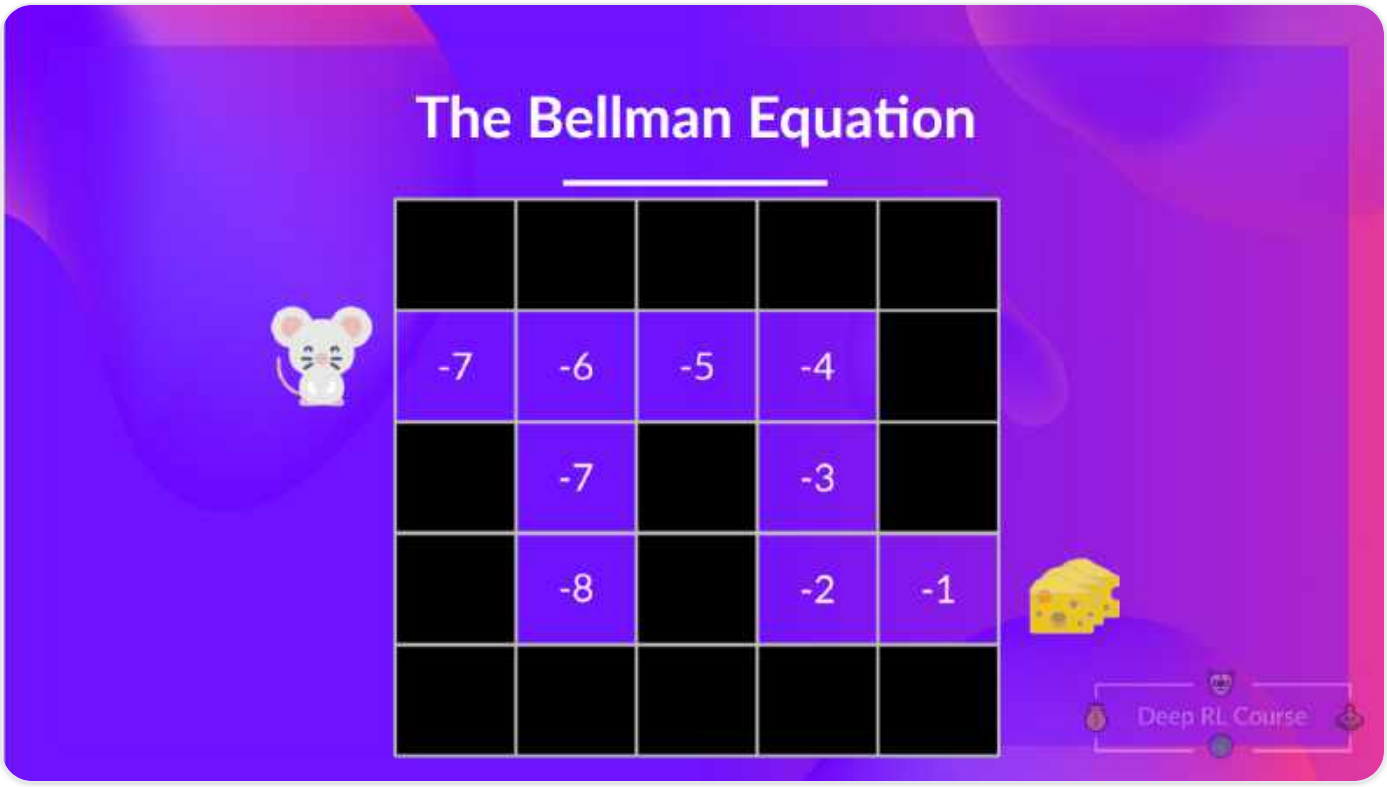
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

The Bellman Equation: simplify our value estimation

The Bellman equation **simplifies our state value or state-action value calculation.**

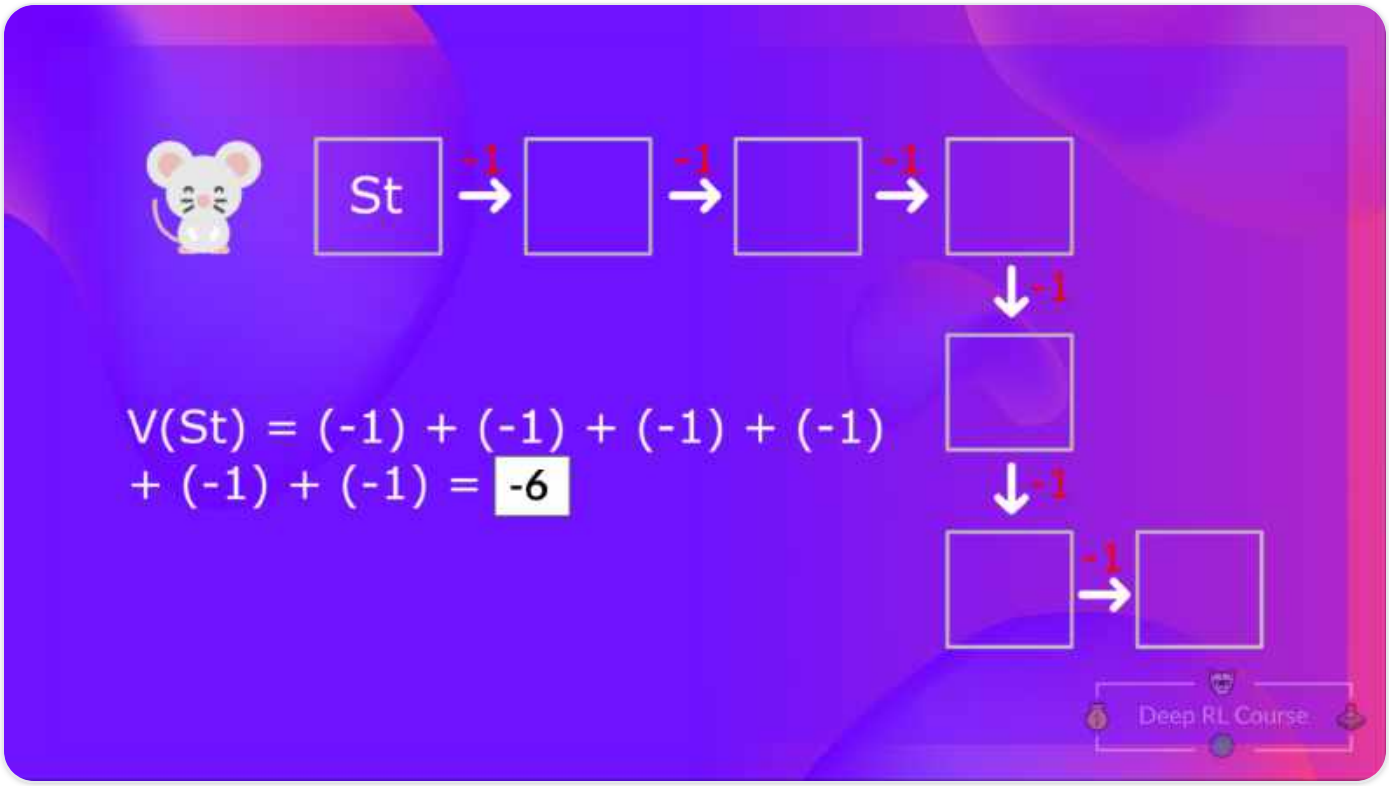


With what we have learned so far, we know that if we calculate $V(S_t)$ (the value of a state), we need to calculate the return starting at that state and th

Ask HuggingChat

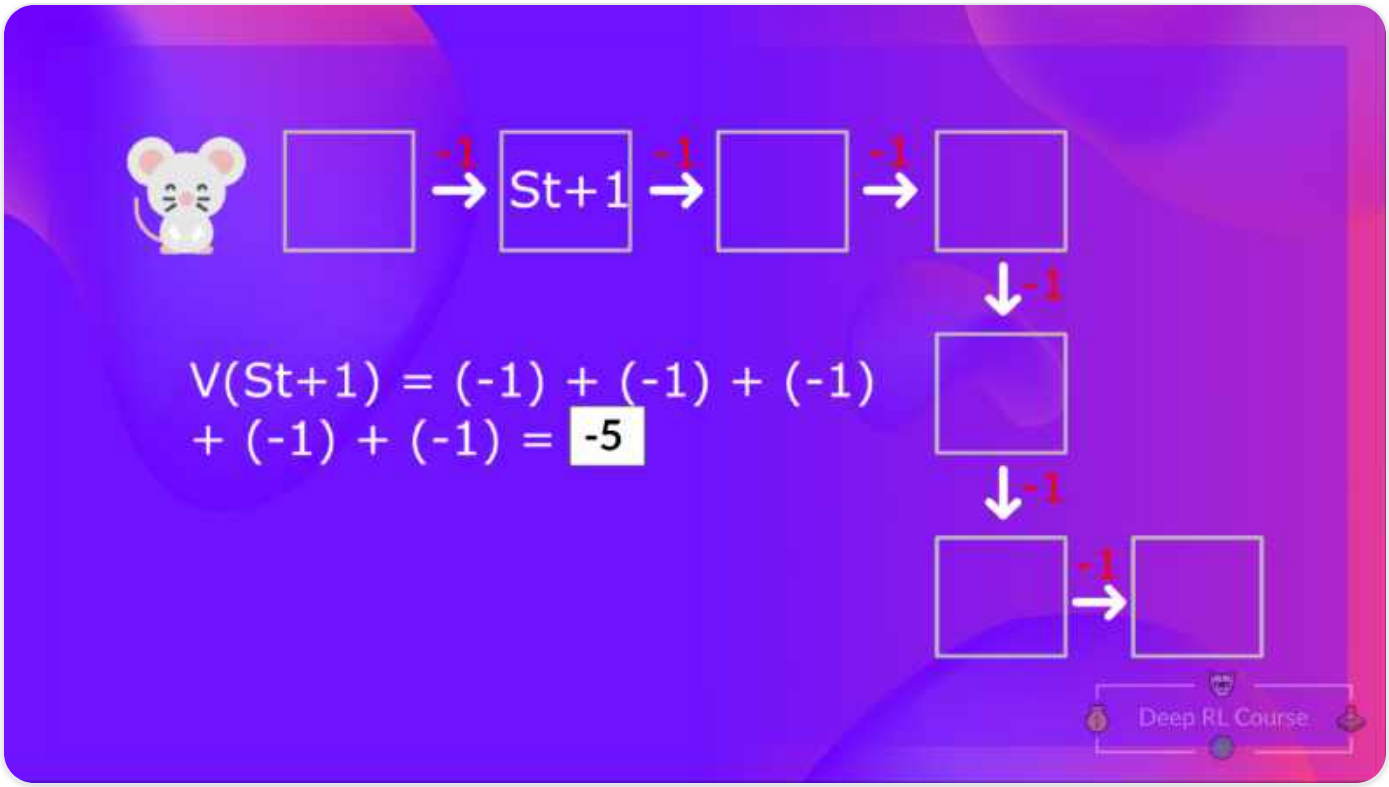
policy we defined in the following example is a Greedy Policy; for simplification, we don't discount the reward).

So to calculate $V(S_t)$, we need to calculate the sum of the expected rewards. Hence:



To calculate the value of State 1: the sum of rewards if the agent started in that state and then followed the greedy policy (taking actions that leads to the best states values) for all the time steps.

Then, to calculate the $V(S_{t+1})$, we need to calculate the return starting at that state S_{t+1}.



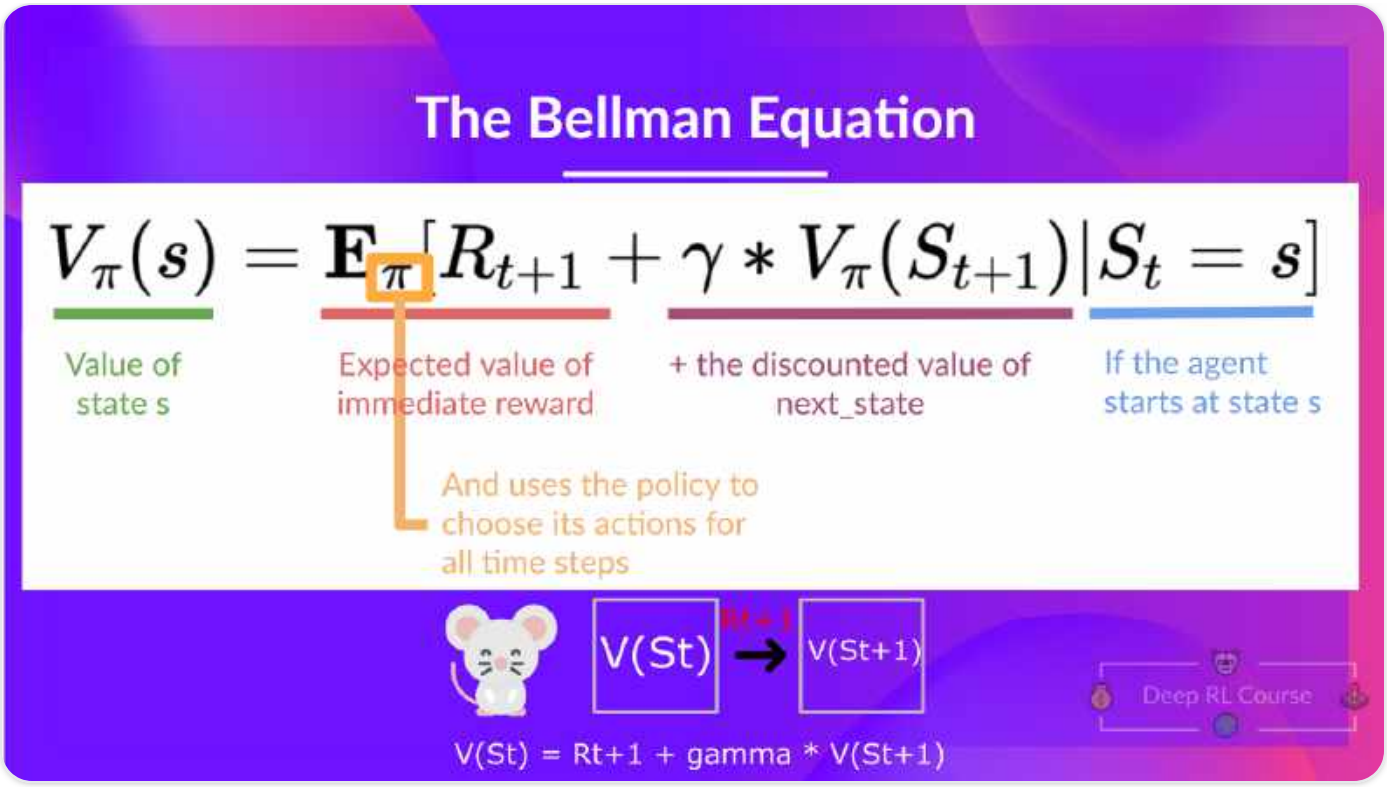
To calculate the value of State 2: the sum of rewards **if the agent started in that state**, and then followed the **policy for all the time steps.**

So you may have noticed, we're repeating the computation of the value of different states, which can be tedious if you need to do it for each state value or state-action value.

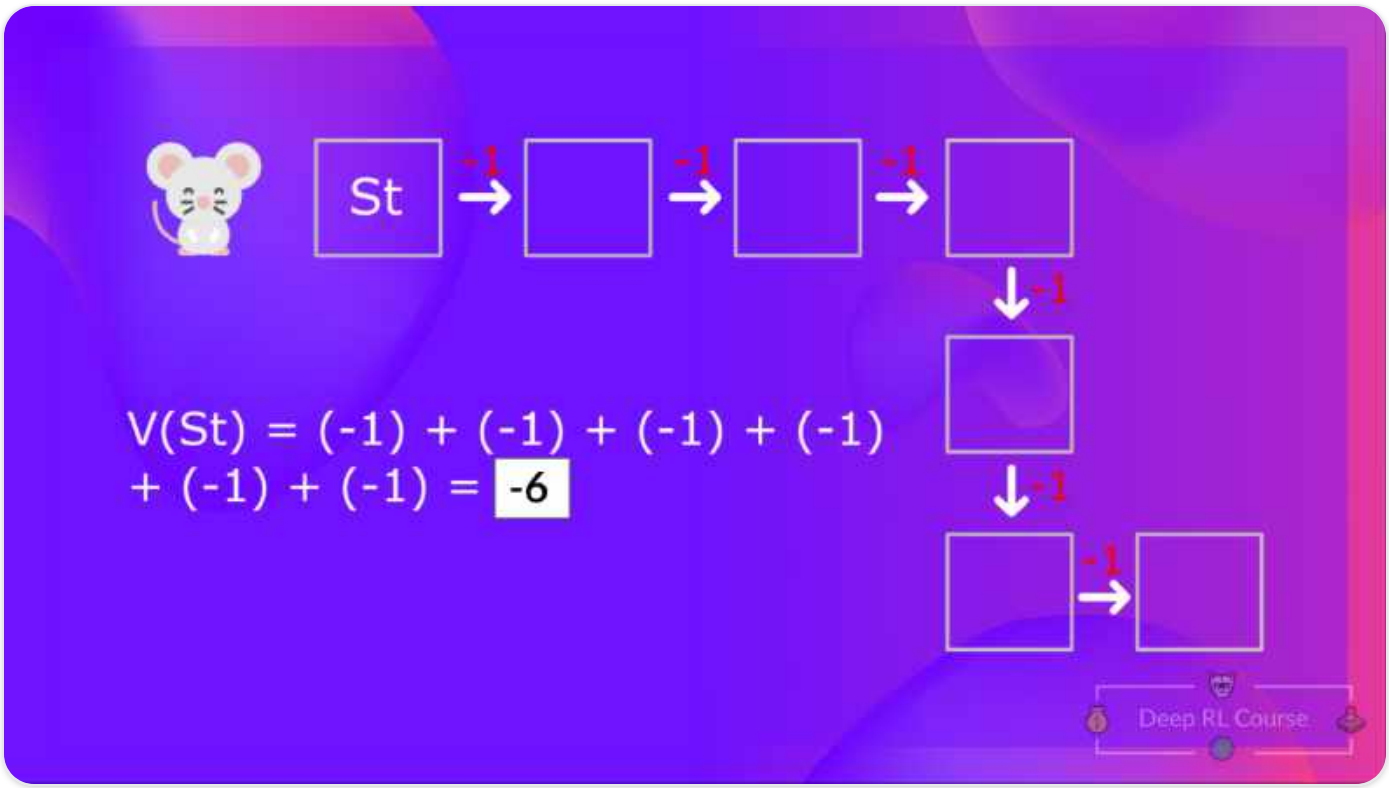
Instead of calculating the expected return for each state or each state-action pair, **we can use the Bellman equation.** (hint: if you know what Dynamic Programming is, this is very similar! if you don't know what it is, no worries!)

The Bellman equation is a recursive equation that works like this: instead of starting for each state from the beginning and calculating the return, we can consider the value of any state as:

The immediate reward R_{t+1} + the discounted value of the state that follows ($\gamma * V(S_{t+1})$) .

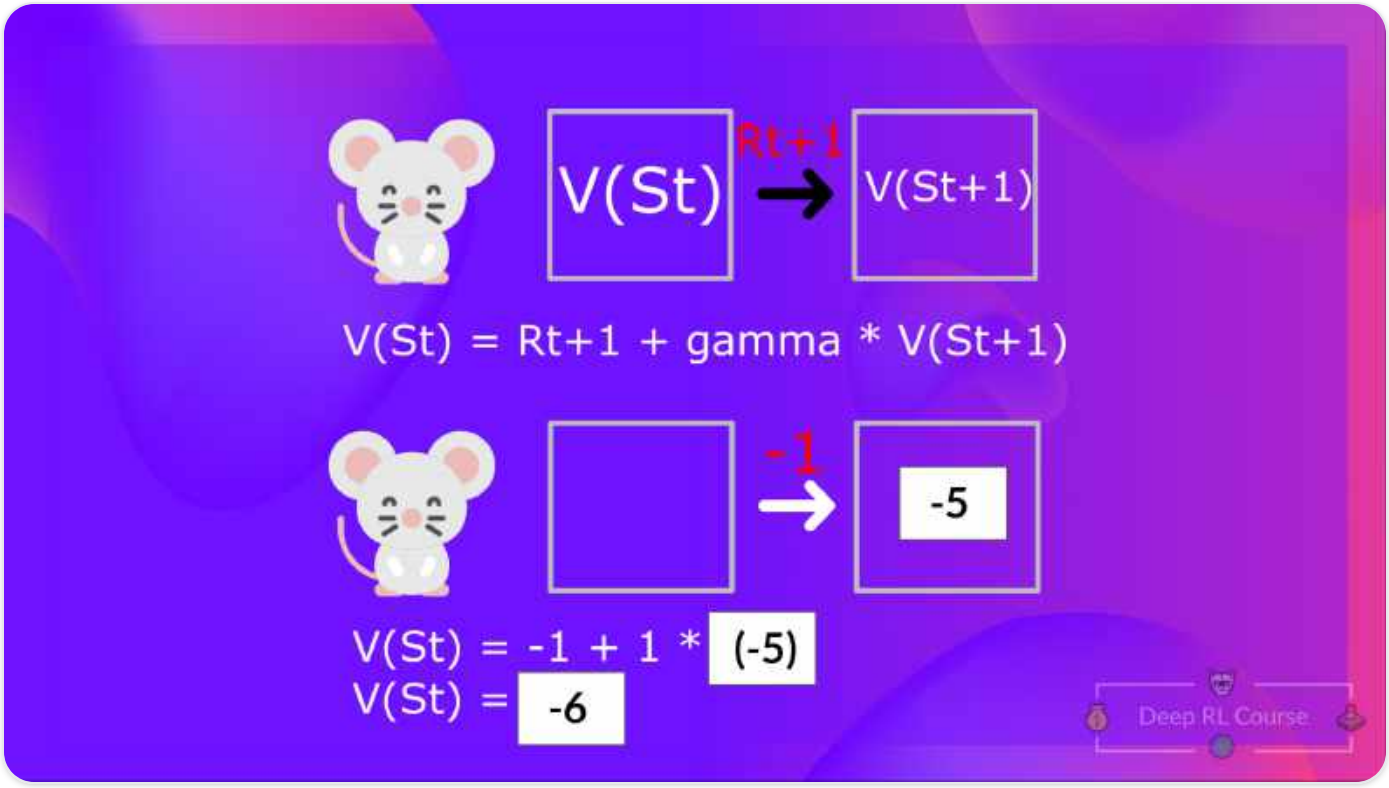


If we go back to our example, we can say that the value of State 1 is equal to the expected cumulative return if we start at that state.



To calculate the value of State 1: the sum of rewards **if the agent started in that state 1** and then followed the **policy for all the time steps.**

This is equivalent to $V(S_t)$ = Immediate reward R_{t+1} + Discounted value of the next state $\gamma * V(S_{t+1})$



For simplification, here we don't discount so gamma = 1.

In the interest of simplicity, here we don't discount, so gamma = 1. But you'll study an example with gamma = 0.99 in the Q-Learning section of this unit.

- The value of $V(S_{t+1})$ = Immediate reward R_{t+2} + Discounted value of the next state ($gamma * V(S_{t+2})$).
- And so on.

To recap, the idea of the Bellman equation is that instead of calculating each value as the sum of the expected return, **which is a long process**, we calculate the value as **the sum of immediate reward + the discounted value of the state that follows.**

Before going to the next section, think about the role of gamma in the Bellman equation. What happens if the value of gamma is very low (e.g. 0.1 or even 0)? What happens if the value is 1? What happens if the value is very high, such as a million?

<> Update on GitHub

← The two types of value-based methods

Monte Carlo vs Temporal Difference Learning →

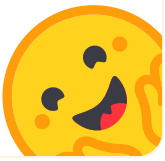
Search models, datasets, users...

Deep RL Course documentation
The Deep Q-Learning Algorithm

Join the Hugging Face community

and get access to the augmented documentation experience

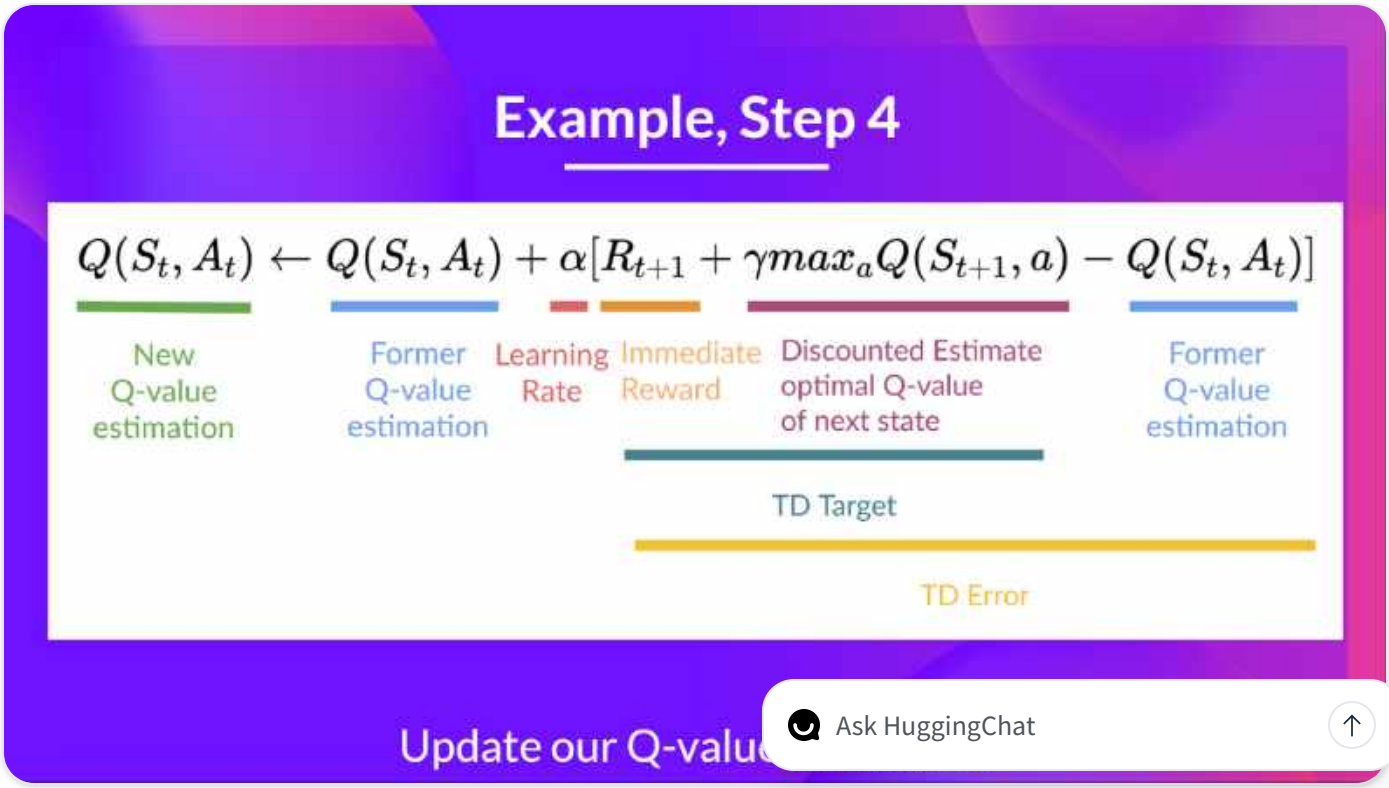
Sign Up to get started



The Deep Q-Learning Algorithm

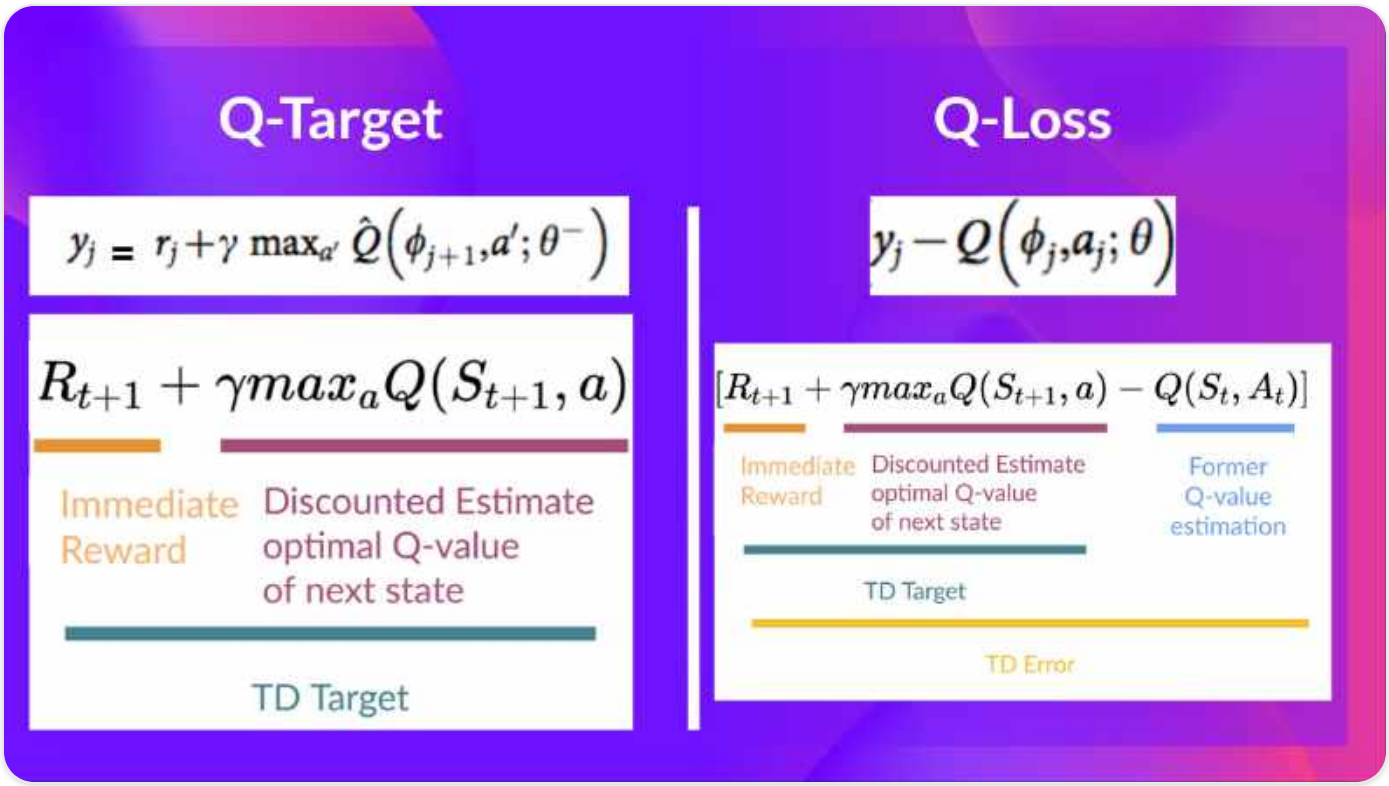
We learned that Deep Q-Learning **uses a deep neural network to approximate the different Q-values for each possible action at a state** (value-function estimation).

The difference is that, during the training phase, instead of updating the Q-value of a state-action pair directly as we have done with Q-Learning:



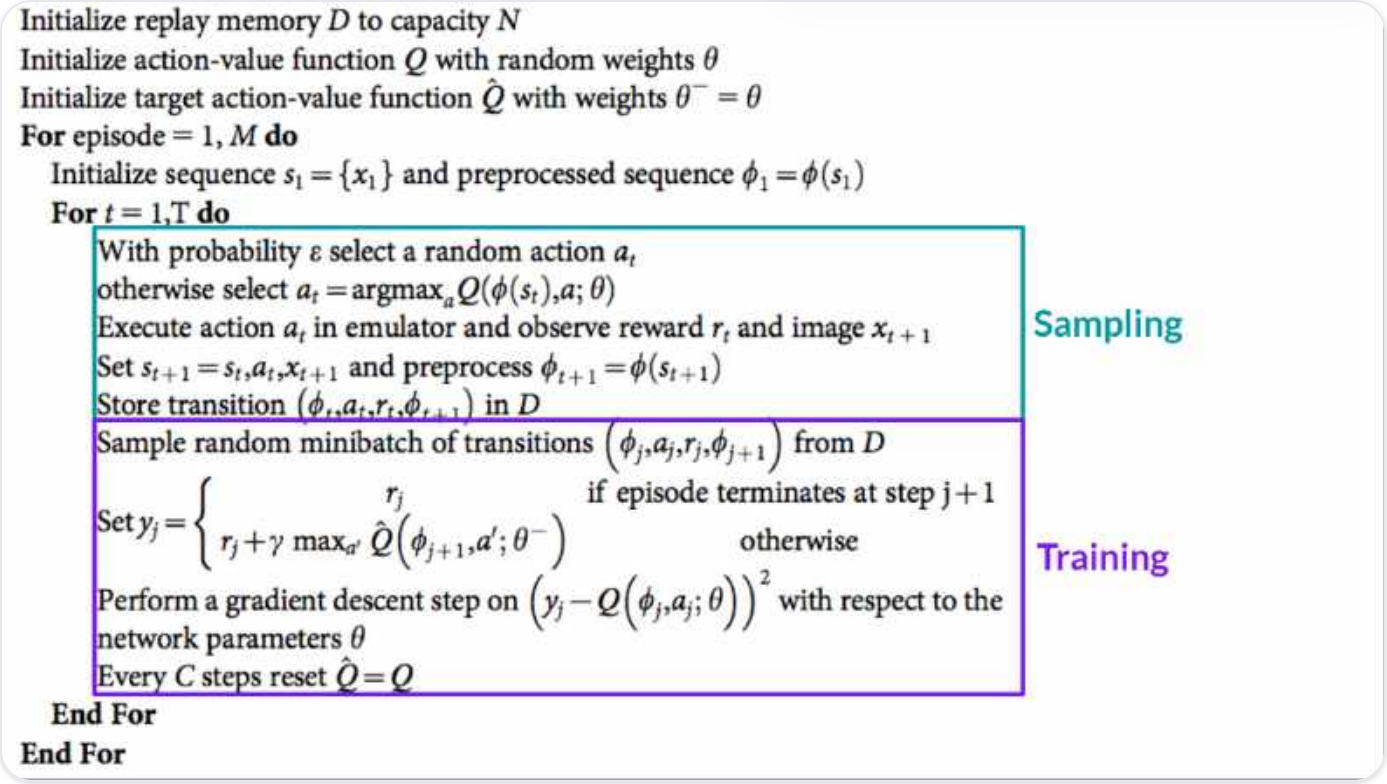
Ask HuggingChat

in Deep Q-Learning, we create a **loss function that compares our Q-value prediction and the Q-target and uses gradient descent to update the weights of our Deep Q-Network to approximate our Q-values better**.



The Deep Q-Learning training algorithm has *two phases*:

- **Sampling**: we perform actions and **store the observed experience tuples in a replay memory**.
- **Training**: Select a **small batch of tuples randomly and learn from this batch using a gradient descent update step**.



This is not the only difference compared with Q-Learning. Deep Q-Learning training **might suffer from instability**, mainly because of combining a non-linear Q-value function (Neural Network) and bootstrapping (when we update targets with existing estimates and not an actual complete return).

To help us stabilize the training, we implement three different solutions:

1. *Experience Replay* to make more **efficient use of experiences**.
2. *Fixed Q-Target* **to stabilize the training**.
3. *Double Deep Q-Learning*, to **handle the problem of the overestimation of Q-values**.

Let's go through them!

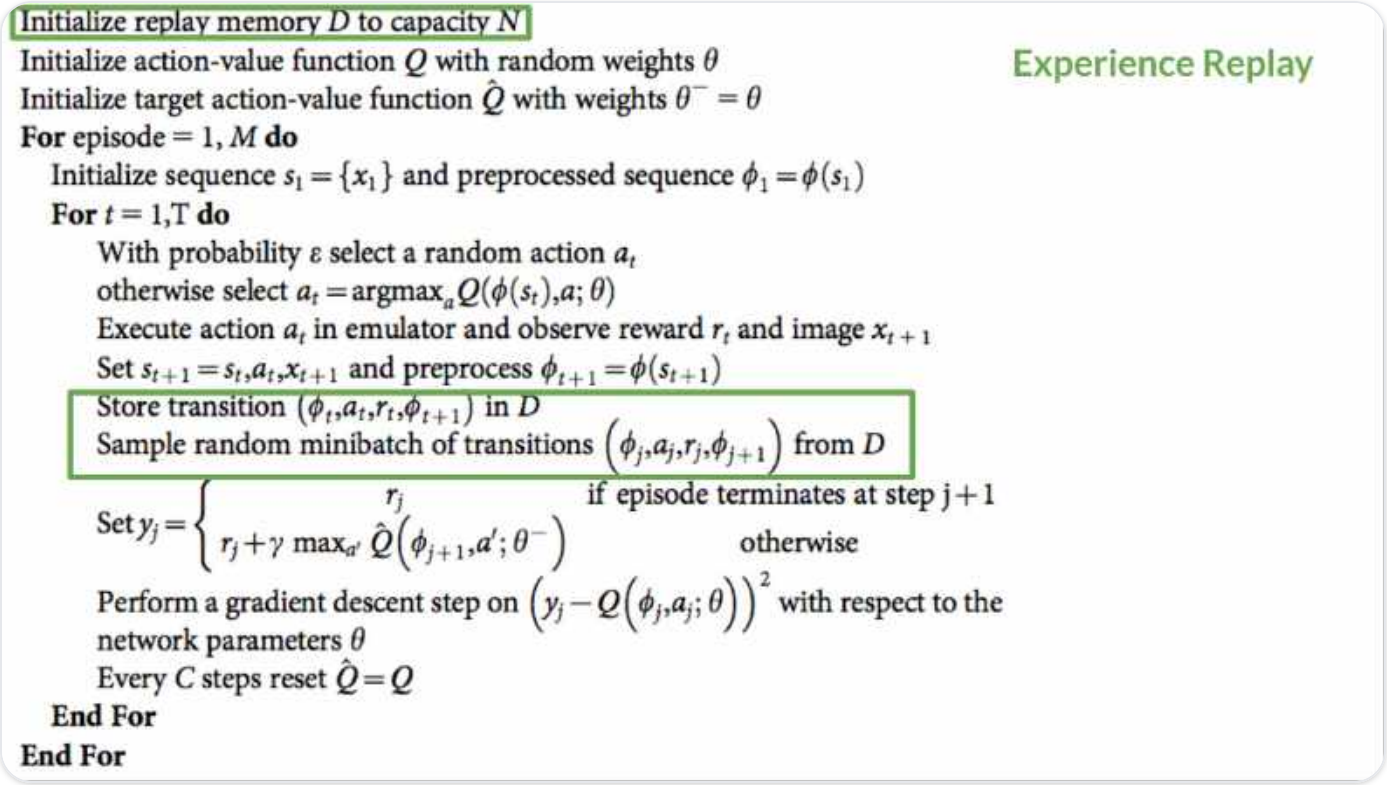
Experience Replay to make more efficient use of experiences

Why do we create a replay memory?

Experience Replay in Deep Q-Learning has two functions:

1. **Make more efficient use of the experiences during the training**. Usually, in online reinforcement learning, the agent interacts with the environment, gets experiences (state, action, reward, and next state), learns from them (updates the neural network), and discards them. This is not efficient.

Experience replay helps by **using the experiences of the training more efficiently**. We use a replay buffer that saves experience samples **that we can reuse during the training.**



⇒ This allows the agent to **learn from the same experiences multiple times**.

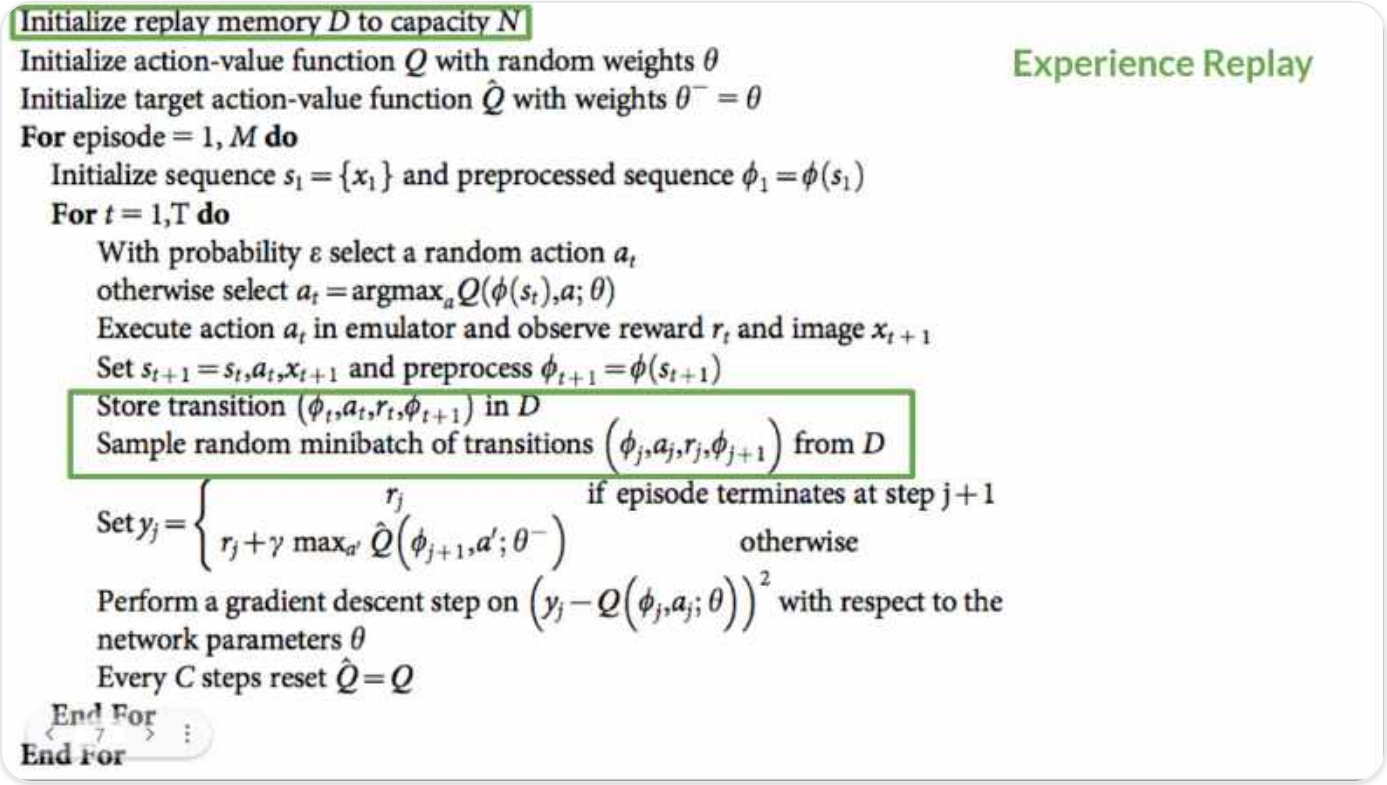
2. **Avoid forgetting previous experiences (aka catastrophic interference, or catastrophic forgetting) and reduce the correlation between experiences**.

- catastrophic forgetting: The problem we get if we give sequential samples of experiences to our neural network is that it tends to forget **the previous experiences as it gets new experiences.** For instance, if the agent is in the first level and then in the second, which is different, it can forget how to behave and play in the first level.

The solution is to create a Replay Buffer that stores experience tuples while interacting with the environment and then sample a small batch of tuples. This prevents **the network from only learning about what it has done immediately before.**

Experience replay also has other benefits. By randomly sampling the experiences, we remove correlation in the observation sequences and avoid **action values from oscillating or diverging catastrophically.**

In the Deep Q-Learning pseudocode, we **initialize a replay memory buffer D with capacity N** (N is a hyperparameter that you can define). We then store experiences in the memory and sample a batch of experiences to feed the Deep Q-Network during the training phase.

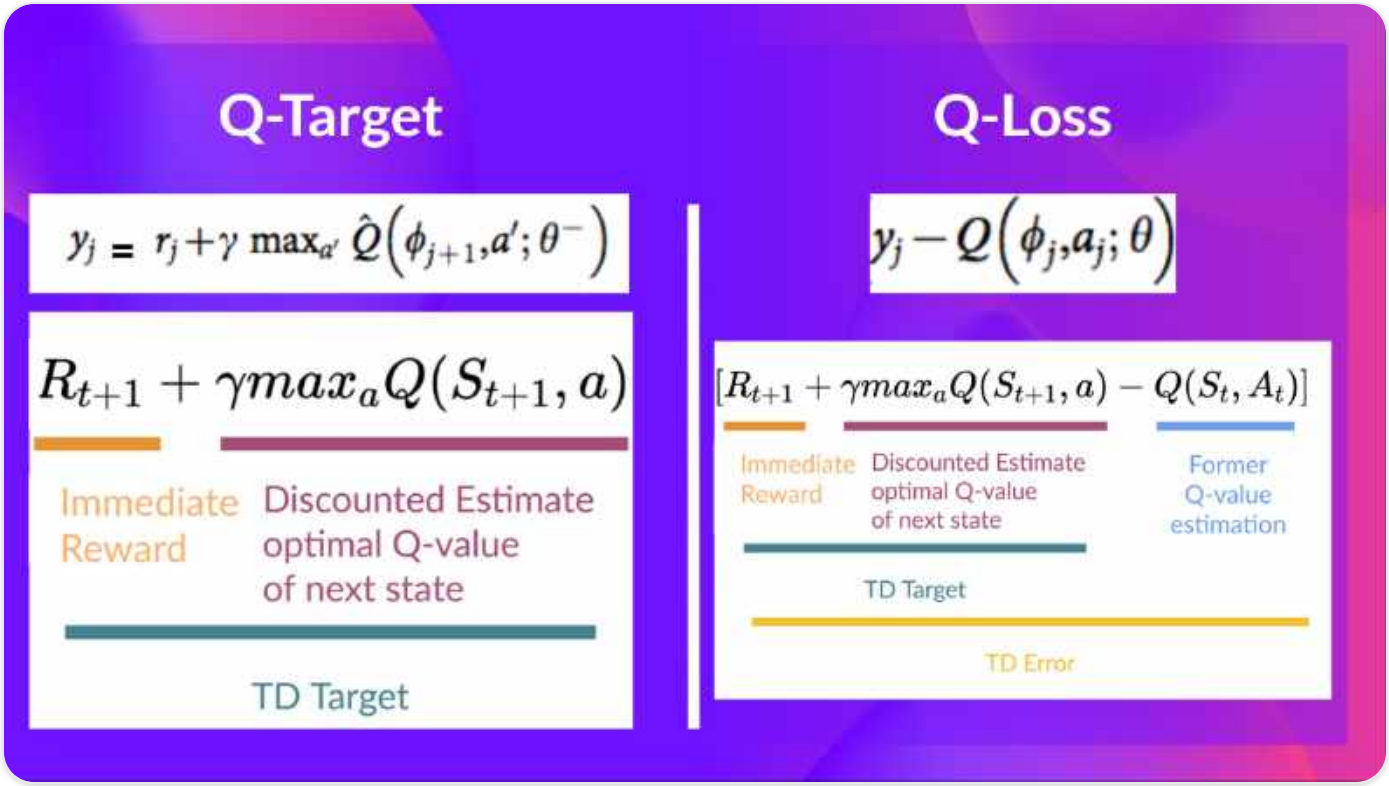


Fixed Q-Target to stabilize the training

When we want to calculate the TD error (aka the loss), we calculate the **difference between the TD target (Q-Target) and the current Q-value (estimation of Q)**.

But we **don't have any idea of the real TD target**. We need to estimate it. Using the Bellman equation, we saw that the TD target is just the reward c

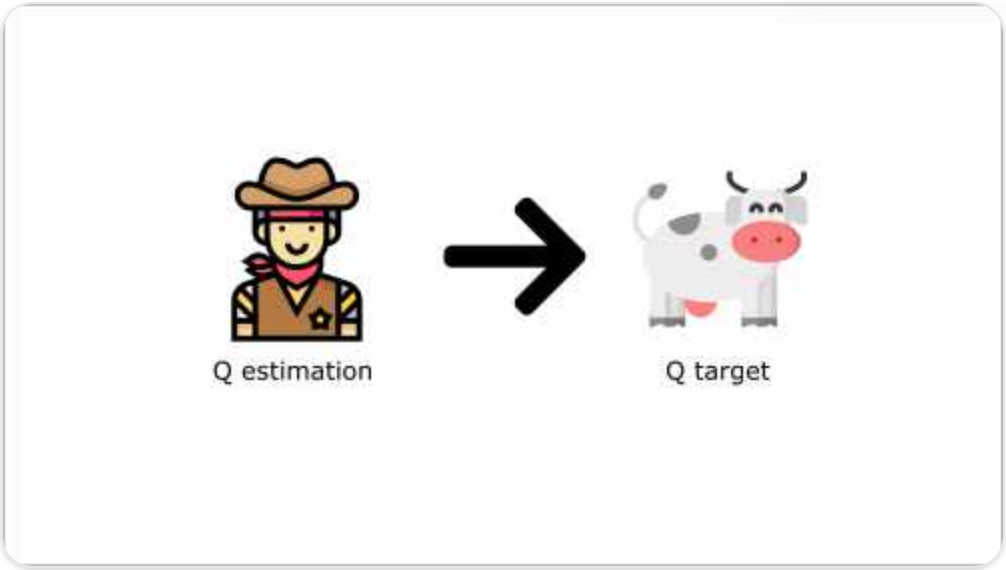
discounted highest Q value for the next state.



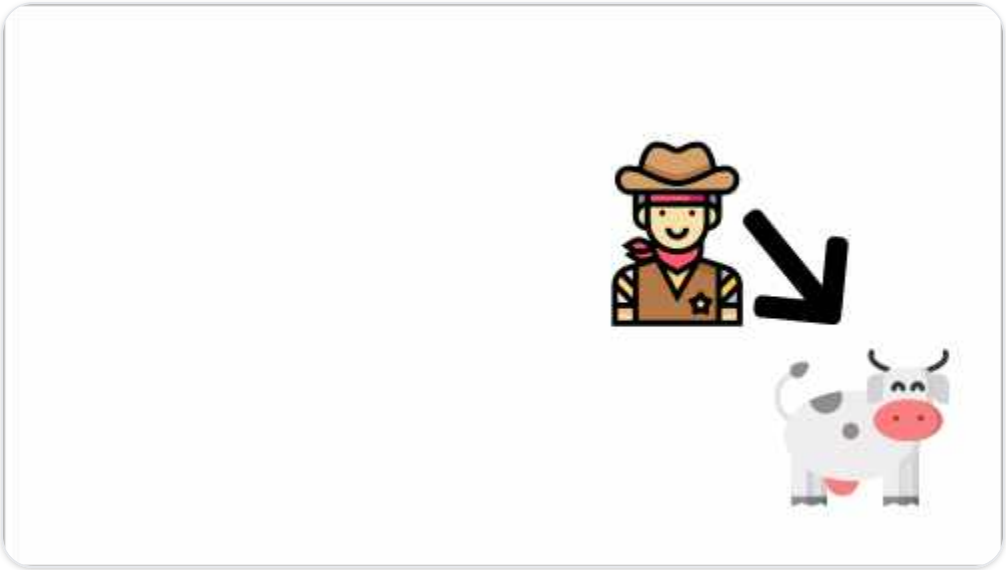
However, the problem is that we are using the same parameters (weights) for estimating the TD target **and** the Q-value. Consequently, there is a significant correlation between the TD target and the parameters we are changing.

Therefore, at every step of training, **both our Q-values and the target values shift.** We're getting closer to our target, but the target is also moving. It's like chasing a moving target! This can lead to significant oscillation in training.

It's like if you were a cowboy (the Q estimation) and you wanted to catch a cow (the Q-target). Your goal is to get closer (reduce the error).

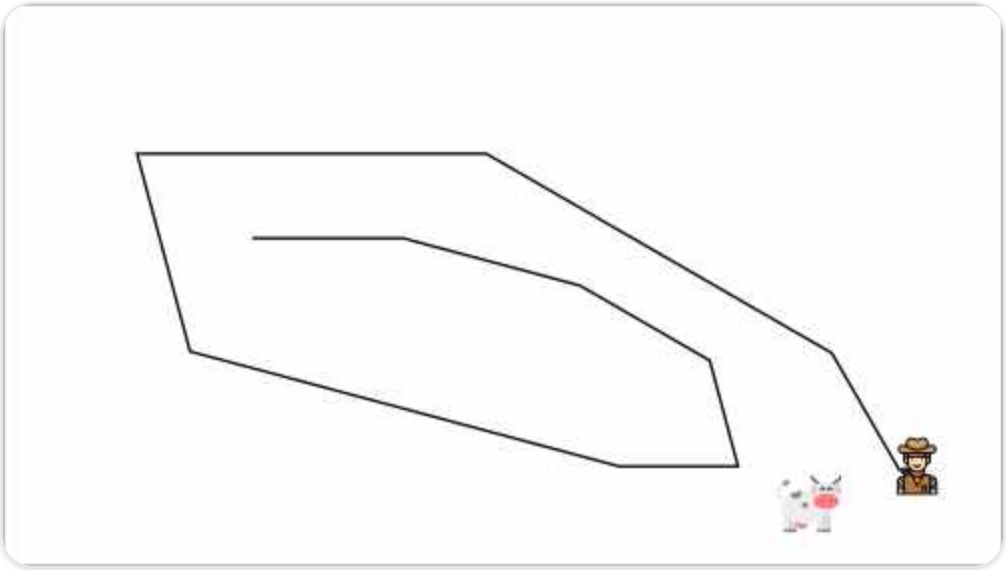


At each time step, you're trying to approach the cow, which also moves at each time step (because you use the same parameters).



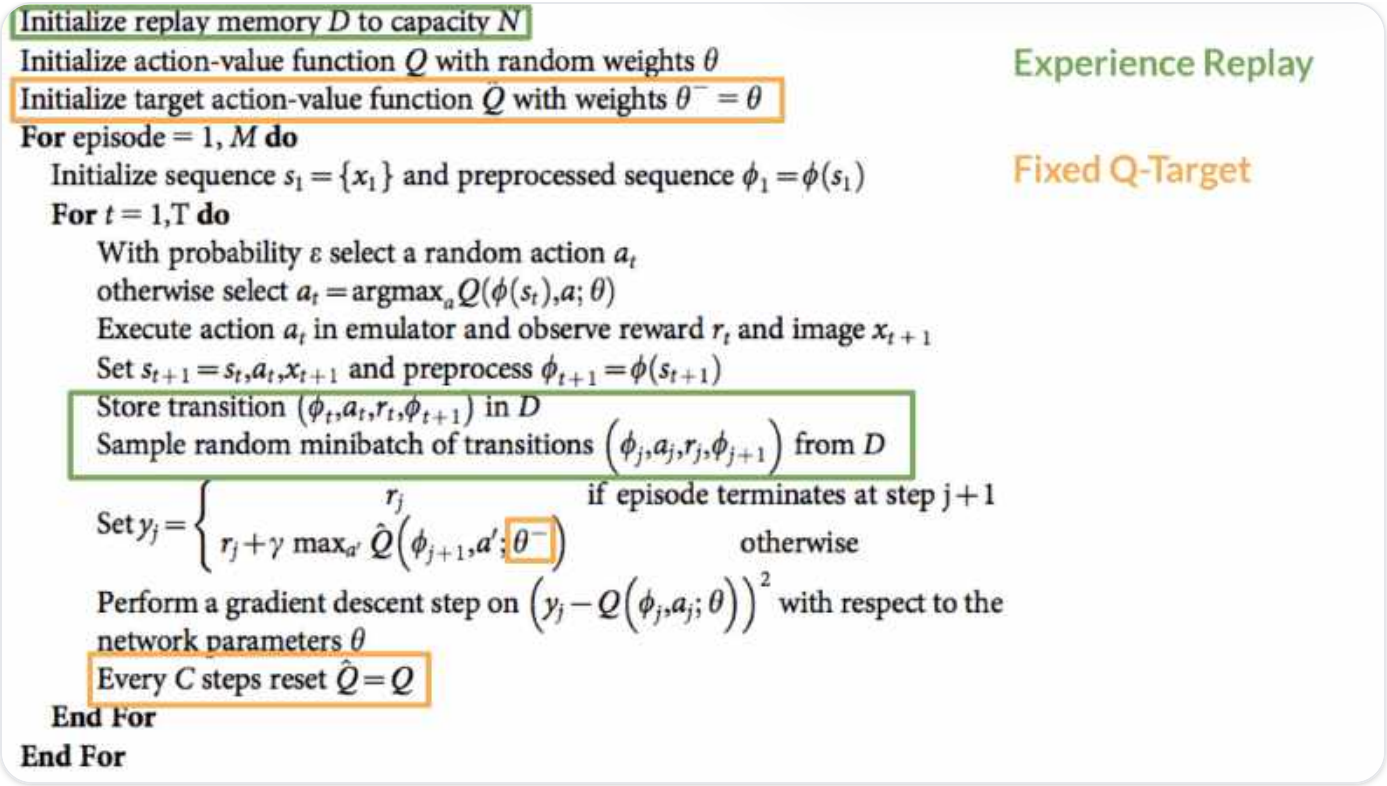


This leads to a bizarre path of chasing (a significant oscillation in training).



Instead, what we see in the pseudo-code is that we:

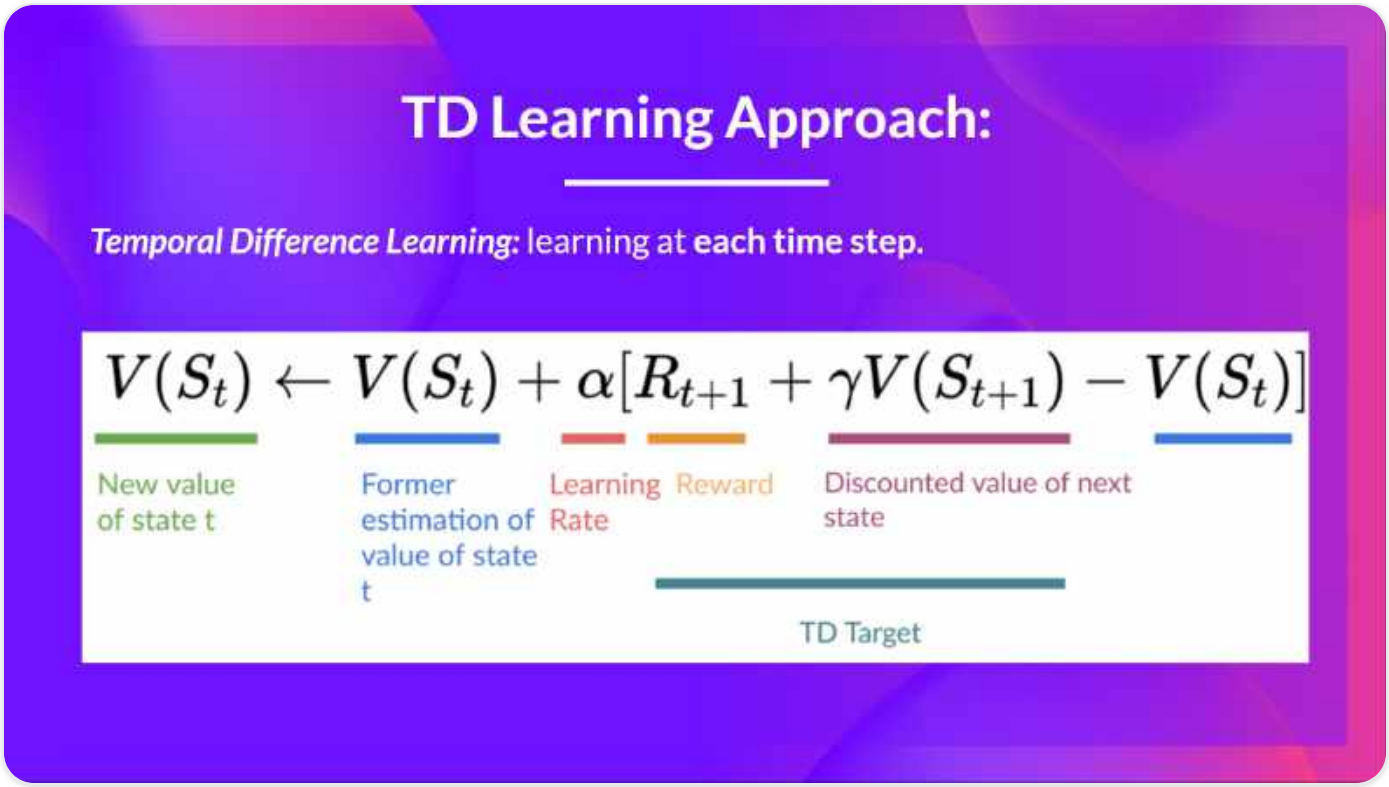
- Use a **separate network with fixed parameters** for estimating the TD Target
- **Copy the parameters from our Deep Q-Network every C steps** to update the target network.



Double DQN

Double DQNs, or Double Deep Q-Learning neural networks, were introduced by Hado van Hasselt. This method **handles the problem of the overestimation of Q-values.**

To understand this problem, remember how we calculate the TD Target:



We face a simple problem by calculating the TD target: how are we sure that **the best action for the next state is the action with the highest Q-value?**

We know that the accuracy of Q-values depends on what action we tried **and** what neighboring states we explored.

Consequently, we don't have enough information about the best action to take at the beginning of the training. Therefore, taking the maximum Q-value (which is noisy) as the best action to take can lead to false positives. If non-optimal actions are regularly **given a higher Q value than the optimal best action, the learning will be complicated.**

The solution is: when we compute the Q target, we use two networks to decouple the action selection from the target Q-value generation. We:

- Use our **DQN network** to select the best action to take for the next state (the action with the highest Q-value).
- Use our **Target network** to calculate the target Q-value of taking that action at the next state.

Therefore, Double DQN helps us reduce the overestima
helps us train faster and with more stable learning.

Since these three improvements in Deep Q-Learning, many more have been added, such as Prioritized Experience Replay and Dueling Deep Q-Learning. They're out of the scope of this course but if you're interested, check the links we put in the reading list.

<> Update on GitHub

← The Deep Q-Network (DQN) Glossary →

Search models, datasets, users...

Deep RL Course documentation
The Deep Q-Network (DQN)

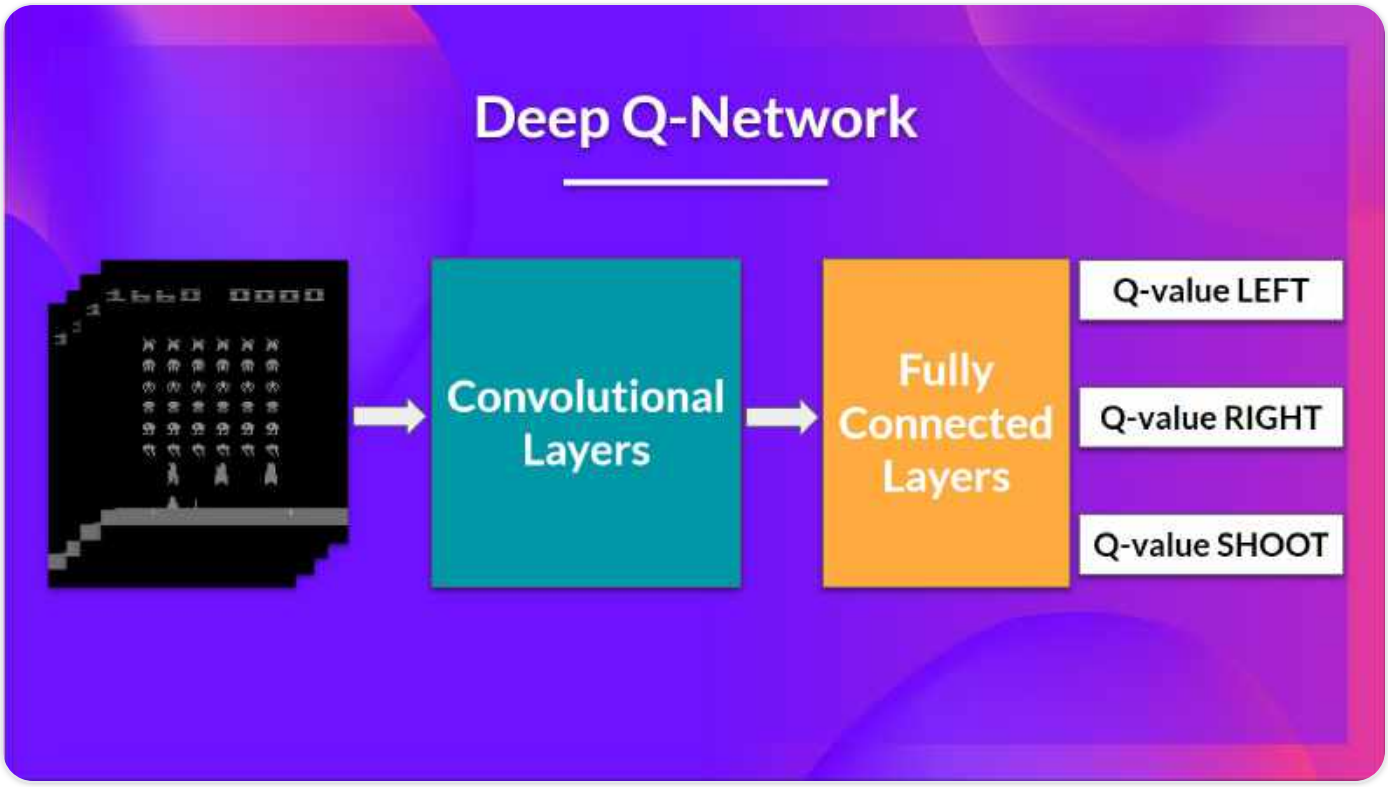
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

The Deep Q-Network (DQN)

This is the architecture of our Deep Q-Learning network:



As input, we take a **stack of 4 frames** passed through the network as a state and output a **vector of Q-values for each possible action at that state**. Then, l

Ask HuggingChat

epsilon-greedy policy to select which action to take.

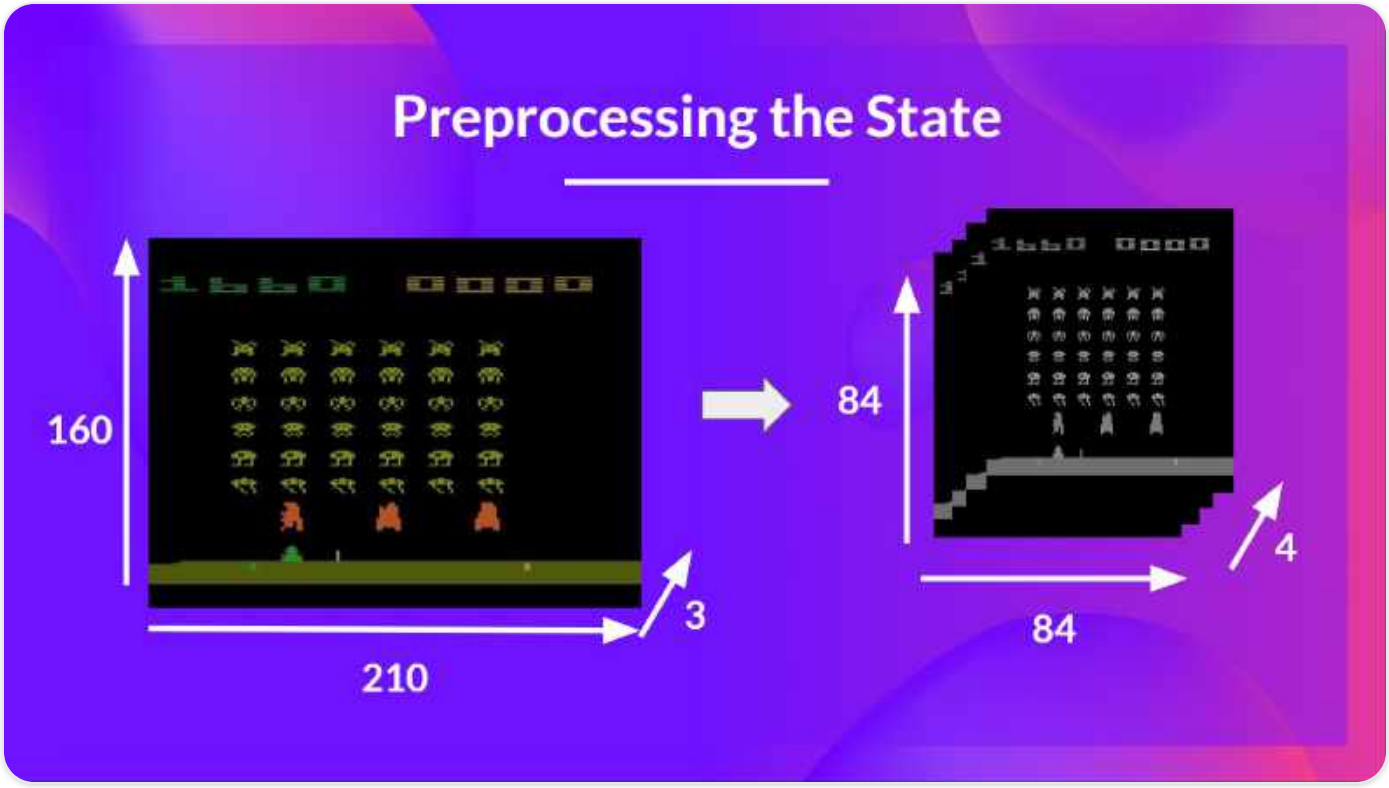
When the Neural Network is initialized, **the Q-value estimation is terrible**. But during training, our Deep Q-Network agent will associate a situation with the appropriate action and **learn to play the game well**.

Preprocessing the input and temporal limitation

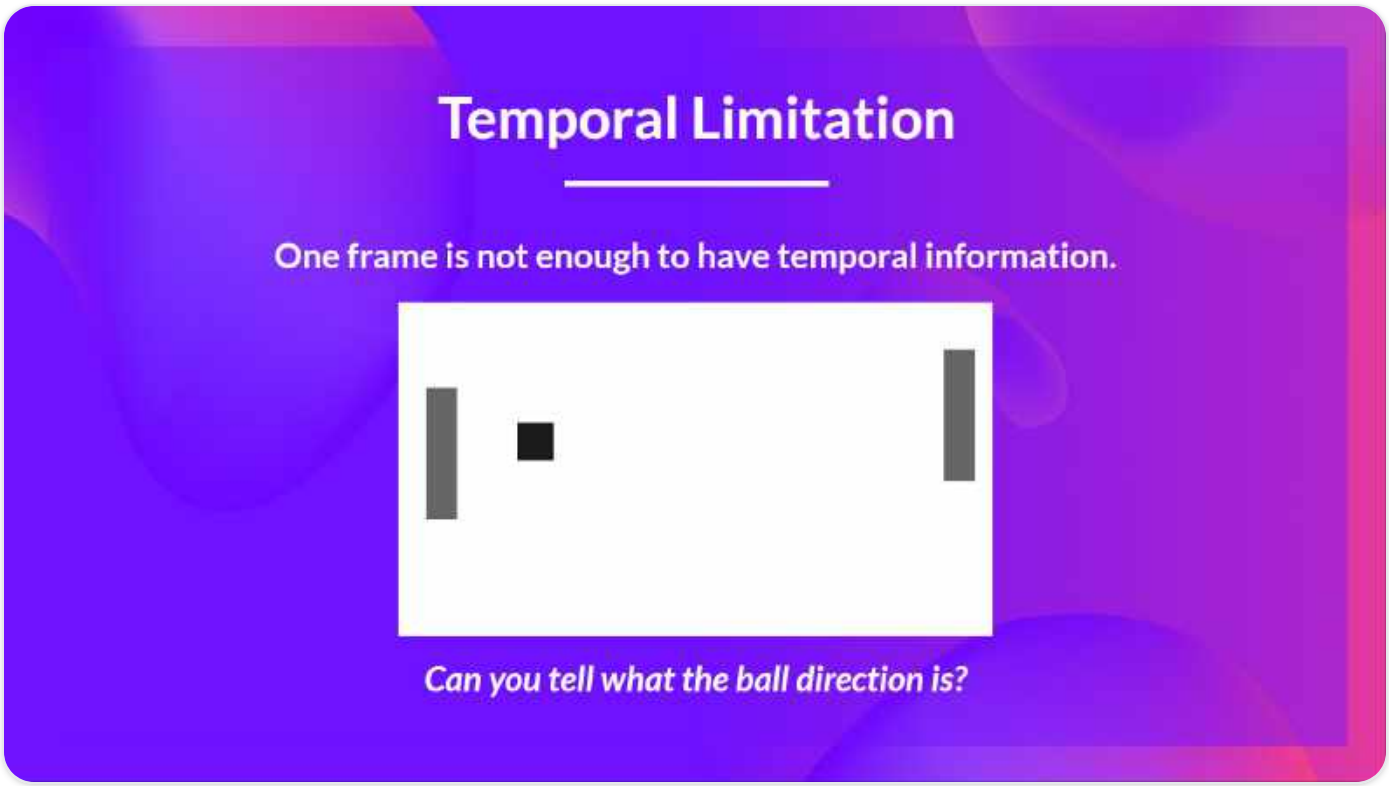
We need to **preprocess the input**. It's an essential step since we want to **reduce the complexity of our state to reduce the computation time needed for training**.

To achieve this, we **reduce the state space to 84x84 and grayscale it**. We can do this since the colors in Atari environments don't add important information. This is a big improvement since we **reduce our three color channels (RGB) to 1**.

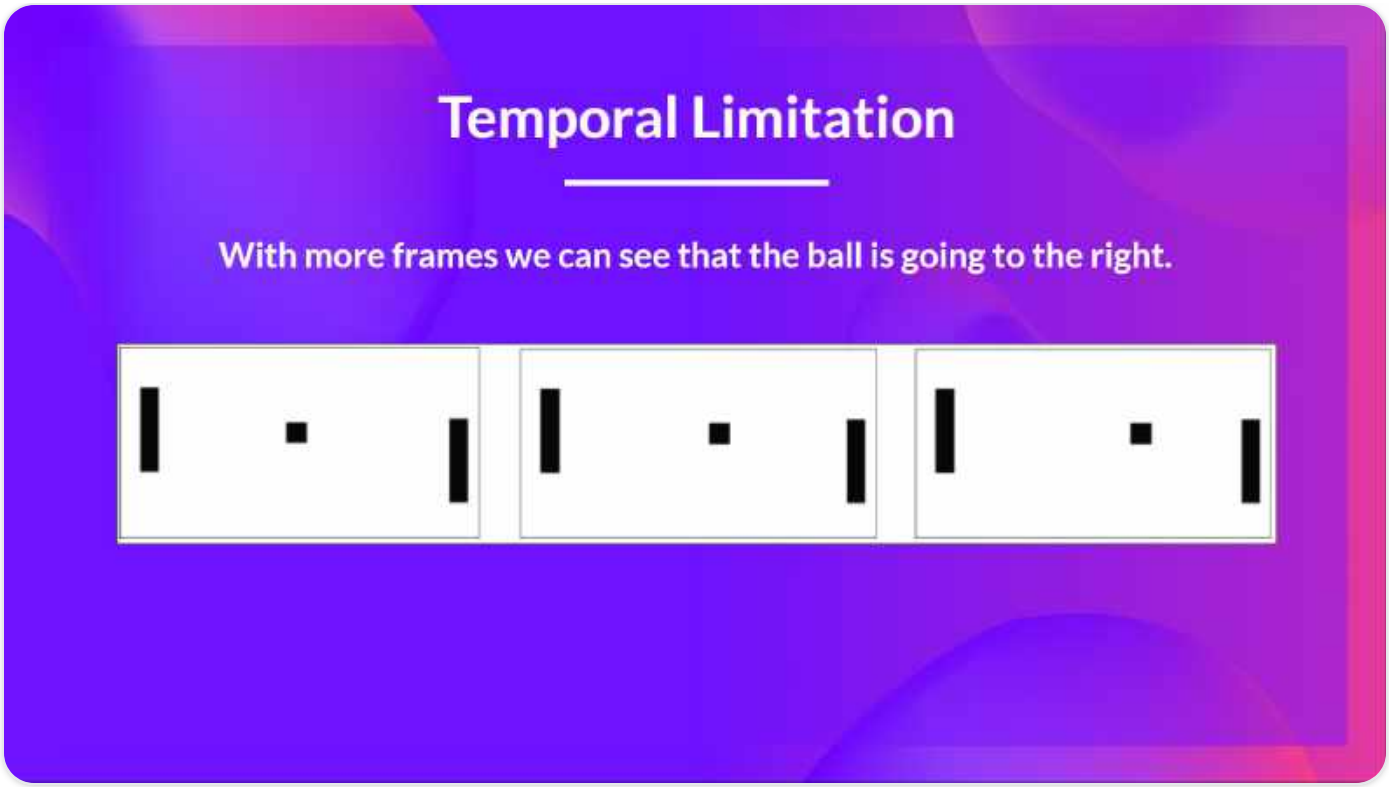
We can also **crop a part of the screen in some games** if it does not contain important information. Then we stack four frames together.



Why do we stack four frames together? We stack frames together because it helps us **handle the problem of temporal limitation**. Let's take an example with the game of Pong. When you see this frame:



Can you tell me where the ball is going? No, because one frame is not enough to have a sense of motion! But what if I add three more frames? **Here you can see that the ball is going to the right**.

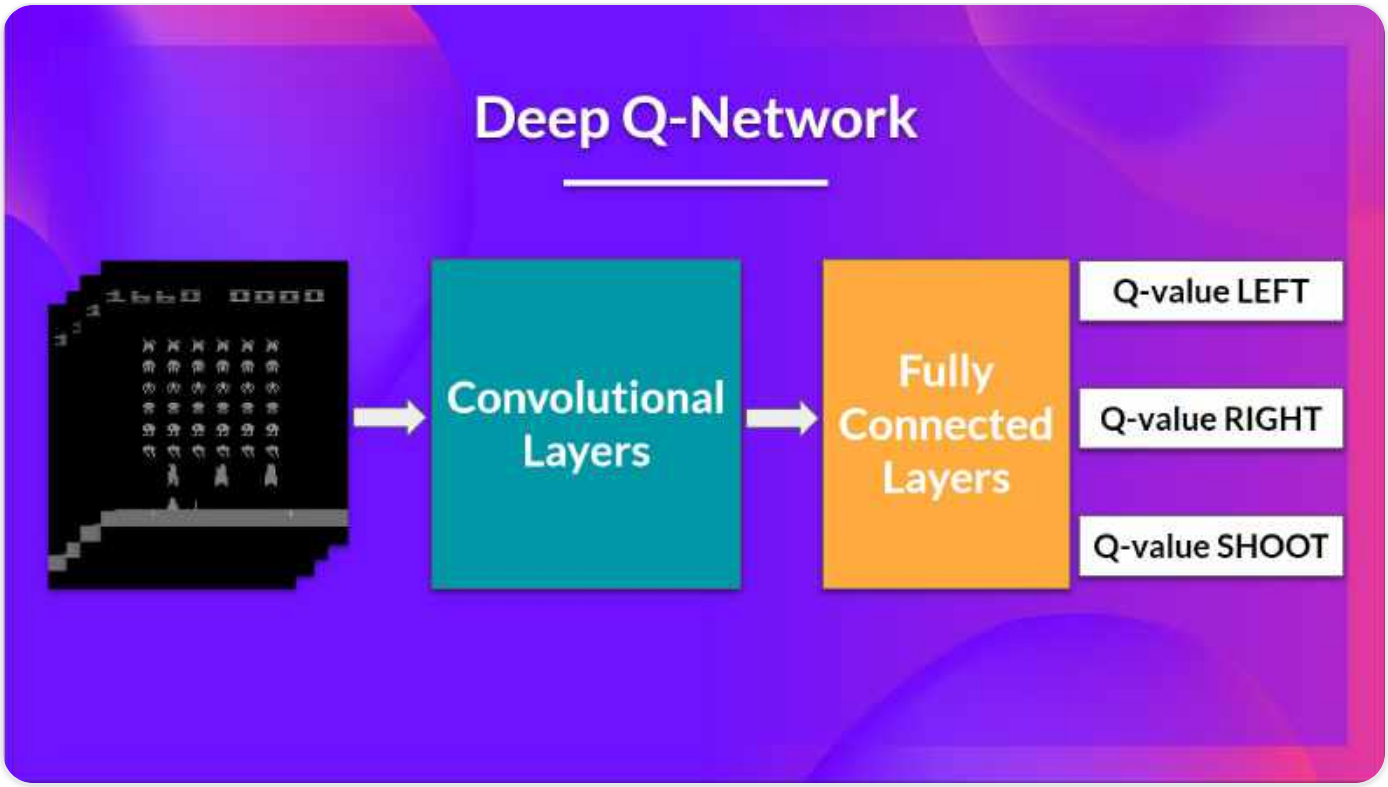


That's why, to capture temporal information, we stack four frames together.

Then the stacked frames are processed by three convolutional layers. These layers **allow us to capture and exploit spatial relationships in images**. But also, because the frames are stacked together, **we can exploit some temporal properties across those frames**.

If you don't know what convolutional layers are, don't worry. You can check out Lesson 4 of this free Deep Learning Course by Udacity

Finally, we have a couple of fully connected layers that output a Q-value for each possible action at that state.



So, we see that Deep Q-Learning uses a neural network to approximate, given a state, the different Q-values for each possible action at that state. Now let's study the Deep Q-Learning algorithm.

<> Update on GitHub

← From Q-Learning to Deep Q-Learning

The Deep Q Algorithm →

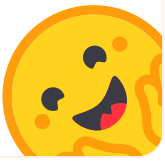
Search models, datasets, users...

Deep RL Course documentation
The environment

Join the Hugging Face community

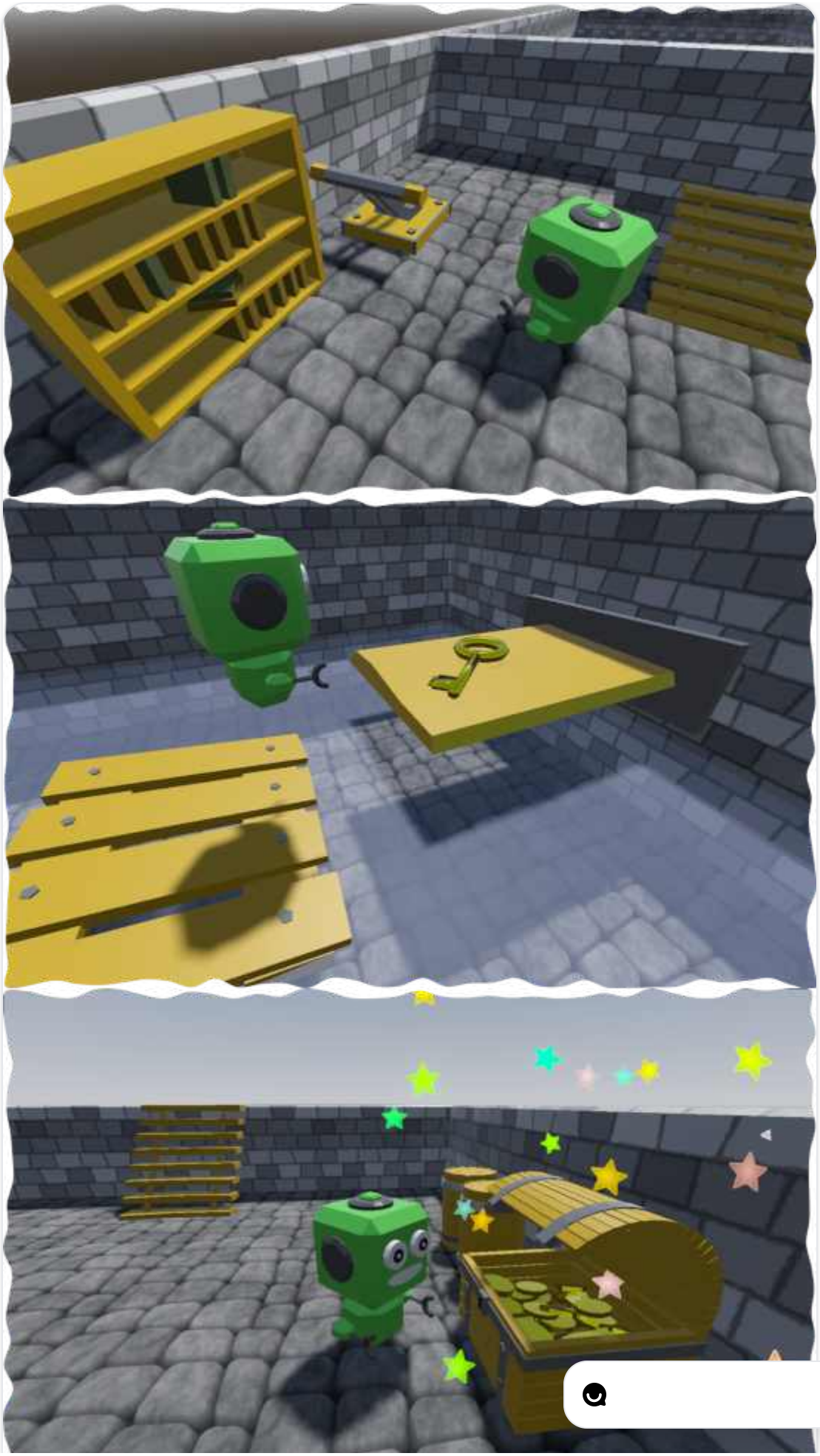
and get access to the augmented documentation experience

Sign Up to get started



The environment

Ask HuggingChat



The tutorial environment features a robot that needs to:

- Pull a lever to raise the stairs leading to the second room,
- Navigate to the key 🔑 and collect it while avoiding falling down into traps, water, or outside the map,
- Navigate back to the treasure chest in the first room, and open it. Victory! 🏆

<> Update on GitHub

← Introduction

Getting started →

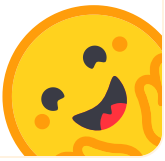
Search models, datasets, users...

Deep RL Course documentation
The Exploration/Exploitation trade-off

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



The Exploration/Exploitation trade-off

Finally, before looking at the different methods to solve Reinforcement Learning problems, we must cover one more very important topic: *the exploration/exploitation trade-off.*

- *Exploration* is exploring the environment by trying random actions in order to **find more information about the environment.**
- *Exploitation* is **exploiting known information to maximize the reward.**

Remember, the goal of our RL agent is to maximize the expected cumulative reward. However, **we can fall into a common trap**.

Let’s take an example:

Ask HuggingChat



In this game, our mouse can have an **infinite amount of small cheese** (+1 each). But at the top of the maze, there is a gigantic sum of cheese (+1000).

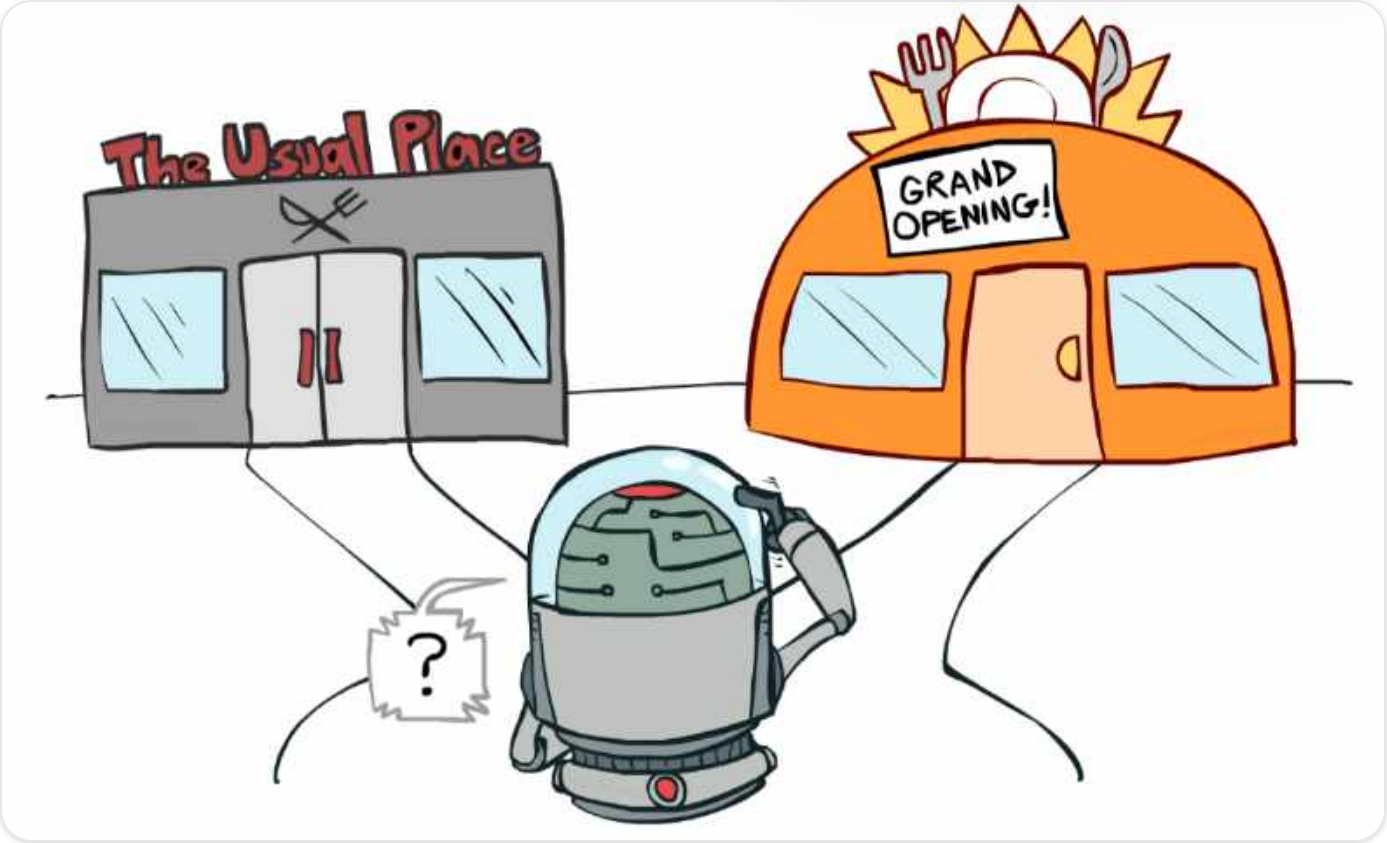
However, if we only focus on exploitation, our agent will never reach the gigantic sum of cheese. Instead, it will only exploit **the nearest source of rewards,** even if this source is small (exploitation).

But if our agent does a little bit of exploration, it can **discover the big reward** (the pile of big cheese).

This is what we call the exploration/exploitation trade-off. We need to balance how much we **explore the environment** and how much we **exploit what we know about the environment.**

Therefore, we must **define a rule that helps to handle this trade-off**. We'll see the different ways to handle it in the future units.

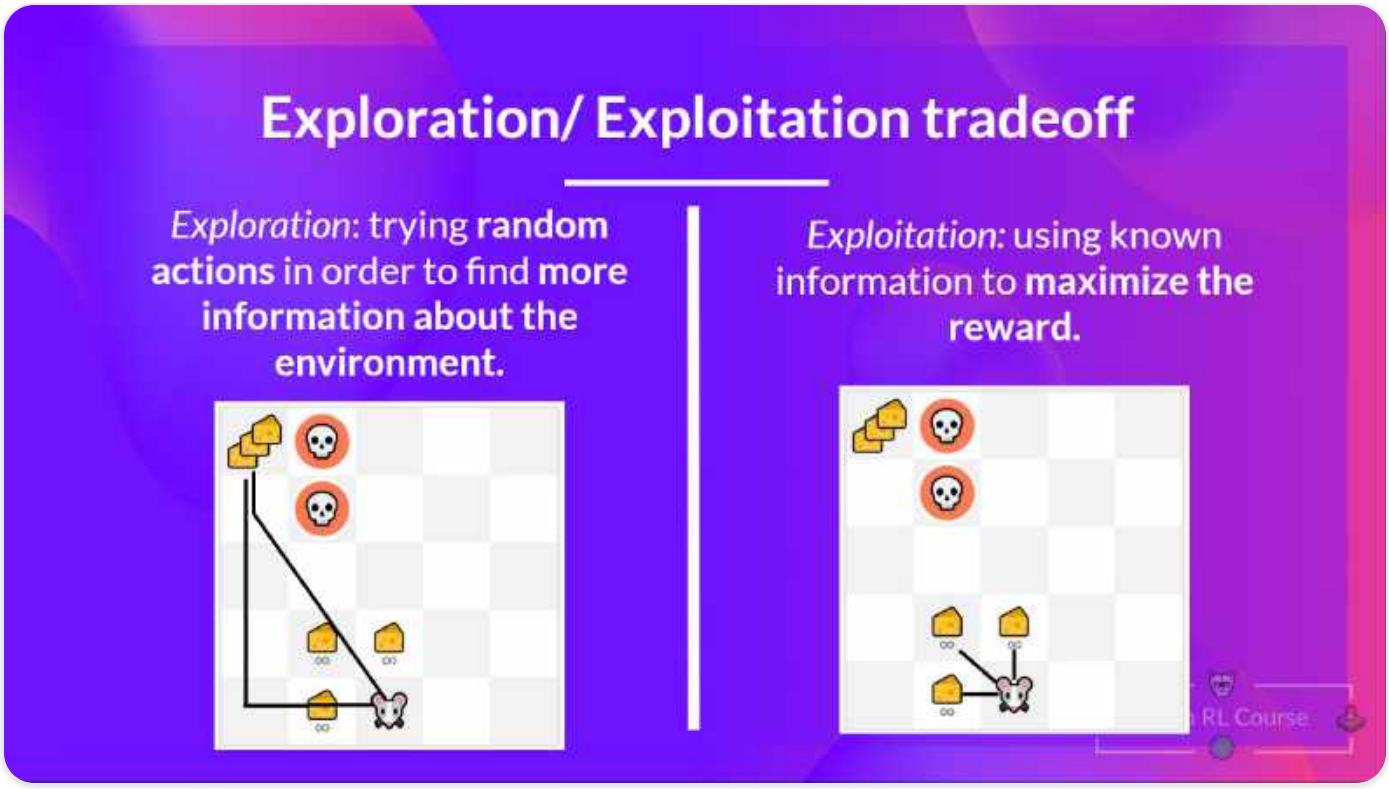
If it's still confusing, **think of a real problem: the choice of picking a restaurant:**



Source: Berkley AI Course

- *Exploitation*: You go to the same one that you know is good every day and **take the risk to miss another better restaurant.**
- *Exploration*: Try restaurants you never went to before, with the risk of having a bad experience **but the probable opportunity of a fantastic experience.**

To recap:



<> Update on GitHub

← The type of tasks

The two main approaches for solving RL problems →

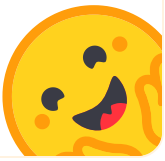
Search models, datasets, users...

Deep RL Course documentation
The intuition behind PPO

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



The intuition behind PPO

The idea with Proximal Policy Optimization (PPO) is that we want to improve the training stability of the policy by limiting the change you make to the policy at each training epoch: **we want to avoid having too large of a policy update.**

For two reasons:

- We know empirically that smaller policy updates during training are **more likely to converge to an optimal solution.**
- A too-big step in a policy update can result in falling “off the cliff” (getting a bad policy) **and taking a long time or even having no possibility to recover.**

Ask HuggingChat



Taking smaller policy updates to improve the training stability

Modified version from RL — Proximal Policy Optimization (PPO) Explained by Jonathan Hui

So with PPO, we update the policy conservatively. To do so, we need to measure how much the current policy changed compared to the former one using a ratio calculation between the current and former policy. And we clip this ratio in a range $[1 - \epsilon, 1 + \epsilon]$, meaning that we **remove the incentive for the current policy to go too far from the old one (hence the proximal policy term).**

<> Update on GitHub

← Introduction

Introducing the Clipped Surrogate Objective Function →

Search models, datasets, users...

Deep RL Course documentation
The Problem of Variance in Reinforce

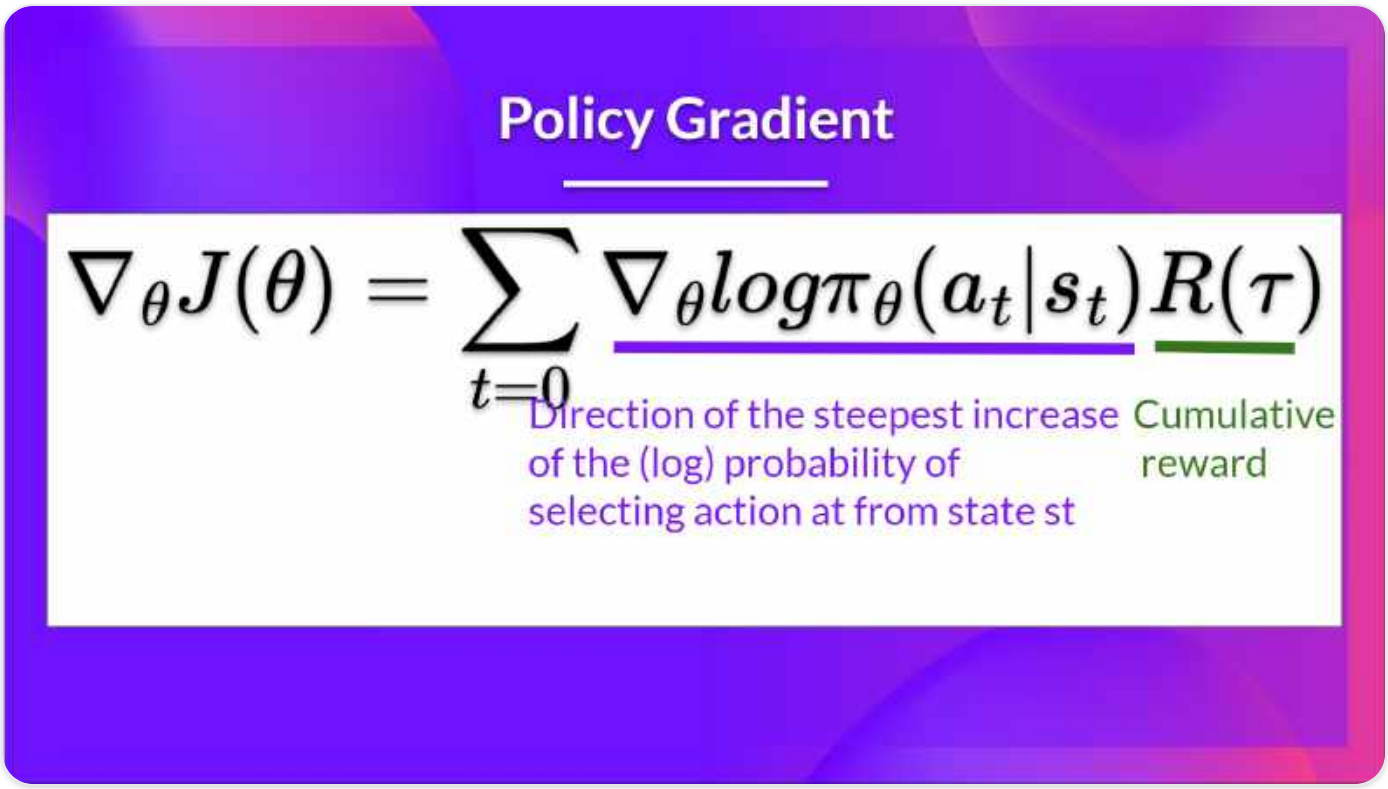
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

The Problem of Variance in Reinforce

In Reinforce, we want to **increase the probability of actions in a trajectory proportionally to how high the return is**.



- If the **return is high**, we will **push up** the probabili

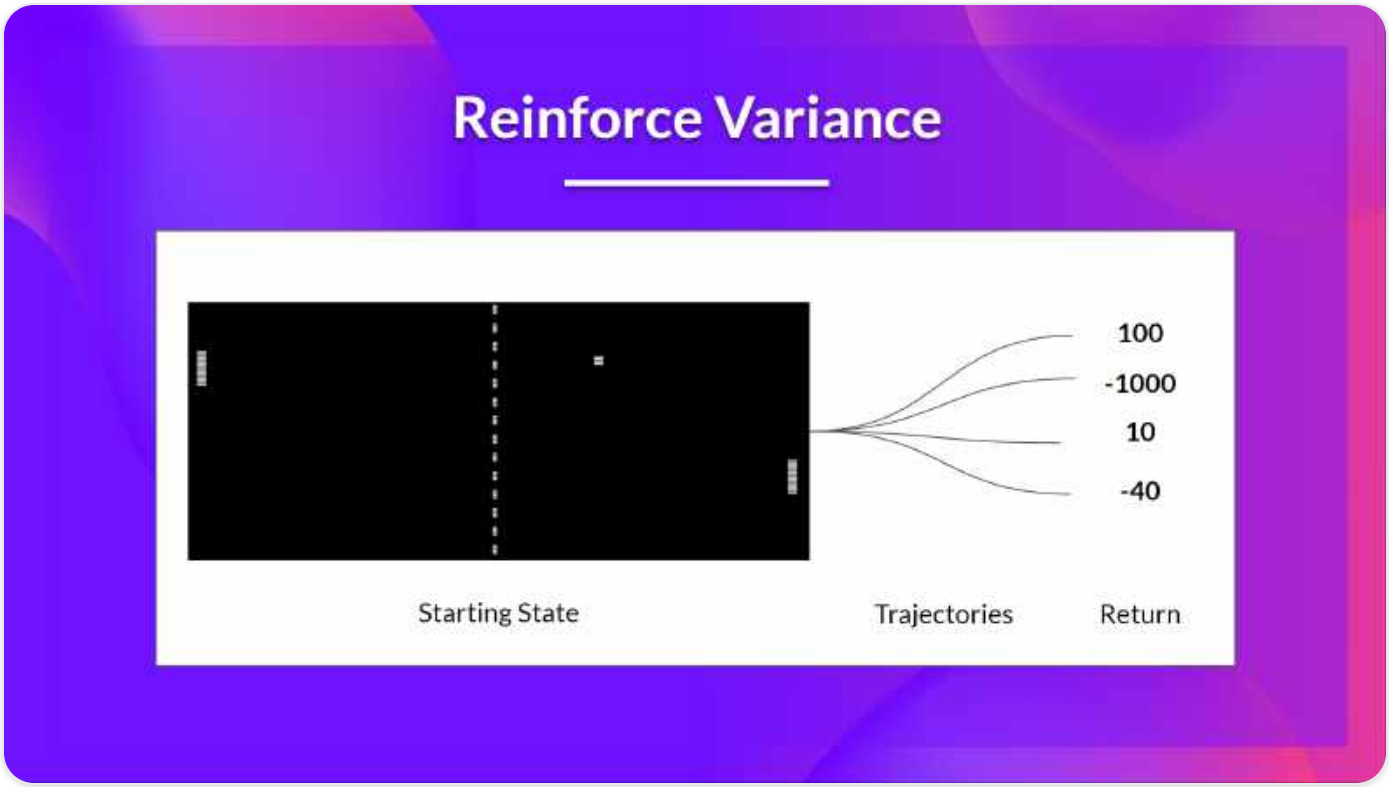
Ask HuggingChat

- Otherwise, if the **return is low**, it will **push down** the probabilities of the (state, action) combinations.

This return $R(\tau)$ is calculated using a *Monte-Carlo sampling*. We collect a trajectory and calculate the discounted return, **and use this score to increase or decrease the probability of every action taken in that trajectory**. If the return is good, all actions will be "reinforced" by increasing their likelihood of being taken. $R(\tau) = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

The advantage of this method is that **it's unbiased. Since we're not estimating the return**, we use only the true return we obtain.

Given the stochasticity of the environment (random events during an episode) and stochasticity of the policy, **trajectories can lead to different returns, which can lead to high variance**. Consequently, the same starting state can lead to very different returns. Because of this, **the return starting at the same state can vary significantly across episodes**.



The solution is to mitigate the variance by **using a large number of trajectories, hoping that the variance introduced in any one trajectory will be reduced in aggregate and provide a "true" estimation of the return.**

However, increasing the batch size significantly **reduces sample efficiency**. So we need to find additional mechanisms to reduce the variance.

If you want to dive deeper into the question of variance and bias tradeoff in Deep Reinforcement Learning, you can check out these two articles:

- Making Sense of the Bias / Variance Trade-off in (Deep) Reinforcement Learning
- Bias-variance Tradeoff in Reinforcement Learning
- High Variance in Policy gradients

<> Update on GitHub

← Introduction

Advantage Actor Critic (A2C) →

Search models, datasets, users...

Deep RL Course documentation
The Pyramid environment

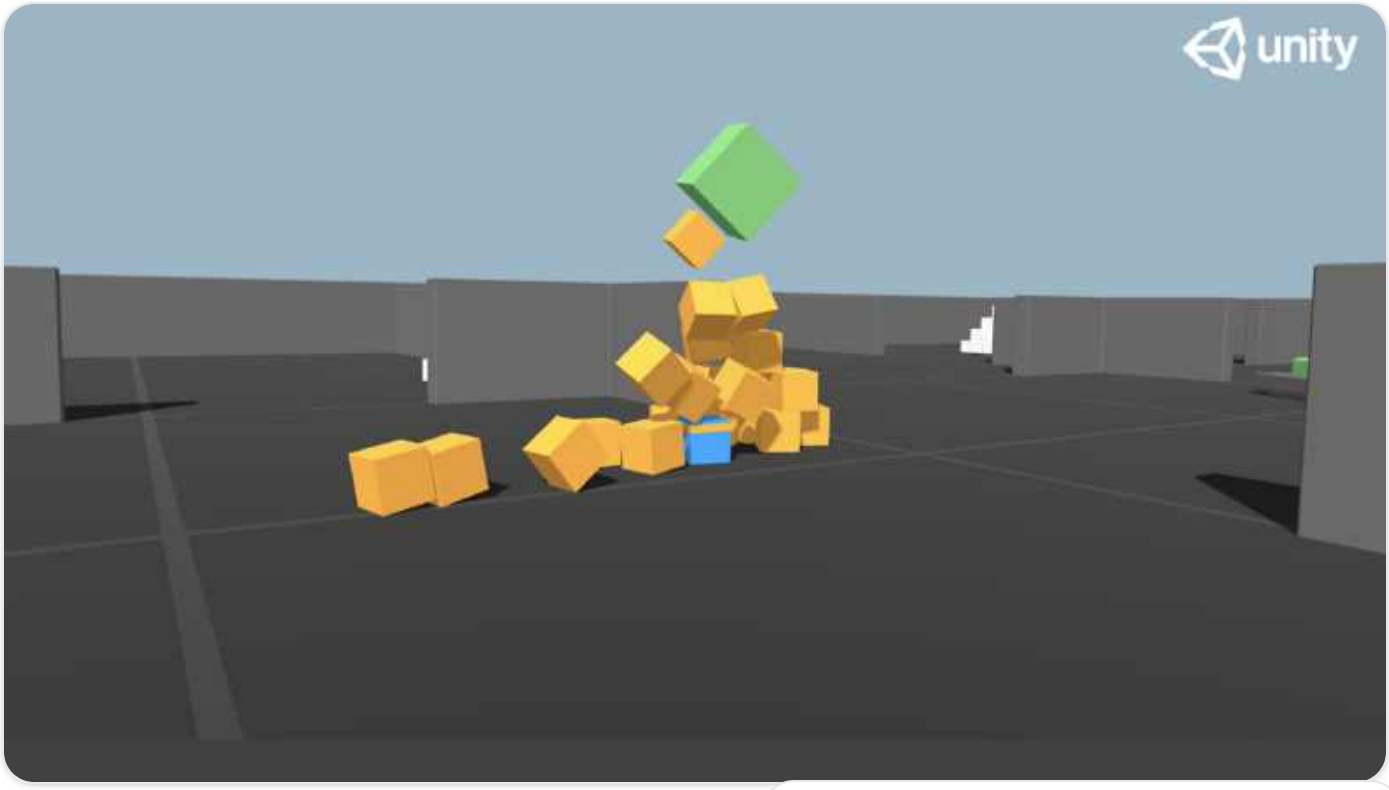
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

The Pyramid environment

The goal in this environment is to train our agent to **get the gold brick on the top of the Pyramid. To do that, it needs to press a button to spawn a Pyramid, navigate to the Pyramid, knock it over, and move to the gold brick at the top**.



Ask HuggingChat

The reward function

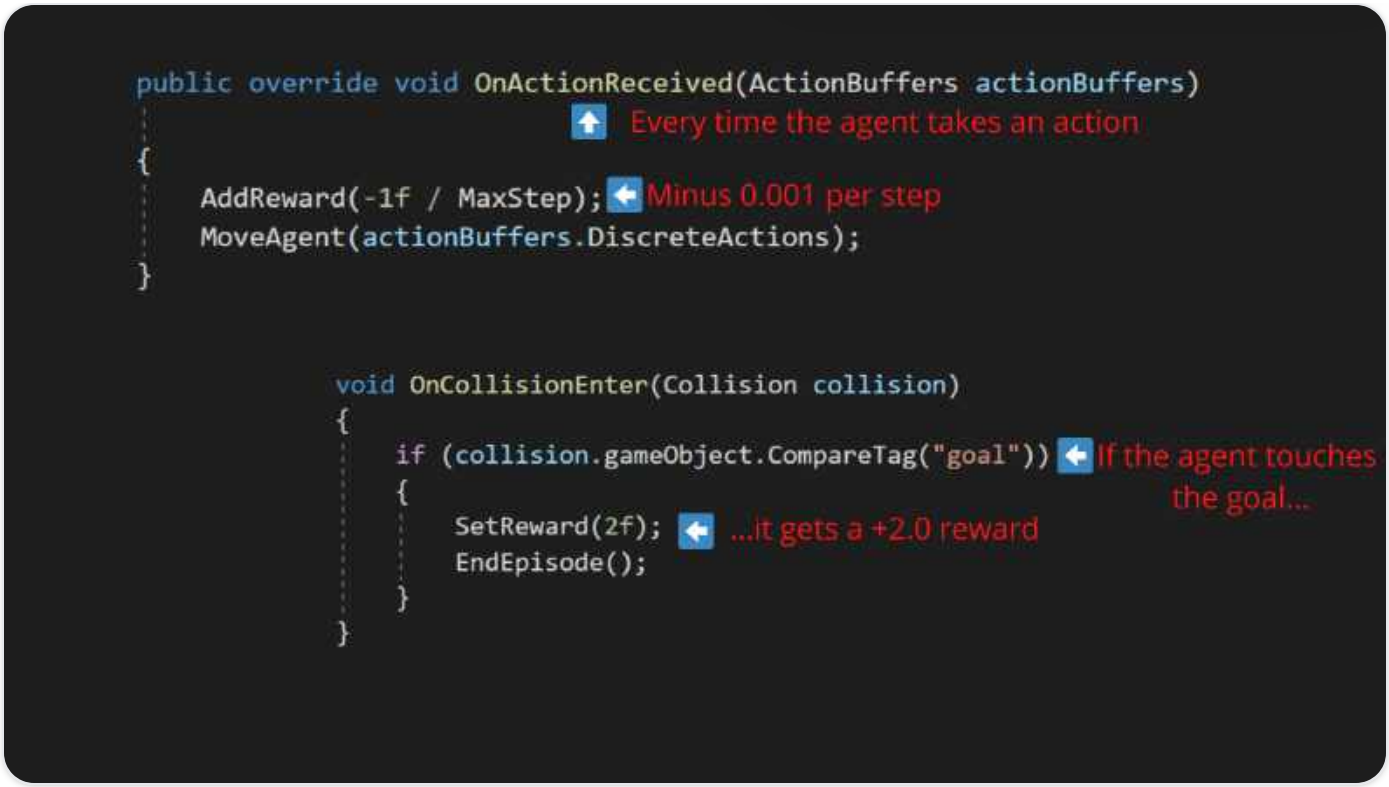
The reward function is:



-0.001 for every step, given this negative reward for every step, will push our agent to go faster.

+2 for moving to golden brick.

In terms of code, it looks like this



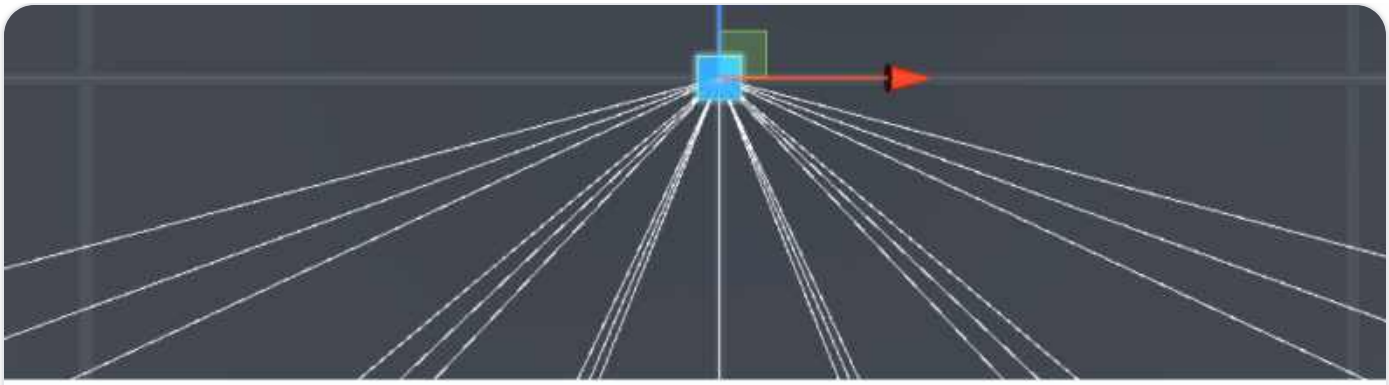
To train this new agent that seeks that button and then the Pyramid to destroy, we'll use a combination of two types of rewards:

- The *extrinsic one* given by the environment (illustration above).
- But also an *intrinsic* one called **curiosity**. This second will **push our agent to be curious, or in other terms, to better explore its environment**.

If you want to know more about curiosity, the next section (optional) will explain the basics.

The observation space

In terms of observation, we **use 148 raycasts that can each detect objects** (switch, bricks, golden brick, and walls.)



3 sets of front raycasts:

- All three detect:
  - Blocks
  - Walls
  - Goal (green cube)
  - Switch on
  - Switch off
  - Stone

We also use a **boolean variable indicating the switch state** (did we turn on or off the switch to spawn the Pyramid) and a vector that **contains the agent's speed**.

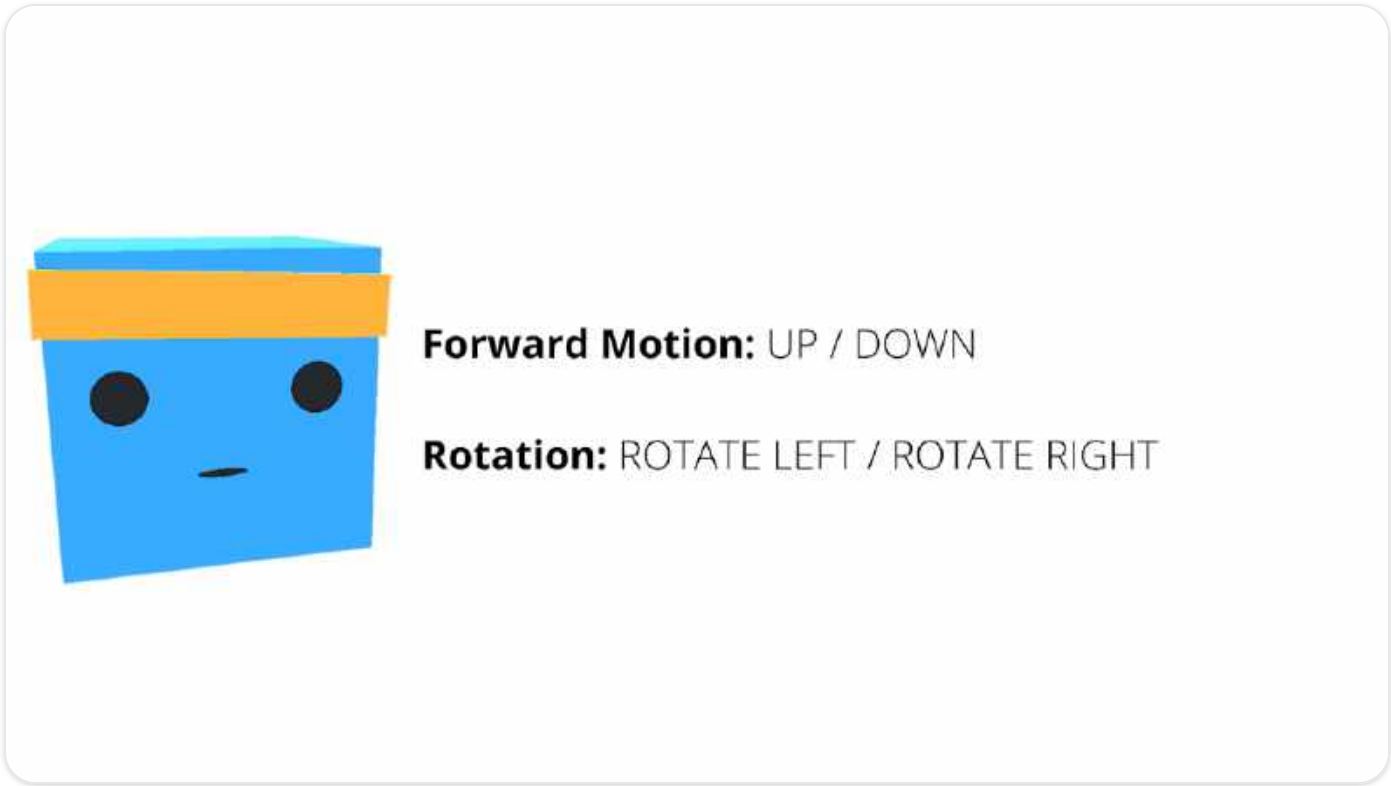
```
public override void CollectObservations(VectorSensor sensor)
{
    if (useVectorObs)
    {
        sensor.AddObservation(m_SwitchLogic.GetState());
        sensor.AddObservation(transform.InverseTransformDirection(m_AgentRb.velocity));
    }
}
```

Is switch button is on or off?

Agent's speed

The action space

The action space is **discrete** with four possible actions:



<> Update on GitHub

← The SnowballTarget environment

(Optional) What is curiosity in Deep Reinforcement Learning? →

Search models, datasets, users...

Deep RL Course documentation
The Reinforcement Learning Framework

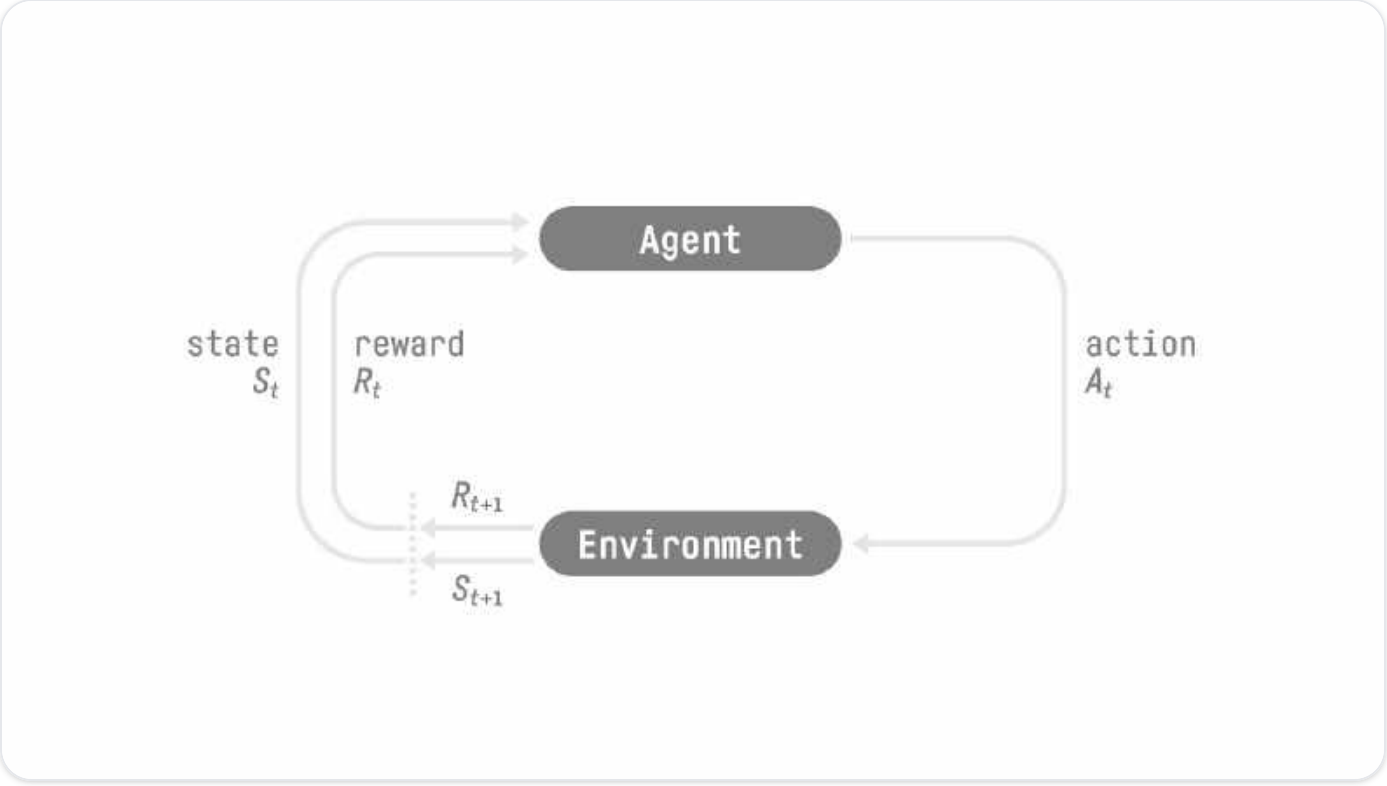
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

The Reinforcement Learning Framework

The RL Process

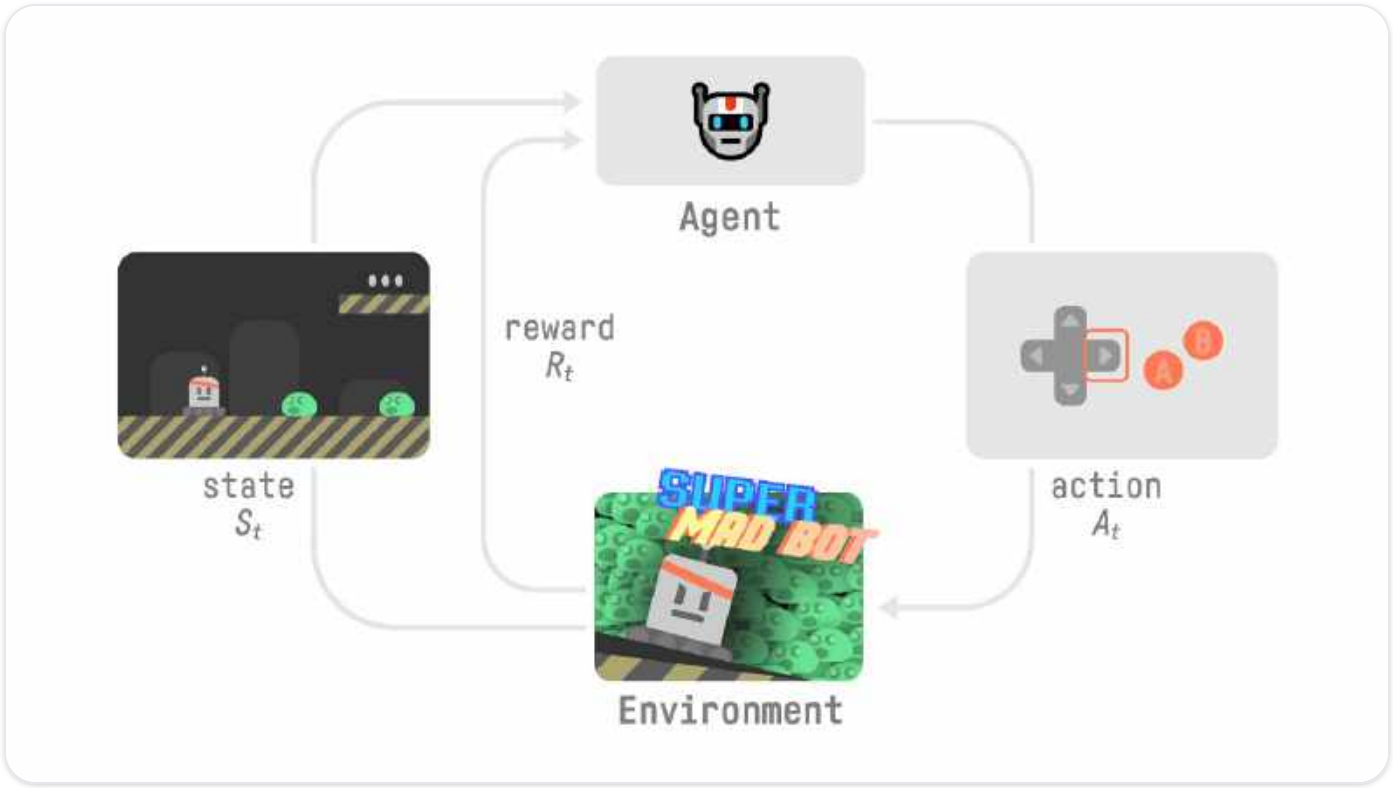


The RL Process: a loop of state, action, reward and next state

Source: Reinforcement Learning: An Introduction, Richard Sutt

Ask HuggingChat

To understand the RL process, let's imagine an agent learning to play a platform game:



- Our Agent receives **state** S_0 from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state** S_0**,** the Agent takes **action** A_0 — our Agent will move to the right.
- The environment goes to a **new state** S_1 — new frame.
- The environment gives some **reward** R_1 to the Agent — we're not dead *(Positive Reward +1)*.

This RL loop outputs a sequence of **state, action, reward and next state.**



The agent's goal is to *maximize* its cumulative reward, **called the expected return.**

The reward hypothesis: the central idea of Reinforcement Learning

⇒ Why is the goal of the agent to maximize the expected return?

Because RL is based on the **reward hypothesis**, which is that all goals can be described as the **maximization of the expected return** (expected cumulative reward).

That's why in Reinforcement Learning, **to have the best behavior,** we aim to learn to take actions that **maximize the expected cumulative reward.**

Markov Property

In papers, you'll see that the RL process is called a **Markov Decision Process** (MDP).

We'll talk again about the Markov Property in the following units. But if you need to remember something today about it, it's this: the Markov Property implies that our agent needs **only the current state to decide** what action to take and **not the history of all the states and actions** they took before.

Observations/States Space

Observations/States are the **information our agent gets from the environment.** In the case of a video game, it can be a frame (a screenshot). In the case of the trading agent, it can be the value of a certain stock, etc.

There is a differentiation to make between *observation* and *state*, however:

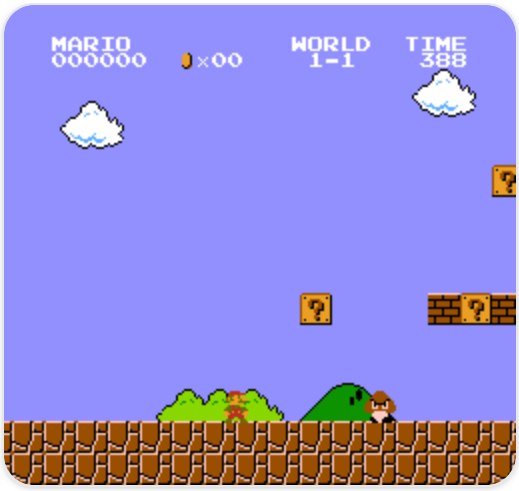
- *State s*: is **a complete description of the state of the world** (there is no hidden information). In a fully observed environment.



In chess game, we receive a state from the environment since we have access to the whole check board information.

In a chess game, we have access to the whole board information, so we receive a state from the environment. In other words, the environment is fully observed.

- *Observation o*: is a **partial description of the state.** In a partially observed environment.



In Super Mario Bros, we only see the part of the level close to the player, so we receive an observation.

In Super Mario Bros, we only see the part of the level close to the player, so we receive an observation.

In Super Mario Bros, we are in a partially observed environment. We receive an observation **since we only see a part of the level.**

> In this course, we use the term "state" to denote both state and observation, but we will make the distinction in implementations.

To recap:



Action Space

The Action space is the set of **all possible actions in an environment.**

The actions can come from a *discrete* or *continuous space*:

- *Discrete space*: the number of possible actions is **finite**.



In Super Mario Bros, we have only 4 possible actions: left, right, up (jumping) and down (crouching).

Again, in Super Mario Bros, we have a finite set of actions since we have only 4 directions.

- *Continuous space*: the number of possible actions is **infinite**.



A Self Driving Car agent has an infinite number of possible actions since it can turn left 20°, 21,1°, 21,2°, honk, turn right 20°…

To recap:

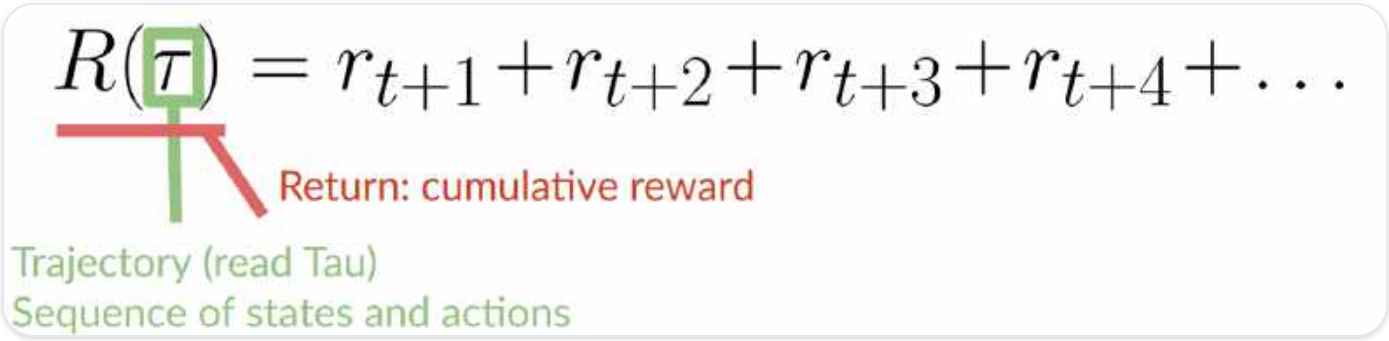


Taking this information into consideration is crucial because it will **have importance when choosing the RL algorithm in the future.**

Rewards and the discounting

The reward is fundamental in RL because it's **the only feedback** for the agent. Thanks to it, our agent knows **if the action taken was good or not.**

The cumulative reward at each time step **t** can be written as:



The cumulative reward equals the sum of all rewards in the sequence.

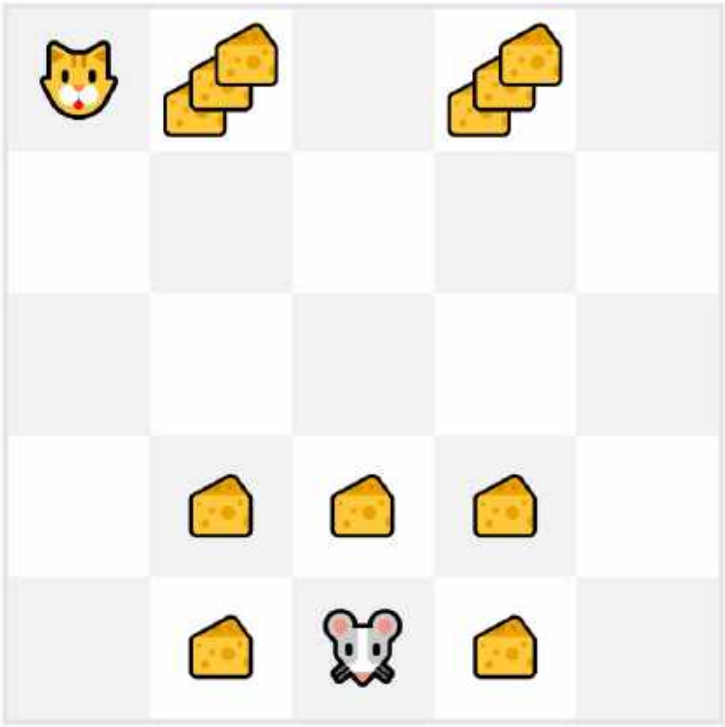
Which is equivalent to:

$$R(\tau) = \sum_{k=0}^{\infty} r_{t+k+1}$$

The cumulative reward = rt+1 (rt+k+1 = rt+0+1 = rt+1)+ rt+2 (rt+k+1 = rt+1+1 = rt+2) + ...

However, in reality, **we can't just add them like that.** The rewards that come sooner (at the beginning of the game) **are more likely to happen** since they are more predictable than the long-term future reward.

Let's say your agent is this tiny mouse that can move one tile each time step, and your opponent is the cat (that can move too). The mouse's goal is **to eat the maximum amount of cheese before being eaten by the cat.**



As we can see in the diagram, **it's more probable to eat the cheese near us than the cheese close to the cat** (the closer we are to the cat, the more dangerous it is).

Consequently, **the reward near the cat, even if it is bigger (more cheese), will be more discounted** since we're not really sure we'll be able to eat it.

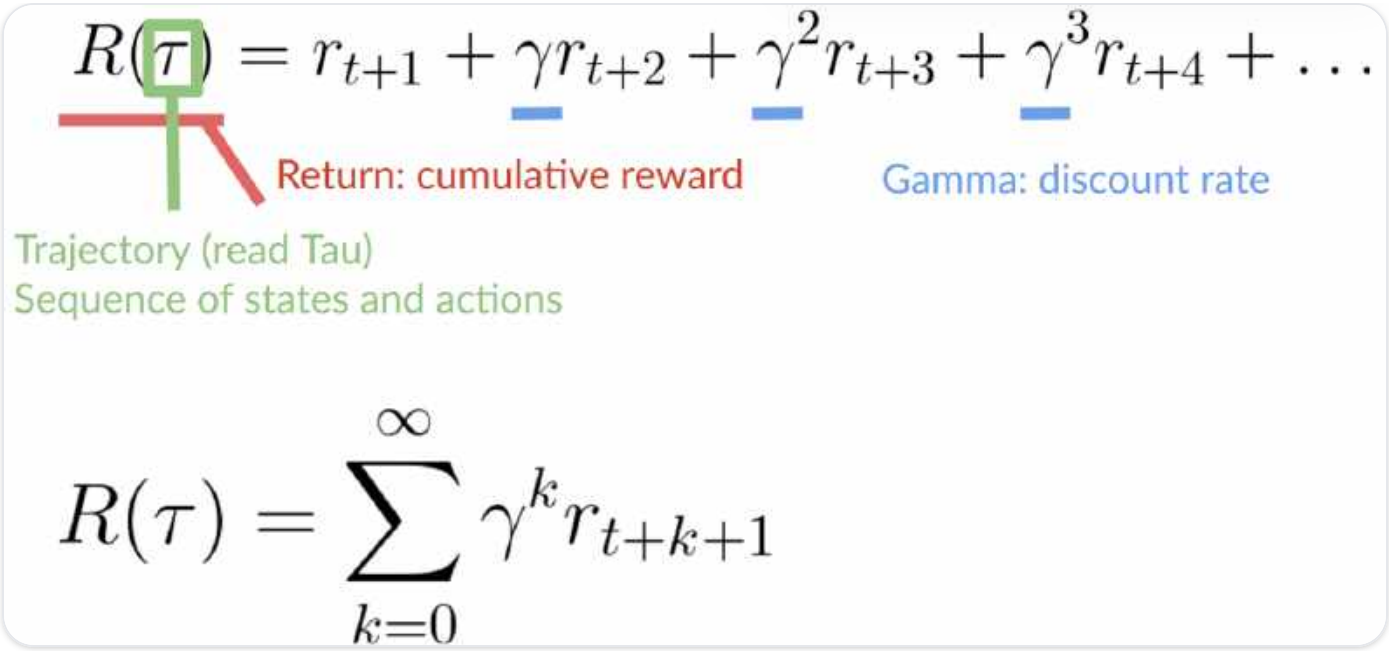
To discount the rewards, we proceed like this:

1. We define a discount rate called gamma. **It must be between 0 and 1.** Most of the time between **0.95 and 0.99**.

- The larger the gamma, the smaller the discount. This means our agent **cares more about the long-term reward.**
- On the other hand, the smaller the gamma, the bigger the discount. This means our **agent cares more about the short term reward (the nearest cheese).**

2. Then, each reward will be discounted by gamma to the exponent of the time step. As the time step increases, the cat gets closer to us, **so the future reward is less and less likely to happen.**

Our discounted expected cumulative reward is:



<> Update on GitHub

← What is Reinforcement Learning?

The type of tasks →

Search models, datasets, users...

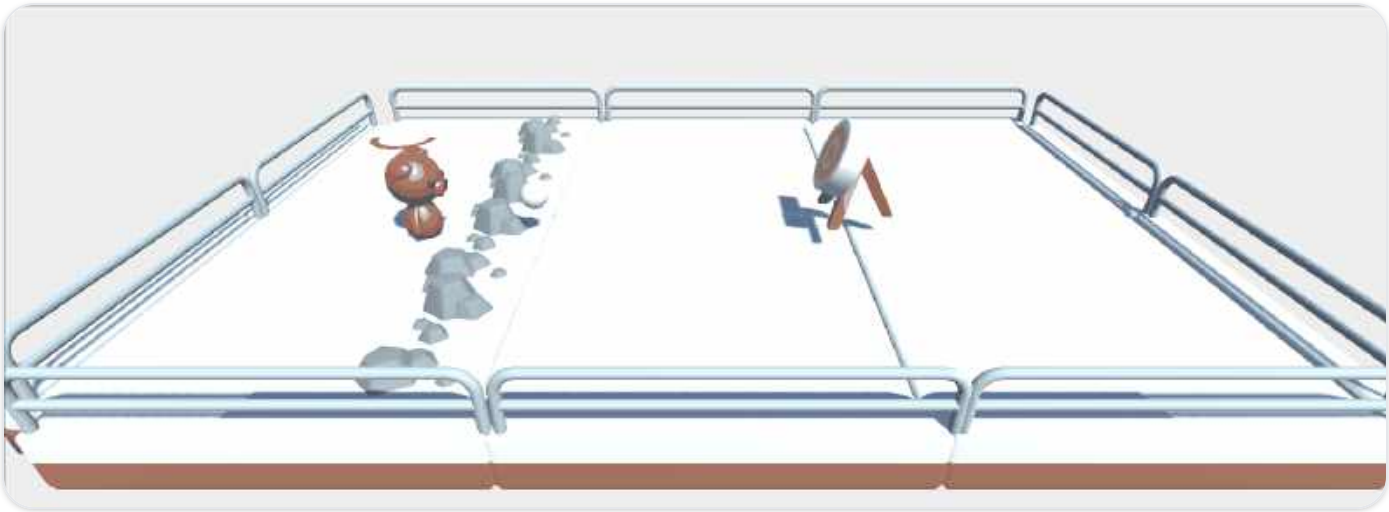
Deep RL Course documentation
The SnowballTarget Environment

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

The SnowballTarget Environment



SnowballTarget is an environment we created at Hugging Face using assets from Kay Lousberg. We have an optional section at the end of this Unit **if you want to learn to use Unity and create your environments**.

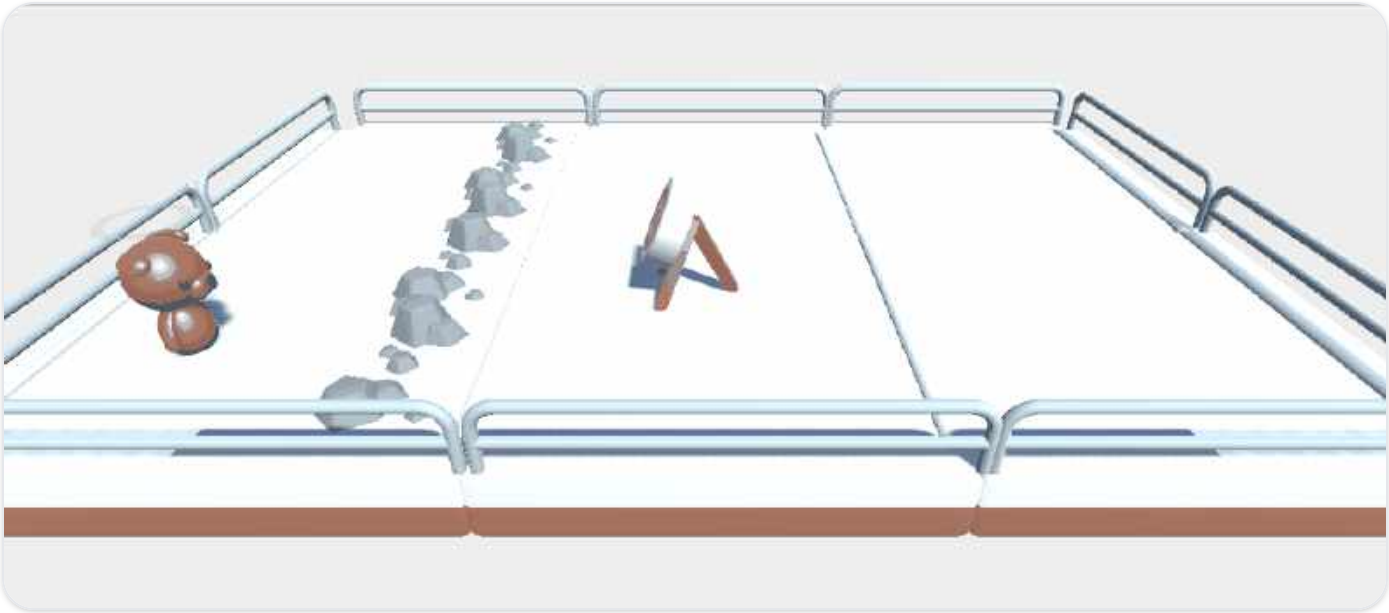
The agent's Goal

The first agent you're going to train is called Julien the bear 🐻. Julien is trained **to hit targets with snowballs**.

Ask HuggingChat

The Goal in this environment is that Julien **hits as many targets as possible in the limited time** (1000 timesteps). It will need **to place itself correctly in relation to the target and shoot**to do that.

In addition, to avoid “snowball spamming” (aka shooting a snowball every timestep), **Julien has a “cool off” system** (it needs to wait 0.5 seconds after a shoot to be able to shoot again).



The agent needs to wait 0.5s before being able to shoot a snowball again

The reward function and the reward engineering problem

The reward function is simple. **The environment gives a +1 reward every time the agent’s snowball hits a target**. Because the agent’s Goal is to maximize the expected cumulative reward, **it will try to hit as many targets as possible**.



We could have a more complex reward function (with a penalty to push the agent to go faster, for example). But when you design an environment, you need to avoid the *reward engineering problem*, which is having a too complex reward function to force your agent to behave as you want it to do. Why? Because by doing that, **you might miss interesting strategies that the agent will find with a simpler reward function**.

In terms of code, it looks like this:

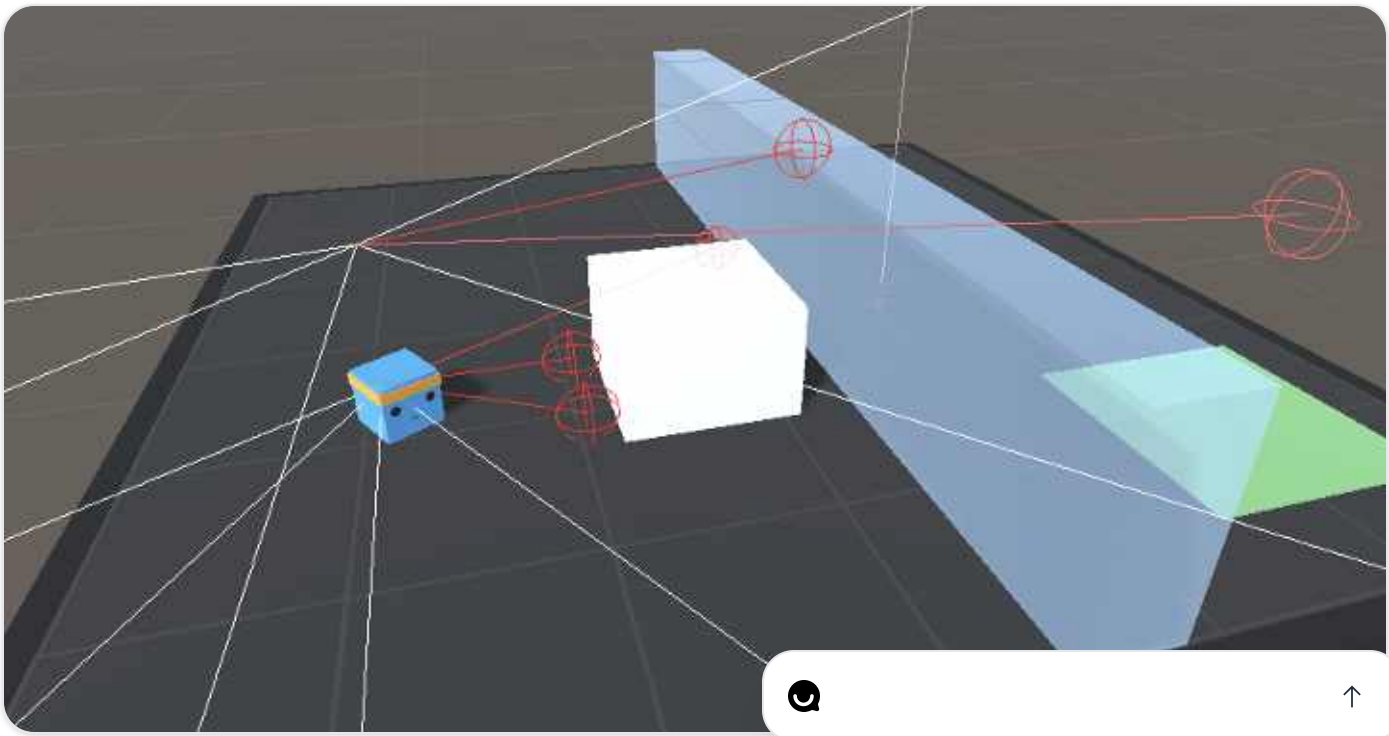
```
private void OnTriggerEnter(Collider collision)
{
    if (collision.gameObject.CompareTag("target"))
    {
        agent.SetReward(1f );   Get a +1.0 reward
        agent.PlaceTarget();    Spawn another target
    }
}
```

If the agent's snowball hits the target

The observation space

Regarding observations, we don't use normal vision (frame), but **we use raycasts**.

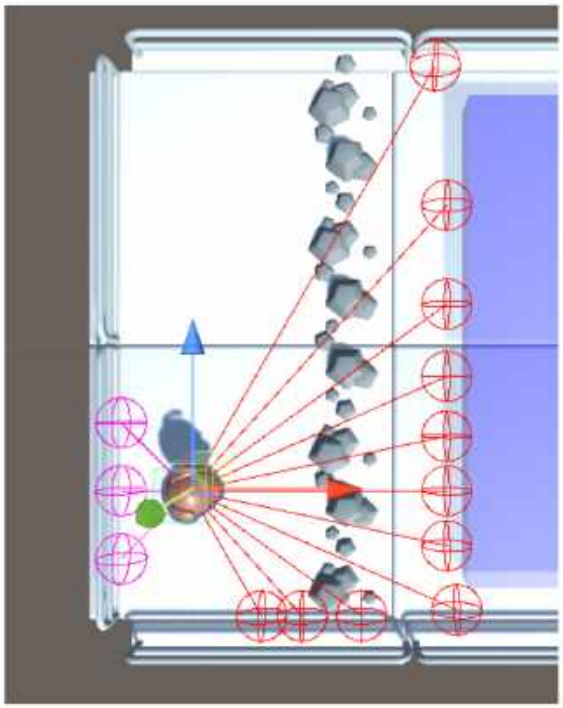
Think of raycasts as lasers that will detect if they pass through an object.



Source: ML-Agents documentation

In this environment, our agent has multiple set of raycasts:

1 set of back raycasts:
- Detect invisible walls



3 sets of front raycasts:
- One set to detect snowballs
- One set to detect targets
- One set to detect frontier (the rocks) and invisible walls

In addition to raycasts, the agent gets a “can I shoot” bool as observation.

```
/// <summary>
/// Collect the observations (vector obs)
/// </summary>
/// <param name="sensor"></param>
8 références
public override void CollectObservations(VectorSensor sensor)
{
    // Can I shoot or not? (Bool)
    sensor.AddObservation(possibleShoot);
}
```

The action space

The action space is discrete:



<> Update on GitHub

← How ML-Agents works?

The Pyramids environment →

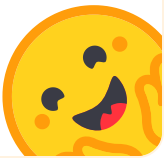
Search models, datasets, users...

Deep RL Course documentation
Train our robot

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Train our robot

> In order to start training, we'll first need to install the imitation library in the same venv / conda env where you installed Godot RL Agents by using: `pip install imitation`

Download a copy of the imitation learning script from the Godot RL Repository.

Run training using the arguments below:

```
sb3_imitation.py --env_path="path_to_ILTutorial_executable" --bc_epochs=100 --gail_timest
```

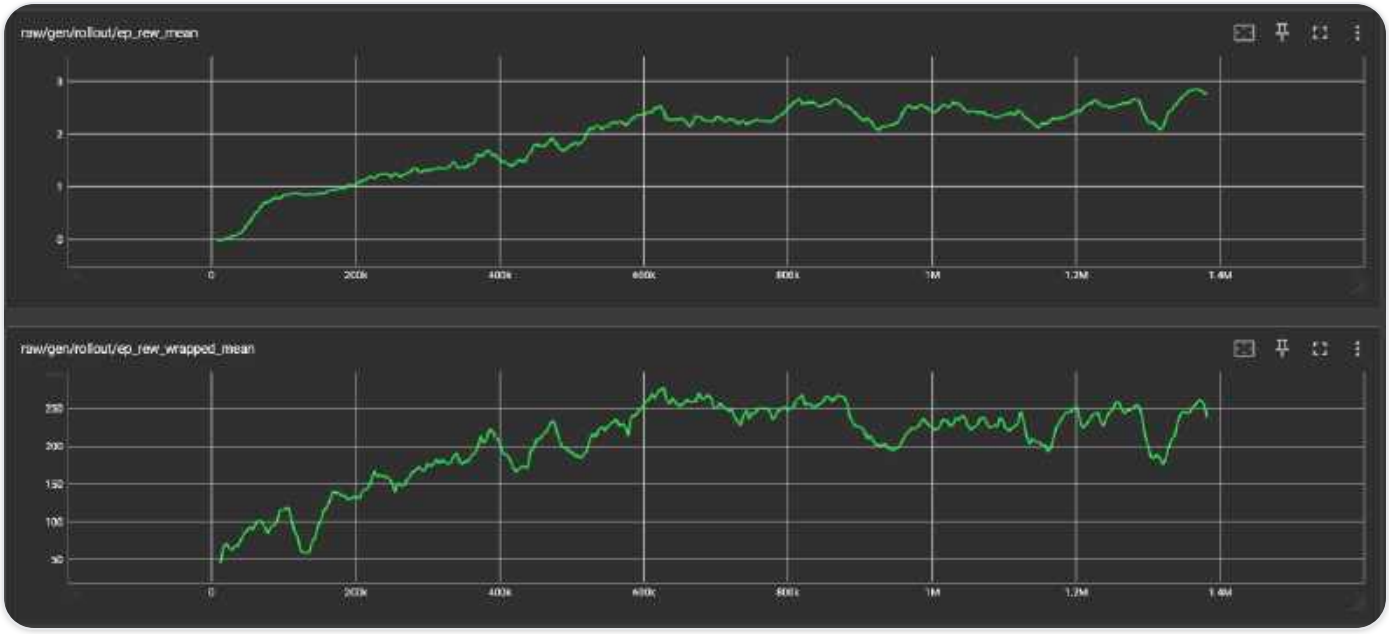
Set the env path to the exported game, and demo files path to the recorded demos. If you have multiple demo files add them with a space in between, e.g. `--demo_files demos.json demos2.json`.

You can also set a large amount of timesteps for `--gail_timesteps` and then manually stop training with `CTRL+C`. I used this method to stop training when the reward started to approach 3, which was at `total_timesteps | 1.38e+06`. That took ~41 minutes, and the BC pre-training took ~5.5 minutes on my PC using CPU for training.

Ask HuggingChat

To observe the environment while training, add the `--viz` argument. For the duration of the BC training, the env will be frozen as this stage doesn't use the env except to get some information about the observation and action spaces. During the GAIL training stage, the env rendering will update.

Here are the `ep_rew_mean` and `ep_rew_wrapped_mean` stats from the logs displayed using tensorboard, we can see that they are closely matching in this case:



> You can find the logs in `logs/ILTutorial` relative to the path you started training from. If making multiple runs, change the `--experiment_name` argument between each.

Even though setting the env rewards is not necessary and not used for the training here, a simple sparse reward was implemented to track success. Falling outside the map, in water, or traps sets `reward += -1`, while activating the lever, collecting the key, and opening the chest each set `reward += 1`. If the `ep_rew_mean` approaches 3, we are getting a good result. `ep_rew_wrapped_mean` is the reward from the GAIL discriminator, which does not directly tell us how successful the agent is at solving the environment.

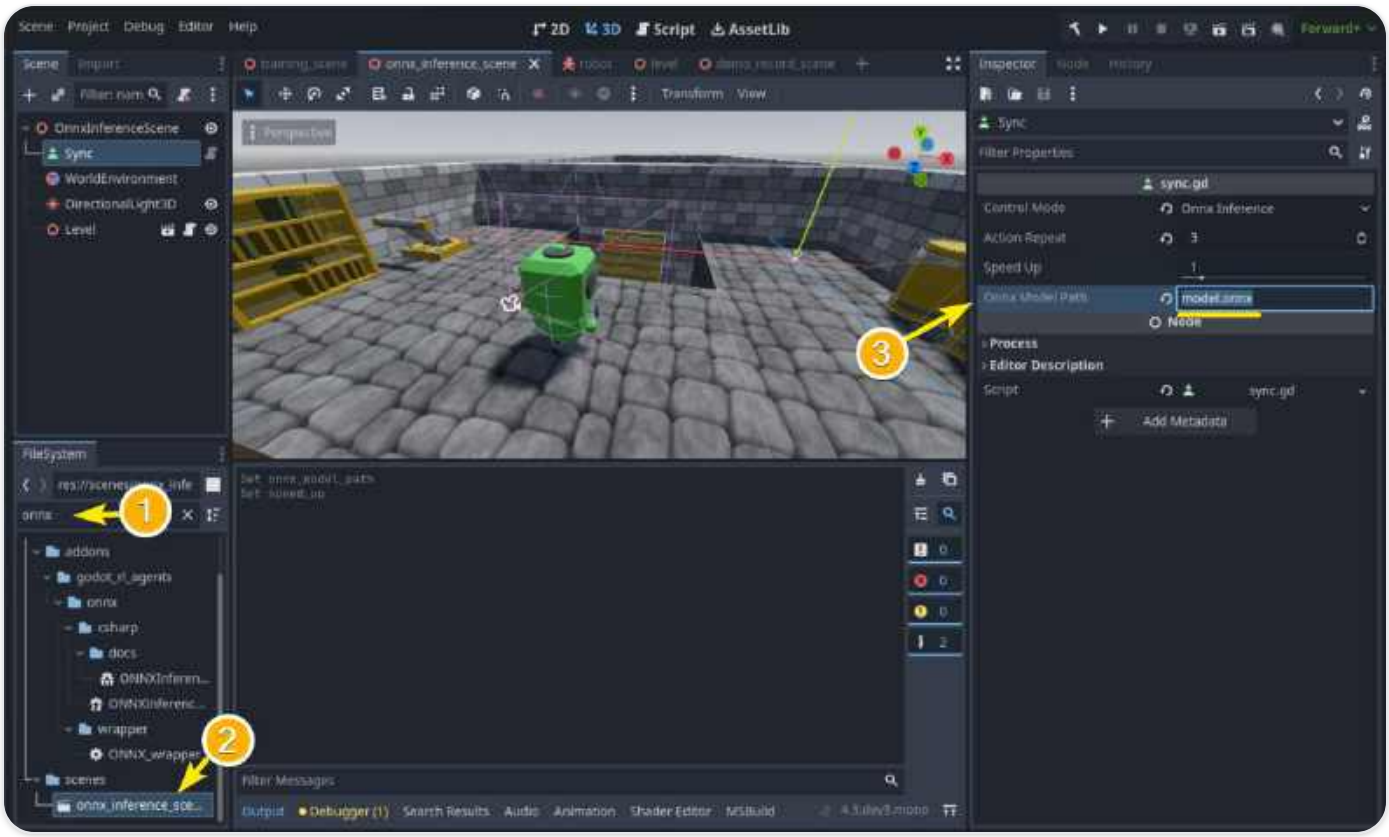
Let's test the trained agent

After training, you'll find a `model.onnx` file in the folder you started the training script from (you can also find the full path to the `.onnx` file in the training log in the console, near the end). **Copy it to the Godot game project folder.**

Open the onnx inference scene

This scene, like the demo record scene, uses only one copy of the level. It also has it's `Sync` node mode set to `Onnx Inference`.

Click on the `Sync` node and set the `Onnx Model Path` property to `model.onnx`.



Press F6 to start the scene and let's see what the agent has learned!

Video of the trained agent:



It seems the agent is capable of collecting the key from both positions (left platform or right platform) and replicates the recorded behavior well. **If you're getting similar results, well done, you've successfully completed this tutorial!** 🏆 👏

If your results are significantly different, note that the amount and quality of recorded demos can affect the results, and adjusting the number of steps for BC/GAIL stages as well as modifying the hyper-parameters in the Python script can potentially help. There's also some run-to-run variation, so sometimes the results can be slightly different even with the same settings.

<> Update on GitHub

← Getting started

(Optional) Customize the environment →

Search models, datasets, users...

Deep RL Course documentation
Train your first Deep Reinforcement Learning Agent

Join the Hugging Face community

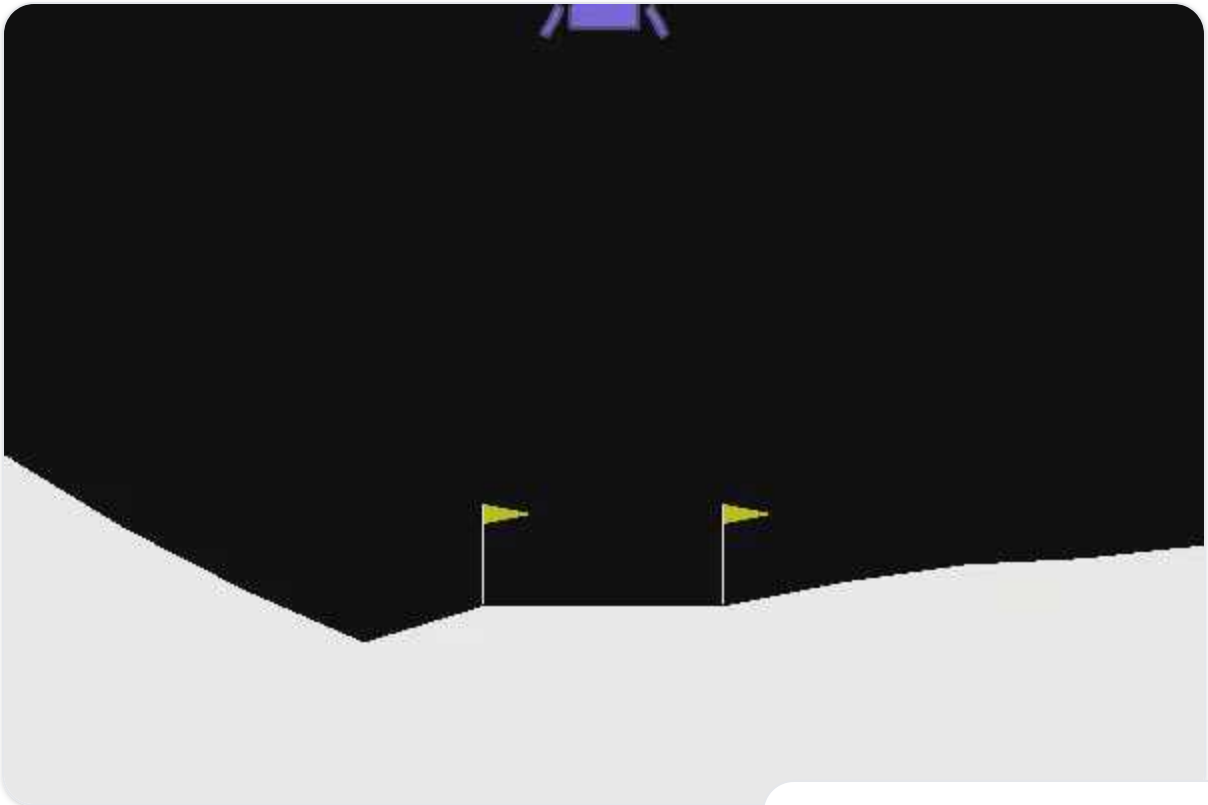
and get access to the augmented documentation experience

Sign Up to get started

Train your first Deep Reinforcement Learning Agent

Ask a question Open in Colab

Now that you've studied the bases of Reinforcement Learning, you're ready to train your first agent and share it with the community through the Hub 🔥 : A Lunar Lander agent that will learn to land correctly on the Moon 🌕



Ask HuggingChat

And finally, you'll **upload this trained agent to the Hugging Face Hub 🤗, a free, open platform where people can share ML models, datasets, and demos.**

Thanks to our leaderboard, you'll be able to compare your results with other classmates and exchange the best practices to improve your agent's scores. Who will win the challenge for Unit 1 🏆 ?

To validate this hands-on for the certification process, you need to push your trained model to the Hub and **get a result of >= 200**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

If you don't find your model, go to the bottom of the page and click on the refresh button.

For more information about the certification process, check this section 👉 https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

And you can check your progress here 👉 https://huggingface.co/spaces/ThomasSimonini/Check-my-progress-Deep-RL-Course

So let's get started! 🚀

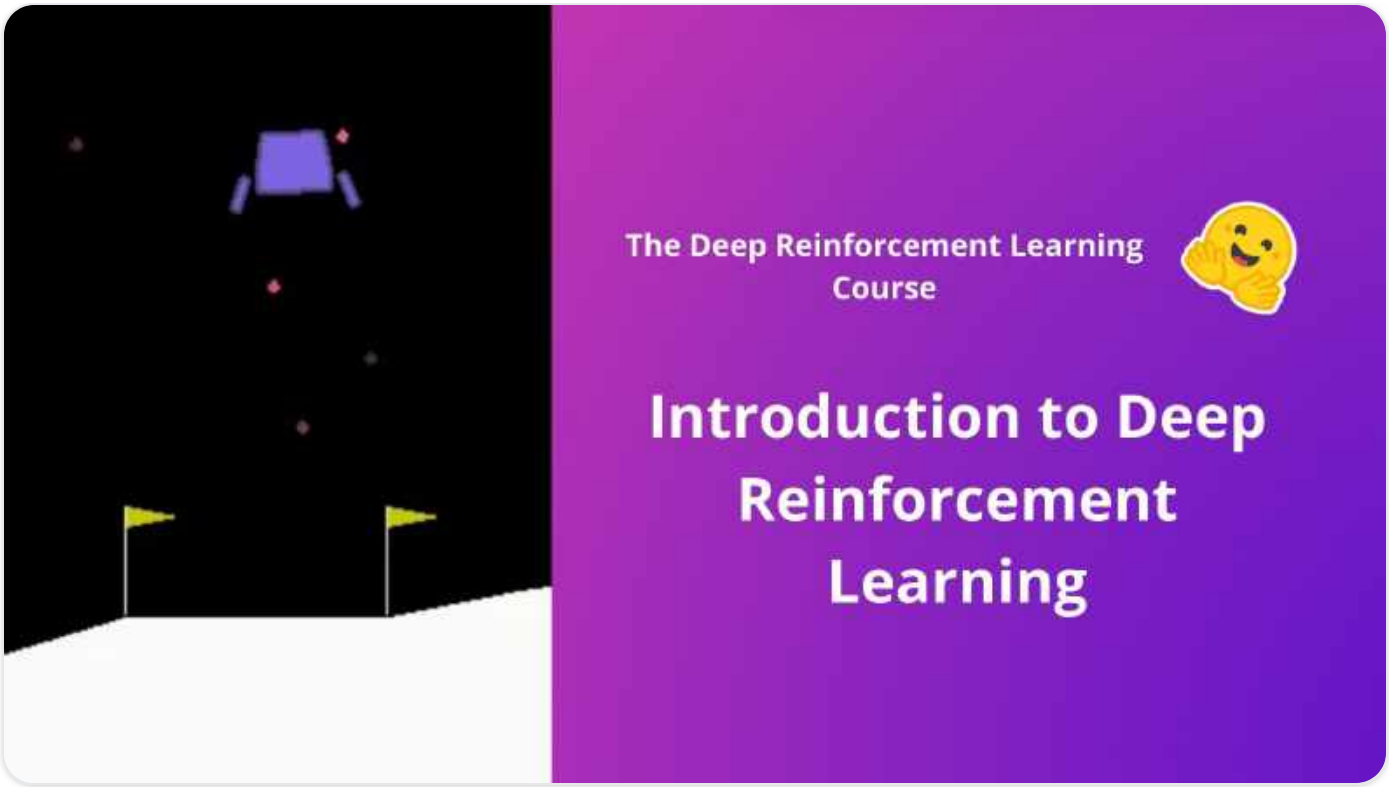
To start the hands-on click on Open In Colab button 👇 :

Open in Colab

We strongly **recommend students use Google Colab for the hands-on exercises** instead of running them on their personal computers.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects** of setting up your environments.

Unit 1: Train your first Deep Reinforcement Learning Agent 🤖



In this notebook, you'll train your **first Deep Reinforcement Learning agent** a Lunar Lander agent that will learn to **land correctly on the Moon** 🌕 . Using Stable-Baselines3 a Deep Reinforcement Learning library, share them with the community, and experiment with different configurations

The environment 🎮

- LunarLander-v2

The library used 📚

- Stable-Baselines3

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the Github Repo.

Objectives of this notebook 🏆

At the end of the notebook, you will:

- Be able to use **Gymnasium**, the environment library.

- Be able to use **Stable-Baselines3**, the deep reinforcement learning library.
- Be able to **push your trained agent to the Hub** with a nice video replay and an evaluation score 🔥.

This notebook is from Deep Reinforcement Learning Course



In this free course, you will:

- 📖 Study Deep Reinforcement Learning in **theory and practice**.
- 👩‍💻 Learn to **use famous Deep RL libraries** such as Stable Baselines3, RL Baselines3 Zoo, CleanRL and Sample Factory 2.0.
- 🤖 Train **agents in unique environments**
- 🎓 **Earn a certificate of completion** by completing 80% of the assignments.

And more!

Check 🗂️ the syllabus 👉 https://simoninithomas.github.io/deep-rl-course

Don't forget to sign up to the course (we are collecting your email to be able to **send you the links when each Unit is published and give you information about the challenges and updates).**

The best way to keep in touch and ask questions is **to join our discord server** to exchange with the community and with us 👉 https://discord.gg/ydHrjt3WP5

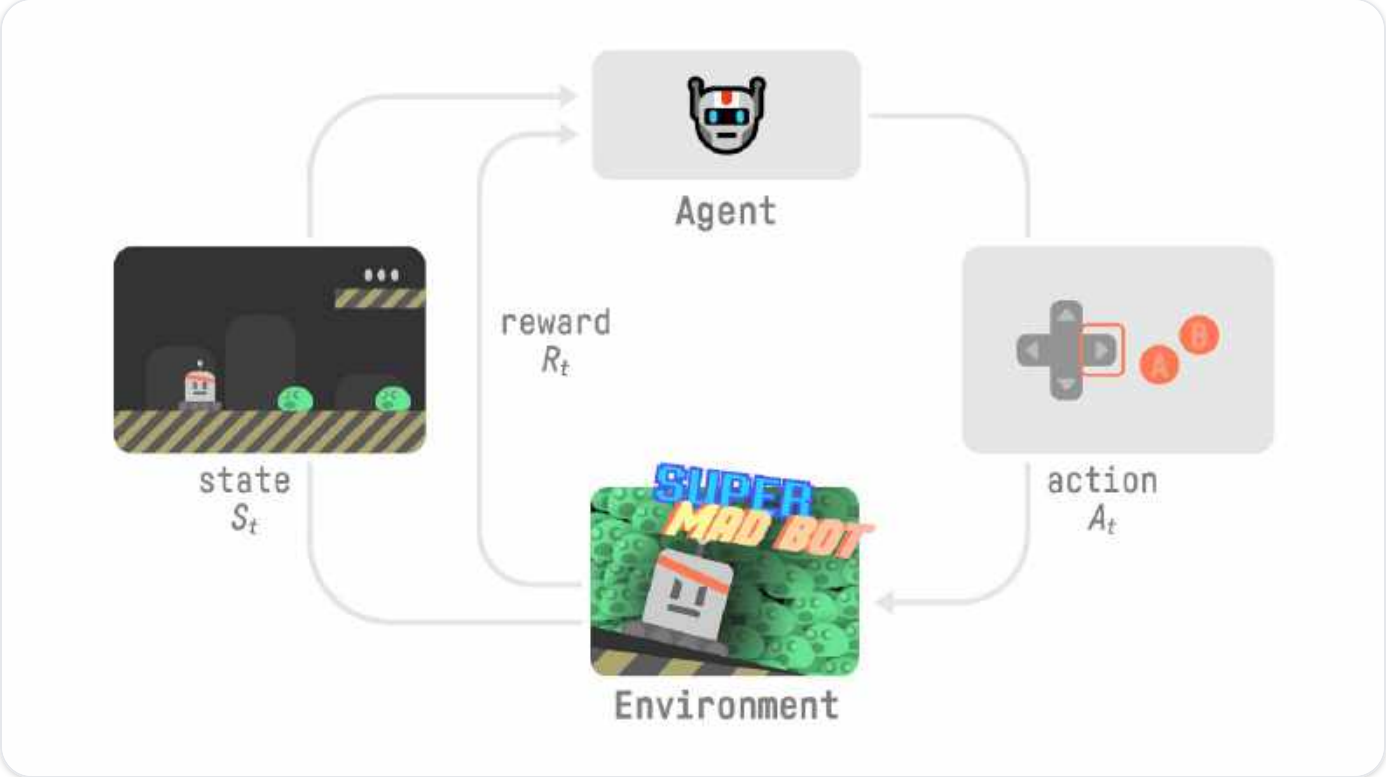
Prerequisites

Before diving into the notebook, you need to:

🔲 📝 Read Unit 0 that gives you all the **information about the course and helps you to onboard** 🤗

🔲 📚 **Develop an understanding of the foundations of Reinforcement learning** (MC, TD, Rewards hypothesis…) by reading Unit 1.

A small recap of Deep Reinforcement Learning 📚



Let's do a small recap on what we learned in the first U

- Reinforcement Learning is a **computational approach to learning from actions**. We build an agent that learns from the environment by **interacting with it through trial and error** and receiving rewards (negative or positive) as feedback.

- The goal of any RL agent is to **maximize its expected cumulative reward** (also called expected return) because RL is based on the *reward hypothesis*, which is that all goals can be described as the maximization of an expected cumulative reward.

- The RL process is a **loop that outputs a sequence of state, action, reward, and next state**.

- To calculate the expected cumulative reward (expected return), **we discount the rewards**: the rewards that come sooner (at the beginning of the game) are more probable to happen since they are more predictable than the long-term future reward.

- To solve an RL problem, you want to **find an optimal policy**; the policy is the "brain" of your AI that will tell us what action to take given a state. The optimal one is the one that gives you the actions that max the expected return.

There are **two** ways to find your optimal policy:

- By **training your policy directly**: policy-based methods.

- By **training a value function** that tells us the expected return the agent will get at each state and use this function to define our policy: value-based methods.

- Finally, we spoke about Deep RL because **we introduce deep neural networks to estimate the action to take (policy-based) or to estimate the value of a state (value-based) hence the name "deep."**

Let's train our first Deep Reinforcement Learning agent and upload it to the Hub 🚀

Get a certificate 🎓

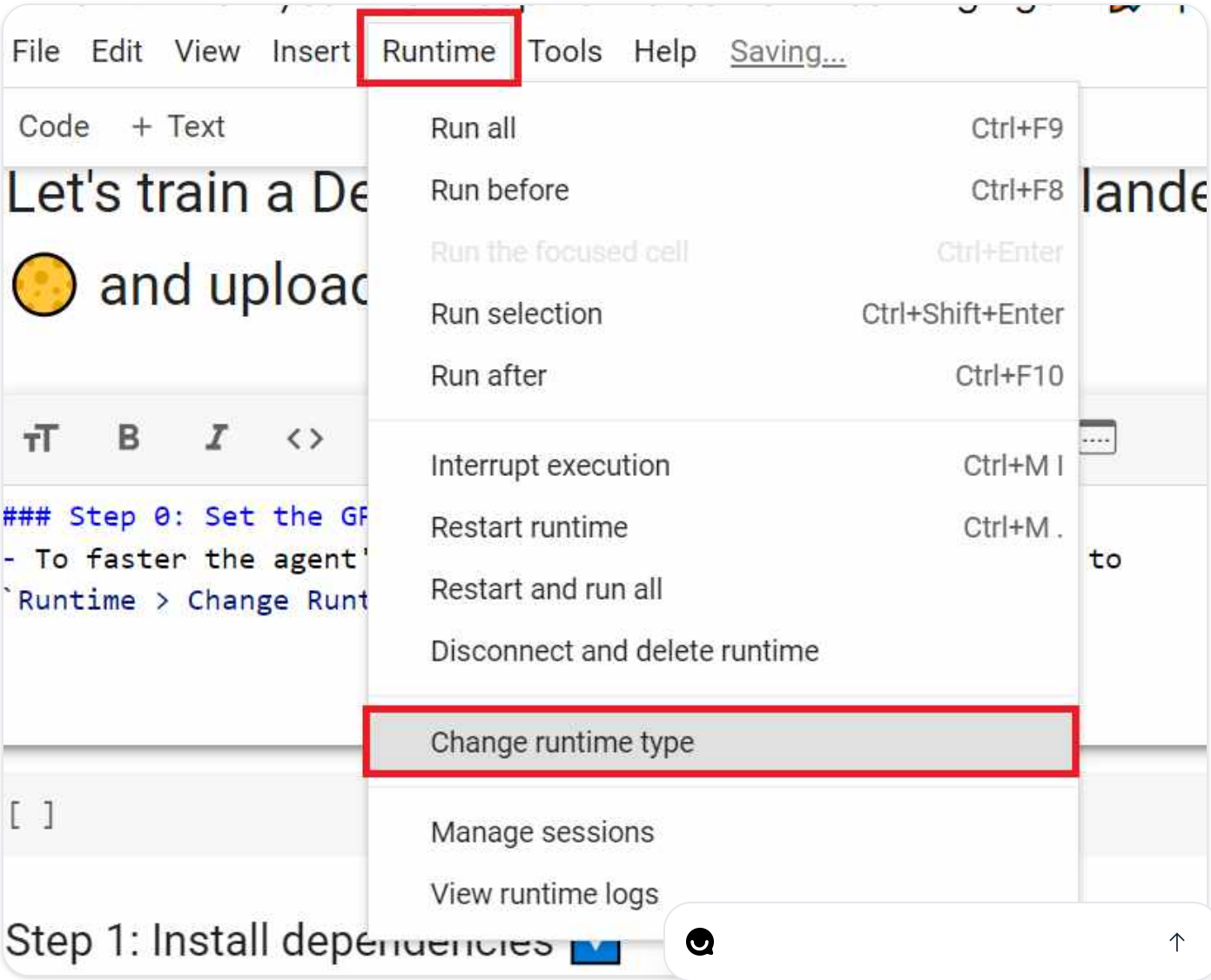
To validate this hands-on for the certification process, you need to push your trained model to the Hub and **get a result of >= 200**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

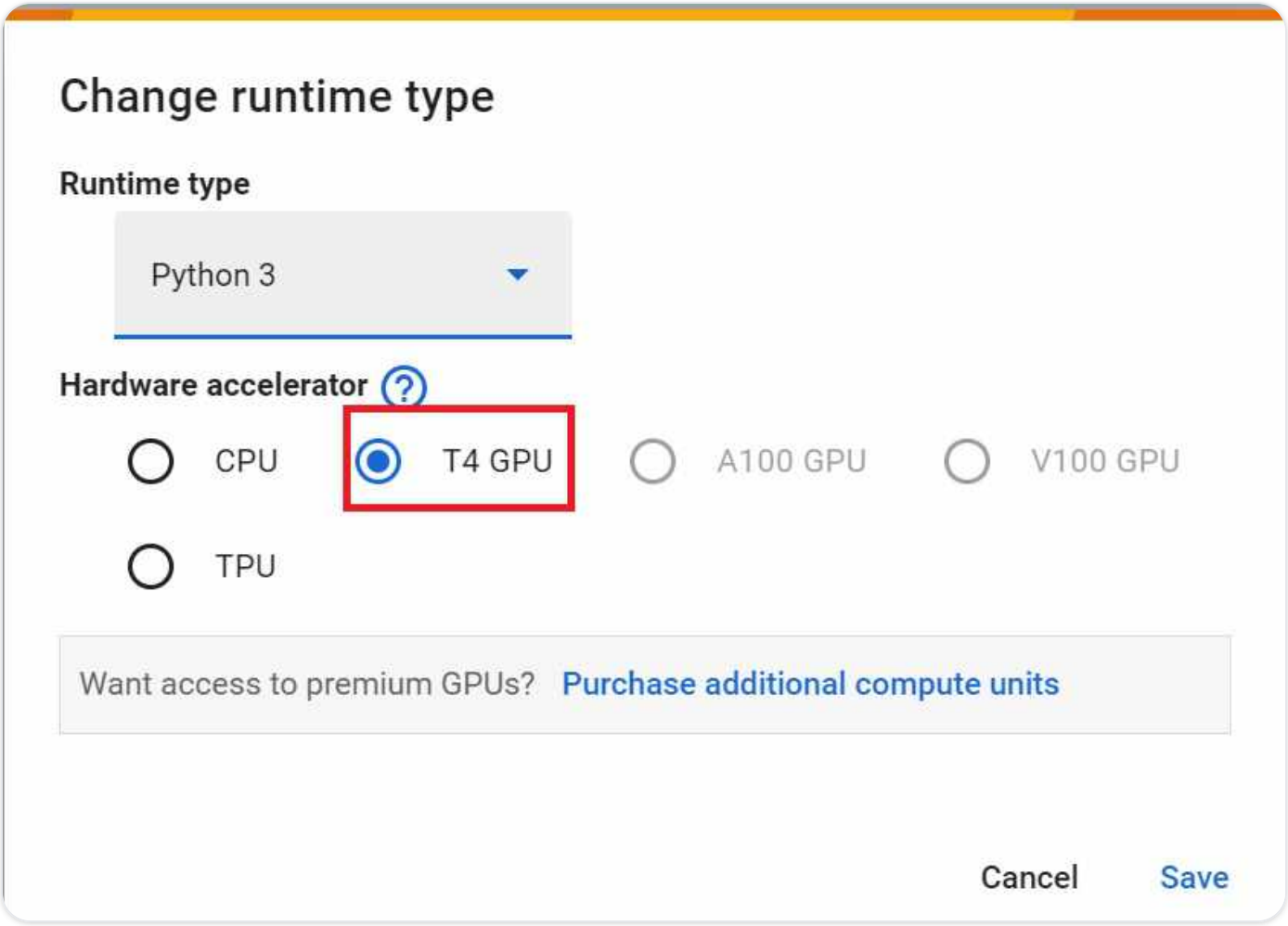
For more information about the certification process, check this section 👉 https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

Set the GPU 💪

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`



- `Hardware Accelerator > GPU`



Install dependencies and create a virtual screen 🔽

The first step is to install the dependencies, we'll install multiple ones.

- `gymnasium[box2d]` : Contains the LunarLander-v2 environment 🌛
- `stable-baselines3[extra]` : The deep reinforcement learning library.
- `huggingface_sb3` : Additional code for Stable-baselines3 to load and upload models from the Hugging Face 🤗 Hub.

To make things easier, we created a script to install all these dependencies.

```
apt install swig cmake
```

```
pip install -r https://raw.githubusercontent.com/huggingface/deep-rl-class/main/notebooks
```

During the notebook, we'll need to generate a replay video. To do so, with colab, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

Hence the following cell will install virtual screen libraries and create and run a virtual screen 🖥️

```
sudo apt-get update
apt install python3-opengl
apt install ffmpeg
apt install xvfb
pip3 install pyvirtualdisplay
```

To make sure the new installed libraries are used, **sometimes it's required to restart the notebook runtime**. The next cell will force the **runtime to crash, so you'll need to connect again and run the code starting from here**. Thanks to this trick, **we will be able to run our virtual screen.**

```
import os

os.kill(os.getpid(), 9)
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Import the packages 📦

One additional library we import is huggingface_hub **to be able to upload and download trained models from the hub**.

The Hugging Face Hub 🤗 works as a central place whe
and datasets. It has versioning, metrics, visualizations a

easily collaborate with others.

You can see here all the Deep reinforcement Learning models available here 👉 https://huggingface.co/models?pipeline_tag=reinforcement-learning&sort=downloads

```
import gymnasium

from huggingface_sb3 import load_from_hub, package_to_hub
from huggingface_hub import (
    notebook_login,
)  # To log to our Hugging Face account to be able to upload models to the Hub.

from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor
```

Understand Gymnasium and how it works 🤖

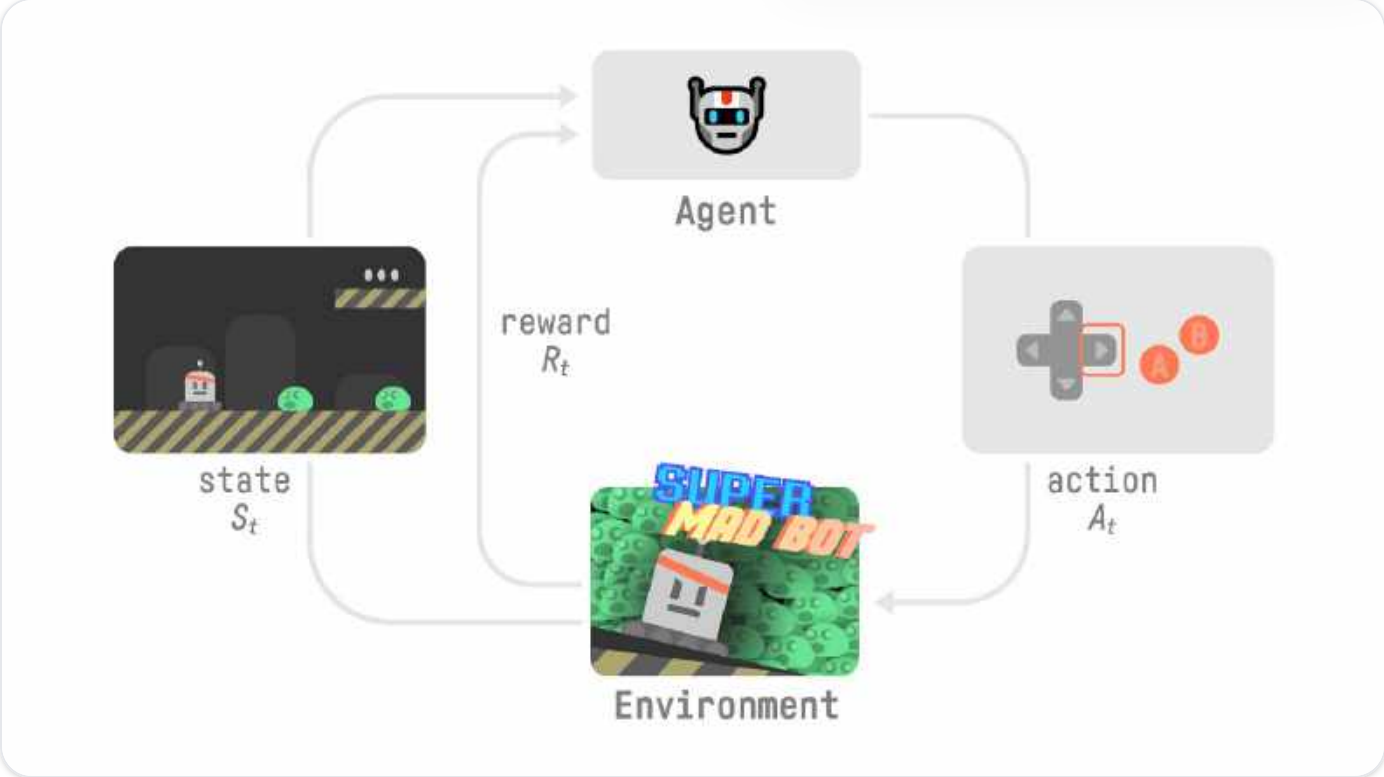
🏋 The library containing our environment is called Gymnasium. **You'll use Gymnasium a lot in Deep Reinforcement Learning.**

Gymnasium is the **new version of Gym library** maintained by the Farama Foundation.

The Gymnasium library provides two things:

- An interface that allows you to **create RL environments**.
- A **collection of environments** (gym-control, atari, box2D…).

Let's look at an example, but first let's recall the RL loop.



At each step:

- Our Agent receives a **state (S0)** from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state (S0),** the Agent takes an **action (A0)** — our Agent will move to the right.
- The environment transitions to a **new state (S1)** — new frame.
- The environment gives some **reward (R1)** to the Agent — we're not dead *(Positive Reward +1)*.

With Gymnasium:

1️⃣ We create our environment using `gymnasium.make()`

2️⃣ We reset the environment to its initial state with `observation = env.reset()`

At each step:

3️⃣ Get an action using our model (in our example we take a random action)

4️⃣ Using `env.step(action)`, we perform this action i

- `observation`: The new state (st+1)
- `reward`: The reward we get after executing the action
- `terminated`: Indicates if the episode terminated (agent reach the terminal state)
- `truncated`: Introduced with this new version, it indicates a timelimit or if an agent go out of bounds of the environment for instance.
- `info`: A dictionary that provides additional information (depends on the environment).

For more explanations check this 👉
https://gymnasium.farama.org/api/env/#gymnasium.Env.step

If the episode is terminated:

- We reset the environment to its initial state with `observation = env.reset()`

Let's look at an example! Make sure to read the code

```
import gymnasium as gym

# First, we create our environment called LunarLander-v2
env = gym.make("LunarLander-v2")

# Then we reset this environment
observation, info = env.reset()

for _ in range(20):
    # Take a random action
    action = env.action_space.sample()
    print("Action taken:", action)

    # Do this action in the environment and get
    # next_state, reward, terminated, truncated and info
    observation, reward, terminated, truncated, info = env.step(action)

    # If the game is terminated (in our case we land, crashed) or truncated (timeout)
    if terminated or truncated:
        # Reset the environment
        print("Environment is reset")
        observation, info = env.reset()
```

```
env.close()
```

Create the LunarLander environment 🌛 and understand how it works

The environment 🎮

In this first tutorial, we're going to train our agent, a Lunar Lander, **to land correctly on the moon**. To do that, the agent needs to learn **to adapt its speed and position (horizontal, vertical, and angular) to land correctly.**

💡 A good habit when you start to use an environment is to check its documentation

👉 https://gymnasium.farama.org/environments/box2d/lunar_lander/

Let's see what the Environment looks like:

```
# We create our environment with gym.make("<name_of_the_environment>")
env = gym.make("LunarLander-v2")
env.reset()
print("_____OBSERVATION SPACE_____ \n")
print("Observation Space Shape", env.observation_space.shape)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

We see with `Observation Space Shape (8,)` that the observation is a vector of size 8, where each value contains different information about the lander:

- Horizontal pad coordinate (x)
- Vertical pad coordinate (y)

- Horizontal speed (x)
- Vertical speed (y)
- Angle
- Angular speed
- If the left leg contact point has touched the land (boolean)
- If the right leg contact point has touched the land (boolean)

```
print("\n _____ACTION SPACE_____ \n")
print("Action Space Shape", env.action_space.n)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

The action space (the set of possible actions the agent can take) is discrete with 4 actions available 🎮:

- Action 0: Do nothing,
- Action 1: Fire left orientation engine,
- Action 2: Fire the main engine,
- Action 3: Fire right orientation engine.

Reward function (the function that will give a reward at each timestep) 💰:

After every step a reward is granted. The total reward of an episode is the **sum of the rewards for all the steps within that episode**.

For each step, the reward:

- Is increased/decreased the closer/further the lander is to the landing pad.
- Is increased/decreased the slower/faster the lander is moving.
- Is decreased the more the lander is tilted (angle not horizontal).
- Is increased by 10 points for each leg that is in contact with the ground.
- Is decreased by 0.03 points each frame a side engi

- Is decreased by 0.3 points each frame the main engine is firing.

The episode receive an **additional reward of -100 or +100 points for crashing or landing safely respectively.**

An episode is **considered a solution if it scores at least 200 points.**

Vectorized Environment

- We create a vectorized environment (a method for stacking multiple independent environments into a single environment) of 16 environments, this way, **we'll have more diverse experiences during the training.**

```
# Create the environment
env = make_vec_env("LunarLander-v2", n_envs=16)
```

Create the Model 🤖

- We have studied our environment and we understood the problem: **being able to land the Lunar Lander to the Landing Pad correctly by controlling left, right and main orientation engine**. Now let's build the algorithm we're going to use to solve this Problem 🚀.
- To do so, we're going to use our first Deep RL library, Stable Baselines3 (SB3).
- SB3 is a set of **reliable implementations of reinforcement learning algorithms in PyTorch**.

💡 A good habit when using a new library is to dive first on the documentation: https://stable-baselines3.readthedocs.io/en/master/ and then try some tutorials.



To solve this problem, we're going to use SB3 **PPO**. PPO (aka Proximal Policy Optimization) is one of the SOTA (state of the art) Deep Reinforcement Learning algorithms that you'll study during this course.

PPO is a combination of:

- *Value-based reinforcement learning method*: learning an action-value function that will tell us the **most valuable action to take given a state and action**.
- *Policy-based reinforcement learning method*: learning a policy that will **give us a probability distribution over actions**.

Stable-Baselines3 is easy to set up:

1️⃣ You **create your environment** (in our case it was do

2️⃣ You define the **model you want to use and instantiate this model** `model = PPO("MlpPolicy")`

3️⃣ You **train the agent** with `model.learn` and define the number of training timesteps

```
# Create environment
env = gym.make('LunarLander-v2')

# Instantiate the agent
model = PPO('MlpPolicy', env, verbose=1)
# Train the agent
model.learn(total_timesteps=int(2e5))
```

```
# TODO: Define a PPO MlpPolicy architecture
# We use MultiLayerPerceptron (MLPPolicy) because the input is a vector,
# if we had frames as input we would use CnnPolicy
model =
```

Solution

```
# SOLUTION
# We added some parameters to accelerate the training
model = PPO(
    policy="MlpPolicy",
    env=env,
    n_steps=1024,
    batch_size=64,
    n_epochs=4,
    gamma=0.999,
    gae_lambda=0.98,
    ent_coef=0.01,
    verbose=1,
)
```

Train the PPO agent 🏃

- Let's train our agent for 1,000,000 timesteps, don't forget to use GPU on Colab. It will take approximately ~20min, but you can use fewer time

- During the training, take a ☕ break you deserved it 🤗

```
# TODO: Train it for 1,000,000 timesteps

# TODO: Specify file name for model and save the model to file
model_name = "ppo-LunarLander-v2"
```

Solution

```
# SOLUTION
# Train it for 1,000,000 timesteps
model.learn(total_timesteps=1000000)
# Save the model
model_name = "ppo-LunarLander-v2"
model.save(model_name)
```

Evaluate the agent 📈

- Remember to wrap the environment in a Monitor.
- Now that our Lunar Lander agent is trained 🚀, we need to **check its performance**.
- Stable-Baselines3 provides a method to do that: `evaluate_policy`.
- To fill that part you need to check the documentation
- In the next step, we'll see **how to automatically evaluate and share your agent to compete in a leaderboard, but for now let's do it ourselves**

💡 When you evaluate your agent, you should not use your training environment but create an evaluation environment.

```
# TODO: Evaluate the agent
# Create a new environment for evaluation
eval_env =

# Evaluate the model with 10 evaluation episodes and deterministic=True
mean_reward, std_reward =
```

```
# Print the results
```

Solution

```
# @title
eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10, determinis
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

- In my case, I got a mean reward of `200.20 +/- 20.80` after training for 1 million steps, which means that our lunar lander agent is ready to land on the moon 🤌🥳.

Publish our trained model on the Hub 🔥

Now that we saw we got good results after the training, we can publish our trained model on the hub 🤗 with one line of code.

📚 The libraries documentation 👉 https://github.com/huggingface/huggingface_sb3/tree/main#hugging-face—x-stable-baselines3-v20

Here's an example of a Model Card (with Space Invaders):

By using `package_to_hub` **you evaluate, record a replay, generate a model card of your agent and push it to the hub**.

This way:

- You can **showcase our work** 🔥
- You can **visualize your agent playing** 👀
- You can **share with the community an agent that others can use** 💾
- You can **access a leaderboard** 🏆 **to see how well**
  your classmates 👉 https://huggingface.co/spaces/huggingface-projects/deep-

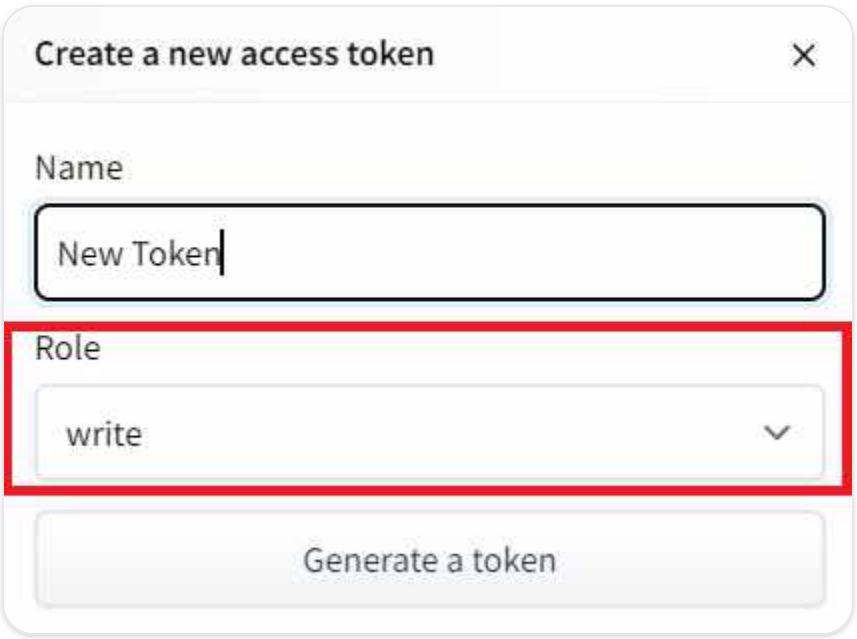
Reinforcement-Learning-Leaderboard

To be able to share your model with the community there are three more steps to follow:

1️⃣ (If it's not already done) create an account on Hugging Face ➡️ https://huggingface.co/join

2️⃣ Sign in and then, you need to store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**



- Copy the token
- Run the cell below and paste the token

```
notebook_login()
!git config --global credential.helper store
```

If you don't want to use a Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

3️⃣ We're now ready to push our trained agent to the 🤗 Hub 🔥 using `package_to_hub()` function

Let's fill the `package_to_hub` function:

- `model`: our trained model.
- `model_name`: the name of the trained model that we defined in `model_save`
- `model_architecture`: the model architecture we used, in our case PPO
- `env_id`: the name of the environment, in our case `LunarLander-v2`
- `eval_env`: the evaluation environment defined in eval_env
- `repo_id`: the name of the Hugging Face Hub Repository that will be created/updated `(repo_id = {username}/{repo_name})`

💡 **A good name is** `{username}/{model_architecture}-{env_id}`

- `commit_message`: message of the commit

```
import gymnasium as gym
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.env_util import make_vec_env

from huggingface_sb3 import package_to_hub

## TODO: Define a repo_id
## repo_id is the id of the model repository from the Hugging Face Hub (repo_id = {organi
repo_id =

# TODO: Define the name of the environment
env_id =

# Create the evaluation env and set the render_mode="rgb_array"
eval_env = DummyVecEnv([lambda: gym.make(env_id, render_mode="rgb_array")])


# TODO: Define the model architecture we used
model_architecture = ""

## TODO: Define the commit message
commit_message = ""

# method save, evaluate, generate a model card and record a replay video of your agent be
```

```
package_to_hub(model=model, # Our trained model
               model_name=model_name, # The name of our trained model
               model_architecture=model_architecture, # The model architecture we used: i
               env_id=env_id, # Name of the environment
               eval_env=eval_env, # Evaluation Environment
               repo_id=repo_id, # id of the model repository from the Hugging Face Hub (i
               commit_message=commit_message)
```

Solution

```
import gymnasium as gym

from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.env_util import make_vec_env

from huggingface_sb3 import package_to_hub

# PLACE the variables you've just defined two cells above
# Define the name of the environment
env_id = "LunarLander-v2"

# TODO: Define the model architecture we used
model_architecture = "PPO"

## Define a repo_id
## repo_id is the id of the model repository from the Hugging Face Hub (repo_id = {organi
## CHANGE WITH YOUR REPO ID
repo_id = "ThomasSimonini/ppo-LunarLander-v2"  # Change with your repo id, you can't push

## Define the commit message
commit_message = "Upload PPO LunarLander-v2 trained agent"

# Create the evaluation env and set the render_mode="rgb_array"
eval_env = DummyVecEnv([lambda: Monitor(gym.make(env_id, render_mode="rgb_array"))])

# PLACE the package_to_hub function you've just filled here
package_to_hub(
    model=model,  # Our trained model
    model_name=model_name,  # The name of our tr
    model_architecture=model_architecture,  # The
```

```
    env_id=env_id,  # Name of the environment
    eval_env=eval_env,  # Evaluation Environment
    repo_id=repo_id,  # id of the model repository from the Hugging Face Hub (repo_id = {
    commit_message=commit_message,
)
```

Congrats 🥳 you've just trained and uploaded your first Deep Reinforcement Learning agent. The script above should have displayed a link to a model repository such as https://huggingface.co/osanseviero/test_sb3. When you go to this link, you can:

- See a video preview of your agent at the right.
- Click "Files and versions" to see all the files in the repository.
- Click "Use in stable-baselines3" to get a code snippet that shows how to load the model.
- A model card (`README.md` file) which gives a description of the model

Under the hood, the Hub uses git-based repositories (don't worry if you don't know what git is), which means you can update the model with new versions as you experiment and improve your agent.

Compare the results of your LunarLander-v2 with your classmates using the leaderboard 🏆 👉 https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard

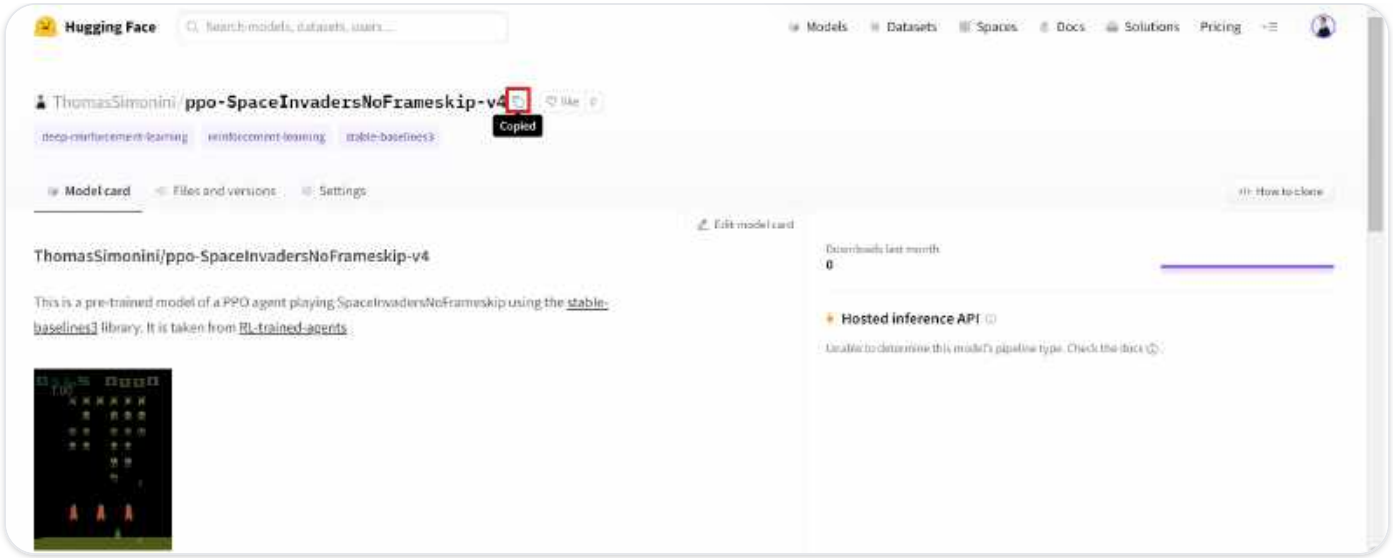
Load a saved LunarLander model from the Hub 🤗

Thanks to ironbar for the contribution.

Loading a saved model from the Hub is really easy.

You go to https://huggingface.co/models?library=stable-baselines3 to see the list of all the Stable-baselines3 saved models

1. You select one and copy its repo_id



2. Then we just need to use load_from_hub with:

- The repo_id
- The filename: the saved model inside the repo and its extension (*.zip)

Because the model I download from the Hub was trained with Gym (the former version of Gymnasium) we need to install shimmy a API conversion tool that will help us to run the environment correctly.

Shimmy Documentation: https://github.com/Farama-Foundation/Shimmy

```
!pip install shimmy
```

```
from huggingface_sb3 import load_from_hub

repo_id = "Classroom-workshop/assignment2-omar"  # The repo_id
filename = "ppo-LunarLander-v2.zip"  # The model filename.zip

# When the model was trained on Python 3.8 the pickle protocol is 5
# But Python 3.6, 3.7 use protocol 4
# In order to get compatibility we need to:
# 1. Install pickle5 (we done it at the beginning of the colab)
# 2. Create a custom empty object we pass as parameter to PPO.load()
custom_objects = {
    "learning_rate": 0.0,
```

```
    "lr_schedule": lambda _: 0.0,
    "clip_range": lambda _: 0.0,
}

checkpoint = load_from_hub(repo_id, filename)
model = PPO.load(checkpoint, custom_objects=custom_objects, print_system_info=True)
```

Let's evaluate this agent:

```
# @title
eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10, determinis
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

Some additional challenges 🏆

The best way to learn **is to try things by your own**! As you saw, the current agent is not doing great. As a first suggestion, you can train for more steps. With 1,000,000 steps, we saw some great results!

In the Leaderboard you will find your agents. Can you get to the top?

Here are some ideas to achieve so:

- Train more steps
- Try different hyperparameters for `PPO`. You can see them at https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html#parameters.
- Check the Stable-Baselines3 documentation and try another model such as DQN.
- **Push your new trained model** on the Hub 🔥

Compare the results of your LunarLander-v2 with your classmates using the leaderboard 🏆

Is moon landing too boring for you? Try to **change the environment**, why not use MountainCar-v0, CartPole-v1 or CarRacing-v0? Check how they work using the gym documentation and have fun 🎉.

Congrats on finishing this chapter! That was the biggest one, **and there was a lot of information.**

If you're still feel confused with all these elements…it's totally normal! **This was the same for me and for all people who studied RL.**

Take time to really **grasp the material before continuing and try the additional challenges**. It's important to master these elements and have a solid foundations.

Naturally, during the course, we're going to dive deeper into these concepts but **it's better to have a good understanding of them now before diving into the next chapters.**

Next time, in the bonus unit 1, you'll train Huggy the Dog to fetch the stick.



Keep learning, stay awesome 🤗

<> Update on GitHub

← Glossary Quiz →

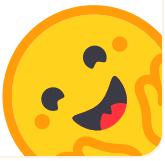
Search models, datasets, users...

Deep RL Course documentation
Two main approaches for solving RL problems

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



Two main approaches for solving RL problems

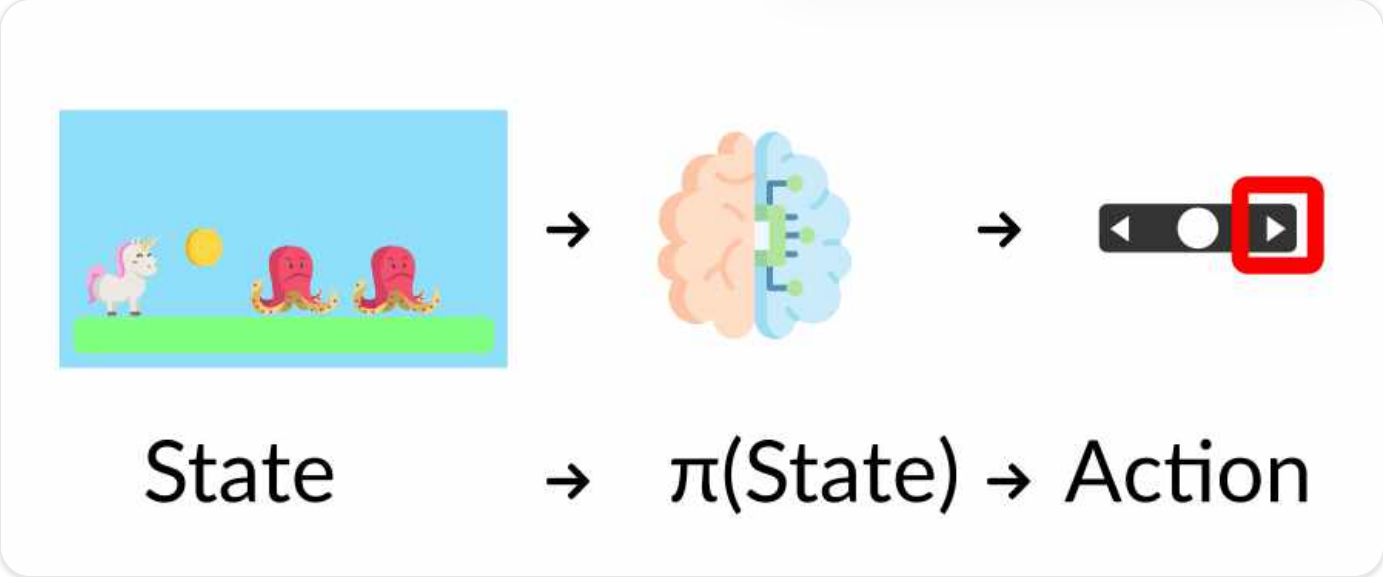
> Now that we learned the RL framework, how do we solve the RL problem?

In other words, how do we build an RL agent that can **select the actions that maximize its expected cumulative reward?**

The Policy π: the agent's brain

The Policy **π** is the **brain of our Agent**, it's the function that tells us what **action to take given the state we are in.** So it **defines the agent's behavior** at a given time.

Ask HuggingChat



Think of policy as the brain of our agent, the function that will tell us the action to take given a state

This Policy **is the function we want to learn**, our goal is to find the optimal policy π*, the policy that **maximizes expected return** when the agent acts according to it. We find this π* **through training.**

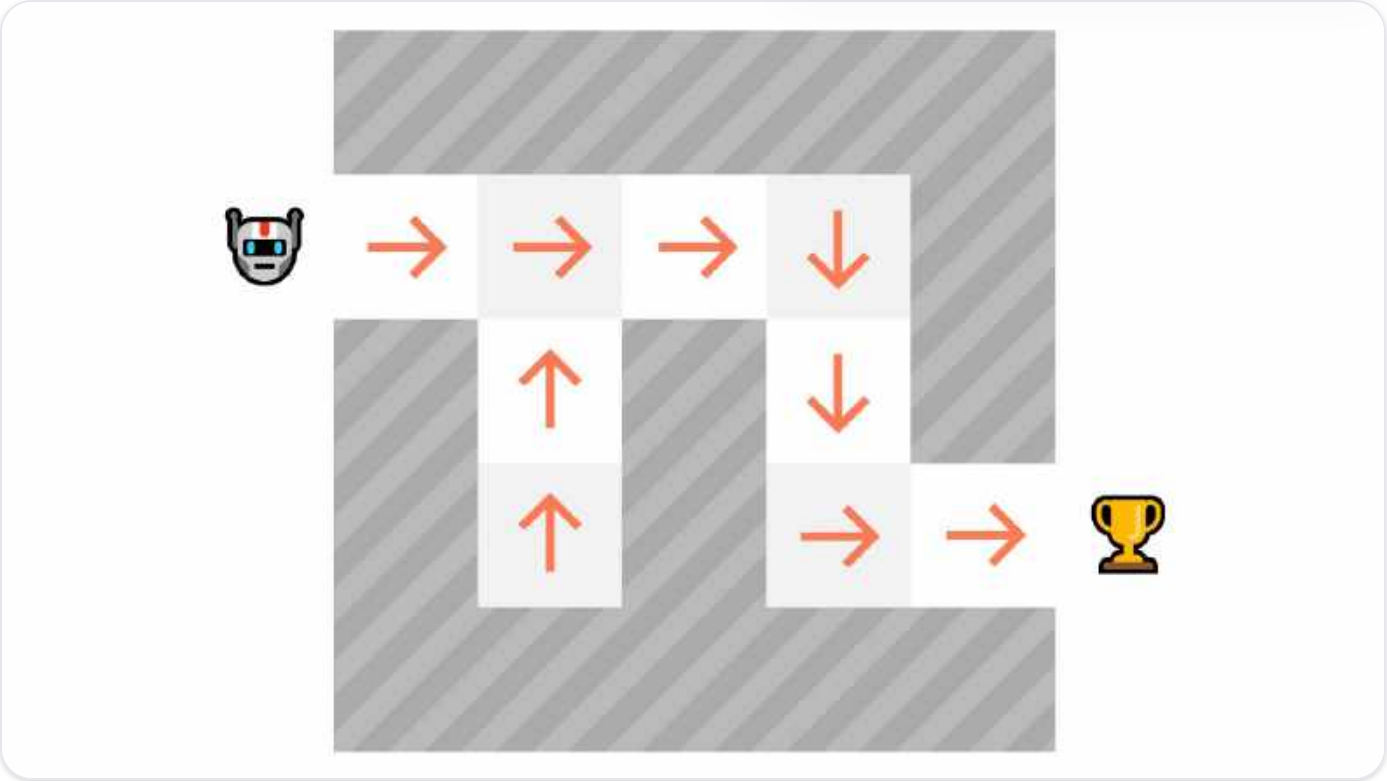
There are two approaches to train our agent to find this optimal policy π*:

- **Directly,** by teaching the agent to learn which **action to take,** given the current state: **Policy-Based Methods.**
- Indirectly, **teach the agent to learn which state is more valuable** and then take the action that **leads to the more valuable states**: Value-Based Methods.

Policy-Based Methods

In Policy-Based methods, **we learn a policy function directly.**

This function will define a mapping from each state to the best corresponding action. Alternatively, it could define **a probability distribution over the set of possible actions at that state.**



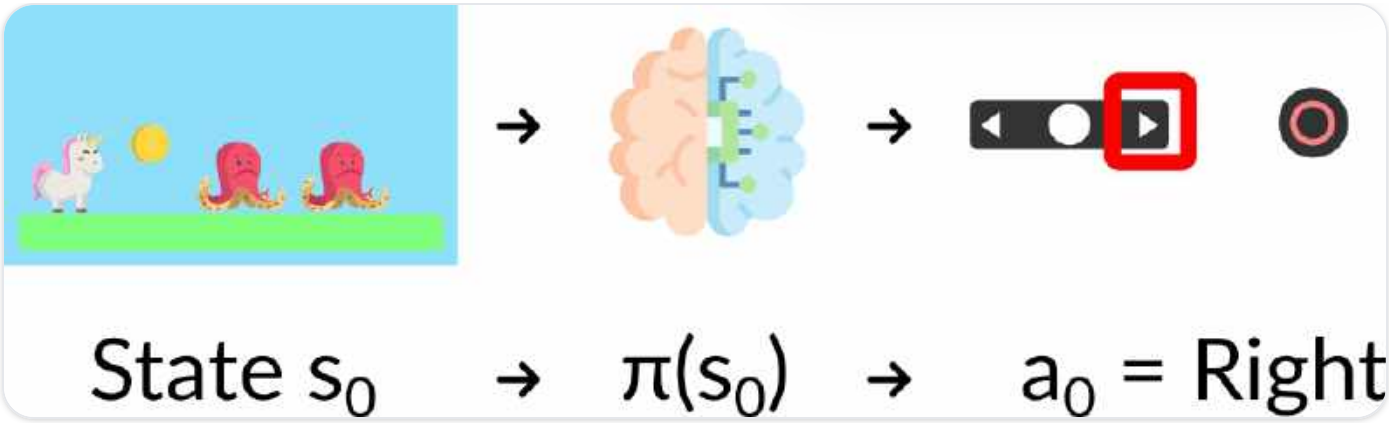
As we can see here, the policy (deterministic) **directly indicates the action to take for each step.**

We have two types of policies:

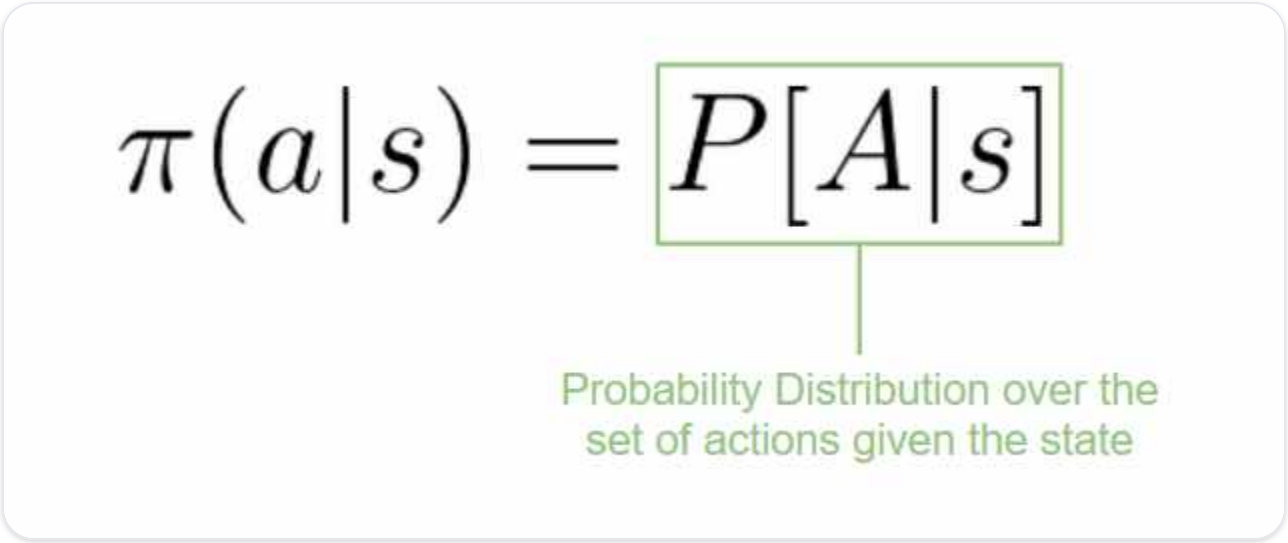
- *Deterministic*: a policy at a given state **will always return the same action.**

$$a = \pi(s)$$

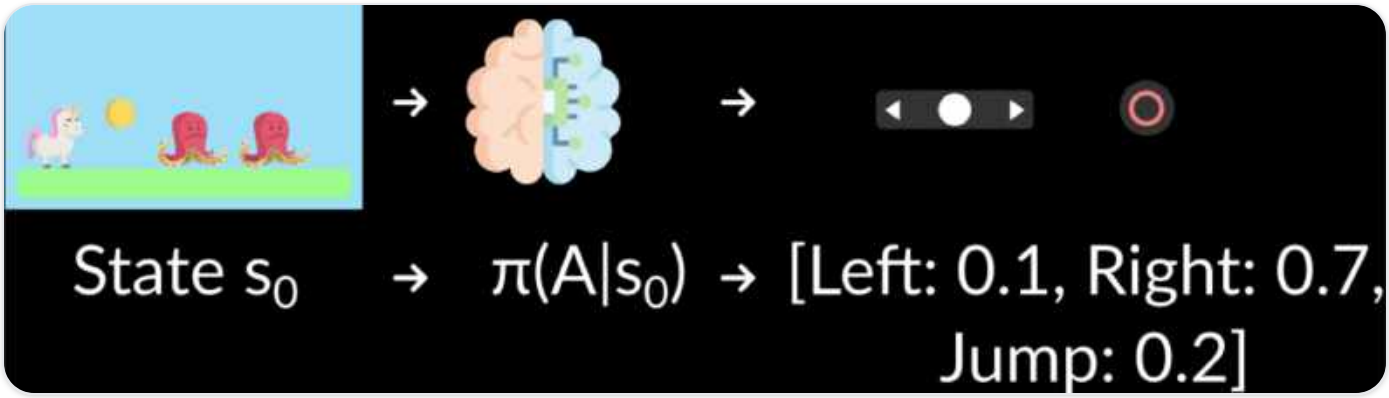
action = policy(state)



- *Stochastic*: outputs **a probability distribution over actions.**

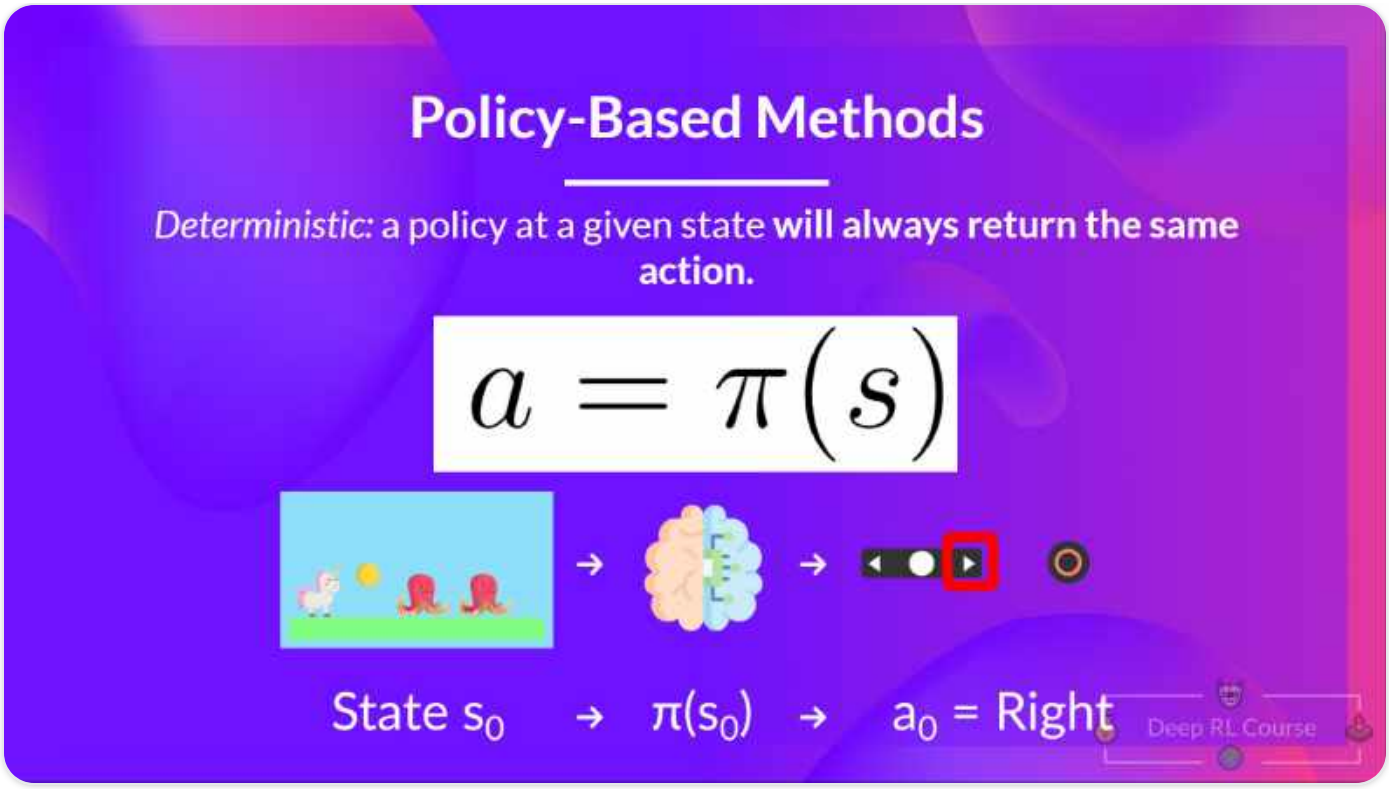


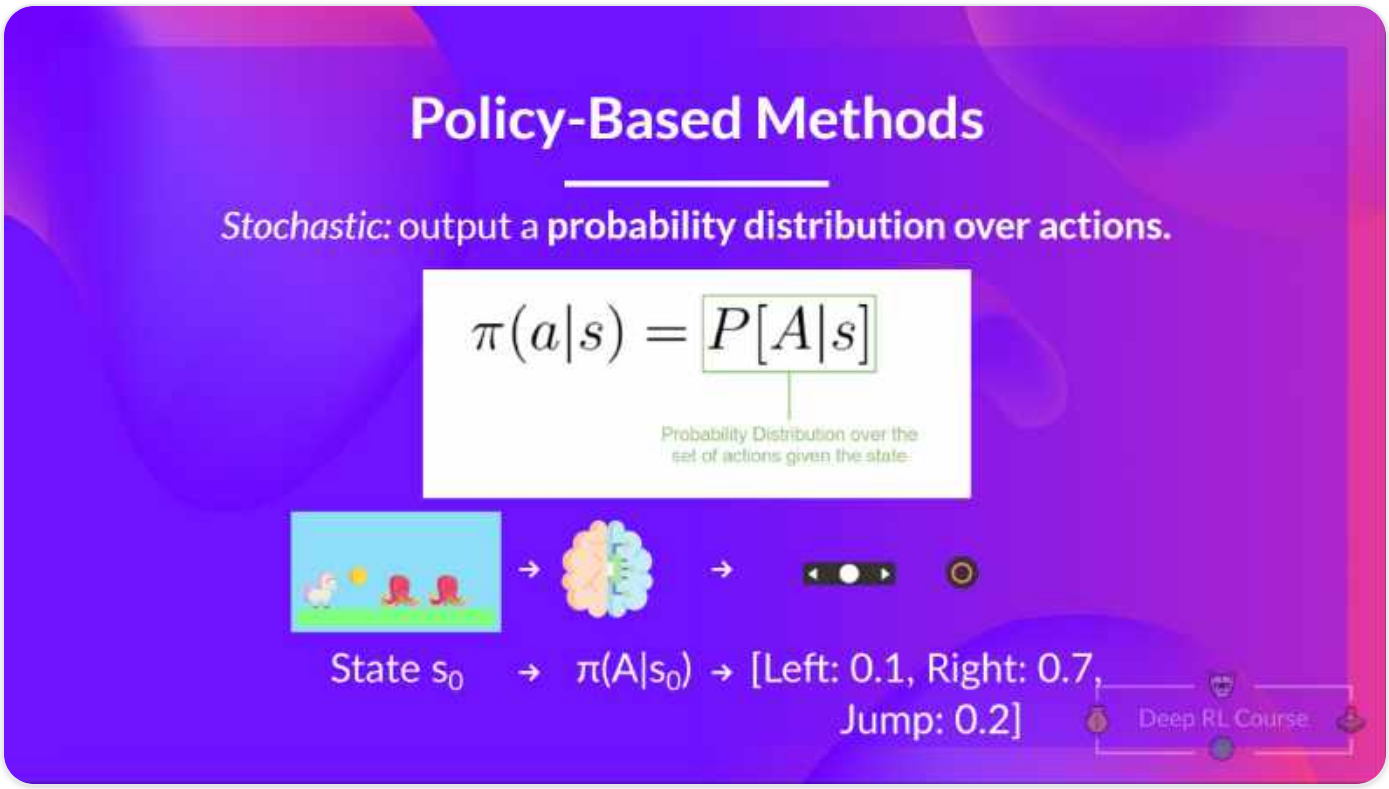
policy(actions | state) = probability distribution over the set of actions given the current state



Given an initial state, our stochastic policy will output probability distributions over the possible actions at that state.

If we recap:





Value-based methods

In value-based methods, instead of learning a policy fun[illegible] we learn a value function that maps a state to the expected value **of being at that state.**

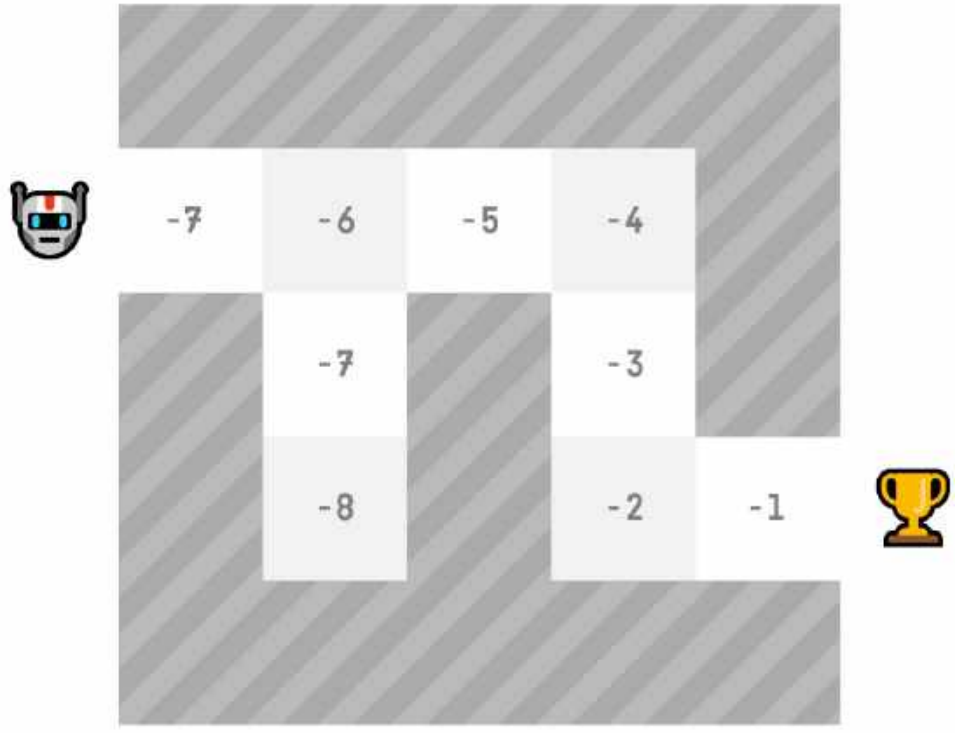
The value of a state is the **expected discounted return** the agent can get if it **starts in that state, and then acts according to our policy.**

"Act according to our policy" just means that our policy is **"going to the state with the highest value".**

$$v_\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s\right]$$

Value function — Expected discounted return — Starting at state s

Here we see that our value function **defined values for each possible state.**



Thanks to our value function, at each step our policy will select the state with the biggest value defined by the value function: -7, then -6, then -5 (and so on) to attain the goal.

Thanks to our value function, at each step our policy w
defined by the value function: -7, then -6, then -5 (and so on) to attain the goal.

If we recap:

Value-Based Methods

The value of a state is the **expected discounted return** the agent can get if it **starts in that state, and then act according to our policy.**

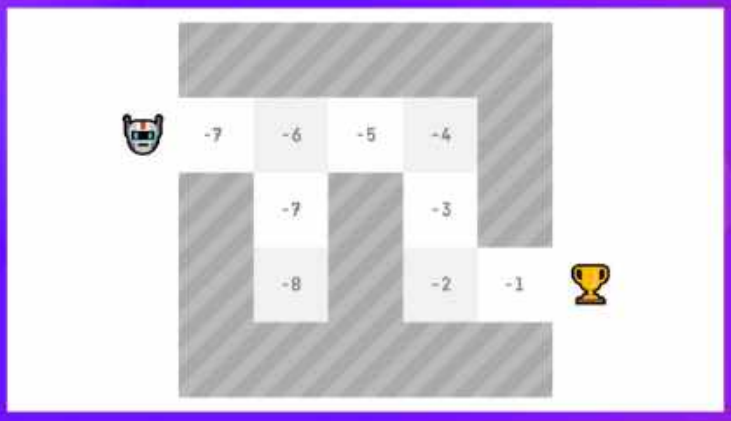
$$v_\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s\right]$$

Value function — Expected discounted return — Starting at state s

"Act according to our policy" just means that our policy is **"going to the state with the highest value".**

Deep RL Course

Value-Based Methods



Thanks to our value function, at each step our policy will select the state with the **biggest value defined by the value function:** -7, then -6, then -5 (and so on) to attain the goal.

Deep RL Course

<> Update on GitHub

← The Exploration/ Exploitation tradeoff

The “Deep” in Deep Reinforcement Learning →

Search models, datasets, users...

Deep RL Course documentation
Two types of value-based methods

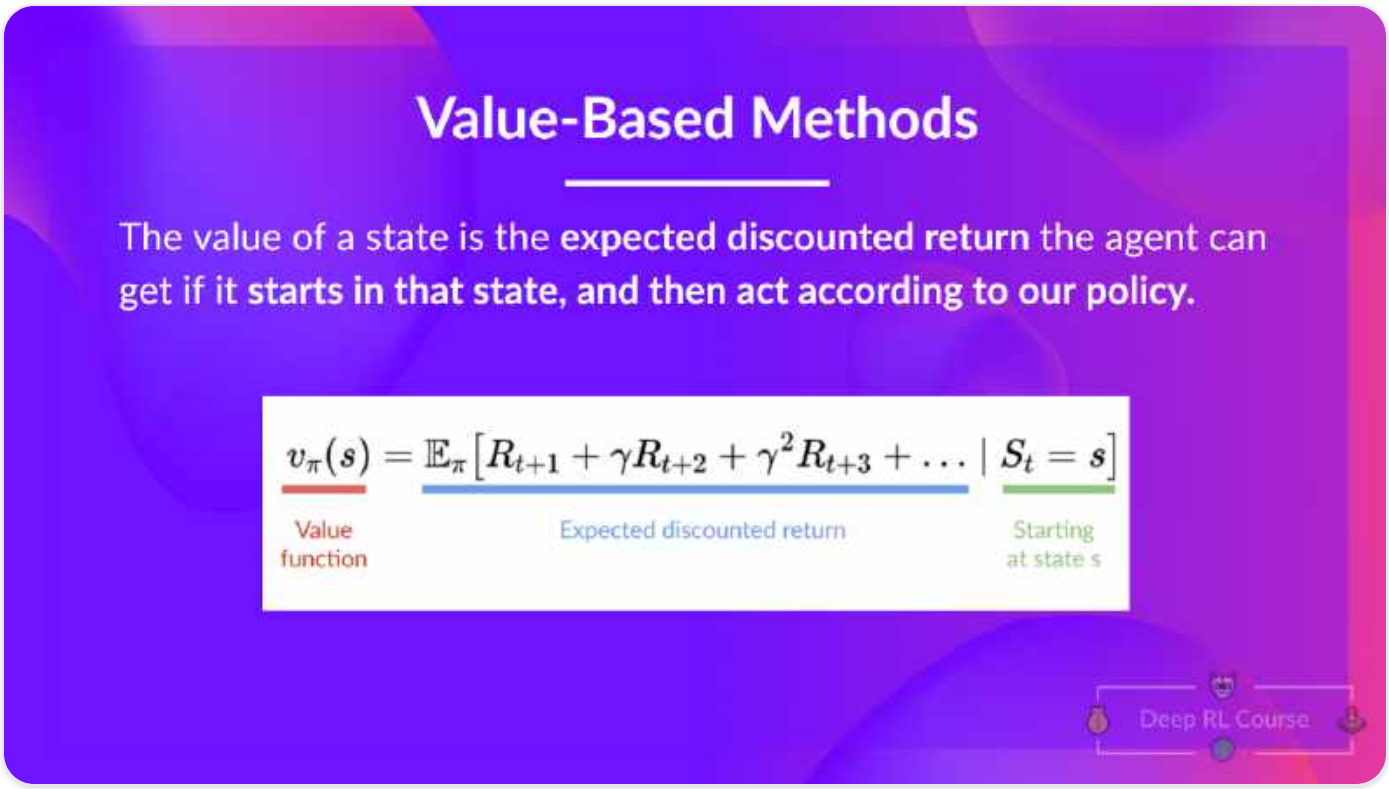
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

Two types of value-based methods

In value-based methods, **we learn a value function** that **maps a state to the expected value of being at that state.**



Ask HuggingChat

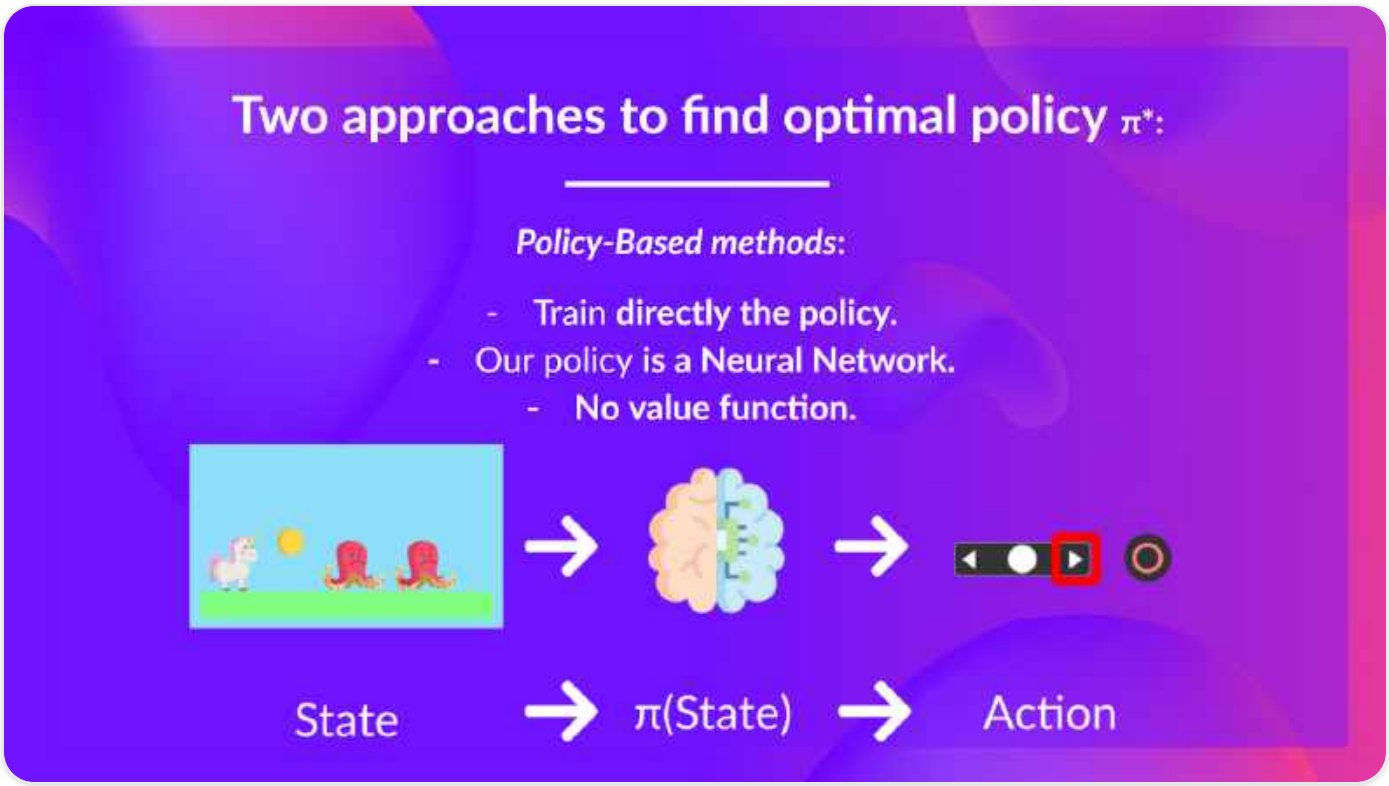
The value of a state is the **expected discounted return** the agent can get if it **starts at that state and then acts according to our policy.**

> But what does it mean to act according to our policy? After all, we don't have a policy in value-based methods since we train a value function and not a policy.

Remember that the goal of an **RL agent is to have an optimal policy π*.**

To find the optimal policy, we learned about two different methods:

- *Policy-based methods:* **Directly train the policy** to select what action to take given a state (or a probability distribution over actions at that state). In this case, we **don't have a value function.**

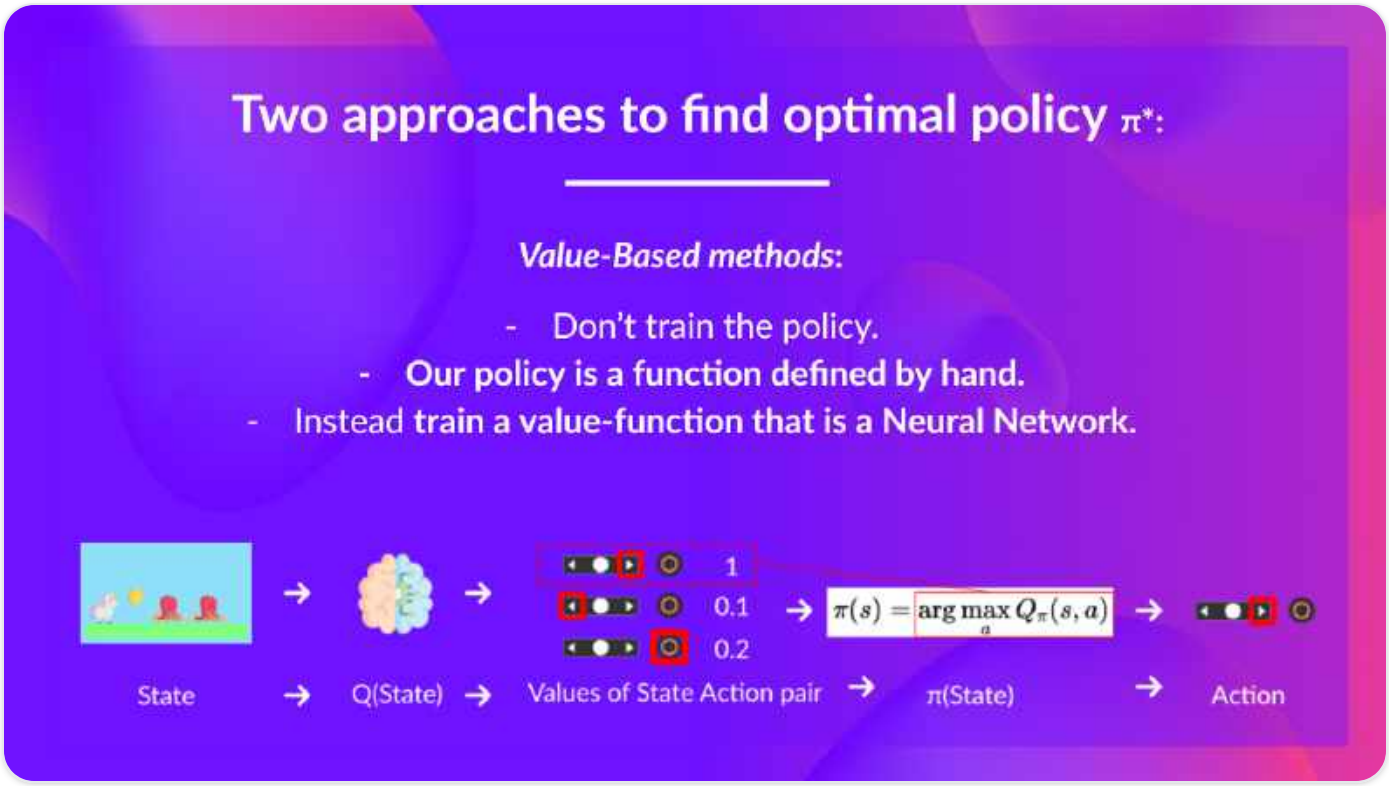


The policy takes a state as input and outputs what action to take at that state (deterministic policy: a policy that output one action given a state, contrary to stochastic policy that output a probability distribution over actions).

And consequently, **we don't define by hand the behav** **define it.**

- *Value-based methods:* **Indirectly, by training a value function** that outputs the value of a state or a state-action pair. Given this value function, our policy **will take an action.**

Since the policy is not trained/learned, **we need to specify its behavior.** For instance, if we want a policy that, given the value function, will take actions that always lead to the biggest reward, **we'll create a Greedy Policy.**



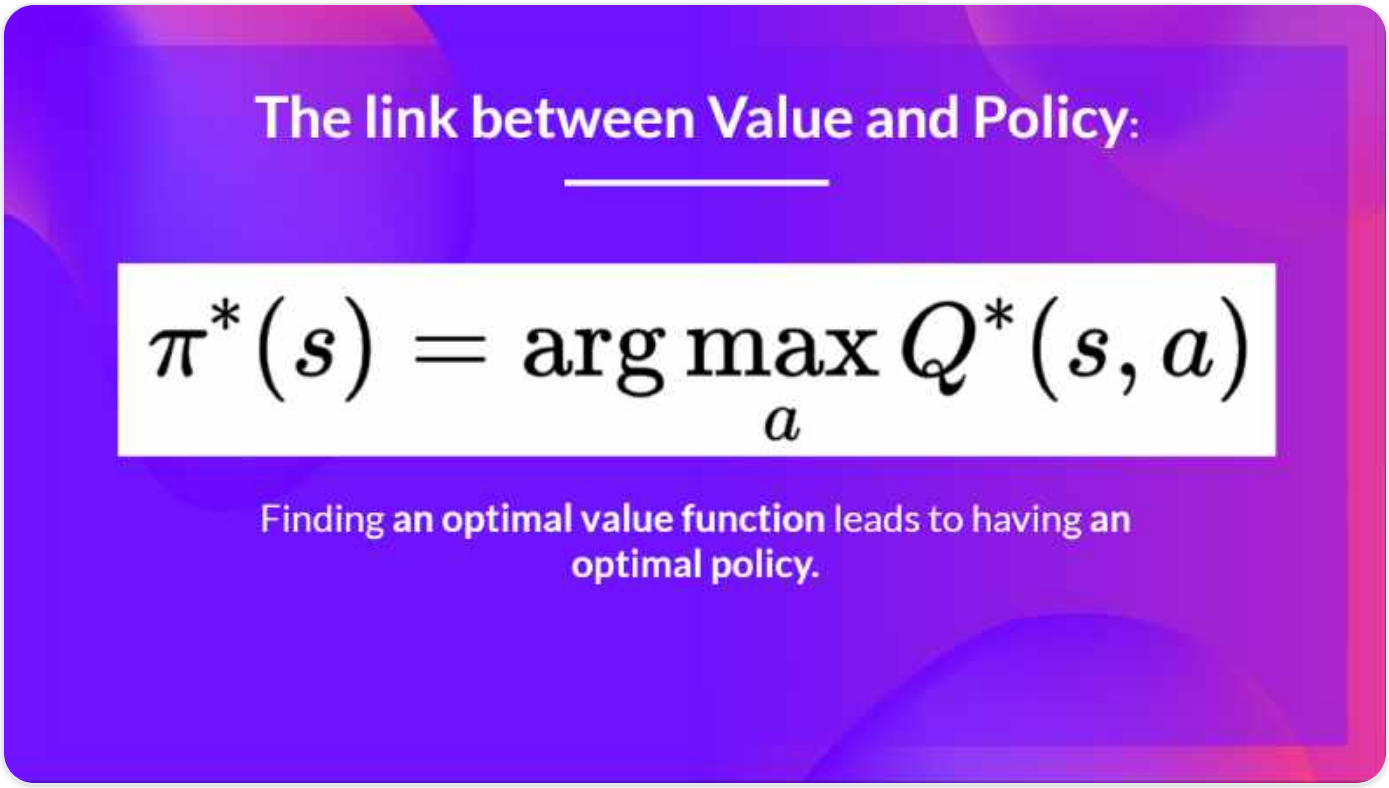
Given a state, our action-value function (that we train) outputs the value of each action at that state. Then, our pre-defined Greedy Policy selects the action that will yield the highest value given a state or a state action pair.

Consequently, whatever method you use to solve your problem, **you will have a policy**. In the case of value-based methods, you don't train the policy: your policy **is just a simple pre-specified function** (for instance, the Greedy Policy) that uses the values given by the value-function to select its actions.

So the difference is:

- In policy-based training, **the optimal policy (denoted π*) is found by training the policy directly.**

- In value-based training, **finding an optimal value function (denoted Q* or V*, we'll study the difference below) leads to having an optimal policy.**

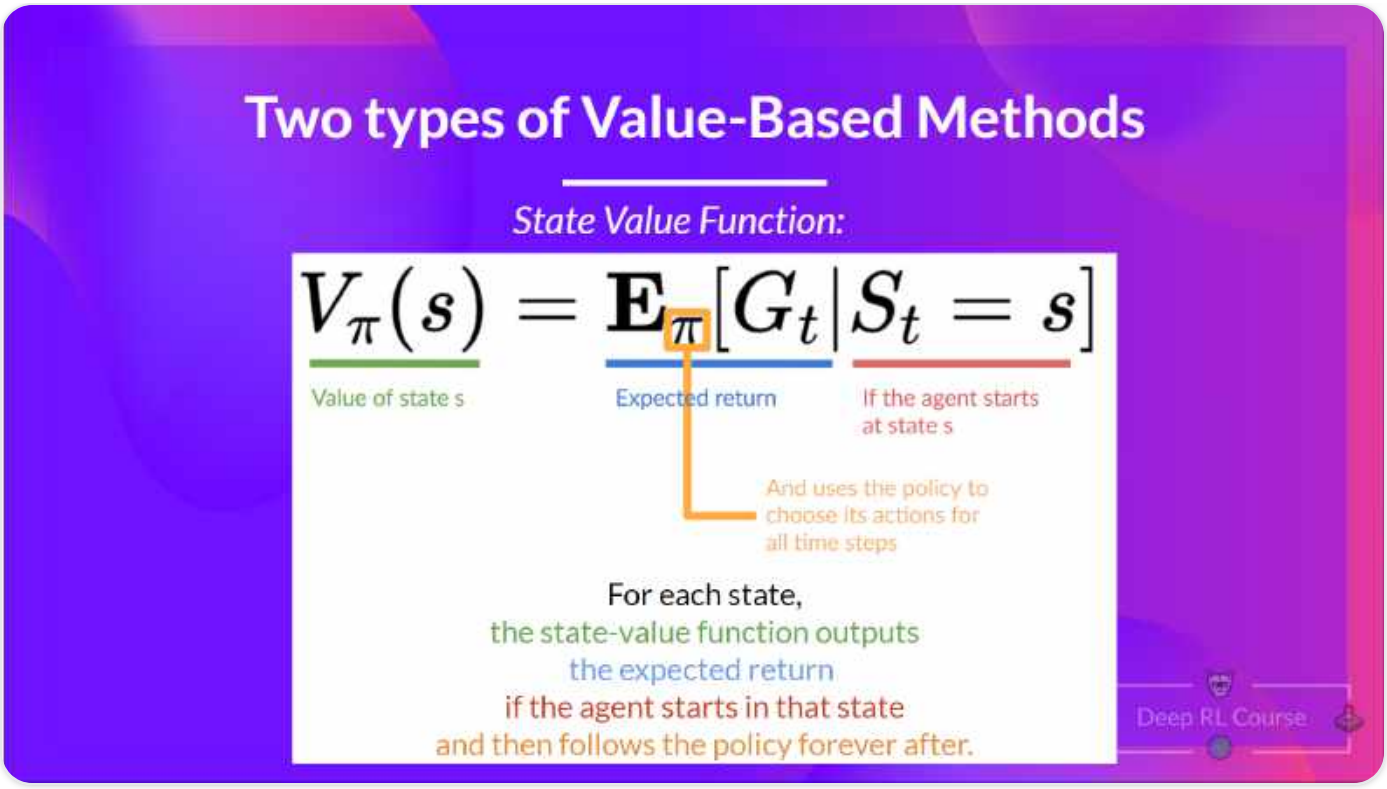


In fact, most of the time, in value-based methods, you'll use **an Epsilon-Greedy Policy** that handles the exploration/exploitation trade-off; we'll talk about this when we talk about Q-Learning in the second part of this unit.

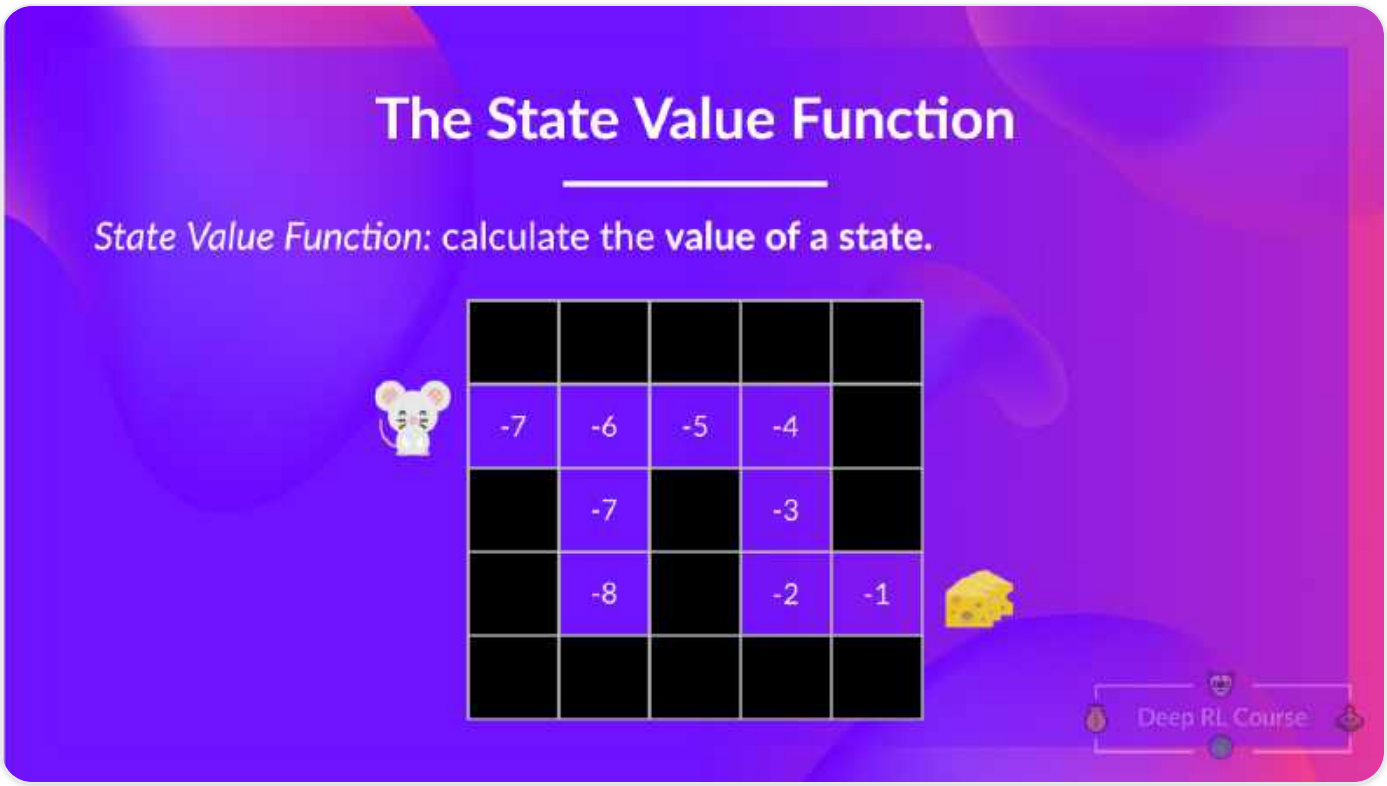
As we mentioned above, we have two types of value-based functions:

The state-value function

We write the state value function under a policy π like this:



For each state, the state-value function outputs the expected return if the agent **starts at that state** and then follows the policy forever afterward (for all future timesteps, if you prefer).

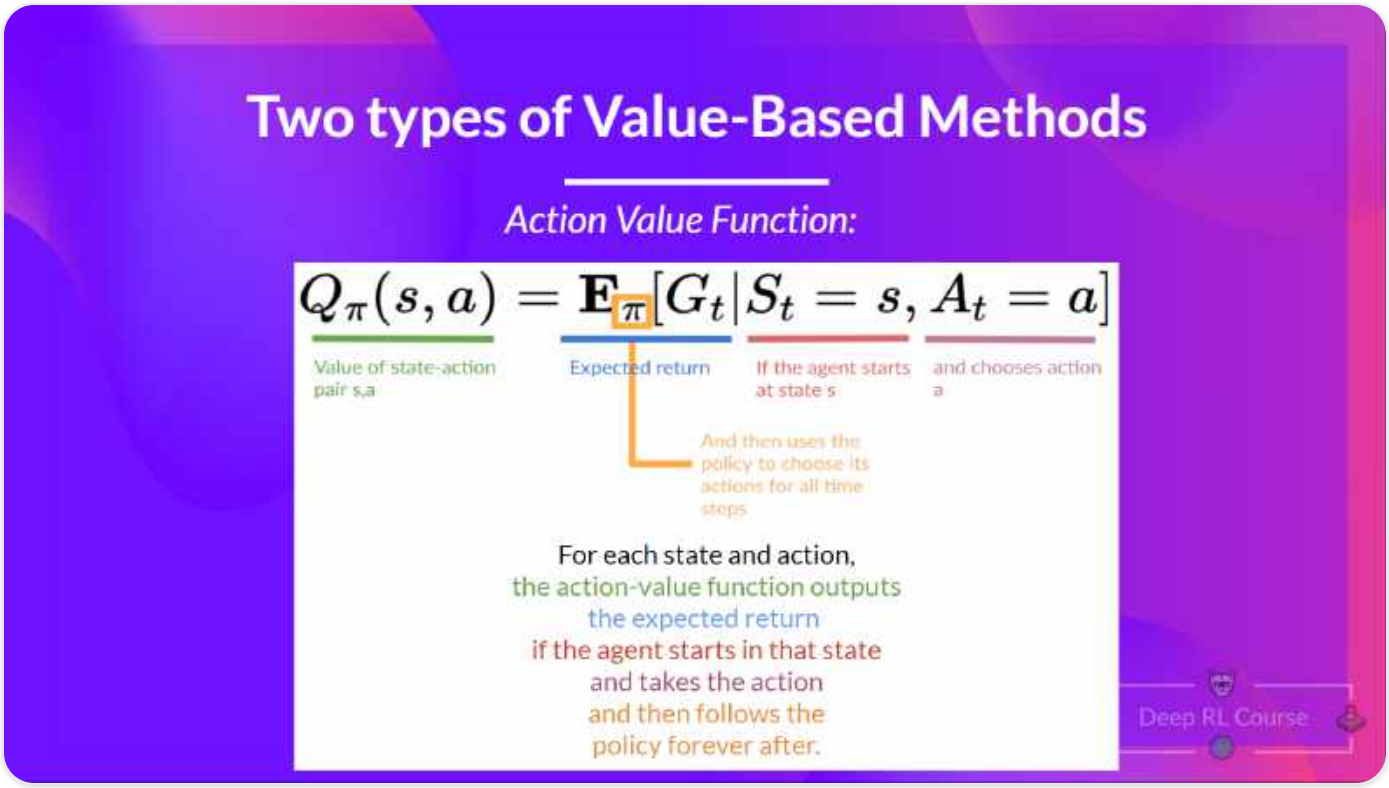


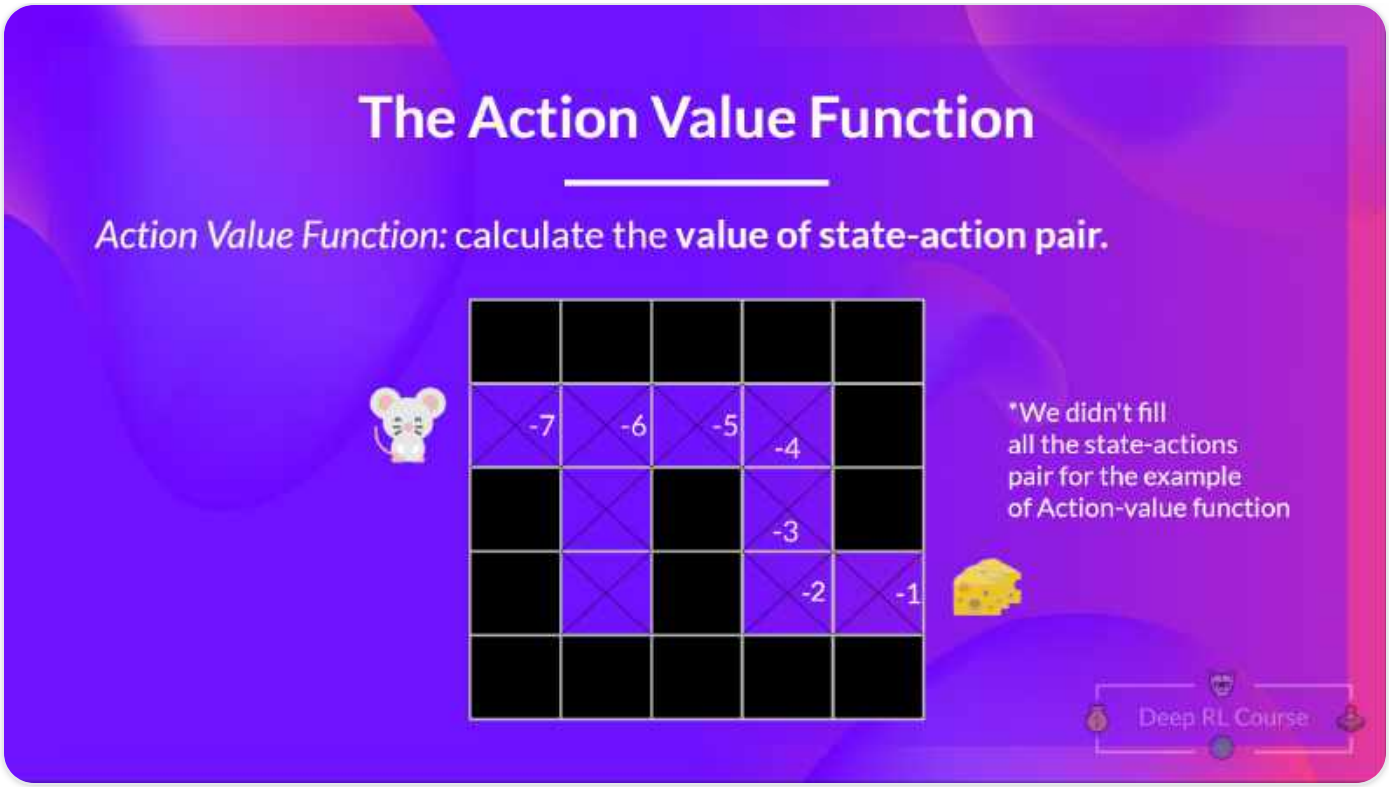
If we take the state with value -7: it's the expected return starti
policy (greedy policy), so right, right, right, down, down, right,

The action-value function

In the action-value function, for each state and action pair, the action-value function **outputs the expected return** if the agent starts in that state, takes that action, and then follows the policy forever after.

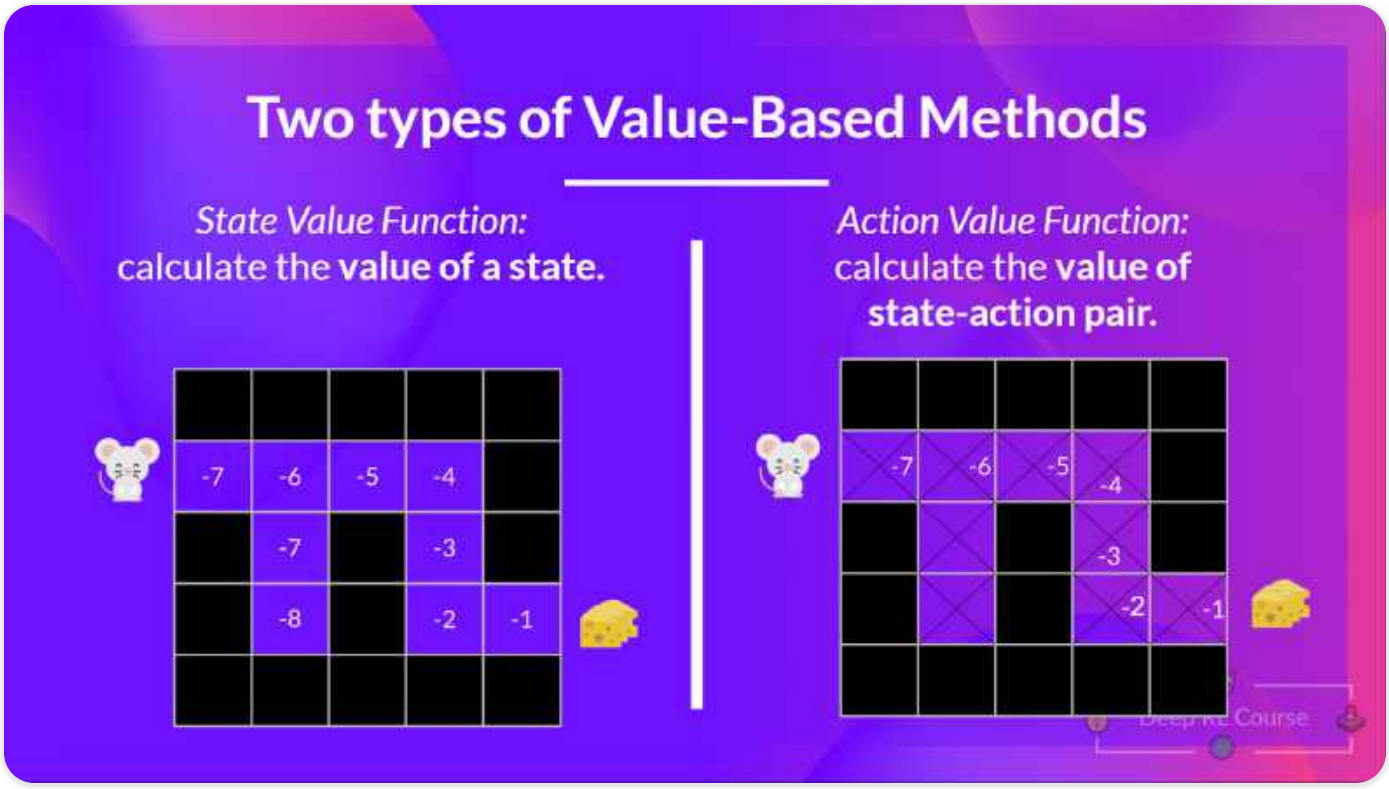
The value of taking action a in state s under a policy π is:





We see that the difference is:

- For the state-value function, we calculate **the value of a state** S_t
- For the action-value function, we calculate **the value of the state-action pair (** S_t, A_t **) hence the value of taking that action at that state.**



Note: We didn't fill all the state-action pairs for the example of Action-value function

In either case, whichever value function we choose (state-value or action-value function), **the returned value is the expected return.**

However, the problem is that **to calculate EACH value of a state or a state-action pair, we need to sum all the rewards an agent can get if it starts at that state.**

This can be a computationally expensive process, and that's **where the Bellman equation comes in to help us.**

<> Update on GitHub

← What is RL? A short recap

The Bellman Equation, simplify our value estimation →

Search models, datasets, users...

Deep RL Course documentation
Type of tasks

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

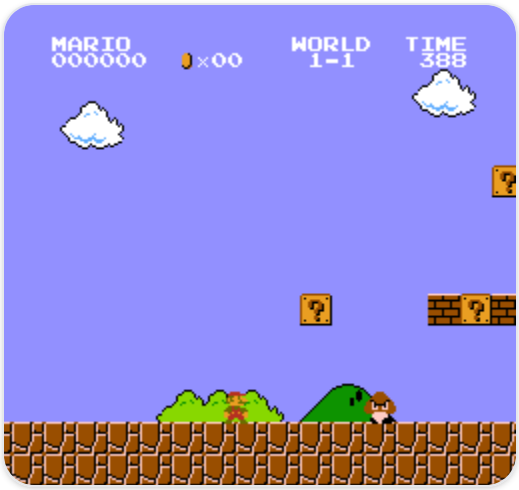
Type of tasks

A task is an **instance** of a Reinforcement Learning problem. We can have two types of tasks: **episodic** and **continuing**.

Episodic task

In this case, we have a starting point and an ending point **(a terminal state). This creates an episode**: a list of States, Actions, Rewards, and new States.

For instance, think about Super Mario Bros: an episode begins at the launch of a new Mario Level and ends **when you're killed or you reached the end of the level.**



Ask HuggingChat

Beginning of a new episode.

Continuing tasks

These are tasks that continue forever (**no terminal state**). In this case, the agent must **learn how to choose the best actions and simultaneously interact with the environment.**

For instance, an agent that does automated stock trading. For this task, there is no starting point and terminal state. **The agent keeps running until we decide to stop it.**



To recap:



<> Update on GitHub

← The Reinforcement Learning Framework

The Exploration/ Exploitation tradeoff →

Search models, datasets, users...

Deep RL Course documentation
Visualize the Clipped Surrogate Objective Function

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

Visualize the Clipped Surrogate Objective Function

Don't worry. **It's normal if this seems complex to handle right now**. But we're going to see what this Clipped Surrogate Objective Function looks like, and this will help you to visualize better what's going on.

	$p_t(\theta) > 0$	A_t	Return Value of min	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	0
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	0
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

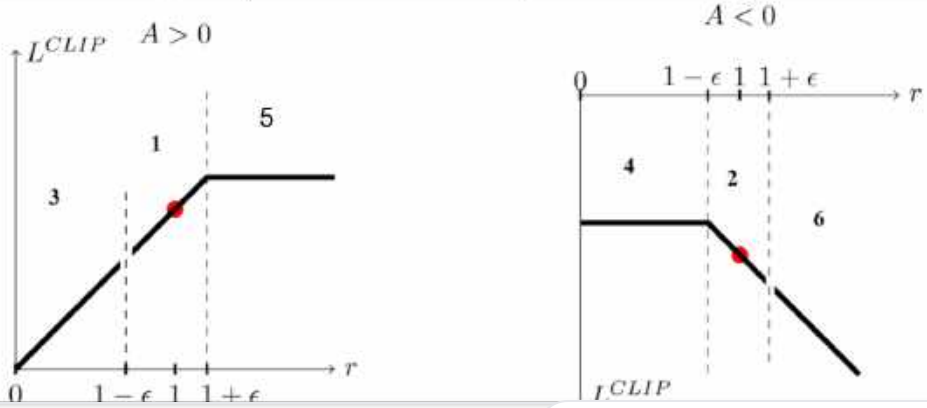


Table from "Towards Delivering a Coherent Self-Contained Exp Bick

Ask HuggingChat

We have six different situations. Remember first that we take the minimum between the clipped and unclipped objectives.

Case 1 and 2: the ratio is between the range

In situations 1 and 2, **the clipping does not apply since the ratio is between the range** $[1 - \epsilon, 1 + \epsilon]$

In situation 1, we have a positive advantage: the **action is better than the average** of all the actions in that state. Therefore, we should encourage our current policy to increase the probability of taking that action in that state.

Since the ratio is between intervals, **we can increase our policy's probability of taking that action at that state.**

In situation 2, we have a negative advantage: the action is worse than the average of all actions at that state. Therefore, we should discourage our current policy from taking that action in that state.

Since the ratio is between intervals, **we can decrease the probability that our policy takes that action at that state.**

Case 3 and 4: the ratio is below the range

	$p_t(\theta) > 0$	A_t	Return Value of min	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	**0**
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	**0**
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

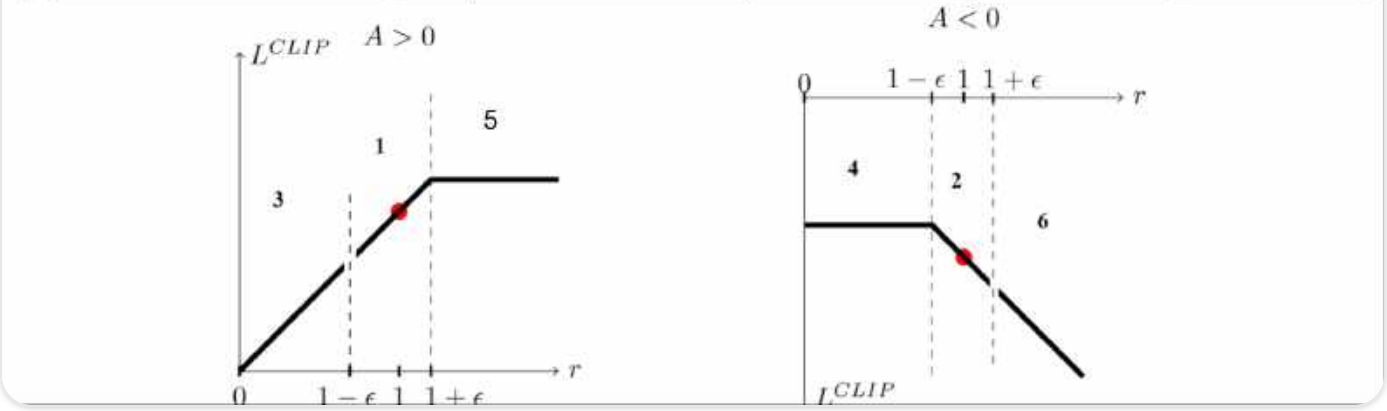


Table from "Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization" by Daniel Bick

If the probability ratio is lower than $[1 - \epsilon]$, the probability of taking that action at that state is much lower than with the old policy.

If, like in situation 3, the advantage estimate is positive (A>0), then **you want to increase the probability of taking that action at that state.**

But if, like situation 4, the advantage estimate is negative, **we don't want to decrease further** the probability of taking that action at that state. Therefore, the gradient is = 0 (since we're on a flat line), so we don't update our weights.

Case 5 and 6: the ratio is above the range

	$p_t(\theta) > 0$	A_t	Return Value of min	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	**0**
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	**0**
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

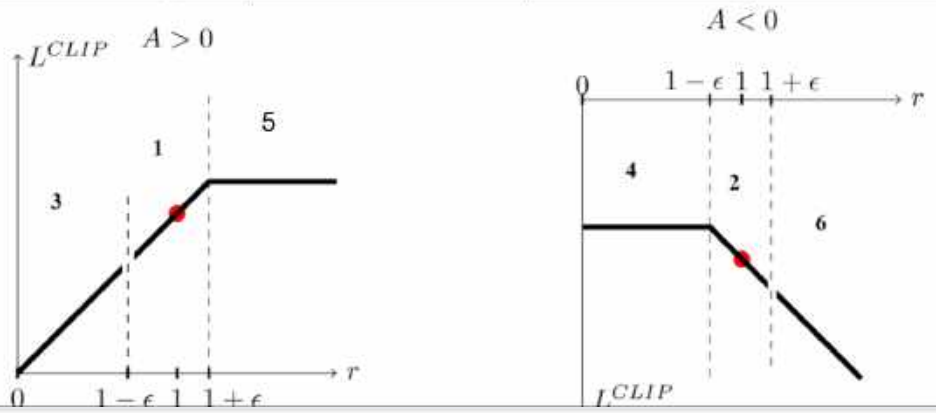


Table from "Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization" by Daniel Bick

If the probability ratio is higher than $[1+\epsilon]$, the probability of taking that action at that state in the current policy is **much higher than in the former policy.**

If, like in situation 5, the advantage is positive, **we don't want to get too greedy**. We already have a higher probability of taking that action at that state than the former policy. Therefore, the gradient is = 0 (since we're on a flat line), so we don't update our weights.

If, like in situation 6, the advantage is negative, we want to decrease the probability of taking that action at that state.

So if we recap, **we only update the policy with the unclipped objective part**. When the minimum is the clipped objective part, we don't update our policy weights since the gradient will equal 0.

So we update our policy only if:

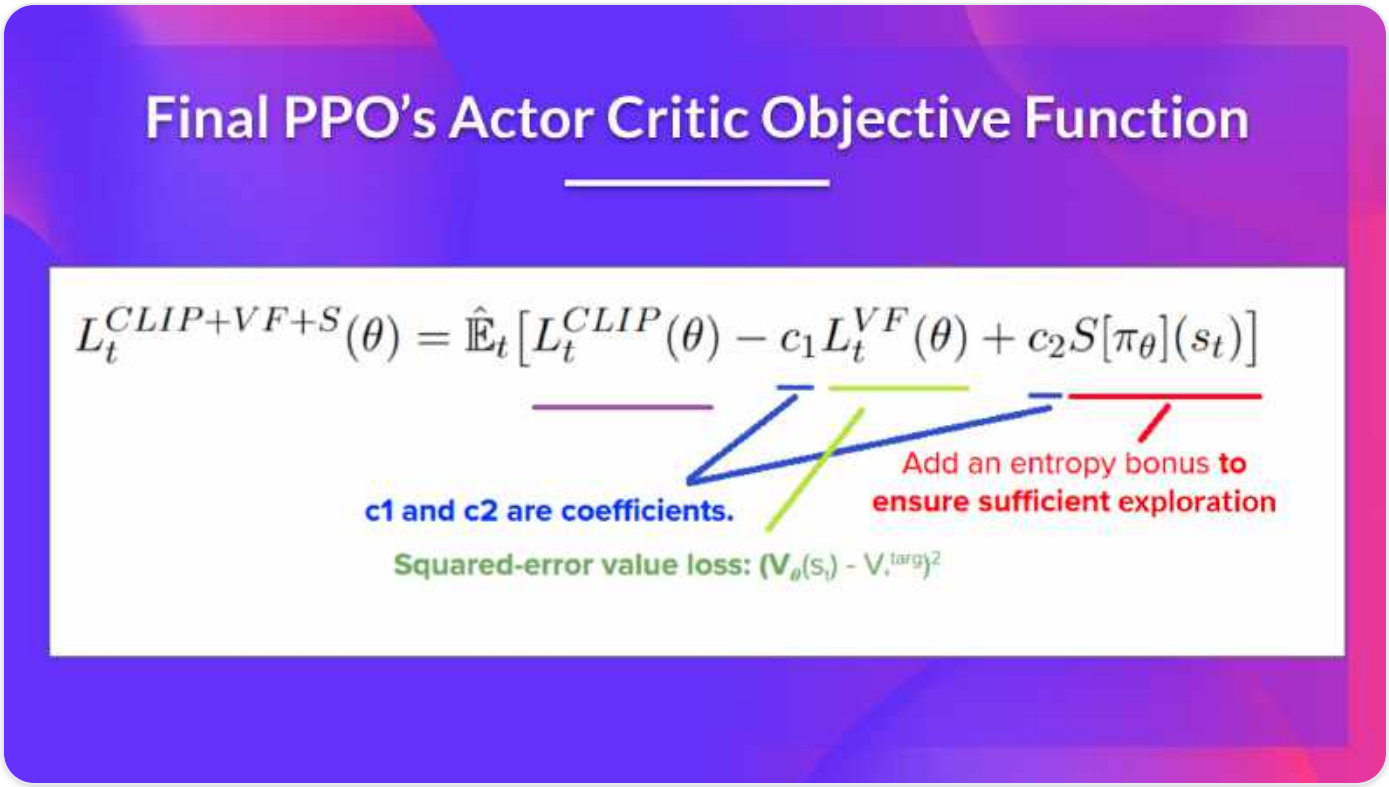
- Our ratio is in the range $[1-\epsilon, 1+\epsilon]$
- Our ratio is outside the range, but **the advantage l**

- Being below the ratio but the advantage is > 0
- Being above the ratio but the advantage is < 0

You might wonder why, when the minimum is the clipped ratio, the gradient is 0. When the ratio is clipped, the derivative in this case will not be the derivative of the$r_t(\theta) * A_t$ but the derivative of either$(1 - \epsilon) * A_t$ or the derivative of$(1 + \epsilon) * A_t$ which both = 0.

To summarize, thanks to this clipped surrogate objective, **we restrict the range that the current policy can vary from the old one.** Because we remove the incentive for the probability ratio to move outside of the interval since the clip forces the gradient to be zero. If the ratio is >$1 + \epsilon$ or < $1 - \epsilon$ the gradient will be equal to 0.

The final Clipped Surrogate Objective Loss for PPO Actor-Critic style looks like this, it's a combination of Clipped Surrogate Objective function, Value Loss Function and Entropy bonus:



That was quite complex. Take time to understand these situations by looking at the table and the graph. **You must understand why this makes sense.** If you want to go deeper, the best resource is the article "Towards Delivering a Coherent Self-Contain Optimization" by Daniel Bick, especially part 3.4.

<> Update on GitHub

← Introducing the Clipped Surrogate Objective Function

PPO with CleanRL →

Search models, datasets, users...

Deep RL Course documentation
Welcome to the 🤗 Deep Reinforcement Learning Course

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

Welcome to the 🤗 Deep Reinforcement Learning Course



Welcome to the most fascinating topic in Artificial Intelligence: **Deep Reinforcement Learning**.

This course will **teach you about Deep Reinforcement Learning from beginner to expert**. It's completely free and open-source!

Ask HuggingChat

In this introduction unit you'll:

- Learn more about the **course content**.
- **Define the path** you're going to take (either self-audit or certification process).
- Learn more about the **AI vs. AI challenges** you're going to participate in.
- Learn more **about us**.
- **Create your Hugging Face account** (it's free).
- **Sign-up to our Discord server**, the place where you can chat with your classmates and us (the Hugging Face team).

Let's get started!

What to expect?

In this course, you will:

- 📖 Study Deep Reinforcement Learning in **theory and practice.**
- 👩‍💻 Learn to **use famous Deep RL libraries** such as Stable Baselines3, RL Baselines3 Zoo, Sample Factory and CleanRL.
- 🤖 **Train agents in unique environments** such as SnowballFight, Huggy the Doggo 🐶, VizDoom (Doom) and classical ones such as Space Invaders, PyBullet and more.
- 💾 Share your **trained agents with one line of code to the Hub** and also download powerful agents from the community.
- 🏆 Participate in challenges where you will **evaluate your agents against other teams. You'll also get to play against the agents you'll train.**
- 🎓 **Earn a certificate of completion** by completing 80% of the assignments.

And more!

At the end of this course, **you'll get a solid foundation from the basics to the SOTA (state-of-the-art) of methods**.

Don't forget to sign up to the course (we are collecting your email to be able to **send you the links when each Unit is published and give you information about the challenges and updates).**

Sign up 👉 here

Course Maintenance Notice 🚧

Please note that this **Deep Reinforcement Learning course is now in a low-maintenance state**. However, it **remains an excellent resource to learn both the theory and practical aspects of Deep Reinforcement Learning**.

Keep in mind the following points:

- *Unit 7 (AI vs AI)* : This feature is currently non-functional. However, you can still train your agent to play soccer and observe its performance.
- *Leaderboard* : The leaderboard is no longer operational.

Aside from these points, all theory content and practical exercises remain fully accessible and effective for learning.

If you have any problem with one of the hands-on **please check the issue sections where the community give some solutions to bugs**.

What does the course look like?

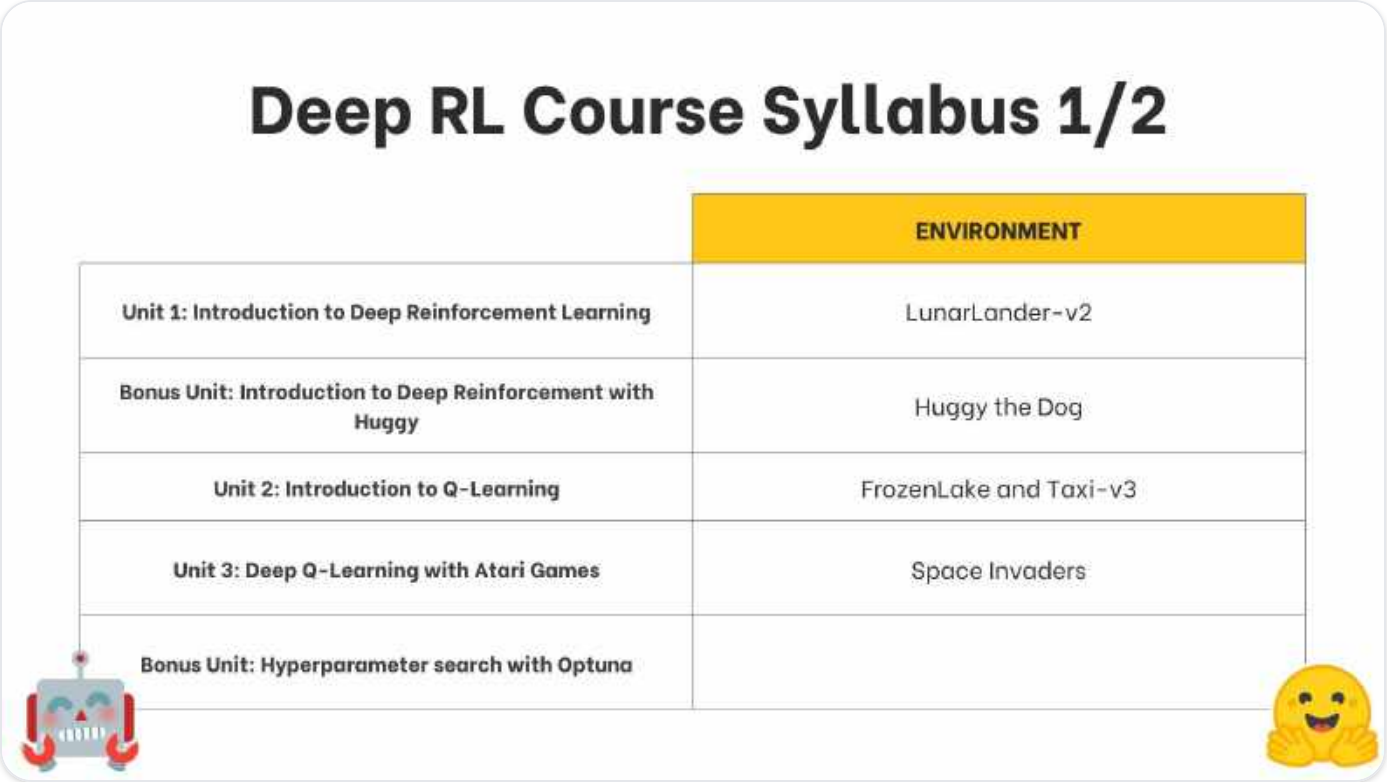
The course is composed of:

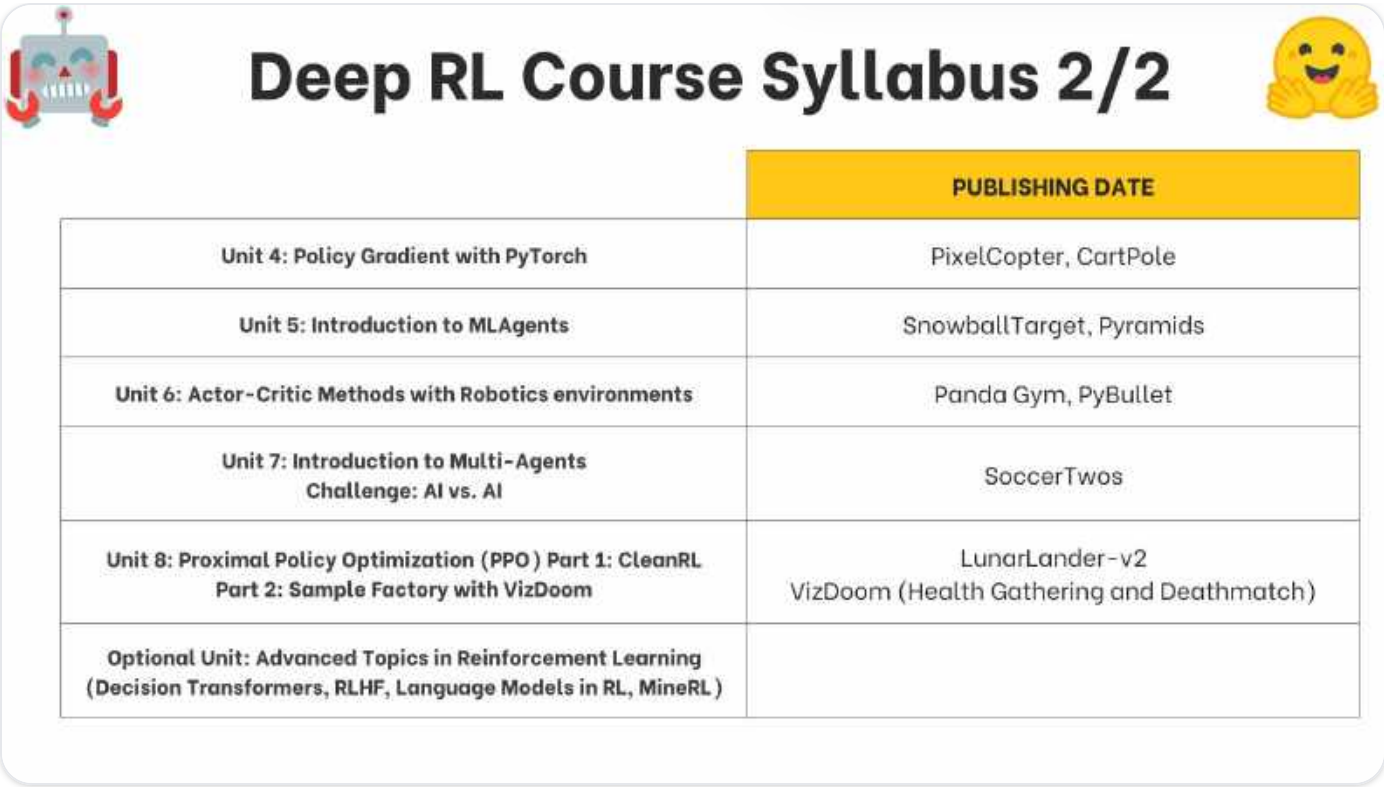
- *A theory part*: where you learn a **concept in theory**.
- *A hands-on*: where you'll learn **to use famous Deep RL libraries** to train your agents in unique environments. These hands-on will be **Google Colab notebooks with companion tutorial videos** if you prefer learning with video format!

- *Challenges*: you'll get to put your agent to compete against other agents in different challenges. There will also be a leaderboard for you to compare the agents' performance.

What's the syllabus?

This is the course's syllabus:





Two paths: choose your own adventure



You can choose to follow this course either:

- *To get a certificate of completion*: you need to complete 80% of the assignments.
- *To get a certificate of honors*: you need to complete 100% of the assignments.
- *As a simple audit*: you can participate in all challenges and do assignments if you want.

There's **no deadlines, the course is self-paced**. Both paths **are completely free**. Whatever path you choose, we advise you **to follow the recommended pace to enjoy the course and challenges with your fellow classmates.**

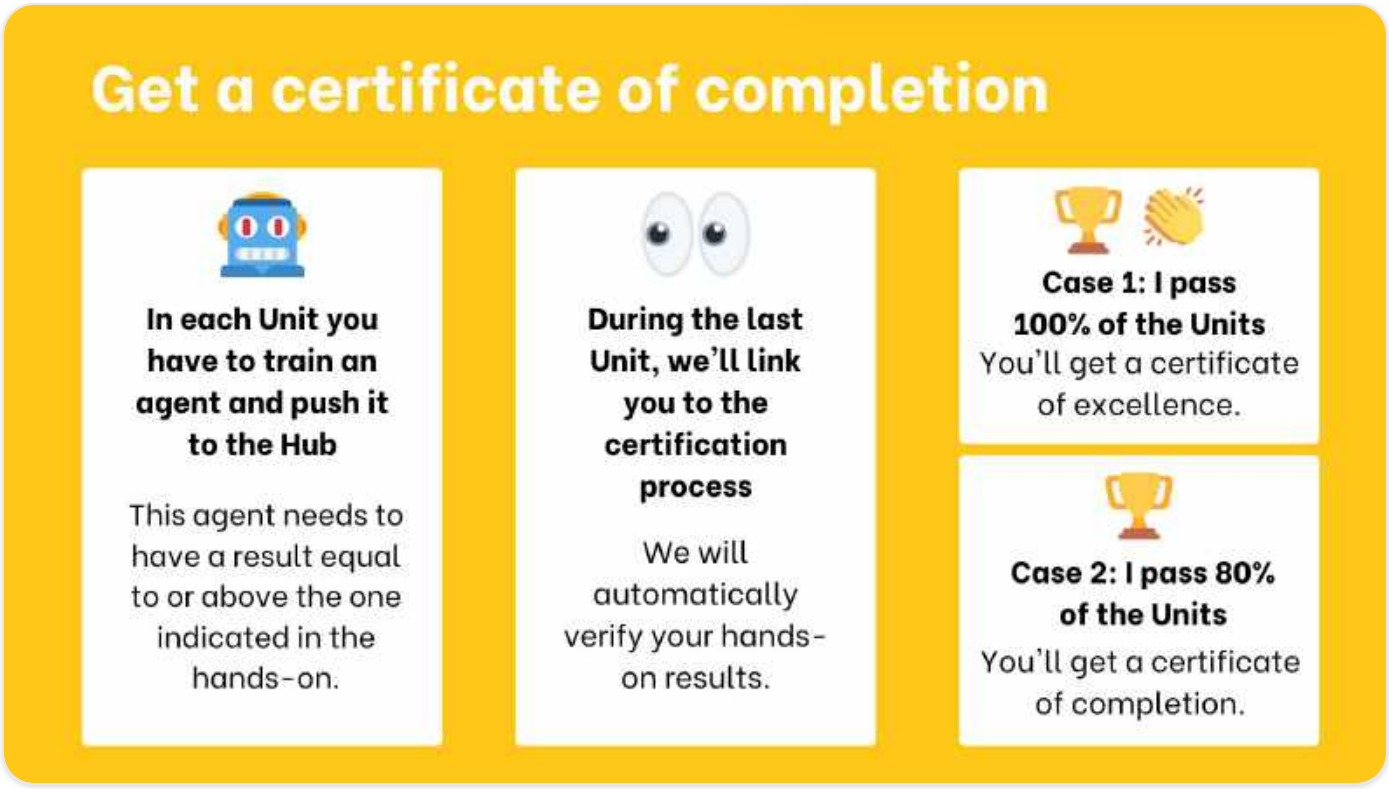
You don't need to tell us which path you choose. **If you get more than 80% of the assignments done, you'll get a certificate.**

The Certification Process

The certification process is **completely free**:

- *To get a certificate of completion*: you need to complete 80% of the assignments.
- *To get a certificate of honors*: you need to complete 100% of the assignments.

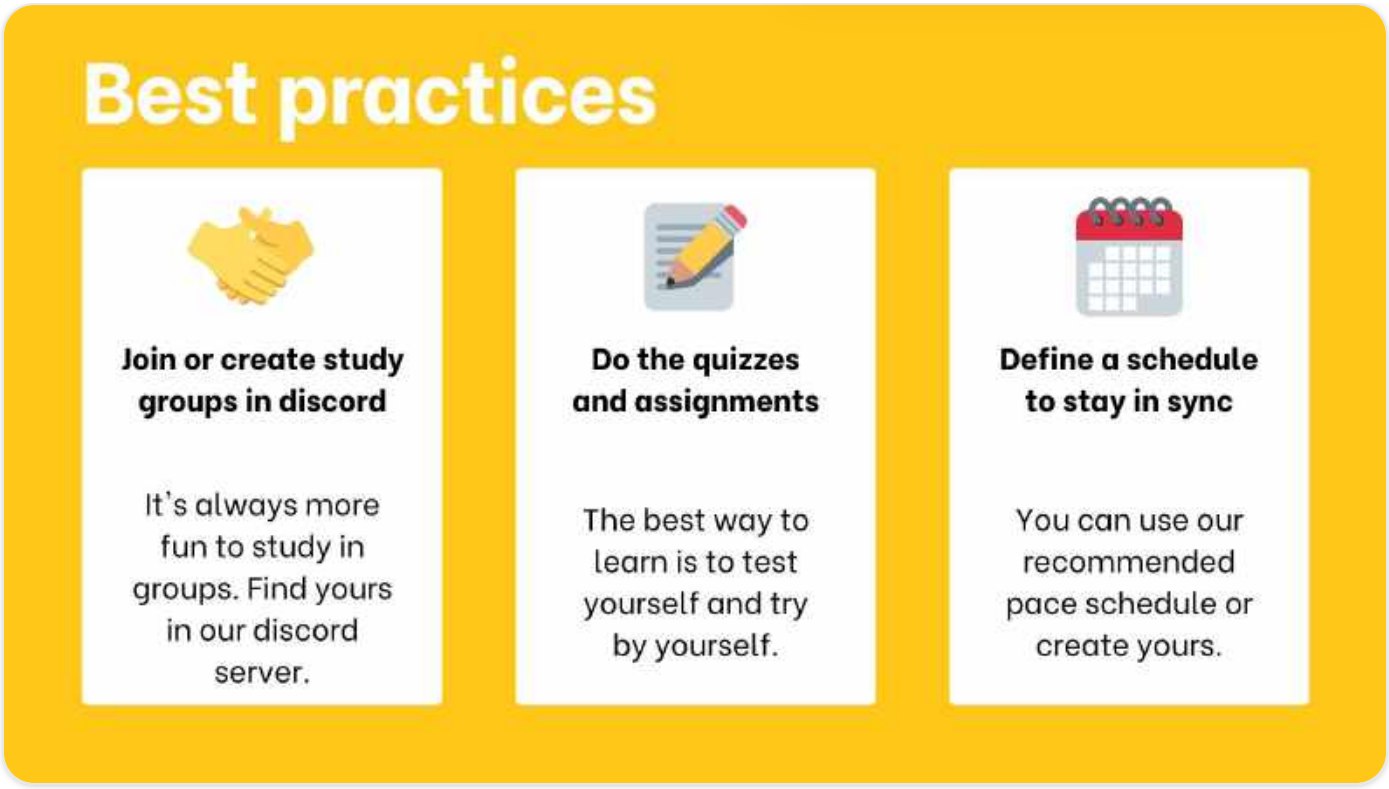
Again, there's **no deadline** since the course is self paced. But our advice **is to follow the recommended pace section**.



How to get most of the course?

To get most of the course, we have some advice:

1. Join study groups in Discord : studying in groups is always easier. To do that, you need to join our discord server. If you're new to Discord, no worries! We have some tools that will help you learn about it.
2. **Do the quizzes and assignments**: the best way to learn is to do and test yourself.
3. **Define a schedule to stay in sync**: you can use our recommended pace schedule below or create yours.



What tools do I need?

You need only 3 things:

- *A computer* with an internet connection.
- *Google Colab (free version)*: most of our hands-on will use Google Colab, the **free version is enough.**
- A *Hugging Face Account*: to push and load models. If you don't have an account yet, you can create one here (it's free).



What is the recommended pace?

Each chapter in this course is designed **to be completed in 1 week, with approximately 3-4 hours of work per week**. However, you can take as much time as necessary to complete the course. If you want to dive into a topic more in-depth, we'll provide additional resources to help you achieve that.

Who are we

About the author:

- Thomas Simonini is a Developer Advocate at Hugging Face 🤗 specializing in Deep Reinforcement Learning. He founded the Deep Reinforcement Learning Course in 2018, which became one of the most used courses in Deep RL.

About the team:

- Omar Sanseviero is a Machine Learning Engineer at Hugging Face where he works in the intersection of ML, Community and Open Source. Previously, Omar worked as a Software

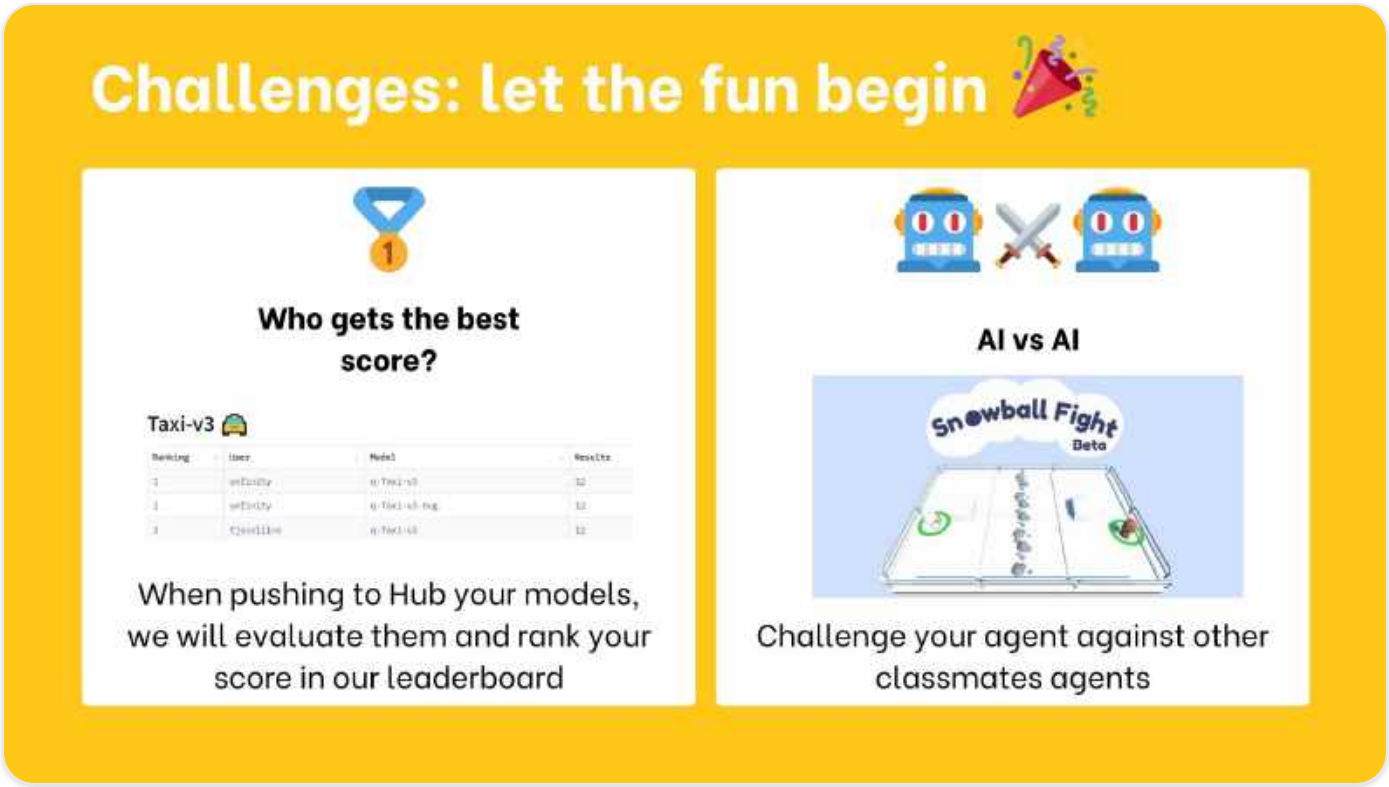
Engineer at Google in the teams of Assistant and TensorFlow Graphics. He is from Peru and likes llamas 🦙.

- Sayak Paul is a Developer Advocate Engineer at Hugging Face. He's interested in the area of representation learning (self-supervision, semi-supervision, model robustness). And he loves watching crime and action thrillers 🔪.

What are the challenges in this course?

In this new version of the course, you have two types of challenges:

- A leaderboard to compare your agent's performance to other classmates'.
- AI vs. AI challenges where you can train your agent and compete against other classmates' agents.



I still have questions

Please ask your question in our discord server #rl-discu

<> Update on GitHub

Setup →

Search models, datasets, users...

Deep RL Course documentation
What are the policy-based methods?

Join the Hugging Face community

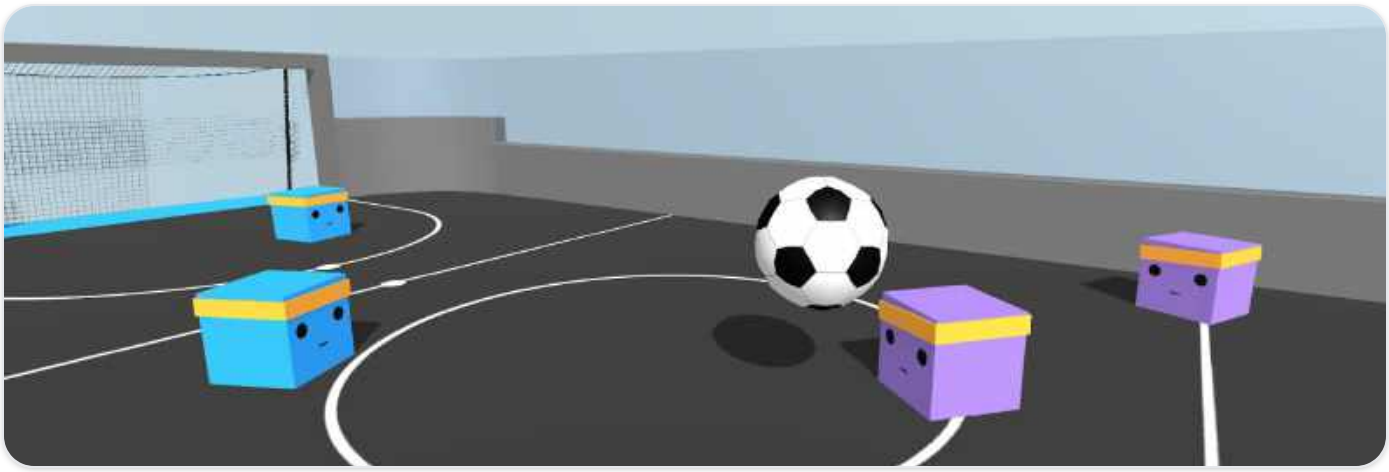
and get access to the augmented documentation experience

Sign Up to get started

What are the policy-based methods?

The main goal of Reinforcement learning is to **find the optimal policy π^* that will maximize the expected cumulative reward**. Because Reinforcement Learning is based on the *reward hypothesis*: **all goals can be described as the maximization of the expected cumulative reward.**

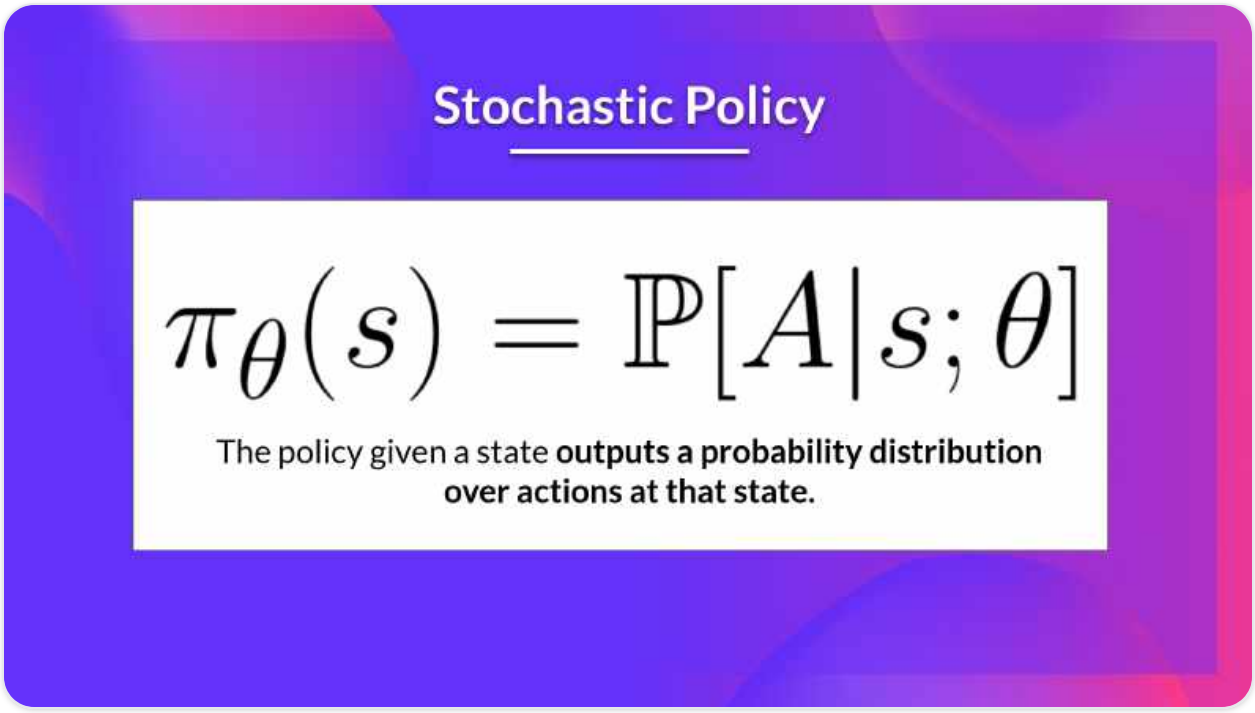
For instance, in a soccer game (where you're going to train the agents in two units), the goal is to win the game. We can describe this goal in reinforcement learning as **maximizing the number of goals scored** (when the ball crosses the goal line) into your opponent's soccer goals. And **minimizing the number of goals in your soccer goals**.



Value-based, Policy-based, and Actor-critic m

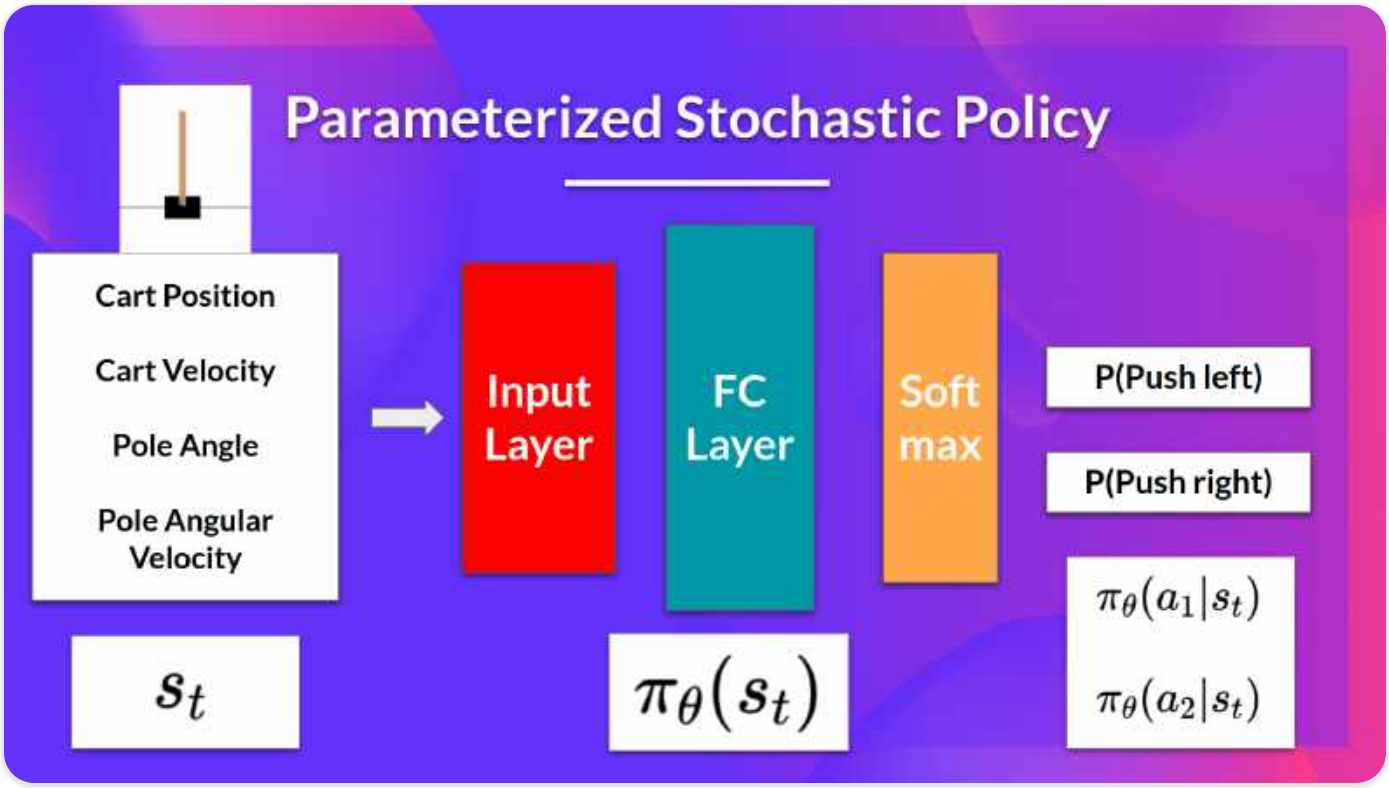
Ask HuggingChat

In the first unit, we saw two methods to find (or, most of the time, approximate) this optimal policy π^*.

- In *value-based methods*, we learn a value function.
  - The idea is that an optimal value function leads to an optimal policy π^*.
  - Our objective is to **minimize the loss between the predicted and target value** to approximate the true action-value function.
  - We have a policy, but it's implicit since it **is generated directly from the value function**. For instance, in Q-Learning, we used an (epsilon-)greedy policy.
- On the other hand, in *policy-based methods*, we directly learn to approximate π^* without having to learn a value function.
  - The idea is **to parameterize the policy**. For instance, using a neural network π_θ, this policy will output a probability distribution over actions (stochastic policy).
  - 

  - Our objective then is **to maximize the performance of the parameterized policy using gradient ascent**.

- To do that, we control the parameter θ that will affect the distribution of actions over a state.



- Next time, we'll study the *actor-critic* method, which is a combination of value-based and policy-based methods.

Consequently, thanks to policy-based methods, we can directly optimize our policy π_θ to output a probability distribution over actions $\pi_\theta(a|s)$ that leads to the best cumulative return. To do that, we define an objective function $J(\theta)$, that is, the expected cumulative reward, and we **want to find the value θ that maximizes this objective function**.

The difference between policy-based and policy-gradient methods

Policy-gradient methods, what we're going to study in this unit, is a subclass of policy-based methods. In policy-based methods, the optimization is most of the time *on-policy* since for each update, we only use data (trajectories) collected **by our most recent version of** π_θ.

The difference between these two methods **lies on how**

- In *policy-based methods*, we search directly for the optimal policy. We can optimize the parameterθ **indirectly** by maximizing the local approximation of the objective function with techniques like hill climbing, simulated annealing, or evolution strategies.
- In *policy-gradient methods*, because it is a subclass of the policy-based methods, we search directly for the optimal policy. But we optimize the parameterθ **directly** by performing the gradient ascent on the performance of the objective function$J(\theta)$.

Before diving more into how policy-gradient methods work (the objective function, policy gradient theorem, gradient ascent, etc.), let's study the advantages and disadvantages of policy-based methods.

<> Update on GitHub

← Introduction The advantages and disadvantages of policy-gradient methods →

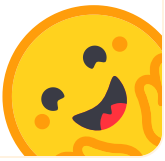
Search models, datasets, users...

Deep RL Course documentation
What is Reinforcement Learning?

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started



What is Reinforcement Learning?

To understand Reinforcement Learning, let’s start with the big picture.

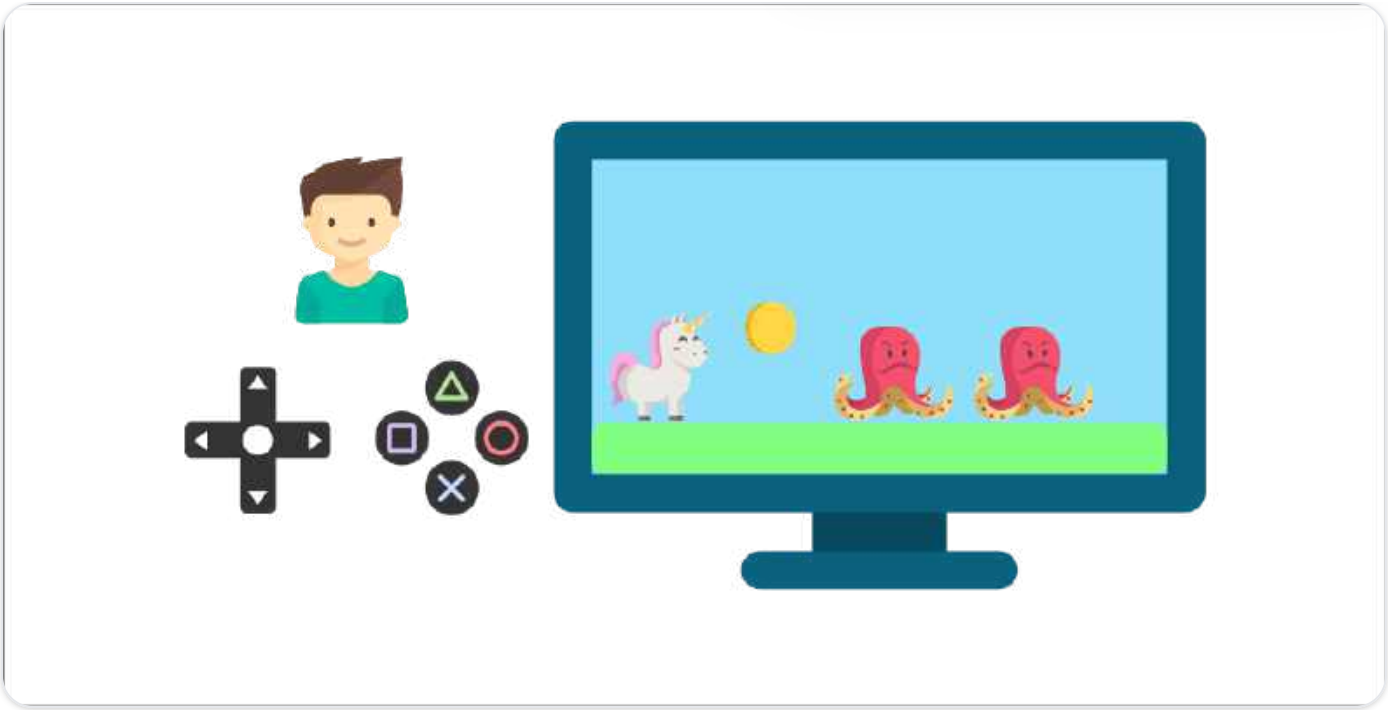
The big picture

The idea behind Reinforcement Learning is that an agent (an AI) will learn from the environment by **interacting with it** (through trial and error) and **receiving rewards** (negative or positive) as feedback for performing actions.

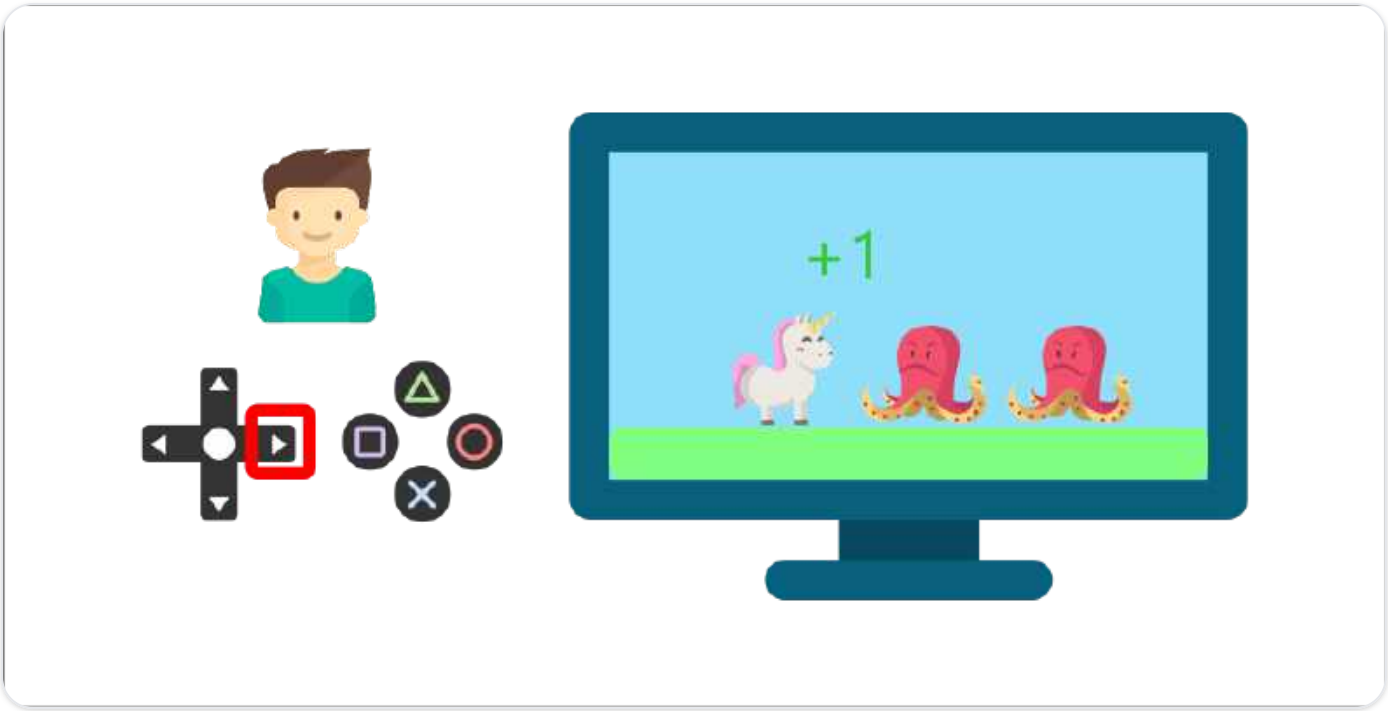
Learning from interactions with the environment **comes from our natural experiences.**

For instance, imagine putting your little brother in front of a video game he never played, giving him a controller, and leaving him alone.

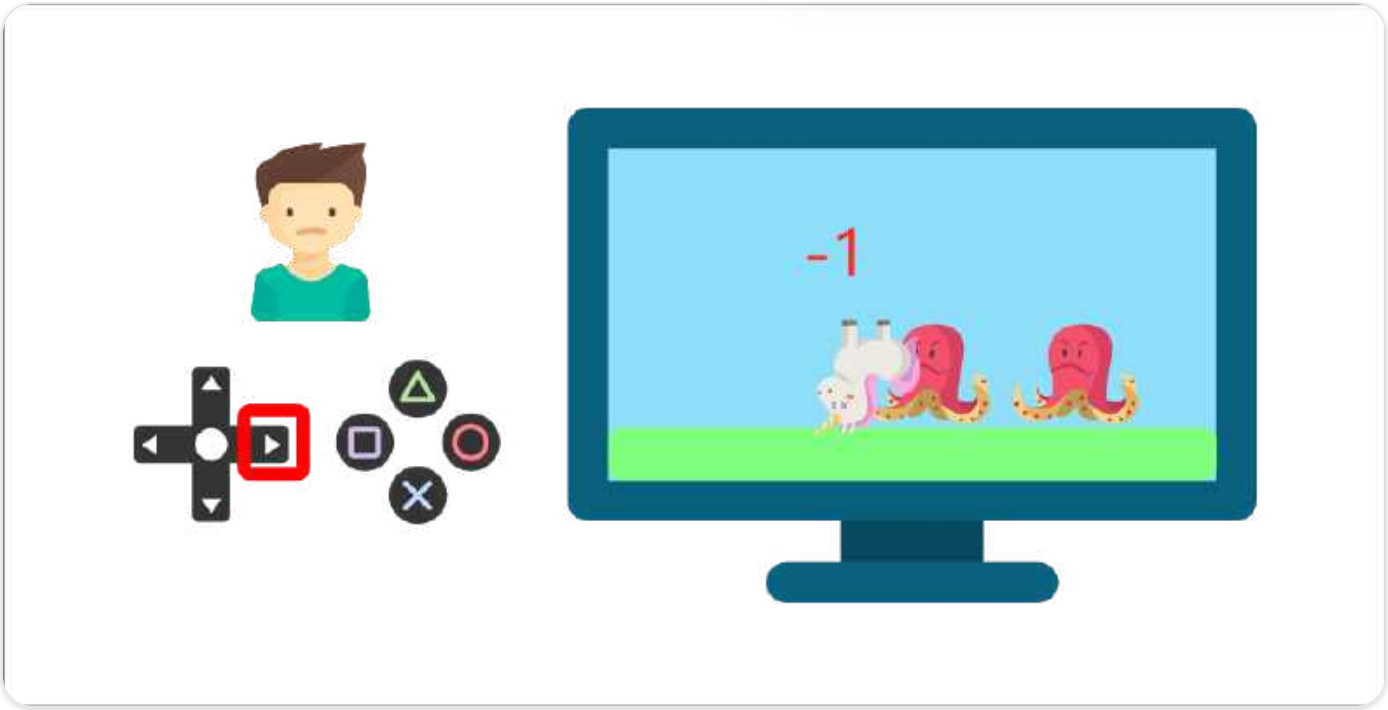
Ask HuggingChat



Your brother will interact with the environment (the video game) by pressing the right button (action). He got a coin, that's a +1 reward. It's positive, he just understood that in this game **he must get the coins.**



But then, **he presses the right button again** and he touches an enemy. He just died, so that's a -1 reward.



By interacting with his environment through trial and error, your little brother understands that **he needs to get coins in this environment but avoid the enemies.**

Without any supervision, the child will get better and better at playing the game.

That's how humans and animals learn, **through interaction.** Reinforcement Learning is just a **computational approach of learning from actions.**

A formal definition

We can now make a formal definition:

> Reinforcement learning is a framework for solving control tasks (also called decision problems) by building agents that learn from the environment by interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback.

But how does Reinforcement Learning work?

<> Update on GitHub

← Introduction

The Reinforcement Learning Framework →

Search models, datasets, users...

Deep RL Course documentation
What is RL? A short recap

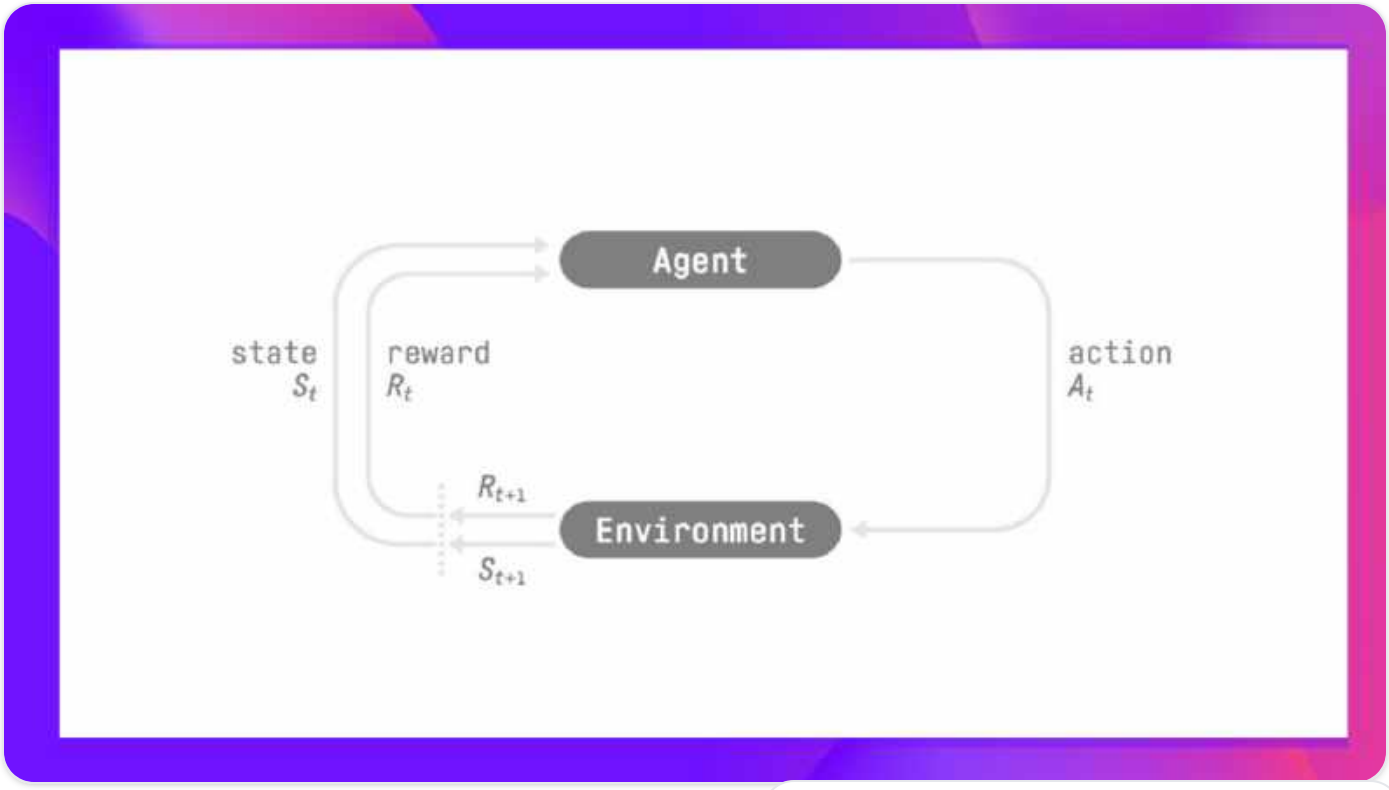
Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

What is RL? A short recap

In RL, we build an agent that can **make smart decisions**. For instance, an agent that **learns to play a video game.** Or a trading agent that **learns to maximize its benefits** by deciding on **what stocks to buy and when to sell.**



Ask HuggingChat

To make intelligent decisions, our agent will learn from the environment by **interacting with it through trial and error** and receiving rewards (positive or negative) **as unique feedback.**

Its goal **is to maximize its expected cumulative reward** (because of the reward hypothesis).

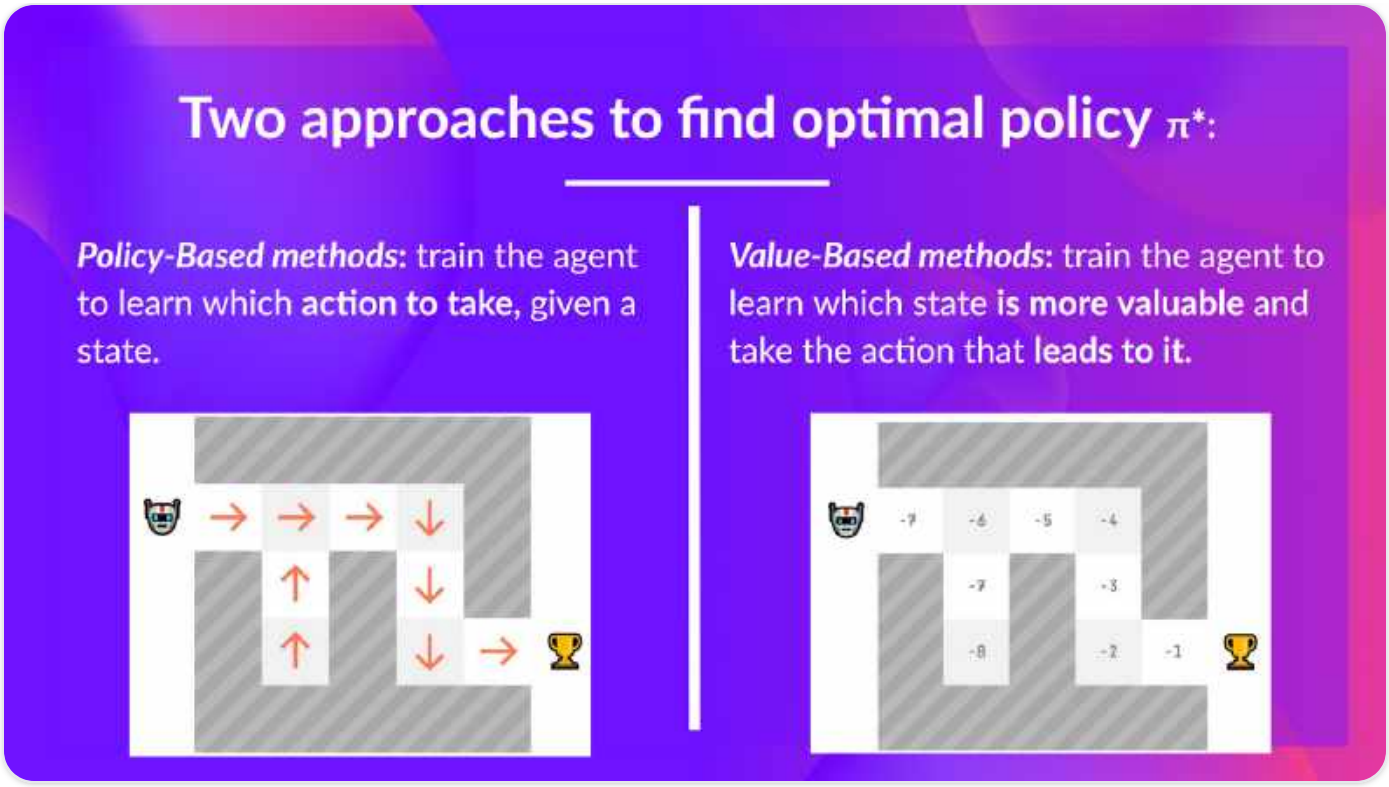
The agent's decision-making process is called the policy π: given a state, a policy will output an action or a probability distribution over actions. That is, given an observation of the environment, a policy will provide an action (or multiple probabilities for each action) that the agent should take.



Our goal is to find an optimal policy π* , aka., a policy that leads to the best expected cumulative reward.

And to find this optimal policy (hence solving the RL problem), there **are two main types of RL methods**:

- *Policy-based methods*: **Train the policy directly** to learn which action to take given a state.
- *Value-based methods*: **Train a value function** to learn **which state is more valuable** and use this value function **to take the action that leads to it**



And in this unit, **we'll dive deeper into the value-based methods.**

<> Update on GitHub

← Introduction

The two types of value-based methods →

Get started

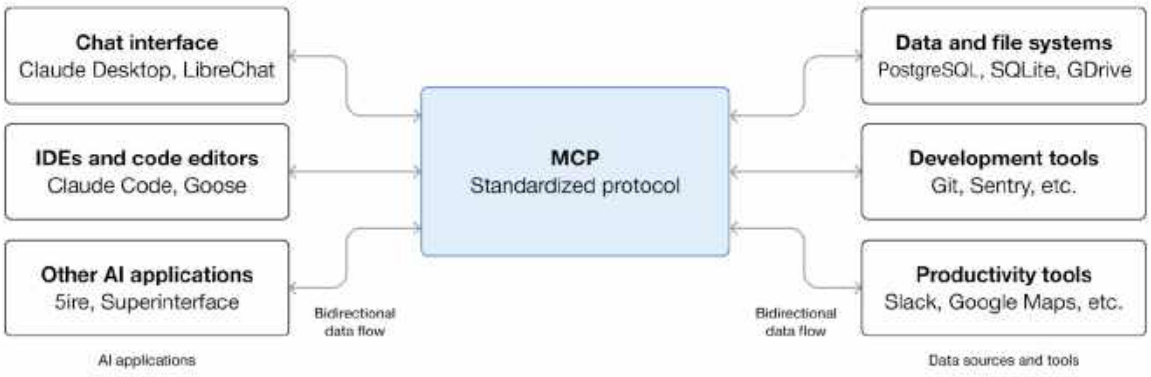
What is the Model Context Protocol (MCP)?

Copy page

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.

Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks.

Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.



What can MCP enable?

Assistant

Responses are generated using AI and may contain mistakes.

Ask a question...

Learn more

Understand concepts

Assistant

Responses are generated using AI and may contain mistakes.